Physical AI: Concepts, Techniques, and Challenges

Physical Intelligence Team, Shanghai AI Laboratory

physicalai@pjlab.org.cn

Preprint version, October 19, 2026

Abstract

In recent years, physical artificial intelligence (physical AI) has experienced rapid development, capturing significant attention in academia and industry. However, the field has yet to reach consensus on its definition, technical landscape, and core challenges. To address this gap, this monograph seeks to define physical AI, present its technical landscape, and examine each key technical subfield in depth. In doing so, it establishes a unified framework for Physical AI: artificial systems that perceive, model, and act upon the physical world through continuous feedback under real-world constraints. Further, this monograph distinguishes physical AI from conventional digital AI, embodied AI, cyber-physical systems, and digital twins. It develops the foundations of physical AI across physics, control, cognition, learning, and reasoning. It reviews key methodologies, including multimodal perception, physical data and simulation, world models, embodied foundation models, motor-control systems, multi-agent coordination, and hardware-software co-design. Furthermore, it examines applications spanning robotics, manufacturing, infrastructure, scientific experimentation, and autonomous services. Particular attention is given to sim-to-real transfer, reliability, real-time execution, evaluation, safety, governance, and long-term deployment. By connecting perception, prediction, decision-making, action, and adaptation in a closed loop, this monograph identifies the technical challenges and research directions required to advance from narrow automation toward more generalizable physical AI and, ultimately, physical artificial general intelligence (Physical AGI).

Contents

I Overview and Foundations **10**

1 Introduction to Physical AI **11**

1.1 Background: A Paradigm Expansion of AI 11

1.1.1 Digital AI and Physical AI 11

1.1.2 Core Characteristics 12

1.1.3 Why Physical AI Is Needed and Increasingly Feasible 13

1.2 Physical AI: Definition and Boundaries 14

1.2.1 A Broad Definition of Physical AI 14

1.2.2 Physical AI vs. Embodied AI 15

1.2.3 Physical AI vs. Cyber-Physical Systems and Digital Twins 16

1.3 Core Elements of Physical AI . . . 17
1.3.1 Physical State Perception and Estimation . . . 18
1.3.2 Physical World Models: Representation and Prediction . . . 18
1.3.3 Physical Decision-Making, Planning, and Control . . . 19
1.3.4 Closed-Loop Feedback and Multiscale Adaptation . . . 19
1.4 The Role of Physical AI in the Evolution of AI . . . 20
1.5 The Application Landscape of Physical AI . . . 21
1.5.1 The Closed Loop and the Distinction from Conventional Automation . . . 21
1.5.2 From Automation to Autonomous Industrial Operation . . . 23
1.5.3 Daily Services: From Dedicated Equipment to Embodied Companions . . . 24
1.5.4 Public Infrastructure: From Single-Vehicle Autonomy to City-Scale Coordination . . . 24
1.5.5 Scientific Inquiry: From Automated Instruments to Autonomous Laboratories 25
1.5.6 System Archetypes across Industries . . . 26
1.6 Summary and Organization of the Monograph . . . 27

2 Technical Foundations 29
2.1 Mathematical Descriptions of the Physical World . . . 29
2.1.1 Geometry and Spatial Representation . . . 29
2.1.2 Physical Dynamics and Interaction Modeling . . . 30
2.1.3 State Representation and Predictive Modeling . . . 31
2.2 Control Theory and Autonomous Systems . . . 32
2.2.1 Foundations of Cybernetics: Feedback and Steady State . . . 32
2.2.2 Classical and Modern Control Methods . . . 35
2.2.3 Hierarchical Architectures of Autonomous Systems . . . 37
2.2.4 Stability and Robustness Theory . . . 38
2.3 Embodied Cognition and the Development of Intelligence . . . 41
2.3.1 Core Claims of Embodied Cognition . . . 41
2.3.2 Perception-Action Coupling and Development . . . 41
2.3.3 Developmental Robotics . . . 42
2.3.4 Embodied Grounding and Multimodal Pretraining . . . 43
2.4 Learning and Reasoning in the Physical World . . . 44
2.4.1 Physics-Informed Learning Architectures . . . 45
2.4.2 Reinforcement Learning for Physical Interaction . . . 46
2.4.3 Predictive Learning and World Models . . . 48

II Methodology **50**

3 Perception and Cognition **51**

3.1 Tasks and Characteristics of Physical Perception . . . 51
3.2 Multimodal Perception and Fusion . . . 52
3.2.1 Multimodal Sensing and Data Acquisition . . . 52
3.2.2 Multimodal Fusion for Joint Perception . . . 58
3.3 3D/4D Spatial Perception . . . 58
3.3.1 3D/4D Scene Representation and Modeling . . . 59
3.3.2 Model-Driven Spatial Perception and Reasoning . . . 60
3.4 High-Level Perception and Cognition . . . 60
3.4.1 Object Recognition and Scene Semantic Understanding . . . 61
3.4.2 Object Properties, States, and Interaction Perception . . . 62
3.4.3 Grounded Perception and Cognition with Vision-Language Models . . . 63

4 Data for Physical AI **66**

4.1 Data Foundations for Physical AI . . . 66
4.1.1 The Spectrum of Physical Data . . . 66
4.1.2 Unified Representation . . . 66
4.1.3 Data Organization . . . 67
4.1.4 Annotation Framework and Workflow . . . 68
4.1.5 Quality Governance . . . 68
4.2 Data Collection and Interaction . . . 68
4.2.1 Collection Design . . . 68
4.2.2 Human Demonstrations . . . 70
4.2.3 Autonomous Data Collection . . . 74
4.2.4 The Interactive Feedback Loop . . . 75
4.3 Physics Engines and Generative Worlds . . . 76
4.3.1 What Simulation Adds to a Dataset . . . 77
4.3.2 Scenes from Scans and from Generators . . . 78
4.3.3 Agent-Built Scenes and Simulations . . . 79
4.3.4 Contact, Fidelity, and Throughput . . . 80
4.3.5 Learned Simulators and Video World Models . . . 81
4.3.6 Synthetic Demonstrations and Pretraining Corpora . . . 81
4.4 Sim-to-Real Transfer and Real-World Alignment . . . 82
4.4.1 What Simulated and Real Data Each Contribute . . . 82
4.4.2 Where the Reality Gap Comes from . . . 83

4.4.3 Calibrating Simulators and World Models with Real Data . . . 84
4.4.4 Randomization, Adaptation, and Co-Training . . . 85
4.4.5 Validating Transfer and Closing the Loop . . . 86
4.5 Scaling Laws and the Data Flywheel for Physical AI . . . 87
4.5.1 The Data Pyramid . . . 87
4.5.2 Scaling Laws and What They Assume . . . 89
4.5.3 What Robot-Learning Studies Show . . . 90
4.5.4 Valuing and Curating Heterogeneous Data . . . 91
4.5.5 Building a Data Flywheel . . . 93
4.5.6 Choosing What to Collect Next . . . 94

5 World Models **95**
5.1 Concepts and Theoretical Frameworks of World Models . . . 96
5.1.1 Definition, Boundaries, and Basic Properties . . . 96
5.1.2 Internal State and Belief Updating Under Partial Observation . . . 98
5.1.3 Understanding-Oriented and Prediction-Oriented Objectives . . . 98
5.1.4 Functional Roles, Query Interfaces, and System Positioning . . . 99
5.2 Representations and Architectures of World Models . . . 100
5.2.1 Generative Architectures in Observation Space . . . 101
5.2.2 Latent-Space Dynamics Architectures . . . 102
5.2.3 3D, 4D, and Object-Centric Architectures . . . 104
5.2.4 Unified Multimodal and Cross-Paradigm Architectures . . . 105
5.3 Training World Models . . . 107
5.3.1 Self-Supervised and Generative Representation Learning . . . 109
5.3.2 Action-Conditioned Dynamics and World-Action Learning . . . 112
5.3.3 Physical Constraints, Counterfactuals, and Uncertainty Modeling . . . 116
5.3.4 Multi-Step Prediction, Multiple Time Scales, and Adaptation . . . 117
5.4 Using and Validating World Models . . . 119
5.4.1 Interactive Simulation and Conditional Queries . . . 120
5.4.2 Imagination, Planning, and Decision Calls . . . 121
5.4.3 Prediction Quality and Physical-Consistency Tests . . . 122
5.4.4 Decision Utility and Applicability Boundaries . . . 124
5.5 Current Challenges and Future Outlook . . . 125
5.5.1 Acquiring Physical Knowledge from Observation to Intervention . . . 125
5.5.2 Reliable Consequence Prediction in Complex and Long-Horizon Processes . . . 126
5.5.3 Closing the Real-World Loop: Real-Time Operation, Transfer, and Continual Improvement . . . 126

6 Foundation Models for Robot Manipulation **128**

6.1 Emergence and Taxonomy . . . 128

6.1.1 Conceptual Scope and Enabling Conditions . . . 129

6.1.2 From Specialized Policies to Reusable Embodied Models . . . 129

6.1.3 Major Model Lineages and Architectural Evolution . . . 130

6.2 Vision-Language-Action Model . . . 133

6.2.1 Action Representation and Generation . . . 133

6.2.2 Paradigm of Vision-Language-Action Models . . . 134

6.2.3 Extending Multimodal Inputs . . . 137

6.3 Pre-Training, Post-Training, and Capability Shaping . . . 138

6.3.1 Pre-Training: Learning General Embodied Representations and Behavioral Priors . . . 138

6.3.2 Supervised Post-Training: From General Priors to Executable Policies . . . 141

6.3.3 Reinforcement Learning-Based Post-Training: Improving Policies Through Action Outcomes . . . 142

6.3.4 Continual Learning: Turning Deployment Experience into Lasting Capabilities 144

6.4 Multi-Embodiment Modeling and Model Adaptation . . . 145

6.4.1 Embodiment-Aware Data Representation and Alignment . . . 146

6.4.2 Shared Representations, Execution Interfaces, and Adaptation . . . 147

6.4.3 Adaptation Validation and Deployment Handoff . . . 150

6.5 Model Deployment and Efficiency Optimization . . . 152

6.5.1 Deployment Architectures and the Execution Pipeline . . . 152

6.5.2 Model Compression and Inference Acceleration . . . 153

6.5.3 Asynchronous Inference and Real-Time Closed-Loop Control . . . 155

6.5.4 Deployment Evaluation and Operational Maintenance . . . 156

6.6 Dexterous Manipulation Foundation Models . . . 157

6.6.1 Action Representations for Multi-Fingered Hands . . . 157

6.6.2 Learning from Human Hands: Pretraining, Alignment, and Post-Training . . . 158

6.6.3 Sharing Actions Across Hands . . . 159

6.6.4 Where Tactile Input Helps a Dexterous Policy . . . 160

6.6.5 Evaluating Dexterous Foundation Models . . . 161

7 Foundation Models for Whole-Body Motor Control **162**

7.1 Scope, Taxonomy, and Control Interfaces . . . 162

7.1.1 Functional Scope and Foundation Models . . . 162

7.1.2 From Control Principles to Reusable Motor Capabilities . . . 163

7.1.3 Architecture and Interfaces . . . 164

7.1.4 Generality, Dexterity, and Evaluation . . . 166
7.2 Body Dynamics and State Estimation . . . 167
7.2.1 Motion-Control Foundations: Floating-Base Dynamics, Contacts, and Actuators . . . 167
7.2.2 Explicit State Estimation: Attitude, Velocity, Contact, and Local Terrain . . . 169
7.2.3 Implicit Estimation from Observation History . . . 171
7.3 Command-Conditioned Motor Control . . . 172
7.3.1 Command Interfaces and Controller Families . . . 173
7.3.2 A Unified Command Space . . . 175
7.3.3 From Commands to Coordinated Actions . . . 176
7.3.4 Contact Penalties and Swing Targets . . . 176
7.3.5 Environment Rewards and Mirror Regularization . . . 177
7.3.6 Couplings Between Commands . . . 178
7.3.7 Disturbance Rejection and External Control . . . 178
7.4 Motor Pretraining and Dexterous Whole-Body Control . . . 178
7.4.1 Motion Tracking . . . 179
7.4.2 Behavior Foundation Model . . . 180
7.4.3 Evaluation Metrics . . . 182
7.4.4 Dexterous Hands: Motor Control at the Finger Level . . . 184
7.4.5 Contact Feedback and Force-Adaptive Body-Hand Coordination . . . 186
7.5 Real-Time Execution, Recovery, and Safety Control . . . 187
7.5.1 Real-Time Execution and the Sim-to-Real Gap . . . 187
7.5.2 Motion Smoothing and Continuous Execution . . . 189
7.5.3 Sim-to-Real Adaptation . . . 191
7.5.4 Balance Protection and Fall Recovery . . . 191

8 Physical Agents **193**
8.1 Definition of Physical Agent . . . 193
8.1.1 Concept and Boundary of Physical Agent . . . 193
8.1.2 Functional Organization and Interaction Loop . . . 195
8.1.3 Autonomy and the Transition from Automation to Agency . . . 197
8.2 Physical Forms of Physical Agent . . . 199
8.2.1 Entity-Level and Composite Forms of Physical Agent . . . 200
8.2.2 System-Level Physical Agents . . . 202
8.2.3 Correspondence Between Form, Capability, and Application . . . 205
8.3 Physical Goal Representation and Task Grounding . . . 207
8.3.1 Physical Goal Representation and Semantic Grounding . . . 207

8.3.2 Task Decomposition and Action Organization . . . 209
8.3.3 Closed-Loop Execution and Failure Recovery . . . 211
8.4 Multi-Agent Collaboration and Distributed Physical AI . . . 212
8.4.1 Organizational Architectures for Distributed Physical AI . . . 213
8.4.2 Coordination Mechanisms for Distributed Physical AI . . . 214
8.4.3 Open Problems and Trends in Distributed Physical AI . . . 217

9 Hardware and Systems 219
9.1 Hardware and Embodiment Taxonomy for Physical AI . . . 219
9.1.1 From Hardware Components to Embodiment . . . 219
9.1.2 Embodied Model-Hardware Co-Design and Multi-Embodiment Adaptation 220
9.1.3 Task-Oriented Embodiment Selection, Validation, and Evaluation . . . 222
9.2 Sensors, Actuators, and Intelligent Materials . . . 223
9.2.1 Acquiring Physical Experience for Learning . . . 224
9.2.2 Closing the Loop from Measurements to Motion . . . 225
9.2.3 Task-State Observability: Cameras, Encoders, and Contact Sensing . . . 225
9.2.4 Electric Motors and the Mechanics of Actuation . . . 228
9.2.5 Fluid Power, Compliance, and Functional Materials . . . 230
9.2.6 Case Study: RobotEra XHAND 1 . . . 231
9.2.7 Case Study: Unitree G1 and Perceptive Locomotion . . . 233
9.3 Device-Edge-Cloud Collaboration and Real-Time Communication . . . 233
9.3.1 Distributed Architecture . . . 233
9.3.2 Device-Side Computing . . . 234
9.3.3 Edge-Side Computing . . . 235
9.3.4 Cloud-Side Agents . . . 236
9.3.5 Real-Time Communication . . . 236
9.3.6 Self-Driving Laboratories . . . 238
9.4 Software Stacks and Development Infrastructure . . . 239
9.4.1 Layered Software Architecture and Physical Interaction Loops . . . 239
9.4.2 Hardware Abstraction, Sensor Data, and Robotics Middleware . . . 241
9.4.3 Real-Time Runtimes and Learned Policy Integration . . . 242
9.4.4 The Embodied Intelligence Harness and Skill Contracts . . . 244
9.4.5 Development Infrastructure and Evidence for Deployment . . . 244
9.5 Reliability and Deployment at Scale . . . 246
9.5.1 From Successful Attempts to Sustained Physical Output . . . 246
9.5.2 Detecting and Diagnosing Failures from Physical Evidence . . . 247

9.5.3 Recovering Progress and Handling Irreversible Actions . . . 248
9.5.4 Learning from Operation Without Concealing Hardware Drift . . . 249
9.5.5 Deploying Compatible Configurations Across a Fleet . . . 249
9.6 Case Studies in Dexterous Hand Hardware . . . 250
9.6.1 Manipulation Tasks and Contact Requirements . . . 251
9.6.2 Hand Design and the Observation-Action Interface . . . 251
9.6.3 Actuation, Sensing, and Electrical Interfaces . . . 252
9.6.4 Demonstration and Data-Collection Hardware . . . 254
9.6.5 From Demonstrated Skills to Integrated Tasks . . . 256
9.7 Case Studies in Humanoid System Integration . . . 257
9.7.1 Embodiment and Whole-Body Coordination . . . 257
9.7.2 From a Learned Policy to Physical Execution . . . 257
9.7.3 Connecting Body Control, Hands, and Task Models . . . 258
9.7.4 Physical and Perceptual Conditions for Task Execution . . . 259
9.7.5 From Integrated Demonstrations to Sustained Operation . . . 260

III Future Outlook 262

10 Physical AGI 263
10.1 Core Capabilities of Physical AGI . . . 263
10.2 Systematic Generalization in Open-World Settings . . . 266
10.3 Human-AI Collaboration and Social Intelligence . . . 268
10.4 Multi-Agent Systems and Collective Physical AI . . . 269
10.5 Autonomous Evolution . . . 271
10.6 Toward the Ultimate Goal of Physical AGI . . . 273

11 Evaluation, Safety, and Governance for Physical AI 275
11.1 Physical-World Evaluators . . . 275
11.1.1 Evaluation Objectives and the Chapter Framework . . . 276
11.1.2 Evidence-Based Evaluation of Physical Agents . . . 277
11.2 Capability Evaluation and Benchmarking . . . 279
11.2.1 Layered Capability Evaluation of Physical Agents . . . 280
11.2.2 Benchmarking Physical Agents: Current Paradigms and Future Directions . . . 282
11.2.3 Application Evaluation and Governance for Physical Agents . . . 287
11.3 Physical Safety and Action Alignment . . . 289
11.3.1 Physical Agent Security . . . 289

11.3.2 Layered Environmental Safety and Action Alignment . . . 291
11.3.3 Predictive Safety and Degradation . . . 293
11.4 System Safety and Risk Governance . . . 295
11.4.1 Cross-Layer Failures and Cascading Risks . . . 295
11.4.2 Risk-Driven System Safety Assessment . . . 297
11.4.3 From Pre-Deployment Verification to Post-Deployment Safeguards . . . 298
11.5 Law, Ethics, Standards, and Global Governance . . . 300
11.5.1 Responsibility Allocation and Traceability . . . 300
11.5.2 Human Control, Privacy, Fairness, and Ethical Boundaries . . . 300
11.5.3 Legal and Regulatory Frameworks for Physical AI . . . 301
11.5.4 Global Cooperation and Governance . . . 302

12 Strategy and Civilizational Outlook 304
12.1 Development Strategies for Physical AI . . . 304
12.1.1 Converging Digital and Physical Foundation Models . . . 305
12.1.2 Self-Evolving Agents for Open-Ended Tasks . . . 305
12.1.3 Hardware-Software Co-Evolution . . . 306
12.1.4 The Internet of Physical Agents: Architecture and Sustained Operation . . . 307
12.1.5 Building the Scientific and Technological Ecosystem for Physical AI . . . 308
12.2 Physical AI and Human Civilization . . . 309
12.2.1 Civilizational Scale and the Physical Basis of Intelligence . . . 310
12.2.2 Toward a Type I Civilization, Planetary-Scale Physical Systems . . . 311
12.2.3 Toward a Type II Civilization, Space Infrastructure and Stellar-Scale Energy . 313
12.3 Summary and Outlook . . . 315

Contributors 316

Terminology and Abbreviations 317

Notation 327

References 343

Part I
Overview and Foundations

"Pure logical thinking cannot yield us any knowledge of the empirical world; all knowledge of reality starts from experience and ends in it."

— Albert Einstein

1 Introduction to Physical AI

For decades, artificial intelligence (AI) operated inside computers primarily, processing digital data and generating symbolic outputs without directly altering the physical world. Recent advances extend AI beyond this digital boundary by enabling it to perceive and understand physical environments, reason about them, and act within them. This emerging capability has given rise to physical AI, which studies how artificial systems shall interact with the physical world. Yet, what exactly is physical AI? What does its technical landscape look like, and what core challenges does it entail? And, what are typical application scenarios of physical AI? This chapter addresses these questions by tracing AI's expansion from digital to physical space, defining physical AI and its boundaries with neighboring fields, and examining its core characteristics, system architectures, and application domains.

1.1 Background: A Paradigm Expansion of AI

1.1.1 Digital AI and Physical AI

Artificial intelligence has developed through several complementary traditions, from symbolic reasoning and knowledge representation to statistical learning and deep neural networks. Deep learning substantially advances the ability of computational systems to extract useful representations from complex data, with major improvements in speech recognition, visual recognition, and related information-processing tasks. These achievements establish representation learning as a central methodological foundation of contemporary AI (LeCun et al., 2015).

In this chapter, *Digital AI* refers to capabilities that operate directly on digital representations, such as text, images, software, and structured records. This definition emphasizes the system's operational interface over the origin of its data. For example, while an image may depict a physical scene, recognizing its contents does not require the system to actually interact with that environment. *Physical AI*, in contrast, applies intelligence to perceive, influence, and adapt to physical processes through direct interaction. Definition 1.1 in Section 1.2 further clarifies this fundamental distinction.

The distinction becomes apparent when comparing the description of an action with its execution. An AI system may correctly explain how to pick up a glass while remaining unable to estimate its mass, establish a stable grasp, detect incipient slip, or adjust contact forces. Successful manipulation depends on relationships among geometry, material properties, sensing, and actuation that cannot be assessed solely through the linguistic plausibility of a response. The relevant criterion is whether the system produces the intended physical outcome under the conditions encountered.

This distinction is related to the symbol grounding problem of how symbols can acquire meaning through a system's own capacities rather than solely through interpretation by an external observer. The formulation by Harnad (1990) identifies the limitations of defining symbols only through other symbols and motivates their connection to nonsymbolic representations. For physical AI, the corresponding engineering problem is to connect concepts such as *support*, *contact*, and *stability* to observations and action consequences.

Grounding should not, however, be reduced to a categorical claim that digital models possess no physical knowledge. Models trained on descriptions, images, videos, or simulated trajectories can acquire useful regularities concerning the physical world. The unresolved issue is whether these regularities support reliable intervention across changes in objects, environments, and operating conditions. Semantic competence, predictive accuracy, and physical competence are related but distinct properties.

Figure 1.1 summarizes this expansion from digital tools and simulated agents to real-world single-agent systems, multi-agent collaboration, and infrastructure-level physical AI. Across these physical deployments, the unit of analysis extends from an isolated model to the integrated system of computation, sensing, actuation, communication, and environment. This perspective has important historical precedents. Brooks (1991) demonstrates an approach to robotics in which intel-

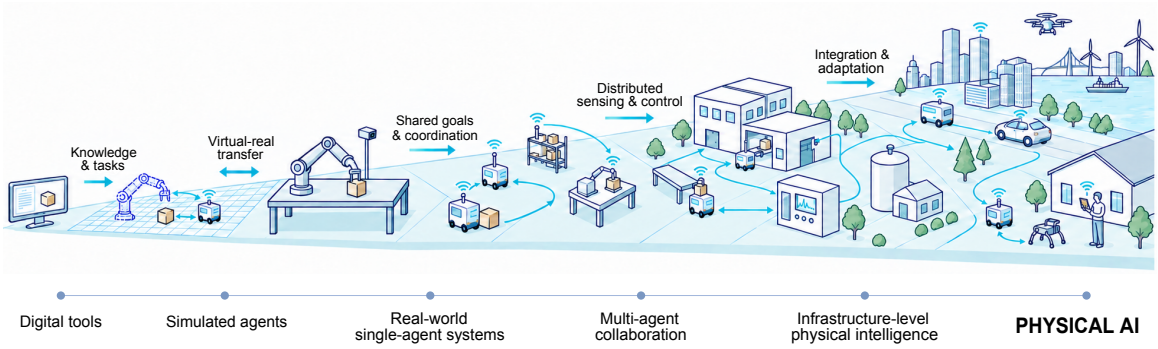


Fig. 1.1 A conceptual expansion in the scope and organization of AI interaction with the physical world, from digital tools and simulated agents to real-world single-agent systems, multi-agent collaboration, and infrastructure-level physical AI. The real-world forms shown fall within the scope of physical AI adopted in this monograph

ligent behavior arises through direct perception-action coupling, challenging the assumption that autonomous behavior must depend on a centralized symbolic description of the world. Contemporary physical AI extends this longstanding concern with situated action by incorporating learned representations, scalable training, and predictive models.

1.1.2 Core Characteristics

Physical AI operates under constraints that cannot be removed by increasing model capacity alone. Three characteristics are particularly important, namely, consequential interaction under uncertainty, perception of evolving physical states, and the integration of empirical learning with physical structure.

Consequential Interaction Under Uncertainty Physical systems are only partially observable. Object properties, external disturbances, and equipment conditions may be unknown or change during execution. Actions also affect the evidence available to the system. Moving a camera changes visibility, probing a surface reveals compliance, and applying force may alter an object's configuration. Perception and action consequently form an interdependent process.

Physical interaction can incur recovery costs and irreversible consequences. A simulated episode can often be restarted from a saved state, whereas a spilled chemical, damaged component, or contaminated sample may require substantial intervention or may be impossible to restore. The operational distinction is that physical systems generally lack an exact, inexpensive reset mechanism. Planning must therefore consider the recoverability of future states as well as the likelihood of immediate task success.

Perception of Evolving Physical States Physical AI requires observations that are temporally and spatially meaningful. Vision can describe appearance and geometry, while force, touch, proprioception, temperature, pressure, and other measurements reveal complementary aspects of interaction. A manipulation system, for example, may need to distinguish visual contact from load-bearing contact and a completed motion from a successfully transferred object.

Although sensors typically produce sampled digital measurements, the processes they measure evolve in continuous time and may include discrete events such as impact, grasp release, or equipment switching. Reliable interpretation therefore depends on sampling rates, calibration, synchronization, coordinate frames, and measurement uncertainty. A correct estimate obtained too late may be unusable for control. More generally, computation and physical dynamics must be analyzed together because execution timing can affect system behavior (Lee, 2008).

Learning Informed by Physical Structure Data provide evidence about real operating conditions, including effects that are difficult to model analytically. Physical knowledge provides structure that can restrict implausible predictions and reduce the burden of learning. Conservation relationships, symmetries, geometric constraints, and governing equations can be incorporated through model architectures, training objectives, simulation, or constrained decision procedures. Physics-informed machine learning formalizes several ways of combining observations with mathematical descriptions of physical processes (Karniadakis et al., 2021).

The appropriate balance depends on the task. Explicit models may be effective when governing relationships are sufficiently understood, whereas learned components can represent uncertain interactions or compensate for model mismatch. Physical priors are themselves conditional on assumptions and operating regimes. For example, an inaccurate friction model or an inappropriate rigid-body approximation can produce systematic errors. Physical consistency therefore requires validation within the task's operating regime, regardless of the model family.

Together, these characteristics shift evaluation toward closed-loop performance. Relevant measures include task completion, robustness to disturbance, response time, resource consumption, and recovery from failure. Prediction accuracy remains valuable, but its practical significance depends on how prediction errors propagate through decisions and physical execution.

1.1.3 Why Physical AI Is Needed and Increasingly Feasible

Physical AI is motivated by tasks that require systems to act on the physical world. Flexible manufacturing requires adaptation to variations in parts and processes. Transportation and energy systems require coordinated responses to changing demand and operating conditions. Scientific experimentation requires procedures to be adjusted in response to measurements. Furthermore, extreme environments, such as deep space and the deep sea, drive the need for local autonomy, as communication constraints and physical hazards severely limit continuous human intervention.

These demands do not imply that established automation is inadequate in every setting. Carefully engineered automation remains effective where tasks and operating conditions can be specified in advance. The additional research challenge is to preserve useful performance when relevant conditions change and exhaustive preprogramming becomes impractical. Physical AI addresses this challenge by integrating interactive learning with perception, inference, prediction, and adaptation.

Several technical developments support this direction. Sensing and local computation provide the practical interfaces for observing and influencing physical processes. Simulation supplies controlled environments for training and evaluation, while methods for transfer from simulation to reality reduce dependence on collecting every training example on physical hardware. For example, Tobin et al. (2017) demonstrate that varying simulated visual conditions can support transfer of a vision model to real robotic tasks. Such results establish a useful transfer mechanism, while leaving its effectiveness dependent on the relationship between training variation and deployment conditions.

Learned physical models provide another foundation. Graph network simulators have demonstrated the ability to represent interacting particles and predict dynamics across fluids, rigid solids, and deformable materials within evaluated settings. These models illustrate how structured learning can support physical prediction beyond a single fixed configuration (Sanchez-Gonzalez et al., 2020). For autonomous action, their potential value lies in evaluating candidate interventions before execution, provided that prediction errors and operating limits are understood.

Large pretrained models also offer reusable semantic knowledge. For example, RT-2 shows that vision-language pretraining can be combined with robot demonstrations to support robotic action and improve generalization in the reported experiments (Zitkovich et al., 2023). This provides evidence that capabilities developed through digital learning can contribute to physical interaction when connected to appropriate action data and interfaces.

These developments make it increasingly feasible to combine digital capabilities, physical models, and control mechanisms to study how systems acquire transferable physical skills while maintaining measurable performance within specified operating conditions.

1.2 Physical AI: Definition and Boundaries

1.2.1 A Broad Definition of Physical AI

At its most general, physical AI concerns the capacity of an artificial system to interact with the physical world. This monograph uses *physical AI* for this artificial-system scope. The term *physical intelligence* is retained for general capacities, bodily mechanisms, and source-specific usage. The following definition makes the notion precise.

Definition 1.1 (Physical AI)**.** Physical AI is the capacity of an artificial system to perceive physical environment states, maintain task-relevant representations of physical relationships and action consequences, and select and execute interventions through physical interfaces, using feedback to achieve objectives under physical, operational, and computational constraints.

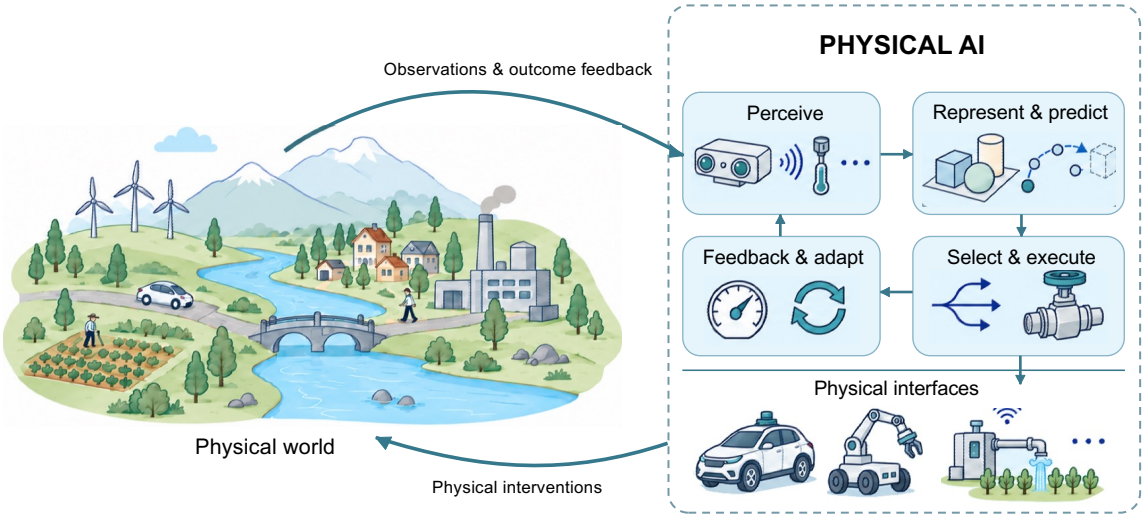


Fig. 1.2 Physical AI as closed-loop interaction between artificial systems and the physical world. Systems observe physical states, maintain task-relevant representations of physical relationships and action consequences, and select and execute interventions through physical interfaces. Outcome feedback guides adaptation toward objectives under physical, operational, and computational constraints

This definition identifies a functional scope rather than a particular architecture. Representations may be explicit, implicit, or distributed across several components. They need not constitute a complete internal simulator, and action selection need not depend on symbolic reasoning. What matters is whether the integrated system can use information about physical conditions to guide and revise its behavior.

Four implications clarify the definition. First, physical grounding requires observations to be connected to relevant quantities, events, or conditions in the world. Second, the system must have a path for intervention, whether through direct actuation or commands to controlled equipment. Third, feedback must inform subsequent behavior, allowing discrepancies between expected and observed outcomes to affect execution. Fourth, claims of intelligent capability should be supported by evidence of task-relevant inference, prediction, planning, or adaptation, rather than inferred merely from the presence of sensors and actuators.

These criteria define a continuum of capability. A fixed feedback regulator and a system that adapts across changing tasks may share essential control principles, yet differ substantially in the breadth of conditions they can handle. The distinction between automation and Physical AI is therefore most useful when expressed through demonstrated capabilities and operating assumptions, rather than an absolute technological boundary.

Terminological care is also necessary because *physical intelligence* has an established, more specific usage in mechanics and robotics. Sitti (2021) uses the term to describe functions such as sensing, actuation, memory, and computation physically encoded in an agent's body. The broader definition of Physical AI adopted here includes such contributions while also encompassing computational and distributed systems that interact with physical processes. It should be understood as this chapter's organizing framework, rather than a claim that the literature has converged on one definition.

Finally, key components should be distinguished from a complete system. A learned simulator, perception model, or action planner may provide an essential Physical AI capability. In isolation, however, it does not satisfy the system-level requirement for closed-loop physical intervention. This distinction allows purely digital modules to be recognized as enabling technologies without attributing autonomous physical competence to them prematurely.

1.2.2 Physical AI vs. Embodied AI

Embodied AI posits that artificial intelligence requires a *body* to house the system and drive its active sensorimotor relationship with the environment (Pfeifer and Iida, 2004). The concept of embodiment focuses on how this morphology, along with sensing and action capabilities, directly shapes environmental interactions and guides behavior. Importantly, embodiment is by no means limited to humanoid or biologically inspired designs. For physically deployed agents, research further highlights how mechanical structures and material properties can intrinsically support adaptive behavior, thereby reducing the control burden on a centralized controller (Pfeifer et al., 2007).

Nor does embodied AI necessarily require a deployed physical robot. Simulated agents are widely used to study navigation, interaction, and learning under embodied constraints (Shridhar et al., 2021). Habitat, for example, was introduced as a platform for training and evaluating virtual embodied agents in three-dimensional environments (Savva et al., 2019). Embodied AI therefore includes both simulated and physically deployed agents.

Within this chapter's framework, physically deployed embodied AI constitutes a major realization of physical AI. Its characteristic analytical focus is the agent and its particular sensing and action capabilities. Physical AI adopts an additional system-level perspective, encompassing arrangements in which sensing, decision-making, and actuation are distributed across infrastructure. Figure 1.3 summarizes these distinctions and their overlap.

Consider an adaptive traffic-management system that estimates network conditions, changes signal timings, and evaluates the resulting traffic flows. Its interaction with the world is mediated by distributed detectors, communication links, and signal controllers rather than a single mobile body. Similar arrangements can be considered for factory coordination or power-system operation, provided that the system actually closes the loop between observations, decisions, and physical consequences. These are illustrative applications of the proposed definition, not claims that every traffic controller, factory scheduler, or grid-management platform possesses advanced intelligence.

Scientific equipment provides a concrete research example. Degrave and colleagues demonstrated deep reinforcement learning for magnetic control of plasma configurations in the TCV tokamak. The controller interacts with a physical process through the machine's measurement and actuation infrastructure (Degrave et al., 2022). Under the present framework, this qualifies as Physical AI even though the system lacks the conventional form of a mobile or manipulative robot.

Such systems are not independent of physical embodiment in the broadest sense, because sensors, actuators, and material processes remain indispensable. Their distinguishing feature is that the physical interface is distributed across an installation or network. Within this monograph's framework, embodied AI emphasizes how an agent's body shapes sensing and action, while physical

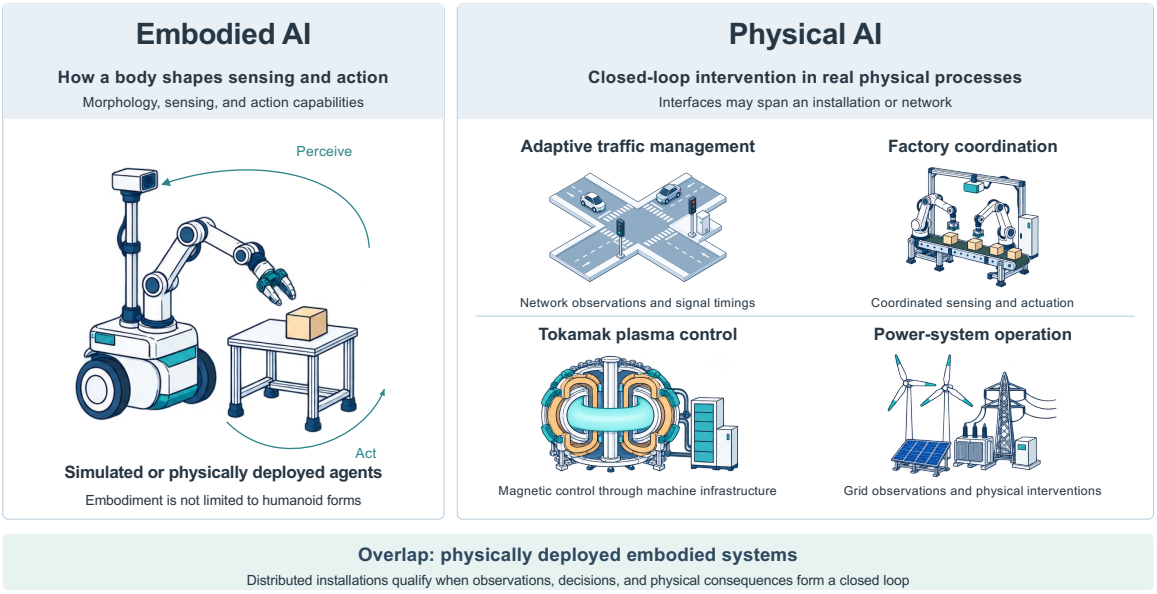


Fig. 1.3 Embodied AI and physical AI within this monograph's framework. Embodied AI emphasizes how an agent's body shapes sensing and action. Physical AI emphasizes closed-loop intervention in real physical processes. The two overlap in physically deployed embodied systems

AI emphasizes closed-loop intervention in real physical processes, including through distributed infrastructure. The two overlap in physically deployed embodied systems.

1.2.3 Physical AI vs. Cyber-Physical Systems and Digital Twins

Physical AI intersects closely with cyber-physical systems (CPS), but the concepts describe different aspects of a system. CPS concerns the integration of computation with physical processes, including the consequences of timing, concurrency, and interaction between software and its physical environment (Lee, 2008). Physical AI concerns the capabilities exercised through that integration, including interpreting conditions, anticipating consequences, choosing interventions, and adapting behavior.

A CPS may operate effectively through fixed control logic without possessing broad inference or adaptation capabilities. Conversely, a physically intelligent system depends on the engineering properties of its computational and physical infrastructure, including dependable sensing, timely communication, and executable control. Physical AI can therefore be understood as a capability layer within many CPS architectures. Describing it as a possible development toward greater autonomy is useful, provided that this does not imply that CPS research excludes advanced cognition or that learning automatically improves an engineered system.

Digital twins occupy a different position. They provide digital representations associated with identifiable physical counterparts, with connections that support updating and comparison between physical and digital states. The literature identifies multiple characteristics and implementation choices, making it important to specify how a proposed twin is connected to its physical counterpart (Jones et al., 2020).

Within a Physical AI architecture, a predictive digital twin can support state estimation, diagnosis, and evaluation of alternative interventions. It may help estimate how a production line would respond to a schedule change or how equipment behavior would change under another operating condition. Such uses require more than a visually accurate replica. The representation must capture the variables and dynamics relevant to the decision.

A digital twin does not necessarily select or execute actions autonomously, and a Physical AI system need not maintain a comprehensive twin. A compact learned dynamics model may be sufficient for a particular control task. Likewise, a generic world model becomes a digital twin only when it is meaningfully associated with a specific physical counterpart and maintained through an appropriate update relationship. Counterfactual predictions remain conditional on the validity of the model and its assumptions.

These distinctions establish complementary roles. CPS describes the integration of computational and physical processes. Embodied AI examines intelligence through situated sensorimotor interaction. Digital twins provide representations tied to physical counterparts. Physical AI, in the framework developed here, identifies the capacity to combine perception, physical understanding, decision-making, and feedback into effective intervention. This system-level perspective provides the basis for the architectural elements discussed in the following sections.

1.3 Core Elements of Physical AI

Definition 1.1 characterizes Physical AI as the capacity to perceive physical states, maintain task-relevant representations, and execute interventions through feedback. In practice, this capacity takes the form of a persistent closed loop. Information processing and decision-making must be translated into physical actions, and subsequent behavior must be revised according to observed outcomes. Physical AI therefore operates through a cycle of perception, modeling, prediction,

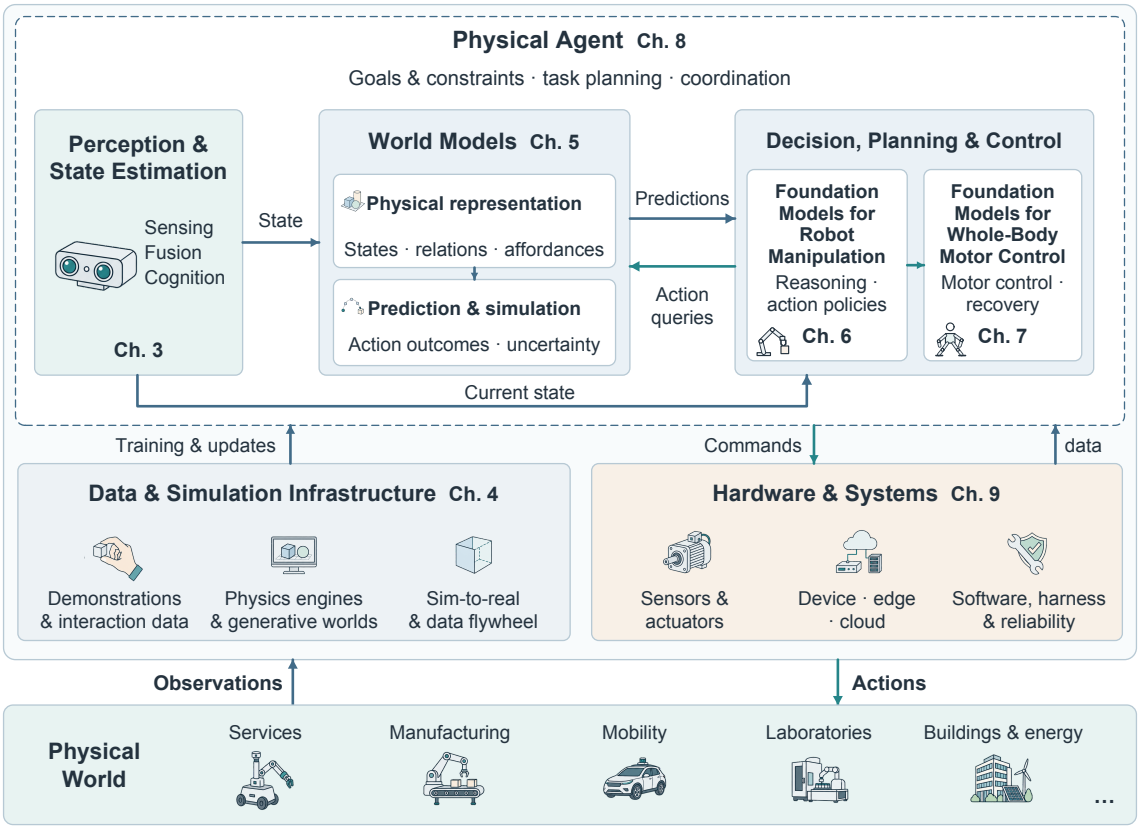


Fig. 1.4 Key elements of Physical AI and their relationships. Chapter labels in the diagram refer to Chapters 3 to 9

decision-making, execution, and feedback, rather than through a one-shot mapping from input to output.

As Section 1.2 emphasizes, Physical AI is not synonymous with robotic intelligence. Whether a system exhibits it depends on its ability to close the loop between perception and action, not on its morphology. The elements discussed in this section apply equally to autonomous vehicles, industrial equipment, automated experimentation platforms, intelligent buildings, and energy infrastructures. In such systems, perception, computation, decision-making, and execution are often distributed across devices, edge nodes, cloud platforms, and physical facilities rather than concentrated within a single device.

Operating in a continuously changing and uncertain world also means that actions are constrained by dynamics, geometry, time, energy, materials, and safety requirements, and that their consequences are not always reversible, as Section 1.1 discusses. A physically intelligent system must therefore answer not only "What should be done?" but also "Can it be done under the current conditions?", "How can it be done safely?", and "Does the execution produce the intended outcome?" Organized around these questions, Physical AI comprises four closely related elements, including physical state perception and estimation, physical world models, physical decision-making together with planning and control, and closed-loop feedback and multiscale adaptation. They correspond respectively to perception, representation, intervention, and feedback in the definition. The first three together constitute the physically intelligent agent, while the fourth connects the agent with the physical world into a continuously running loop. The relations among these elements, together with the data, hardware, and physical world on which they depend, are illustrated in Figure 1.4.

1.3.1 Physical State Perception and Estimation

Physical AI begins with information about the environment, task-relevant objects, and the system itself. Vision conveys identity, pose, geometry, and motion. Force and tactile sensing reveal contact, friction, and material properties (Dahiya et al., 2010), while acoustic sensing detects collisions and vibrations. Proprioception measures the system's own configuration and load, and industrial systems add process variables such as temperature, pressure, and current. Chapter 9 treats the sensors themselves. The concern here is turning their signals into an estimate of physical state.

Measurements are incomplete, noisy, delayed, and often misaligned across modalities, so calibration, multimodal fusion, and state estimation are needed to obtain a temporally consistent state with explicit uncertainty (Kalman, 1960a; Baltrušaitis et al., 2019). Perception also supports active information acquisition. When observations are insufficient for a decision, the system can change its viewpoint, reposition sensors, or probe objects to estimate mass, stiffness, or friction, folding information acquisition into the decision loop (Bajcsy, 1988).

1.3.2 Physical World Models: Representation and Prediction

State estimates are organized into a world model, which describes the environment, the actions available to the system, and how physical states evolve over time (Ha and Schmidhuber, 2018; LeCun, 2022). It has two interdependent aspects. Representation captures what the model holds about the current world, and prediction anticipates what the world becomes after a given action.

Physical representations cover objects, poses, geometry, and spatial relations, together with velocities, masses, contacts, forces, material properties, affordances, and task constraints. A cup is at once a shape with a pose and an object that can be grasped, holds liquid, and spills when tilted. Representations may be explicit (maps, object lists, equations of motion, relational graphs), implicit (learned from data), or hybrid, combining physical priors with learning to retain interpretability while correcting for unmodeled effects. An effective world model is updated with each new observation and distinguishes directly observed, estimated, and unobserved states.

Prediction estimates the consequences of a particular action, covering trajectories, contact outcomes, equipment conditions, task success, and safety hazards over time scales from milliseconds

to hours. Predictions may come from physical equations and simulators (Raissi et al., 2019) or from transition patterns learned from data (Sanchez-Gonzalez et al., 2020). Practical systems combine the two and revise predictions and their uncertainty online. In scenario simulation, the decision process submits candidate actions and the world model returns predicted outcomes with their uncertainty, enabling counterfactual comparison before anything is executed. This simulation runs online, unlike the physics engines and generative environments of Chapter 4, which are used offline to produce training data and test policies.

1.3.3 Physical Decision-Making, Planning, and Control

Physical decision-making selects actions from task objectives, the current state, predicted outcomes, and real-world constraints, usually through a hierarchy (Kaelbling and Lozano-Pérez, 2011; Sutton et al., 1999). At the top, the system sets goals and constraints and, when several physical agents share a task, allocates and coordinates subtasks among them. Foundation models for robot manipulation reason about the task and scene, decompose goals into executable steps, and produce action policies. Foundation models for whole-body motor control provide reusable feedback policies for executing movement goals and correcting deviations (Zeng et al., 2025; 2026). The levels differ mainly in time scale, from minutes or hours for task allocation to seconds for policies and milliseconds for motor commands. The boundary separating task-level reasoning and action-policy generation from motor control and feedback correction varies across systems.

Execution takes many forms, from joints and end effectors to steering and braking, instrument scheduling, or valves, pumps, and switches. An action is thus best understood as an observable intervention that changes a physical object, environment, or process. Such interventions must be feasible, safe, and timely. A logically valid plan can still fail because of collisions, actuator limits, or latency, so safety checks precede execution, and monitoring and emergency stops stay active throughout.

1.3.4 Closed-Loop Feedback and Multiscale Adaptation

Physical AI revises behavior according to the observed consequences of action. After each command, the system perceives again, checks whether the intended result is achieved, and updates its state estimate together with its world model and plan. When outcomes deviate from predictions, it identifies the cause and compensates, retries, replans, or terminates safely.

The loop resides in no single module but in the connections among the elements. Perception supplies the state, the world model answers action queries from the decision process, commands act on the world through actuators, and sensors return the consequences to perception (Figure 1.4). The loop thus closes through the physical world rather than inside the agent. Fast reactive control, such as tightening a grip when an object begins to slip, may bypass the world model and respond directly to the current state.

Feedback operates at three nested scales. The fastest loop connects motor control and feedback correction with the actuators and sensors, adjusting position, velocity, or force in real time. The task-level loop connects task-level reasoning and action-policy generation with the world model and revises plans as the task progresses. The slowest passes through the offline data and simulation infrastructure, where deployment data drive continual learning (Parisi et al., 2019) and form a data flywheel. In multi-agent and infrastructure-scale systems these loops may be distributed across devices that coordinate toward a common objective, and humans may join them by providing goals, demonstrations, supervision, and intervention.

Physical AI therefore lies not in producing a plausible action or plan, but in sustaining this cycle of observation, prediction, action, verification, and revision under real physical constraints, whether the system is a single robot or distributed infrastructure.

1.4 The Role of Physical AI in the Evolution of AI

Section 1.1 contrasts digital intelligence, whose objects of computation and evaluation are recorded content, with the Physical AI defined in Definition 1.1. This subsection develops three claims about what that contrast means for the trajectory of AI. Physical AI is the stage at which artificial intelligence moves from understanding and generating digital information to perceiving, predicting, and changing physical processes. It builds on digital intelligence rather than replacing it and provides one of the important paths toward artificial general intelligence (AGI).

New Capabilities Beyond Digital Intelligence The first extension concerns what the system perceives. Digital intelligence works on symbols and pixels that come from prior collection, curation, and encoding. Physical AI deals with physical states directly (Section 1.1), including the pose and velocity of objects, forces and friction at contact surfaces, pressure and temperature in fluids, and the composition and structure of materials. These states are continuous and evolve over time. They can only be partially observed through noisy sensors, and vision, touch, force, and other modalities must be fused within a common spatiotemporal frame. The object of perception thus shifts from previously recorded content to the current state of a world that keeps changing.

The second extension concerns what the system produces. Because physical systems lack an inexpensive reset (Section 1.1), a grasp, an injection, or a turn of the steering wheel changes the subsequent state of the system and often cannot be undone. Physical AI must therefore predict, before acting, how the world will evolve if a given action is taken. This prediction is interventional in nature and depends on causal relations and dynamics, not merely on statistical correlation (Schölkopf et al., 2021).

Building on Digital Intelligence These new capabilities do not make Physical AI a replacement for digital intelligence. The common-sense and domain knowledge accumulated in foundation models is the basis on which Physical AI understands and plans tasks (Ichter et al., 2023). In turn, Physical AI grounds that knowledge in measurable changes of state and tests it through real interaction. A large language model can search the literature, propose a hypothesis, and write an experimental protocol, but whether the protocol succeeds depends on accurate pipetting, stable temperature control, and correct sample handling. Only when digital reasoning is closed with physical execution can AI take part in the full process from posing a question and running an experiment to drawing a conclusion.

Section 1.1 explains that Physical AI enlarges the unit of analysis from the model to the system, and this enlargement has concrete consequences for how capability is attributed. The same policy may perform very differently on robot arms of different precision or in workshops with different lighting, and task success also depends on engineering conditions such as sensor calibration, fixture design, and safety interlocks. The shift also changes how general capability should be understood and measured, which leads to the third claim.

A Path Toward AGI Benchmarks in digital environments are prone to overfitting, and it is hard to rule out memorization caused by contamination of the training data. The physical world is open, continuous, and impossible to enumerate in advance, and whether a task has been completed is decided directly by physical outcomes. Moravec's paradox points in the same direction, highlighting that perceptual and motor skills that humans find effortless are among the hardest for machines to acquire (Moravec, 1988). Physical AI therefore provides a demanding test of generality along four axes, including generalization to new objects and scenes as well as new task combinations, transfer across embodiments and tasks as well as from simulation to reality (Tobin et al., 2017; OpenAI et al., 2020), robustness to disturbances and sensor degradation as well as environmental drift, and long-term autonomy over days or months with little human intervention. Because mistakes in the physical world are often irreversible, evaluation cannot rest on task success rate alone. Real-time performance, sample efficiency, energy consumption, risk controllability, and recoverability after failure must be considered together, and they frequently trade off against one another. Physical AI is not the only such path, as mathematical reasoning, scientific discovery, and social interaction

advance general capability from other directions, but it is the one in which capability is checked against physical reality.

Locating Physical AI on the Trajectory of AI Taking the degree of coupling between AI and the physical world as the axis, the position of Physical AI can be stated more concretely. At the loosely coupled end, AI acts as a *digital tool* that assists humans with design, analysis, and decision-making but leaves execution entirely to them. Next, *simulated agents* perceive and act in simulators or digital twins (Jones et al., 2020), where trial and error is cheap and safe but limited by the gap between simulation and reality. Physical AI proper begins when AI closes the loop in the real world. It starts with *single-agent systems*, such as a robot, vehicle, drone, or autonomous laboratory platform, where most current embodied intelligence research is concentrated. It then extends to *multi-agent collaboration*, in which groups of agents accomplish tasks that no single agent can, and finally to *infrastructure-level Physical AI*, in which distributed sensing, computing, and actuation resources embedded in cities, power grids, factories, and research facilities are scheduled and governed through common protocols. Section 1.5.6 returns to these three stages as the recurring physical compositions of deployed systems. In a network of autonomous laboratories spread across institutions, for example, the question is no longer only whether an experiment can be completed, but also how resources are allocated, how processes are audited, and how data ownership and reproducibility are established. These stages coexist and support one another rather than replacing each other. Digital tools and simulation continue to supply knowledge, data, and safety validation to physical systems. Physical AI thus occupies the tightly coupled end of this axis, where the value of AI is realized as a change in physical processes rather than only as the production of information. The application landscape in Section 1.5 makes this coupling concrete across industries, and Chapter 10 examines Physical AGI as a long-term research goal.

1.5 The Application Landscape of Physical AI

Physical AI connects the interpretation of physical state with decisions that change it, and its applications span individual robots, autonomous vehicles, instrument networks, factories, cities, and scientific facilities. These settings differ in scale by orders of magnitude, yet the questions that determine whether a system is useful remain largely the same, concerning what the task requires, what the environment allows, what resources are available, and which decisions may be taken without a human in the loop. Whether a particular configuration amounts to a deployable system is ultimately reflected in the cost of the valid work it produces. The subsections that follow first state the closed loop that every application instantiates, together with the four dimensions along which applications are described, namely *task*, *environment*, *resources*, and *autonomy*. They then survey four domains, from industrial operation through everyday services and public infrastructure to scientific inquiry, and finally extract the system archetypes that recur across those domains.

1.5.1 The Closed Loop and the Distinction from Conventional Automation

Every application of Physical AI instantiates one closed loop as shown in Figure 1.5. *Physical perception* acquires signals from the environment through sensors. *State understanding* assembles these signals into estimates of the objects, agents, and processes present, together with the representations and world models of how they evolve. *Prediction and imagination* extrapolate the estimated state forward and evaluate candidate actions, whether through learned dynamics or explicit physical simulation. *Decision making and planning* select and sequence actions under task and process constraints. *Physical action* executes the selected behavior through control and actuation, and the resulting *environmental feedback* becomes the input to the next cycle of perception. These stages give Definition 1.1 its operational form. Section 1.3 groups them into four elements comprising physical state perception and estimation, world modeling for representation and prediction, decision-making with planning and control, and closed-loop feedback with multiscale adaptation. The loop as a whole, rather than any single stage, defines an application. A system that perceives without acting merely observes its environment, and one that acts without state understanding merely executes a prescribed motion.

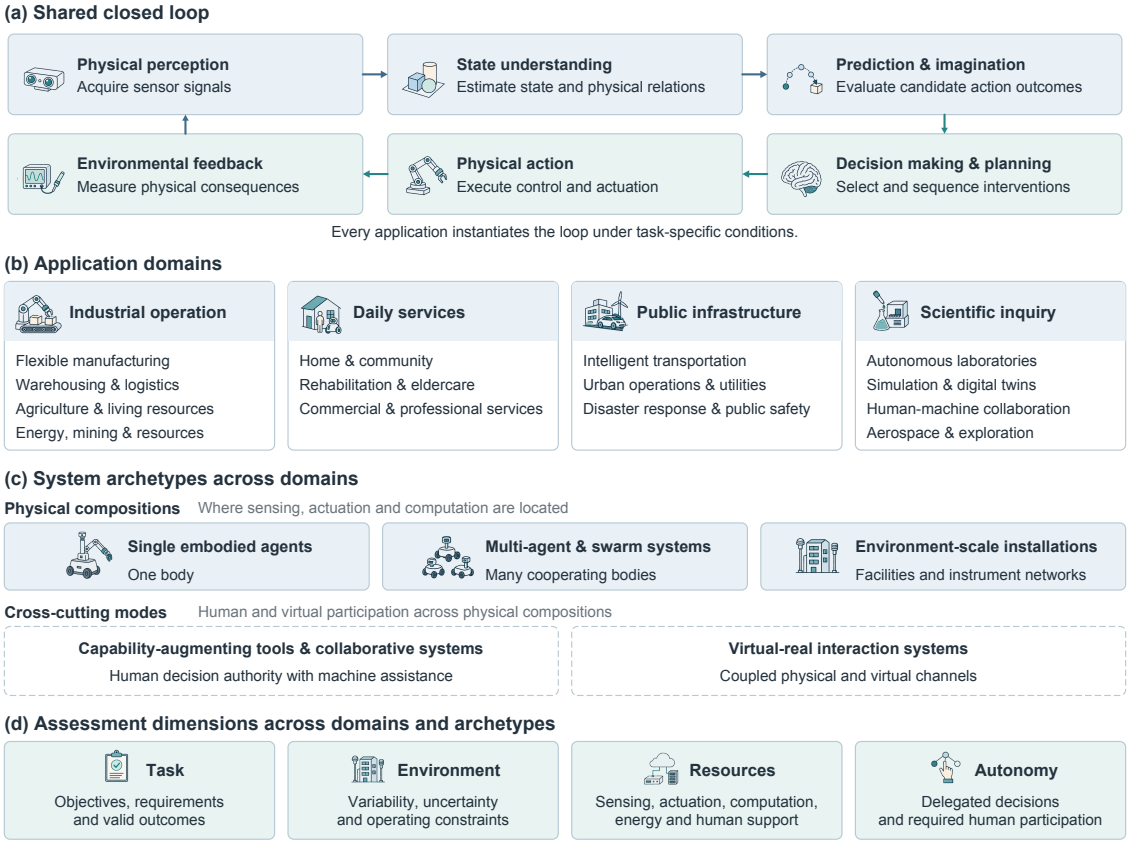


Fig. 1.5 The application landscape of Physical AI. All applications instantiate the same closed loop. The four domains illustrate its operation under different conditions. Five system archetypes (three physical compositions and two cross-cutting modes) and four assessment dimensions apply across domains

Conventional automation already incorporates feedback control, so the loop itself is not new (Åström and Murray, 2008). What distinguishes Physical AI is how the loop is sustained when conditions depart from those anticipated at design time, through adapting to changing conditions, interpreting unfamiliar tasks, and transferring knowledge across operations. Classical control holds a process near a setpoint under modeled disturbances. Physical AI re-estimates state, re-plans, and recovers when the task or the environment changes. The distinction is graded rather than binary. Applications occupy a spectrum from fixed procedures with local adaptation, such as an industrial manipulator re-grasping a shifted part, to open-ended task interpretation, in which a system must infer what is wanted from instruction, demonstration, or context (Ichter et al., 2023).

The position of an application on this spectrum is determined by the task rather than by the industry in which it appears. An application can be characterized along four dimensions, namely task, environment, resources, and autonomy. Resources include sensing, actuation, computation, energy, and human support. Autonomy specifies which decisions are delegated and when human participation is required (Parasuraman et al., 2000). These dimensions distinguish requirements that industry categories tend to merge. Within one factory, moving sealed containers and manipulating deformable materials impose different demands on contact control and recovery, and a laboratory may coordinate fixed instruments without a single mobile body. System composition and decision authority therefore require separate descriptions.

1.5.2 From Automation to Autonomous Industrial Operation

Industrial production, logistics, and agriculture involve transportation, manipulation, inspection, and process regulation. Physical AI offers value when variability makes fixed procedures costly to maintain. Parts arrive in unfamiliar configurations, crops vary in maturity, or equipment behavior changes with wear. The underlying pattern is common to these sectors. Fixed, engineered procedures are replaced by state estimation and adapted planning, while the process constraints within which the adaptation must operate remain fixed.

Flexible Manufacturing and Lights-Out Production In an adaptive production system, visual and tactile observations update estimates of object and process state, while planning and control adjust operations within process constraints (Lee et al., 2015; Monostori et al., 2016). Variability enters as small batches, frequent product changeovers, and parts presented in unfixtured configurations, and the value is realized when changeover no longer requires re-engineering (Van Brussel et al., 1998). The benefit is measured by product quality, changeover time, resource consumption, and the effort of setting up a new task, while task variability and the resources required for changeover are the binding constraints. The limit of this development is the *lights-out factory*, in which no standing human presence remains on the floor. Lights-out operation does not eliminate human work. It moves that work to exception handling, supervision, and maintenance, and the cost of these activities becomes a primary economic variable rather than an overhead (Bainbridge, 1983).

Warehousing, Logistics, and Supply-Chain Orchestration Warehouses and distribution networks concentrate transportation and manipulation at high repetition rates, including automated storage and retrieval, mobile-robot fleets, sortation, palletizing and depalletizing, and parcel handling (Wurman et al., 2008). The distinguishing problem is not the execution of any single motion but *orchestration*, which involves allocating physical work across many machines as demand fluctuates, replanning after disruption, and keeping physical flows consistent with the inventory records and delivery commitments carried by information systems (Gerkey and Matarić, 2004). Local motion competence and network-level coordination are separate achievements. An operation can automate the former while the latter remains manual, and the gap between the two is closed in practice either by consistent state estimates or by buffer stock.

Agriculture, Forestry, Fishery, and Animal Husbandry Agricultural settings concentrate the constraints of Physical AI, including biological variability, unstructured outdoor terrain, weather-dependent timing, and long deployment cycles. Operations under development span the production sequence, including inspection and patrol, seeding and planting, mechanical weeding, fertilization, plant protection, harvesting, and grading and sorting of produce (Bac et al., 2014), together with livestock monitoring and aquaculture operations (Berckmans, 2017; Føre et al., 2018). Value appears in two forms, namely precision, in which seed, chemical, and water inputs are reduced by acting at the resolution of individual plants or animals, and timeliness, in which operations are performed within short agronomic windows. The acceptance criteria are correspondingly biological and seasonal rather than volumetric, such as fruit harvested without bruising at the correct maturity, weeding achieved with a specified reduction in chemical use, or illness detected early enough for intervention.

Energy, Mining, and Resource Systems Energy and mineral systems are high-risk and large-scale, including generation equipment, transmission corridors, pipelines, refineries, and open-pit and underground mines. Applications combine inspection of assets (such as turbine blades, transmission lines, and pipe corridors) (Nguyen et al., 2018), autonomous haulage and drilling (Wang and Huang, 2017; Marshall et al., 2008), and process regulation distributed across many devices. Equipment behavior changes with wear, which motivates continuously updated estimates of degradation in place of fixed maintenance schedules (Jardine et al., 2006). Limited access and the high consequence of failure shift the economics toward predictive operation and remote supervision. In these systems, safety cases rather than throughput determine which functions may be

automated, and the allocation of authority between site automation and remote operation centers is itself a design decision.

1.5.3 Daily Services: From Dedicated Equipment to Embodied Companions

Service applications place Physical AI in environments that are built for people rather than for machines. At one end are dedicated devices (single-purpose appliances that perform one task under human setup), and at the other are personalized embodied companions intended to share unstructured space with untrained users. Consumer-grade embodied products, from desktop companions to inexpensive legged and humanoid platforms, make this trajectory visible at retail prices, although the distance between demonstrations of interactive behavior and dependable household service remains substantial (Srinivasa et al., 2012).

Human participation is an explicit design variable in these applications. Supervision, shared control, and selective delegation assign different responsibilities to users, operators, and machines. In human-facing services, decision authority and timely assistance matter alongside completion rates, so evaluation includes service quality, accessibility, and the burden placed on users and caregivers (Hoffman, 2019; Nikolaidis et al., 2016).

Home and Community Services A household assistant must accommodate changing layouts and object locations, because neither residents nor their belongings remain arranged to suit a stored map (Wisspeintner et al., 2009). Cleaning, tidying, laundry, and meal preparation combine manipulation with continual re-estimation of where things are and what state they are in. Community services extend the same capabilities into semi-public space, including delivery to doors and lockers, patrol and monitoring, and check-ins on residents who live alone. In these settings, acceptance is determined not by peak capability but by the frequency and manner in which the system requests help. If a robot completes 90% of its tasks unaided but leaves the remaining 10% unfinished in mid-task, it shifts the burden to its users rather than removing it (Forlizzi and DiSalvo, 2006).

Medical Rehabilitation and Eldercare A rehabilitation system must respect clinically established limits on force, range, and intensity while still challenging the patient enough for therapeutic effect. Applications include shared-control therapy devices that adjust assistance to measured performance, hospital logistics and delivery, continuous monitoring of activity, and assistance with activities of daily living (Kyrarini et al., 2021). Clinical deployment adds requirements that are foreign to industrial settings. Interventions must demonstrate safety and efficacy under the conventions of clinical study (Lo et al., 2010; Esquenazi et al., 2012), responsibility for decisions must be traceable to accountable parties, and eldercare in particular must weigh privacy and dignity alongside capability (Broadbent et al., 2009). In this domain, Physical AI cannot be separated from its regulatory and evidentiary context.

Commercial and Professional Services Retail, hospitality, professional cleaning, and food preparation occupy a middle ground, with spaces more structured than homes, because they are designed around service processes, yet occupied by the public and by staff whose cooperation cannot be assumed. Tasks include shelf monitoring and restocking, floor care, reception and guidance, and food handling to hygienic standards. Interaction with customers and employees is part of the task itself rather than a distraction from it, which places interface design and predictable, legible behavior on the critical path alongside manipulation competence. The economics are those of semi-structured spaces. Repetition is higher than in domestic service, while the tolerance for bespoke installation is lower than in industry.

1.5.4 Public Infrastructure: From Single-Vehicle Autonomy to City-Scale Coordination

Transportation, energy networks, and public infrastructure extend the concerns of the preceding sections across larger spatial and temporal scales. The unit of analysis shifts from one machine to populations of devices whose local actions must combine into system-level outcomes.

Intelligent Transportation Vehicles and drones combine local perception and control with routing and coordination (Yurtsever et al., 2020; Murray and Chu, 2015). For road vehicles, the difficult problems are increasingly systemic rather than vehicular, including mixed traffic in which autonomous and human drivers share conventions, fleet scheduling together with charging and cleaning for shared robotaxi operation, and the interfaces between vehicle autonomy and traffic management (Stern et al., 2018; Varaiya, 2013). For low-altitude aircraft, coordination is the founding problem. Airspace organization, geofencing, and conflict resolution for large numbers of small vehicles must exist before individual autonomy is useful at scale (Prevot et al., 2016). In both cases, the binding constraint is the coordination of many local planners toward network-level service quality.

Urban Operations and Public Facilities Buildings, power systems, and water networks distribute sensing and actuation across many devices with no mobile body at all, an arrangement treated as a distinct system archetype in Section 1.5.6. Model predictive control of building climate, demand response across a power network, and leakage detection in a water distribution network illustrate the range (Afram and Janabi-Sharifi, 2014; Siano, 2014; Chan et al., 2018). Communication capacity and consistent state estimates constrain coordination between local actions and system-wide outcomes. Value is measured by the reliability and continuity of service, energy and resource efficiency, and responsiveness to faults, observed over the scales of cities and seasons, which places a premium on sustained and validated operation rather than on peak performance.

Disaster Response and Public Safety Extreme environments impose additional restrictions. Disaster response combines difficult access, limited energy, and delayed or unavailable human intervention, making recovery mechanisms and bounded local autonomy particularly important. Search and rescue in collapsed structures, firefighting and hazardous-material intervention, and post-disaster inspection all require systems that decide and recover locally within a pre-authorized envelope and degrade safely when communication fails (Murphy, 2014; Delmerico et al., 2019; Tranzatto et al., 2022). Public-safety applications sharpen the question of authority more than any other setting in this chapter. The conditions under which a machine may act, and the parties who are answerable for its actions, must be specified before capability is deployed rather than after.

1.5.5 Scientific Inquiry: From Automated Instruments to Autonomous Laboratories

Scientific applications use physical action to acquire evidence and guide inquiry. An autonomous laboratory selects experiments, operates instruments, measures outcomes, and revises subsequent choices (King et al., 2009; Burger et al., 2020; MacLeod et al., 2020; Dai et al., 2024). Scientific value depends on measurement validity, sample traceability, and reproducible interpretation, alongside experimental throughput.

Automated and Self-Driven Laboratories The A-Lab illustrates this organization within a defined inorganic-materials synthesis workflow, integrating robotic operations, computational information, automated characterization, and active learning (Szymanski et al., 2023). Its subsequent correction underscores the need to validate material identification and novelty claims separately from successful workflow execution (Szymanski et al., 2026). Workflow autonomy (the reliable execution of planned operations) and epistemic autonomy (the production of claims that survive scrutiny) are different achievements, and evaluations in this field tend to measure the former while the value lies chiefly in the latter. Self-driven laboratory programs accordingly treat the validity chain from sample to published claim as part of the system to be engineered.

Physical Simulation and Digital Twins In scientific inquiry, simulation and digital twins, which Section 1.2.3 distinguishes from generic world models, serve as instruments for experiment design, parameter estimation, and controlled testing. Validation takes two forms. For policy training, it concerns whether simulated variation supports performance in reality. For an operational digital twin, it concerns the accuracy, timeliness, and uncertainty of the estimates used in decisions. The two uses impose different obligations. A training environment must cover the variations a policy

will meet, while an operational twin must declare how far ahead, and with what confidence, its estimates may be trusted.

Human-Machine Collaborative Scientific Cognition Autonomous laboratories change the division of cognitive labor in science rather than removing the scientist. In deployed systems, objective setting, prioritization among hypotheses, and the acceptance of evidence remain human responsibilities, while machines contribute breadth of search, uniformity of execution, and complete procedural memory. The interpretability of proposed experiments conditions trust. A suggestion whose rationale can be examined can also be disputed, and such dispute is one of the mechanisms by which science corrects itself. The productive design question, familiar from Section 1.5.1, is which parts of inquiry to delegate. Within a research program, the answer to this question is itself something to be established experimentally.

Aerospace and Space Exploration Space exploration combines scientific objectives with the most constrained communication, energy, and mass budgets in this chapter. Planetary rovers schedule a sol of activity within command latency (Jet Propulsion Laboratory, 2021), on-orbit systems service and assemble spacecraft, and in-situ instruments analyze samples that cannot be returned. Underwater operations share the pattern of difficult access, limited energy, and delayed or absent intervention (Wynn et al., 2014). Both settings inherit bounded local autonomy as their organizing principle of deciding locally within a pre-authorized envelope, recording faithfully, and deferring the remainder to later analysis. This is the same principle identified for disaster response in Section 1.5.4, applied where the return of data is slower still.

1.5.6 System Archetypes across Industries

Beneath the domain labels above, deployed systems assume a small number of recurring physical compositions, together with two modes that cut across them. Three compositions locate sensing and actuation together with computation in one body, in many bodies, or in the facility itself, while the two modes place either a human operator or a simulated world inside the loop. These compositions retrace the progression from single-agent systems through multi-agent collaboration to infrastructure-level Physical AI described in Section 1.4. Taken together, the five archetypes span the domains surveyed above, each of which can be read through the four dimensions of Section 1.5.1. Figure 1.5 shows both views together.

Single Embodied Agents The first composition concentrates sensing, actuation, computation, and energy in a single mobile body, as in the household assistant (Fu et al., 2025), the inspection quadruped (Miki et al., 2022), the planetary rover, the delivery drone (Loquercio et al., 2021). Its defining property is that resources and autonomy are co-located and bounded by one morphology, which in turn fixes the reachable task set. Most public attention attaches to this archetype, although the surveys above show it to be one composition among three.

Multi-Agent and Swarm Systems The second composition distributes work across many cooperating bodies, as in warehouse fleets, agricultural machine teams, drone formations, cooperative transport (Dorigo et al., 2021). Capability emerges from task allocation, communication, and consensus rather than from any single member, and the fleet thereby acquires properties that its individual members lack, such as tolerance of the failure of one member (Werfel et al., 2014; Tolstaya et al., 2020; Zhu et al., 2024). Assessment correspondingly shifts to fleet-level quantities such as throughput, availability, and the reliability of the coordination mechanism itself.

Environment-Scale Installations The third composition embeds sensing and actuation in the facility itself, as in automated warehouses, lights-out factories, autonomous laboratories (Gao et al., 2025), intelligent buildings, instrumented grids. No mobile body need exist. The laboratory described in Section 1.5.1, which coordinates fixed instruments without any mobile body, belongs to this composition. Engineering attention shifts from the agility of a body to the interfaces of a fa-

cility, including how materials, information, and people enter and leave, and the system boundary becomes an explicit design decision rather than the surface of a robot.

Capability-Augmenting Tools and Collaborative Systems The first of the two cross-cutting modes extends a single person's capabilities through intelligent hand tools, exoskeletons, augmented-reality task guidance, and collaborative robot cells (de Looze et al., 2016; Matheson et al., 2019). The human retains decision making, while Physical AI supplies perception, guidance, and force at the point of work. Autonomy is deliberately low, and integration with the human is the design center rather than a constraint. Because responsibility remains with the operator, this archetype provides the economic baseline against which more autonomous alternatives are judged (International Organization for Standardization, 2025).

Virtual-Real Interaction Systems The other cross-cutting mode spans physical and simulated environments through sim-to-real policy training (Tobin et al., 2017), operational digital twins, teleoperation with haptic and mixed-reality interfaces (Hokayem and Spong, 2006), and hardware-in-the-loop test benches. The virtual channel may support development, remain part of operation, or serve both roles. Action may originate from a human operator, a learned policy, or a simulator, and its physical effect depends on how the resulting command is executed. The validation questions of Section 1.5.5, concerning whether simulated variation covers reality and how far estimates may be trusted, thereby become structural properties of the system rather than external checks.

The archetypes compose rather than compete with one another. An autonomous laboratory, for example, is an environment-scale installation that contains single agents, is trained through virtual-real interaction, is maintained by humans with augmenting tools, and is coordinated as a multi-agent system when replicated across sites. The physical composition fixes much of the resource profile of an application before any algorithm is selected, and the two modes then determine where authority resides and how much of the loop runs in simulation. For this reason, the four dimensions of Section 1.5.1 are best assessed per archetype rather than per industry.

1.6 Summary and Organization of the Monograph

This chapter introduces Physical AI as a system-level capacity to perceive physical states, maintain task-relevant representations of physical relationships and action consequences, and select and execute interventions through feedback under physical, operational, and computational constraints. It contrasts this capacity with digital intelligence, makes the contrast precise in Definition 1.1, and marks its boundaries against embodied AI, cyber-physical systems, and digital twins. It then decomposes the capacity into four interdependent elements (Section 1.3), locates it on the axis of coupling between AI and the physical world (Section 1.4), and shows how deployed applications instantiate one closed loop under four assessment dimensions, across three physical compositions and two cross-cutting modes (Section 1.5). A single criterion runs through these perspectives. What a system can do is determined less by any individual model than by the integrated loop of perception, prediction, decision, action, and feedback, and by the physical and operational conditions under which the loop is sustained.

The remainder of this monograph develops that loop. Chapter 2 assembles the technical foundations shared by all four elements, including mathematical descriptions of the physical world, control theory and autonomous systems, embodied cognition, and learning and reasoning. The chapters that follow then take up the elements in order. Chapter 3 covers perception and state estimation. Chapters 4 and 5 cover physical data, simulation environments, and world models as resources that support prediction. Chapters 6 and 7 cover the foundation models and motor-control systems that realize decision-making and execution. Chapter 8 develops the definition, organization, task execution, and coordination of physical agents. Chapter 9 covers the hardware and systems that must sustain the loop reliably over long-term operation. The final chapters return to the questions left open here concerning how far the path toward general Physical AI extends (Chapter 10), how capabilities are evaluated and governed (Chapter 11), and what development strategies and longer-term outlook follow from them (Chapter 12).

The definitions and distinctions established in this chapter provide the vocabulary for all that follows. Whether a system amounts to Physical AI, where it sits between fixed automation and open-ended autonomy, and how much of the loop runs in simulation or under human authority are judged not by stated architecture but by demonstrated capability under specified conditions.

2 Technical Foundations

Building on the framework introduced in Chapter 1, this chapter first establishes foundations of Physical AI, including the mathematical modeling, control theory, cognitive AI, and usage of machine learning. These four points are organized around the closed physical interaction loop: observations inform state estimates, models and goals guide action, physical contact generates feedback, and learning updates behavior. Particularly, Section 2.1 introduces mathematical descriptions of geometry, dynamics, state, and uncertainty, which provide a shared language for later discussion. Section 2.2 explains how the core feedback control is used to build a robust and safe embodied system. On top of the control theory, Section 2.3 examines how to enable embodiment with perception, skill, and physical grounding of language and multimodal representations. Finally, Section 2.4 moves towards the modern machine learning-based methods. This closed-loop formulation provides the shared theoretical backbone for the perception, modeling, control, hardware, and safety chapters that follow.

2.1 Mathematical Descriptions of the Physical World

Physical AI requires an internal representation of the physical world that enables an agent to perceive, predict, and interact with its environment. Unlike purely symbolic systems, embodied agents must reason about continuous geometry, physical dynamics, uncertainty, and interaction constraints. Mathematical descriptions provide the foundation for this process by defining how physical entities are represented, how their states evolve over time, and how observations relate to underlying physical reality.

Classical robotics has developed rigorous mathematical formulations for geometry, motion, and dynamics, while modern embodied intelligence extends these concepts toward learned representations of complex environments. Together, geometric representations, physical dynamics, and predictive state descriptions form the foundation for physical world modeling.

2.1.1 Geometry and Spatial Representation

Euclidean Space and Rigid-Body Transformations The physical world is naturally described in three-dimensional (3D) Euclidean space. The position of an object can be represented as a point $\mathbf{x}_{\text{pos}} \in \mathbb{R}^3$, while the complete pose of an object requires both translation and rotation.

A rigid transformation is represented as

$$\mathbf{x}'_{\text{pos}} = \mathbf{R}\mathbf{x}_{\text{pos}} + \mathbf{t}, \tag{2.1}$$

where $\mathbf{R} \in \text{SO}(3)$ is a rotation matrix in the special orthogonal group in three dimensions and $\mathbf{t} \in \mathbb{R}^3$ represents translation. The rotation matrix satisfies

$$\mathbf{R}^\top \mathbf{R} = \mathbf{I}, \qquad \det(\mathbf{R}) = 1. \tag{2.2}$$

Although rotation matrices provide an intuitive geometric interpretation, they contain redundant parameters. Unit quaternions provide a compact representation of orientation and avoid singularities associated with Euler angles.

For robotics and embodied systems, rigid transformations in the special Euclidean group $\text{SE}(3)$ are commonly represented as

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix}. \tag{2.3}$$

The corresponding Lie algebra $\mathfrak{se}(3)$ provides a local representation of transformations and is widely used in state estimation, simultaneous localization and mapping (SLAM), and motion optimization (Solà, 2017; Forster et al., 2017).

Explicit Geometric Representations A physical environment can be represented using explicit geometric structures that directly describe spatial entities. Point clouds represent surfaces through discrete sample sets of the form

$$\mathcal{P}_{\text{cloud}} = \{\mathbf{p}_1, \mathbf{p}_2, ..., \mathbf{p}_{N_{\text{point}}}\}, \qquad \mathbf{p}_i \in \mathbb{R}^3. \tag{2.4}$$

Point clouds preserve metric geometry and naturally arise from sensors such as light detection and ranging (LiDAR) devices and depth cameras. They are widely used for localization, mapping, manipulation, and 3D scene understanding.

Mesh representations describe surfaces using vertices, edges, and faces. Compared with point clouds, meshes explicitly encode surface connectivity and are therefore important for simulation, collision detection, and physical interaction.

Implicit Representations of Physical Environments Instead of explicitly storing geometric elements, implicit representations describe the environment through continuous functions.

A signed distance function (SDF) represents geometry as the zero-level set of a continuous function:

$$\mathcal{S}_{\text{surf}} = \{\mathbf{x}_{\text{pos}} | f(\mathbf{x}_{\text{pos}}) = 0\}. \tag{2.5}$$

SDFs provide useful geometric properties such as surface distance and occupancy, making them suitable for collision checking, motion planning, and reconstruction.

Neural implicit representations extend this idea by parameterizing continuous scene functions with neural networks. Neural radiance fields (NeRFs) represent a scene through a continuous mapping from spatial coordinates and viewing directions to density and appearance as

$$(\sigma_{\text{rad}}, \mathbf{c}_{\text{rgb}}) = F_\theta^{\text{NeRF}}(\mathbf{x}_{\text{pos}}, \mathbf{d}), \tag{2.6}$$

where σ_{rad} represents density and $\mathbf{c}_{\text{rgb}}$ represents color. Such representations enable high-quality reconstruction and novel-view synthesis, providing a foundation for simulation-ready environments (Mildenhall et al., 2020).

Topology and Connectivity Geometric representations describe where objects exist, but embodied agents must additionally understand how spaces and objects are connected. Topological representations abstract environments into nodes and edges, where nodes represent meaningful states or locations and edges represent feasible transitions.

For navigation, topology captures reachability and connectivity of environments. For manipulation, object topology describes structural relationships among parts and possible interaction paths. Combining geometry and topology allows agents to reason not only about spatial configuration but also about possible actions.

2.1.2 Physical Dynamics and Interaction Modeling

Kinematics Kinematics describes how the configuration of a physical system changes without considering the forces causing the motion. For articulated robots, forward kinematics describes the relationship between joint space and task space as

$$\mathbf{T} = f(\mathbf{q}), \tag{2.7}$$

where $\mathbf{q}$ represents joint variables and $\mathbf{T} \in \text{SE}(3)$ represents the end-effector pose.

The inverse kinematics problem estimates joint configurations required to achieve a desired pose:

$$\mathbf{q} \in f^{-1}(\{\mathbf{T}\}), \tag{2.8}$$

where $f^{-1}(\{\mathbf{T}\})$ denotes the set of joint configurations that reach the desired pose.

Inverse kinematics is fundamental for manipulation, locomotion, and motion planning, but can be challenging due to redundancy, constraints, and multiple feasible solutions (Khalil and Dombre, 2004).

Dynamics and Equations of Motion Dynamics describes how physical states evolve under forces and control inputs. For articulated systems, robot dynamics can be expressed as

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} + \mathbf{J}(\mathbf{q})^\top \mathbf{w}_{\text{ext}}, \tag{2.9}$$

where $\mathbf{M}$ represents inertia, $\mathbf{C}$ represents velocity-dependent effects, $\mathbf{g}$ represents gravity, and $\boldsymbol{\tau}$ represents actuator torque. The external wrench $\mathbf{w}_{\text{ext}} \in \mathbb{R}^6$ stacks force and moment. The geometric Jacobian $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6\times n}$ maps the n joint rates to linear and angular velocity at the wrench reference point, expressed in the same coordinate frame.

The Newton-Euler formulation provides an efficient recursive computation of rigid-body dynamics, while the Lagrangian formulation derives motion equations from energy functions:

$$L_{\text{mech}}(\mathbf{q}, \dot{\mathbf{q}}) = E_{\text{kin}}(\mathbf{q}, \dot{\mathbf{q}}) - E_{\text{pot}}(\mathbf{q}). \tag{2.10}$$

These formulations provide the mathematical basis for simulation, control, and physical reasoning (Featherstone, 2008).

Contact Mechanics and Interaction Physical AI requires reasoning about interaction between objects. Unlike free-space motion, contact introduces constraints involving collision, friction, and force transmission.

The Coulomb friction model relates tangential and normal forces through

$$\|\mathbf{f}_{\text{t}}\| \leq \mu \|\mathbf{f}_{\text{n}}\|, \tag{2.11}$$

where μ denotes the friction coefficient.

Contact dynamics are often formulated as constrained optimization problems involving non-penetration constraints and friction cones. Accurate contact modeling is essential for grasping, manipulation, locomotion, and tool use (Posa et al., 2014; Stewart and Trinkle, 1996).

2.1.3 State Representation and Predictive Modeling

State-Space Representation A physical system is commonly described through a state variable $\mathbf{x}_t$ that summarizes the information necessary to predict future evolution. The state transition process can be represented as

$$\mathbf{x}_{t+1} = F(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t, \tag{2.12}$$

where $\mathbf{u}_t$ represents actions and $\mathbf{w}_t$ represents disturbances.

The observation process connects physical states with sensory measurements through

$$\mathbf{y}_t = h_{\text{obs}}(\mathbf{x}_t) + \boldsymbol{\epsilon}_t. \tag{2.13}$$

This formulation provides the mathematical foundation for perception, estimation, planning, and control.

Latent States and Compact Physical Representations In realistic environments, the complete physical state is rarely directly observable. Sensors provide incomplete and noisy measurements, while variables such as object properties, hidden geometry, and contact states may remain unknown.

Therefore, modern Physical AI systems often learn latent state representations of the form

$$\mathbf{z}_t = f_\theta(o_t), \tag{2.14}$$

where o_t represents observations and $\mathbf{z}_t$ represents a compact latent description of the environment.

A useful latent representation should preserve task-relevant physical information while discarding irrelevant variations. Such representations provide the foundation for predictive learning and world models discussed in Section 2.4 and Chapter 5.

Uncertainty and Predictive Representations Physical environments are inherently uncertain due to incomplete observations, stochastic dynamics, and unknown parameters. Therefore, physical state representations should incorporate uncertainty:

$$p(\mathbf{x}_t \mid \mathbf{y}_{1:t}) \tag{2.15}$$

rather than assuming exact knowledge of the state.

Probabilistic state estimation methods, including Kalman filtering and Bayesian inference, provide classical solutions for uncertainty estimation. Modern learning-based approaches extend these ideas by learning predictive representations directly from large-scale sensory experience.

These mathematical descriptions provide the bridge between explicit physical models and learned world models. Classical representations define the structure of the physical world, while learned representations enable scalable prediction and reasoning in complex environments.

2.2 Control Theory and Autonomous Systems

The mathematical descriptions in the previous section provide models of the physical world, and control theory in this section explains how an embodied agent can use measurements and actions to regulate its interaction with that world. Embodied intelligence requires continuous exchange of energy and information with the physical world. A robot must therefore not only infer an appropriate action but also realize it through actuators subject to finite torque, uncertain contact, delayed sensing, and disturbances that cannot be enumerated in advance. Control theory provides the formal language for this interaction. It asks how measurements should modify action, how models may be used without being trusted unconditionally, and under what conditions the resulting closed loop remains stable, accurate, and safe.

This section examines closed-loop autonomous operation, in which sensing, planning, and control repeatedly respond to the consequences of action. Motor-current and joint controllers react within microseconds to milliseconds. Whole-body controllers coordinate forces over tens of milliseconds, and task planners and behavior executives operate over seconds or minutes. Learning systems may update models over an entire deployment. Although these layers differ in representation and method, each compares an intended condition with an observed one and acts on the discrepancy. Feedback therefore connects classical control, optimization, planning, and learning-based autonomy rather than belonging to any one of them. The following subsections introduce feedback, control methods, and hierarchical autonomy. They also analyze stability, robustness, adaptation, and safety.

2.2.1 Foundations of Cybernetics: Feedback and Steady State

A feedback control system uses measurements of a physical system's behavior to adjust its control inputs toward a desired target, which forms a feedback loop of measurement and correction. This section first introduces this feedback loop, and then discusses its transient and steady-state behavior, to assess how effectively a system approaches and maintains that target.

Negative Feedback Principle Wiener (1948) introduces cybernetics as a framework for studying control and communication in systems whose actions affect the conditions they subsequently observe. In this framework, a plant receives a control input $u(t)$, evolves under its own dynamics and external disturbances, and produces an output $y(t)$. For output tracking, a controller compares the measured output with a reference $r(t)$ and uses the tracking error to determine the next control command. The error, and the control input in the simplest case of proportional control with gain K_{P}, are defined as

$$e_{\mathrm{track}}(t) = r(t) - y(t), \qquad u(t) = K_{\mathrm{P}} e_{\mathrm{track}}(t), \tag{2.16}$$

where the proportional law is the simplest instance of feedback. Integral and derivative action extend this relation in the proportional-integral-derivative controller introduced below.

Controllers may also use an estimate of the underlying state when the relevant variables are not directly measured. The loop is called negative feedback when the controller acts so as to reduce the measured deviation, as when a thermostat increases heating below its set point or a balancing vehicle generates torque opposing body tilt. Feedback is most valuable when the model is incomplete. If an actuator delivers less torque than commanded, or a manipulator grasps an object of unknown mass, a precomputed input cannot detect the resulting error. Feedback uses observations of the changed motion to compensate within the limits of sensing, actuation, and closed-loop bandwidth. Feedback thereby converts certain forms of uncertainty into observable error and permits repeatable behavior from imperfect components.

Negative feedback is not synonymous with stability. A correction whose sign is appropriate at low frequency may destabilize the loop once sensing, computation, or actuation introduces sufficient delay. Excessive gain has the same effect, and strong reaction to high-frequency measurement noise may excite unmodeled flexible modes. Feedback design must therefore shape the magnitude and phase of the loop response, not merely reverse the sign of the error (Åström and Murray, 2008). The signals entering the loop must also be distinguished. A reference describes intended behavior, a disturbance acts on the plant from outside, measurement noise corrupts the information available to the controller, and model uncertainty denotes the discrepancy between representation and mechanism. These signals can produce similar visible effects yet require different remedies, so an embodied controller is a negotiated balance among tracking, disturbance rejection, noise attenuation, actuator effort, and robustness. Feedback further depends on what is measured. Because the full state is rarely available, practical loops incorporate an estimator that fuses a dynamic model with sensor streams. Estimation and control become coupled processes in which actions determine which future observations become available.

Open-Loop and Closed-Loop Control Two basic control arrangements are open-loop and closed-loop control, illustrated in Figure 2.1 (Franklin et al., 2019; Dorf and Bishop, 2017; Åström and Murray, 2008). An open-loop controller selects its input without measurements of the controlled outcome during execution. Such control is appropriate when the plant and environment are sufficiently repeatable, when feedback is unavailable, or when the maneuver is too fast for a sensor-to-actuator loop, but it cannot correct errors that become observable only after execution begins. Closed-loop control uses observations to revise action, and its advantage is made precise by the sensitivity functions. For a single-input, single-output linear system in the standard negative unity-feedback configuration with plant $P(s)$ and controller $C(s)$, the loop transfer function $L(s) = P(s)C(s)$ yields

$$S(s) = \frac{1}{1 + L(s)}, \qquad T(s) = \frac{L(s)}{1 + L(s)}, \qquad S(s) + T(s) = 1, \tag{2.17}$$

where T maps the reference to the output, S maps an additive output disturbance to the output, and measurement noise reaches the output through $-T$. Where $|L|$ is large and the loop is stable, $|S|$ is generally small, whereas noise is transmitted through T. The multivariable case is analogous, with multiplication order significant. Feedback thus does not erase uncertainty but redistributes sensitivity across frequencies, subject to stability margins, delay, and unmodeled dynamics (Skogestad and Postlethwaite, 2005).

Capable robotic systems combine feedback with feedforward. Inverse dynamics, for example, predicts the torques required for a desired motion, while feedback corrects residual errors caused by inaccurate parameters, friction, contact, or estimation error. This two-degree-of-freedom structure also clarifies the role of learned models, which improve the nominal action while the conventional loop rejects deviations within a known operating envelope. The open-loop/closed-loop distinction is therefore a design spectrum rather than a dichotomy. Feedforward motion may be embedded within a slower feedback loop, and symbolic goals may be issued to feedback controllers with local termination conditions. Physical feedback also carries costs. Sensors consume power and bandwidth, estimation and communication add latency, and a loop may regulate the wrong variable with high precision, as when position control damages a fragile object because contact force is unmonitored (Raibert and Craig, 1981; Hogan, 1985).

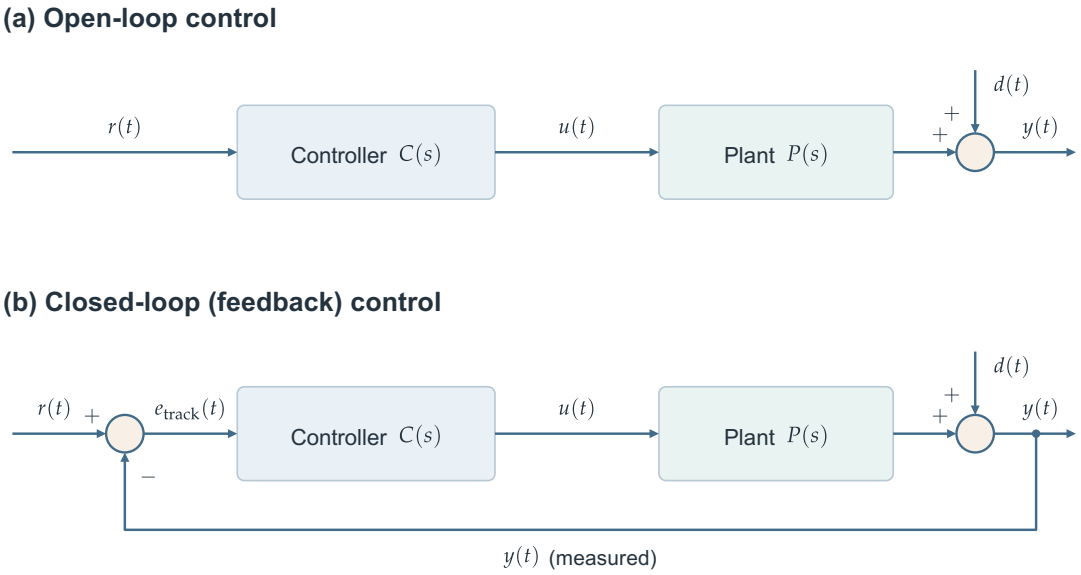


Fig. 2.1 Open-loop and closed-loop control. (a) In open-loop control, the controller maps the reference $r(t)$ directly to the plant input $u(t)$, so an output disturbance $d(t)$ passes to the output $y(t)$ uncorrected. (b) In closed-loop control, the measured output is fed back and compared with the reference, and the controller acts on the error $e_{\text{track}}(t) = r(t) - y(t)$. For a stable closed loop, the feedback attenuates the output-disturbance response over frequencies at which the loop-gain magnitude $|L(\mathrm{j}\omega)|$ is sufficiently large. $C(s)$ and $P(s)$ denote the controller and plant transfer functions, respectively

Steady-State Error and Integral Action Transient performance describes behavior while a system approaches its target. For a system that converges to a steady state, steady-state performance characterizes the residual behavior after transients have decayed (Ogata, 2010; Franklin et al., 2019). For a constant reference $r(t)$, with the tracking error $e_{\text{track}}(t) = r(t) - y(t)$ as defined above, the steady-state error is

$$e_{\text{ss}} = \lim_{t \to \infty} e_{\text{track}}(t), \tag{2.18}$$

provided that the limit exists and the closed loop is stable. A proportional controller may fail to eliminate this error when a constant input is needed to balance a persistent load. For instance, a mobile robot ascending a slope requires a constant drive force to balance gravity, and because proportional action generates its command solely in proportion to the present error, such a nonzero force can be produced only while a nonzero error persists, so the vehicle settles into a steady climb at a speed below the reference. Integral action instead accumulates the error in an integrator state $\eta_{\text{I}}(t)$ and adds an integral term $u_{\text{I}}(t)$ to the control input,

$$\eta_{\text{I}}(t) = \int_0^t e_{\text{track}}(\tau)\,\mathrm{d}\tau, \qquad u_{\text{I}}(t) = K_{\text{I}}\eta_{\text{I}}(t), \tag{2.19}$$

where K_{I} is the integral gain. A persistent deviation therefore continues to modify the command and, subject to closed-loop stability and sufficient control authority, is driven to zero. The internal model principle formalizes this mechanism. Under closed-loop stability and the solvability conditions of output regulation, exact asymptotic tracking or rejection that is robust to small plant-parameter variations requires a model capable of generating the reference or disturbance signal within the feedback path (Francis and Wonham, 1976). The integral action discussed above is the simplest instance of this principle. An integrator generates constant signals, and a loop containing an integrator can therefore track a constant reference or reject a constant disturbance exactly. Robust tracking of sinusoidal references likewise requires the controller to contain a corresponding oscillatory model at the reference frequency.

Integral action has its own failure mode under actuator saturation. An actuator, such as an electric motor or a hydraulic drive, delivers only bounded torque or force, so the applied control input $u(t)$ stops following the command whenever the requested value exceeds this physical limit. During saturation, the integrator may continue accumulating error although additional command cannot affect the plant, and the stored term produces a large overshoot when the system re-enters the controllable range. Anti-windup schemes limit or discharge this accumulation by clamping the integrator, feeding back the difference between requested and applied input, or conditioning integration on the actuator state (Kothare et al., 1994; Tarbouriech and Turner, 2009). For embodied systems, saturation must be treated as part of the model rather than as a rare implementation detail. Note that the significance of zero steady-state error is task-dependent. Contact tasks often tolerate deliberate pose error in exchange for regulated force, and legged robots accept bounded periodic error in exchange for lower impact and energy consumption.

2.2.2 *Classical and Modern Control Methods*

One of the most widely used control systems is proportional-integral-derivative (PID) control. It maps three interpretable features of the error into control action. In its ideal continuous-time form,

$$u(t) = K_{\mathrm{P}} e_{\mathrm{track}}(t) + K_{\mathrm{I}} \int_0^t e_{\mathrm{track}}(\tau)\,\mathrm{d}\tau + K_{\mathrm{D}} \frac{\mathrm{d}e_{\mathrm{track}}(t)}{\mathrm{d}t}, \tag{2.20}$$

where K_{P}, K_{I}, and K_{D} are the proportional, integral, and derivative gains, and $e_{\mathrm{track}}(t)$ is the tracking error defined above. The proportional term $K_{\mathrm{P}} e_{\mathrm{track}}(t)$ responds to the present error, the integral term $K_{\mathrm{I}} \int_0^t e_{\mathrm{track}}(\tau)\,\mathrm{d}\tau$ to its accumulated history, and the derivative term $K_{\mathrm{D}}\,\mathrm{d}e_{\mathrm{track}}/\mathrm{d}t$ to its local trend. For position or angle error with force or torque output, proportional action resembles a virtual spring, derivative action resembles a virtual damper, and integral action resembles a slowly adjusting bias that offsets gravity or friction. In a torque-controlled robot this analogy connects PID directly to impedance design. It also exposes the hazard of aggressive tuning. A stiff virtual spring improves free-space positioning while generating unsafe forces after unexpected contact, so that gains encode a physical interaction policy rather than a numerical convergence rate alone.

In practice, PID implementations modify the ideal equation in several ways. First, the derivative term is filtered against measurement noise and applied to the output rather than the reference to avoid derivative kick. Integral action also requires anti-windup. Additionally, actuator dead zones, quantization, rate limits, and computation delay can dominate performance if ignored (Åström and Hägglund, 2005). Tuning rules such as Ziegler-Nichols (Ziegler and Nichols, 1942) provide useful initial gains, but they do not replace validation on hardware. Practical implementations typically nest current, velocity, and position loops of decreasing bandwidth. When this bandwidth separation is weak, the independently tuned loops interact and may destabilize the mechanism. PID control is best suited to regulation tasks with few variables and nearly constant dynamics. Multivariable coupling, nonminimum-phase behavior, hard constraints, and large changes in the operating range call for model-based methods instead. Even then, PID often persists at the actuator interface, because its failure modes are familiar and its behavior is inspectable.

State Space and Optimal Control Another model is the state-space model, which represents the internal configuration required to predict future evolution. A linear time-invariant model has the form

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{E}\mathbf{w}(t), \qquad \mathbf{y}(t) = \mathbf{C}_{\mathrm{obs}}\mathbf{x}(t) + \mathbf{v}^{\mathrm{noise}}(t), \tag{2.21}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the state, $\mathbf{u} \in \mathbb{R}^m$ the control input, $\mathbf{w}$ a process disturbance, $\mathbf{y}$ the measurement, $\mathbf{v}^{\mathrm{noise}}$ measurement noise, and $\mathbf{A}$, $\mathbf{B}$, $\mathbf{E}$, $\mathbf{C}_{\mathrm{obs}}$ are matrices of compatible dimensions. A nonlinear system $\dot{\mathbf{x}} = f_{\mathrm{c}}(\mathbf{x}, \mathbf{u}, \mathbf{w})$ can be linearized about an equilibrium or nominal trajectory. Controllability describes whether the input can move every state direction over a finite interval, and observability describes whether the initial state can be recovered from the input-output history. The weaker properties of stabilizability and detectability suffice for stabilization and estimator convergence, requiring only that every uncontrollable or unobservable mode already be stable. As a result, no controller compensates for an actuator lacking authority over an unstable direction,

and no model complexity recovers a contact state invisible to the measurements (Åström and Murray, 2008). When the full state is not measured, an observer estimates it. For a linear model with zero-mean white process and measurement noises of known covariance, the Kalman filter is the minimum-mean-square-error estimator among linear estimators, and it is the optimal estimator without that restriction when the initial state and the noises are jointly Gaussian (Kalman, 1960a). Because estimation error enters the loop as a dynamic signal, estimator bandwidth, outlier handling, and covariance calibration are part of controller design rather than preprocessing details.

The linear-quadratic regulator (LQR) (Kalman, 1960b) renders the trade-off between regulation and control effort explicit. For the system $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$, it minimizes

$$J_{\text{cost}} = \int_0^\infty \left(\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{u}^\top \mathbf{R}_{\text{u}}\mathbf{u} \right) \, \mathrm{d}t, \tag{2.22}$$

with $\mathbf{Q} \succeq 0$ and $\mathbf{R}_{\text{u}} \succ 0$. Assuming $(\mathbf{A}, \mathbf{B})$ stabilizable[1] and $(\mathbf{A}, \mathbf{Q}^{1/2})$ detectable, the unique stabilizing positive-semidefinite solution $\mathbf{\Pi}$ of

$$\mathbf{A}^\top \mathbf{\Pi} + \mathbf{\Pi}\mathbf{A} - \mathbf{\Pi}\mathbf{B}\mathbf{R}_{\text{u}}^{-1}\mathbf{B}^\top \mathbf{\Pi} + \mathbf{Q} = 0, \tag{2.23}$$

yields $\mathbf{K} = \mathbf{R}_{\text{u}}^{-1}\mathbf{B}^\top \mathbf{\Pi}$ and the state-feedback law $\mathbf{u} = -\mathbf{K}\mathbf{x}$ (Lewis et al., 2012). The optimum is conditional on the specified model, cost, horizon, and information pattern. $\mathbf{u}^\top \mathbf{R}_{\text{u}}\mathbf{u}$ is an effort surrogate that equals physical energy consumption only under appropriate modeling. Trajectory tracking applies time-varying LQR to deviations linearized along a nominal trajectory, and applying the gain to a Kalman estimate yields the linear-quadratic-Gaussian controller under the separation principle. The principal limitation is constraint handling. LQR does not natively enforce joint limits, contact, collision avoidance, or torque saturation, and clipping an optimal command after computation alters the closed-loop dynamics and can invalidate the nominal analysis. Constrained quadratic programs impose instantaneous limits, while model predictive control extends the optimization across future steps.

Model Predictive Control Model predictive control (MPC) is also commonly used and repeatedly solves a finite-horizon optimal control problem using the latest state estimate. A representative nominal discrete-time formulation is

$$\begin{aligned}
\min_{\mathbf{u}_0,\ldots,\mathbf{u}_{H_{\text{MPC}}-1}} \quad & \sum_{k=0}^{H_{\text{MPC}}-1} \ell(\mathbf{x}_k, \mathbf{u}_k) + V_{\text{f}}(\mathbf{x}_{H_{\text{MPC}}}) \\
\text{subject to} \quad & \mathbf{x}_0 = \hat{\mathbf{x}}_{\text{now}}, \\
& \mathbf{x}_{k+1} = f_{\text{d}}(\mathbf{x}_k, \mathbf{u}_k), && k = 0, \ldots, H_{\text{MPC}} - 1, \\
& \mathbf{x}_k \in \mathcal{X}_{\text{adm}}, \quad \mathbf{u}_k \in \mathcal{U}, && k = 0, \ldots, H_{\text{MPC}} - 1, \\
& \mathbf{x}_{H_{\text{MPC}}} \in \mathcal{X}_{\text{term}} \subseteq \mathcal{X}_{\text{adm}},
\end{aligned} \tag{2.24}$$

where $\ell(\mathbf{x}_k, \mathbf{u}_k)$ and $V_{\text{f}}(\mathbf{x}_{H_{\text{MPC}}})$ are the stage and terminal costs, f_{d} is the discrete-time plant dynamics, $\hat{\mathbf{x}}_{\text{now}}$ is the most recent state estimate, $\mathcal{X}_{\text{adm}}$ and $\mathcal{U}$ are the admissible state and input sets, and $\mathcal{X}_{\text{term}}$ is a terminal constraint set. Only the first optimized input is applied. The horizon then shifts forward, a new estimate is obtained, and the problem is solved again. When the optimization remains feasible and is solved as specified, this receding-horizon procedure induces a feedback law on the current state estimate.

MPC is attractive for embodied systems because constraints are part of the decision problem. A manipulator can bound joint angles, velocities, torques, or predicted collision distances, and a legged

[1] A pair $(\mathbf{A}, \mathbf{B})$ is stabilizable when every eigenmode of $\mathbf{A}$ that cannot be influenced by the input is already asymptotically stable. Likewise, $(\mathbf{A}, \mathbf{Q}^{1/2})$ is detectable when every eigenmode of $\mathbf{A}$ that is unobservable through $\mathbf{Q}^{1/2}\mathbf{x}$ is already asymptotically stable. These are necessary and sufficient conditions for the existence of stabilizing linear state feedback and a linear observer with asymptotically stable error dynamics, respectively. Here detectability refers to the cost output $\mathbf{Q}^{1/2}\mathbf{x}$.

robot can also consider friction and feasible contact forces. For nominal regulation, recursive feasibility and asymptotic stability can be established through a positive-definite stage cost, a terminal controller that respects the constraints, a positively invariant terminal set, and a terminal cost decrease condition, provided that the initial problem is feasible (Mayne et al., 2000). Design choices are consequential. Short horizons reduce computation but limit how far ahead the controller predicts. Hard constraints declare violations inadmissible, whereas selected constraints may be softened with slack variables when requirements conflict. With linear dynamics, linear constraints, and convex quadratic costs, MPC is a convex quadratic program. Nonlinear MPC typically uses iterative numerical optimization, often with warm starts and sparse solvers.

Prediction does not eliminate the need for feedback or robustness. Model mismatch grows along the horizon, particularly during impacts, slip, or deformable contact. Tube MPC, stochastic MPC, chance constraints, or min-max formulations reserve margins against possible deviations. The system must define the response to infeasibility or solver timeout, whether retaining the previous command, switching to a stabilizing backup, or braking. Controller selection should follow the structure of the task, and the methods are frequently composed. MPC supplies whole-body references, an intermediate controller stabilizes deviations, and embedded PID loops realize actuator commands.

2.2.3 Hierarchical Architectures of Autonomous Systems

The Classical Sense-Plan-Act Pipeline Sense-Plan-Act decomposes autonomous behavior into perception, planning, and control. Perception converts sensor data into estimates of robot and environment state. Planning selects goals, actions, or trajectories. Control converts plans into actuator commands while correcting deviations (Gat, 1998). The decomposition is misleading when read as a one-shot sequence. By the end of a long perception-and-planning cycle, the environment may have changed and localization uncertainty may have grown. Autonomous operation therefore requires repeated sensing and replanning, with components running concurrently under explicit latency budgets, because an accurate estimate of an old state can be less useful than a coarser estimate of the present one.

One design requirement for this pipeline is that information passed between the perception, planning, and control stages should include uncertainty and validity, not only nominal values (Siciliano and Khatib, 2016). For example, a pose estimate without covariance encourages downstream modules to treat perception as certain, and a path without timing assumptions leaves control unable to distinguish acceptable tracking error from an obsolete plan. Interfaces should include reference frames, units, update rates, tolerances, and failure conditions, particularly when learned components are integrated with model-based control (Brugali and Scandurra, 2009). Feedback paths should also cross stage boundaries. Control influences perception through active sensing, and a geometrically valid path may be dynamically infeasible. Finally, real-time performance is governed by worst-case timing rather than average throughput. Occasionally, a planner that requires 2 seconds may be unacceptable for collision avoidance. A predesigned degradation behavior provides a backup solution. Also, scheduling priorities must follow physical urgency, because motor stabilization and contact protection cannot wait for task-level planning.

Behavior Trees and Finite-State Machines Finite-state machines (FSMs) can also formalize task execution as discrete modes and transitions triggered by events or conditions, with each state activating corresponding controllers or planners. Hybrid automata extend the representation by associating continuous dynamics with each mode (Henzinger, 1996), a natural description of systems that alternate among free motion, contact, and impact. However, a limitation of a flat FSM is combinatorial growth. As recovery cases and task variants accumulate, transitions proliferate and behavior becomes difficult to inspect. Hierarchical state machines mitigate this growth, but their interrupt, priority, and concurrency semantics must be specified explicitly. A logically valid transition may be physically unsafe if it switches references discontinuously or transfers control without resetting estimator and integrator states.

Behavior trees organize task logic through a rooted tree that is evaluated, or "ticked," repeatedly. Internal control-flow nodes implement sequences, fallbacks, and parallel combinations. Leaf nodes evaluate conditions or execute actions. Every node returns one of the three statuses *Success*, *Failure*, or *Running* to its parent. A sequence ticks its children until one returns *Running* or *Failure*. A fallback ticks its children until one returns *Running* or *Success*. Because a subtree exposes a small status interface, it can be reused across tasks, and repeated evaluation supports reactive switching (Colledanchise and Ögren, 2018; Iovino et al., 2022). Behavior trees do not themselves solve planning or control. A leaf labeled *grasp object* still requires perception, control, and recovery logic, and apparent modularity can conceal resource conflicts between branches. The choice between the representations should follow execution semantics. FSMs suit a small number of modes with well-defined transitions and admit formal verification, whereas behavior trees favor hierarchical composition and reusable recovery strategies, with an independent monitor retaining override authority for safety-related functions. Task-level switching also interacts with continuous control. Stability of each mode does not imply stability under arbitrary switching. Common and multiple Lyapunov functions, dwell-time conditions, guards, and reset maps address different classes of switched and hybrid systems (Liberzon, 2003), and the task executive and the low-level controller must share explicit entry, exit, and state-transfer rules.

Coordinating Reactive and Deliberative Systems Reactive control maps current observations directly to actions, with little or no explicit look-ahead, whereas deliberative control constructs and evaluates possible futures. Neither is sufficient alone. Pure deliberation becomes slow or brittle once its world model is stale. Pure reaction may cycle, become trapped, or sacrifice a long-term goal. Layered architectures therefore emerged to combine the two (Brooks, 1986; Gat, 1998). A useful architecture separates time scales while preserving channels for intervention. Its layers range from actuator limits, balance, and contact stability through short-horizon motion generation and skill selection to mission-level reasoning over goals and resources. Each layer has a characteristic update rate, abstraction, authority, and failure response. Higher layers parameterize lower layers, and lower layers report feasibility, progress, and exceptions rather than merely accepting commands.

Coordination can be expressed through references and invariant sets. A local controller accepts a high-level goal only if it lies within a verified operating envelope, and it modifies or rejects commands that approach limits of balance, collision distance, or actuator temperature, reporting the reason. Learned policies fit naturally into this view. A policy may propose task choices, waypoints, residual torques, or complete actions, and its placement determines the consequences of error, because a high-level proposal can be checked by planning and constrained control whereas an end-to-end torque policy admits little downstream correction. When layers disagree, summing commands can produce an action intended by neither. Priority arbitration, constrained optimization, and supervisory switching provide explicit alternatives, and hysteresis and minimum dwell times prevent chattering. The resulting system is a hierarchy of closed loops rather than a linear pipeline. Each layer monitors variables appropriate to its horizon and receives feedback from below and from the environment, and autonomy arises when these loops maintain short-term viability while advancing long-term objectives.

2.2.4 Stability and Robustness Theory

Feedback enables a controller to respond to deviations from a reference, while stability and robustness analysis establishes the closed-loop guarantees under specified conditions. This subsection reviews Lyapunov methods, barrier functions, and robust and adaptive control as formal tools for making such guarantees explicit.

Lyapunov Stability The stability of a closed-loop system depends on the specified equilibrium, trajectory, set, or orbit. For an equilibrium $\mathbf{x} = \mathbf{0}$, Lyapunov stability requires that trajectories starting sufficiently close remain close, asymptotic stability additionally requires convergence, and exponential stability specifies a convergence-rate bound (Khalil, 2002). These statements are not interchangeable, and a local certificate asserts nothing about initial conditions outside its certified

region of attraction. Lyapunov's direct method establishes stability without solving the differential equation. Let f be locally Lipschitz on a neighborhood $\mathcal{D}_{\text{Lyap}}$ of the equilibrium, with $f(\mathbf{0}) = \mathbf{0}$. If a continuously differentiable function $V : \mathcal{D}_{\text{Lyap}} \to \mathbb{R}$, called a Lyapunov function candidate and distinct from the terminal cost V_{f} used in model predictive control above, satisfies

$$V(\mathbf{0}) = 0, \qquad V(\mathbf{x}) > 0, \qquad \dot{V}(\mathbf{x}) = \nabla V(\mathbf{x})^{\top} f(\mathbf{x}) < 0, \quad \mathbf{x} \in \mathcal{D}_{\text{Lyap}} \setminus \{\mathbf{0}\}, \tag{2.25}$$

then the equilibrium is locally asymptotically stable. If these conditions hold on all of $\mathbb{R}^n$ and V is proper and radially unbounded, the equilibrium is globally asymptotically stable. If $\dot{V}$ is only negative semidefinite, stability follows, but asymptotic convergence requires additional reasoning such as LaSalle's invariance principle (Khalil, 2002). Every stability claim should be paired with its domain, assumptions, and convergence notion.

For control-affine dynamics $\dot{\mathbf{x}} = f(\mathbf{x}) + g_{\text{in}}(\mathbf{x})\mathbf{u}$, a control Lyapunov function identifies inputs that render the derivative sufficiently negative and can be imposed as a constraint in an online optimization that remains close to a nominal command. This construction bridges nonlinear control and learning. A neural network may propose a command that a model-based filter modifies when the constraint would otherwise be violated. Claims that a learned certificate renders a neural controller unconditionally stable are nevertheless too broad. The positivity and decrease conditions must hold over a stated domain, not merely on sampled training points, and verification may require analytic bounds, interval methods, mixed-integer formulations, sum-of-squares relaxations, or reachability analysis. Neural representation changes the parameterization of the candidate, not the logical requirements of the theorem.

Stability also differs from safety. An asymptotically stable robot may pass through an obstacle on its way to the goal. Safety is commonly formulated as forward invariance of a set $\mathcal{C}_{\text{safe}} = \{\mathbf{x} : h_{\text{safe}}(\mathbf{x}) \geq 0\}$. For a differentiable relative-degree-one constraint and control-affine dynamics, a zeroing control barrier function requires an admissible input $\mathbf{u} \in \mathcal{U}$ satisfying

$$L_f h_{\text{safe}}(\mathbf{x}) + L_{g_{\text{in}}} h_{\text{safe}}(\mathbf{x})\mathbf{u} + \alpha_{\text{safe}}(h_{\text{safe}}(\mathbf{x})) \geq 0, \tag{2.26}$$

where $L_f h_{\text{safe}} = \nabla h_{\text{safe}}^{\top} f$, $L_{g_{\text{in}}} h_{\text{safe}} = \nabla h_{\text{safe}}^{\top} g_{\text{in}}$, and α_{safe} is a locally Lipschitz extended class-$\mathcal{K}$ function. If a well-posed feedback law selects such an input and the regularity assumptions of the barrier theorem hold, trajectories starting in $\mathcal{C}_{\text{safe}}$ remain in $\mathcal{C}_{\text{safe}}$ (Ames et al., 2017). Feasibility can fail under input limits or conflicting barriers, so the inequality is not an unconditional safety guarantee. Lyapunov and barrier constraints can be combined in a quadratic program, with safety taking priority over convergence when relaxation is necessary. Formal claims must finally encompass computation and implementation. A continuous-time proof may not survive slow sampling, solver timeout, communication delay, or state-estimation error. A credible assurance case therefore connects the theorem to timing tests, uncertainty bounds, fault handling, and a validated operating domain, with formal analysis and empirical stress testing supporting one another.

Robust Control Robust control studies whether stability and performance persist across a specified family of plants and disturbances. The uncertainty set is the central object. It may represent bounded parameter variation, neglected high-frequency dynamics, or uncertain contact stiffness, and no controller is robust to unspecified uncertainty of arbitrary magnitude. A common objective bounds the induced gain from a disturbance $\mathbf{w}$ to a performance output $\mathbf{z}_{\text{perf}}$. In H_∞ control, the closed-loop transfer matrix $\mathbf{T}_{zw}$ from the disturbance $\mathbf{w}$ to the performance output $\mathbf{z}_{\text{perf}}$ is designed so that

$$\|\mathbf{T}_{zw}\|_\infty = \sup_{\omega \in \mathbb{R}} \bar{\sigma}(\mathbf{T}_{zw}(\mathrm{j}\omega)) < \gamma_{\text{gain}}, \tag{2.27}$$

where $\bar{\sigma}$ denotes maximum singular value. For a stable proper linear time-invariant interconnection with zero initial condition, this is the induced L_2-gain bound $\|\mathbf{z}_{\text{perf}}\|_2 < \gamma_{\text{gain}}\|\mathbf{w}\|_2$ for nonzero $\mathbf{w} \in L_2$. Weighting functions specify which signals and frequency ranges matter, and γ_{gain} is a certified performance bound for the chosen interconnection, not a probability of success in an arbitrary environment (Zhou et al., 1996).

Frequency-domain concepts clarify why high nominal performance can erode tolerance to uncertainty. Gain and phase margins describe only particular scalar perturbations in a single loop, singular values generalize sensitivity analysis to multivariable systems, and structured singular-value methods distinguish repeated, real, complex, and dynamic uncertainties. Finer uncertainty models reduce conservatism but demand stronger plant knowledge. The guarantee covers only the uncertainty that analysis has identified and bounded. Robust performance also trades against nominal agility, and excessive conservatism is itself hazardous when a vehicle cannot maneuver quickly enough to avoid a collision. Learning reduces model error but introduces new uncertainty. A residual dynamics model may improve tracking in familiar conditions while extrapolating poorly after a payload or surface change. A practical pattern lets the learned component improve performance inside a confidence region while a robust backup or safety filter limits consequences outside it, preserving the separation between empirical performance and a separately justified safety mechanism (Brunke et al., 2022).

Adaptive Control Adaptive control changes controller parameters online in response to observed behavior (Åström and Wittenmark, 1995). It is useful when the dynamics vary beyond what fixed gains can tolerate, as when a manipulator grasps an unknown payload or tire forces change with road conditions. A canonical analysis considers dynamics linearly parameterized by an unknown constant vector $\boldsymbol{\theta}$. The controller maintains an estimate $\hat{\boldsymbol{\theta}}$, updated from tracking errors and measured regressors. The control and update laws are designed together so that a Lyapunov argument establishes boundedness and, under stated assumptions, tracking convergence. This structure is more specific than unconstrained online learning, because the form of uncertainty, the update signal, and the stability objective are stated in advance (Åström and Wittenmark, 1995). Gain scheduling selects among precomputed controllers, direct adaptive control updates controller parameters, and indirect adaptive control estimates plant parameters and redesigns the controller. Guarantees proved for constant $\boldsymbol{\theta}$ do not transfer unchanged to time-varying parameters, where bounded rates of variation yield ultimate bounds rather than exact asymptotic convergence.

Accurate tracking does not imply parameter convergence. For a regressor $\boldsymbol{\phi}_{\mathrm{PE}}(t) \in \mathbb{R}^{d_{\mathrm{reg}}}$, a standard persistent-excitation condition requires constants $T_0 > 0$ and $\mu_{\mathrm{PE}} > 0$ such that

$$\int_t^{t+T_0} \boldsymbol{\phi}_{\mathrm{PE}}(\tau)\boldsymbol{\phi}_{\mathrm{PE}}(\tau)^\top \, \mathrm{d}\tau \succeq \mu_{\mathrm{PE}}\mathbf{I}_{d_{\mathrm{reg}}} \qquad \text{for all } t \geq 0. \tag{2.28}$$

Under the assumptions of a given adaptive law, such excitation supports convergence because the data distinguish all parameter directions. A robot holding a constant pose may regulate successfully while learning almost nothing about inertial coupling, whereas deliberate excitation improves identification but may conflict with task performance, comfort, or safety. Actions in adaptive control therefore influence both immediate regulation and future knowledge (Bar-Shalom and Tse, 1974).

Classical adaptive laws can drift when disturbances, unmodeled dynamics, or sensor bias violate their assumptions. Projection operators confine estimates to plausible bounds, dead zones prevent adaptation to noise-dominated errors, and leakage or σ-modification discourages unbounded parameter growth, typically replacing exact asymptotic tracking with uniform ultimate boundedness. Modern learned adaptation extends beyond low-dimensional parameter estimation to latent-context identification, dynamics-model updating, and policy adaptation, but stability does not follow from low prediction loss alone. A conservative architecture confines adaptation to a nominal model or policy inside an envelope maintained by independent constraints, monitoring, and fallback behavior. Stability, robustness, adaptation, and safety answer distinct questions about what trajectories do, whether that behavior persists over a bounded uncertainty set, how the controller changes as information is acquired, and whether forbidden states are avoided. Dependable autonomy requires an explicit argument for each property, and it requires every feedback loop to declare what it regulates, what uncertainty it tolerates, and where its guarantees end.

2.3 Embodied Cognition and the Development of Intelligence

Embodied cognition examines how intelligence depends on the interaction between an agent's body and its environment. Perception and action shape one another, and concepts gain meaning through sensory and motor experience. For Physical AI, this view calls for models and representations that are developed and tested through physical interaction.

2.3.1 Core Claims of Embodied Cognition

Embodied cognition is not a single theory but a family of claims about the unit of analysis for intelligence. Classical accounts often separate sensing, internal reasoning, and actuation. Embodied accounts emphasize that perception depends on possible movement, bodily dynamics change the control problem, and action alters the observations available next. Embodied, embedded, enactive, and extended cognition make distinct claims about bodily realization, real-time environmental dependence, autonomous sense-making, and the possible inclusion of external resources in a cognitive system (Wilson, 2002; Clark, 1997; Varela et al., 1991; Clark and Chalmers, 1998). For Physical AI, the common methodological lesson is to analyze the coupled agent-body-environment system. Merely attaching sensors and actuators is not enough. The body and interaction history must materially organize the capability being explained.

Gibson's ecological approach shifts perception from recovering detached object properties to detecting information relevant to an acting organism (Gibson, 1979). Its central concept, the *affordance*, is relational rather than a fixed label. The same ledge may afford stepping, climbing, or no safe action for bodies with different abilities (Chemero, 2003). An affordance-centered robot must therefore ask not only "What is this?" but also "What can this body do here under its current kinematic, dynamic, and safety constraints?" Sensorimotor theories complement this view by treating perception as mastery of action-dependent sensory regularities (O'Regan and Noë, 2001). Camera motion can reveal depth, wrist rotation can disambiguate shape, and tactile exploration can reveal compliance. Brooks's situated robotics work in 1991 and subsequent dynamical analyses show how robust behavior can arise from such tight real-time coupling without an exhaustive central reconstruction (Brooks, 1991; Beer, 2000). These results challenge mandatory complete world models, but they do not show that long-horizon planning or reasoning about absent events can dispense with internal state.

Intelligence also requires the ability to combine learned concepts and apply them to new situations (Fodor and Pylyshyn, 1988). However, these concepts must be connected to real-world experience. Harnad's symbol-grounding problem highlights that symbols cannot obtain meaning only from relations to other symbols (Harnad, 1990). Barsalou's perceptual-symbol theory suggests that concepts are grounded in perceptual and motor experiences (Barsalou, 1999; 2008). For example, the concept of "heavy" can be associated with the effort required to lift an object and the adjustments needed to maintain a stable grip. Thus, intelligent systems need representations that support reasoning and generalization while remaining connected to perception and action.

A robot's physical structure can simplify certain movements, while interaction with the environment provides information that helps it make better decisions (Pfeifer and Bongard, 2006; Wilson and Golonka, 2013). Rather than replacing internal models, physical interaction can complement them by providing feedback and helping robots adapt their behavior. This raises the question of how robots acquire and improve their knowledge and skills through repeated interactions with the physical world.

2.3.2 Perception-Action Coupling and Development

Action does more than follow perception. It can also improve perception. Action can help to select viewpoints, create contacts, and determine which perceptual signals may covary. Bajcsy (1988) formalizes active perception by proposing that sensing strategies be chosen jointly with task goals. Held and Hein's 1963 yoked-control experiment further distinguishes receiving visual stimulation from causing it, supporting a developmental role for action-contingent experience under its

controlled conditions (Held and Hein, 1963). In robotics, an observation paired with the agent's command, proprioception, and resulting state transition contains information absent from passive video. New motor skills also create developmental cascades. Reaching, locomotion, and tool use change what can be observed and manipulated, thereby changing the data distribution for later learning (Adolph and Hoch, 2019).

One aspect of development is learning sensorimotor coordinates and a body schema. Hand-eye coordination requires image displacement, head pose, arm configuration, joint motion, and contact to become mutually predictive. Metta et al. (1999) show that an artificial system can refine a visuomotor reaching map through experience without a fully specified prior kinematic calibration. Body-schema research generalizes this result to revisable relations among commands, configurations, contacts, and sensory consequences (Hoffmann et al., 2010). The exploration policy matters. Uniform motor babbling may oversample redundant regions, whereas goal babbling selects desired outcomes and learns the inverse mapping needed to reach them (Rolf et al., 2010). Internal models and sensorimotor coupling are therefore complementary. A model compactly predicts bodily contingencies, while intervention and feedback recalibrate it after tool use, wear, damage, or payload change.

The morphology of the physical body can also serve as a dynamical prior for learning and control. Morphological computation describes how geometry, inertia, and compliance reduce or transform what a controller must compute (Paul, 2006). McGeer's passive walkers generate gait structure from gravity and mechanics under restricted conditions, and minimally actuated descendants in later work extend the principle to level-ground walking (McGeer, 1990; Collins et al., 2005). Compliant bodies can likewise absorb impacts and provide fading-memory dynamics, although formal results apply only under specific assumptions (Hauser et al., 2011). These examples show how a task-matched body supplies a dynamical prior that learning can exploit. The gain is also task-specific, so morphology and learned control must be co-designed rather than treated as alternatives.

Lastly, self-generated experience creates a final selection problem concerning what the agent should practice next. Early curiosity systems reward expected improvement of a predictive model, while learning-progress methods direct attention toward activities that are neither mastered nor irreducibly unpredictable (Schmidhuber, 1991; Oudeyer et al., 2007). Deep curiosity methods extend action-conditioned prediction error to high-dimensional observations, but evidence from video games does not resolve physical safety, wear, or stochastic "noisy-TV" effects (Pathak et al., 2017). Taken together, active sensing determines what to observe, the body determines how action unfolds, body schemas relate commands to consequences, and intrinsic motivation allocates practice. These mechanisms explain how a learning trajectory can become structured. Developmental robotics asks how they can be integrated across an embodied lifetime.

2.3.3 Developmental Robotics

Developmental robotics studies how robots acquire and reorganize their abilities through experience over time. This subsection examines developmental constraints, lifelong and open-ended learning, imitation, and social interaction as complementary ways to guide that process.

Developmental robotics views intelligence as a gradual learning process rather than a collection of independently designed abilities. Autonomous mental development and cognitive developmental robotics study how robots are used both as constructive tests of developmental hypotheses and as systems that may benefit from developmental constraints (Weng et al., 2001; Asada et al., 2001; Lungarella et al., 2003). Development entails more than ordering examples by difficulty. The learner's body, reachable goals, representations, and social relations can all change. Morphological growth or progressively released degrees of freedom can therefore structure exploration by temporarily reducing the search space (Lungarella and Berthouze, 2002). Infant development is valuable here as a source of mechanisms such as self-calibration, staged constraints, and skill reuse, rather than as a schedule robots must literally reproduce.

Another important research direction is lifelong robot learning, which focuses on how robots continuously learn new tasks while retaining and reusing previously acquired knowledge and

skills (Thrun and Mitchell, 1995; Parisi et al., 2019). Robotics adds active data collection, temporally correlated experience, and simultaneous change in platform and environment, which static-dataset protocols often omit (Lesort et al., 2020). Also, retention is only one component of development. An agent may preserve old task performance without inventing goals, composing skills, or changing its state and action representations. Open-ended learning therefore adds representational redescription and goal discovery (Doncieux et al., 2018). Progress-based goal exploration offers one operational mechanism. Agents generate and order goals according to current learning progress, allowing acquired skills to become stepping stones for more complex behavior (Oudeyer et al., 2007; Forestier et al., 2022). A developmental architecture must consequently balance plasticity, consolidation, and expansion rather than optimize only final reward.

Additionally, individual exploration is expensive in the high-dimensional spaces of humanoid manipulators. Imitation learning is introduced to reduce this cost by directing attention toward actions another agent has made relevant. Still, it introduces a correspondence problem between different bodies, viewpoints, and intentions (Schaal, 1999). Demonstrations should therefore be treated as priors over goals and action structure, not as trajectories to copy blindly, and embodiment-specific practice remains necessary.

Lastly, social learning also requires shared reference. Constructive models show how saliency and sensorimotor regularities can scaffold joint attention (Nagai et al., 2003), while sociable-robot studies show that timing, gaze, and expression shape the feedback humans provide (Breazeal, 2003). Language games with AIBO further demonstrate that social feedback can participate in forming shared perceptual categories and words (Steels and Kaplan, 2001). The resulting relation is reciprocal. Language directs attention and decomposes goals, while action constrains linguistic meaning (Cangelosi et al., 2010). Developmental robotics thus integrates autonomous exploration, continual retention, and social scaffolding over one trajectory. In the studies reviewed here, however, these mechanisms are typically demonstrated separately and over short periods. Evaluation should measure skill diversity, retention, compositional reuse, adaptation to bodily change, safety cost, and tutor effort. This social-developmental account also provides the transition to symbols. Language becomes meaningful when shared expressions are connected to the learner's evolving perceptual and action repertoire.

2.3.4 Embodied Grounding and Multimodal Pretraining

For robots to operate effectively in the physical world, they need both broad knowledge of the world and the ability to perform physical actions. Multimodal pretraining on images, text, and videos provides useful semantic knowledge and motion priors, but this knowledge alone is not sufficient for reliable robot control (Bisk et al., 2020). Robots also need embodied experience to understand which actions are feasible and what consequences they may produce. Embodied grounding aims to connect these two sources of knowledge, linking what robots perceive and understand to what they can actually do. As illustrated in Figure 2.2, this perspective helps explain the roles of different embodied AI approaches, including vision-language-action (VLA) models, world-action models (WAMs), and multimodal agents.

Multimodal pretraining provides a scalable source of prior knowledge. Early systems explicitly connect speech to visual objects and compositional instructions to executable robot plans (Roy and Pentland, 2002; Tellex et al., 2011). At greater scale, CLIP learns broad image-language correspondences, while R3M extracts temporally informative representations from human video for downstream robot learning (Radford et al., 2021; Nair et al., 2023). Image-text data primarily contribute semantic and referential knowledge. Human video additionally conveys motion patterns and likely object interactions. However, neither directly specifies a particular robot's reachability, actuation limits, contact dynamics, or action costs. Such data therefore provide valuable *semantic and motion priors*, rather than sufficient evidence of executable competence.

Embodied experience supplies complementary evidence by connecting an agent's actions with observed outcomes. Robot demonstrations illustrate how tasks can be accomplished with a particular embodiment, while self-generated interactions reveal feasible actions and their consequences.

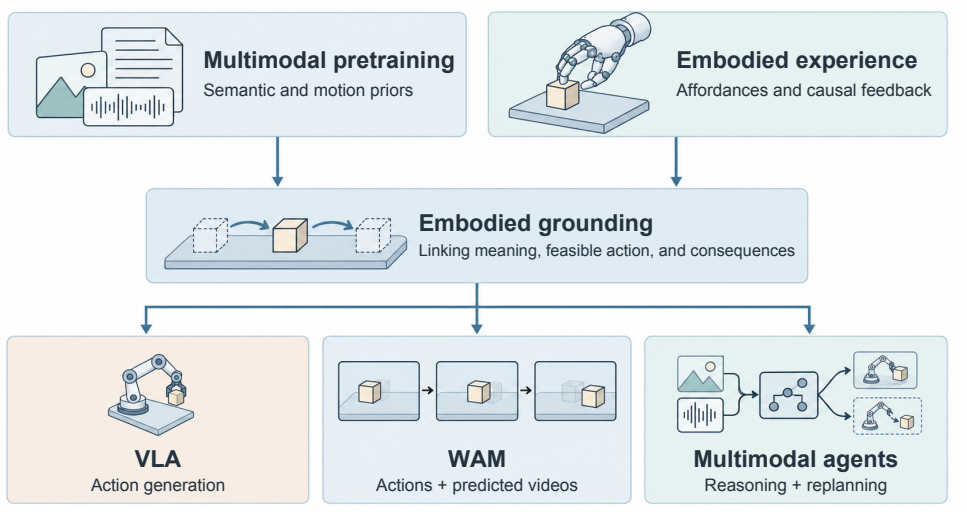


Fig. 2.2 Grounding and multimodal pretraining

Work on intuitive physical prediction, robotic poking, and action-conditioned visual foresight exemplifies the transition from observation toward intervention-based learning (Battaglia et al., 2013; Agrawal et al., 2016; Finn and Levine, 2017). Modular systems also make the division between meaning and feasibility explicit. CLIPort combines pretrained visual-language semantics with spatial manipulation, and SayCan evaluates language-model skill proposals using embodiment-specific value functions (Shridhar et al., 2022; Ichter et al., 2023). Yet successful skill selection is not equivalent to learning new affordances or revising semantic beliefs after failure. Robust grounding requires both physically informed action and feedback that can improve subsequent decisions.

Recent embodied models increasingly integrate these complementary sources. VLA models learn language-conditioned control, progressing from RT-1 and RT-2 through heterogeneous cross-robot training to systems such as $\pi_{0.5}$ (Physical Intelligence et al., 2025), which co-trains on robot actions, web data, and semantic subtasks for open-world generalization. EgoScale further explores human egocentric video as scalable motor supervision (Brohan et al., 2023; Zitkovich et al., 2023; Open X-Embodiment Collaboration, 2024; Physical Intelligence et al., 2025; Zheng et al., 2026b). WAMs, including DreamZero and Cosmos Policy, extend action learning by predicting future observations alongside actions, potentially supporting consequence-aware planning (Wang et al., 2026b; Ye et al., 2026a; Kim et al., 2026a). General-purpose multimodal agents, as explored in preliminary evaluations of GPT-6 Astra, may instead contribute task-level reasoning, failure diagnosis, and replanning, but are not established substitutes for reliable low-level control (OpenAI, 2026; Su et al., 2026; Zhang et al., 2026b). These are complementary modeling emphases rather than successive stages. The shared challenge is to turn broad multimodal priors and embodiment-specific experience into reliable, adaptive behavior.

2.4 Learning and Reasoning in the Physical World

Physical AI requires more than representing the physical world. A learning-based physical AI agent must learn the underlying regularities of physical processes, reason about possible future states, and acquire effective behaviors through interaction. Unlike learning methods that primarily exploit statistical correlations in static data, Physical AI requires models that respect geometric constraints, dynamical laws, and causal consequences of actions.

Modern approaches therefore combine three complementary principles. Physical knowledge provides inductive biases, machine learning extracts unknown structures from experience, and rein-

forcement learning enables agents to improve through interaction. Together, these components provide the foundation for learning and reasoning in physical environments.

2.4.1 Physics-Informed Learning Architectures

Physical Inductive Biases Physical environments contain strong structural regularities, including symmetry, conservation laws, locality, and relational interactions. Incorporating these properties into machine learning models reduces the hypothesis space and improves data efficiency.

A fundamental example is equivariant learning. Physical predictions should transform consistently under changes of coordinate systems. For a transformation g_{group} from a symmetry group, an equivariant model satisfies

$$f(T_{g_{\text{group}}}(x)) = T'_{g_{\text{group}}}(f(x)). \tag{2.29}$$

For 3D physical systems, SE(3)-equivariant networks preserve rotational and translational consistency and have been widely applied to molecular modeling, geometric reasoning, and robotics (Thomas et al., 2018; Satorras et al., 2021).

Graph Neural Networks for Physical Systems Many physical environments naturally consist of interacting entities. Particles, objects, robot links, and articulated components can be represented as graphs of the form

$$G = (\mathcal{V}, \mathcal{E}), \tag{2.30}$$

where nodes describe physical entities and edges represent interactions.

Graph neural networks learn system evolution through message passing of the form

$$\mathbf{h}_i^{\text{node},t+1} = \phi_{\text{node}}\left(\mathbf{h}_i^{\text{node},t}, \sum_{j\in\mathcal{N}(i)} \psi_{\text{msg}}(\mathbf{h}_i^{\text{node},t}, \mathbf{h}_j^{\text{node},t}, \mathbf{e}_{ij}^{\text{edge}})\right), \tag{2.31}$$

where $\mathbf{h}_i^{\text{node},t} \in \mathbb{R}^{d_{\text{node}}}$ is a node-feature vector and $\mathbf{e}_{ij}^{\text{edge}} \in \mathbb{R}^{d_{\text{edge}}}$ is an edge-feature vector. The message function is $\psi_{\text{msg}} : \mathbb{R}^{d_{\text{node}}} \times \mathbb{R}^{d_{\text{node}}} \times \mathbb{R}^{d_{\text{edge}}} \to \mathbb{R}^{d_{\text{msg}}}$, and the update function is $\phi_{\text{node}} : \mathbb{R}^{d_{\text{node}}} \times \mathbb{R}^{d_{\text{msg}}} \to \mathbb{R}^{d_{\text{node}}}$. The positive integer dimensions d_{node}, d_{edge}, and d_{msg} need not be equal.

This representation preserves relational structures and enables learning of complex physical dynamics, including particle simulation and object interaction modeling (Sanchez-Gonzalez et al., 2020).

Physics-Informed Neural Networks When governing equations are partially known, physical laws can be directly incorporated into the learning objective. Given a differential equation

$$\mathcal{N}_{\text{PDE}}[u^{\text{field}}](x,t) = 0, \tag{2.32}$$

a neural approximation $u_\theta^{\text{field}}(x,t)$ can be trained by minimizing both data error and physical inconsistency:

$$\mathcal{L} = \mathcal{L}_{\text{data}} + \lambda\mathcal{L}_{\text{physics}}. \tag{2.33}$$

The physical constraint is enforced through

$$\mathcal{L}_{\text{physics}} = \sum_i \|\mathcal{N}_{\text{PDE}}[u_\theta^{\text{field}}](x_i,t_i)\|^2. \tag{2.34}$$

Physics-informed neural networks (PINNs) provide a framework for combining sparse observations with known physical principles and have been widely used for solving and learning differential equations (Raissi et al., 2019).

Neural Differential Equations Instead of manually specifying analytical dynamics, neural differential equations use neural networks to parameterize continuous-time evolution as

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}_\theta^{\text{drift}}(\mathbf{x}, t). \tag{2.35}$$

Neural ordinary differential equations (Neural ODEs) provide a flexible framework for learning continuous latent dynamics (Chen et al., 2018). Stochastic extensions further model uncertainty through

$$\mathrm{d}\mathbf{x}_t = \mathbf{f}_\theta^{\text{drift}}(\mathbf{x}_t, t)\,\mathrm{d}t + \mathbf{G}_\theta^{\text{diff}}(\mathbf{x}_t, t)\,\mathrm{d}\mathbf{W}_t^{\text{Wiener}}, \tag{2.36}$$

where $\mathbf{x}_t \in \mathbb{R}^{d_{\text{state}}}$ is the state and $\mathbf{W}_t^{\text{Wiener}} \in \mathbb{R}^{d_{\text{noise}}}$ is a standard Wiener process. For the deterministic and stochastic dynamics above, $\mathbf{f}_\theta^{\text{drift}} : \mathbb{R}^{d_{\text{state}}} \times [0, \infty) \to \mathbb{R}^{d_{\text{state}}}$ is the drift mapping. The diffusion mapping is $\mathbf{G}_\theta^{\text{diff}} : \mathbb{R}^{d_{\text{state}}} \times [0, \infty) \to \mathbb{R}^{d_{\text{state}} \times d_{\text{noise}}}$. The positive integers d_{state} and d_{noise} need not be equal, and $\mathrm{d}\mathbf{W}_t^{\text{Wiener}}$ denotes the Wiener increment.

These approaches bridge classical dynamical systems and modern representation learning by allowing models to learn unknown physical processes while preserving temporal structure.

2.4.2 Reinforcement Learning for Physical Interaction

Markov Decision Processes and Physical Interaction Learning Physical AI requires not only predicting the world but also selecting actions that influence future states. Reinforcement learning formulates this process as sequential decision-making.

A Markov decision process (MDP) is defined as

$$(\mathcal{S}, \mathcal{A}, P^{\text{tr}}, R, \gamma), \tag{2.37}$$

where $\mathcal{S}$ represents states, $\mathcal{A}$ represents actions, P^{tr} represents transition dynamics, and R represents rewards.

The goal of an agent is to learn a policy

$$\pi(a|s) \tag{2.38}$$

that maximizes cumulative reward:

$$J(\pi) = \mathbb{E}_\pi[\sum_t \gamma^t r_t]. \tag{2.39}$$

However, physical agents rarely observe complete states. A robot may observe an object's appearance and motion without directly knowing its mass, friction coefficient, or contact condition. Sensor noise, occlusion, and delayed measurements further limit the information available for decision-making.

A partially observable Markov decision process (POMDP) extends the MDP with an observation space $\mathcal{O}$ and an observation model:

$$(\mathcal{S}, \mathcal{A}, P^{\text{tr}}, R, \mathcal{O}, O, \gamma), \tag{2.40}$$

where $O(o_{t+1} \mid s_{t+1}, a_t)$ specifies the probability of receiving observation o_{t+1} after action a_t leads to state s_{t+1}, and $\gamma \in [0, 1)$ is the discount factor. The physical state remains Markovian, but an individual observation need not contain enough information to predict future outcomes.

Decisions therefore depend on both past observations and past actions:

$$h_t = (o_0, a_0, o_1, \ldots, a_{t-1}, o_t), \qquad h_0 = (o_0), \qquad \pi(a_t \mid h_t). \tag{2.41}$$

Past actions matter because they help explain subsequent measurements. A change in an object's position may result from the robot's own push or from an external disturbance.

The history can be summarized by a belief state

$$b_t(s) = P(s_t = s \mid h_t), \tag{2.42}$$

which represents uncertainty about the current physical state. With a known transition and observation model, the belief is a sufficient statistic of the history for decision-making, allowing a policy to be written as $\pi(a_t \mid b_t)$. Belief updates combine predictions from the previous action with evidence from new observations.

Practical systems may approximate this inference using state estimators, recurrent networks, or policies conditioned on a recent interaction history. For example, lifting an object provides force measurements that help infer its weight and guide subsequent control. POMDPs thus make explicit how uncertain perception, memory, and action jointly shape physical decision-making (Sutton and Barto, 2018).

Reward Design and Task Specification The reward function defines the objective of an embodied agent. Designing appropriate rewards is challenging because physical tasks often involve long horizons and complex constraints.

Sparse rewards provide feedback at infrequent events, such as successful grasping or completion of a navigation task. Although simple, they create exploration difficulties.

Dense rewards introduce intermediate objectives, such as distance reduction, energy efficiency, or trajectory smoothness. However, poorly designed dense rewards may lead to unintended behaviors.

Curriculum learning gradually increases task complexity and allows agents to acquire complex physical skills from simpler subtasks.

Policy Optimization Policy-gradient methods improve a parameterized policy using the outcomes of its actions. With the history-conditioned policy introduced above, a trajectory-based expression is

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau_{\text{ep}} \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid h_t) A^{\text{adv},\pi_\theta}(h_t, a_t) \right], \tag{2.43}$$

where τ_{ep} is a sampled interaction trajectory and $A^{\text{adv},\pi_\theta}(h_t, a_t) = Q^{\pi_\theta}(h_t, a_t) - V^{\pi_\theta}(h_t)$ is the advantage, defined as the expected return of the selected action relative to the policy's average return at that history. For fully observed systems, the current state s_t can replace h_t. Actor-critic methods use a learned value estimator to approximate the advantage while updating the policy.

Proximal Policy Optimization (PPO) uses a clipped surrogate objective to limit the incentive for large policy changes:

$$\begin{aligned} L^{\text{CLIP}}(\theta) &= \mathbb{E}_t \left[\min \left(\rho_t^{\text{PPO}}(\theta) \widehat{A}_t^{\text{adv}}, \text{clip}\left(\rho_t^{\text{PPO}}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}\right) \widehat{A}_t^{\text{adv}} \right) \right], \\ \rho_t^{\text{PPO}}(\theta) &= \frac{\pi_\theta(a_t \mid h_t)}{\pi_{\theta_{\text{old}}}(a_t \mid h_t)}, \end{aligned} \tag{2.44}$$

where θ_{old} denotes the policy parameters used to collect the batch, $\widehat{A}_t^{\text{adv}}$ is an estimated advantage, $\epsilon_{\text{clip}} > 0$ controls clipping, and $\mathbb{E}_t$ denotes averaging over sampled steps. The symbol ρ_t^{PPO} distinguishes the policy probability ratio from the reward r_t. These methods are widely used for locomotion and manipulation (Schulman et al., 2017). Optimization stability and the physical closed-loop guarantees discussed in Section 2.2 remain separate questions.

Offline Reinforcement Learning Unlike simulation environments, real physical systems cannot collect unlimited interaction data. Offline reinforcement learning addresses this limitation by learning policies from previously collected datasets, including demonstrations and robot trajectories.

This paradigm is particularly important for Physical AI because large-scale datasets can provide diverse physical experiences without requiring unsafe exploration on real hardware.

2.4.3 *Predictive Learning and World Models*

This subsection develops the predictive-learning mechanisms underlying the action-consequence relation discussed in Section 2.3. The focus is on constructing internal states, learning how they evolve under actions, and using predicted outcomes for decision-making. The mathematical notion of state and the motivation for latent representations are introduced in Section 2.1.3.

Learning Latent Physical Dynamics Under partial observability, an observation feature and an internal predictive state serve different purposes. An encoder extracts features from the current measurement, while a state-update model integrates them with past actions and the previous internal state:

$$e_t^{\text{enc}} = f_\theta(o_t), \qquad \mathbf{z}_t = H_\theta(\mathbf{z}_{t-1}, a_{t-1}, e_t^{\text{enc}}), \tag{2.45}$$

where e_t^{enc} denotes an observation feature, $\mathbf{z}_t$ an internal state, and H_θ a learned update function. This recurrent form is one way to summarize the interaction history h_t used in the POMDP formulation of Section 2.4.2. The internal state should retain task-relevant information about hidden properties and contact conditions. A learned latent vector does not automatically constitute a calibrated belief distribution.

An action-conditioned dynamics model then predicts the next internal state as

$$\mathbf{z}_{t+1} \sim p_\theta(\cdot \mid \mathbf{z}_t, a_t), \tag{2.46}$$

where p_θ is a learned transition distribution. A deterministic predictor, $\hat{\mathbf{z}}_{t+1} = F_\theta(\mathbf{z}_t, a_t)$, is a simplified alternative. Training uses recorded observations, actions, and outcomes to learn useful state updates and transitions. Known physical structure can constrain these models through the architectural and loss-based approaches in Section 2.4.1. Their adequacy depends on preserving the distinctions that affect future outcomes, including uncertainty that remains after observing the interaction history.

Joint Embedding Predictive Architectures The Joint-Embedding Predictive Architecture (JEPA) approach learns representations by predicting target features from context features. I-JEPA predicts representations of masked image regions from visible context (Assran et al., 2023), while V-JEPA extends feature prediction to masked spatiotemporal regions in video (Bardes et al., 2024). A schematic prediction is

$$\hat{\mathbf{z}}_{\text{target}} = g_\phi(\mathbf{z}_{\text{context}}, m_{\text{target}}), \tag{2.47}$$

where $\mathbf{z}_{\text{context}}$ represents the visible context, m_{target} specifies target positions, and g_ϕ is the predictor. Predictions are matched to features produced by a target encoder. In these implementations, a stop-gradient target and an exponentially averaged target encoder help prevent representation collapse.

Such representations can supply observation features for a predictive world model. Learning from passive images or videos, however, does not by itself identify the effects of executable robot actions. An action-conditioned dynamics model additionally requires experience linking the agent's commands to resulting changes. This distinction separates predictive representation learning from the action-grounded models needed for physical planning and policy improvement.

World Models, Imagination, and Decision-Making Given a current internal state, a dynamics model can roll forward candidate actions. For a deterministic model and an action sequence $a_{t:t+H_{\text{roll}}-1}$ of length H_{roll}, the rollout is

$$\hat{\mathbf{z}}_t = \mathbf{z}_t, \qquad \hat{\mathbf{z}}_{t+i+1} = F_\theta(\hat{\mathbf{z}}_{t+i}, a_{t+i}), \quad i = 0, \ldots, H_{\text{roll}} - 1. \tag{2.48}$$

A stochastic model instead produces a distribution of possible rollouts. A planner evaluates predicted outcomes against the task objective and constraints, executes a selected action, and updates the internal state from new observations. The receding-horizon control principle is developed in Section 2.2.2.

Model-based reinforcement learning can also use imagined trajectories to improve a policy. Dreamer learns behavior through rollouts in a learned latent world model (Hafner et al., 2020), and

DayDreamer applies world-model-based learning to physical robots (Wu et al., 2023). Prediction thus supports both selecting actions and learning better action policies. Model errors can accumulate over long rollouts, and a policy may exploit inaccurate predictions, so useful imagination requires validation against observed physical outcomes.

These mechanisms connect state representation, physical priors, and reinforcement learning through prediction of action consequences. Chapter 5 develops world-model architectures, training objectives, uncertainty, and validation in detail. Chapter 6 and Chapter 7 examine their relationship to learned manipulation and motor capabilities.

Part II
Methodology

"The world is its own best model."
— Rodney Brooks

3 Perception and Cognition

This chapter dives deeply into perception and cognition. One of the primary differences between normal AI systems and physical AI systems is that physical AI uses perception and cognition to bridge the physical environment with the actual AI system. The key part for that bridge is the multimodal sensing, which brings information about this complex and diverse world into digital form, and produces a compressed representation.

Therefore, the perception and cognition module aims to obtain a coherent and comprehensive understanding of the physical environment from raw sensor observations. This understanding grounds world models in sensory evidence and supports prediction, planning, and action. This chapter traces that pipeline from low-level multimodal sensing and sensor fusion, 3D spatial and four-dimensional (4D) spatiotemporal perception, to high-level semantic and interaction reasoning.

3.1 Tasks and Characteristics of Physical Perception

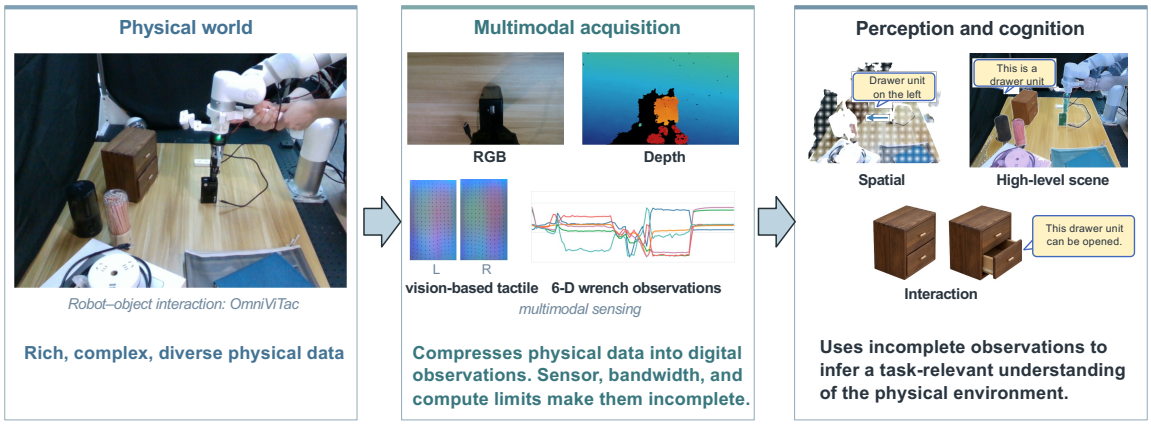


Fig. 3.1 Embodied perception from multimodal observations. A complex physical environment is sampled through multiple sensors, producing a compressed and incomplete digital representation. Perception and cognition integrate these observations to infer the environment state relevant to an embodied agent's task

Perception provides the fundamental interface between the physical world and embodied intelligence. Unlike symbolic or language-based environments, where information is already presented to a model in discrete and machine-readable forms, the physical world does not directly expose its underlying state. It is continuous, high-dimensional, dynamic, and only partially observable. As illustrated in Fig. 3.1, an embodied agent can access this world only through sensors that transform physical signals into digital observations. Before reasoning, prediction, planning, or action can take place, these observations must therefore be organized into representations that a computational model can interpret and use.

This transformation is inherently incomplete. Cameras have finite resolution and fields of view, depth sensors contain missing or uncertain measurements, force and tactile sensors observe only local contacts, and all sensing modalities operate with finite sampling rates and measurement noise. Occlusion further prevents parts of the environment from being observed at a given time. Consequently, the digital observations available to an embodied agent constitute only a partial and compressed representation of the underlying physical world. Perception must infer task-relevant structure from this incomplete evidence rather than assuming direct access to the true physical state.

This distinction makes perception particularly important for Physical AI. Downstream models reason and act on the basis of what has been observed and represented, rather than on the physical

world itself. Errors or missing information in perception can therefore propagate to state estimation, prediction, reasoning, planning, and ultimately physical action. A missed obstacle may lead to a collision. An incorrect depth estimate may produce an invalid grasp. An incorrect estimate of object or material state may cause an otherwise reasonable plan to fail. Reliable Physical AI thus depends not only on the capability of downstream models, but also on the accuracy, completeness, consistency, and uncertainty of the perceptual representations on which they operate.

Physical perception addresses this problem by acquiring multimodal observations and progressively transforming them into representations of the environment that are useful for embodied tasks. Beyond recognizing *what* is present, a physical perception system must determine *where* entities are, *how* they move and change over time, *what* physical and semantic states they are in, and *how* they can be interacted with. Perception is therefore not merely a preprocessing stage for downstream intelligence, but a continuous process of constructing and updating an internal representation of the physical world.

Physical perception can be broadly organized into three levels. *Low-level perception* acquires observations through sensory modalities. These include color cameras using red, green, and blue (RGB) channels, together with depth sensors, LiDAR, infrared and event cameras, tactile and force sensors, and proprioception. These modalities capture complementary aspects of appearance, geometry, motion, temperature, contact, and the agent's own physical state. *Mid-level perception* organizes these observations into spatially and temporally consistent representations, including 3D geometry, motion, and dynamic 3D/4D scene structure. *High-level perception and cognition* further associates these representations with semantic entities, physical properties and states, functional relationships, affordances, and interaction possibilities, enabling task-dependent interpretation and reasoning.

Across these levels, physical perception is characterized by *multimodality, spatiotemporal consistency, physical grounding, uncertainty, and interaction-oriented understanding*. Multimodal observations compensate for the limitations of individual sensors, while spatiotemporal modeling connects partial observations across viewpoints and time. Physical grounding associates semantic interpretation with measurable geometry, state, and physical evidence, and explicit treatment of uncertainty reflects the unavoidable incompleteness of sensing. Interaction-oriented understanding further focuses perception on the information that matters for action, including contact regions, object states, material conditions, and possible state transitions.

These requirements drive physical perception beyond isolated single-modality recognition toward integrated systems that continuously combine multimodal sensing, 3D/4D spatial modeling, semantic understanding, physical measurement, and interaction feedback. From this perspective, perception extends beyond recognition to make the physical world observable, representable, and actionable to an embodied intelligent system.

3.2 Multimodal Perception and Fusion

3.2.1 Multimodal Sensing and Data Acquisition

Multimodal sensing and data acquisition refer to the processes by which an embodied system measures the physical state of itself and its environment and converts heterogeneous measurements into digital observations. Perceiving and interacting with a physical environment require information about several complementary properties, including appearance, geometry, motion, temperature, and contact state. No single sensing modality can reliably and completely observe all of these properties. Embodied systems therefore commonly combine RGB cameras, depth cameras, LiDAR, force and tactile sensors, and proprioceptive sensors. Exteroceptive sensors observe the external environment, proprioceptive sensors measure the internal configuration and motion of the robot, and contact sensors characterize physical interaction at the robot-environment interface. Together, these measurements form the observational basis for subsequent perception, state estimation, reasoning, planning, and control. Section 9.2 provides additional details.

RGB Cameras RGB cameras are the most widely used exteroceptive sensors in embodied intelligence. They measure visible optical radiation and provide dense observations of scene color, texture, boundaries, and appearance. These observations contain rich semantic cues but do not directly encode metric depth or contact state.

To describe how an RGB camera converts incident light into image measurements, this chapter first considers its radiometric image-formation model (Healey and Kondepudy, 1994; Bhuyan, 2019; Holst and Lomheim, 2007). During an exposure, each photosite integrates the incident light according to its wavelength-dependent response. For a photosite associated with color channel $i_{\mathrm{rgb}} \in \{\mathrm{R}, \mathrm{G}, \mathrm{B}\}$ and spatial location $\mathbf{u}_{\mathrm{pix}} = (u_{\mathrm{pix}}, v_{\mathrm{pix}})$, the resulting raw measurement can be expressed as

$$y^{\mathrm{raw}}_{i_{\mathrm{rgb}}}(\mathbf{u}_{\mathrm{pix}}, t) = Q^{\mathrm{quant}}_{i_{\mathrm{rgb}}}\left(\int_{t}^{t+\Delta t} \int_{\Lambda_{\mathrm{vis}}} S_{i_{\mathrm{rgb}}}(\lambda) E_{\lambda}(\mathbf{u}_{\mathrm{pix}}, \tau)\, \mathrm{d}\lambda\, \mathrm{d}\tau + \varepsilon_{i_{\mathrm{rgb}}}(\mathbf{u}_{\mathrm{pix}}, t) \right), \tag{3.1}$$

where $y^{\mathrm{raw}}_{i_{\mathrm{rgb}}}(\mathbf{u}_{\mathrm{pix}}, t)$ is the raw measurement of an individual photosite, $E_{\lambda}(\mathbf{u}_{\mathrm{pix}}, \tau)$ denotes the spectral irradiance incident on the sensor, Δt is the exposure duration, and $S_{i_{\mathrm{rgb}}}(\lambda)$ is the effective spectral responsivity of channel i_{rgb}, incorporating wavelength-dependent optical transmission, color-filter response, quantum efficiency, and photoelectric conversion. The term $\varepsilon_{i_{\mathrm{rgb}}}(\mathbf{u}_{\mathrm{pix}}, t)$ represents sensor noise, while $Q^{\mathrm{quant}}_{i_{\mathrm{rgb}}}(\cdot)$ summarizes electronic amplification, analog-to-digital conversion, and quantization. A camera image is therefore an exposure-time integral rather than an instantaneous sample.

The collection of photosite measurements forms a raw mosaic image $Y^{\mathrm{raw}}_{\mathrm{C}}$, which is subsequently transformed by an image signal processor (ISP) into an RGB image

$$I^{\mathrm{RGB}} = F_{\mathrm{ISP}}(Y^{\mathrm{raw}}_{\mathrm{C}}), \tag{3.2}$$

where I^{RGB} denotes the output RGB image and $F_{\mathrm{ISP}}(\cdot)$ denotes the camera-specific processing pipeline, which may include demosaicing, white balancing, denoising, color correction, tone mapping, gamma encoding, and compression. Consequently, the final RGB values are not generally proportional to scene radiance, and their photometric interpretation depends on both the camera response and the processing pipeline (Debevec and Malik, 2023; Brooks et al., 2019).

Complementary to the radiometric model, which describes how pixel values are formed, a geometric image-formation model specifies how 3D scene points are projected onto the two-dimensional (2D) image plane (Hartley and Zisserman, 2003; Szeliski, 2011; Torralba et al., 2024). Under the pinhole camera model, the projection of a point $\mathbf{p}_{\mathrm{C}} = (X_{\mathrm{C}}, Y_{\mathrm{C}}, Z_{\mathrm{C}})^{\top}$ expressed in the camera coordinate frame is represented in homogeneous coordinates as

$$\tilde{\mathbf{u}}_{\mathrm{pix}} \propto \mathbf{K}_{\mathrm{cam}} \mathbf{p}_{\mathrm{C}}, \qquad \tilde{\mathbf{u}}_{\mathrm{pix}} = (u_{\mathrm{pix}}, v_{\mathrm{pix}}, 1)^{\top}, \tag{3.3}$$

where $\propto$ denotes equality up to a nonzero scale and $\mathbf{K}_{\mathrm{cam}}$ is the camera intrinsic matrix, conventionally written as

$$\mathbf{K}_{\mathrm{cam}} = \begin{bmatrix} f_{\mathrm{x}} & \gamma_{\mathrm{cam}} & c_{\mathrm{x}} \\ 0 & f_{\mathrm{y}} & c_{\mathrm{y}} \\ 0 & 0 & 1 \end{bmatrix}, \tag{3.4}$$

where f_{x} and f_{y} are the effective focal lengths measured in pixels, $(c_{\mathrm{x}}, c_{\mathrm{y}})$ is the principal point, and γ_{cam} is the skew parameter between the image axes, which is commonly assumed to be zero in modern digital cameras. The intrinsic matrix maps normalized camera coordinates to pixel coordinates. Lens distortion is modeled separately. Its parameters are estimated during camera calibration and subsequently used to undistort or rectify the captured images (Zhang, 2000).

Within embodied systems, RGB cameras are commonly mounted on the head or torso to provide egocentric observations, on the wrist or end effector to capture close-range interactions, or in the environment to provide a global view of the robot and workspace. These visual streams constitute a

primary source of exteroceptive information for semantic scene understanding, object localization, visual navigation, and visuomotor control. Visual navigation models such as ViNT use camera observations to learn navigational affordances under a general goal-reaching objective (Shah et al., 2023), while large-scale manipulation datasets and policies increasingly combine synchronized multi-view imagery with language and proprioceptive state. DROID, for example, records three synchronized stereo RGB camera streams, together with robot states and actions, exterior-camera calibration, and natural-language instructions across 86 manipulation tasks (Khazatsky et al., 2024). Generalist VLA models such as π_0 and $\pi_{0.5}$ similarly combine camera observations with language instructions and robot state to generate actions across diverse tasks and robot embodiments (Black et al., 2025a; Physical Intelligence et al., 2025). Despite their low cost and semantic richness, RGB cameras do not directly measure metric geometry or physical contact, and their reliability remains sensitive to illumination changes, saturation, specular reflections, motion blur, and occlusion.

Depth Cameras and LiDAR Depth cameras and LiDAR provide metric observations of scene geometry, but their measurement principles and sampling patterns are not determined by the device category alone. Some depth cameras employ stereo triangulation, whereas others use time-of-flight principles closely related to LiDAR. A useful distinction is therefore made between *axial depth* Z_{depth}, defined as the coordinate of a scene point along the camera optical axis, and *ray range* r, defined as the Euclidean distance from the sensing origin along a viewing or emission ray. The systems considered here are organized by how distance is inferred, distinguishing *triangulation-based depth sensing* from *propagation-based optical ranging*.

Triangulation-based systems infer depth from correspondences between calibrated rays originating from spatially separated camera or projector centers. For a rectified stereo pair with baseline B_{stereo}, common horizontal focal length f_{x}, and aligned principal points, the horizontal disparity $d_{\text{stereo}}(\mathbf{u}_{\text{pix}}) = u_{\text{pix,L}} - u_{\text{pix,R}}$ determines the axial depth. The resulting depth can then be back-projected into the reference-camera coordinate frame using

$$\begin{aligned} Z_{\text{depth}}(\mathbf{u}_{\text{pix}}) &= \frac{f_{\text{x}} B_{\text{stereo}}}{d_{\text{stereo}}(\mathbf{u}_{\text{pix}})}, \\ \mathbf{p}_{\text{C}}(\mathbf{u}_{\text{pix}}) &= Z_{\text{depth}}(\mathbf{u}_{\text{pix}}) \mathbf{K}_{\text{cam}}^{-1} \tilde{\mathbf{u}}_{\text{pix}}, \end{aligned} \tag{3.5}$$

where f_{x} and $d_{\text{stereo}}(\mathbf{u}_{\text{pix}})$ are expressed in pixels, $\mathbf{K}_{\text{cam}}$ is the intrinsic matrix of the reference camera, and $\tilde{\mathbf{u}}_{\text{pix}} = (u_{\text{pix}}, v_{\text{pix}}, 1)^\top$ is the homogeneous image coordinate (Hartley and Zisserman, 2003). The second relation assumes that the depth image stores axial depth rather than distance along the viewing ray. Passive stereo relies on naturally occurring scene texture to establish correspondence. Active-stereo systems retain the same binocular geometry but project an unstructured infrared pattern to add texture in weakly textured regions. The pattern assists left-right image matching and need not be decoded into projector coordinates. The Intel RealSense D400 series is a representative example (Keselman et al., 2017). Structured-light systems also recover depth by triangulation, but use a calibrated camera-projector pair and establish correspondences between observed image locations and encoded projector coordinates (Zanuttigh et al., 2016). Thus, although active stereo and structured light employ active illumination, both remain triangulation-based because depth is inferred from spatial correspondence rather than optical propagation time.

Propagation-based sensors emit optical signals and infer distance from temporal or phase properties of the returned signal rather than from spatial disparity. For a pulsed time-of-flight LiDAR, the measured round-trip propagation delay Δt_i determines the range of the ith return. Combining this range with the calibrated beam direction gives its three-dimensional position as

$$\begin{aligned} r_i &= \frac{c_{\text{light}}\, \Delta t_i}{2}, \\ \mathbf{p}_i &= \mathbf{o} + r_i \mathbf{d}_i, \end{aligned} \tag{3.6}$$

where c_{light} is the speed of light, $\mathbf{o}$ is the sensing origin, and $\mathbf{d}_i$ is a unit vector specifying the calibrated beam direction (Royo and Ballesta-Garcia, 2019). A return may additionally include a

measure of return strength or intensity a_i^{lidar} and an acquisition timestamp t_i. Per-return timing is important for compensating motion distortion, or *deskewing*, when a scanning LiDAR assembles a point cloud over a nonzero time interval. Time-of-flight depth cameras apply a related propagation-based principle at many pixels over a two-dimensional detector array. Depending on the sensor design, they estimate distance from the direct time of flight of optical pulses or from the phase shift of modulated illumination (Horaud et al., 2016). The stored value may represent ray range or may be converted into axial depth according to the sensor's calibration and output convention.

The native measurements of these sensors induce different spatial structures. A camera depth image can be back-projected into an organized point cloud whose samples preserve the image lattice, whereas LiDAR measurements generally form an irregular, sparse point cloud indexed by beam direction and acquisition time. After calibration and coordinate transformation, either representation may be converted into voxels, signed-distance fields, or continuous neural implicit fields for downstream perception and planning.

In embodied systems, depth observations are particularly useful for close-range geometry, including object-pose estimation, grasping, and workspace reconstruction. PerAct, RVT, and Act3D, for example, demonstrate how calibrated RGB and depth (RGB-D) observations can be transformed into volumetric, multi-view, or continuous three-dimensional representations for robotic manipulation (Shridhar et al., 2023; Goyal et al., 2023; Gervet et al., 2023). LiDAR is more commonly employed on mobile and field robots, where its typically longer sensing range and direct metric measurements support localization, mapping, obstacle avoidance, and exploration. CERBERUS provides a representative example in subterranean environments (Tranzatto et al., 2022). These modalities nevertheless exhibit different failure modes. Passive stereo correspondence becomes unreliable in textureless, repetitive, or occluded regions. Active stereo and structured light reduce texture ambiguity but remain limited by projector range, strong ambient illumination, interference, and transparent or specular surfaces. Propagation-based ranging can likewise be degraded by low-return surfaces, strong ambient illumination, multipath interference, and adverse atmospheric conditions.

Force and Tactile Sensors Force and tactile sensors provide measurements of physical interaction that cannot be obtained directly from conventional vision. Force and torque sensors measure the resultant loads transmitted through a robot component, whereas tactile sensors measure local contact conditions over a sensing surface. This distinction concerns the spatial support and physical interpretation of the measurements rather than the transduction material alone. Resistive, capacitive, piezoelectric, optical, and magnetic mechanisms may be used in different sensor configurations.

A multi-axis force/torque sensor measures the resultant force and moment acting on a robot component, commonly at the wrist, ankle, or end effector. A six-axis sensor reports the wrench, expressed in a declared sensor coordinate frame, as

$$\begin{aligned} \mathbf{w}_S &= [\, \mathbf{f}_S^\top, \boldsymbol{\tau}_S^\top \,]^\top, \\ \mathbf{w}_S &= \mathbf{C}_{\text{cal}}(\mathbf{v}_{\text{sens}} - \mathbf{v}_{\text{sens},0}), \end{aligned} \tag{3.7}$$

where $\mathbf{f}_S \in \mathbb{R}^3$ and $\boldsymbol{\tau}_S \in \mathbb{R}^3$ are the measured force and torque, $\mathbf{v}_{\text{sens}}$ denotes the raw electrical outputs, $\mathbf{v}_{\text{sens},0}$ is the zero-load offset, and $\mathbf{C}_{\text{cal}}$ is a calibration and decoupling matrix. The second relation is a common linear approximation for strain-based sensors, which infer applied loads from the deformation of an elastic structure (Cao et al., 2021). Such sensors provide accurate aggregate interaction loads but do not resolve how those loads are distributed across multiple contact points. Joint-torque sensors and motor currents can also support external-force estimation through a robot dynamics model, but these indirect estimates should not be treated as equivalent to a directly measured endpoint wrench.

Tactile sensors measure contact locally at the robot surface. Their outputs may describe contact location, surface deformation, normal and tangential loading, or slip. According to their dominant sensing and readout mechanisms, tactile sensors commonly used in embodied systems can be broadly organized into vision-based tactile sensors and taxel-based tactile arrays.

Vision-based tactile sensors illuminate a compliant elastomer and use an internal camera to observe its deformation during contact. Their native digital observation is an image rather than a direct force measurement:

$$I_{\text{tac}}(\mathbf{u}_{\text{pix}}, t) = \mathcal{F}_{\text{opt}}\big(\delta(\mathbf{x}_{\text{pos}}, t); \boldsymbol{\theta}_{\text{tac}}\big) + \varepsilon_I(\mathbf{u}_{\text{pix}}, t), \qquad \widehat{\mathbf{z}^{\text{tac}}}_t = G_{\text{tac}}(I_{\text{tac}}), \tag{3.8}$$

where δ denotes the elastomer deformation, $\mathcal{F}_{\text{opt}}$ summarizes the illumination and imaging process with sensor parameters $\boldsymbol{\theta}_{\text{tac}}$, and $G_{\text{tac}}(\cdot)$ is a calibrated or learned estimator of contact quantities $\widehat{\mathbf{z}^{\text{tac}}}_t$, such as contact geometry, force, shear, or slip. The GelSight study establishes high-resolution optical measurement of contact geometry and force-related deformation (Yuan et al., 2017), while the DIGIT study demonstrates a compact, lower-cost design suitable for robotic hands (Lambeta et al., 2020). Because their output is image-like, these sensors also support visual representation learning. Sparsh, for example, learns transferable representations from unlabeled tactile images (Higuera et al., 2025).

Taxel-based sensors use arrays of discrete tactile elements distributed over a compliant surface. A pressure-sensitive taxel primarily measures normal contact stress, whereas more general taxels may also respond to tangential stress or local deformation. For taxel i with sensitive region $\mathcal{A}_i$ and area $A_i = \int_{\mathcal{A}_i} \mathrm{d}A$, a generic calibrated measurement model is

$$\begin{aligned} \overline{\boldsymbol{\sigma}}_i(t) &= \frac{1}{A_i} \int_{\mathcal{A}_i} \boldsymbol{\sigma}_{\text{c}}(\mathbf{x}_{\text{pos}}, t)\, \mathrm{d}A, \\ \mathbf{y}_i(t) &= g_i^{\text{taxel}}\big(\overline{\boldsymbol{\sigma}}_i(t), T_{\text{temp}}(t)\big) + \boldsymbol{\varepsilon}_i(t), \end{aligned} \tag{3.9}$$

where $\boldsymbol{\sigma}_{\text{c}} = [p_{\text{press}}, s_1^{\text{shear}}, s_2^{\text{shear}}]^\top$ contains normal pressure p_{press} and two tangential stress components, and $g_i^{\text{taxel}}(\cdot)$ represents the sensor-specific transduction and calibration. Resistive, piezoresistive, and capacitive arrays are widely used for compact quasi-static contact sensing. Piezoelectric elements are particularly responsive to changes in load and vibration. Magnetic or inductive designs can provide compliant multi-axis measurements. These mechanisms differ in sensitivity, bandwidth, hysteresis, drift, wiring complexity, and spatial resolution (Dahiya et al., 2010; Meribout et al., 2024).

Some specialized tactile devices additionally measure vibration, acoustic emission, or temperature. High-bandwidth vibration and acoustic signals can reveal impact, sliding, texture, and incipient slip, while thermal measurements can contribute to material recognition. These signals are less commonly used as standalone tactile modalities in embodied systems and are often incorporated as complementary channels in multimodal tactile sensors.

The appropriate sensor depends on the spatial scale and information required by the task. Six-axis force/torque sensors are mature choices for aggregate load feedback in compliant control, insertion, assembly, and human-robot interaction. Vision-based tactile sensors are particularly attractive in current robot-learning research because they provide dense contact geometry and can be processed with image-based models, whereas electrical or magnetic taxel arrays can be integrated into compact fingertips, finger links, palms, and larger-area robotic skin. Force and tactile measurements support contact detection, grasp stabilization, slip detection, texture recognition, object reorientation, and dexterous manipulation. RH20T illustrates their role in multimodal robot learning by recording force together with visual, audio, robot-state, action, and language information across diverse manipulation skills (Fang et al., 2024b).

These sensing families also have distinct limitations. Force/torque sensors do not localize individual contacts without additional geometric assumptions. Taxel arrays may exhibit hysteresis, drift, cross-talk, and limited spatial resolution. Vision-based tactile sensors are sensitive to illumination and camera-calibration errors, as well as elastomer wear and viscoelastic effects such as hysteresis and stress relaxation. Their physical estimates additionally depend on calibrated or learned mappings from deformation to contact quantities. All tactile modalities have a local field of view and become informative primarily after contact. They are therefore commonly combined with vision and proprioception. Vision estimates where interaction should occur, proprioception supplies the robot configuration, and force or tactile feedback verifies the resulting physical interaction.

Proprioceptive Sensors Proprioceptive sensors measure the internal state and motion of the embodied platform. For an articulated robot, a representative proprioceptive observation is

$$\mathbf{z}_{\text{prop}}(t) = [\,\mathbf{q}^\top, \dot{\mathbf{q}}^\top, \boldsymbol{\tau}^\top, \mathbf{i}^\top, \mathbf{a}_{\text{imu}}^\top, \boldsymbol{\omega}_{\text{imu}}^\top\,]^\top, \tag{3.10}$$

where $\mathbf{q}$ and $\dot{\mathbf{q}}$ are joint positions and velocities, $\boldsymbol{\tau}$ denotes measured or estimated joint torque, $\mathbf{i}$ is motor current, and $\mathbf{a}_{\text{imu}}$ and $\boldsymbol{\omega}_{\text{imu}}$ are specific force and angular velocity from an inertial measurement unit. Depending on the platform, proprioception may additionally include wheel odometry, base orientation, gripper aperture, actuator temperature, or foot-contact switches.

These signals support forward kinematics, state estimation, trajectory tracking, balance, and body-aware control. They also provide the configuration needed to transform head-, wrist-, and body-mounted sensor data into a common coordinate system. Joint torque and motor current can support external-force estimation through an explicit robot dynamics model, but the resulting estimate remains sensitive to transmission, friction, and model error. In learning-based embodied systems, proprioceptive state is commonly combined with visual and language inputs so that action generation is conditioned on both the external scene and the current robot configuration (Black et al., 2025a; Khazatsky et al., 2024).

Multisensor Acquisition, Calibration, and Time Synchronization Combining sensors requires more than recording their outputs. Different devices use different coordinate frames, sampling rates, exposure intervals, clock domains, communication paths, and latency conventions. Reliable fusion therefore depends on calibrated spatial and temporal relationships between measurements.

Spatial calibration comprises intrinsic calibration of each imaging or ranging sensor and extrinsic calibration between sensor frames. If a point measured in sensor frame S_m is to be expressed in a reference frame S_0, the homogeneous-coordinate transformation is

$$\widetilde{\mathbf{p}}_{\mathrm{S}_0} = {}^{\mathrm{S}_0}\mathbf{T}_{\mathrm{S}_m}\widetilde{\mathbf{p}}_{\mathrm{S}_m}, \qquad {}^{\mathrm{S}_0}\mathbf{T}_{\mathrm{S}_m} \in \mathrm{SE}(3). \tag{3.11}$$

For moving sensors, this transform is composed with the robot kinematic state at the measurement time. Calibration may also include image rectification, depth-to-color registration, tactile-frame localization, force-sensor bias estimation, and calibration of the transformation from sensor output to physical units.

Temporal alignment requires a model of each sensor clock. A common affine approximation relates sensor time t_m to a reference time t_0 as

$$t_m = \alpha_m^{\text{clk}} t_0 + \beta_m^{\text{clk}}, \tag{3.12}$$

where β_m^{clk} is the clock offset and α_m^{clk} represents the relative clock rate. A timestamp must also be associated with a clearly defined event. For a camera, that event may be the beginning, midpoint, or end of exposure. For a ranging pulse, it may be emission or reception. A timestamp may also mark the arrival of a sample at the host computer. Joint estimation of spatial and temporal calibration is often necessary because an unmodeled time offset during robot motion can be mistaken for an extrinsic-calibration error (Furgale et al., 2013).

Software synchronization aligns independently acquired streams through timestamps, buffering, interpolation, and nearest-neighbor association. It is flexible but cannot recover information lost through inaccurate timestamps or variable transport delay. Hardware synchronization distributes a common clock, trigger, or pulse-per-second signal and is preferable when the platform undergoes rapid motion or contact transitions. Even with hardware triggering, an acquisition system should account for exposure duration, rolling-shutter readout, sensor processing delay, network latency, and the time required to assemble scanning measurements.

The final synchronized record should preserve the raw measurements, calibrated measurements, timestamps, coordinate-frame definitions, calibration parameters, and validity indicators. An RGB image, depth frame, LiDAR return, tactile image, wrench sample, and joint state should either refer to the same physical instant or carry sufficient timing information to be transformed or interpolated to a common instant. This requirement is particularly important for visual-tactile learning,

force-guided manipulation, and high-speed navigation, where small temporal errors can associate observations with different robot poses or contact regimes. Careful acquisition and calibration are thus integral to perception and determine the reliability of all subsequent multimodal fusion.

3.2.2 Multimodal Fusion for Joint Perception

Multimodal fusion combines complementary observations to estimate the state of the robot, its surroundings, and their interactions. RGB images describe appearance. Depth and LiDAR provide geometric measurements. Force, tactile, and proprioceptive signals supply information about contact and robot motion. A useful organizing principle is the stage at which information is combined, whether at the level of observations, intermediate features, or task-level estimates. These levels describe possible operations rather than mutually exclusive architectures (Hall and Llinas, 1997; Baltrušaitis et al., 2019).

Observation-Level Integration and Cross-Modal Enhancement When measurements have been spatially registered and temporally aligned, they can be associated in a common coordinate frame. For example, a registered RGB image and depth map associate appearance with measured distance at corresponding image locations. RGB-guided depth completion additionally uses image information to infer depth where measurements are sparse or missing (Ma and Karaman, 2018). Such estimates must remain distinguishable from measured depth. An appearance edge need not coincide with a geometric boundary. Validity masks, calibration, and timestamps are therefore part of the fused observation, especially near occlusions or when the sensors move.

Feature-Level Fusion for Joint Perception Modality-specific encoders can extract representations that are then combined through concatenation, learned weighting, or attention. The features may be aligned on an image grid or mapped into a shared three-dimensional or bird's-eye-view space. FuseNet combines color and depth features for semantic segmentation (Hazirbas et al., 2017). BEVFusion combines camera and LiDAR features in a bird's-eye-view representation for three-dimensional perception (Liu et al., 2023c). In embodied systems, visual, tactile, and proprioceptive features can likewise be related through robot kinematics and contact geometry, even when pixel-wise correspondence is unavailable. Feature fusion allows task-dependent interactions but can be sensitive to missing modalities and errors in spatial or temporal alignment.

Estimate-Level Fusion Alternatively, each sensor stream can produce an estimate, such as object pose, occupancy, or contact state, before the estimates are combined. This approach permits modality-specific processing and can accommodate different sampling rates, provided that the estimates refer to compatible states and their uncertainties and dependencies are accounted for (Hall and Llinas, 1997). Because high-level estimates discard some measurement detail, they may offer less opportunity to exploit fine cross-modal correspondences. In practice, the fusion level is chosen according to the perceptual task, available calibration, measurement reliability, and computational constraints. A system may combine several levels.

3.3 3D/4D Spatial Perception

Three-dimensional spatial perception estimates scene geometry, object locations, and spatial relationships. 4D perception additionally represents changes over time, including object motion and deformation. Figure 3.2 illustrates alternative 3D representations of the same object. For an embodied agent, these estimates should support interaction with the physical environment, not only recognition or novel-view rendering. Because sensors observe only limited, noisy projections of the world, spatial perception maintains a task-relevant estimate rather than a complete copy of physical reality.

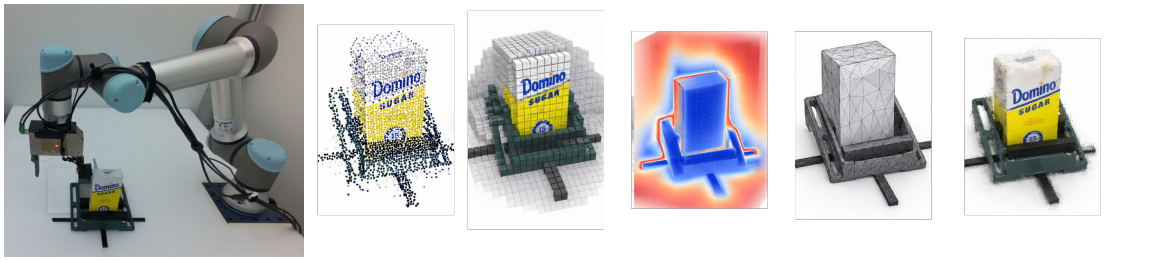


Fig. 3.2 Schematic illustration of different 3D representations of the same object. The 3D visualizations are illustrative and are not ground-truth reconstructions or outputs computed from the source data. The scene image is from Suresh et al. (2022)

3.3.1 3D/4D Scene Representation and Modeling

Scene representations differ in the properties they encode and the queries they support. In a calibrated pinhole camera, a world point $\mathbf{p}_W$ projects to image coordinates according to

$$\mathbf{p}_C = \mathbf{R}_{CW}\mathbf{p}_W + \mathbf{t}_{CW}, \qquad \tilde{\mathbf{u}}_{pix} \propto \mathbf{K}_{cam}\mathbf{p}_C, \tag{3.13}$$

where $\mathbf{R}_{CW}$ and $\mathbf{t}_{CW}$ define the transform from world to camera coordinates, and $\mathbf{K}_{cam}$ is the intrinsic matrix (Hartley and Zisserman, 2003). Given axial depth $Z_{depth}(u_{pix}, v_{pix})$, the corresponding camera-frame point is $\mathbf{p}_C = Z_{depth}(u_{pix}, v_{pix})\mathbf{K}_{cam}^{-1}\tilde{\mathbf{u}}_{pix}$. Thus, RGB-D data can be lifted into 3D and registered to robot or world coordinates, subject to calibration, depth error, and temporal alignment.

Point clouds retain measured or predicted surface samples, but do not by themselves define continuous surfaces, object interiors, or free space. Occupancy grids represent cells as occupied, free, or unknown, often with probabilistic occupancy. OctoMap is a probabilistic octree-based example (Hornung et al., 2013). Signed-distance fields encode distance to a surface, with the zero level set representing that surface under a chosen sign convention. They support geometric queries such as clearance and collision checking. Voxblox incrementally constructs Euclidean SDFs for planning (Oleynikova et al., 2017). Meshes explicitly represent surfaces and can support rendering or collision queries when constructed for that purpose. None of these representations alone supplies all physical properties required for interaction, such as friction, mass, articulation, or contact dynamics. Their coverage is also limited by sensor visibility and spatial resolution.

NeRFs represent a scene by mapping 3D position and viewing direction to volume density and view-dependent radiance, $(\sigma_{rad}, \mathbf{c}_{rgb}) = F_\theta^{NeRF}(\mathbf{x}_{pos}, \mathbf{d})$ (Mildenhall et al., 2020). 3D Gaussian Splatting (3DGS) represents a scene using anisotropic Gaussian primitives optimized for efficient rendering (Kerbl et al., 2023). These appearance-oriented representations can support high-quality novel-view synthesis, but rendering quality alone does not establish accurate metric geometry or collision-ready surfaces.

For dynamic scenes, the representation must also describe temporal change. Time-dependent radiance fields and Gaussian models can reconstruct or render scenes at different times. 4D Gaussian Splatting is one example that models dynamic scenes with Gaussian primitives and learned deformations (Wu et al., 2024a). When motion estimation or interaction prediction is required, the system must additionally maintain quantities such as object identity, pose or shape trajectories, and correspondences over time. A temporally plausible rendering does not by itself guarantee physically valid motion.

Learning-based reconstruction has reduced the need to optimize a separate model from scratch for every scene. For example, VGGT directly predicts camera parameters as well as geometric attributes such as depth maps, point maps, and point tracks from one or more images (Wang et al., 2025a). pixelSplat predicts a Gaussian radiance-field representation from image pairs (Charatan

et al., 2024). These predictions can complement geometric reconstruction, but scale, coordinate-frame alignment, calibration, and task-specific accuracy still require attention. Geometry in regions not constrained by observations is inferred rather than measured.

World models can combine scene representation with prediction or generation, but these functions should be distinguished. A renderer that generates plausible views is not necessarily a model that predicts physically valid outcomes under robot actions. Atlas, for example, is described by its developers as producing point-cloud or Gaussian-splat outputs from images and generating novel views. Its developers also state that unseen regions may be inferred to fill gaps (World Labs Team, 2026). A related but distinct capability is tool-assisted 3D authoring. OpenAI reports that GPT-6 Astra can generate computer-aided design (CAD) models from multi-view renders with tools and, in the reported demonstration, is used to create an editable house scene in Blender and a walk-through version in Unreal Engine (OpenAI, 2026).[2] These developer-reported examples demonstrate 3D generation and software-mediated modeling. Their use for robot interaction requires distinguishing generated content from sensor-supported estimates and validating geometry and motion against physical constraints.

3.3.2 Model-Driven Spatial Perception and Reasoning

Learned models can associate geometric representations with semantic concepts and use these associations to answer spatial queries or guide actions. The approaches below differ mainly in where spatial information is retained and how it is grounded in observations.

Models Using Explicit Spatial Representations An explicit map can associate semantic features or object labels with points, surfaces, or regions in a common coordinate frame. OpenScene, for instance, co-embeds features of 3D scene points with image and text features, enabling open-vocabulary queries about observed space (Peng et al., 2023). Such maps allow geometric grounding and repeated queries, although their coverage and semantic accuracy depend on observation quality, registration, and the underlying visual features.

Spatial Perception with Vision-Language Models Vision-language models can infer object relations and answer spatial questions directly from images and text without necessarily maintaining a persistent 3D map. SpatialVLM shows that spatially grounded training can improve qualitative and quantitative spatial question answering (Chen et al., 2024). These outputs can help interpret instructions and propose likely relationships, but a linguistic answer should not be treated as a verified metric measurement or reachability judgment, particularly when viewpoint, occlusion, or unknown scale creates ambiguity.

Agent-Based Spatial Perception and Reasoning An agent can couple perception to action. It maintains a spatial state, chooses where to look or move, and revises its estimates as new observations arrive. SayNav incrementally builds a 3D scene graph to ground language-model plans for navigation in unfamiliar environments (Rajvanshi et al., 2024). VoxPoser grounds language instructions in 3D value maps and uses model-based planning to generate closed-loop manipulation trajectories (Huang et al., 2023a). These examples illustrate how spatial representations and learned knowledge can inform action. Reliable operation still requires checking proposed plans against current geometry, robot configuration, collision constraints, and physical limits, and updating the estimate from feedback during execution.

3.4 High-Level Perception and Cognition

High-level perception organizes sensory observations and spatial representations into entities, attributes, states, and relations that are relevant to a task. Cognition builds on these representations by integrating prior knowledge, retaining relevant history, reasoning about spatial and physical

[2]Thomas Ricouard, OpenAI Developers Blog, *Architectural Visualization with Astra*, September 4, 2026.

relationships, and identifying what additional evidence is needed when current observations are insufficient.

For example, picking up a cup requires more than detecting and localizing it. The system may need to estimate its contents and physical state, identify an appropriate contact region, and determine whether the intended interaction is feasible. Similarly, monitoring an automated experiment requires associating a vessel with the correct sample, interpreting changes in material state, and relating visual observations to instrument measurements and experimental history. Both cases require representations that connect semantic understanding with physical evidence and persist beyond a single observation. This section develops a progression from object- and scene-level semantic perception to the understanding of physical properties, states, and interaction possibilities. It finally addresses grounded reasoning with vision-language models, memory, measurements, and active perception.

3.4.1 Object Recognition and Scene Semantic Understanding

Object- and scene-level semantic perception provides the foundation for high-level physical understanding. It identifies the entities present in an environment, localizes them in observations, maintains their identities over time, and organizes their semantic and spatial relationships. These representations provide the targets and contextual structure required for subsequent state estimation, interaction perception, and task-level reasoning.

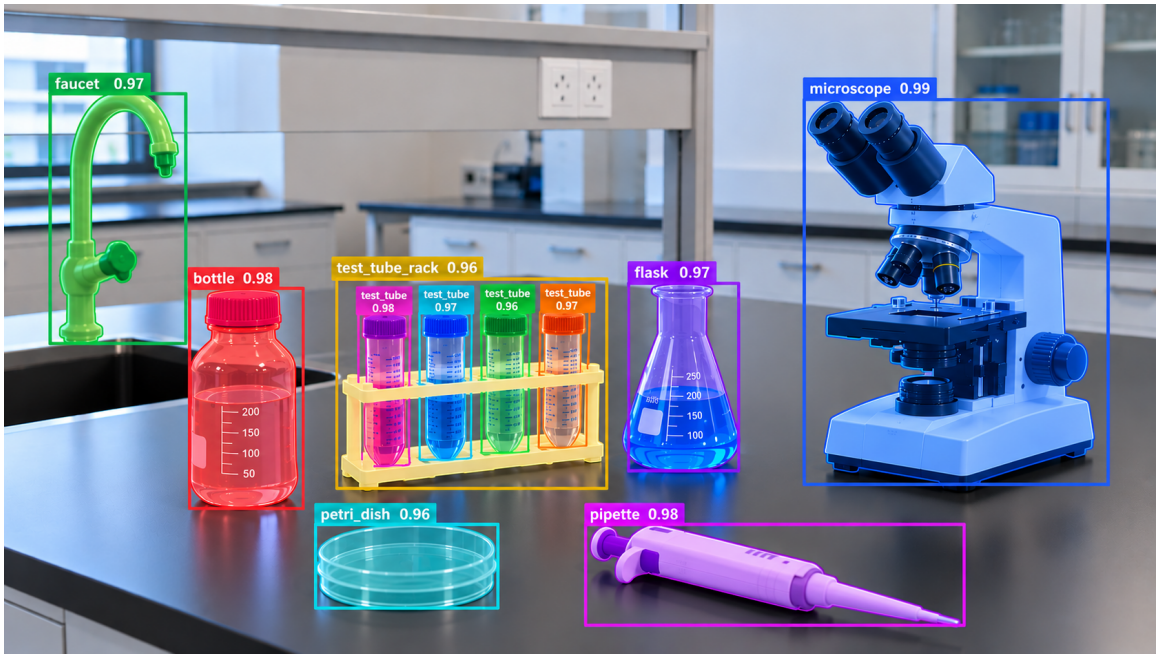


Fig. 3.3 Illustration of object detection and instance segmentation. Bounding boxes indicate the locations and categories of detected objects, while colored masks delineate individual object instances at the pixel level

Object Detection, Segmentation, and Tracking Object detection predicts object categories and locations, typically represented as bounding boxes. Semantic segmentation assigns category labels at the pixel level, whereas instance segmentation distinguishes individual object instances within each category, as illustrated in Fig. 3.3. Mask R-CNN jointly predicts object categories, bounding boxes, and instance masks (He et al., 2017), while DETR formulates object detection as Transformer-based set prediction (Carion et al., 2020). Tracking extends recognition across time

by maintaining object identities using appearance, motion, and geometric information. ByteTrack, for example, incorporates lower-confidence detections during association to recover partially occluded objects (Zhang et al., 2022). Together, detection, segmentation, and tracking provide object categories, spatial regions, and persistent identities that support target selection and state estimation. A cup temporarily occluded by a gripper, for example, should remain associated with the same contents, physical state, and interaction history after it becomes visible again.

Open-vocabulary recognition extends this capability beyond a predefined set of object categories. Grounding DINO localizes objects specified through textual descriptions (Liu et al., 2024), while the Segment Anything Model (SAM) generates segmentation masks from spatial prompts such as points and bounding boxes (Kirillov et al., 2023). Combining language-grounded detection with promptable segmentation connects task descriptions to observable image regions, allowing a system to identify and segment relevant targets without requiring an exhaustive task-specific label set.

Scene-Level Semantic Perception and Understanding Recognizing individual objects is only the first step toward understanding a physical environment. Scene-level perception organizes entities according to their spatial, semantic, and functional relationships. Scene graphs, for example, represent objects or regions as nodes and relationships such as support, containment, proximity, or attachment as edges. These structured relationships provide context for interpreting both individual objects and the overall scene. For example, cups and a kettle located on the same countertop may jointly indicate a beverage-preparation area. In a laboratory, scene relationships may associate a sample with its vessel, a vessel with an instrument, and an observation with the experimental stage at which acquisition occurs. Such associations make isolated sensor observations interpretable within the context of an ongoing task or physical process.

Context can also help resolve ambiguous observations. An object that appears partially behind a drawer may be contained inside it or merely occluded by it, while a visible liquid region may belong to one of several overlapping transparent vessels. Scene representations should therefore retain not only predicted relationships but also the evidence and uncertainty associated with them. New observations can then confirm, reject, or revise previously inferred relationships.

These semantic representations answer questions about *what* entities are present, *where* they are, and *how* they are related. Physical interaction, however, requires additional information about *what properties they have*, *what states they are in*, and *what actions they afford*. These aspects are considered next.

3.4.2 Object Properties, States, and Interaction Perception

High-level physical perception extends object recognition beyond identifying *what an object is* toward understanding *what properties it has*, *what state it is in*, and *how it can be interacted with*. Such understanding requires connecting semantic observations with physical properties, functional structures, interaction possibilities, and feedback from the environment.

Observable Attributes and Latent Physical Properties Objects exhibit directly observable attributes, such as color, texture, shape, and visible geometry, as well as physical properties that are only indirectly observable, including mass, friction, stiffness, compliance, and fragility. Visual appearance provides useful cues for these properties but is often insufficient for reliable estimation. Visually similar objects, for example, may differ substantially in mass because of their contents or internal structure.

Multimodal sensing and physical interaction provide complementary evidence for resolving such ambiguities. Deformation under load can reveal compliance, object motion during controlled contact can constrain friction estimates, and force measurements during lifting can provide evidence about mass. These estimates remain dependent on contact conditions, sensing quality, and dynamical assumptions, and should therefore retain uncertainty. Their primary role in physical perception

is not necessarily to recover exact physical parameters, but to provide sufficient information for selecting appropriate contact regions, forces, and interaction strategies.

States, Functional Parts, and Affordances Recognizing an object category alone does not determine how the object should be used or manipulated. Object function describes its typical role, while functional parts identify regions that support particular interactions. A cup, for example, may function as a container, while its handle provides a functional region for grasping.

Affordances further describe the actions that are possible for a particular agent under the current physical and task conditions. Unlike object categories or intrinsic properties, affordances are relational. They depend jointly on object geometry, physical state, environmental context, agent capabilities, and task requirements. A handle may be graspable by one gripper but inaccessible to another, and an otherwise graspable cup may be unsuitable for lifting when its contents are likely to spill. In an automated laboratory, relevant affordances may instead concern access to a vessel opening, available capacity, or whether a valve can be operated in its current state.

Interaction perception connects these semantic and physical descriptions to possible actions and state transitions. It estimates discrete conditions such as open, closed, grasped, or slipping, as well as continuous quantities such as deformation and contact force. It also associates functional parts with candidate interactions and evaluates whether an observed response is consistent with the intended action. Methods such as Where2Act demonstrate how actionable regions and interaction proposals can be inferred for articulated objects (Mo et al., 2021). Such proposals nevertheless remain candidates rather than executable actions and must be checked against reachability, collision, object state, and task constraints.

Physical interaction also reveals information that may be unavailable from external vision. During contact, relevant regions can become occluded, while incipient slip, local deformation, and contact loading may be difficult to infer visually. Tactile and force sensing provide complementary observations of these conditions. GelSight, for example, demonstrates how vision-based tactile sensing can capture local contact geometry and deformation (Yuan et al., 2017). Observed slip, deformation, or unexpected resistance can then update the estimated interaction state and support adjustments to contact location, grasp force, or subsequent actions.

Material and Process-State Perception High-level physical perception is not limited to robot-object contact. In autonomous laboratories and other process-oriented environments, the target of perception may instead be a material state or a process variable. Relevant observations include liquid levels, phase boundaries, precipitates, vessel contents, and changes occurring during an experiment. Distinguishing the container from its contents is particularly important because material regions may lie within, overlap with, or be partially occluded by the surrounding vessel. Reflections, refraction, foam, transparency, and occlusion can further make visible boundaries ambiguous.

Visual observations should therefore be connected to calibrated measurements and process context. Estimating a liquid level may require vessel geometry, camera calibration, or a visible scale, while determining material identity may require sample metadata or analytical measurements. Other quantities, including temperature, pressure, mass, and spectral measurements, may be obtained directly from instruments or extracted from their displays. Regardless of the acquisition mechanism, observations should retain their units, timestamps, sample associations, and measurement quality. Maintaining these relationships allows semantic interpretation to remain grounded in the physical process being observed.

3.4.3 Grounded Perception and Cognition with Vision-Language Models

Vision-language models (VLMs) provide a flexible semantic interface between visual observations, task instructions, and prior knowledge. In Physical AI, however, semantic interpretation alone is insufficient. Model predictions must ultimately be grounded in specific entities, physical states, spatial geometry, measurements, and interaction history. VLMs should therefore be viewed as

one component of a broader perception and cognition system rather than as a replacement for geometric estimation, physical sensing, memory, or measurement.

Semantic, Property, and Affordance Understanding VLMs can associate visual appearance with semantic concepts, material cues, object functions, and typical interactions. This capability is particularly useful when a system encounters unfamiliar objects or must interpret them in the context of a task. Given an image and an instruction to move a cup, for example, a ceramic appearance may suggest fragility and careful handling, while a visible handle may indicate a candidate interaction region. PhysObjects illustrates the use of physical-concept annotations to improve the understanding of properties relevant to manipulation (Gao et al., 2024).

Such predictions should be interpreted as hypotheses rather than direct physical measurements. A visual model may suggest that an object is heavy, fragile, or slippery, but quantitative or safety-critical judgments generally require additional sensing or interaction evidence. Semantic knowledge therefore provides useful priors that can guide subsequent observation and interaction without replacing physical verification.

Semantic knowledge must also be grounded in specific locations in the current environment. RoboPoint, for example, predicts image-space affordance points from visual observations and language instructions, linking semantic requests to candidate interaction regions (Yuan et al., 2025). These predictions help identify *where* an interaction may occur but do not by themselves specify a feasible physical action. Depth, geometry, object state, collision constraints, and robot kinematics are still required to convert semantic interaction targets into executable contact poses and motions.

Reasoning with Geometry, Memory, and Measurements High-level physical reasoning frequently requires information beyond the current image. Spatial questions may require calibrated geometry, questions about change require previous observations, and process-state interpretation may depend on event history or instrument measurements. A grounded cognition system therefore combines semantic reasoning with explicit geometric representations, persistent memory, and quantitative measurements.

Relevant information can be retrieved through object, sample, or task identity and associated with the current observation. Geometry provides spatial structure, memory provides temporal context, and sensors or instruments provide quantitative physical evidence. VLMs can integrate these sources to interpret a request, relate observations to the intended task, and identify missing or inconsistent evidence. Memory management, geometric computation, and numerical measurement remain explicit system functions even when their outputs are consumed by a multimodal model.

PaLM-E demonstrates one architecture for incorporating visual observations, continuous state estimates, and textual representations into an embodied language model (Driess et al., 2023). More generally, physical information may be provided to a reasoning model through structured records, tool outputs, or textual representations. Regardless of the interface, physical semantics should be preserved, including units, timestamps, coordinate frames, object identities, and measurement uncertainty.

Consider determining whether a vessel has reached a target fill level. Visual perception first identifies the vessel and candidate liquid boundary, while calibration or vessel geometry converts image evidence into a physical estimate. Memory provides the target level and relevant filling history, and complementary measurements can be consulted when the visual evidence is ambiguous. The VLM helps interpret the task and associate the available evidence with the correct sample, while geometric and measurement tools support the quantitative comparison. This division of roles prevents semantic plausibility from being mistaken for physical certainty.

Active Perception and Feedback-Based Updating Physical cognition becomes particularly important when the available observations do not uniquely determine the current state. Rather than forcing a decision from incomplete evidence, a high-level reasoning module, potentially imple-

mented with a VLM, can maintain competing hypotheses, identify unresolved variables, and propose additional observations or controlled interactions that reduce uncertainty.

The resulting process forms a perception-cognition loop. The system observes the environment, interprets the available evidence, identifies relevant uncertainty, acquires additional evidence, and updates its internal state. Additional evidence may come from a new camera viewpoint, a different sensing modality, an instrument measurement, or a controlled physical probe. The choice of observation should consider expected information gain, acquisition cost, execution risk, and potential disturbance to the physical process.

For example, when observing unfamiliar storage furniture, a change in viewpoint may reveal hinges or rails and help distinguish rotational from translational motion. If ambiguity remains, a gentle probe with measured motion and force can provide further evidence about the underlying mechanism. In a laboratory, an apparent liquid boundary may instead correspond to a reflection or foam edge. A different viewpoint or illumination condition can test the visual hypothesis, while a mass measurement may provide independent evidence. In both cases, active perception should target the specific uncertainty rather than simply acquire more observations.

Feedback from execution provides another source of evidence. A failed grasp, unexpected resistance, observed slip, or inconsistent measurement can revise the estimated object state, interaction hypothesis, or process interpretation. Perception and cognition are therefore not restricted to a single observation-decision sequence. They continuously update their representation of the physical environment as new evidence becomes available.

Limitations, Multiple Time Scales, and Outlook High-level semantic reasoning remains vulnerable when model outputs become detached from geometry, measurement, or physical history. Plausible semantic predictions do not guarantee correct localization, physical feasibility, or accurate state estimation. Evaluation should therefore extend beyond language quality to include entity grounding, spatial relations, physical-property judgments, state estimation, affordance prediction, and the appropriate use of new evidence. Occlusion, viewpoint changes, ambiguous material boundaries, and conflicting instrument readings provide useful settings for testing whether a system can recognize uncertainty and request additional evidence rather than produce an unsupported conclusion.

Another challenge arises from the multiple time scales of physical perception. Contact and force measurements may change rapidly, visual observations arrive at another rate, and material, thermal, or experimental processes may evolve over much longer intervals. A shared physical state should therefore preserve acquisition time, validity, uncertainty, and evidence provenance. High-level semantic reasoning can query this state when needed without requiring every sensor update to pass through a language model. This separation enables high-frequency physical sensing and lower-frequency semantic reasoning to coexist within the same perception architecture.

Looking forward, a research direction is the development of persistent and grounded physical cognition that combines multimodal memory, 3D spatial reasoning, quantitative measurements, and active perception. Such systems should not only recognize objects and describe scenes, but also maintain up-to-date estimates of physical states, identify missing evidence, and select informative observations or interactions when uncertainty matters. For autonomous laboratories, this requires connecting sample identity, material observations, instrument measurements, and experimental history. For robotic manipulation, it requires linking semantic targets to geometry, physical properties, interaction states, and execution feedback.

Progress should ultimately be assessed by whether these capabilities improve the interpretation and control of physical systems. The assessment should determine whether a system can correctly identify relevant entities, maintain their evolving states, recognize when its evidence is insufficient, acquire information that resolves the ambiguity, and revise its interpretation under real feedback. From this perspective, VLMs provide a flexible semantic and reasoning interface within a broader physical perception and cognition system, while grounded sensing, geometry, memory, and interaction provide the evidence that keeps such reasoning connected to the physical world.

4 Data for Physical AI

Perception and cognition are only passive to understand our physical world. Still, to build an active embodiment, interaction data in the real world is critical, as it provides the empirical foundation for AI systems to understand the actual physical dynamic environment and how to interact with it. Since this data records how actions alter physical states under known operating conditions, one can learn a control policy, calibrate a simulation pipeline, or build a validation benchmark. This chapter focuses on how to build a robust data collection pipeline for these purposes. It first introduces the data formats, curation pipelines, and real-world collection strategies spanning human teleoperation, autonomous exploration, and interactive feedback. It then examines how physics engines and generative world simulators expand data coverage, and how sim-to-real transfer aligns synthetic trajectories with physical hardware. Finally, it analyzes physical data scaling laws, the role of heterogeneous cross-embodiment datasets, and closed-loop data flywheels driven by deployment feedback.

4.1 Data Foundations for Physical AI

Physical data link observations, actions, and their consequences under specified operating conditions. A drawer image may support recognition. Learning to open it also requires the applied commands, contact events, and resulting motion. The same distinction applies to industrial processes, traffic control, and automated experiments. This section defines the evidence and record structure needed to interpret such data. Subsequent sections address its collection, generation, alignment, and growth.

4.1.1 The Spectrum of Physical Data

Data sources differ in how directly they support physical execution. The *Data Pyramid for Embodied Manipulation* organizes real-robot data, Universal Manipulation Interface (UMI)-style demonstrations, egocentric and exocentric observations, simulation data, and general vision-language data around the tension between scalability and robot alignment (Ye et al., 2026b). Here, the complementary question is what each record actually establishes. Video can reveal an object's motion without identifying the force or command that caused it. A simulated trajectory can include exact action labels without establishing the corresponding real-world response.

Table 4.1 distinguishes these forms of evidence. They can coexist within one dataset and do not constitute a ranking of quality. In particular, reconstructed poses and latent actions are *action proxies*. Converting them into executable supervision requires a mapping to the target embodiment and controller. Representative released datasets spanning robot execution, portable demonstrations, and human observation are cataloged in Table 4.3 in Section 4.2.

For interaction data, let s_t denote the underlying physical state, o_t the observation, and a_t the action. A useful conceptual record is

$$\tau_{\text{ep}} = \left(c, g, \left\{ \left(o_t, \zeta_t^{\text{state}}, a_t^{\text{cmd}}, a_t^{\text{exec}}, e_t^{\text{event}}, \Delta t_t \right) \right\}_{t=0}^{T}, y \right), \tag{4.1}$$

where c records the embodiment, environment, and collection conditions, and g is the goal. The optional field ζ_t^{state} is a state estimate or belief, while e_t^{event} records events such as contact or intervention. The field Δt_t gives elapsed time, and y records the outcome. The command a_t^{cmd} is distinguished from a_t^{exec}, which denotes the measured execution response rather than a second command. The available fields vary by source. Missing measurements require validity masks, and inferred values require provenance.

4.1.2 Unified Representation

A common schema must preserve the meaning of its fields. In the RT-X experiments, Open X-Embodiment converts actions into a common seven-dimensional end-effector representation, but

Table 4.1 Evidence levels in physical data and their limits

Evidence	Provides	Does not establish
Observation	Appearance, scene context, visible motion	Executable commands or contact forces
Action proxies	Reconstructed poses, trajectories, or latent actions	Feasibility on the target embodiment
Execution	Recorded commands and measured responses	Coverage of unfamiliar states or failures
Closed-loop experience	Policy-induced states, interventions, and outcomes	Correct behavior or reliable annotations
Synthetic evidence	Controlled variation, scalable trajectories, dense model-derived labels	Real-world dynamics or sensor behavior

retains differences in coordinate frames and in absolute, relative, or velocity control. The same vector can therefore induce different motions on different robots (Open X-Embodiment Collaboration, 2024). Matching dimensions enables shared processing but does not establish physical equivalence.

Each record should retain the embodiment, tool geometry, control interface, and calibration needed to interpret it. Coordinate transforms must specify their direction and reference frames. Sensor acquisition, command issue, and physical response times must also remain distinguishable. Pairing an image with a command issued before its exposure can change the supervision target. Sampling rate, action duration, and controller update rate should be recorded separately.

Measured, estimated, and generated quantities require distinct provenance. A reconstructed wrist pose is an estimate, and a simulator's contact force is a model output. Neither should silently replace a physical measurement. Derived fields should identify their source records, generating method and version, and any available uncertainty or validity bounds. A shared coordinate frame resolves geometric ambiguity but leaves embodiment-dependent limits and dynamics to be accounted for.

Minimum Action Specification An action field should identify its issuer and target, control mode, reference frame and tool point, units, duration and effective time, applicable limits or saturation, controller gains where relevant, and the associated execution feedback.

4.1.3 Data Organization

Physical records are organized into steps, transitions, segments, and episodes. These boundaries carry learning semantics. Reinforcement Learning Datasets (RLDS), for example, distinguishes the last recorded step from an environment terminal state. A truncated episode does not establish that the task terminated, and its final observation need not have a valid outgoing action. Training views must preserve these distinctions instead of treating every stored row as a complete transition.

Raw records, standardized records, and training views serve different purposes. Raw records preserve the original signals and metadata. Standardized records make units, timestamps, coordinates, and episode boundaries explicit. Training views specify selection, segmentation, annotation, and augmentation. Keeping these stages traceable allows a calibration or labeling error to be corrected without repeating collection.

Every episode should retain a stable identifier and links to its source, embodiment, operator or policy version, calibration, and processing history. Segments and generated variants should retain their parent identifiers. Dataset splits can then keep related records together and hold out the task, scene, object, or embodiment relevant to the generalization claim. Splitting adjacent frames at random cannot establish transfer to unseen physical conditions.

4.1.4 Annotation Framework and Workflow

A recording may show an object moving, a temperature rising, or a machine stopping. The signals are recorded, but what happened? Annotations identify the objects and materials involved, the operating conditions, the actions taken, and their consequences. They may mark an object in an image, contact during a motion, stages of a production process, or the outcome of an experiment. The same record can then support different learning tasks, including recognizing a material, predicting its response, choosing an action, or judging whether the outcome meets a goal.

Annotation starts with rules that different people and tools can apply consistently. A small trial reveals unclear definitions and disputed boundaries before labeling expands. Automated tools can propose labels, while reviewers examine uncertain cases and spot-check the rest against the recordings. Each label keeps a link to its evidence and records whether it comes from measurement, human judgment, inference, or simulation. Uncertainty remains visible, and corrections update the labels without rewriting the underlying evidence.

4.1.5 Quality Governance

Quality checks should distinguish recording defects from difficult but valid experience. A force spike may indicate contact. A stalled gripper may reveal an execution failure. Neither warrants deletion solely because the signal is irregular. Checks should cover signal integrity, consistency across timestamps and coordinate frames, correspondence between commands and responses, and agreement between event annotations and outcomes.

Learning utility also depends on the learner and the intended supervision. The robomimic study systematically varies demonstration quality, observations, and learning algorithms, showing that these choices materially affect policy performance (Mandlekar et al., 2022). A valid failure trajectory can support outcome prediction while providing unsuitable actions for nominal behavior cloning. Data validity, task outcome, and suitability as an imitation target should therefore be recorded separately.

Dataset documentation makes these judgments inspectable. Following *Datasheets for Datasets*, a release should describe its composition, collection process, intended uses, and limitations (Gebru et al., 2021). For physical data, this should include embodiment and control specifications, calibration history, missing modalities, outcome definitions, and known recording defects. A versioned manifest should link released samples to their processing code and access or reuse conditions. These records provide the basis for the collection procedures in Section 4.2 and the synthetic-data validation developed in Section 4.3 and Section 4.4.

4.2 Data Collection and Interaction

Data collection and interaction proceed through collection design, human demonstrations, autonomous collection, and interactive feedback in four linked steps. In this view, producing data means actively sampling the physical world. The emphasis is on distributions, supervision, efficiency, and safety. Implementation details of individual devices are addressed only where they constrain the evidence that can be collected.

4.2.1 Collection Design

Collecting physical data is not indiscriminate recording of real-world processes. It is an active sampling of the space of reachable states, possible actions, and actual outcomes. Initial states, task goals, acting agents, control constraints, disturbances, and termination conditions all shape the resulting distribution. The collection policy is therefore part of the data-generating mechanism. A sound design first identifies the capability to be developed or the hypothesis to be tested, and then determines what to collect.

Design typically starts with the intended deployment distribution and works backward to define task families. A coverage matrix can span objects, scenes, initial states, goals, disturbances, embod-

iments, operators or policies, outcomes, and risks. Frequent states require sufficient density to establish basic competence. Rare but consequential states require targeted collection. Matched comparison cases can weaken spurious correlations with backgrounds, positions, or operator styles. The number of task names is not a measure of coverage. "Tidy the desk" can probe very different capabilities as object combinations, occlusion, starting poses, and time pressure change.

The variable specification should distinguish controlled factors, observable quantities, and potential confounders. In robotic manipulation, one can vary object positions and materials, container states, illumination, and disturbances. Traffic and industrial systems also require records of topology, load, shifts, operating rules, and decisions by multiple actors. Intelligent laboratory systems should retain reagent batches, instrument states, step dependencies, and measurement uncertainty. These conditions must be represented in task templates and metadata if performance differences are to be attributed to plausible causes rather than described merely as isolated successes or failures.

Sensor configuration further bounds what can be observed. Measurement principles, mounting locations, and coverage determine which quantities can be measured directly. Range, noise, and calibration determine how trustworthy those measurements are. An uninstalled or uncovered channel should be marked as missing, rather than represented by a zero reading. Contact forces inferred from images or a dynamics model must likewise be distinguished from directly measured forces. Temporal resolution depends jointly on sensor bandwidth, sampling rate, filtering, and the transmission path. A higher logging rate does not necessarily provide more usable information.

Force and tactile sensing illustrate these constraints. A six-axis wrist force/torque sensor measures the net force and moment, not the distribution of fingertip contacts. A tactile array covers only its sensitive surface. Some force sensors support sampling at kilohertz rates (ATI Industrial Automation, n.d.), whereas vision-based tactile sensors are limited by camera frame rate. The DIGIT prototype, for example, operates at 60 frames/s (Lambeta et al., 2020). Directly reducing a high-rate force stream to the video frame rate can hide collision peaks and transient slip. The raw time series should therefore be retained, with a separately aligned stream generated after antialiasing. Collection also requires calibration and zeroing, together with compensation of wrist readings for tool weight and inertial effects. Quality checks are also required for thermal drift, hysteresis, saturation, and dropped frames. Calibration versions, filter settings, and anomaly flags should accompany the data. The confidence assigned to contact labels should account for both synchronization error and measurement uncertainty.

Success criteria, failure categories, safety limits, and reset rules must be defined before collection. Success should be expressed in verifiable object states, process conditions, or operational metrics rather than left to an operator's impression. Failures should distinguish perception errors, planning errors, control deviations, unstable contact, timeouts, safety aborts, and external disturbances. An incomplete reset, or physical traces left by a previous trial, can also compromise the independence of subsequent data. When an outcome is difficult to determine automatically, sensor-based rules can be combined with visual checks and human review in a staged assessment.

Temporal segmentation is likewise part of collection design. An episode should start at a verifiable initialization or reset and end in success, failure, timeout, or an abort. Skill segments and action chunks should have explicit temporal meanings. The acquisition system must distinguish the rates of sensor sampling, model decisions, action updates, and low-level servo control. It should record the issue, delivery, and actual effective time of each command. Source timestamps, coordinate-frame calibration, latency measurement, and completeness monitoring are minimum requirements. Detailed hardware and software implementation belongs in Chapter 9.

Recent large-scale real-world datasets illustrate different organizational approaches. DROID shows how relatively uniform hardware and protocols can support distributed collection across sites. RoboMIND brings multiple embodiments, task protocols, and demonstrations of failure with recorded causes into one platform. AgiBot World scales trajectories through centralized facilities, standard procedures, and human-in-the-loop verification. Humanoid Everyday further includes whole-body locomotion, manipulation, and human-robot interaction (Khazatsky et al., 2024; Wu

et al., 2025b; AgiBot-World-Contributors et al., 2025; Zhao et al., 2026). Together, these efforts show that scaling requires reproducible task definitions, staffing procedures, equipment states, and quality judgments, as well as more devices.

Collection performance should be measured by *effective data yield*. Beyond raw hours and trajectory counts, useful measures include minutes of meaningful interaction, the share of recordings that pass training quality checks, incremental coverage after deduplication, the yield of rare events, equipment availability, human and machine time, time spent resetting, and the final retention rate from raw records to the training release. Consecutive stationary frames, repeated successes, and multiple derived versions of the same behavior inflate volume without necessarily adding new experience. Efficient collection uses limited physical interaction to close identifiable capability gaps.

The output of collection design is thus an executable data-collection protocol, not merely a task list. It specifies the target distribution, variable combinations, episode boundaries, success and failure definitions, safety constraints, data schema, quality gates, and cost metrics. Human demonstrations, autonomous execution, and interactive corrections are different ways of gathering evidence under this plan.

4.2.2 *Human Demonstrations*

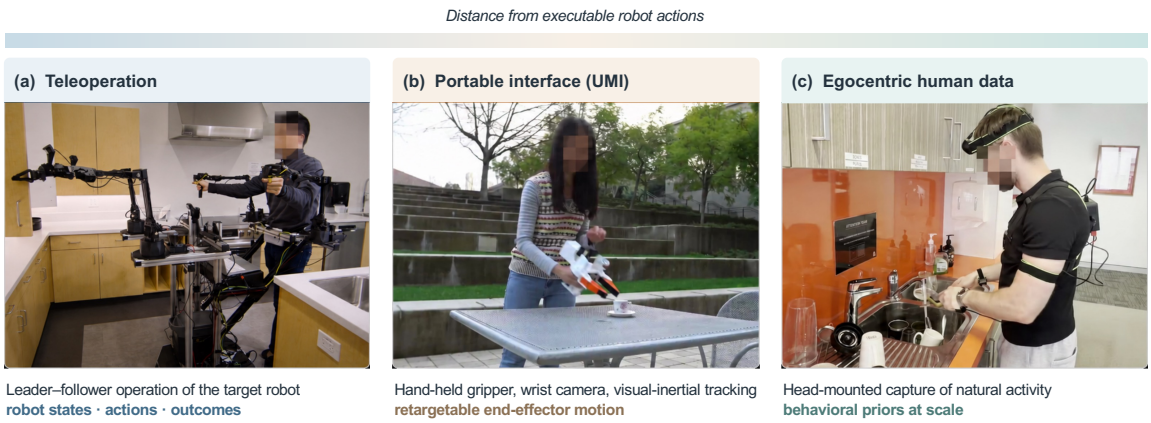


Fig. 4.1 Human-demonstration data collection paradigms, arranged by their *distance from executable robot actions*. (a) Teleoperation drives the target robot through leader-follower mechanisms and records robot observations, actions, and outcomes directly (Mobile ALOHA, photo from Stanford News). (b) Portable interfaces such as UMI record retargetable end-effector motion in natural environments using a hand-held gripper, a wrist-mounted camera, and visual-inertial tracking (photo from the UMI project). (c) Egocentric human data capture natural activity at scale with head-mounted devices, providing broad behavioral priors without native robot states (photo from ABC News)

Human demonstration converts human intent into behavioral evidence that a machine can learn from and, ideally, execute on the target embodiment. Methods can be arranged by their distance from executable robot actions (Figure 4.1). Teleoperation and kinesthetic teaching with the target robot in the loop directly produce embodied actions. Portable tools, motion capture, and wearable systems record motions that can be retargeted. Egocentric and Internet videos provide broad observations and behavioral priors. These sources complement one another in action fidelity, coverage of natural settings, equipment occupancy, and processing cost.

Master-slave robot arms, spatial controllers, and immersive teleoperation operating on the target embodiment can record robot observations, control commands, embodiment states, contacts, and outcomes together. ALOHA uses matched leader and follower mechanisms for fine-grained bimanual demonstrations (Zhao et al., 2023). GELLO provides a low-cost, kinematically matched

interface that helps an operator perceive joint limits (Wu et al., 2024b). Open-TeleVision incorporates stereoscopic views and active head motion into the teleoperation loop (Cheng et al., 2025). The collection interface is therefore not a neutral accessory. Its mapping, feedback delay, viewpoint strategy, and operator burden systematically shape trajectories.

Portable, robot-independent interfaces bring demonstrations into natural environments. UMI records relative end-effector motion with a handheld gripper, wrist-mounted cameras, and visual-inertial tracking, then accounts for target-robot reachability and latency at deployment (Chi et al., 2024). DexWild uses a portable, human-centered system to extend dexterous demonstrations in everyday settings (Tao et al., 2025b). FreeTacMan synchronizes visual and tactile collection, so that contact feedback can support both the operator and supervision of contact phases (Wu et al., 2026a). These methods reduce occupancy of the target robot and expand scene coverage, but still have to address tracking errors, scale, action retargeting, and the target embodiment's dynamics.

Humanoid and mobile manipulation extend demonstration beyond tabletop end-effector motion to whole-body coordination. TWIST2 uses portable full-body teleoperation without external motion capture to bring legged locomotion, torso, both arms, hands, and head-mounted views into one loop (Ze et al., 2025). The central requirement is not simply more degrees of freedom. Navigation, support, manipulation, and active vision must remain temporally consistent. A recording should also distinguish actions supplied by the operator from compensation supplied by a stabilizer or whole-body controller. Otherwise, high-level intent and low-level control corrections become conflated.

Egocentric human data offer natural behavior at greater scale. EgoDex synchronizes egocentric images with three-dimensional head, upper-body, and hand poses, turning video into a more calibratable behavioral record (Hoque et al., 2026). AoE explores sustained collection of natural egocentric experience and joint training with a smaller amount of robot teleoperation data (Yang et al., 2026a). ActiveMimic jointly recovers camera and wrist trajectories from single-camera in-the-wild egocentric video, emphasizing the relationship between active viewpoints and manipulation motion (Lin et al., 2026). These approaches broaden the range of objects, scenes, behavioral phases, and viewpoint priors. Human-video records themselves, however, do not natively supply robot joint states, gripper signals, tactile measurements, or feedback from actual robot execution.

Turning human or Internet videos into robot supervision requires alignment of camera frames, recovery of scale, mapping from hands or bodies to tools, correction for kinematic reachability, temporal alignment, contact assumptions, and confidence assessment. Point tracks, wrist poses, body poses, and latent actions are action proxies with different levels of reliability. They should not all be labeled as ground-truth robot actions. Table 4.2 compares the supervision, executability, and limitations of interaction data sources. More defensible training strategies include learning visual and temporal representations first and then post-training on robot data, using separate loss heads for human and robot data, weighting examples by confidence, or restricting explicit action supervision to reliable channels.

Demonstration quality also depends on how people are organized. A collection plan should retain anonymized operator identifiers, levels of training, interface configurations, and work periods. Task rotation, fatigue monitoring, spot checks of replays, and coverage across operators can reduce style bias. Expert demonstrations may not visit the error states encountered by novices, while natural human movements may violate a robot's safety constraints. The value of a demonstration lies in how clearly it conveys intent, constraints, and an executable path.

Human data thus form a gradient from rich semantics to reliable executable actions. A sound combination uses observational data to broaden tasks and scenes, wearable and tool-based data to reduce the gap in action semantics, and interactions on the target embodiment to calibrate execution. Whether cross-domain data actually help must ultimately be established through closed-loop evaluation on robots or other real systems. Section 9.6.4 follows these collection choices into a dexterous-hand system, where wrist tracking, finger capture, and contact sensing supply different parts of the demonstration.

Table 4.2 Supervisory value and principal limitations of interaction data sources

Source	Main supervision	Executability	Main bias and suitable use
Target-embodiment teleoperation	Robot observations, commands, executed motions, contacts, and outcomes	High	Affected by interfaces, delays, and operator styles. Suitable for behavior cloning and skill fine-tuning
Portable human demonstration	Camera, tool, and body trajectories in natural scenes	Medium	Requires calibration, retargeting, and reachability filtering. Expands object and contact coverage
Egocentric or public video	Visual content, semantics, task steps, and behavioral timing	Low	Lacks native actions and physical feedback. Supports representations, latent actions, and high-level priors
Autonomous policy rollout	Policy-induced states, actions, events, and outcomes	High	May repeat behavior or amplify policy bias. Supports learning from boundaries, failures, and online experience
Human intervention and recovery	Warning signs, takeovers, corrections, recovery, and consequences	High	Selected by intervention triggers. Supports correction, recovery, and safety-oriented post-training

Table 4.3 makes this spectrum concrete through representative publicly available datasets. Its focus is real-world manipulation and human activity relevant to the collection methods above. Reading the scale together with the supervision distinguishes robot execution records from tool-mediated demonstrations and human behavioral observations. The official resource links provide entry points for inspecting each release.

Table 4.3 Representative real-world datasets for Physical AI. Each entry identifies a release or collection, its reported scale, the available supervision, and official access points

Dataset / release	Scale and coverage	Overview and supervision	Official resources
A. Real-robot interaction datasets			
Open X-Embodiment (OXE)	>1,000,000 trajectories, 22 embodiments (reported collection)	Aggregates heterogeneous robot datasets in a common format for learning across embodiments. *Signals:* images, robot states/actions, and language where available. Modalities vary by source.	Project Data / code *Source-specific terms*
BridgeData V2	60,096 trajectories, 24 environments, 13 skills	Combines teleoperated and scripted WidowX manipulation across varied household-style settings. *Signals:* RGB, robot states/actions, and language, with depth and additional views in subsets.	Project / code Data
DROID	76,000 trajectories, 350 h, 564 scenes	Uses a shared Franka platform for distributed collection of manipulation in varied real environments. *Signals:* wrist/external stereo video, robot states/actions, calibration, and language.	Project / code Data

Continued on the next page.

Table 4.3 Representative real-world datasets for Physical AI (continued)

Dataset / release	Scale and coverage	Overview and supervision	Official resources
RH20T	>110,000 sequences, 147 tasks	Pairs contact-rich robot manipulation with human demonstrations to study multimodal skill transfer. *Signals:* RGB-D, robot states/actions, force/torque, audio, and human video, with tactile data in one configuration.	Project / data / code
RoboMIND Original release	107,000 trajectories, 4 embodiments, 479 tasks	Applies a unified teleoperation protocol across single-arm, dual-arm, and humanoid platforms. *Signals:* multi-view images, robot states/actions, language, and failure demonstrations with causes.	Project / code Data
AgiBot World Beta (2025)	1,003,672 trajectories, 100 robots	Scales bimanual manipulation through coordinated collection across replicated real-world scenarios. *Signals:* multi-camera video, robot states and issued actions, task descriptions, and segment annotations.	Project / code Data
B. UMI-style tool demonstrations and wearable hand capture			
UMI In-the-wild cup collection	1,400 demonstrations, 30 locations	Captures cup arrangement with a handheld gripper for transfer to compatible robot end effectors. *Signals:* wrist RGB, visual-inertial tool trajectories, and gripper width, provided in a processed Zarr release.	Project Data / code
FastUMI-100K	>100,000 trajectories, 54 tasks	Extends UMI-style collection to a large set of single-arm and bimanual household demonstrations. *Signals:* wrist fisheye RGB, relative tool poses, gripper states, and text, provided in a LeRobot-format release.	Project / code Data
DexWild Released collection	9,505 episodes (human + robot), 93 environments	Combines portable human-hand demonstrations with robot data for dexterous policy transfer. *Signals:* hand-mounted images, tracked wrist motion, and hand articulation, with human and robot subsets.	Project / code Data
C. Human egocentric and synchronized ego-exo datasets			
EPIC-KITCHENS-100	100 h, 45 kitchens, approximately 90,000 action segments	Records unscripted kitchen activities for learning object-centric actions and their temporal structure. *Signals:* first-person video/audio, narrations, and verb-noun action segments.	Project / data / code
Ego4D Original collection	3,670 h, 74 locations, 9 countries	Covers diverse daily activities to support first-person perception, episodic memory, and anticipation. *Signals:* egocentric video, narrations, and benchmark annotations, with extra sensors in subsets.	Project Data access *Agreement required*

Continued on the next page.

Table 4.3 Representative real-world datasets for Physical AI (continued)

Dataset / release	Scale and coverage	Overview and supervision	Official resources
Ego-Exo4D V2 release	5,035 takes, 1,286.30 video h, including 221.26 ego h	Synchronizes first- and third-person views of skilled activities for linking observed motion and behavior. *Signals:* ego/exo video, audio, gaze, camera trajectories, and task-specific body/hand pose annotations.	Project / docs Data access *Agreement required*
EgoDex	829 h, 194 tabletop tasks	Couples egocentric manipulation video with tracked human motion for learning dexterous behavior. *Signals:* RGB, 3D head/upper-body/hand poses, and language captured using Apple Vision Pro.	Project / data / code
HOT3D	833 min, 19 participants, 33 rigid objects	Provides calibrated egocentric hand-object interactions for evaluating 3D tracking and pose estimation. *Signals:* synchronized head-mounted views, motion-capture hand/object poses, and scanned object models.	Project / data / code

Reading the Table Scales follow the named releases and official documentation checked on October 9, 2026. Trajectories, video hours, and synchronized takes are different units. OXE aggregates other datasets, so row totals are not additive. BridgeData V2 includes 50,365 teleoperated trajectories and 9,731 scripted rollouts. DexWild uses the released data-card count, which differs from the paper's 9,290 human demonstrations. Tool and human poses are action proxies requiring embodiment alignment, not native robot commands. Resource links lead to official data, code, or download instructions. Data-access and reuse terms remain specific to each release.

4.2.3 Autonomous Data Collection

Autonomous data collection should be understood as an agent actively sampling the space of possible experiences, rather than a robot repeating its current policy indefinitely. Passive production logs reflect the natural frequency of events but are often dominated by routine states. An active system can instead select tasks and initial conditions according to novelty, uncertainty, prediction error, failure density, gaps in coverage, or risk priority. If a collection policy repeatedly succeeds only in familiar regions, the sample count may grow while behavioral diversity and the boundaries of competence remain unchanged.

AutoRT illustrates an early use of foundation models to orchestrate collection. A vision-language model describes the scene, a language model proposes and filters tasks, and the system schedules their execution using existing policies or human operators (Ahn et al., 2024). Its contribution in this context is not to replace physical control with a language model, but to bring scene understanding, task discovery, safety rules, and available policy resources into a common orchestration process. A high-level model may likewise identify gaps in coverage for factories, traffic systems, mines, or laboratories, while concrete actions remain subject to control authority, real-time constraints, and safety interlocks.

A systematic study of scaling autonomous robot data collection finds that environment instrumentation, reliable object resets, success detection, and maintaining stable conditions remain major costs. In some tasks, adding a small number of human demonstrations is more effective than adding many autonomous episodes (Mirchandani et al., 2025). These findings show how autonomous collection shifts some labor from episode-by-episode teleoperation to environment interfaces, online assessment, and risk control. If that infrastructure is unreliable, scaling up may merely accumulate repetitive, mislabeled, or dangerous data.

The distinctive value of autonomous interaction is its access to states rarely covered by successful human demonstrations. PlayWorld uses autonomous play to collect contact-rich and long-tail robot-object interactions for learning action-conditioned changes in the world (Yin et al., 2026b). The lesson for the present section concerns data production. Open-ended play can complement demonstrations with failures, slips, precursors to collisions, and nonexpert trajectories. The use of such data to train generative world models is discussed in Section 4.3 and Chapter 5.

Effective active sampling must balance information gain with representativeness. A collection budget can be divided into three parts. One maintains a baseline density for states common in deployment, another targets regions where the current model is uncertain or incurs high loss, and the third explores rare risks and new tasks. A task selector should also suppress near-duplicate trajectories and avoid favoring short tasks merely because success is easy to detect. For high-risk states, constrained environments, low-speed operation, safe proxy tasks, or simulation-based prescreening should take priority over deliberately inducing accidents.

A complete autonomous episode should retain the policy version, model inputs, raw outputs, commands after safety filtering, actual execution, random seed, initial state, reset method, termination reason, success-detector version, and any human involvement. In particular, a policy-initiated abort, a safety-system intervention, a human emergency stop, and an equipment failure must be distinguished because they carry different training semantics. Continuous operation also requires monitoring for clock drift, frozen sensors, saturated actions, repeated tasks, degradation of the scene, and failed resets, so that the pipeline does not continue blindly accumulating data after the physical system has ceased to behave as intended.

Multiple robots can collect data in parallel, but the number of concurrent robots is not the only measure of efficiency. As fleets grow, task scheduling, spare parts, networks, storage, calibration, and on-site supervision become more complex. Drift and maintenance history across devices may also introduce hidden domain differences. A more mature system coordinates equipment health, task queues, risk levels, gains in coverage, and human workload, and admits each new data batch into training only after it passes a quality gate rather than adding every operating log directly to the model.

The scope of autonomous collection should be clear. This section concerns the selection and execution of interactions in the real world. Scaling simulated agents and generative scenes belongs to Section 4.3, the simulation-reality gap to Section 4.4, and the longer-term deployment and retraining flywheel to Section 4.5. Autonomy is, first, a means of collecting informative experience and, second, a means of reducing the human cost of each individual data point.

4.2.4 The Interactive Feedback Loop

The most valuable supervision often lies at the boundary of a policy's competence. Complete, successful demonstrations describe a manifold of nominal behavior, whereas autonomous deployment exposes the distribution of states induced by the current policy itself. Interactive supervision connects the two. A failed episode, the warning signs before failure, the instant of human takeover, corrective actions after takeover, and the recovery segment that returns the system to a feasible state have different learning semantics. They should not be collapsed into a single "failure" label.

The collection system should retain the policy's original actions, commands after safety filtering, the human's modifications, and the actual execution results, together with the reasons for intervention, its start and end times, and the subsequent outcome. If only the smooth trajectory after takeover is saved, a model may incorrectly attribute the human's behavior to the policy. If only the segment before failure is retained, it cannot learn how to recover. Linking warning, trigger, takeover, correction, recovery, and outcome within one event chain supports risk prediction, corrective policies, and recovery skills.

Human-in-the-Loop Sample-Efficient Robotic Reinforcement Learning (HIL-SERL) connects initial demonstrations and online human interventions with reinforcement learning on real robots, showing that human-in-the-loop data can provide continued supervision on the policy's execution

distribution, beyond offline imitation (Luo et al., 2025). Hi-ORS further combines online rejection sampling with flexible human correction, exploiting recovery segments under comparatively stable supervision targets (Lu et al., 2025). Together these works point to a shift from a single sequence of "collect, then train" to a short-cycle loop among execution, evaluation, and updating.

HELP further organizes human-in-the-loop involvement as a scarce supervision resource. Different staff members undertake multi-robot monitoring, physical resets, and high-value remote takeovers, while rollout segmentation distinguishes progress-making, idle, failure-inducing, and recovery segments (Zhai et al., 2026). This exploration suggests that a scalable loop need not assign one person to continuously control each robot. One person can monitor several robots, intervene when risk warrants it, and concentrate limited human effort on states with the greatest learning value. The results still depend on the platform and task. No particular ratio of people to robots should be treated as a universal rule.

The quality gate is an intermediate layer of the loop. Before each round of data enters training, it should undergo completeness checks, physical-consistency checks, event segmentation, review of successes and failures, near-duplicate suppression, and provenance audits. After training, improvements must also be verified on held-out scenes, objects, operators, and embodiments. A new version that improves only on the collection distribution, or raises average success at the cost of safety and recovery, does not establish an effective feedback loop.

Safety is a system variable that determines what may be collected, when to intervene, and how the resulting data may be used. Task rules, policy uncertainty, kinematic and dynamic constraints, low-level monitoring, software and hardware interlocks, and human oversight should provide protection together. New policies should progress from offline checks through constrained environments and low-speed operation without a payload before normal operation. In settings involving people, hazardous materials, public roads, or critical infrastructure, near misses, controlled perturbations, and safe proxy tasks should take priority. Long-tail data should not be sought by causing real accidents.

Interactive feedback loops also apply to Physical AI systems that do not have a single robot body. Human takeover in a traffic system may mean reverting to a previous signal plan. Recovery in a factory may mean degrading process parameters and isolating equipment. Correction in a laboratory may mean stopping a procedure, recalibrating, or changing experimental conditions. Whatever the system, its recommendations, authorization decisions, issued commands, actual process, alarms, human interventions, and final results should remain linked in the same event chain.

The efficiency of the loop should be measured by capability gains per human-hour or robot-hour, alongside effective minutes of interaction, the fraction of usable episodes, rare-event yield, takeover frequency, recovery success, gains in coverage, and safety incidents. Raw operating time is merely a proxy for cost and cannot stand in for data value. The output of a short-cycle loop consists of training batches that pass quality gates and traceable model versions. The longer-cycle flywheel of deployment, failure discovery, renewed collection, and retraining is developed in Section 4.5.

4.3 Physics Engines and Generative Worlds

Section 4.1 defines physical data as records that link observations and actions to their consequences. Among these records, failures and the recoveries that follow them are the most informative and also the hardest to obtain (Section 4.2). A policy that must recover when a grasped object starts to slip needs to experience many slips, and a legged robot that must get up after a fall needs to fall, yet physical systems cannot be reset cheaply (Section 1.1). In simulation, a slip can be staged 1,000 times, and a simulated fall causes no physical damage to robot hardware. A model computes the recorded consequences, so their physical consistency has to be tested before the data can be trusted.

Every synthetic trajectory depends on three choices. The world in which it unfolds may be authored by hand, reconstructed from scans, or generated. The consequences of each action may be computed by a physics engine or by a network trained on data. The actions may come from planners, reinforcement-learning experts, or adapted human demonstrations. The section first con-

Table 4.4 Sources of synthetic experience and what must be checked before use

Source	Role	Main benefit	Must be checked
Reconstructed scenes and digital cousins	The world: geometry and appearance taken from real sites, or variations of them	Deployment-like layouts and appearance	Collision geometry, articulation, physical parameters
Generated assets and tasks	The world and its goals	Object, scene, and task diversity	Scale, mass and inertia, joint axes, task feasibility
Physics engine	Consequences, from hand-written dynamics and contact	Privileged state, dense labels, counterfactual replay	Parameter values, missing mechanisms, solver artifacts
Learned simulator	Consequences, from a network trained on solver or real trajectories	Fast rollouts, materials that are hard to model	Extrapolation, conservation, long-horizon drift
Observation generator	Appearance only, with physics computed by an engine	Visual diversity over fixed interactions	Label preservation near contacts, thin parts, and occlusions
Video world model	Consequences and observations, learned from video	Broad visual and behavioral priors	Action grounding, physical consistency, exploitation by the policy
Demonstration generator	Actions, adapted from human seed demonstrations	Many demonstrations from few	Coverage of strategies, missing failures, controller match

siders what simulation adds to a dataset and then takes the three choices in turn. Table 4.4 summarizes the sources involved and what must be checked before their output is used. In what follows, synthetic data are any data produced by a model. Simulated data come from a physics engine, and generated data from a learned model. Most examples are drawn from manipulation and locomotion, where the evidence is richest, but the same questions arise for the simulators and digital twins of process plants, traffic systems, and laboratories.

4.3.1 What Simulation Adds to a Dataset

In the notation of Section 4.1, a simulator turns chosen actions into trajectories through two conditional distributions,

$$s_{t+1} \sim P^{\mathrm{tr}}_{\xi_{\mathrm{sim}}}(\cdot \mid s_t, a_t), \qquad o_t \sim O_{\psi_{\mathrm{render}}}(\cdot \mid s_t), \tag{4.2}$$

where the physical and numerical parameters ξ_{sim} (masses, friction coefficients, time step, solver tolerance) govern the dynamics and the observation parameters ψ_{render} (camera models, lighting, sensor noise) govern what is sensed. Delays in the real system, such as actuator response time or sensor latency, have to be represented in s_t or in an explicit history buffer. Otherwise the simulator assumes that commands act instantly and that sensors report the present. Two further ingredients complete a simulated dataset, namely the distribution of initial states and tasks from which episodes start and the predicate that decides whether an episode meets the success criterion. The dynamics, the observation model, the initial conditions, and the success predicate can each differ from reality, and Section 4.4 classifies the reality gap along these four lines.

The most visible gain is access to the full state. Object poses, contact forces, and material parameters can be recorded at every step, even though a deployed policy cannot observe them directly. This privileged information is valuable for training teachers, critics, and state estimators (Section 7.2 and Section 7.3), provided it never leaks into the deployed policy's inputs. Each synthetic field should therefore be tagged as simulator state, rendered observation, or inferred quantity.

A deeper gain is the exact and inexpensive reset, which physical systems lack. Episodes restart from saved states, thousands of environments run in parallel, and the same initial state can be replayed under different actions, friction coefficients, or disturbances to see what changes the outcome. Counterfactual replay of this kind supports curricula, failure diagnosis, and targeted generation. Its findings concern the model, however. A policy's sensitivity to a simulated friction coefficient says something about a real material only after the simulator has been checked against measurements.

Simulated episodes need the same conventions as real trajectories and the same care in task design. An action vector means different things under torque, position, and Cartesian control, so the action specification of Section 4.1 belongs with every episode. Favorable starting poses hide difficult approaches, and a reset triggered as soon as an object slips ends the episode before the recovery that a deployed policy will need to learn. Success is best defined by the physical outcome of interest, such as a plug that is fully seated or a lid that stays closed, instead of by whatever the simulator makes easy to check.

4.3.2 Scenes from Scans and from Generators

Hand-built scenes are slow to author and tend to be few and tidy, so the training data available to policies cover a narrow range of environments. Reconstruction from real scans and automatic generation both widen that slice. Reconstruction has considerably reduced the appearance gap for visual policies. Neural radiance fields and Gaussian splatting (Section 3.3) recover geometry and appearance, although collision geometry, articulation, and physical parameters must still be added before a scene can be simulated. With these additions, reconstructed scenes can largely stand in for real ones. RGB manipulation policies trained entirely on data from SplatSim, which renders a simulator's scenes with Gaussian splats, achieve 86.25% average success in the reported SplatSim experiments zero-shot on real hardware, against 97.5% for policies trained on real data (Qureshi et al., 2025). RialTo fine-tunes policies with reinforcement learning inside a reconstructed replica of the deployment scene itself, reporting an average improvement of 67% in success over baselines under perturbed object poses, distractors, and disturbances (Villasevil et al., 2024).

Some pipelines use reconstruction without simulating dynamics at all. Real2Render2Real turns a phone scan of an object and a single video of a person manipulating it into thousands of rendered robot demonstrations (Yu et al., 2025a). The object's motion is copied from the human and collision modeling is switched off, so no model computes the consequences of the robot's actions. Even so, in the reported Real2Render2Real experiments, policies trained on data rendered from one human video match policies trained on 150 teleoperated demonstrations. The approach works when the object's path alone defines the task, and it cannot handle tasks that depend on force.

Reconstruction also raises the question of how faithful a training scene needs to be. A digital twin is tied to one physical site and kept up to date with it. A digital cousin, by contrast, is a generated environment that preserves what matters for interaction in a real scene, such as where objects are and how they can be grasped or opened, without copying it exactly. In the reported digital-cousin experiments, policies trained across a set of cousins achieve 90% zero-shot success in the original real scene, whereas policies trained on a single faithful twin achieve 25% (Dai et al., 2025).

Generating scenes at scale requires assets, and a training asset must be a physical specification as well as a shape. It needs consistent scale, collision geometry, mass and inertia, joint axes and limits, and material parameters. A photorealistic cabinet with a misplaced hinge axis adds visual variety but nothing a policy can practice on. Articulation is particularly hard to produce automatically. Articulate-Anything has a vision-language model write articulation code from text, images, or video and revise it after inspecting the result. On the PartNet-Mobility benchmark, the reported Articulate-Anything evaluation shows that this raises the success rate of reconstructed articulations from 8.7 to 11.6% for earlier methods to 75% (Le et al., 2025).

Generative models can supply tasks as well as assets. GenSim and RoboGen prompt a language model to generate tasks, write the code that builds their scenes, and produce the supervision needed to learn them (Wang et al., 2024d;e). RoboCasa populates kitchens with assets from text-

to-3D and text-to-image models, and RoboCasa365 scales the approach to 365 tasks across 2,500 kitchen layouts (Nasiriany et al., 2024; 2026). A generated task is usable only if it resolves to identifiable objects, admissible actions, and a success predicate that can be checked, so generation pipelines usually execute each proposed task and discard those that a solver cannot complete.

4.3.3 Agent-Built Scenes and Simulations

The methods above use a language model for one part of a scene at a time. Agents built on language and vision-language models now assemble whole scenes, and increasingly whole simulations, by writing code, running it, and inspecting the result. Early systems focus on layout. Holodeck asks GPT-4 which objects a described room should contain and how they relate spatially, then places Objaverse assets with a constraint solver (Yang et al., 2024c). Fine-tuning on Holodeck scenes raises a navigation agent's success in unfamiliar room types, such as music rooms and gyms, from 4.1% to 20.4%, against 17.0% for scenes with the same assets in procedural layouts.

Later systems also supply physical properties. In SceneSmith, designer and critic agents build a room in stages, from the floor plan to furniture to small objects, while a vision-language model estimates each object's mass and material from rendered views and articulated furniture is drawn from a library (Pfaff et al., 2026). The generated rooms contain about 71 objects, against 11 to 23 for earlier generators, and 95.6% of the objects stay in place under simulation. SAGE uses a similar generate-and-critique loop to build scenes for a given task (Xia et al., 2026). Parameters inferred from appearance are enough to keep a scene standing, but contact-sensitive tasks still need them calibrated.

General-purpose models released in 2026 can carry out the whole real-to-sim process. In its 2026 release materials, OpenAI presents 3D world and asset generation as a main capability of GPT-6 Astra (OpenAI, 2026), and reports published within weeks of release describe several groups using it to build simulation scenes (Li et al., 2026d).[3] In Agentic RSR, a Codex agent running GPT-6 Astra turns an RGB video of a robot workspace into a MuJoCo scene with collision geometry and joints, then develops a policy program by running it there (Li et al., 2026d). Across 18 tasks on 2 robot arms, 10 succeed in simulation and 8 of those also succeed on the real robots. The authors find that the agent behaves differently across interfaces and runs, so the model version belongs in the record of any data it generates.

Benchmarks show where such agents still fall short. In Video2World, agents rebuild a simulated world and the demonstrated behavior from a video of a robot or a person performing a task (Tang et al., 2026). The best of 9 agent systems achieves 25.5% task success over 222 instances, against 58.8% for human-assisted reconstructions, and GPT-6 Astra achieves only 10.7% although it recovers object shapes more accurately than any other agent. On indoor scans, the reported LiteReality-Agent evaluation shows 24 interpenetrating furniture pairs for GPT-6 Astra and none for the specialized LiteReality-Agent pipeline (Huang et al., 2026b), and on 4DCodeBench good static reconstruction does not carry over to deformation, fluids, or fracture (Shen et al., 2026). Accurate geometry does not guarantee a scene in which the task can be performed.

Two recent systems close part of this gap by adding structure around the model. Text2Sim splits the writing of a Genesis simulation among planner, writer, and critic agents, and the writers consult Debug Cards, short notes distilled from 99 demonstrations in graphics papers on how simulations of a given kind fail and how to repair them (Xiong et al., 2026). On 42 held-out prompts, its automatic physical-quality score is 95.3, against 86.6 for direct use of its own underlying model (GPT-5.6-Sol). Without the cards, the score falls to 91.3. WorldAgent instead verifies every construction and simulation step in Blender, checking interpenetration, support, numerical stability, and causal order against scene geometry and simulator states, so it also catches overlaps hidden from the camera (Wang et al., 2026a). With GPT-6 Astra as the underlying model, WorldAgent reduces the worst interpenetration during simulation to about 1/5 of that from direct prompting.

[3] Max Bickley and Ashley Goldstein, NVIDIA Technical Blog, *How to Use AI Agents to Prepare 3D Scenes for Simulation*, September 16, 2026.

In these evaluations, neither system is checked against measured behavior, and both infer physical parameters instead of measuring them.

World generators skip the agent altogether. Marble turns text, images, or video into Gaussian splats with a collision mesh, and such worlds have been imported into MuJoCo and Isaac Sim. The mesh is a single static surface, so the robot and the objects it manipulated were added from asset libraries (World Labs Team, 2025a).[4] Genie 3 generates an interactive environment from a prompt in real time and keeps it consistent for several minutes, and DeepMind's SIMA agent has pursued goals inside such worlds.[5] A world of this kind has no explicit state or parameters to inspect, so, like the video world models discussed later in this section, it can be judged only by the consequences it predicts.

4.3.4 Contact, Fidelity, and Throughput

Once a world is built, a model must compute what happens in it, and the accuracy required depends on the task. Articulated rigid-body dynamics suffice for most legged locomotion and for pick-and-place with rigid objects, whereas cloth, cables, food, and soft tissue need deformable or continuum models. A useful test is whether the model reproduces the events that change the correct action, such as a grasp that slips, a peg that jams, or a support that gives way.

Most such events happen at contact. Section 2.1 describes contact through non-penetration and Coulomb friction, and engines differ mainly in how they enforce these two conditions. Classical time-stepping schemes pose them as a complementarity problem, in which two bodies at a contact either separate or push on each other, never both at once (Stewart and Trinkle, 1996). MuJoCo relaxes them into soft constraints and solves a convex optimization problem at each step, obtaining smooth, well-posed dynamics at the price of small interpenetrations and some compliance (Todorov et al., 2012). For many tasks the relaxation does no harm, and a calibrated compliance can even describe a rubber fingertip better than a perfectly rigid model. In tight-tolerance insertion or stacking, however, a soft contact model can let a part slide into place where the corresponding real part would bind.

Deformable objects make non-penetration harder to enforce. Incremental Potential Contact guarantees, under its algorithmic assumptions, that simulated deformable bodies never pass through each other and that mesh elements never turn inside out (Li et al., 2020). Trajectories computed with it are free of interpenetration artifacts, but the method says nothing about the stiffness, damping, or friction of a particular sponge or garment, and these values must be measured before the recorded forces and deformations can be trusted.

Numerical artifacts are a separate hazard, because reinforcement learning can favor policies that exploit them. A policy can learn to exploit interpenetration, spurious impulses from numerical jitter, or contacts that stick when they should slide, and the time step, collision geometry, solver tolerance, and constraint softness determine which of these artifacts the optimizer can find. A cheap safeguard is to replay a sample of accepted trajectories with a smaller time step, tighter tolerances, or perturbed contact parameters, and to discard behaviors whose success does not survive. The check is best run before a large dataset is produced, since an artifact that goes unnoticed is copied into thousands of examples.

Throughput determines how much variation can be explored. Isaac Gym keeps physics simulation and policy training on the same graphics processing unit (GPU) and reports speedups of two to three orders of magnitude over pipelines that simulate on the central processing unit (CPU) (Makoviychuk et al., 2021), which makes it possible to train quadruped walking policies in minutes (Rudin et al., 2022). For policies that use images, the bottleneck moves to rendering. ManiSkill3 parallelizes rendering on the GPU along with physics and reports that, with rendering enabled, it runs 10 to 1,000 times faster than other platforms on its benchmarks (Tao et al., 2025a).

[4] World Labs, *Scaling Robotic Simulation with Marble*, 2025.

[5] Jack Parker-Holder and Shlomi Fruchter, Google DeepMind, *Genie 3: A New Frontier for World Models*, August 5, 2025.

Frame rates are nonetheless a poor guide to the cost of data. They vary with scene complexity and contact density, and they leave out resets, policy inference, filtering, and storage. A better measure is the simulated counterpart of the effective data yield of Section 4.2, which counts only trajectories that pass the quality gates and weighs their cost against how much they improve the policy.

4.3.5 Learned Simulators and Video World Models

Hand-written engines are limited to what their authors can model. Learned models trade explicit physical structure for coverage of phenomena that are hard to write down, and they can replace the solver, the renderer, or both. The price is that their errors are harder to diagnose. An error in an engine can usually be traced to a parameter, a missing mechanism, or a solver setting, whereas an error in a learned model is spread across its weights. Measurements on the physical system then become the main check on its physical consistency.

Learned simulators replace the solver but keep the state-based interface of Eq. (4.2). Graph Network-based Simulators and MeshGraphNets, for example, learn how particles and meshes evolve for fluids, granular media, cloth, and deformable solids (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021). In its evaluation, MeshGraphNets runs 1 to 2 orders of magnitude faster than the solvers used to produce its training data. Such models are reliable mainly within their training regime. Over long rollouts their errors accumulate, and conservation laws hold only approximately unless they are built into the architecture (Section 2.4).

Observation generators leave the physics to an engine and replace only the renderer, changing the lighting, textures, backgrounds, and clutter that the camera sees. Cosmos-Transfer, for instance, generates realistic video conditioned on segmentation, depth, and edge maps rendered by a simulator (NVIDIA et al., 2025b). The risk is that the generator alters something the labels depend on. If it shifts a handle by a few centimeters while restyling the scene, and the simulator keeps the original pose, a correct grasp label becomes wrong. Agreement between image and simulator state therefore needs checking near contacts, around thin structures, and under occlusion, where such errors are most likely.

Video world models replace both, learning the consequences of actions directly from recordings (Chapter 5) (Bruce et al., 2024; Yang et al., 2024b; NVIDIA et al., 2025a). UniSim, an action-conditioned video simulator, learns from image, robot, and navigation data and is used to train policies that transfer to real robots. For generating data, the first question is which training data the model has received. A model trained on demonstrations rarely observes failures or unusual contact, so it tends to predict that actions succeed. Training on autonomous play, as in PlayWorld (Section 4.2), exposes a world model to the slips, collisions, and near misses that demonstrations omit (Yin et al., 2026b).

Because video models produce pixels, actions have to be recovered before their output can train a policy. DreamGen fine-tunes an image-to-video model on the target embodiment and uses it to generate videos of familiar and new tasks (Jang et al., 2025). The videos are then labeled with actions, either by an inverse-dynamics model, which infers the action taken between consecutive frames, or by a latent-action model, which infers an abstract action code. Starting from teleoperation data for a single pick-and-place task in one environment, a humanoid trained on these generated trajectories acquires 22 new behaviors in seen and unseen environments. The recovered actions are action proxies in the sense of Section 4.1, and they can be screened by replaying them in a physics simulator and keeping those whose simulated motion matches the generated video (Kim et al., 2026b). A policy can also be fine-tuned inside a world model, on imagined successes or on rewards that a vision-language model assigns to imagined rollouts (Guo et al., 2026b; Sharma et al., 2026a), at the risk of learning to exploit the model's errors (Section 5.4).

4.3.6 Synthetic Demonstrations and Pretraining Corpora

The third choice is the agent. Behavior can come from motion planners that use the simulator's exact state, from reinforcement-learning experts trained with privileged observations and then distilled into policies that use deployable sensors (Section 7.2), from programs written by language

models, or from a few human demonstrations adapted to new situations. Each source leaves its mark on the data. Planners produce smoother and more direct motions than an operator would produce, reinforcement learning may find strategies that a person would not demonstrate, and adapted demonstrations carry human habits along with human skill.

MimicGen shows how far a few demonstrations can be stretched. It splits each demonstration into segments defined relative to the objects involved, transforms the segments to new object poses, stitches them together, and keeps the results that succeed when executed in simulation. In MimicGen, about 200 human demonstrations yield more than 50,000 demonstrations across 18 tasks with varied scenes, objects, and robot arms. Policies trained on the synthesized demonstrations perform comparably to policies trained on additional human demonstrations (Mandlekar et al., 2023). Later systems extend the method to long-horizon skill sequences and to bimanual dexterous hands (Garrett et al., 2025; Jiang et al., 2025). MimicGen's output, however, recombines the strategies in the seed demonstrations without adding new ones, and because failed executions are discarded, the resulting training data provide few examples of failure or recovery.

Synthetic demonstrations should pass the same quality gates that Section 4.2 applies to real batches, along with checks specific to synthesis. These checks cover action semantics that match the deployed controller, privileged fields that are marked, and outcomes that survive the solver checks described earlier. Rejected samples are worth keeping, sorted by the reason for rejection, because different reasons call for different responses. Invalid geometry, a mismatch between the rendered observation and the simulator state, and an infeasible task all point to defects in the generator. A valid execution that ends in failure is ordinary failure data, unsuitable as an imitation target but useful for learning to predict outcomes and to recover. If generator versions, parameter distributions, solver and renderer settings, and random seeds are kept on record, a problem found later can be fixed at its source before more data are produced from it.

Synthetic data have also moved beyond narrow skills into pretraining. Simulated trajectories make up a large part of the training mixture of the GR00T N1 humanoid model, and GraspVLA, pretrained on 1 billion frames of simulated grasping, transfers zero-shot to real objects (NVIDIA et al., 2025c; Deng et al., 2025b). The most direct comparison comes from InternData-A1. In InternData-A1, a model with the π_0 architecture, pretrained only on its 630,000 simulated trajectories, matches the official π_0 on simulated, real, and long-horizon dexterous tasks. The official π_0 had been pretrained largely on real robot data (Tian et al., 2025). Results of this kind are strongest where outcomes depend mainly on geometry, as in grasping and pick-and-place. Where outcomes depend on contact dynamics, a synthetic corpus is only as useful as the calibration of the engine that produced it, and calibration requires real data.

4.4 Sim-to-Real Transfer and Real-World Alignment

A policy trained on synthetic data inherits the errors of its source, and how much those errors matter depends on the task. In legged locomotion, policies trained entirely in simulation with randomized dynamics and learned actuator models now routinely transfer without real-world fine-tuning. Manipulation is harder. Contact, deformation, and vision all differ between simulation and reality, and transfer there typically involves some real data. This section begins with what simulated and real data each contribute. It then follows the order in which the reality gap is commonly addressed. This sequence consists of finding where simulation and reality disagree, correcting the simulator with measurements, training policies that are robust to the remaining error, and checking the result in closed loop on the physical system.

4.4.1 What Simulated and Real Data Each Contribute

Choosing between simulated and real data means trading scale and label accuracy against the reliability of the dynamics (Table 4.5). Simulation offers scale, exact perception labels, and free resets (Section 4.3), but its dynamics, and therefore the consequences it computes, are only as accurate as its contact, material, and actuator models. Real data have the reverse profile. They are limited by robot time, operators, and the cost of resetting the world, and their observations are imperfect.

Table 4.5 Complementary strengths of simulated and real data

Aspect	Simulated data	Real data
Scale and cost	Millions of episodes, with cost dominated by building scenes and by compute	Limited by robots, operators, and resets, with cost growing with every episode
Perception and labels	Exact poses, contacts, masks, and depth, although rendering may look unrealistic	Sensor noise, calibration error, missing depth, with labels that must be estimated or annotated
Dynamics	Only as accurate as the contact, material, and actuator models	Real consequences, including effects that no model captures, though their causes may go unmeasured
Rare and dangerous states	Staged and replayed on demand	Rare, costly, and sometimes unsafe to collect
Main role	Breadth, privileged supervision, pretraining, stress tests	Training where dynamics matter, calibration of simulators and world models, validation

Cameras add noise and blur, depth sensors drop out on shiny surfaces, calibration drifts, and object poses or contact forces must be estimated instead of read off. Their dynamics, on the other hand, are the real ones. A real trajectory reflects the actual friction, compliance, backlash, and delay of the system, including effects that no simulator models, even without direct measurements of those effects. Neither source dominates. Simulation is most useful where exact labels and broad coverage matter and the dynamics are easy to model, while real data are hard to replace where the outcome depends on contact, deformation, or actuation that simulators approximate poorly.

Because real data are scarce, how they are spent matters as much as how much is collected. They serve three purposes, namely training policies directly, calibrating the simulators and world models that generate synthetic data, and validating both the generators and the policies trained on their output. The three purposes compete for the same evidence, and evidence spent on one cannot serve as an independent check on another. Trajectories used to fit a simulator cannot show that the fit is accurate, and trials used to select a policy cannot certify it.

4.4.2 Where the Reality Gap Comes from

The four ingredients of a simulated dataset listed in Section 4.3 also give a natural way to classify the reality gap (Table 4.6). A dynamics gap means that $P^{\mathrm{tr}}_{\tilde{\zeta}_{\mathrm{sim}}}$ in Eq. (4.2) is wrong, either because its parameters are off or because it lacks a mechanism. Errors in mass, inertia, friction, and compliance alter accelerations and contact outcomes, and actuators add discrepancies of their own through gains, torque limits, backlash, and delay. An observation gap means that $O_{\psi_{\mathrm{render}}}$ misrepresents what the sensors report, whether in appearance, camera calibration, depth dropouts on shiny or transparent surfaces, proprioceptive bias, or timing. Touch is a particular weak point, since many tactile sensors respond through elastomer deformation and internal optics that simulators capture only roughly. The remaining two gaps are easier to miss. A distribution gap arises when training does not cover the initial states, objects, and tasks the robot actually meets. An objective gap arises when simulated success is not real success, as when a success predicate credits an insertion that reaches the right depth but would not hold in a real fixture.

Each gap calls for a different remedy, so the gap has to be identified before anything is changed. A delayed command is fixed by calibrating the interface, an unmodeled deformation needs a better model, and an action that is executed correctly but is wrong for the task needs new supervision. Telling these cases apart requires the commanded and executed actions, events, and outcomes in the trajectory record (Section 4.1), which a binary success flag does not provide.

Table 4.6 Components of the reality gap and how they are diagnosed and addressed

Gap	Typical symptom	Diagnostic	Main remedies
Dynamics ($P^{\mathrm{tr}}_{\xi_{\mathrm{sim}}}$)	Overshoot, sluggish tracking, unexpected slip or jamming, wrong deformation	Replay identical command sequences in simulation and on hardware and compare the responses	System identification, actuator or residual models, dynamics randomization
Observation ($O_{\psi_{\mathrm{render}}}$)	A correct plan executed at the wrong place or time, and failures under particular lighting or materials	Compare sensor streams for matched states and measure end-to-end latency	Calibration, latency modeling, sensor noise models, visual randomization
Distribution (initial states and tasks)	Failures concentrated on unseen objects, layouts, or initial states	Stratify evaluation by condition	Targeted real or synthetic collection, broader scene and task generation
Objective (success predicate)	Episodes counted as successes in simulation that fail in reality	Audit success predicates against physical outcomes	Redefine success and reward and relabel data

A gap of any kind is amplified in closed loop. An early mistake moves the robot into a state that no demonstration covered, the policy's next action there is worse, and errors snowball. For imitation learning, this intuition has a precise form. With per-step task costs bounded in $[0, 1]$, a cloned policy with error probability ϵ_{BC} under the expert state distribution can incur an excess expected cost of order $T^2\epsilon_{\mathrm{BC}}$ over T steps (Ross and Bagnell, 2010). Dataset Aggregation (DAgger) collects expert labels on states visited during rollouts that may mix expert and learner policies. Under the assumptions of its analysis and with sufficiently many iterations, some learned iterate has excess expected task cost relative to the expert bounded by $u_{\mathrm{dev}} T \epsilon_{\mathrm{DAgger},N_{\mathrm{iter}}} + O(1)$. Here $\epsilon_{\mathrm{DAgger},N_{\mathrm{iter}}}$ is the minimum average surrogate loss achievable within the policy class over the state distributions induced by the rollout policies across N_{iter} iterations, and u_{dev} bounds the increase in the expert's remaining cost after one deviating action (Ross et al., 2011). The constant u_{dev} is a measure of recoverability, the property that Section 1.1 argues planning must take into account when resets are not free.

The analysis assumes access to an expert policy that can label the states visited during data collection. A privileged teacher in simulation can supply such labels when its policy is defined on those states, and teacher-student distillation usually labels the states that the student visits. Human operators find it hard to say what the robot should have done in a state they are not controlling, which is why intervention-based collection (Section 4.2) lets them take over when the policy goes wrong. Either way, training data must include the states that the deployed policy actually reaches.

4.4.3 Calibrating Simulators and World Models with Real Data

Calibration adjusts a model's parameters until it reproduces real measurements. With $\mathcal{D}_{\mathrm{r}}$ denoting a set of real trajectories, system identification can be written as Bayesian inference over the simulator parameters,

$$p(\xi_{\mathrm{sim}} \mid \mathcal{D}_{\mathrm{r}}) \propto p(\mathcal{D}_{\mathrm{r}} \mid \xi_{\mathrm{sim}})\, p(\xi_{\mathrm{sim}}), \tag{4.3}$$

where the prior $p(\xi_{\mathrm{sim}})$ encodes datasheet values and physical bounds and the likelihood measures how well the simulator with parameters ξ_{sim} reproduces the recorded observations. When states are only partially observed, the likelihood must also account for state and sensor uncertainty. A point estimate is often enough in practice, but the full posterior keeps track of ambiguities that matter later, for instance when randomization ranges are chosen.

The data used for identification must be informative. As Section 2.2 explains, accurate tracking alone does not imply parameter convergence. Persistent excitation of the regressor supports pa-

rameter convergence under the assumptions of the specified adaptive law. Identification likewise requires data that distinguish the relevant parameter directions. Slow motions in free space reveal little about actuator bandwidth, grasps that never slip reveal little about sliding friction, and different combinations of motor strength, payload, and gain can produce nearly identical trajectories. Calibration experiments should therefore excite the mechanisms the task depends on, for example with fast reversals for actuator response, controlled pushes for friction, and loaded lifts for inertia, while staying within safe limits.

Where first-principles models fall short, models learned from data often do better. Learned actuator networks, which map commanded positions and recent history to delivered torque, are central in the early legged-transfer study (Section 7.2) (Hwangbo et al., 2019). ASAP applies data-driven correction to a whole humanoid. A motion-tracking policy trained in simulation is run on the real robot, and the recorded rollouts train a delta-action model, a network that adds a correction to each action inside the simulator so that simulated motion matches the recorded real-robot motion. The policy is then fine-tuned in the corrected simulator. In tests of transfer between two simulators and deployment on a real Unitree G1, the reported ASAP evaluation shows that the approach tracks agile whole-body motions more accurately than system identification, domain randomization, or a learned correction to the simulator's dynamics (He et al., 2025a).

Learned corrections can, however, fit the data for the wrong reason. A correction trained on one set of motions may absorb a calibration error elsewhere in the system, and testing at speeds, loads, and contact points outside the fitting data exposes such errors. The same tests separate parameter errors, which calibration can fix, from structural ones, which it cannot. No value of ξ_{sim} repairs a model that omits cable twisting or a compliant gripper pad.

Nor is a single best-fit simulator always necessary. SimOpt instead calibrates a distribution over parameters. It adjusts the distribution so that simulated rollouts of the current policy match real ones, retrains the policy under the updated distribution, and repeats. Policies trained in this way transfer to real swing-peg-in-hole and drawer-opening tasks (Chebotar et al., 2019). A simulator fitted to one policy's behavior can remain wrong in regions that policy never visits, so calibration data should also cover the behaviors that later policies are likely to try.

Learned simulators can be calibrated more directly, because their parameters can be updated by gradient descent on real data. NeRD replaces the dynamics and contact solvers of a conventional simulator with a learned model that can be fine-tuned on real trajectories (Xu et al., 2025b). For video world models, VLAW alternates between improving the model and improving the policy. Real rollouts of a vision-language-action policy refine the world model, and the refined model generates rollouts for training the policy (Guo et al., 2026a). On a real robot, the authors report that the method raises success by 39.2 percentage points over the pretrained policy and by 11.6% over fine-tuning on the same real rollouts filtered for success. In that evaluation, the refined world model thus adds value beyond the real data used to build it. Establishing the accuracy of the model would still require fresh trajectories from conditions outside the fitting data.

4.4.4 Randomization, Adaptation, and Co-Training

Calibration narrows the uncertainty that measurements can resolve, and the policy must tolerate whatever remains. Domain randomization trains it to do so across a distribution $p_{\eta_{\text{sim}}}(\xi_{\text{sim}})$ over simulator parameters, shaped by design parameters η_{sim},

$$\max_{\theta} \; \mathbb{E}_{\xi_{\text{sim}} \sim p_{\eta_{\text{sim}}}} \left[J(\pi_\theta; \xi_{\text{sim}}) \right], \tag{4.4}$$

where $J(\pi_\theta; \xi_{\text{sim}})$ is the expected return of the policy in the world with parameters ξ_{sim}. Training across varied models aims to reduce reliance on any one exact model, and randomizing appearance and dynamics in this way underlies much of sim-to-real transfer in perception and control (Tobin et al., 2017; Peng et al., 2018b).

Choosing $p_{\eta_{\text{sim}}}$ in Eq. (4.4) is a data-design problem. Parameters should be resampled on the timescale at which they actually vary. An object's mass is fixed within an episode, sensor noise

changes with every reading, and actuator response drifts slowly with temperature and wear. Dependencies should be explicit, since appearance and friction are correlated for a given material.

Wide ranges are not enough by themselves. A range that contains the true value is of little use if the distribution places almost no probability near it, and ranges that are too broad produce cautious policies without fixing the failures seen in deployment. Moderate variation around a well-chosen nominal value often helps more than pinning each parameter to a single estimate, as the digital-cousin result of Section 4.3 suggests, and Section 4.5 reports a similar effect of diversity in real data. The posterior in Eq. (4.3) is a natural source for both the nominal values and their spread, and evaluation should report performance for groups of conditions as well as the average over the distribution.

Automatic domain randomization automates part of this design by widening each range as the policy meets the performance criterion within the current one, which turns randomization into a curriculum. OpenAI uses it to train a Shadow Dexterous Hand to solve a Rubik's cube. In ten reported trials, the hand solves cubes needing 15 face rotations 6 times and solves the hardest cubes (which need 26 face rotations) 2 times (OpenAI et al., 2019). The result also depends on a cube instrumented with sensors, a classical solver that chooses the sequence of face rotations, and months of large-scale training.

Adaptive policies deal with the remaining uncertainty differently. Instead of performing well under every condition, they infer the current one from recent history and adjust to it, as in the teacher-student and rapid motor adaptation methods of Section 7.2 (Kumar et al., 2021). Training them requires data that contain changes and transients, such as a payload added in the middle of a task or the onset of slip, as well as conditions that are unknown but fixed. When adaptive rollouts are later reused for training, the inferred context should be stored with each example, because an action that is correct for a heavy payload becomes a wrong label once the payload information is dropped.

In vision-based manipulation, where both the visual and the physical gap are large, a common alternative is to train on simulated and real data at once and let each cover the other's weaknesses (Table 4.5). Co-training minimizes a weighted sum of losses over the two sources, the mixture objective of Eq. (9.1) with one simulated and one real component. Across tasks on a robot arm and a humanoid, the reported co-training study shows that adding simulation data improves real-world performance by an average of 38%, and the benefit persists when the real dataset grows to 400 demonstrations (Maddukuri et al., 2025). A controlled study of planar pushing from pixels examines when and why the benefit appears (Wei et al., 2025). In that study, simulation data raise success two- to sevenfold when real data are scarce, but the gains plateau unless real data grow too. Narrowing the physics gap matters more than improving visual fidelity. Unexpectedly, the best policies can distinguish simulated inputs from real ones, and a modest visual difference between the domains helps. A mechanistic analysis attributes this to a balance between aligning representations across domains, so that knowledge transfers, and keeping the domains distinguishable, so that the policy can apply the right dynamics to each (Lei et al., 2026).

The same pushing study finds that mixing the sources during training outperforms pretraining in simulation and then fine-tuning on real data, and that fine-tuning a co-trained policy does not consistently help. These comparisons involve single tasks and modest datasets. Whether broad multi-task pretraining followed by fine-tuning helps is a separate question, taken up in Section 4.5.

4.4.5 Validating Transfer and Closing the Loop

Most of the evaluation that guides development happens away from hardware, yet the criterion remains closed-loop performance on the physical system (Section 1.1), so every proxy evaluator has to be checked against it. SIMPLER builds simulated counterparts of real manipulation setups, matching camera views, appearance, and control. It compares policy rankings in simulation with their rankings on real robots (Li et al., 2025b). Agreement is measured by the Pearson correlation between simulated and real success rates and by the mean maximum rank violation (MMRV), which penalizes the simulator for misordering two policies in proportion to the gap between their real success rates. By these measures, simulated evaluation tracks real results far better than offline

metrics such as the error of predicted actions on held-out demonstrations. World models can serve the same purpose. In WorldGym, success rates measured inside an action-conditioned video model correlate with real success at $r = 0.78$ on tasks from the BridgeData V2 tabletop setup (Quevedo et al., 2026).

Simulated and real trials can also be combined statistically instead of merely compared. SureSim treats the problem as prediction-powered inference. A small number of paired real and simulated trials estimate the simulator's bias, and that estimate corrects a large number of simulated trials (Badithela et al., 2025). The result is a valid confidence interval on real performance, which raw simulated success rates cannot provide, and an interval of similar width needs 20 to 25% fewer hardware trials than one estimated from hardware trials alone. Like any agreement between a proxy and reality, the correction holds for the policies and conditions for which it is measured and must be rechecked when either changes.

Consider a fleet of mobile manipulators that open drawers in office kitchens. At one site, pulls repeatedly stall after an approach and grasp that look correct in the logs. Engineers rebuild the scene in simulation from the logs and a scan, and the simulated robot reproduces the approach but opens the drawer without difficulty. Since the approach is reproduced and the stall is not, the discrepancy most likely lies in the simulated drawer rather than in perception or in the grasp. Pulls with a force-sensing gripper reveal a soft-close damper and a detent that the simulated drawer lacks. Once the damper and detent are modeled and their parameters fitted (Eq. (4.3)), randomization is restricted to damping and detent force around the identified values, and new scenes vary handle position and pull direction near the failure. The retrained policy is screened in simulation and then tested on held-out drawers at that site and at others, to confirm that the fix generalizes beyond the cabinet where the failure occurs.

The example also shows the many points at which real data enter a pipeline nominally trained in simulation, and claims of transfer should account for all of them. "Zero-shot" usually means that the policy was not fine-tuned on the target embodiment, yet real data may still have been used to identify parameters, reconstruct scenes, pretrain perception, or select checkpoints. Reporting these uses makes the contribution of simulation measurable. It also sharpens a question that co-training raises and that Section 4.5 takes up. The question concerns how many synthetic trajectories one real trajectory is worth.

4.5 Scaling Laws and the Data Flywheel for Physical AI

Language-model scaling studies show predictable improvements as data, parameters, and compute grow together, and progress in vision-language-action models now depends mainly on data (Section 6.2). Whether physical data will yield equally predictable gains is far from clear. Every trajectory comes from a particular body in a particular world and costs robot time and usually human time. The practical question concerns allocation. It covers which data to collect or generate and in what proportions. It also covers how to divide scarce physical interaction among training, calibration, and validation. A further aspect is when a robot system can collect what it needs through its own operation. Such a system, a data flywheel, is worth running only if the capability it adds outweighs its cost. Throughout, value is measured by closed-loop performance on the physical system, not by loss, frame counts, or simulated success.

4.5.1 The Data Pyramid

Section 4.2 describes human data as a gradient from rich semantics to executable actions. Figure 4.2 extends that gradient to all physically collected data and adds the dimension of scale, with layers widening toward the base as cost per sample falls. The trajectory record of Section 4.1 gives a way to judge relevance. Data are most relevant when they record the target embodiment issuing executable commands under deployment conditions, together with the measured consequences of those commands. Each layer below the apex gives up part of this record. Handheld and wearable devices capture real contact and robot-like actions without a robot, human video captures real physics without robot actions, and web data supply semantics without physical interaction. The

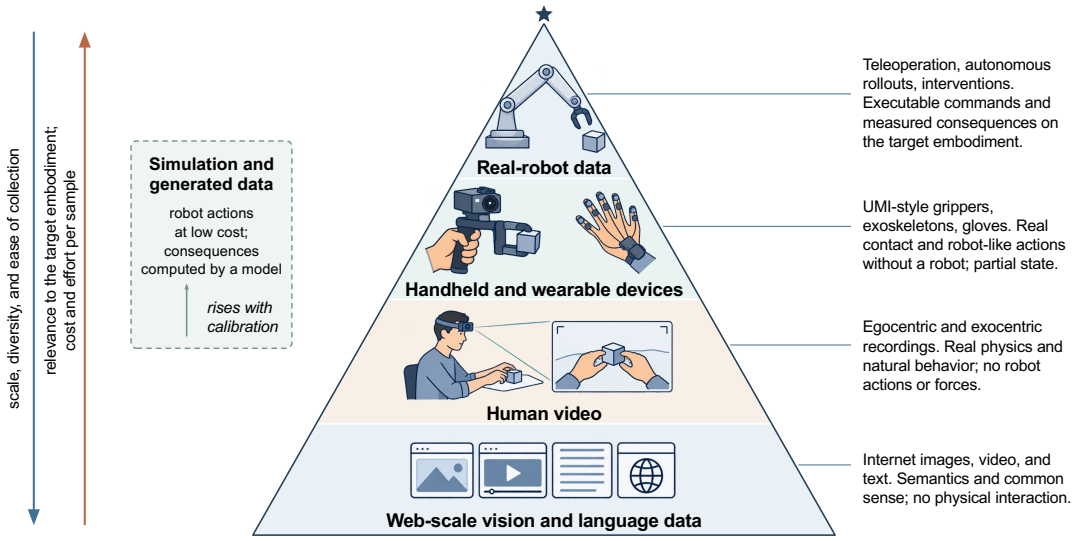


Fig. 4.2 The data pyramid for Physical AI. For physically collected data, relevance to the target embodiment and cost per sample rise together toward the apex, where the target embodiment acts in the physical world. Lower layers are cheaper, larger, and more diverse but supervise physical execution less directly. Simulated and generated data are drawn separately because they supply robot actions at low cost while computing their consequences with a model, so their effective position depends on calibration. Adapted from NVIDIA et al. (2025c) and Ye et al. (2026b). The order of the layers reflects relevance and cost, not dataset quality

most relevant data are also the most expensive, since every hour of them needs a robot, an operator, and resets, and they are correspondingly scarce. Goldberg estimates that the largest teleoperation datasets amount to about a year of robot experience, whereas the text and images used to train vision-language models would take a person some 100,000 years to read and view (Goldberg, 2025).

Simulation does not fit this ordering, which is why the figure draws it separately. It supplies executable actions on a model of the target embodiment at almost no marginal cost. The reported GR00T N1 study produces 780,000 simulated trajectories, equivalent to about 6,500 hours of human demonstration, in 11 hours (NVIDIA et al., 2025c). Because its consequences are computed, its relevance depends on calibration (Section 4.4), and simulated corpora have so far done best where geometry decides the outcome (Section 4.3). Section 9.2 places simulation alongside handheld devices in a related pyramid organized by the kind of evidence each source provides.

A capable system needs both ends of the pyramid. The apex provides executable actions and real consequences on the system's own body. The base provides diversity. Chen and colleagues argue that diversity sets the ceiling on how far a model can generalize and that, for now, only internet-scale data supply enough of it (Chen et al., 2026b). Much of this chapter concerns lowering the cost of relevant data and raising the value of cheap data through calibration, retargeting, and filtering.

Within the physically collected layers, the collection method largely determines cost and yield. Section 4.2 describes how each method works. The paragraphs below compare their economics, including what an hour of collection costs, how much it yields, and what the resulting data lack.

Egocentric video is the cheapest to collect and the easiest to scale, since it needs only a head-mounted camera or glasses. Its drawback is the embodiment gap, since human hands differ from robot grippers and video records neither commands nor forces. The gap can be narrowed by design. If the robot is built to resemble the human and both are recorded with the same sensors, human data can be worth more per hour than robot data. In EgoMimic, the reported evaluation shows that an hour of egocentric recording yields about 1,400 demonstrations against 135 from tele-

operation, and adding an hour of human data improves the co-trained policy more than adding an hour of robot data (Kareer et al., 2025).

A handheld gripper costs a few hundred dollars and needs no robot. Using UMI, 3 people collect 1,400 demonstrations across 30 locations in 12 person-hours, and on 1 task collection runs more than 3 times faster than teleoperation with a 3D mouse (Chi et al., 2024). Because the fingers match the robot's, gripper-level actions transfer directly. The arm itself goes unrecorded, so its joint states and dynamics are missing and reachability has to be checked at deployment. Forces are also absent unless tactile sensors are added.

Teleoperation yields the most relevant data and the least of it. The hardware has become cheaper. A kinematically matched leader arm costs under $300, and a complete bimanual system, robots included, costs about $20,000 (Wu et al., 2024b; Zhao et al., 2023). A robot and an operator, however, remain in the loop for every hour. The reported DROID data-collection effort involves 50 collectors at 13 institutions gathering 76,000 trajectories over one year (Khazatsky et al., 2024). As Section 4.2 notes, the data also carry the imprint of the interface, since latency, limited haptic feedback, and operator style shape the trajectories.

Exoskeletons and hand trackers fall between the handheld gripper and teleoperation, both in cost and in how closely they match the robot. An exoskeleton with the same kinematics as the target arm records joint-level motion directly. AirExo-2 costs about $600 for a dual-arm rig, and in the reported user study its operators complete a task in 5.7 s on average against 17.3 s with joint-space teleoperation (Fang et al., 2025). Gloves and motion-capture systems extend joint-level recording to dexterous hands, and the DexCap study reports collection 3 times faster than teleoperation (Wang et al., 2024a). In exchange, the operator has to wear the rig, the data must be retargeted whenever the human and robot hands differ, and force feedback is limited.

Across the four methods, the cheaper the source, the further its data sit from the target embodiment. How much each source is worth for a given task has to be measured. Scaling studies are the usual instrument, but in their standard form they rest on assumptions that physical data do not meet.

4.5.2 Scaling Laws and What They Assume

Scaling laws come from language modeling. Kaplan et al. (2020) find that the test loss of language models falls as a power law in model size, dataset size, and training compute, with some trends spanning more than seven orders of magnitude. A common form separates the contributions of the parameter count N_{par} and the amount of training data N_{data},

$$L_{\text{scale}}(N_{\text{par}}, N_{\text{data}}) = L_{\text{floor}} + a_{\text{scale}} N_{\text{par}}^{-\alpha_{\text{scale}}} + b_{\text{scale}} N_{\text{data}}^{-\beta_{\text{scale}}}, \qquad a_{\text{scale}}, b_{\text{scale}}, \alpha_{\text{scale}}, \beta_{\text{scale}} > 0, \quad (4.5)$$

where L_{scale} is a held-out loss and L_{floor} an irreducible floor. With compute $C_{\text{train}} \approx 6 N_{\text{par}} N_{\text{data}}$, about six floating-point operations per parameter per training token, minimizing Eq. (4.5) at fixed C_{train} makes the optimal N_{par} and N_{data} grow as powers of C_{train}. The exponents prove harder to pin down than the form. The original study concludes that most additional compute should go into parameters, whereas Hoffmann et al. (2022) find that parameters and data should grow in equal proportion as compute increases. Later work traces most of the difference to how parameters and compute are counted and to optimizer settings (Pearce and Song, 2024; Porian et al., 2024). An exponent, in other words, reflects the measurement protocol as well as the model family.

The allocation argument also rests on the three assumptions that every unit of data is equally informative, that compute is the only cost, and that lower loss means a better model. Physical data strain all three. Robot trajectories are highly correlated, so much of a robot dataset behaves like repeated data. Even in language modeling, repeating data for more than about 4 epochs yields rapidly diminishing returns (Muennighoff et al., 2023). Collecting robot data consumes robot time, operators, and resets that never appear in C_{train}. Section 12.1 calls this resource physical interaction capacity and argues that it must grow along with investment in compute. Held-out loss, finally, is a weak guide to closed-loop success. In the robomimic study, the checkpoint with the lowest validation loss performs significantly worse than the checkpoint with the highest rollout success (Mandlekar et al., 2022).

Table 4.7 Selected evidence on scaling in robot learning and related domains

Study	Varied	Setting	Measured	Finding and caveat
Lin et al. (2025)	Environments, objects, demonstrations	Single-task imitation with handheld-gripper data	Real success in new environments with new objects	Rough power law in environments and objects, demonstrations saturate, single tasks only
Ai et al. (2025)	Number of training embodiments	Locomotion on about 1,000 generated bodies	Performance on unseen bodies	Adding embodiments beats adding data on fixed ones, locomotion only
Pearce et al. (2025)	Model size, data, compute	Pretraining of imitation policies and world models	Pretraining loss	Language-like power laws, with coefficients that depend on tokenizer, task, and architecture
Baniodeh et al. (2025)	Compute, model size, data	Motion forecasting and planning on 500,000 hours of driving	Loss, open- and closed-loop metrics	Power law in compute, closed-loop metrics also improve, optimal model size grows 1.5 times as fast as data
Barreiros et al. (2026)	Pretraining data and tasks	Multitask dexterous manipulation	Blind real and simulated success after fine-tuning	3 to 5 times less task data needed, mixed results without fine-tuning
Generalist AI (2025)	Pretraining data, model size	Over 270,000 hours of real manipulation data	Validation loss and action-prediction error	Power law in data, small models stop improving, company blog, not peer reviewed
Xu et al. (2026)	Required precision	Simulated high-precision manipulation	Demonstrations needed for a fixed success rate	$\log N_{\text{demo}} = m_{\text{fit}}/(P_{\text{tol}} - c_{\text{lim}}) + n_{\text{fit}}$ for $P_{\text{tol}} > c_{\text{lim}}$. The fitted limit depends on sensing and expert quality.

4.5.3 What Robot-Learning Studies Show

Evidence for scaling in robot learning accumulates, but controlled studies remain rare. The cited meta-analysis of 327 papers finds that robot performance generally improves as a power law in data, model size, and compute, and that it improves faster than performance on language tasks (Sartor and Thompson, 2025). Because meta-analyses average over very different tasks and metrics, the clearest evidence comes from studies that vary one factor at a time, summarized in Table 4.7. Some of these studies measure loss rather than success. Pretraining studies of imitation policies and world models, for example, find language-like power laws in loss, with coefficients that depend on the tokenizer, task, and architecture (Pearce et al., 2025).

The most consistent finding is that diversity matters more than volume. In a study of single-task imitation with handheld grippers, generalization to new environments and objects follows a rough power law in the number of environments and objects seen in training, while extra demonstrations in environments already covered stop helping beyond a threshold (Lin et al., 2025). Guided by this finding, the study reports that 4 people collect enough data in one afternoon to train policies that succeed about 90% of the time on 2 tasks in new environments with unseen objects. Locomotion shows the same pattern across bodies. Adding procedurally generated embodiments to training improves generalization to unseen ones substantially more than adding data for a fixed set, and the best policy transfers zero-shot to real Unitree Go2 and H1 robots (Ai et al., 2025).

Broad pretraining pays off chiefly by making new tasks cheaper to learn. In the Toyota Research Institute's study of large behavior models, evaluated in blind real and simulated trials, fine-tuning

a pretrained model matches single-task training with 3 to 5 times less task data, and the advantage grows under distribution shift (Barreiros et al., 2026). Without fine-tuning, the pretrained model gives mixed results. Pretraining may also extend what a model can handle. In the first training phase of $\pi_{0.5}$, 97.6% of the examples come from sources other than mobile manipulators working in homes, such as other robots and web data, yet the reported model controls mobile manipulators in homes that had not appeared in its training data (Physical Intelligence et al., 2025).

The split of additional compute between model size and data has been measured directly in at least one physical domain. In motion forecasting and planning for driving, closed-loop metrics improve along with the loss as compute increases, and the compute-optimal model size grows 1.5 times as fast as the dataset (Baniodeh et al., 2025). Driving logs accumulate passively across a fleet and are far cheaper than manipulation data, so the result does not carry over directly, but it shows that the measurement can be made in a physical setting.

Industry now reports manipulation datasets far larger than academic ones. In a blog post, Generalist AI describes pretraining on more than 270,000 hours of real-world manipulation data (Generalist AI, 2025). It reports that the post-training validation error falls as a power law in the amount of pretraining data, and that roughly billion-parameter models stop improving with more data while seven-billion-parameter and larger models keep improving. These are prediction errors on the company's own tasks rather than closed-loop success rates, and they have not been peer reviewed.

Precision imposes a different kind of limit. In the cited simulation study, the demonstration count N_{demo} needed for a fixed success rate is modeled by the empirical fit $\log N_{\text{demo}} = m_{\text{fit}}/(P_{\text{tol}} - c_{\text{lim}}) + n_{\text{fit}}$, with fitted coefficients $m_{\text{fit}} > 0$ and n_{fit}. Here P_{tol} is the length tolerance and c_{lim} the fitted system limit. For $P_{\text{tol}} > c_{\text{lim}}$, this fit predicts that N_{demo} diverges as P_{tol} approaches c_{lim} from above (Xu et al., 2026). The limit depends on the whole system, and adding a wrist camera or using a better expert moves it toward finer precision. For such tasks, better sensing can pay off more than additional demonstrations.

All such curves are hard to measure precisely. Real-robot success rates are estimated from tens or hundreds of trials, so each point carries a wide confidence interval (Section 9.5) and a fitted exponent is loosely constrained. With that caveat, the evidence supports consistent gains from diversity across environments, objects, embodiments, and data sources, and from pretraining followed by fine-tuning. It does not yet establish universal exponents, compute-optimal model sizes for robot policies, or a dependable link between loss and closed-loop success.

Better evidence requires purpose-built experiments that vary the quantity and the diversity of data independently while keeping the training procedure fixed. Such experiments would report new trajectories apart from extra passes over old ones, evaluate on held-out conditions that stay the same between runs, and check predictions made from small runs against a larger run held in reserve. A plateau in such an experiment deserves attention in its own right, since it may point to missing sensing, an inadequate model, or an unsuitable objective instead of a shortage of data.

4.5.4 Valuing and Curating Heterogeneous Data

Raw volume overstates what a dataset contains. Experience is nested, with frames inside trajectories, trajectories inside tasks and scenes, and scenes inside embodiments and sites, so counts of hours and frames treat many correlated samples as independent. This is why Section 4.2 measures collection by effective data yield, and scaling studies should likewise report growth in independent interactions and their spread over tasks, scenes, objects, and embodiments.

How many samples from one layer are worth one sample from another can be measured in specific settings. In the InternData-A1 study (Section 4.3), fewer than 1,600 simulated samples match the performance of 200 real ones on 4 tasks with closely aligned simulated and real setups, a ratio below 8:1 (Tian et al., 2025). EgoMimic's hour-for-hour comparison, described above, gives a corresponding rate for human data. Both rates depend on how closely the source matches the task, on how much contact is involved, and on how the sources are weighted in training, and neither should be expected to hold elsewhere.

Table 4.8 Selected hubs and large public datasets for robot learning, with access conditions as of October 2026

Resource	Contents	Access	Link
Hubs and indexes of many datasets			
LeRobot hub	Community robot datasets in the LeRobot format, numbering in the tens of thousands	Licenses set per dataset, with some gated	`huggingface.co/datasets?other=LeRobot`
Open X-Embodiment	60 datasets from 34 labs, 22 embodiments, over 1 million trajectories in RLDS format (Open X-Embodiment Collaboration, 2024)	Open	`robotics-transformer-x.github.io`
ARIO	About three million episodes from 258 series and 321,064 tasks in a unified format (Wang et al., 2024f)	Open, with original licenses retained	`imaei.github.io/project_pages/ario`
TensorFlow Datasets (TFDS) catalog	Robotics section with RLDS versions of many robot datasets	Open	`www.tensorflow.org/datasets/catalog/overview`
OpenDataLab	Dataset platform hosting AgiBot World among other collections	Registration	`opendatalab.com`
Awesome Embodied Data Pyramid	Curated list of datasets organized by pyramid layer (Ye et al., 2026b)	Open	`github.com/worldbench/awesome-embodied-data-pyramid`
Large single-source datasets			
AgiBot World	About 1 million trajectories over 217 tasks from 100 robots (AgiBot-World-Contributors et al., 2025)	Gated, CC BY-NC-SA 4.0	`agibot-world.com`
DROID	About 76,000 Franka manipulation trajectories from 564 scenes (Khazatsky et al., 2024)	Open download	`droid-dataset.github.io`
RoboMIND	107,000 trajectories on four embodiments and 479 tasks, including failures (Wu et al., 2025b)	Gated, Apache 2.0	`x-humanoid-robomind.github.io`
RH20T	Over 110,000 contact-rich sequences with force, audio, and tactile data (Fang et al., 2024b)	CC BY-SA 4.0 or non-commercial subsets	`rh20t.github.io`
BridgeData V2	60,096 trajectories in 24 environments on a low-cost arm (Walke et al., 2023)	CC BY 4.0	`rail-berkeley.github.io/bridgedata`
Ego4D	3,670 hours of egocentric video (Grauman et al., 2022)	License agreement	`ego4d-data.org`
EgoDex	829 hours of egocentric video with tracked 3D hand poses (Hoque et al., 2026)	CC BY-NC-ND	`github.com/apple/ml-egodex`

Curation can matter as much as collection. CUPID uses influence functions to estimate how a trained policy's closed-loop return would change if a given demonstration were removed, and ranks every demonstration by its effect on performance (Agia et al., 2025a). On the simulated robomimic benchmark, training on less than a third of the data, selected in this way, produces diffusion policies that match the state of the art. Hardware experiments show similar gains. Related methods weight whole datasets, as Re-Mix does for Open X-Embodiment by choosing mixture weights that maximize worst-case performance across its constituent datasets (Hejna et al., 2025a), or score demonstrations by the mutual information between states and actions (Hejna et al., 2025b). A scaling curve plotted against raw demonstration counts can therefore partly reflect how much harmful or redundant data a collection contains.

Knowing which kinds of diversity matter helps both collection and retrieval. A study using procedurally generated datasets with controlled composition identifies camera poses and the spatial arrangement of objects as the dimensions most relevant in that study, both for diversifying new collections and for retrieving relevant demonstrations from existing datasets. Retrieval guided by these findings raises the real-world performance of policies trained with DROID data by up to 70% (Saxena et al., 2025).

Retrieval and curation presuppose that data can be found. A growing number of hubs collect robot datasets, and Table 4.8 lists some of the largest. Aggregators such as the LeRobot hub, Open X-Embodiment, and ARIO gather datasets in common formats, making cross-dataset training and retrieval practical, while large single-source collections offer consistent protocols at scale. Licenses and access conditions range from open to gated and non-commercial, and, as the documentation practices of Section 4.1 require, they should travel with any dataset derived from these sources.

4.5.5 Building a Data Flywheel

Flywheels have worked best where success can be detected cheaply, because the robot can then label its own experience. Large-scale autonomous collection on real robots therefore focuses on grasping in early studies, where a robot can tell by itself whether it is holding the object. One early study reports more than 800,000 grasp attempts on up to 14 robots (Levine et al., 2018), and the QT-Opt study uses more than 580,000 attempts to train a vision-based Q-function that grasps unseen objects with 96% success (Kalashnikov et al., 2018). Where success is harder to judge, people or learned models have to supply the labels. Sirius-Fleet uses a visual world model to predict whether a robot's upcoming actions are likely to fail and asks a human to intervene when they are. As the policy improves, the failure predictor adapts and requests for help become rarer (Liu et al., 2025a).

Post-training has since moved to the fleet level. RECAP, used to train $\pi^*_{0.6}$, combines demonstrations, autonomous experience, and expert corrections in batched offline reinforcement learning (Section 6.3) (Amin et al., 2026). SOP streams on-policy experience and intervention signals from a fleet to a central learner that runs either interactive imitation, in which humans correct the robot during execution, or RECAP (Pan et al., 2026). In its experiments, the wall-clock time needed to reach 80% success falls from about 174 minutes with 1 robot to 127 with 2 and 72 with 4, a 2.4-fold speedup from 4 times as many robots. The human-in-the-loop methods of Section 4.2 are the short-cycle version of the same idea.

One iteration of such a loop can be written so that the three uses of real data become explicit. Let $\mathcal{D}_k$ be the accumulated training data at iteration k, π_k the current policy, W_k^{sim} the simulator or world model, and $\mathcal{E}_{\text{eval}}$ a held-out evaluation set collected under conditions that training never sees. Then

$$\begin{aligned} \mathcal{B}_k^{\text{data}} &= \text{Collect}(\pi_k, \mathcal{A}_k^{\text{acq}}), \\ \mathcal{D}_{k+1} &= \mathcal{D}_k \cup \text{Curate}(\mathcal{B}_k^{\text{data}}), \\ (\tilde{\pi}, W_{k+1}^{\text{sim}}) &= \text{TrainAndCalibrate}(\mathcal{D}_{k+1}, W_k^{\text{sim}}), \\ \pi_{k+1} &= \text{Accept}(\tilde{\pi}, \pi_k; \mathcal{E}_{\text{eval}}), \end{aligned} \tag{4.6}$$

where $\mathcal{A}_k^{\text{acq}}$ is the collection plan, Curate applies the quality gates of Section 4.2, and Accept keeps the candidate $\tilde{\pi}$ only if it outperforms π_k in closed loop on $\mathcal{E}_{\text{eval}}$. Each batch $\mathcal{B}_k^{\text{data}}$ feeds both policy

training and, through the procedures of Section 4.4, the recalibration of W_k^{sim}, while $\mathcal{E}_{\text{eval}}$ stays outside both. Even so, repeated acceptance decisions gradually fit the policy to $\mathcal{E}_{\text{eval}}$ itself, so the evaluation set has to be refreshed from time to time, with a final set held back for reporting.

For the drawer fleet of Section 4.4, Collect in Eq. (4.6) uploads every operator takeover and every pull whose force profile leaves the expected envelope, while subsampling routine successes. Curate turns takeovers into recovery demonstrations attributed to the operator, TrainAndCalibrate retrains the policy and refits the drawer model, and Accept compares the result with the frozen previous version on held-out drawers at the failing site and at others. The release process that follows, including replay testing, staged deployment, and release gates, is described in Section 9.4.

4.5.6 Choosing What to Collect Next

Section 4.2 discusses how active sampling selects tasks and states within a collection campaign. At the level of the flywheel the choice is broader, because the next unit of effort may go to a new site, a calibration experiment, corrective demonstrations, a batch of simulated scenarios, or held-out trials for validation. A natural criterion is the expected improvement per unit cost,

$$x^\star = \arg\max_x \ \frac{\mathbb{E}\left[\Delta J \mid x\right]}{c_{\text{acq}}(x)}, \tag{4.7}$$

where ΔJ is the resulting change in closed-loop performance on held-out conditions and $c_{\text{acq}}(x)$ is the cost of collecting, curating, and learning from x. Physical options are expensive, since resets take time and some trials carry risk. Synthetic options are cheaper but less certain to help. In practice, candidates are ranked with heuristic scores such as novelty, disagreement between models, gaps in coverage, and predicted failure (Agia et al., 2025b; Xu et al., 2025a). Each score can mislead. An unfamiliar background may look novel without adding any physical information, and persistent uncertainty may come from sensor noise instead of a gap the policy could learn to close (Section 10.5).

Since Eq. (4.7) can at best be approximated before training, a collection strategy has to be judged after the fact, by retraining on the selected data and testing the result. The comparison should be made at a matched budget against reasonable alternatives, in particular stratified sampling over known conditions. It should also include a frozen baseline evaluated under the same conditions (Section 12.1), so that a better strategy is not confused with a larger investment. All costs belong in the same account, from robot occupancy and resets to interventions, annotation, simulation, and training. For repeatable tasks, total cost per accepted output is a convenient summary. Flywheels complicate the comparison further, because each policy changes the data collected next and because hardware, sites, and operators drift between iterations.

The measurements that would settle the central questions of this chapter are still largely missing. Most published scaling curves for robot learning put demonstrations, hours, frames, or tokens on the horizontal axis. Few relate closed-loop capability to the total amount of real interaction, counting the simulation and human effort that come with it. Such curves would show whether the next robot-hour is better spent on training data, on calibration, or on evaluation. They would also show whether a flywheel is actually improving the system. Chapter 10 considers what it would take for loops of this kind to sustain open-ended growth in capability.

5 World Models

Previous chapters discuss how perception captures physical scenes at the current moment, and physical data record how it evolves over time (Chapter 4). With them, we can present a more comprehensive account of world modeling, which studies how an environment evolves over time and how external actions or interventions change that evolution. This is also an important step for the physical agent that will be discussed in the later section, as it helps a physical agent to predict, before acting, how the world will evolve and how its own action would change that evolution. Figure 5.1 illustrates this closed-loop interaction among the physical environment, the world model, and the querying agent. Within the technical stack of this book, world models build on the state-space formulations of Chapter 2, the perception of Chapter 3, and the data and simulation of Chapter 4 to provide action-conditioned predictions for the manipulation and whole-body control models in Chapters 6 to 7.

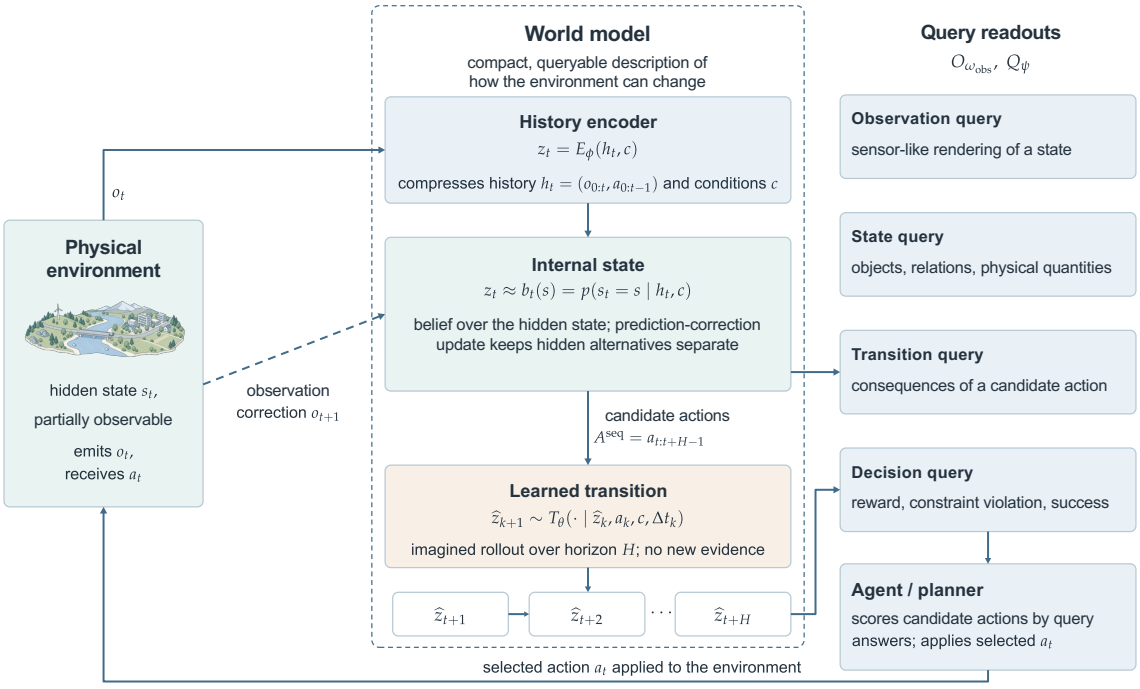


Fig. 5.1 A world model as a compact, queryable description of how the environment can change. Solid arrows carry information available at deployment. The dashed arrow carries the observation correction that revises the internal state after each step. A particular model may expose only a subset of the four query interfaces

Based on this, this chapter frames physical world modeling in five parts. Section 5.1 establishes the conceptual framework (state, belief, query, and evidence) and clarifies what a world model is and is not. Section 5.2 surveys how the internal state is represented, ranging from generative observation-space models to latent dynamics, 3D and object-centric structures, and unified multimodal architectures. Section 5.3 develops training strategies, from self-supervised pretraining to action-conditioned and interaction-driven learning. Section 5.4 addresses how predictions are deployed and validated across forecast accuracy, physical consistency, and downstream decision utility. Section 5.5 concludes with open challenges and the long-term role of world models in Physical AI.

Additionally, *A Definition and Roadmap for World Models* organizes physical world modeling around finite-resource compression, local observation, asynchronous multimodal evidence, and the rela-

tionship between understanding and prediction (Chen et al., 2026b). This perspective organizes the chapter's discussion of current systems and future directions. The scope of the chapter includes robots, scientific instruments, and distributed physical processes rather than only visually observed manipulation.

5.1 Concepts and Theoretical Frameworks of World Models

World models are compact, queryable descriptions of how an environment can change. They are not required to reproduce every physical detail. Their purpose is to retain the distinctions needed by the questions that an agent must answer, including what is present, what is hidden, what will happen under an action, and which action is preferable. This section fixes the common vocabulary of *state*, *transition*, *query*, and *evidence* used throughout the chapter. A model is useful when these elements are connected and its claims can be checked against new observations.

5.1.1 Definition, Boundaries, and Basic Properties

Physical scenes contain objects, fields, energy, and interactions at many scales, while an agent has finite sensors, memory, and time. A world model therefore compresses observations into an internal state from which future states and task-relevant outputs can be queried. The compression is task-relative. A logo may be irrelevant when predicting whether a cup will spill, but essential when identifying a reagent. A lower-dimensional representation is not automatically a better one. It is better only when it preserves the distinctions needed by the intended queries.

The same visible image can correspond to different velocities, contact modes, temperatures, or internal masses. A stationary drawer can be unlocked or held by a latch, and the same pull then has different consequences. A world model must use history, interaction, and background conditions to maintain these hidden alternatives. Operationally, its functions include inference from evidence, conditional evolution over time, and readout of observations or physical and task quantities. These functions may be modular or share parameters. A particular model can expose only a subset of them, but its advertised interfaces should have testable meanings.

A Development Trajectory Organized by Underlying Problems Three enduring questions explain the main technical lineages. First, how can an agent infer a sufficient state from incomplete measurements? State-space estimation and filtering make a distinction between an evolving state and its noisy observations. Kalman's work provides a foundational linear formulation (Kalman, 1960a). Second, how can an internal model improve action rather than merely fit observations? Dyna connects real experience, model learning, planning, and acting, so the learned model supplies additional experience for decision learning (Sutton, 1991). Third, how can these functions operate on high-dimensional sensory data? Learned encoders and recurrent dynamics replace hand-specified state variables with representations shaped by prediction and control. The *World Models* architecture separates visual compression, temporal memory, and a controller. PlaNet performs planning through learned latent dynamics. Dreamer trains behavior using imagined latent trajectories (Ha and Schmidhuber, 2018; Hafner et al., 2019; 2020).

These developments do not form a sequence in which each new architecture makes its predecessors obsolete. Explicit states remain useful when physical variables are measurable, generative models retain information for rich observation queries, and decision-oriented latent models can avoid reconstructing details that a task does not use. Recent multimodal and world-action models extend the same underlying questions to the sharing of semantic, predictive, and behavioral information. Their shared foundation is the connection among state inference, conditional prediction, and action learning. The discussion below compares implementations of these functions rather than presenting successive product generations.

Predictive Sufficiency Rather than a Miniature Copy of the World The central abstraction is a state that preserves answers, not necessarily a state that reproduces every observation. Predictive state representations make this idea explicit by representing a controlled system through predic-

tions of future observation tests rather than through a prescribed list of hidden variables (Littman et al., 2002). Learned embeddings, object states, and physical fields are different ways of approximating such predictive sufficiency. A useful idealized criterion is

$$p(y_{t+k}^{q_{\text{query}}} \mid h_t, A^{\text{seq}}, c) \approx p_\Theta(y_{t+k}^{q_{\text{query}}} \mid E_\phi(h_t, c), A^{\text{seq}}, c), \qquad q_{\text{query}} \in \mathcal{Q},\ A^{\text{seq}} \in \mathcal{A}_H, \tag{5.1}$$

where $\mathcal{Q}$ is the set of supported queries, $y_{t+k}^{q_{\text{query}}}$ the outcome of query q_{query} at offset k, $\mathcal{A}_H$ a declared set of input sequences over horizon H, and A^{seq} one such externally supplied sequence. The history is h_t, c records known conditions, E_ϕ compresses history into an internal state, and Θ parameterizes the predictive distribution. The left side describes the environment under the specified experimental input protocol. The right side describes its learned approximation. This is a design criterion, not a guarantee of identifiability from passive recordings. Its tolerance, horizon, and query set must be stated.

The criterion explains why compression and generality trade off. A representation sufficient to decide whether a drawer opens can omit texture but must preserve the latch hypothesis. A representation intended to answer new visual questions may need much more detail. A state sufficient for ranking a particular policy's actions need not be sufficient for estimating forces, synthesizing views, or optimizing a different objective. Broader query coverage therefore requires richer targets, retained evidence, or a mechanism for refining the representation on demand. Changes of latent coordinates do not change predictive capability when all interfaces transform consistently, so physically meaningful readouts matter more than requiring every latent coordinate to have a unique physical name.

Boundaries with Neighboring Systems Several neighboring systems should be distinguished by their interfaces. A video generator can reproduce motion without correctly responding to a specified intervention. A physics engine advances a supplied state but need not infer that state from open-ended observations. A digital twin is tied to a particular physical counterpart, whereas a policy outputs actions without necessarily exposing conditional futures. Table 5.1 summarizes the distinctions used here. These are functional distinctions. One implementation may combine several rows, and a model's name does not establish all the listed capabilities.

The state boundary must specify which variables are explicit, which are conditions, and which are unknown disturbances. Precision and horizon are part of this choice. A temperature interval can suffice for a safe-touch query but not for metrology. A short transport may treat material parameters as fixed while a long heating process cannot. Local boundaries also require attention to external inputs. A container can exchange heat and material with its surroundings. Missing measurements should remain marked as missing and should not create unjustified certainty. Claims of generality should state the tested state range, intervention interface, timescale, and query precision.

Table 5.1 Neighboring concepts and the capabilities their interfaces do not establish. The rows describe functions and may overlap within one system

System	Primary object	Typical interface	Not established by the interface alone
Video generator	Observations and temporal appearance	Images or videos under conditions	Correct interventions or physical quantities
Physics engine	Explicit state and physical dynamics	State propagation, contact, and forces	State inference from open observations
Digital twin	A specified physical counterpart	Monitoring, updating, and prediction	Transferable modeling across entities
Policy model	Goal-conditioned behavior	Actions, skills, or control targets	Independently queryable consequences
World model	Internal state and conditional evolution	Inference, prediction, and queries	A complete agent or controller

5.1.2 *Internal State and Belief Updating Under Partial Observation*

A POMDP distinguishes the environmental state, the actions applied to it, and the measurements available to an agent (Kaelbling et al., 1998). Let s_t be the physical state at discrete index t, o_t a possibly multimodal observation, and a_t the control input applied over the subsequent interval. Let c denote the known embodiment, environment, and sensor conditions. Here a discrete index orders updates. It need not imply equally spaced measurements. The history $h_t = (o_{0:t}, a_{0:t-1})$ contains observations and past inputs, with their timestamps and validity metadata understood. The belief over the hidden physical state is the conditional distribution

$$b_t(s) = p(s_t = s \mid h_t, c). \tag{5.2}$$

The argument s is a possible state value. b_t represents uncertainty about which value is current. A learned internal representation z_t is distinct from s_t. It may encode a point estimate, parameters of a distribution, or information sufficient for selected future queries. Calling a vector a belief does not by itself give it probabilistic semantics.

Belief maintenance alternates between prediction and correction. For a transition kernel T and an observation likelihood O, the Bayesian filtering recursion is

$$\begin{aligned} \widetilde{b}_{t+1}(s') &= \int T(s' \mid s, a_t, c, \Delta t)\, b_t(s)\, \mathrm{d}s, \\ b_{t+1}(s') &= \frac{O(o_{t+1} \mid s', c)\widetilde{b}_{t+1}(s')}{\int O(o_{t+1} \mid u, c)\widetilde{b}_{t+1}(u)\, \mathrm{d}u}, \end{aligned} \tag{5.3}$$

where s' is a candidate next state, u a dummy integration variable, Δt the elapsed physical time, and $\widetilde{b}_{t+1}$ the predictive prior before observing o_{t+1}. The denominator normalizes the posterior when the observation has nonzero probability. Integrals become sums for discrete states. The recursion assumes a Markov state and an observation model conditionally independent of earlier history given that state and c. Delays or actuator memory that violate these assumptions must be included in the state or conditioning record. Learned filters approximate this recursion rather than inheriting its correctness merely from architectural resemblance.

The distinction is concrete during grasping. If the gripper moves upward but the object remains on the table, the observation should revise the attachment hypothesis. If the camera is occluded, lack of visible motion need not mean a failed grasp. A delayed temperature reading belongs to its acquisition time, not its arrival time. Using it may require correcting an earlier state and propagating the correction through intervening actions. Asynchronous processing therefore requires temporal bookkeeping, not compulsory resampling of every sensor to one frequency.

Keeping alternatives separate can be more meaningful than averaging them. The mean of two possible object locations may lie inside an obstacle, and a mean of sticking and sliding contact is not a realizable contact mode. Partial-observation uncertainty concerns the current hidden state. Aleatoric uncertainty concerns variability unresolved at the chosen modeling level. Epistemic uncertainty concerns limited knowledge of model parameters or mechanisms (Kendall and Gal, 2017). These categories can interact, and their decomposition depends on the state and sensor specification. Extra sensing may resolve a hidden contact, whereas collecting diverse interactions may reduce model error. Simply increasing the number of sampled futures addresses neither problem automatically.

5.1.3 *Understanding-Oriented and Prediction-Oriented Objectives*

Understanding is best stated as a set of testable abilities, including identifying entities and relations, recognizing operating modes, distinguishing variables that control subsequent change, and identifying what present evidence cannot determine. A fluent explanation is not sufficient. For example, a valve model must distinguish a command, the mechanical position, and the resulting flow. A grasp model must distinguish a closed gripper from a stable attachment. The understanding-prediction distinction in Chen et al. (2026b) provides a useful way to organize these requirements without treating them as mutually exclusive capabilities.

Prediction supplies a complementary pressure. Given a history and conditions, the model should produce a measurable future state, observation, or distribution. Copying a background and extrapolating smooth motion may solve short video prediction while ignoring contact. Longer horizons, action perturbations, and event-level targets test whether the state retains the missing distinctions. Distributional prediction matters when several qualitatively different outcomes remain possible. A single mean trajectory can represent none of them.

Reconstruction is a useful information diagnostic but not proof of understanding. DIAMOND demonstrates that visual detail can affect learned behavior in Atari, illustrating why discarding apparently minor observations may be harmful (Alonso et al., 2024). Conversely, a model can render a convincing container without connecting liquid level, acceleration, and spillage. Decision-oriented abstractions are legitimate when they preserve distinctions needed to compare actions. MuZero learns quantities needed for planning without requiring reconstruction of the original observation, showing that control sufficiency and observation reconstruction are different targets (Schrittwieser et al., 2020). Neither example establishes a universally preferred representation for physical modeling.

Understanding also requires separating reusable relations from accidental correlations. If every heavy training container is red, color may predict lifting success until the correlation changes. A useful representation should allow mass hypotheses to be revised after force feedback without having to change color identity. This example does not require every latent coordinate to have a human name. It requires an intervention-relevant distinction to remain available to downstream queries. Prediction and understanding can thus be mutual checks. An explanation proposes a testable consequence, while a prediction error exposes a missing variable or an inadequate transition.

Representation Learning, Forecasting, and Identification These goals differ in the evidence they require. Representation learning can exploit correlations across regions or times. Forecasting must respect the observations available at its issue time. Identifying an intervention response additionally requires the action interface and relevant confounders to be accounted for. A passive video predictor can learn that a human usually opens a drawer after reaching toward it, while still being unable to distinguish a push from a pull applied by a robot. Likewise, fitting a trajectory does not uniquely identify mass and friction when several parameter combinations produce the same motion. An informative probe is one that makes these alternatives predict measurably different outcomes. This connects world modeling to state estimation and system identification, rather than treating physical knowledge as an automatic consequence of video scale.

Learning with a JEPA offers a complementary response to the information problem by predicting selected representations of observations rather than all their low-level details. LeCun's architectural proposal places this idea within a larger system of perception, predictive modeling, memory, and action selection (LeCun, 2022). The reusable principle is selective prediction. Whether the selected information supports causal forecasting, planning, or physical explanation is determined by the conditioning, training data, and readout tests developed below.

5.1.4 Functional Roles, Query Interfaces, and System Positioning

The role of a world model is determined by the questions it answers. An observation query produces a sensor-like output. A state query returns objects, relations, or physical quantities. A transition query predicts an action's consequences. And a decision-related query estimates reward, constraint violation, or task success. Figure 5.1 assembles these roles into a reference architecture. A compact conceptual implementation is

$$\begin{aligned} z_t &= E_\phi(h_t, c), \qquad \widehat{z}_{t+1} \sim T_\theta(\cdot \mid z_t, a_t, c, \Delta t), \\ \widehat{o}_{t+1} &\sim O_{\omega_{\text{obs}}}(\cdot \mid \widehat{z}_{t+1}, c), \qquad \widehat{y}_{t+1} = Q_\psi(\widehat{z}_{t+1}, g, c), \end{aligned} \tag{5.4}$$

where E_ϕ is a history encoder, T_θ a learned transition, $O_{\omega_{\text{obs}}}$ an observation decoder, and Q_ψ a task readout. Their subscripts denote parameter sets, which may overlap in a shared network. The goal or query specification is g, y is the requested task quantity, a hat denotes a prediction, and $\sim$

denotes sampling. Q_ψ is a generic query head, not necessarily an action-value function. Eq. (5.4) is an interface-level synthesis of state-space and learned-model formulations, not a requirement that these functions be separate networks (Kaelbling et al., 1998; Hafner et al., 2019).

In the figure, the history encoder E_ϕ compresses the observation-action history h_t and known conditions c into the internal state z_t, which approximates the belief $b_t(s)$ over the hidden physical state. The prediction-correction update keeps hidden alternatives separate. The learned transition T_θ then rolls out imagined latent states $\widehat{z}_{t+1}, \ldots, \widehat{z}_{t+H}$ under a candidate action sequence, and the readouts $O_{\omega_{\text{obs}}}$ and Q_ψ answer observation, state, transition, and decision queries over the inferred state and the imagined rollout. Figure 5.1 also makes two boundaries explicit. The first separates inference from imagination. The internal state is updated from real observations only at the correction step, whereas the rollout is advanced by the learned transition alone and accumulates model error with horizon. The second separates the model from its consumer. Readouts answer queries over the inferred state and the imagined futures, but the scoring of candidate actions and the choice of the applied a_t belong to the agent or planner. A model may support transition and decision queries without an observation decoder. Evaluation should follow the query interfaces it exposes.

This decomposition helps localize failure. A wrong rendered texture with a correct measured trajectory suggests a decoding problem. A wrong contact event suggests a state or dynamics problem. A correct prediction that never influences action suggests a decision-interface problem. Such diagnoses are hypotheses to test, since errors may also propagate across modules.

The world model can sit inside a model-predictive controller, provide training-time imagination to a policy, supply predictive features to an action model, or expose a shared state service to several agents. The surrounding loop remains perception, state inference, prediction, decision, execution, and feedback. Actions change both the world and the observations available next. This connects the chapter to physical data acquisition in Chapter 4 and to the foundation-model and motor-control interfaces in Chapters 6 to 7. World modeling is one component of Physical AI. Sensing, actuation, safety, and recovery do not follow automatically from a predictive network.

The remaining sections separate four questions concerning how information is represented, how parameters are learned, how predictions are used and verified, and how the full loop can improve. In particular, training with future observations, predicting future observations at runtime, and executing actions with real feedback are different operations. This chapter makes their information boundaries explicit throughout.

5.2 Representations and Architectures of World Models

No single representation dominates all physical queries. Observation generators retain visible detail, latent dynamics can reduce imagination cost, and structured three-dimensional or object-centric states make entities and relations explicit. These are composable design routes rather than disjoint model classes. Following the distinction between functional and architectural taxonomies in Chen et al. (2026b), it is useful to ask separately *what is predicted*, *how dynamics are represented*, *which functions share computation*, and *which modules run at deployment*. A diffusion model can operate over pixels, compressed video latents, or object states. A geometric representation can support both rendering and planning. And any of these can be coupled to a policy.

Figure 5.2 organizes representative world models along two complementary dimensions of functional role and architectural representation. The functional axis distinguishes rendering observations, simulating state evolution, and generating actions for decision making. The architectural axis distinguishes observation-level generation, structured world representations, and latent-space dynamics. These categories are not mutually exclusive. A model family may span several functions, and a system may combine multiple representational choices.

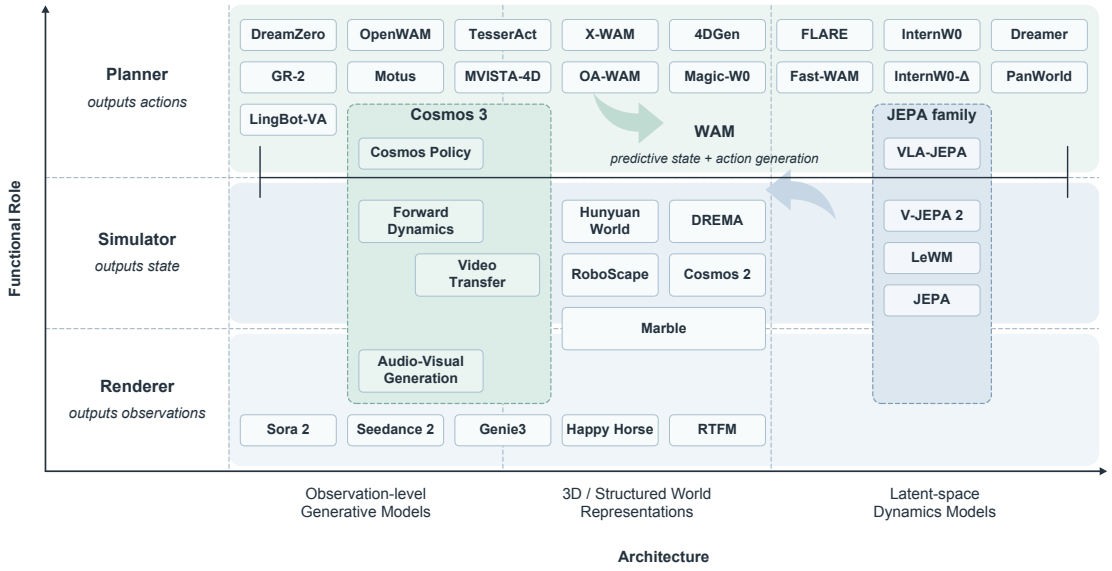


Fig. 5.2 Two-dimensional taxonomy of representative world models. The vertical axis distinguishes functional roles. Renderers produce observations, simulators predict states, and planners produce actions. The horizontal axis distinguishes architectural routes, including observation-level generative models, 3D or structured world representations, and latent-space dynamics models. Spanning regions indicate model families or systems that support multiple roles or combine representational choices. World-action models (WAMs) couple predictive modeling with action generation. Placement is schematic rather than a performance ranking. Planning capability may depend on additional policy or search components

5.2.1 Generative Architectures in Observation Space

An observation-oriented model predicts images, video, audio, or other sensor streams. For an externally specified action sequence, a causal autoregressive factorization is

$$p_\theta(o_{1:N_{\text{step}}} \mid o_0, a_{0:N_{\text{step}}-1}, c) = \prod_{t=0}^{N_{\text{step}}-1} p_\theta(o_{t+1} \mid o_{0:t}, a_{0:t}, c), \tag{5.5}$$

where N_{step} is the final sequence index, θ the model parameters, and $i{:}j$ an inclusive sequence of indices. The elapsed times are included in c for this expression. Each action is aligned to the interval in which it acts. The equation assumes that future prescribed actions do not affect earlier observations. With actions selected in response to future measurements, conditioning retrospectively on the entire executed action sequence can reveal additional information. Such observational conditioning is not automatically equivalent to an intervention.

Tokenization and Generative Mechanism Autoregressive models predict a sequence of discrete or continuous tokens from preceding context. Their ordering makes causal rollout natural, but generating many spatial and temporal tokens can require substantial sequential computation. Diffusion and flow-based models instead generate a frame or chunk through iterative denoising or transport, enabling parallel updates within the chunk while requiring multiple network evaluations. Chunkwise generation combines these choices. Chunks advance causally in physical time, while tokens within each chunk can interact bidirectionally. Physical rollout time and the artificial denoising time are different axes. Section 5.3.1 defines the corresponding training objectives.

An observation-oriented generator need not perform all computation in pixel coordinates. A variational autoencoder (VAE) can compress observations, a generative model can predict the compressed variables, and a decoder can reconstruct the output. Latent diffusion demonstrates this division between perceptual compression and generative computation (Rombach et al., 2022). It reduces the number of generated elements but does not, by itself, turn video latents into a sufficient physical state. The architectural distinction from an abstract latent-dynamics model concerns the representation's role and supervision, not whether a tensor is called a latent.

Conditioning and Interaction Genie learns latent controls from videos and uses them for interactive generation, whereas UniSim learns conditional real-world video simulation from heterogeneous data (Bruce et al., 2024; Yang et al., 2024b). These examples illustrate different intervention interfaces. A latent action inferred from visual changes is not automatically a robot command. Camera motion, actor motion, and object motion can be confounded. Executable control requires a mapping to an embodiment and verification of the resulting response. Language can specify desired behavior, but the same sentence may leave force, speed, contact, and timing unspecified.

Observation generation is useful when detail matters for perception, when imagined data will train a visual policy, or when a person must inspect a predicted future. Its limitations include high spatial cost, temporal drift, and the possibility of replacing a wrong state with a plausible texture. A video of a drawer opening should be checked for the commanded displacement, latch state, object identity, and occluded contents, not only visual sharpness. Multiple rendered views are valuable only when they describe a compatible underlying scene. These requirements motivate persistent latent states, geometric structure, and action-grounded supervision rather than ruling out generative prediction.

Broad Video Priors and Spatially Indexed Memory Cosmos illustrates how pretrained video models, tokenizers, and data processing can form a reusable platform for physical-AI adaptation (NVIDIA et al., 2025a). Such a prior can supply appearance and motion regularities before task-specific action data are available. Its contribution is different from learning the calibrated response of a particular actuator. Downstream conditioning and validation must establish that interface. This separation between broad pretraining and local grounding remains useful as individual video backbones change.

Persistent visual context can also be organized spatially rather than only as a recent temporal window. World Labs' RTFM uses an autoregressive diffusion frame model with camera poses and retrieval of relevant posed frames as spatial memory (World Labs Team, 2025b). It illustrates a learned rendering route that need not construct an explicit mesh at every update. Revisiting a place queries stored evidence about that place rather than forcing an ever-growing sequence through a fixed context window. Camera-conditioned view generation, however, primarily specifies how observation changes with viewpoint. Controlling a camera and predicting an object's response to force are distinct interfaces. This distinction is important when a navigable generated world is later used for interaction.

5.2.2 Latent-Space Dynamics Architectures

Latent models encode observation history and propagate a compact state. A generic form is

$$\begin{aligned} z_t &= E_\phi(h_t, c), \qquad \widehat{z}_t = z_t, \\ \widehat{z}_{t+k+1} &\sim T_\theta(\cdot \mid \widehat{z}_{t+k}, a_{t+k}, c, \Delta t_k), \\ \widehat{y}_{t+k+1} &= Q_\psi(\widehat{z}_{t+k+1}, g, c), \quad k = 0, \dots, H-1, \end{aligned} \tag{5.6}$$

where H is the rollout horizon, Δt_k the duration of its kth interval, E_ϕ the history encoder, and Q_ψ the query readout. The symbols z and $\widehat{z}$ distinguish inferred and imagined states. y may be an observation, reward, physical variable, or event. The readout can be omitted during intermediate rollout steps. A useful state preserves the conditional distribution of the intended queries. Histories merged into one representation should not demand incompatible answers after the same future

input. A recently heated vessel and a long-cooled one may look alike while requiring different temperature forecasts.

Generative State-Space Models PlaNet combines deterministic recurrent memory with stochastic latent variables, while Dreamer uses learned latent dynamics to train an actor and critic through imagination (Hafner et al., 2019; 2020). A recurrent component can retain predictable history, and stochastic variables can represent outcomes that remain uncertain. Training distinguishes a prior predicted without the next observation from an approximate posterior corrected by that observation. Reconstruction and auxiliary targets encourage informative states. Prior-posterior regularization makes them predictable. Overly strong regularization can suppress observation information, whereas a weakly trained prior can fail when real frames are absent. The loss and its multi-step extensions are developed in Section 5.3.

JEPA: Predicting in a Learned Target Space A JEPA relates a context representation to a target representation through a predictor. Its defining choice is the prediction target, not a particular recurrent layer, transformer size, or policy head. I-JEPA learns image representations by predicting target-region embeddings from visible context. It is a spatial representation learner, not by itself an action-conditioned temporal simulator (Assran et al., 2023). V-JEPA extends feature prediction to masked spatiotemporal regions of video, using a predictor and a slowly updated target encoder rather than a pixel-reconstruction objective (Bardes et al., 2024). In both cases, what is hidden and what remains visible determine whether the task teaches spatial completion, temporal interpolation, or future anticipation.

V-JEPA 2 connects large-scale action-free representation learning to a separately post-trained action-conditioned latent predictor. Its planning variant uses robot observations, proprioception, and recorded actions to predict future features, then evaluates candidate actions against an image goal in representation space (Assran et al., 2025). Recorded interactions ground the predictor in robot actions. Self-supervision constructs targets from these records without task-success annotations or a hand-designed reward for each training example.

The architectural advantage is that a predictor can avoid synthesizing every unpredictable texture while retaining information useful for recognition or control. The corresponding obligation is to show that contacts, relative motion, object identity, and material cues have survived the target representation. A deterministic embedding forecast need not describe the distribution of possible futures, and its distance to a goal need not be a calibrated measure of physical reachability. Observation reconstruction and predictive embeddings are therefore complementary choices. A model can learn from both, or attach an optional decoder without making rendering a prerequisite for each decision. Section 5.3.1 develops the target update and loss, while Section 5.4.3 discusses tests that do not privilege models with a visual decoder.

Decision-Oriented Abstraction and State Maintenance MuZero and TD-MPC-style systems emphasize reward, value, or latent consistency sufficient for decision making rather than unrestricted observation queries (Schrittwieser et al., 2020; Hansen et al., 2024). DreamerV3 retains a generative latent model while improving robustness of model and behavior learning across different domains through normalization and loss balancing (Hafner et al., 2025). These examples distinguish the choice of representation from the numerical techniques needed to train it reliably. A small state is efficient only if it preserves the distinctions required by the objective. Changing the goal or downstream query can expose previously harmless information loss.

Latent coordinates need not correspond one-to-one to physical quantities. A feature from which mass can be decoded may still entangle mass with shape, and an arbitrary coordinate edit need not realize a meaningful mass intervention. A history encoder should also distinguish an observation-corrected state from one propagated without new evidence. Its memory may be recurrent, attention-based, spatially retrieved, or hierarchically summarized. What matters is whether the representation preserves consequential hidden alternatives through occlusion and updates them when new evidence arrives. Tests should compare posterior updates, pure open-loop prediction, and feedback-interleaved prediction at the same physical horizons.

5.2.3 3D, 4D, and Object-Centric Architectures

Structured states make geometry, entities, and relations directly queryable. They can use occupancy or bird's-eye grids, object slots, scene graphs, particles, meshes, or continuous fields. Their relation to world modeling depends on the associated transition. A static 3D reconstruction supports novel views but does not specify what a push will do. A 4D reconstruction describes temporal observations but may not predict a new intervention. The spatial perception methods in Chapter 3 become a world-model substrate when they are connected to evolving attributes, interactions, and conditional prediction.

Object-centric methods allocate separate representations to entities and model their interactions. Slot Attention provides a mechanism for learning object-centric representations from observations (Locatello et al., 2020). Graph-network simulators and MeshGraphNets illustrate dynamics expressed through local interactions among particles or mesh elements (Sanchez-Gonzalez et al., 2020; Pfaff et al., 2021). These structures can make interaction locality and changes in object count easier to express than an undifferentiated global vector. However, the structure is an inductive bias, not a guarantee. Learned slots may exchange identities, an object may split or merge, and contact can change the interaction graph discontinuously.

Geometry and physical attributes should be distinguished. Shape alone does not identify mass, friction, stiffness, temperature, or internal contents. A deformable state may need both configuration and a constitutive description. A liquid requires boundary conditions and appropriate conservation accounting. PhysGaussian couples Gaussian-based rendering with continuum-mechanics simulation, illustrating how visual representation and physical evolution can share a substrate within a specified regime (Xie et al., 2024). The same representation must still be supplied with suitable physical parameters and verified against the intended materials and interactions.

Hybrid representations can assign different resolutions to different objects or processes. A rigid container may be represented as an object, while its contents are a field or a small set of effective variables. A coarse query could ask whether spilling is likely. A fine query might require the flow near an outlet. Expanding the representation only where the query requires it is a plausible way to balance cost and detail. The levels must share coordinate frames, identity, units, boundary fluxes, and physical time so that a coarse state is compatible with a refined prediction.

The structured route is attractive for metric queries, contact reasoning, and compositional changes. It introduces costs in reconstruction, correspondence, annotation, and numerical integration, and it may inherit a simulator's modeling assumptions. A representation built around rigid objects can fail on cloth or fragmentation. A fixed field resolution can miss thin contacts. Validation should test identity through occlusion, changed topology, altered parameters, and action-dependent response. Structured models can be combined with learned latent residuals and generative observation heads, rather than being treated as alternatives to neural representation learning.

Hierarchies, Fields, and Physical-Prediction Structure The Hierarchical Relation Network (HRN), developed by Mrowca and colleagues, represents bodies with hierarchies of particles and learns their interactions through graph-based computation (Mrowca et al., 2018). Its lasting contribution is the separation of local interactions from larger-scale object structure, allowing rigid and deformable dynamics to share a representation without pretending that all objects are single rigid slots. Hierarchical modeling can reduce the distance over which information must propagate, but the useful grouping and coupling still depend on the physical process. The related Physion evaluation program tests future object contact across varied physical scenarios and distinguishes learning from images from prediction with privileged state (Bear et al., 2021). It provides an empirical counterpart to the architectural question of which state distinctions are needed to forecast an event.

Continuous-field models broaden the scope beyond articulated robots and discrete objects. Fourier neural operators learn mappings between parameter or input fields and solution fields for families of partial differential equations (Li et al., 2021). They can serve as components for fluid, thermal, or other distributed-process predictions. A field-to-solution surrogate need not include perception,

uncertainty, or an action policy. It supplies one transition or query component of a larger world model. Generalization across spatial resolution also differs from generalization across boundary conditions or physical regimes. Coupling a learned field solver to objects requires consistent interfaces for forces, fluxes, time, and units.

Persistent Spatial Generation and Multimodal Reconstruction The spatial-intelligence route associated with World Labs should be distinguished from, rather than conflated with, the physical-dynamics work above. Marble generates persistent, editable 3D environments from multimodal prompts and supports explicit scene outputs (World Labs Team, 2025a). It illustrates the value of a scene that can be revisited and queried from different viewpoints, instead of a sequence whose geometry is reconstructed anew at every frame. The public Atlas description further combines text, posed images, and depth within a shared spatial context using an autoregressive diffusion architecture, with reconstruction and space-time generation interfaces (World Labs Team, 2026). These are developer-described systems. Their available interfaces should not be expanded into unsupported claims about arbitrary contact dynamics or physical precision.

The underlying distinction is between reconstructing evidence, completing unobserved regions, and evolving a state under interventions. A model may do all three, but their outputs carry different evidential status. Adding real views can constrain a previously imagined region. Creating a visually plausible hidden room does not measure that room. An explicit 3D scene can supply geometry for a simulator while leaving collision properties, mass, and material parameters to be assigned or identified. Conversely, an implicit renderer can maintain useful spatial consistency without exposing a simulation-ready mesh. The choice of explicit or implicit representation should therefore follow the required queries, not a blanket identification of 3D output with physical understanding.

5.2.4 Unified Multimodal and Cross-Paradigm Architectures

Unification concerns several different design decisions, including sharing an input interface, sharing a representation, learning multiple objectives together, and executing multiple functions in one runtime. A system may unify one level while retaining separate modules at another. Describing the level of unification is more informative than calling the entire architecture unified.

Multimodal Evidence and Representation Sharing Vision, touch, proprioception, sound, temperature, and process readings can update a common state while retaining modality-specific noise and timing. Fusion can occur at the input, in intermediate state representations, or at query time. Early fusion preserves rich interactions but couples token budgets and preprocessing. Separate encoders with state-level exchange preserve sensor-specific processing while introducing alignment interfaces. Query-time fusion is modular but can lose information already discarded by each encoder. In a lifting task, disagreement between vision and torque may indicate an incorrect mass hypothesis, a grasp failure, or a sensor fault. Averaging the signals into an unexplained scalar is not a solution.

Understanding and generation also have different representational needs. Show-o combines autoregressive and diffusion-based modeling within a unified transformer. BAGEL studies shared multimodal understanding and generation with specialized representational pathways (Xie et al., 2025; Deng et al., 2025a). These attempts show that shared processing need not require identical tokenizers or identical prediction targets. They motivate joint semantic and generative learning, but successful image understanding and synthesis do not independently verify a model's response to physical interventions.

Coupling Future Prediction and Action Generation Let $A_t^{\text{seq}} = a_{t:t+H_a-1}$ denote an action chunk of length H_a, and let $F_t^{\text{future}} = o_{t+1:t+H_o}$ denote future observations over H_o observation steps. A

statistical joint model admits the factorizations

$$\begin{aligned} p_\Theta(A_t^{\text{seq}}, F_t^{\text{future}} \mid h_t, g, c) &= p_\Theta(A_t^{\text{seq}} \mid h_t, g, c)\, p_\Theta(F_t^{\text{future}} \mid A_t^{\text{seq}}, h_t, g, c) \\ &= p_\Theta(F_t^{\text{future}} \mid h_t, g, c)\, p_\Theta(A_t^{\text{seq}} \mid F_t^{\text{future}}, h_t, g, c), \end{aligned} \tag{5.7}$$

where Θ denotes the model's parameters. The action and observation horizons must refer to compatible physical intervals even when their sample counts differ. The first factorization suggests action proposal followed by consequence prediction. The second suggests a future subgoal followed by inverse-dynamics action generation. An observed conditional distribution is not automatically an interventional model, and an attractive generated goal is not necessarily reachable.

Unified World Models (UWM) couples video and action diffusion with independent modality-specific noise levels, using the conditioning pattern to obtain policy, forward-dynamics, inverse-dynamics, and video-generation interfaces (Zhu et al., 2025). This illustrates how the same parameter set can support different queries when the training distribution and masks explicitly cover those queries. It does not imply that every joint generator reliably supports every conditional query. A cascaded design is easier to inspect but can compound errors and latency. A jointly sampled design enables stronger coupling but must control incompatible objectives and future-information leakage.

DreamZero supplies another representative use of a pretrained video prior for joint future-video and action modeling (Ye et al., 2026a). Together with UWM, it illustrates a broad design axis in which action supervision can shape a model of anticipated observations instead of being added only to a static visual encoder. The systems need not share the same architecture, sampling procedure, or adaptation protocol. OpenWAM's composable framework makes this distinction explicit by separating prediction, action generation, their interaction structure, and inference choices for controlled comparison (Wang et al., 2026c). Its methodological lesson is to compare one interface choice at a time, rather than attributing every difference between complete systems to "world modeling."

A third route uses future supervision to shape a representation available from present evidence, without generating future observations during deployment. InternVLA-A1.5 trains foresight tokens through a frozen video model and removes the video branch at inference (Ma et al., 2026a). InternW0-Δ combines a video expert, an action expert, frozen VLM semantics, and training-only 4D distillation. Causal Imprint connects future-derived supervision to observation-conditioned action features (Miao et al., 2026). Such systems unify predictive learning with action generation without requiring a full future rollout for every policy call.

Expert Specialization and Runtime Organization A mixture-of-transformers can retain separate parameters and token streams while allowing masked attention between experts. The mask specifies which observations, predicted variables, and action tokens can influence one another. Shared attention does not imply unrestricted bidirectional access. A cache can be reused across action-denoising steps only if its inputs and all upstream dependencies remain unchanged. Architectural independence, gradient isolation during training, and asynchronous execution are three distinct properties.

InternW0 illustrates a complementary runtime design in which slower video prediction supplies cached context, and a faster action expert adapts that context to recent observations through an observation-conditioned routing interface (Cai et al., 2026b). This differs from training away the future generator. In InternW0-Δ, future-video tokens are omitted at inference, but the video expert still processes observed context and Causal Imprint once before action-only denoising (Miao et al., 2026). Neither arrangement should be summarized simply as "the video module is free." Their evidence boundaries are taken up again when decision utility is evaluated in Section 5.4.4.

Comparative Strengths and Composable Choices Table 5.2 compares the four routes by what they make convenient, what they cost, and what must be validated. The entries describe tendencies, not universal performance rankings. Actual cost depends on backbone size, representation resolution, number of sampling steps, and deployment implementation.

Table 5.2 Architectural routes, practical trade-offs, and principal validation questions. The routes are composable. Multimodal unification is also a cross-cutting organization choice

Route	Useful interface / strength	Main cost or limitation	Critical validation
Observation generation	Detailed, inspectable futures and synthetic sensor data	Large output budget, decoding and sampling cost, plausible but wrong detail	Action-conditioned change, identity, and long-horizon consistency
Latent dynamics	Compact rollout, inexpensive task readout, planning	Objective-dependent information loss, latent drift, limited new queries	Predictive sufficiency, open-loop stability, and calibrated readouts
3D / object-centric	Metric geometry, entities, and local interactions	Reconstruction and correspondence, structural and simulator assumptions	Contact, topology, identity, physical parameters, and interventions
Multimodal / unified	Shared evidence and priors, multiple query interfaces	Objective interference, alignment and missing-data issues, runtime coupling	Information boundaries, cross-query agreement, missing modalities, and total cost

A design can therefore combine an object-centric state, latent stochastic transitions, an optional video decoder, and a policy trained from future supervision. The right combination is determined by the required queries and evidence. Cross-query consistency is useful. Rendering a state and re-estimating it should preserve relevant quantities. However, two heads can share the same bias, so their agreement is not independent physical validation. Section 5.5 returns to hierarchical, multi-timescale, and selectively activated architectures as future directions rather than treating a single fully shared backbone as inevitable.

Reading Representative Work Through Common Design Questions Table 5.3 summarizes representative implementations of the preceding principles. It is a map of mechanisms, not an exhaustive catalog or a ranking. A new model can be located by its retained state, supervisory target, intervention interface, and deployed computation without adding a new supposedly disjoint category for every paper.

The table also exposes missing interfaces. An image representation learner still needs a temporal, action-conditioned predictor to support control queries. A persistent scene generator still needs dynamics for manipulation. And a successful policy still needs separate tests before being used as a general predictive simulator. These are opportunities for composition, not reasons to exclude the corresponding lines of research from world modeling.

5.3 Training World Models

Training must make the internal state useful for future queries rather than merely reconstruct familiar observations. A complete account specifies the data available to each objective, the variables predicted, the parameters updated, the rollout horizon, and the information excluded from deployment. Representation learning, dynamics learning, policy learning, physical constraints, and uncertainty estimation solve related but different problems. Their combination is a design choice, not one mandatory loss shared by all world models.

A practical training program can start from scratch, adapt pretrained components in stages, or connect frozen specialists through learned interfaces. Broad observation pretraining supplies reusable visual and temporal structure. Action-conditioned training grounds intervention semantics. Downstream post-training adapts the model to a task or embodiment. Interaction-driven updates address the states reached by the deployed system. Some systems combine these stages, and some do not require all of them. PlaNet illustrates joint latent-model learning, V-JEPA 2 separates broad representation learning from action-conditioned post-training, and UWM combines

Table 5.3 Representative lineages organized by enduring modeling questions. Examples instantiate the mechanisms discussed in the text. They do not share one evaluation protocol

Underlying question	Representative work	Mechanism to compare
How is history compressed for control?	World Models, PlaNet, Dreamer	State inference, recurrent dynamics, and imagination
Which predictions are sufficient for decisions?	MuZero, TD-MPC2	Reward/value and task-relative latent consistency
Can useful futures be predicted without pixels?	I-JEPA, V-JEPA, V-JEPA 2	Target embeddings, masking, and action grounding
How are interactive observations generated?	DIAMOND, Genie, UniSim, Cosmos	Generative targets, controllable inputs, and rollout memory
How are spatial persistence and reconstruction maintained?	Marble, RTFM, Atlas	Explicit scenes or posed-view memory Measured versus completed content
How is physical structure exploited?	HRN, graph/mesh simulators, PhysGaussian, neural operators	Entities, local interactions, physical priors, and fields
How are prediction and action coupled?	UWM, DreamZero, InternVLA-A1.5, InternW0, InternW0-Δ	Joint generation, predictive features, and training-runtime separation

predictive and behavioral supervision within a shared generative framework (Hafner et al., 2019; Assran et al., 2025; Zhu et al., 2025). Figure 5.3 summarizes this landscape by showing the entry points along the top, the data sources on the left, the four training stages in the center, and the world-action interfaces on the right.

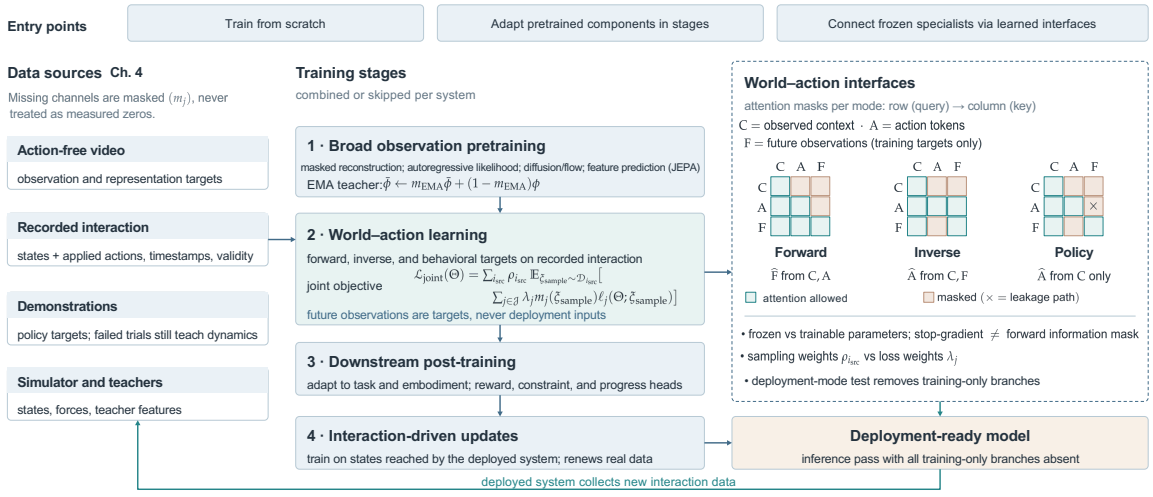


Fig. 5.3 Training world models. Three entry points (top) lead into a staged training pipeline (center), fed by the data sources of Chapter 4 (left). Interaction-driven updates retrain on the states reached by the deployed system, closing the loop through new data. The right panel defines the world-action interfaces as attention masks over observed-context (C), action (A), and future-observation (F) tokens

The data foundations are developed in Chapter 4. Their training consequences differ. Action-free videos can supervise observation or representation prediction. Recorded controls can supervise action-conditioned transitions. Demonstrations can supervise a policy. And simulator states, force readings, or teacher features provide additional targets with distinct provenance. A valid failed trajectory can teach dynamics and outcomes without being an appropriate nominal imitation target. Missing channels must be masked rather than treated as measured zeros.

The four stages in the center of Figure 5.3 carry different objectives. Broad observation pretraining applies masked reconstruction, autoregressive likelihood, diffusion or flow, and feature-prediction targets, with an exponential-moving-average teacher where the target representation must stay stable. World-action learning adds forward, inverse, and behavioral targets on recorded interaction. Downstream post-training adapts the model to a task and embodiment. And interaction-driven updates retrain on the states the deployed system actually reaches. That last stage closes the loop only through new data, which is what distinguishes continual improvement from repeated fitting of a fixed corpus. In the right panel, the forward mode predicts $\widehat{\mathrm{F}}$ from C and A, the inverse mode predicts $\widehat{\mathrm{A}}$ from C and F, and the policy mode predicts $\widehat{\mathrm{A}}$ from C only, with the future-to-action path masked out as a leakage route (red cross). In all three modes the context rows are barred from reading F, so the context representation remains a pure function of observed evidence even when the action query may consult the future. These interfaces are reading modes over the same token groups, selected by attention masks, rather than three separate models. One network can be trained to serve all of them provided the masks are enforced exactly, as Section 5.3.2 discusses. The deployment-ready model is then an inference pass with all training-only branches absent.

5.3.1 Self-Supervised and Generative Representation Learning

Self-supervised objectives extract structure from observations without requiring every physical variable to be annotated. Temporal prediction, masked reconstruction, contrastive discrimination, and target-representation prediction impose different information bottlenecks. A reconstruction target retains observable detail. A feature target retains distinctions present in its encoder. A temporal target encourages information predictive over its selected horizon. None automatically identifies mass, contact, or causality when the observations do not distinguish them.

Autoregressive Likelihood and Masked Reconstruction Let $x = (x_1, \ldots, x_{N_{\text{tok}}})$ be a target sequence of N_{tok} tokens, κ the available conditioning information, and $\mathcal{D}$ the training distribution. A maximum-likelihood objective is

$$\mathcal{L}_{\text{AR}} = -\mathbb{E}_{(x,\kappa)\sim\mathcal{D}} \left[\sum_{i=1}^{N_{\text{tok}}} \log p_\theta(x_i \mid x_{<i}, \kappa) \right]. \tag{5.8}$$

The index i orders tokens, $x_{<i}$ contains preceding target tokens, and θ denotes trainable parameters. The conditioning can include past observations, a goal, and supplied actions, depending on the advertised query. Discrete tokens yield cross-entropy training. Continuous outputs require an explicit conditional density. During teacher-forced training, preceding targets are real tokens, whereas free generation conditions on the model's own outputs. Thus a low teacher-forced loss does not establish stable recursive prediction.

Masked reconstruction instead withholds selected spatial or temporal observations and predicts them from visible context. A mask that reveals observations on both sides of a hidden frame trains interpolation or smoothing, not causal future prediction. Such supervision can still be valuable for representation pretraining, but a causal predictor needs a corresponding future-blind training or adaptation stage. The mask pattern, temporal window, and conditioning availability should be recorded alongside the loss.

Diffusion and Flow-Matching Objectives For a clean continuous target $\mathbf{x}$, a noise-prediction diffusion loss takes the form

$$\begin{aligned} \mathbf{x}^{(j)} &= \sqrt{\bar{\alpha}_j}\,\mathbf{x} + \sqrt{1-\bar{\alpha}_j}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}), \\ \mathcal{L}_{\text{diff}} &= \mathbb{E}_{\mathbf{x},\kappa,j,\boldsymbol{\epsilon}}\left[w_j \left\| \boldsymbol{\epsilon}_\theta(\mathbf{x}^{(j)}, j, \kappa) - \boldsymbol{\epsilon} \right\|_2^2 \right], \end{aligned} \tag{5.9}$$

where j indexes artificial noise levels, $\bar{\alpha}_j$ is the retained signal fraction, $\mathbf{I}$ the identity covariance, w_j a nonnegative loss weight, and $\boldsymbol{\epsilon}_\theta$ the learned noise predictor. The expectation samples targets, conditions, noise levels, and Gaussian noise. This is a conditional adaptation of denoising diffusion training (Ho et al., 2020). The physical timestep remains distinct from j. Predicting clean data or a different noise-data combination changes the parameterization and weighting, not the underlying requirement to learn a conditional distribution.

A simple flow-matching construction uses the following linear data-to-noise path

$$\begin{aligned} \mathbf{x}^{\sigma_{\text{flow}}} &= (1-\sigma_{\text{flow}})\mathbf{x} + \sigma_{\text{flow}}\boldsymbol{\epsilon}, \qquad \sigma_{\text{flow}} \sim \text{Unif}[0,1], \\ \mathcal{L}_{\text{FM}} &= \mathbb{E}_{\mathbf{x},\kappa,\sigma_{\text{flow}},\boldsymbol{\epsilon}}\left[\left\| \mathbf{v}_\theta^{\text{flow}}(\mathbf{x}^{\sigma_{\text{flow}}}, \sigma_{\text{flow}}, \kappa) - (\boldsymbol{\epsilon} - \mathbf{x}) \right\|_2^2 \right], \end{aligned} \tag{5.10}$$

where σ_{flow} is continuous artificial time, $\text{Unif}[0,1]$ the uniform distribution, and $\mathbf{v}_\theta^{\text{flow}}$ a learned velocity field in the target space. The target velocity is $\boldsymbol{\epsilon} - \mathbf{x}$ under this convention. Sampling starts at noise and integrates from $\sigma_{\text{flow}} = 1$ toward 0. Reversing the path convention reverses the velocity sign. This is a representative conditional flow-matching objective, not a unique choice of path or solver (Lipman et al., 2023).

The target $\mathbf{x}$ can be a future-video latent, an action chunk, or a structured physical quantity. Video and action branches can have separate noise levels and masks, as in UWM (Zhu et al., 2025). This allows examples with missing action labels to train the video objective without fabricating control supervision. Tokenization and normalization determine how the squared loss weights different channels. An angular coordinate, translation, and contact force should not compete merely according to their raw numerical scales. Fewer inference steps lower cost but introduce a solver and approximation trade-off that must be measured at the task interface.

Stochastic State-Space Representation Learning A generative latent model must learn both an observation-informed posterior and a prior that can advance without new observations. For clarity, consider a fixed initial latent z_0, a Markov prior $p_\theta(z_t \mid z_{t-1}, a_{t-1}, c)$, an observation likelihood $O_{\omega_{\text{obs}}}(o_t \mid z_t, c)$, and a filtering variational distribution $q_\phi(z_t \mid z_{t-1}, h_t, c)$. A representative sequence objective is

$$\begin{aligned} \mathcal{L}_{\text{SSM}} = \mathbb{E}_{\mathcal{D}} \sum_{t=1}^{N_{\text{step}}} \Big[& \mathbb{E}_{q_\phi(z_{1:t} \mid h_t, c, z_0)}\left[-\log O_{\omega_{\text{obs}}}(o_t \mid z_t, c) \right] \\ & + \beta_{\text{KL}}\, \mathbb{E}_{q_\phi(z_{1:t-1} \mid h_{t-1}, c, z_0)} D_{\text{KL}}\big(q_\phi(z_t \mid z_{t-1}, h_t, c) \,\|\, p_\theta(z_t \mid z_{t-1}, a_{t-1}, c) \big) \Big]. \end{aligned} \tag{5.11}$$

The variational sequence distribution is the product of its filtering factors. D_{KL} is Kullback-Leibler divergence, and β_{KL} weights prior-posterior agreement. Elapsed times are suppressed in c. With $\beta_{\text{KL}} = 1$, this is the negative evidence lower bound for the stated model, conditional on z_0. Learning an uncertain initial state adds its corresponding initial term. Other β_{KL} values yield a regularized objective rather than the same exact likelihood bound. The construction combines variational learning with sequential latent dynamics (Kingma and Welling, 2014; Hafner et al., 2019).

The reconstruction term trains the encoder and decoder to retain evidence. The divergence trains the prior to match the observation-informed posterior while limiting information that the prior cannot predict. Reparameterized sampling allows gradients through continuous latent variables. In practice, balancing or stopping selected gradient paths, protecting a minimum latent information

budget, and adding reward or event targets can stabilize learning. Their details should be reported, because identical symbolic losses can optimize different parameters under different gradient rules. A deterministic recurrent memory may be included alongside the stochastic state, provided its dependencies are included in the model definition.

Predicting Target Representations For a context encoder E_ϕ, predictor P_θ, and target encoder $E_{\bar{\phi}}$, a generic joint-embedding objective is

$$\mathcal{L}_{\text{repr}} = \mathbb{E}_{\mathcal{D},\mathcal{M}_{\text{mask}}}\left[d_{\text{feat}}\Big(P_\theta(E_\phi(x_{\text{vis}}), \mathcal{M}_{\text{mask}}, \kappa), \text{sg}[E_{\bar{\phi}}(x_{\text{target}})]\Big)\right], \tag{5.12}$$

where $\mathcal{M}_{\text{mask}}$ specifies target positions or masked regions, x_{vis} the visible context, x_{target} the withheld target, d_{feat} a feature distance, and sg the stop-gradient operator. The target parameters $\bar{\phi}$ can be frozen or updated by an exponential moving average of ϕ, rather than by backpropagation through this loss. I-JEPA and V-JEPA 2 instantiate related predictive-embedding mechanisms at different spatial and temporal scales (Assran et al., 2023; 2025).

The predictor and context encoder learn to make retained target information predictable. Stop-gradient alone is not a universal proof against collapse. The target update, masking, architecture, and representation statistics must work together. A collapsed representation can achieve a trivial small distance, and a semantic target can discard fine motion. Comparing downstream readouts, representation variation, and controlled physical predictions is therefore necessary. The main trade-off concerns which information is retained, how expensive the target is to produce, and which later queries the representation supports.

From Masked JEPA Pretraining to Causal Latent Dynamics A typical training iteration first selects a clip and its visible and target regions. The context encoder receives only the visible content. The target encoder processes the target evidence, possibly with the wider context specified by the method, and produces the features to be matched. The predictor is conditioned on the target positions and, when the task requires them, actions and elapsed time. After the student update, an exponential-moving-average teacher can be updated as

$$\bar{\phi} \leftarrow m_{\text{EMA}}\bar{\phi} + (1 - m_{\text{EMA}})\phi, \qquad 0 \le m_{\text{EMA}} < 1, \tag{5.13}$$

where ϕ is the updated student-encoder parameter vector, $\bar{\phi}$ the teacher vector, and m_{EMA} the momentum coefficient. The teacher is not updated by the predictive-loss gradient. A larger m_{EMA} changes targets more slowly but can leave them behind a rapidly adapting student. This target-update mechanism is used in the JEPA lineage (Assran et al., 2023; Bardes et al., 2024). Freezing an independent pretrained teacher is a different design.

For causal dynamics learning, the visible context must stop at the forecast origin, and the predictor must associate a proposed control sequence with later target features. V-JEPA 2's action-conditioned post-training illustrates this additional step (Assran et al., 2025). Bidirectional pretraining remains useful, but its masked-completion score should not be relabeled as open-loop prediction accuracy. A goal-conditioned planner also needs a meaningful discrepancy between predicted and goal features and a valid action-search procedure. Representation quality, transition quality, and search quality are separately testable.

Feature regression has a statistical limit worth making explicit. With a fixed target representation and squared error, the population-optimal deterministic prediction is its conditional mean. With scalar absolute error, it is a conditional median. Neither objective by itself recovers multiple modes or assigns probabilities to them. A scene with two plausible future contact states can therefore require a stochastic latent predictor, multiple hypotheses, or a probabilistic event head. The appropriate objective depends on whether the downstream consumer needs a reusable feature, a most likely outcome, or a calibrated distribution. This applies to JEPA-style targets and to predictive-feature distillation in world-action models alike.

5.3.2 *Action-Conditioned Dynamics and World-Action Learning*

Actions connect prediction to intervention, but an action record must have a precise meaning. Distinguish a proposed command, the command actually applied after clipping or safety filtering, and the subsequent measured physical response. A deployable forward model can condition on the applied input or model the actuator transformation from a proposed command. It must not require the future measured response as an input to predict that same response. Controller mode, reference frame, units, duration, delays, and saturation belong in the record. The notation a_t refers to the declared input interface, not an unspecified mixture of command and consequence.

Forward, Inverse, and Behavioral Targets A forward model predicts the next observation or state from current evidence and an input. An inverse model predicts an input compatible with two states or observations. A policy predicts an input from current evidence and a goal. These targets are related by the joint distribution in Eq. (5.7), but they are not equivalent. Several actions can reach the same endpoint, and matching endpoints can hide different contact forces or unsafe intermediate states. Inverse-dynamics training therefore needs a multimodal output distribution or an explicit criterion for selecting among compatible actions.

The three targets correspond to the three world-action interfaces in the right panel of Figure 5.3, defined as attention masks over observed-context tokens C, action tokens A, and future-observation tokens F. A *forward dynamics model* (FDM) predicts $\widehat{\mathrm{F}}$ from C and A. Given what is observed and a candidate input, it forecasts the consequence, which is exactly the transition query of Figure 5.1 and the rollout mechanism used in planning and imagination. An *inverse dynamics model* (IDM) predicts $\widehat{\mathrm{A}}$ from C and F. It reads how the scene actually changed and infers a compatible input. An IDM that requires the realized future observation cannot use that observation to choose the preceding action online. In the training interface considered here, it labels recorded interaction with action estimates, recovers latent actions from action-free video, and shapes representations that encode how actions change the world. Even here, only the action query may read the future. The context rows of the inverse mask bar C from attending to F, so context features, which may be cached or shared with the policy mode, remain deployable summaries of observed evidence rather than carriers of future information. A *policy* mode predicts $\widehat{\mathrm{A}}$ from C alone, possibly with a goal encoded in the context. The future-to-action attention path is therefore masked out entirely (the crossed-out cell in Figure 5.3). A single leaky edge would let action tokens read future observations at training time and silently lose that input at test time.

Sharing one network across the three modes is attractive because the modes provide complementary supervision. The FDM grounds what actions do, the IDM grounds which actions produce observed change, and the policy converts both into executable behavior. The cost is discipline in the masks and in gradient routing. A stop-gradient blocks a parameter update but not a forward information path, so frozen and detached components must be distinguished from masked ones, and a deployment-mode forward pass with all training-only branches absent is the decisive test that the policy mode's inputs are exactly those available at runtime.

Behavior cloning learns demonstrated actions, using a likelihood, regression, diffusion, or flow objective. Diffusion Policy exemplifies a conditional distribution over action sequences rather than a single independent action at each step (Chi et al., 2023). A failed action can remain useful for a forward model and an outcome predictor while being excluded from a nominal imitation objective. Reward, termination, contact-event, and task-progress heads can make a state more useful for decisions, but they supervise different quantities. A progress label is not a value target unless its connection to future return has been established.

Multi-Source and Multi-Objective Optimization A useful way to document joint training is

$$\mathcal{L}_{\text{joint}}(\Theta) = \sum_{i_{\text{src}}=1}^{N_{\text{src}}} \rho_{i_{\text{src}}} \, \mathbb{E}_{\xi_{\text{sample}} \sim \mathcal{D}_{i_{\text{src}}}} \left[\sum_{j \in \mathcal{J}} \lambda_j \, m_j(\xi_{\text{sample}}) \, \ell_j(\Theta; \xi_{\text{sample}}) \right], \tag{5.14}$$

where N_{src} is the number of data sources, $\mathcal{D}_{i_{\text{src}}}$ the distribution of source i_{src}, $\rho_{i_{\text{src}}} \geq 0$ its sampling weight with $\sum_{i_{\text{src}}} \rho_{i_{\text{src}}} = 1$, and ξ_{sample} a sampled training record. The set $\mathcal{J}$ indexes enabled objectives, ℓ_j their per-example losses, $\lambda_j \geq 0$ their weights, and $m_j(\xi_{\text{sample}}) \in \{0, 1\}$ indicates whether the target exists and is appropriate. The parameter collection is Θ. Per-dimension masks and normalization by valid target counts are applied inside ℓ_j when needed. This equation is a bookkeeping template, not an assertion that the objectives have one common probabilistic interpretation.

The distinction between sampling weights and loss weights matters. Increasing a robot source's sampling rate increases all of its enabled objectives, whereas increasing an action-loss coefficient changes the balance within sampled records. Loss scales also depend on token counts and units. A long video target can dominate a short action chunk unless normalization is deliberate. Joint training can share useful structure but can also let appearance gradients erase precise control information or let narrow action data degrade broad semantics. Freezing pretrained modules, staged unfreezing, separate experts, restricted gradient paths, and alternating updates offer different compromises between preservation and adaptation.

Future Supervision Without Future-Input Leakage Future observations are legitimate training targets. The restriction is on the forward information available to the deployed policy. A predictive feature branch can map present history to future-relevant features and align them with a detached future teacher. Alternatively, a frozen generative model can receive learned conditioning and send a loss gradient back through that conditioning while its own weights remain fixed. Freezing weights is not the same as disabling the gradient with respect to the model's input. InternVLA-A1.5 uses this latter mechanism to train foresight conditioning (Ma et al., 2026a).

InternW0-Δ uses latent-change supervision and detached future-feature alignment for Causal Imprint, alongside training-only 4D teacher targets (Miao et al., 2026). The general lesson is to document every route by which future information could reach action activations, including shared attention, side conditioning, cached features, and preprocessing. A direct mask between future and action tokens is insufficient if an intermediate token first reads the future and then passes it onward. A detached target blocks a gradient path. A causal attention restriction blocks a forward information path. Both can be required, but they solve different problems. Temporal "causality" of a mask also does not by itself establish causal identification of the physical system.

A Worked Multi-Supervision Construction World-action training becomes clearer when each auxiliary target is written separately. InternW0-Δ provides a concrete example with video and action flow losses, clean-latent change regression, future-feature alignment, and training-only 4D distillation (Miao et al., 2026; Shanghai AI Laboratory, 2026). The following normalized formulation explains their roles using this chapter's notation. It is not a prescription for the paper's exact layer indices, tensor layout, or loss coefficients.

Let u_t^{vid} be a clean video-autoencoder latent for the current observation, and let $U_t^+ = (u_{t+\tau_{\text{stride}}}^{\text{vid}}, \ldots, u_{t+N_{\text{slice}}\tau_{\text{stride}}}^{\text{vid}})$ contain N_{slice} future latent slices with sampling stride τ_{stride}. The autoencoder is frozen in this construction. The action chunk is A_t^{seq}. Let $q_t = P_\theta^{\text{q}}(h_t, g, c)$ denote predictive tokens computed only from observed history, instruction, and known conditions. A readout $\widehat{D}_t$ from these tokens is trained against consecutive clean-latent differences given by

$$
\begin{aligned}
D_{t,i} &= u_{t+i\tau_{\text{stride}}}^{\text{vid}} - u_{t+(i-1)\tau_{\text{stride}}}^{\text{vid}}, \qquad i = 1, \ldots, N_{\text{slice}}, \\
D_t &= [D_{t,1}; \ldots; D_{t,N_{\text{slice}}}], \qquad \mathcal{L}_{\text{change}} = \mathbb{E}\left[\frac{\|\widehat{D}_t - \text{sg}[D_t]\|_2^2}{d_{\text{D}}}\right],
\end{aligned}
\tag{5.15}
$$

where brackets denote stacking, d_{D} is the number of valid scalar target entries, and the expectation averages training records. The norm is the Euclidean norm of the concatenated valid scalar latent entries. The readout includes any output reshaping or projection needed to match dimensions. Padded entries are omitted. A latent difference emphasizes change rather than static appearance,

but it is not a physical velocity. Its units are feature units, and the sampling interval still matters. Computing differences before noise injection avoids supervising independent diffusion noise as if it were motion. Camera changes and encoder instability can also create latent differences, so this objective needs complementary semantic or geometric checks.

A second target can transfer richer intermediate features. Let $\mathbf{q}_{t,i_{\text{tok}}}^{(\ell_{\text{s}})}$ be a predictive token at student layer ℓ_{s} and spatial position i_{tok}, and let $\mathbf{f}_{t,i_{\text{tok}}}^{+,\ell_{\text{T}}}$ be a spatially corresponding feature of a future slice at target layer ℓ_{T}. With compatible feature dimensions, a representative alignment objective is

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{t,i_{\text{tok}}}\left[1 - \cos_{\text{sim}}\left(\mathbf{q}_{t,i_{\text{tok}}}^{(\ell_{\text{s}})}, \text{sg}[\mathbf{f}_{t,i_{\text{tok}}}^{+,\ell_{\text{T}}}]\right)\right]. \tag{5.16}$$

The function $\cos_{\text{sim}}(\mathbf{u},\mathbf{v}) = \mathbf{u}^{\top}\mathbf{v}/(\|\mathbf{u}\|_2\|\mathbf{v}\|_2)$ measures direction similarity for nonzero feature vectors, with numerical stabilization used in implementation. Layer indices and spatial correspondence are part of the target definition. The future feature can come from another branch of the same trainable model. Detaching this target prevents this loss from updating it, while other objectives may still update its parameters. Such a target is not the same as a permanently frozen teacher. Compared with clean-latent regression, feature alignment transfers the target branch's abstraction but inherits its biases and changes during training.

An external geometry-and-motion teacher supplies a third, complementary signal. Let $\mathbf{r}_t^{\text{T}} = R_{\text{T}}(o_{t:t+H_{\text{o}}}, c)$ be a descriptor extracted from a recorded clip and $\mathbf{r}_t^{\text{S}} = R_\theta^{\text{S}}(h_t, g, c)$ the student descriptor predicted from available observations. The horizon H_{o} is the observation horizon. R_{T} is frozen and may access training-only future frames, while R_θ^{S} may not. A corresponding objective is

$$\mathcal{L}_{\text{geom}} = \mathbb{E}_{\text{valid}}\left[\frac{\|\mathbf{r}_t^{\text{S}} - \text{sg}[\mathbf{r}_t^{\text{T}}]\|_2^2}{d_{\text{r}}}\right], \tag{5.17}$$

where d_{r} is the descriptor dimension and the expectation includes only samples with valid teacher outputs. Descriptor components should be normalized according to their semantics. Unlike a hard physical constraint, this objective transfers an estimated geometric or motion prior. It does not establish exact metric geometry, correct forces, or conservation. The auxiliary student readout can be removed after training when its role was only to shape a representation already used by the policy.

The combined objective is then

$$\begin{aligned}\mathcal{L}_{\text{WAM}} = \lambda_{\text{v}}\mathcal{L}_{\text{FM}}^{\text{v}}(U_t^{+}) + \lambda_{\text{a}}\mathcal{L}_{\text{FM}}^{\text{a}}(A_t^{\text{seq}}) + \lambda_{\text{ch}}\mathcal{L}_{\text{change}} \\ + \lambda_{\text{al}}\mathcal{L}_{\text{align}} + \lambda_{\text{ge}}\mathcal{L}_{\text{geom}}.\end{aligned} \tag{5.18}$$

The superscripts v and a identify separately conditioned video and action flow objectives of Eq. (5.10). The nonnegative λ coefficients balance their contributions. Source-specific weights and validity masks follow Eq. (5.14). This combination assigns distinct jobs to reconstructable future detail, executable behavior, change prediction, and teacher knowledge. Ablations should remove or vary these jobs independently, matching data and optimization budgets, rather than comparing only the total loss.

A limitation follows from the conditioning. When q_t does not receive a candidate action sequence, it predicts future-relevant information under the behavior represented in its data, not the separate outcome of every possible intervention. Its value can be improved action features without a standalone counterfactual simulator. Joint video-action training should therefore declare whether its video branch sees supplied actions, generated actions, or neither. The behavioral benefit of future supervision and the correctness of action-conditioned physical prediction are different claims.

Parameters, Information Paths, and a Training Iteration Table 5.4 distinguishes the pathways in this construction. Observation encoders, frozen semantic models, detached targets, and trainable

world experts play different roles even when their outputs are all called features. A forward dependency restriction concerns the information available to activations. A stop-gradient concerns which parameters a loss updates. Freezing a module's weights does not necessarily stop derivatives through its computation to an upstream trainable input.

Table 5.4 Information and optimization roles in predictive-feature world-action training. This is a general implementation checklist, not a claim that all world-action models use identical branches

Branch or target	Training information and updates	Deployment role
Observed-context features	Present/past observations Backbone updated if trainable	Available context May be cached
Frozen semantic conditioning	Current observations and instruction Encoder weights fixed	Retained when the action head depends on it
Future video branch	Noised future target and allowed context Video loss updates its expert	Optional Omitted in observation-only predictive-feature execution
Predictive change tokens	Observed context only Future outcomes provide losses, not inputs	Retained as action conditioning
Detached future features	Future branch supplies targets This alignment loss cannot update the target path	Not required as realized future inputs
External geometry teacher and auxiliary readout	Recorded clip targets Student learns and teacher stays fixed	Both removable if used only for distillation
Action expert	Noised actions and deployment-valid context Action loss trains the policy path	Generates the executable action chunk

One minibatch update proceeds as follows. Construct a time-consistent observed context, action chunk, future targets, and validity masks. Encode frozen inputs and retrieve any valid cached teacher descriptors. Noise the video and action targets according to their declared schedules, while keeping the observed conditioning and clean change targets separate. Run the experts with the intended directed dependencies, compute only the supported losses, normalize them by valid entries, and update the trainable parameter groups. If a moving-average teacher is used, update it after the student step. Finally, test an inference-mode forward pass with all training-only future and teacher branches absent. The policy must still have every input it requires.

This last test should inspect indirect as well as direct paths. An observed token that reads a future token at one layer can leak that future to the policy at a later layer even when future-to-action attention is masked. Temporal preprocessing can leak in the same way. Encoding an observed frame together with future frames through a noncausal video encoder can contaminate its latent. Cache reuse additionally requires independence from the evolving action and artificial-time inputs, not merely independence from future observations. These checks turn a verbal claim of "training-only supervision" into an auditable computation graph.

Learning a World Model Versus Learning Within One Model-based reinforcement learning adds a second optimization problem in which a policy is improved using transitions generated by a learned model. In Dreamer, imagined trajectories train behavior while the learned dynamics provide the rollout mechanism (Hafner et al., 2020). Model parameters can be held fixed during policy optimization even when derivatives through the model are used to update policy parameters. Conversely, updating the model to maximize its own predicted reward would not necessarily improve its physical accuracy.

Dyna-style alternation uses real experience to improve the model and model-generated experience to improve decisions (Sutton, 1991). MBPO highlights a useful compromise by using short model rollouts branched from real states, limiting reliance on distant model predictions (Janner

et al., 2019). Longer imagination expands possible credit assignment but exposes the policy to compounding errors and model exploitation. Offline data create an additional support problem. A proposed policy may select actions absent from the training distribution. Policy improvement should therefore be coupled to rollout-length selection, conservative use of uncertain predictions, renewed real data, and independent physical evaluation rather than judged only by imagined return.

5.3.3 Physical Constraints, Counterfactuals, and Uncertainty Modeling

Physical structure can enter through loss penalties, architectural restrictions, or a hybrid of known dynamics and learned corrections. Physics-informed neural networks demonstrate the use of governing-equation residuals as learning signals (Raissi et al., 2019). Structured simulators illustrate complementary restrictions arising from a representation and its update rules (Sanchez-Gonzalez et al., 2020; Xie et al., 2024). The appropriate choice depends on which laws, parameters, and boundary conditions are actually known.

Soft Penalties and Structural Constraints For a predicted physical trajectory $\widehat{S} = (\widehat{s}_t, \ldots, \widehat{s}_{t+H})$ under actions A_t^{seq}, a representative penalty is

$$\mathcal{L}_{\text{phys}} = \mathbb{E}\left[\sum_{j=1}^{J_{\text{eq}}} \alpha_j^{\text{eq}} \|f_j^{\text{eq}}(\widehat{S}, A_t^{\text{seq}}, c)\|_2^2 + \sum_{j=1}^{J_{\text{ineq}}} \beta_j^{\text{ineq}} \|[g_j^{\text{ineq}}(\widehat{S}, A_t^{\text{seq}}, c)]_+\|_2^2\right]. \tag{5.19}$$

The residual $f_j^{\text{eq}} = 0$ specifies an equality, $g_j^{\text{ineq}} \leq 0$ an inequality, $[u]_+ = \max(u, 0)$ applied componentwise, and $J_{\text{eq}}, J_{\text{ineq}}$ the numbers of each constraint. The nonnegative coefficients $\alpha_j^{\text{eq}}, \beta_j^{\text{ineq}}$ include suitable scaling by units and tolerances. The expectation averages training records and sampled predictions. These coefficients are local to this equation. Constraints must act on physical readouts or physically defined latent variables, not on arbitrary embedding coordinates without an interpretation.

Examples include joint limits, nonpenetration, contact consistency, and energy or mass balance under known boundary fluxes. A system exchanging heat or liquid with its surroundings should obey a balance equation, not an inappropriate constant-energy or constant-mass penalty. Tolerances should reflect measurement precision and modeling assumptions. A strong incorrect constraint biases the model. A weak penalty may leave violations large. Enforcing symmetries or integrating known mechanics can reduce violation by construction, but only for the constrained property and under the numerical method's assumptions. It cannot identify an unmeasured friction coefficient or certify all downstream behavior.

Counterfactual and Intervention Supervision Paired rollouts can compare different actions from matched initial conditions. A resettable simulator can hold state and external inputs fixed while varying an intervention. In a physical experiment, the same comparison requires reproducible initial conditions, controlled confounders, or a justified statistical design. Outcome diversity from an unconditioned video generator is not evidence that it learned how a particular action changes the outcome. Synthetic counterfactuals supervise the simulator's mechanisms. Independent measurements are needed before transferring those conclusions to reality.

A useful training contrast varies the commanded magnitude, direction, or duration while keeping nonintervened conditions comparable. It can supervise the sign and scale of the response as well as which objects should remain unchanged. Care is required near discontinuities. A slightly larger force may change sticking to sliding rather than yield a slightly larger displacement. A smooth penalty that averages these modes can suppress the event the model is supposed to predict.

Predictive Distributions and Uncertainty Sources Deterministic regression is cheap but cannot express multiple plausible outcomes. A conditional Gaussian supplies a mean and variance. Mixtures, stochastic latent states, diffusion, or flows can represent more complex distributions. For a

scalar measured target y and condition κ, a Gaussian negative log-likelihood, omitting an additive constant, is

$$\mathcal{L}_{\text{Gauss}} = \frac{1}{2}\mathbb{E}_{\mathcal{D}}\left[\frac{(y-\mu_\theta(\kappa))^2}{\sigma_\theta^2(\kappa)} + \log\sigma_\theta^2(\kappa)\right], \tag{5.20}$$

where μ_θ is the predicted conditional mean and $\sigma_\theta^2 > 0$ the variance. The residual term penalizes inaccurate concentrated predictions, while the logarithmic term discourages arbitrarily inflating variance. Positivity can be enforced through a log-variance parameterization with numerically appropriate bounds. This example illustrates heteroscedastic likelihood learning (Kendall and Gal, 2017). Its Gaussian assumption is inadequate when distinct contact modes produce well-separated outcomes.

An ensemble adds variation across fitted models. For M equally weighted scalar predictors with means μ_m and variances σ_m^2 at the same condition, the mixture variance obeys

$$\bar{\mu} = \frac{1}{M}\sum_{m=1}^{M}\mu_m, \qquad \operatorname{Var}(Y \mid \kappa) = \frac{1}{M}\sum_{m=1}^{M}\sigma_m^2 + \frac{1}{M}\sum_{m=1}^{M}(\mu_m - \bar{\mu})^2, \tag{5.21}$$

where Y is the predicted random quantity and m the ensemble index. The identity separates within-model dispersion from between-model disagreement. Deep ensembles and probabilistic ensemble dynamics use this kind of diversity for predictive uncertainty and planning (Lakshminarayanan et al., 2017; Chua et al., 2018). However, disagreement is only a proxy for epistemic uncertainty, and the within-model term can include unresolved state ambiguity as well as observation and process noise. Correlated model errors can leave an ensemble confidently wrong.

Uncertainty must also propagate consistently across time. A hidden but fixed mass should be sampled once for a trajectory hypothesis rather than independently every step. Sensor noise may vary at each observation. More detailed distributions increase sampling and calibration costs. Deterministic models can still be useful when their operating range is narrow and externally monitored, but they should not be presented as calibrated probabilistic simulators. Section 5.4.3 explains how uncertainty is evaluated rather than inferred from the mere presence of stochastic layers.

5.3.4 Multi-Step Prediction, Multiple Time Scales, and Adaptation

One-step accuracy can hide recursive drift. A generic horizon-weighted objective is

$$\mathcal{L}_{\text{multi}} = \mathbb{E}_{\mathcal{D}}\left[\sum_{k=1}^{H} w_k\,\ell\left(\widehat{y}_{t+k|t}, y_{t+k}\right)\right], \qquad w_k \geq 0. \tag{5.22}$$

The prediction $\widehat{y}_{t+k|t}$ is generated from evidence available at t, recursively applying the supplied actions without incorporating intermediate real observations. The target y_{t+k} is a later measurement or explicitly identified teacher target. w_k weights horizon k, H is the maximum horizon, and ℓ is a suitable regression or probabilistic loss. A probabilistic $\widehat{y}$ denotes a distribution, not necessarily one sampled value. Supervision should score that distribution appropriately rather than force every sample to match one realized future.

Teacher Forcing, Free Rollout, and Curriculum Teacher forcing provides stable local supervision but exposes the model mainly to data states. Free rollout trains on its own evolving predictions and addresses the deployment distribution more directly, at the cost of unstable early trajectories and longer gradient paths. Scheduled sampling mixes real and generated preceding inputs (Bengio et al., 2015). It is a training heuristic, not generally the original sequence-likelihood objective. A sensible comparison therefore includes teacher-forced error, open-loop error, and task behavior rather than assuming that mixed inputs always improve all three.

Latent overshooting in PlaNet extends consistency pressure beyond one transition by matching multi-step predictive distributions to observation-informed states (Hafner et al., 2019). Related multi-step objectives can supervise physical readouts or future embeddings. Longer windows

improve exposure to delayed effects, but require more memory and can overweight a particular target encoder's errors. A curriculum can begin with short horizons and then include longer windows and contact-centered segments, while preserving enough ordinary motion to avoid a distorted training distribution. Truncating backpropagation lowers cost but changes which temporal dependencies receive direct gradient signals.

Multiple Timescales and Temporal Consistency Different sensors and physical variables evolve at different rates. Training should include elapsed-time conditioning, modality-validity masks, and action durations rather than assuming one frame equals one control interval. Slow semantic context, faster visual dynamics, and high-rate force feedback can share state through explicit timing interfaces. Interpolation is appropriate only when its assumptions hold. Interpolating across an impact can erase the discontinuity.

For a Markov transition kernel under the same constant control input a and conditions c, time consistency takes the form

$$T_\theta(z'' \mid z, a, c, \Delta_1 + \Delta_2) \approx \int T_\theta(z'' \mid u, a, c, \Delta_2) T_\theta(u \mid z, a, c, \Delta_1)\, \mathrm{d}u, \tag{5.23}$$

where z, z'' are initial and final latent states, u is an intermediate latent state, and $\Delta_1, \Delta_2 > 0$ are physical durations. This is a distribution-level Chapman-Kolmogorov consistency condition under the stated Markov and fixed-input assumptions. It is not generally valid to replace the intermediate distribution by its mean in nonlinear dynamics. If actions or external conditions change, both sides must use the same time-varying input schedule. If the compressed state is not Markov, residual inconsistency may reveal missing memory rather than numerical error.

Post-Training, Distillation, and Continued Interaction Task-specific post-training can refine action interfaces, contact targets, or selected dynamics. InternW0 adds available force and tactile histories and predicts future contact channels during contact-aware post-training (Cai et al., 2026b). Such adaptation requires valid contact supervision. Missing physical channels cannot be recovered simply by adding a new output head. Distillation can transfer an expensive teacher's features or predictions to a smaller or less frequently executed branch, but the teacher's uncertainty and errors should remain distinguishable from measured truth.

Local adaptation should separate belief correction, physical-parameter identification, and updates to learned network weights. A failed grasp may revise a contact hypothesis without changing the model. Repeated measured slip may justify identifying friction. Systematic errors across conditions may require a new training round. Replay, regularization, and selective parameter updates address the trade-off between new learning and retention. Elastic weight consolidation is one example of limiting changes to parameters important for previous tasks (Kirkpatrick et al., 2017), not a guarantee against forgetting in an arbitrary world model. New versions should be accepted only after tests of both new and retained capabilities.

Staging Auxiliary Supervision and Adapting the Deployed Interface A multi-objective recipe can be organized around a stable behavioral and predictive baseline rather than enabling every target from the first update. InternW0-Δ first performs joint pretraining and subsequently adds cached 4D supervision during continued pretraining, before task- and embodiment-specific post-training (Miao et al., 2026). The general choice is between early joint shaping and later auxiliary refinement. Early auxiliary losses can guide representation formation, but noisy or mismatched teachers can interfere before the action interface becomes useful. Later introduction reduces that early interference but may require a smaller learning rate or replay to avoid disturbing established behavior. Neither schedule is a universal optimum.

Post-training should match the actual deployment contract. A target robot may change valid action dimensions, normalization, camera configuration, actuation delay, or the relation between predicted and executed chunk lengths. Adapt the relevant input/output mappings and condition the model on these differences. Do not reinterpret an identical numerical vector as an identical physical action. If successive chunks overlap or part of a chunk has already been committed to execution,

the training condition should identify that prefix and the loss should target the remaining controllable portion. Where online feedback is used to choose future actions, it must be separated from the fixed action sequence assumed by an open-loop dynamics objective.

Frozen-encoder and teacher caches can reduce repeated training work, but only while their dependencies remain fixed. Changing an encoder checkpoint, a crop, a view layout, an instruction, or the temporal window can invalidate a cached feature. Offline geometric descriptors should retain their source clip, teacher version, confidence, and normalization. Activation checkpointing trades extra computation for memory. Mixed precision reduces storage and arithmetic cost but needs checks for unstable losses and gradients. Report these implementation choices separately from reductions in the amount of training data or the number of optimization steps. A faster experiment pipeline is valuable, but it is not itself evidence of better learned dynamics.

For model selection, monitor the component losses alongside open-loop predictions and closed-loop behavior. A lower teacher-feature error can coexist with worse action accuracy if the representation discards contact detail. Likewise, a lower action loss on demonstration states can conceal failure on states visited by the policy. Compare a frozen baseline, a joint-training variant, and an auxiliary-supervision variant on fixed held-out conditions, with unchanged target preprocessing and comparable budgets. This distinguishes the contribution of predictive supervision from a larger backbone, a different data mixture, or additional optimization.

Training Trade-Offs and Reproducible Reporting Table 5.5 summarizes choices that should be reported together rather than hidden inside a total loss. At minimum, an experiment should state its data mixture and splits, temporal sampling, target normalization, trainable and frozen modules, gradient paths, optimizer and learning-rate schedule, effective batch size, sequence length, number of updates, numerical precision, and training and inference budgets. No fixed recipe is optimal across video generation, latent planning, and multimodal control.

The common criterion is whether the resulting model answers the intended queries on evidence it could actually receive. Training losses are useful diagnostics, but the relationships among reconstruction, dynamics, uncertainty, and behavior must be established through the validation protocols below.

Table 5.5 Training choices and their principal trade-offs. Each comparison requires matched data or compute where feasible, plus evaluation on the actual prediction or control interface

Choice	Potential benefit	Cost or failure to check
Observation vs. feature targets	Detail and query breadth vs. compact predictive abstraction	Expensive nuisance modeling vs. omitted physical information
Frozen vs. jointly updated modules	Prior preservation and cheaper optimization vs. task adaptation	Restricted adaptation vs. objective interference and forgetting
Short vs. long rollout training	Stable local learning vs. delayed-effect and drift supervision	Myopic accuracy vs. memory cost and difficult optimization
Deterministic vs. distributional models	Low overhead vs. multiple outcomes and probabilistic queries	Hidden ambiguity vs. calibration and sampling cost
Soft vs. structural physics	Flexible constraints vs. restricted admissible predictions	Residual violations vs. bias from incorrect assumptions
Offline vs. interaction-driven updates	Reproducible reuse vs. coverage of policy-induced failures	Unseen-action errors vs. physical cost and retention risk

5.4 Using and Validating World Models

Using a world model means making a query under a declared information boundary and, when appropriate, using its answer to choose an intervention. Validation should separate the quality of a forecast from its physical consistency and from the utility of the decision it supports. A model

can produce physically possible but inaccurate futures, accurate short-term forecasts that do not improve a policy, or useful actions without exposing a general-purpose predictive interface. This section develops protocols that make these distinctions measurable.

5.4.1 Interactive Simulation and Conditional Queries

An interactive call starts from an inferred state, belief, or observation history and returns one or more conditional futures. A query may request an image, object trajectory, contact event, temperature, boundary crossing, or task outcome. It should specify the available evidence, intervention interface, physical horizon, environment conditions, and required precision. For a partially known scene, the returned uncertainty should include unresolved state alternatives, not only randomness introduced during decoding.

State Posterior, Future Prediction, and Retrospective Checking The state posterior summarizes evidence available up to the prediction time. A future forecast propagates that posterior without using the future measurements to be predicted. For an observation-generating latent model, a representative factorization is

$$\begin{aligned} &p_{\theta,\phi,\omega_{\text{obs}}}(o_{t+1:t+H} \mid h_t, A_t^{\text{seq}}, c) \\ &\quad = \int q_\phi(z_t \mid h_t, c) \prod_{k=0}^{H-1} \left[T_\theta(z_{t+k+1} \mid z_{t+k}, a_{t+k}, c, \Delta t_k) O_{\omega_{\text{obs}}}(o_{t+k+1} \mid z_{t+k+1}, c) \right] \mathrm{d}z_{t:t+H}, \end{aligned} \tag{5.24}$$

where h_t is the observed history, $A_t^{\text{seq}} = a_{t:t+H-1}$ a supplied action sequence of H intervals, q_ϕ the inferred latent-state posterior, T_θ the learned transition, and $O_{\omega_{\text{obs}}}$ the observation likelihood. The integration marginalizes the current and future latent states $z_{t:t+H}$. Δt_k specifies each physical interval. The symbols $\phi, \theta, \omega_{\text{obs}}$ denote fixed fitted parameters in this expression. It is posterior prediction with respect to the current latent state, not necessarily a Bayesian average over uncertain network parameters. The Markov and observation assumptions are those introduced in Section 5.1.2. A model with additional dependencies must include them explicitly (Kaelbling et al., 1998; Hafner et al., 2019).

After real execution, the withheld observations can be used to check the forecast and then update the state. This *retrospective validation of an earlier prediction* is different from reconstructing a future frame after that frame has already entered an inference network. Likewise, smoothing a past state using later evidence can be useful for diagnosis but cannot be reported as an online forecast made with less information. Evaluation logs should retain the prediction, model version, conditioning evidence, and issue time before the target outcome is revealed.

Eq. (5.24) describes a query interface, not a requirement that every architecture evaluate an explicit density. Sample-based models can approximate the predictive distribution. Deterministic models return a restricted point prediction. A latent-only model can answer a physical or task query through a readout rather than an observation decoder. In all cases, a goal specifies what is desired and should not be treated as evidence that the goal will occur. Scenario assumptions, such as a proposed payload or a fixed external temperature, must remain distinguishable from measured conditions.

Interactive Correction and Information Acquisition Real feedback should update the inferred state before the next query. A prediction made before a grasp slips may no longer be a valid context for later actions even if its video remains visually coherent. Asynchronous systems should therefore expose the age and observation support of cached predictions, together with the conditions that trigger correction, regeneration, or rejection.

Predictions can also help choose what to observe next. If two action rankings differ mainly because contact is uncertain, an additional tactile measurement may be more useful than another distant camera frame. This is an information-acquisition decision within the partially observable control problem (Kaelbling et al., 1998). Its value depends on an observation model, the cost of sensing,

and whether the proposed measurement can actually resolve the ambiguity. Querying the same model repeatedly does not create independent evidence about the physical state.

5.4.2 Imagination, Planning, and Decision Calls

Planning proposes candidate actions, predicts their consequences, and ranks them using task objectives and constraints. An imagined collection of branches can be written as

$$\mathcal{C}_{\text{cand}} = \left\{ \left[\left(a_{t+k}^{(b)}, \widehat{z}_{t+k+1}^{(b)} \right) \right]_{k=0}^{K_b-1} : b = 1, \ldots, N_{\text{roll}} \right\}, \tag{5.25}$$

where $\mathcal{C}_{\text{cand}}$ is a collection of ordered imagined paths, N_{roll} is the number of candidate branches, b indexes a branch, and K_b is its number of intervals. The action $a_{t+k}^{(b)}$ produces the imagined next state $\widehat{z}_{t+k+1}^{(b)}$ from the preceding state in that branch. Branches may also be repeated under different state hypotheses or stochastic outcomes. Increasing N_{roll} explores alternatives. Increasing K_b reveals later consequences but also exposes more model error.

For candidates with a common horizon H, a useful illustrative objective is

$$U(A_t^{\text{seq}} \mid b_t, g, c) = \mathbb{E}_{\text{model}} \left[\sum_{k=0}^{H-1} \gamma^k \left(r_{t+k} - \lambda C_{t+k}^{\text{run}} \right) - \eta_{\text{pen}} \, \mathbf{1}\{\mathcal{V}_{t:t+H}^{\text{viol}}\} \,\middle|\, b_t, A_t^{\text{seq}}, g, c \right]. \tag{5.26}$$

The utility U scores the action sequence $A_t^{\text{seq}} = a_{t:t+H-1}$ starting from current belief b_t, goal g, and conditions c. The expectation averages the model's predicted state and outcome uncertainty. The transition reward r_{t+k} measures task benefit, $C_{t+k}^{\text{run}} \geq 0$ a per-interval resource cost such as time or energy, and $\gamma \in (0, 1]$ the discount factor for equally spaced intervals. The coefficient $\lambda \geq 0$ converts resource cost into utility units. The indicator $\mathbf{1}\{\mathcal{V}_{t:t+H}^{\text{viol}}\}$ equals one when a specified violation occurs anywhere along the trajectory. $\eta_{\text{pen}} \geq 0$ penalizes that trajectory-level event once. Both the reward and the per-step cost are inside the sum. For unequal durations, discounts and accumulated costs should instead follow physical time. This is a generic decision objective, not a world-model fitting loss or a safety certificate. Constrained MPC provides the broader control framework (Mayne et al., 2000).

A planner may optimize actions by sampling, tree expansion, or differentiating through a learned transition. PlaNet uses latent-space planning, while probabilistic ensemble methods such as PETS propagate alternative dynamics and state outcomes during action evaluation (Hafner et al., 2019; Chua et al., 2018). A separately trained actor may instead amortize this computation into direct action generation. The actor can be trained using imagined trajectories without running a planner at deployment, as in Dreamer (Hafner et al., 2020). Planning at inference and policy learning through imagination therefore have different runtime costs and failure modes.

Receding-horizon execution applies only an initial portion of the selected sequence and replans from new real evidence. Without this correction, action optimization may deliberately enter regions where prediction errors create unrealistically high reward. Longer imagined rollouts are not automatically better. They can change action ranking for the wrong reason. Model-based policy optimization addresses this issue by controlling the use and length of imagined experience (Janner et al., 2019). Uncertainty-sensitive ranking can compare candidate outcomes across hidden-state hypotheses, but agreement among several copies of a biased model is not independent confirmation.

Hierarchical imagination can compare coarse routes or process phases before refining a selected interval. Candidates should terminate at a common physical horizon or comparable completion event, with appropriate terminal-value treatment. Otherwise a shorter trajectory can appear preferable merely because its later costs are omitted. Efficient execution can also reuse observation features or invoke detailed prediction only near consequential events. These savings must be evaluated together with the error introduced by stale context, reduced branching, or omitted detail, rather than by model throughput alone.

Table 5.6 Validation objects, suitable evidence, and common overclaims. The evaluation protocol must preserve the information available at the forecast issue time

Object	Evidence or protocol	Example readout	Overclaim to avoid
Observation forecast	Held-out, time-matched outcomes Declared sample budget	Pixel or perceptual error, conditional and distribution checks	Overall realism establishes action correctness
State persistence	Occlusion, reappearance, and history changes	Identity errors Position, orientation, or attribute drift	Single-frame recognition establishes memory
Action response	Controlled changes and matched initial conditions	Response magnitude, direction, and event timing	Any action sensitivity establishes a correct intervention model
Physical consistency	Independent geometry, force, and process measurements	Penetration, contact consistency, justified balance residuals	Plausibility establishes quantitative accuracy or causality
Predictive probability	Repeated held-out outcomes and separate calibration data	Brier or CRPS, coverage and width, reliability by horizon	Diverse samples establish calibrated uncertainty
Decision utility	Matched-policy and resource comparisons on real outcomes	Task score, completion time, failures, and latency	Prediction gains establish better real decisions

5.4.3 Prediction Quality and Physical-Consistency Tests

Prediction quality asks whether a forecast agrees with the subsequently observed outcome or its conditional distribution. Physical consistency asks whether the predicted trajectory respects relevant identities, mechanisms, and constraints. These are separate axes. Several trajectories may be physically possible, while only one occurs per trial. A visually similar output may still violate contact or respond incorrectly to an action. Table 5.6 organizes the evidence needed for each claim.

Prediction Quality at a Fixed Information Boundary Report one-step prediction from newly observed context, open-loop multi-step prediction without future measurements, and feedback-interleaved prediction with explicitly scheduled observations separately. Also specify whether future controls were fixed before the forecast or recorded from a feedback policy. Conditioning retrospectively on a policy's later actions can reveal information about later states. It is not equivalent to forecasting an independently supplied intervention sequence.

Observation forecasts can be compared using pixel errors, perceptual feature distances such as learned perceptual image patch similarity (LPIPS), and distribution-level video measures such as Fréchet video distance (FVD) (Zhang et al., 2018; Unterthiner et al., 2018). These answer different questions. Pixel error is sensitive to small spatial misalignment. Perceptual metrics inherit the features of their evaluator. A good aggregate distribution score does not establish that each sample follows its supplied action. Multimodal futures further complicate paired image error. A legitimate but unobserved outcome can be penalized even when the predicted distribution is appropriate. Report the sample budget and avoid presenting best-of-many selection as the performance of a single unselected prediction.

State and event queries require measurements matched to their meaning. Object positions and velocities can be evaluated in physical units. Orientations need an appropriate rotational distance. Contact or phase-change events require detection and timing errors. For a scalar physical quantity, a horizon-specific summary is

$$\mathrm{RMSE}(k) = \sqrt{\frac{1}{N_{\mathrm{eval}}} \sum_{i=1}^{N_{\mathrm{eval}}} \left(\widehat{y}^{(i)}_{t_i+k|t_i} - y^{(i)}_{t_i+k} \right)^2}, \tag{5.27}$$

where N_{eval} is the number of independent evaluation trials, i indexes a trial, t_i is its forecast origin, k the forecast offset on a declared sampling grid, y the measured quantity, and $\widehat{y}_{t_i+k|t_i}$ the point forecast issued at t_i. Root mean squared error has the same physical unit as y. For multiple quantities, report errors separately or declare the normalization instead of adding meters, newtons, and temperatures without scaling. Plotting or tabulating the error against elapsed time exposes drift hidden by an average over horizons. A probabilistic forecast also needs a distributional score, not only error of its mean.

Physical Consistency and Intervention Response Evaluate persistence of object identity and hidden attributes through occlusion, compatibility of geometry across views, non-penetration within measurement tolerance, contact creation and release, and agreement between predicted motion and known control changes. When forces, boundary fluxes, and physical parameters are available, check momentum, energy, or material balances under the corresponding assumptions. Conservation of an isolated system must not be imposed on an open or externally actuated process. These tests should use the physical quantities underlying the constraints of Section 5.3.3, not merely whether an auxiliary loss is small during training.

Contact and other mechanism changes deserve event-centered reporting. Most recorded frames may show easy steady motion, so an aggregate average can conceal incorrect impact timing, slip onset, or release. Partition errors into free motion, contact formation, sustained contact, release, and relevant process transitions. Controlled changes in action direction, magnitude, duration, or timing test whether the model predicts the *right* change, rather than merely any change. Where possible, paired trials should hold the initial conditions and non-intervened factors fixed. Simulation supports precise resets. Physical repeats require reporting reset variability and measurement uncertainty.

Physics judged only from rendered images remains an indirect test. A perception model used to extract trajectories can introduce correlated errors, and a language-model judge can prefer plausible-looking motion. Such evaluators should be validated against independent labels or measurements for the claimed quantity. A state predictor without a visual decoder can instead be assessed through its native physical outputs or a readout fitted only on training data. Requiring every architecture to produce a video would conflate query availability with dynamics quality.

Benchmark Tasks and Information Parity Physion illustrates evaluation of a physical event by asking a model to predict, from an initial visible segment, whether specified objects will contact during the unseen continuation, rather than requiring every model to generate a video (Bear et al., 2021). The task permits comparisons among learned visual features, explicit states, and dynamical predictions, provided that their input privileges and readout training are reported. A model given simulator particle states has a different information budget from one given RGB observations. Their gap can help diagnose perception versus dynamics, but cannot be interpreted as a matched-input architectural ranking.

The same principle applies across model families. Spatial generators need held-out-view and revisit consistency tests with measured regions distinguished from completed regions. JEPA-style models need fixed, independently trained readouts or action-conditioned rollout tests, not only similarity to their own changing teacher. Joint world-action models need both policy evaluation and any separately claimed forecast evaluation. No single rendering metric, contact classifier, or robot success rate covers all these interfaces. A useful benchmark suite therefore combines common physical questions with native-interface tests and publishes what information each model receives.

Probability, Calibration, and Sharpness A forecast distribution should place probability on outcomes in a way that is both accurate and informative. Proper scoring rules reward the predictive distribution rather than a conveniently selected sample (Gneiting and Raftery, 2007). For a binary event, a scalar observation, and its predictive distribution, two representative scores are

$$\text{BS} = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} (p_i - e_i^{\text{obs}})^2, \qquad \text{CRPS}(F_{\text{CDF}}, y) = \mathbb{E}|Y - y| - \tfrac{1}{2}\mathbb{E}|Y - Y'|, \tag{5.28}$$

where BS is the Brier score, $p_i \in [0,1]$ the predicted probability of event $e_i^{\text{obs}} \in \{0,1\}$, and N_{eval} the number of trials. CRPS is the continuous ranked probability score for a scalar measurement y and predictive cumulative distribution F_{CDF}. Y and Y' are independent draws from F_{CDF}, assumed to have finite first moments. Lower values are better. CRPS can be estimated from samples without evaluating a density. A log score is appropriate when a normalized predictive density or probability mass is available. A diffusion training loss is not itself the log likelihood of a deployed forecast.

Calibration asks whether probabilities or intervals agree with empirical frequencies. For example, cases assigned a contact probability near a specified value should exhibit contact at approximately that frequency. A nominal prediction interval should be assessed by empirical coverage *and* width, since an excessively broad interval can cover almost every outcome while being unhelpful. Calibration curves and post-hoc calibration techniques are useful diagnostics, but calibration on one distribution need not persist after a shift (Guo et al., 2017). Marginal coverage alone does not establish correctness within each task, contact mode, horizon, or material group. Sample diversity and ensemble disagreement should therefore be tested against realized outcomes rather than interpreted as uncertainty quality by construction.

Splits, Aggregation, and Reproducibility Separate model fitting, calibration or threshold selection, and final testing. Group related frames and derived samples by trajectory, scene, object combination, and collection session as appropriate to the generalization claim. Report results by physical horizon, event type, and change of conditions before aggregating them. Confidence intervals should account for trajectory-level dependence. Treating adjacent frames as independent trials substantially overstates the amount of evidence. Include failure cases and invalid predictions in a declared accounting scheme rather than silently removing them from the denominator.

The rows in Table 5.6 are complementary, not interchangeable levels of one score. A model may pass state-persistence tests while failing intervention tests, or yield useful decisions while offering no calibrated probability interface. The evaluation report should identify supported claims and missing evidence separately. Decision-level evaluation is developed next.

5.4.4 Decision Utility and Applicability Boundaries

The operational question is whether model calls improve real outcomes. Compare systems with and without the relevant world-model function while matching the policy proposal mechanism, observation history, action interface, training data, and inference budget as closely as possible. For a specified real-task score J_{real}, define

$$\Delta J = J_{\text{real}}(\pi_{\text{with model}}) - J_{\text{real}}(\pi_{\text{reference}}), \tag{5.29}$$

where $\pi_{\text{with model}}$ is the policy or complete decision procedure using the tested model function, $\pi_{\text{reference}}$ the comparison procedure, and ΔJ their difference under the same outcome definition and evaluation distribution. The score may be success rate, measured task quality, or another explicitly specified objective. It is not the model's own predicted reward. Report initial conditions, repetitions, uncertainty intervals, failures, interventions, and resource costs. A positive average gain can conceal systematic failure on the very candidates the model ranks most confidently.

Ablations should replace one function at a time. A persistence predictor tests the value of temporal evolution beyond current state. An action-shuffled predictor tests sensitivity to the supplied control. Removing future supervision tests its contribution to the learned representation. Matched history and parameter budgets help separate predictive learning from a larger encoder or additional context. These controls diagnose mechanisms but do not individually prove a causal explanation for an entire system's improvement.

Efficiency has several boundaries, including observation acquisition and transfer, encoding, predictive prefill, action or video sampling, communication, and physical execution. Report model-side action-update time separately from sensor-to-command latency, full query latency, and successful task throughput. Asynchronous prediction moves work off a critical path but does not remove its computation or the risk of stale predictions. Hardware, precision, batch size, context length, action

horizon, denoising steps, and update rates should accompany timings. Neither the inverse of average inference time nor a high low-level servo rate establishes closed-loop responsiveness under all conditions.

Applicability boundaries should cover objects, materials, actions, temporal horizons, modalities, and embodiments. A model may generalize to new appearance while failing under new friction, or predict free motion while failing after a contact switch. Test combinations of mechanism changes as well as isolated shifts. Distinguish recovery through fresh observation, belief correction, physical-parameter identification, and network retraining. These require different resources and support different transfer claims. Uncertainty, modality disagreement, and residuals can trigger shorter horizons, additional sensing, or fallback, but their thresholds need validation against actual errors. Independent execution constraints and safety mechanisms remain system responsibilities.

Deployment validation should establish whether a world model improves outcomes under matched comparisons, within stated boundaries, and at a cost the system can afford. Its query interfaces must specify the limits of those claims.

5.5 Current Challenges and Future Outlook

The preceding sections suggest that progress should be described through capabilities and their evidence rather than a single architecture label. Broad pretrained priors, explicit physical structure, and efficient action interfaces contribute different strengths. Their integration raises questions about what can be learned from existing observations, what requires new intervention, and how a model can remain useful while its environment and operating conditions change.

5.5.1 Acquiring Physical Knowledge from Observation to Intervention

Videos provide rich evidence about appearance and motion but usually omit commanded forces, contact states, and other variables that determine the response to an intervention. Different mechanisms can explain the same observed trajectory. Larger observational datasets can improve coverage without resolving this ambiguity when the distinguishing variable is never measured or excited. The relevant challenge is to connect reusable visual and semantic priors with evidence that identifies executable action consequences.

Action-free pretraining followed by grounded interaction learning offers one division of labor, as illustrated by the separate representation and action-conditioned stages of V-JEPA 2 (Assran et al., 2025). Other systems can jointly train on several evidence types or transfer a prior through distillation. The choice should follow the gap to be closed. Unfamiliar object semantics, an unobserved material property, a new control interface, and a missing deformation mode are different problems. Transferring a representation does not remove the need to calibrate units, timing, and embodiment-dependent dynamics.

Multimodal measurements and targeted probes can expose distinctions that passive video leaves unresolved. A controlled lift with load measurements constrains mass. A slide under measured force constrains friction within a contact model. A thermal observation distinguishes visually identical process states. Such experiments should be chosen for their ability to separate plausible explanations, not merely to maximize visual novelty. Their cost and risk must be included in learning efficiency. For instruments and infrastructure, the intervention may be a process-setting change rather than a robot motion, but the same requirement holds. Predicted effects must be checked against the measurements relevant to the mechanism.

A further challenge is preserving uncertainty where evidence remains insufficient. A model should represent several compatible explanations instead of inventing a unique physical parameter from appearance. Uncertainty reduction through new sensing, identification through repeated interaction, and correction of structural model error should remain distinct. This distinction gives the closed loop a scientific role. Action tests an explanation, and the resulting discrepancy determines what knowledge should be revised.

5.5.2 *Reliable Consequence Prediction in Complex and Long-Horizon Processes*

Contact switches, non-rigid deformation, and coupled physical processes can turn small local errors into qualitatively different futures. Long-horizon prediction therefore requires more than a longer context window. It needs persistent state, representations of consequential events, appropriate time resolution, and uncertainty that remains useful as the forecast extends. A model may need to predict a phase boundary accurately without reproducing every intermediate texture.

The architectural routes in Table 5.2 are likely to be combined in several ways, but no single combination is established as universally preferable. One design direction connects compact predictive states to detailed generative readouts on demand. Another combines object- or field-based physical structure with learned residual dynamics. A third coordinates slower semantic and process models with faster contact or motion models. Each requires an interface that preserves identity, units, physical time, and uncertainty. A hierarchy is not useful when its levels describe incompatible worlds.

Combining routes along any of these directions raises a representation question. A shared substrate can support understanding, prediction, and action, while specialized encoders and heads preserve the information each function needs. Shared parameters are only one implementation. Aligned representations, directed cross-expert communication, and training-only predictive supervision offer alternatives, as discussed in Section 5.2.4. The roadmap's distinction between functional roles and architectural choices is useful here. A common physical description should support compatible queries without forcing all modalities into one update frequency or every task into one decoding objective (Chen et al., 2026b).

Future comparisons should therefore test composition rather than only larger monolithic models. Does adding structure improve held-out contact response? Does a generative head retain information needed by a new query? Does selective prediction save computation without making action ranking less reliable? Longer horizons and richer modalities should be assessed at matched resource budgets, with prediction errors decomposed by mechanism. These tests can reveal when an interface is missing information, when a transition is wrong, and when extra detail simply does not help the task.

A persistent challenge is deciding what must remain explicit across abstraction levels. A coarse state may encode that liquid will spill, while a fine field predicts where it flows. The two outputs should refer to the same container, contents, and boundary conditions. Learned refinement must recover detail compatible with the coarse state, not silently revise the initial condition until a preferred prediction becomes possible. The model may retain raw evidence or an external memory so that a new query can request information omitted from its current compact state. This makes memory retrieval and adaptive resolution part of world modeling, rather than equating longer token context with a sufficient physical state.

A second challenge is modular identification. If a prediction fails, the system should distinguish a wrong observation interpretation, an incorrect physical parameter, a missing interaction mechanism, and an inadequate decision objective. Composable models make these distinctions easier to test when their interfaces have clear semantics. A monolithic model can offer the same diagnostic access through controlled readouts and interventions. The long-term comparison is therefore not simply modular versus end-to-end. It is how reliably a system can localize error, acquire the evidence needed to correct it, and preserve previously valid knowledge at an acceptable cost.

5.5.3 *Closing the Real-World Loop: Real-Time Operation, Transfer, and Continual Improvement*

A deployed world model participates in a continuing loop among observation, understanding, prediction, action, and external feedback. It may help identify an uncertain state, propose informative experiments, screen candidate actions, or supply imagined experience for policy learning. The physical process then produces evidence that the model did not choose. This separation between internal imagination and external verification is essential. Repeated agreement between a

generator and an evaluator sharing its assumptions does not establish improved knowledge of the world.

Real-time operation requires allocating computation while the environment continues to change. Cached context, asynchronous experts, selective decoding, and short action chunks can reduce particular costs, but each creates a responsibility for freshness and correction. Transfer adds another problem. A new body, tool, sensor, or process can change what an action means. Reusable physical knowledge should be separated from the local parameters and interfaces that require new evidence. Systems such as InternW0 and InternW0-Δ show concrete ways to separate training and runtime work, not a guarantee that one update schedule transfers unchanged to every physical system.

Continual improvement should use diagnosis to choose the scale of an update. Fresh evidence may be enough to correct the current belief. Repeated discrepancies may justify identifying a small set of physical parameters. A persistent missing mechanism may require new data and network retraining. These responses should be recorded separately, together with the amount of real interaction, human intervention, and computation required. Updating every model after every observation can be wasteful, while never updating parameters can leave systematic errors unresolved.

A practical improvement cycle retains real failures and corrections, selects data that target identified gaps, updates the relevant model or policy components, and evaluates the candidate version on both new conditions and a retained regression suite. Synthetic rollouts can broaden practice near a known weakness, but acceptance must include evidence not produced by the same model. Replay and parameter regularization can help retain previous skills. They must still be checked for interference and loss of adaptability. Chapter 4 treats the data-side loop, while the present chapter specifies the predictive interfaces whose validity must survive each update.

The long-term aim extends beyond more elaborate imagined worlds. It is a system in which better representations produce more informative forecasts, forecasts support better real interventions, and the resulting evidence produces measurable correction. Progress should be reported through predictive accuracy, physical consistency, calibrated uncertainty, and improved physical outcomes, together with the costs of training, adaptation, operation, and recovery. This closes the chapter's organizing chain from state and transition to query and evidence.

These challenges converge on a single claim about priorities. Perception models tell an agent what is present. Policy models tell it what to do. A world model has the potential to help an agent understand what will happen, what would have happened instead, and why. In this sense the world model is one of the most important foundation models of Physical AI, providing a substrate on which perception, decision, and control can all stand. Its importance lies in the potential to connect testable predictions of the world with better decisions. More accurate predictions of task-relevant dynamics can support safer exploration, cheaper learning, and more reliable decisions across tasks and embodiments. For a world model to support decisions, its predictions need sufficient accuracy at the relevant resolution. Building machines that act in the world therefore depends, to a large extent, on building models of it and on making the limits of their evidence and predictions explicit.

6 Foundation Models for Robot Manipulation

Physical AI not only discusses how to perceive the world dynamics, but also instructs robots to actually interact with it. In robotics, manipulation is one of the important forms of physical action, which includes grasping, moving, assembling, and transforming objects through controlled contact, and beyond. This chapter examines foundation models for robot manipulation, which are the core of an embodied system that maps multimodal observations and task instructions to behavioral goals or low-level motor actions. Because open-ended environments present an unbounded variety of objects and tasks, engineering each manipulation skill individually is impractical. Instead, foundation models learn web-scale multimodal priors from physical robot trajectories, enabling a single pretrained model to be reused across tasks, environments and embodiments.

The chapter begins with the evolution of manipulation foundation models, focusing on the vision-language-action (VLA) architectures that dominate current practice. It then examines the pre-training and post-training that shape the model's capabilities, the model's adaptation across embodiments, and its deployment under real-time and resource constraints. It closes by applying these ideas to multi-fingered hands, where high-dimensional actuation, intricate contact transitions, and scarce demonstration data push every stage of the pipeline to its limits.

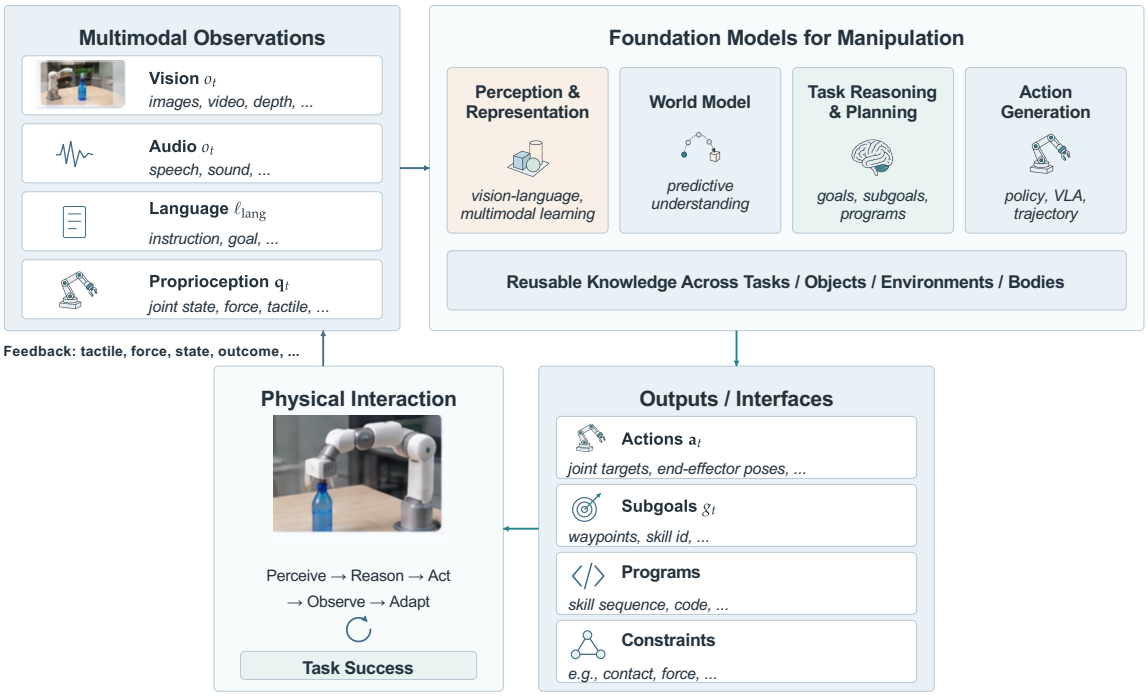


Fig. 6.1 Foundation models for robot manipulation

6.1 Emergence and Taxonomy

An embodied agent should transform an instruction into a sequence of physically meaningful decisions. Recognizing a bottle is only one part of opening it. The agent must identify the requested outcome, choose a grasp and an opening strategy, interpret resistance during contact, and determine whether the cap has actually been released. Foundation models for robot manipulation aim to make these capabilities reusable across tasks, objects, environments and sometimes even bodies. They bring together multimodal representation learning, generalist robot policies and models of action consequences. Figure 6.1 provides an overview of foundation models for robot manipulation. The key question is how broadly acquired knowledge can guide ongoing physical interactions.

6.1.1 Conceptual Scope and Enabling Conditions

In this chapter, a *foundation model for robot manipulation* is a pretrained model that can adapt to new tasks. It draws on multimodal evidence and task context to make task-relevant decisions, set behavioral goals, or generate robot actions. The foundation model property concerns the reuse of learned structure. A single model should provide a useful starting point for many downstream tasks or operating conditions. Neither a large parameter count nor a natural-language interface establishes this property on its own.

Let o_t denote exteroceptive observations, $\mathbf{q}_t$ denote proprioceptive measurements, $\mathbf{a}_t$ denote an action at the model's declared control interface, ℓ_{lang} denote a task instruction, and e describe the embodiment. The interaction history is defined as follows.

$$h_t = (o_{0:t}, \mathbf{q}_{0:t}, \mathbf{a}_{0:t-1}), \qquad \mathbf{A}_t^{\text{seq}} = [\mathbf{a}_t^\top; \ldots; \mathbf{a}_{t+H-1}^\top]. \tag{6.1}$$

An action-producing model implements $\pi_\theta(\mathbf{A}_t^{\text{seq}} \mid h_t, \ell_{\text{lang}}, e)$. A higher-level model may instead output a subgoal g_t, a skill identifier, a program, or a constraint on the desired interaction. These outputs are different interfaces to physical behavior, and should not be treated as interchangeable. An instruction to "grasp the bottle" leaves substantially more work to downstream modules than a sequence of end-effector poses or joint targets.

Four developments have made this reuse practical. Vision-language pretraining supplies broad semantic representations. Increasingly diverse robot datasets provide action-grounded supervision. Sequence and generative models capture temporal and multimodal behavior. Training infrastructure and simulation make repeated optimization and controlled data variation feasible. These conditions are complementary. Additional computation cannot recover action semantics that were never recorded, and simulated diversity is useful only to the extent that learned behavior transfers to the target physical conditions.

The distinction from adjacent chapters follows these interfaces. The perception models of Chapter 3 extract task-relevant evidence from observations, while Chapter 4 addresses the data that support learning. The world models of Chapter 5 represent conditional evolution and answer questions about possible consequences. Foundation models for robot manipulation use evidence and predictive knowledge to decide what behavior to pursue. The foundation models for whole-body motor control discussed in Chapter 7 regulate the realization of that behavior under body dynamics and contact constraints. One network may implement several of these functions, and a VLA model may extend into motion generation. The distinction concerns the responsibility of an output, not the size, name, or location of a network. Chapter 8 then considers how these capabilities are assembled into agents that manage memory, tools, and extended activities.

For example, during cap opening, task reasoning first identifies which bottle to open and what the desired open state is. Prediction then estimates what will happen if the agent grasps and rotates it. Action generation proposes a motion, and local control adjusts that motion based on contact feedback. The same tactile event may cause either a local correction, such as reducing grip force, or a task-level revision, such as regrasping. Effective coordination therefore requires feedback in both directions.

6.1.2 From Specialized Policies to Reusable Embodied Models

Figure 6.2 summarizes the transition from specialized visuomotor policies to reusable embodied models.

These early learned visuomotor policies often acquire one skill on one setup. Their representations are optimized for a narrow demonstration distribution, so even a change in camera, object, instruction, or robot can require substantial retraining. Multitask policies change the unit of learning from an isolated skill to a task-conditioned behavioral repertoire. RT-1 demonstrates instruction-conditioned robot control at scale, while the Open X-Embodiment study investigates the transfer available from pooling trajectories across robot platforms (Brohan et al., 2023; Open X-Embodiment

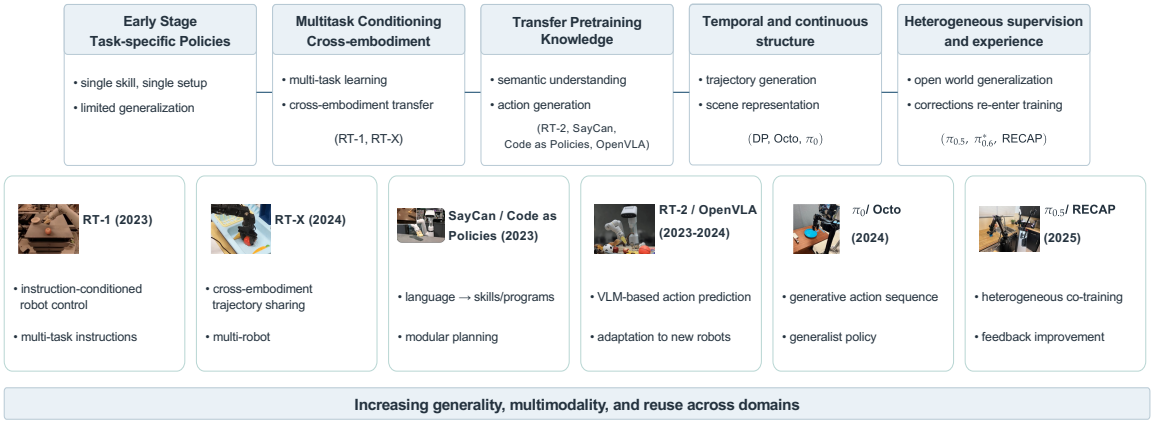


Fig. 6.2 Algorithm development framework

Collaboration, 2024). These developments make data diversity and interface compatibility central modeling issues.

A second development is the transfer of semantic knowledge from language and vision-language pretraining. In modular systems, a language model can select grounded skills or generate programs that invoke perception and control functions. SayCan and Code as Policies illustrate these two approaches (Ichter et al., 2023; Liang et al., 2023a). In direct action models, Zitkovich et al. (2023) incorporate robot action prediction into vision-language training with RT-2, and Kim et al. (2025b) release OpenVLA as a pretrained VLM-based policy available for adaptation. This kind of transfer can broaden object and instruction understanding, but it does not supply the missing dynamics or contact experience of an unfamiliar robot.

A third development focuses on temporal structure and continuous behavior. Diffusion Policy demonstrates multimodal action-sequence generation for visuomotor control, Octo combines a generalist policy with adaptable observation and action interfaces, and π_0 connects a pretrained VLM to a flow-based action expert (Chi et al., 2023; Ghosh et al., 2024; Black et al., 2025a). These designs make a useful distinction between the representation needed to understand a scene and the computation needed to generate a coherent trajectory. They also show that a foundation model for robotics need not be an autoregressive language model throughout.

Finally, broad pretraining is increasingly connected with heterogeneous supervision and deployment experience. $\pi_{0.5}$ studies open-world generalization through heterogeneous co-training, while the RECAP approach underlying $\pi^*_{0.6}$ uses experience and corrections to improve a pretrained policy (Physical Intelligence et al., 2025; Amin et al., 2026). The progression is consequently not a simple replacement of control by larger language models. It is the integration of semantic priors, physical action interfaces, predictive representations, and feedback-driven improvement. Each component addresses a different limitation of demonstration-only learning.

6.1.3 Major Model Lineages and Architectural Evolution

The main model lineages can be compared along the three axes of primary knowledge source, learning objective, and decision interface. Generalist policies emphasize reusable behavior learned from robot trajectories. VLAs introduce broad vision-language knowledge into action generation. And world-action models couple behavior with predictions of environmental change. Table 6.1 summarizes these emphases. The categories overlap. A generalist policy can be a VLA, and a VLA can contain a world model. Direct, hierarchical, and prediction-assisted execution are architectural choices that cut across all three lineages.

Table 6.1 Three model lineages compared by knowledge source, learning target, and decision interface. The categories are complementary rather than exclusive

Lineage	Knowledge source	Main learning target	Decision interface
Generalist robot policy	Diverse robot trajectories, task labels	Shared behavioral priors across tasks and setups	Actions, chunks, or skills RT-1, RT-X, Octo
Vision-language-action model	Vision-language pretraining plus robot interaction	Semantic grounding and conditioned action generation	Action tokens or continuous trajectories RT-2, OpenVLA, π_0
World-action model	Video and interaction data with available action labels	Coupled action and future-state prediction	Actions with predicted outcomes, or prediction-assisted decisions Unified World Models

Generalist Robot Policies These policies learn primarily from collections of robot trajectories, usually conditioned on language, goal images, or task identifiers. Shared training allows reaching, grasping, placing, and other recurring structures to contribute to multiple tasks. RT-X explores such sharing across embodiments, and Octo provides a policy initialization that can be adapted to new observation and action spaces (Open X-Embodiment Collaboration, 2024; Ghosh et al., 2024). Generality here comes from how broadly the behavior covers and transfers, not from conversational ability. Language may serve as a conditioning representation without making the policy itself a pretrained language model. The main limitation is coverage. Robot trajectories provide physically grounded actions but are costly to collect and can be concentrated in a few environments or control interfaces.

Vision-Language-Action Models These models connect visual observations and linguistic goals with a declared action space. A prominent route initializes the policy from a pretrained VLM so that the policy does not have to learn object knowledge and instruction semantics entirely from robot demonstrations. Action supervision then grounds that knowledge in the robot's control interface. The phrase VLA specifies a multimodal policy interface more clearly than it specifies a single architecture. Discrete autoregression, deterministic regression, diffusion, and flow matching are alternative ways of producing actions within this broad class. Their differences concern precision, multimodality, learning signals, and execution cost, as developed in Section 6.2. Semantic transfer is especially useful for interpreting an unfamiliar instruction, but successful interpretation still requires a motor repertoire capable of realizing it.

Predictive and Joint World-Action Models A world model can choose actions in several ways, including evaluating proposed actions, generating subgoal observations, or sharing a representation with a policy. Joint video-action modeling, illustrated by Unified World Models, couples predictive and behavioral learning (Zhu et al., 2025). These couplings should be distinguished. A model that predicts a future image that appears to satisfy the task goal does not thereby show that the robot can reach that image. Likewise, predicting a future and an action together does not automatically provide a calibrated answer to every counterfactual action query. The benefit of prediction must be demonstrated through better decisions or more transferable representations.

One formulation models $p_\theta(\mathbf{A}_t^{\text{seq}}, O_t^+ \mid h_t, \ell_{\text{lang}}, e)$, where O_t^+ denotes future observations over a declared horizon. With appropriate conditioning and training, the model can relate a proposed behavior to its expected outcome, or use a desired outcome to help infer behavior. Action-free video can supervise the predictive component while action-labeled robot data ground executable decisions. This is a potential advantage of joint learning, but the missing robot actions in human video cannot be recovered merely by renaming visual motion as an action. Viewpoint, embodiment, and unobserved forces still constrain transfer.

Architectures for Organizing Decisions A direct policy maps the current task context straight to actions. A hierarchical system exposes subgoals or skills between task reasoning and motion generation. A prediction-assisted system brings anticipated outcomes into learning or action selection. Language and program-mediated planners can compose existing behaviors, provided that their calls are grounded in perception, skill preconditions, and execution feedback. A higher-level process can also maintain context while a faster process updates motion using recent observations. GR00T N1 provides an example of tightly coupled vision-language and action-generation modules (NVIDIA et al., 2025c). Gemini Robotics 1.5 illustrates the coordination of embodied reasoning with action capabilities (Gemini Robotics Team et al., 2025). Architectural separation does not mean asynchronous execution. Whether modules run independently, share cached features, or wait at synchronization points is a deployment property that must be specified separately.

Connecting Policy, Prediction, and Evaluation The relationships among these arrangements can be expressed with the following shared task representation and three possible output interfaces

$$\begin{aligned} z_t &= f_\theta(h_t, \ell_{\text{lang}}, e), \\ \mathbf{A}_t^{\text{seq}} &\sim \pi_\psi(\cdot \mid z_t, \ell_{\text{lang}}, e), \\ \widehat{z}_{t+1:t+H} &\sim p_\phi(\cdot \mid z_t, \mathbf{A}_t^{\text{seq}}, e), \\ \widehat{v}_t &= V_{\omega_{\text{critic}}}(z_t, \ell_{\text{lang}}, e). \end{aligned} \tag{6.2}$$

Here z_t summarizes available evidence, the policy proposes behavior, the transition model predicts its consequences, and the evaluator estimates a task-related quantity. Not every system contains all three heads, and the equations do not require shared parameters. They distinguish functions that are often conflated in discussion of unified models.

There are several ways to connect them. A policy may use a predicted subgoal as conditioning, a planner may score sampled action chunks through a world model, or predictive training may improve an encoder that is used without online rollout. In a joint generative model, what the model samples (an action, a future observation, or both) depends on how it is conditioned and masked. Each choice changes which questions can be asked at inference time and which computational costs lie on the control path.

Evaluation also requires precise semantics. A progress score estimates how far an observed process has advanced. A success classifier estimates whether a terminal criterion holds. A value function estimates future return under a specified policy and reward. A model trained for one of these tasks is not automatically a substitute for the others. During cap opening, observing rotation may indicate motion without demonstrating release. Outcome feedback must therefore be grounded in the property the task actually requires.

Interpreting Generalization Claims Embodied generalization has several independent dimensions. A model may transfer to new object appearances while still relying on the same workspace and robot. It might understand a new composition of familiar instructions while lacking the corresponding motor skill. Or it might adapt to a new embodiment only after target demonstrations. Claims should specify which factors change and which remain fixed. In particular, zero-shot execution, few-shot adaptation, and large-scale fine-tuning are distinct capabilities.

Closed-loop evaluation should test the decisions that make reuse valuable, including selecting the intended object among distractors, completing unfamiliar combinations of known stages, revising behavior after a disturbance, and transferring with a stated amount of new data. A useful experimental design varies task, environment, and embodiment separately before testing their combination. Retaining performance on previously supported conditions matters as much as improvement on the new condition. Offline action prediction error alone cannot establish these properties because it does not measure the states visited by the executing policy.

The remainder of this chapter follows the resulting development path. Section 6.2 examines action representations and multimodal architectures. Section 6.3 explains how different supervision

sources shape capabilities. Section 6.4 addresses the physical meaning of transfer between bodies. Section 6.5 then asks how these capabilities can be delivered within the timing and resource constraints of an actual robot.

6.2 Vision-Language-Action Model

Many current VLA models start from a pretrained VLM and extend it to output actions, a modality its pretraining never covered. This section begins with how actions are represented and generated, then turns to how they are combined with visual and linguistic inputs in a single sequence model, and closes with how this model is extended to sensory inputs beyond images and language.

6.2.1 Action Representation and Generation

Within a VLA, action is treated as a third modality alongside images and language, although it behaves quite differently from both. Actions are continuous rather than symbolic, their dimensionality and control rate vary with the embodiment, and consecutive actions are sampled at high frequency and are strongly correlated. Since action is also the model's output, how it is represented decides both how demonstrations are encoded during training and how commands are decoded at run time. Writing $\mathbf{a}_t \in \mathbb{R}^d$ for the action at step t and $\mathbf{A}_t^{\text{seq}} = [\mathbf{a}_t^\top; \dots; \mathbf{a}_{t+H-1}^\top]$ for an action chunk of horizon H, modern VLAs predict a whole chunk at once instead of a single step (Zhao et al., 2023; Chi et al., 2023). Doing so limits the accumulation of errors over time, produces smoother motion, and spreads the cost of one forward pass over H control steps. Existing methods fall into two families according to how $\mathbf{A}_t^{\text{seq}}$ is fed into the sequence model and how it is generated. The *discrete* route converts $\mathbf{A}_t^{\text{seq}}$ into tokens that the language backbone predicts, whereas the *continuous* route lets a dedicated head generate real-valued chunks directly. The two routes trade off training efficiency, inference speed and the preservation of the pre-trained VLM in opposite ways, which is why recent hybrid schemes combine discrete training with continuous execution.

Discrete Action Representations Once a chunk is mapped to tokens $z^{\text{act}}_{1:N_{\text{acttok}}}$, the VLA is trained with the ordinary next-token objective

$$\mathcal{L}_{\text{AR}} = -\sum_{k=1}^{N_{\text{acttok}}} \log p_\theta\big(z_k^{\text{act}} \,\big|\, z_{<k}^{\text{act}},\, o_t,\, \ell_{\text{lang}}\big). \tag{6.3}$$

This requires no change to the VLM architecture, and since robot data and web data are trained with the same loss, the backbone remains a language model shaped by both. The methods differ mainly in the tokenizer that produces $z^{\text{act}}_{1:N_{\text{acttok}}}$. The simplest, *per-dimension binning*, splits each action dimension into a fixed number of uniform bins (256 in RT-1 (Brohan et al., 2023), RT-2 (Zitkovich et al., 2023) and OpenVLA (Kim et al., 2025b)). RT-2 writes the bin indices as text, so the VLM outputs an action as a string of integers, while OpenVLA reassigns the least-used tokens of the Llama vocabulary to action bins. Binning is easy to implement but scales poorly, because a chunk becomes $H \cdot d$ tokens that must be decoded one after another, neighboring tokens at high control rates are nearly identical and therefore carry little learning signal, and precision can never exceed the bin width. *Compression-based tokenization* turns this redundancy into an advantage. FAST (Pertsch et al., 2025) applies a discrete cosine transform to each action dimension, quantizes the coefficients so that most high-frequency ones become zero, and compresses the result with byte-pair encoding. The resulting sequences are much shorter than those produced by binning, the same tokenizer transfers across embodiments, and π_0-FAST reaches the performance of the flow-matching π_0 with less training compute. *Learned quantizers* take a different approach and train a vector-quantized autoencoder (van den Oord et al., 2017) whose codebook entries act as short motion primitives. Residual vector quantization (RVQ) increases resolution by quantizing the remaining error over several stages, and VQ-VLA (Wang et al., 2025b) shows that training the tokenizer on more real and simulated trajectories improves both long-horizon success and smoothness. The price is an embodiment-specific decoder that has to be deployed together with the policy. Sequential autoregressive decoding of long action-token chunks through a billion-parameter backbone can incur

substantial inference latency. Parallel decoding in OpenVLA-OFT (Kim et al., 2025a) predicts all action tokens in a single pass and thus raises throughput, but it no longer models the joint distribution over tokens explicitly.

Continuous Action Representations On the continuous route, an output module produces $\mathbf{A}_t^{\text{seq}} \in \mathbb{R}^{H\times d}$ directly. A simple option is an L_1 action-regression head, as used in OpenVLA-OFT (Kim et al., 2025a). For direct point regression, expected elementwise L_1 loss is minimized by conditional coordinatewise medians. Action Chunking with Transformers (ACT) uses an L_1 reconstruction term together with a weighted KL-divergence term within a conditional variational autoencoder (Zhao et al., 2023). Such methods can perform well on benchmarks, but a single point prediction need not capture distinct valid action modes. Other generative heads model the full, possibly multimodal, action distribution. Diffusion Policy (Chi et al., 2023) denoises action chunks conditioned on observations, and Octo (Ghosh et al., 2024), RDT-1B (Liu et al., 2025c) and GR00T N1 (NVIDIA et al., 2025c) scale the idea up with diffusion transformers. Within VLAs, the reference design is π_0 (Black et al., 2025a), in which a PaliGemma backbone encodes images and language and a separate, smaller *action expert* attends to the backbone tokens and is trained with conditional flow matching. With an interpolant $\mathbf{A}_t^{\text{seq},\tau_{\text{flow}}} = \tau_{\text{flow}}\mathbf{A}_t^{\text{seq}} + (1-\tau_{\text{flow}})\boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \in \mathbb{R}^{H\times d}$ with independent entries $\varepsilon_{ij} \sim \mathcal{N}(0,1)$, the expert learns the velocity field

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{\tau_{\text{flow}},\boldsymbol{\varepsilon}} \left\| \mathbf{v}_\theta^{\text{flow}}(\mathbf{A}_t^{\text{seq},\tau_{\text{flow}}}, \tau_{\text{flow}}, o_t, \ell_{\text{lang}}) - (\mathbf{A}_t^{\text{seq}} - \boldsymbol{\varepsilon}) \right\|_{\text{F}}^2, \tag{6.4}$$

where $\|\cdot\|_{\text{F}}^2$ sums squared entries over the H action steps and d action coordinates. At inference the velocity field is integrated with a few Euler steps, and because the backbone prefix is cached, each step requires a forward pass through the expert only. Diffusion and flow-matching action heads avoid action-bin quantization and can represent multimodal action distributions. Compact action expert modules and a small number of sampling steps can make inference efficient. However, in the cited VLA comparisons, training with a continuous action expert converges more slowly than autoregressive action-token training, and gradients from a randomly initialized expert can interfere with the pretrained backbone. Gradient control and co-training with VLM data mitigate this interference in the reported settings (Driess et al., 2025). Co-training with web data, or interleaving actions with language reasoning, also requires additional machinery. It is therefore natural to use tokens for learning and continuous generation for execution, and several recent systems combine the two in this way. $\pi_{0.5}$ (Physical Intelligence et al., 2025) pre-trains the whole model autoregressively on FAST tokens and adds a flow-matching expert only during post-training, where it optimizes $\mathcal{L} = \mathcal{L}_{\text{AR}} + \alpha_{\text{mix}}\mathcal{L}_{\text{FM}}$ with the two action representations masked from each other. At run time the model first decodes text and then runs a few denoising steps of the expert. Knowledge insulation (Driess et al., 2025) merges the two stages into one. The backbone is trained on FAST tokens and vision-language data while a stop-gradient prevents the flow-matching loss from reaching it. The reported model trains faster, runs faster, and generalizes better than either pure route. These systems point toward a division of labor in which discrete tokens shape the backbone's representation and a lightweight continuous generator handles control. Which tokenizer shapes the backbone best, and how the two representations should be scheduled across pre-training, post-training and continual learning, are still open questions.

6.2.2 Paradigm of Vision-Language-Action Models

The VLA paradigm starts from a simple observation. A policy $\pi_\theta(\mathbf{A}_t^{\text{seq}} \mid o_t, \ell_{\text{lang}})$ that maps images o_t and an instruction ℓ_{lang} to an action chunk $\mathbf{A}_t^{\text{seq}}$ takes the same inputs as a vision-language model, so a policy initialized from a VLM and fine-tuned on robot demonstrations can inherit knowledge that the VLM has learned from billions of web image-text pairs. Almost every current VLA pairs the vision encoder and language backbone of a VLM with an action output module and couples the two in one of two mainstream ways (Figure 6.3). This section first describes this common template before tracing its development.

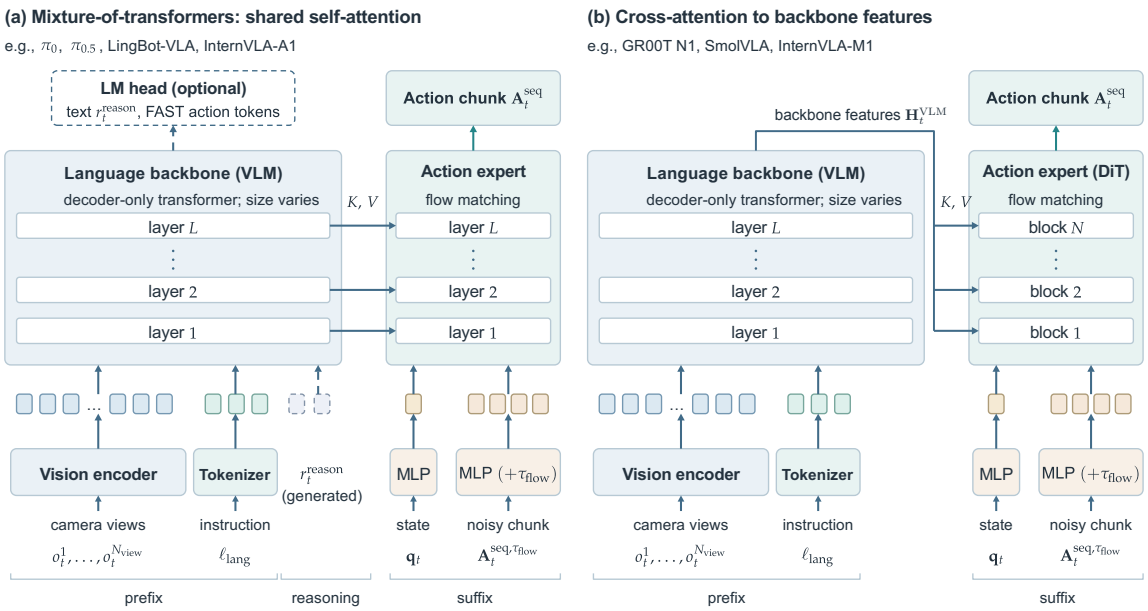


Fig. 6.3 Two mainstream VLA structures. (a) In a mixture-of-transformers structure, the expert has its own weights but the same depth as the backbone, and in every layer its tokens attend to the backbone's keys and values through a shared self-attention. The backbone can additionally generate reasoning text r_t^{reason} or FAST action tokens through an optional language-model head. (b) In a cross-attention structure, a separate diffusion transformer reads backbone token-feature matrix $\mathbf{H}_t^{\text{VLM}}$ through cross-attention, so its depth, size and rate are independent of the backbone

Components Both structures in Figure 6.3 share three components. The first is a *vision encoder*, usually the SigLIP or DINOv2 vision transformer (ViT) inherited from the VLM, which maps each camera image to a few hundred patch tokens. When there are several views, each is encoded separately and the tokens are concatenated. The second is a *vision-language backbone*, typically a decoder-only transformer with 2 to 7 billion parameters in open models (Llama 2, Qwen-VL, PaliGemma) and a frontier-scale model in closed systems, which processes the image tokens together with the tokenized instruction and produces the representation from which actions are read out. The third is an *action output module*, one of the heads described in Section 6.2.1, which turns this representation into an action chunk. Discrete action tokens need nothing beyond the backbone's own language-model head, as in RT-2 and OpenVLA, whereas current generalist VLAs mostly follow the continuous route and add a smaller action expert trained with flow matching or diffusion. Proprioceptive state $\mathbf{q}_t$ enters alongside them, either as discretized tokens in the prefix or through a small embodiment-specific multilayer perceptron (MLP), and other sensing modalities are deferred to Section 6.2.3.

Two Mainstream Structures With a continuous expert in place, the key architectural question becomes how the expert reads from the backbone, and current VLAs answer it in one of two ways (Figure 6.3). In a *mixture-of-transformers* (MoT) design (π_0 (Black et al., 2025a), $\pi_{0.5}$ (Physical Intelligence et al., 2025), LingBot-VLA (Wu et al., 2026b), InternVLA-A1 (Cai et al., 2026a)), the expert is a narrower transformer with the same depth as the backbone. The two keep separate weights but share a single self-attention operation in every layer, so the action tokens simply form another block of the same sequence and read the backbone's keys and values layer by layer (Figure 6.3a). This gives the tightest possible coupling, since the expert can draw on every level of the backbone's representation and the backbone's language-model head remains available for reasoning text or FAST tokens in the same sequence. The downside is that the expert is bound to the backbone's depth and clock, and its gradients reach every backbone layer, which is what knowledge insula-

tion (Driess et al., 2025) is designed to stop. In a *cross-attention* design (GR00T N1 (NVIDIA et al., 2025c), SmolVLA (Shukor et al., 2025), InternVLA-M1 (Chen et al., 2025b)), the expert is a separate diffusion transformer (DiT) over state and action tokens that reads backbone token-feature matrix $\mathbf{H}_t^{\text{VLM}}$ through cross-attention layers interleaved with self-attention (Figure 6.3b). The position and width of this interface are then free design choices, and published systems make them differently. GR00T N1 conditions its DiT on middle-layer rather than final-layer features of the VLM, and SmolVLA keeps only the lower half of a small VLM's layers so that the whole model has just 0.45B parameters. InternVLA-M1 inserts a querying transformer whose learnable queries cross-attend to intermediate layers of a 3B VLM, passes the resulting fixed set of tokens to an 86M diffusion action expert, and attenuates the expert's gradients before they reach the backbone. Decoupling allows the expert to be sized and clocked independently of the VLM, which dual-system layouts exploit by running a slow VLM above a fast expert, as GR00T N1 does. The price is a narrower channel, because the expert sees the backbone only through the features or queries it is given and does not take part in the backbone's own attention. In practice, MoT is preferred when the expert and the backbone are trained jointly on one sequence, and cross-attention when the expert has to be small, fast or run at its own rate.

Sequence Organization Whatever the structure, the inputs of one control step are organized as one ordered sequence, canonically

$$\underbrace{[\,\text{img}(o_t^1), \ldots, \text{img}(o_t^{N_{\text{view}}}),\ \text{txt}(\ell_{\text{lang}})\,]}_{\text{prefix}}\ \underbrace{[\,\text{txt}(r_t^{\text{reason}})\,]}_{\text{reasoning}}\ \underbrace{[\,\text{emb}(\mathbf{q}_t),\ \text{emb}(\mathbf{A}_t^{\text{seq},\tau_{\text{flow}}})\,]}_{\text{suffix}}, \tag{6.5}$$

where N_{view} is the number of views, r_t^{reason} is an optional intermediate text output (a subtask, plan or chain of thought), and $\mathbf{A}_t^{\text{seq},\tau_{\text{flow}}}$ is the noised continuous action chunk at flow time τ_{flow}. Discrete action tokens instead use $z_{1:N_{\text{acttok}}}^{\text{act}}$ as defined above. In an MoT the whole sequence passes through shared self-attention, which follows a block mask rather than a plain causal one. Prefix tokens attend to each other bidirectionally, as in prefix-LM training of PaliGemma-style VLMs, while the reasoning block is causal because it is generated token by token. The suffix attends to everything before it and bidirectionally within itself, since all H actions are produced together, but no earlier block attends to the suffix. This keeps the prefix key-value cache independent of the actions, so that it can be reused across denoising steps. A cross-attention design arrives at the same structure by construction, because the backbone processes only the prefix and the reasoning block while the expert processes the suffix and reads the backbone through cross-attention. Here too the backbone features are computed once per observation and reused across denoising steps. When discrete and continuous representations coexist in training, as in $\pi_{0.5}$ (Physical Intelligence et al., 2025) and knowledge insulation (Driess et al., 2025), the two suffixes are additionally masked from each other so that the flow expert cannot copy the FAST tokens.

Development The template described above takes shape over several years. RT-1 (Brohan et al., 2023) predates the use of VLMs but sets conventions that later work retains. A transformer with 35 million parameters, trained on about 130,000 teleoperated episodes from 13 robots, learns to follow several hundred instructions. Its actions are discretized into 256 bins per dimension and emitted as tokens. Its generalization grows with the breadth of tasks and robots rather than with model size. RT-2 (Zitkovich et al., 2023) then co-fine-tunes internet-scale VLMs (PaLI-X 55B, PaLM-E 12B (Driess et al., 2023)) on web data together with robot episodes whose actions are written as integer strings, and the resulting policy generalizes semantically in ways that no robot dataset could have taught it, for instance picking up the object "that could be used as a hammer". RT-2, however, is neither open nor backed by shared data. Open X-Embodiment (Open X-Embodiment Collaboration, 2024) addresses the data gap by pooling 60 datasets from 22 embodiments into more than a million episodes and showing positive transfer across robots, and OpenVLA (Kim et al., 2025b) addresses openness by releasing a 7B open-weight policy trained on this corpus.

The current generation keeps this foundation but adopts continuous action experts and trains on data far beyond public collections, and its progress has come on three fronts. The first front is scale,

starting with π_0 (Black et al., 2025a), which pairs a 3B PaliGemma backbone with a 300M flow-matching expert and trains it on roughly 10k hours of data from 7 robot configurations. $\pi_{0.5}$ (Physical Intelligence et al., 2025) adds heterogeneous co-training and subtask prediction, which allows the same policy to work in homes that had not appeared in its training data, GR-3 (Cheang et al., 2025) reproduces the template on a Qwen2.5-VL backbone, and LingBot-VLA (Wu et al., 2026b) reports steady gains as its bimanual data grow from 3k to 20k hours, with no sign of saturation.

The second front is hierarchical control, motivated by whole-body and long-horizon tasks for which a single synchronous model is too slow. GR00T N1 (NVIDIA et al., 2025c) establishes the open dual-system template for humanoids, with a 2B VLM feeding a diffusion transformer that runs at 120 Hz, and the Gemini Robotics series (Gemini Robotics Team et al., 2025) pairs a frontier VLA with a separate embodied-reasoning model that plans in natural language and orchestrates the VLA.

On the third front, supervision has moved beyond imitation. $\pi^*_{0.6}$ (Amin et al., 2026) learns from its own deployment experience through reinforcement learning, while $\pi_{0.7}$ (Physical Intelligence et al., 2026) conditions on subgoal images generated by a world model and shows early signs of compositional generalization to tasks that have no demonstrations in its training data. InternVLA-A1 (Cai et al., 2026a) unifies understanding, future-frame generation and action in one mixture-of-transformers, and its successor InternVLA-A1.5 (Ma et al., 2026a) replaces explicit future-frame generation with a small set of learnable foresight tokens supervised by a frozen pre-trained video generator, so that the policy inherits the generator's dynamics priors while the video branch is discarded at inference. X-VLA (Zheng et al., 2026a) shows that a 0.9B transformer with embodiment-specific soft prompts can match far larger models. Overall, the field has settled on the template and the two structures described above, and the main axis of scaling has moved from parameters, with open-weight VLAs still at 1 to 7B, to data, which now amounts to tens of thousands of hours supplemented by human video, synthetic trajectories and autonomous experience.

6.2.3 Extending Multimodal Inputs

Images and language are the default inputs of a VLA because they are the modalities used for VLM pretraining, but they leave out much of what manipulation depends on, such as events preceding the current frame, how far away objects are, and what happens at the moment of contact. An additional input can enter the sequence of Section 6.2.2 in one of two places. It can be added to the prefix as *extra tokens* that the backbone attends to together with images and text, or it can be given to the suffix and the action module as a *conditioning input* that bypasses the backbone. Writing m_t^{multi} and $m_t^{\text{multi}\prime}$ for inputs on the two routes, the canonical sequence becomes

$$\underbrace{[\,\text{img}(o_t^1),\ldots,\text{img}(o_t^{N_{\text{view}}}),\ \text{enc}(m_t^{\text{multi}}),\ \text{txt}(\ell_{\text{lang}})\,]}_{\text{prefix}}$$
$$\underbrace{[\,\text{txt}(r_t^{\text{reason}})\,]}_{\text{reasoning}}\ \underbrace{[\,\text{emb}(\mathbf{q}_t),\ \text{emb}(m_t^{\text{multi}\prime}),\ \text{emb}(\mathbf{A}_t^{\text{seq},\tau_{\text{flow}}})\,]}_{\text{suffix}}. \tag{6.6}$$

Multiple camera views, including the wrist camera, are already handled by the vision encoder as described in Section 6.2.2, and proprioceptive state $\mathbf{q}_t$ usually takes the second route, so the rest of this section reviews three further extensions, namely temporal history, 3D geometry and contact.

Temporal History and Memory Most VLAs condition only on the current frame, even though many tasks depend on the past, for example when the arm hides the target, when an object has been placed under a container, or when the robot must keep track of which stages of a task are already done. Concatenating past frames into the prefix, as Octo (Ghosh et al., 2024) does, brings sharply diminishing returns beyond one extra frame while adding hundreds of tokens per frame and view, and it also invites causal confusion (de Haan et al., 2019), in which the policy copies the behavior implied by past observations instead of reacting to the current one. Longer memory therefore comes down to compression, and two recent systems approach it in different ways.

MEM (Torne et al., 2026), which underlies $\pi_{0.6}$ (Amin et al., 2026) and $\pi_{0.7}$ (Physical Intelligence et al., 2026), keeps memory inside the sequence at two time scales. Over the short term, a video encoder compresses recent frames, packing six of them into the token budget of a single frame in $\pi_{0.7}$. Over the long term, a high-level policy rewrites a natural-language summary at every step and places it in the reasoning slot r_t^{reason}, which supports tasks that require up to 15 minutes of memory. RoboTTT (Jiang et al., 2026) instead moves memory out of the sequence and into the weights. Its test-time training layers update a small fast-weight network with one gradient step per timestep, so history is stored in these weights and inference cost stays constant however long the context becomes. With this design the context reaches 8K timesteps, or about 5 minutes, and pre-training with an 8K-timestep context outperforms pre-training with 1K timesteps, which suggests that context length may be a scaling axis for robot policies just as it is for language models.

3D Information An RGB image projects the scene onto a plane, so grasp height, clearance and relative depth must be inferred, whereas depth sensors and point clouds supply this geometry directly. The most direct way to use this geometry is to feed it into the model. SpatialVLA (Qu et al., 2025) turns depth from a monocular estimator into 3D positional encodings for the image tokens, while PointVLA (Li et al., 2026a) converts point clouds into native 3D tokens that only the action module consumes. Explicit geometry improves precision on spatially demanding tasks, but the 3D encoder lacks web-scale pre-training, and commodity depth sensors fail on transparent and reflective objects. Spatial Forcing (Li et al., 2026b) takes a second route and uses geometry only as a training signal. Having found that the visual tokens of an RGB-trained VLA carry little recoverable depth structure, it aligns an intermediate layer of these tokens with the features that the geometry foundation model VGGT (Wang et al., 2025a) extracts from the same images. Since VGGT is needed only during training, inference is unchanged, and training nevertheless becomes faster and more data-efficient, which shows that an input can shape the backbone's representation without ever entering its sequence.

Force and Tactile Sensing Contact-rich tasks such as insertion, screwing and handling fragile objects depend on forces that vision cannot observe, and the wrist camera is usually occluded at exactly the moment of contact. Contact information can enter a VLA in three ways. Images from vision-based tactile sensors such as GelSight (Yuan et al., 2017) can be fed to the vision encoder as an additional view, albeit across a domain gap. Force-torque readings (Cao et al., 2021) can be treated like proprioception and placed in the suffix. And dedicated tactile encoders that cover several sensor types, such as Sparsh (Higuera et al., 2025), can produce tokens for the backbone. Tactile-VLA (Huang et al., 2025a) follows the token route, fusing tactile tokens with vision and language inside the VLM and outputting target positions and forces for a hybrid position-force controller, whereas ForceVLA (Yu et al., 2025b) follows the conditioning route, fusing force-torque tokens with VLM features in a mixture-of-experts module on the action path. TA-VLA (Zhang et al., 2025d), a systematic study of torque inputs, comes out in favor of the conditioning route for such signals, finding that torque helps more in the action decoder than in the encoder and that a torque history summarized into a single token works best.

6.3 Pre-Training, Post-Training, and Capability Shaping

Figure 6.4 summarizes the overall training pipeline for embodied VLA models, which proceeds through three stages. Pre-training uses multi-source data to learn general representations and behavioral priors. Supervised post-training, including SFT, turns these priors into executable policies. Reinforcement learning-based post-training, including RFT, improves the policy through action outcomes. The remainder of this section discusses each stage in turn and then addresses continual learning, which converts deployment experience into lasting capabilities (Figure 6.5).

6.3.1 Pre-Training: Learning General Embodied Representations and Behavioral Priors

Vision-language models can recognize objects, interpret instructions, and reason using commonsense knowledge, but these capabilities do not automatically translate into reliable robotic manip-

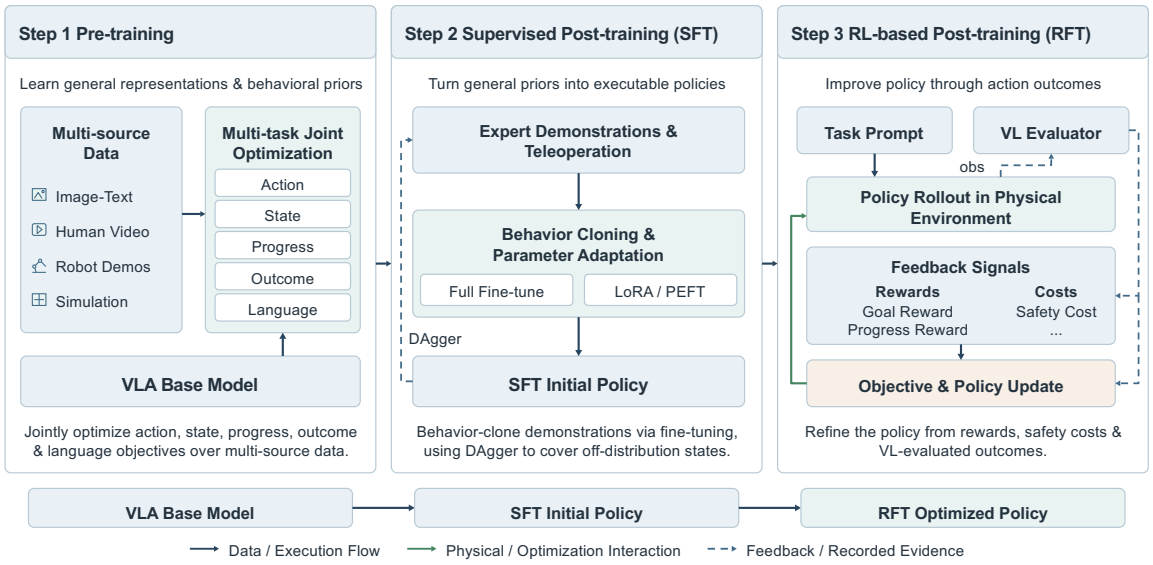


Fig. 6.4 Overview of the embodied VLA training pipeline. Pre-training combines multi-source data under multi-task objectives to produce a VLA base model. Supervised fine-tuning (SFT) during supervised post-training behavior-clones demonstrations into an executable initial policy. And reinforcement fine-tuning (RFT) during reinforcement learning-based post-training refines the policy from rewards, safety costs, and vision-language evaluator feedback. The stages are discussed in Sections 6.3.1 to 6.3.3

ulation. Embodied pretraining must integrate language, visual inputs, robot proprioceptive states, actions, and task outcomes within a common temporal framework, enabling the model to learn relationships among instructions, states, actions, and outcomes. Conditioned on observation histories, previous actions, task instructions, and embodiment characteristics, the model learns how to select actions in the current state. Observations may include not only RGB images but also depth, tactile measurements, force and torque readings, joint states, and equipment telemetry.

Embodiment characteristics include degrees of freedom, end-effector type, control frequency, workspace, and action limits. Explicitly incorporating these conditions helps distinguish shared task structure from embodiment-specific execution mechanisms, providing a basis for transfer across tasks and robot platforms.

Different data sources offer complementary supervision. Internet image-text data provide semantic knowledge about objects, scenes, and tasks. Videos of human activity reveal action sequences, tool use, and state transitions. Robot demonstrations supply action labels associated with specific control interfaces. And simulated interactions provide relatively inexpensive data on contact, forces, collisions, and failures that are difficult to collect at scale. Mixing these sources is intended to combine semantic, temporal, and physical information, rather than merely increase the number of training examples.

This complementarity also offers a way to address the scarcity of labeled robot actions. Video prediction, inverse dynamics, and latent action models can extract task-relevant behavioral representations from human videos, which an embodiment adaptation module can subsequently decode into executable actions. Studies such as UniVLA (Bu et al., 2025) show that task-centered latent action representations can exploit video data spanning different embodiments and viewpoints. Nevertheless, their physical executability still requires calibration through robot interaction. Acquiring behavioral knowledge from human video therefore also requires aligning that knowledge with robot control interfaces, sensor feedback, and physical outcomes.

To support this alignment, embodied pretraining commonly combines action prediction, language understanding, state recognition, task progress estimation, and outcome prediction. These objectives correspond to different forms of supervision available in embodied data. Consider a bottle-opening task as an example. Action supervision specifies the robot actions needed to approach, grasp, and rotate the cap, and is typically obtained from teleoperated or scripted demonstrations recorded in the robot's own action space. State supervision describes the relevant object and contact conditions, such as the poses of the bottle and cap and whether the end effector is in contact with the cap. In simulation these labels come from ground-truth physics, while on real robots they are estimated by the perception stack or annotated offline. Progress supervision identifies the current stage of the task, such as approach, grasp, rotation, release, or completion. This supervision can be supplied by scripted stage detectors, human annotators, or a vision-language evaluator of the kind used in reinforcement learning-based post-training. Outcome supervision indicates whether the bottle has ultimately been opened, which can be determined from a terminal physical observation or task-specific success criterion. Language supervision associates the visual and behavioral sequence with the task instruction, such as "open the bottle", and is available whenever trajectories are paired with instructions or narrations. These five objectives train distinct, complementary capabilities.

These objectives can be summarized by a weighted multi-task loss of the form

$$\mathcal{L}_{\text{pre}} = \lambda_{\text{a}}\mathcal{L}_{\text{action}} + \lambda_{\text{s}}\mathcal{L}_{\text{state}} + \lambda_{\text{p}}\mathcal{L}_{\text{progress}} + \lambda_{\text{o}}\mathcal{L}_{\text{outcome}} + \lambda_{\ell_{\text{lang}}}\mathcal{L}_{\text{language}}, \tag{6.7}$$

where the coefficients balance action learning, state understanding, progress estimation, outcome prediction, and language supervision. This schematic loss combines objectives whose labels may be available on different samples. Each loss is evaluated only where its supervision is available, with missing labels handled by masking the corresponding objective.

Table 6.2 summarizes how this masking scheme applies to each data source, which objectives in Eq. (6.7) it can support, and which model components the resulting gradients primarily update. Because available supervision varies by data source, the same foundation model is trained by applying different subsets of the objectives to different data sources. The resulting training process aligns semantic, temporal, state, and action representations without requiring a homogeneous label space across the entire dataset.

Table 6.2 Correspondence between heterogeneous data sources, the supervision they provide, the objectives of Eq. (6.7) they support, and the model components primarily updated. Objectives whose labels a source does not provide are masked rather than assigned artificial targets. A dash means that missing labels are not specified

Data source	Available supervision	Supported objectives	Primary module updated	Missing labels
Internet image-text	Object/scene semantics, instruction pairing	$\mathcal{L}_{\text{language}}$ ($\mathcal{L}_{\text{state}}$ from captions)	Vision-language backbone	Action, progress masked
Human video	Temporal order, task stages, narration	$\mathcal{L}_{\text{progress}}$, $\mathcal{L}_{\text{language}}$, latent-action $\mathcal{L}_{\text{action}}$	Backbone, latent-action or embodiment adapter	Executable action masked
Robot demonstrations, teleoperation	Executable actions, proprioception Stages and outcomes when annotated	$\mathcal{L}_{\text{action}}$, $\mathcal{L}_{\text{state}}$ ($\mathcal{L}_{\text{progress}}$, $\mathcal{L}_{\text{outcome}}$)	Action expert (and adapters)	Unannotated items masked
Simulation	Dense state, contact, action, outcome	All five	Backbone and action expert	—
Deployment logs (interventions, failures)	Outcomes, corrections, takeovers	$\mathcal{L}_{\text{outcome}}$ Value/critic targets at later stages	Value head, policy via post-training	Per record type

How such multi-task, multi-embodiment data are curated, balanced, and mixture-weighted is a data engineering question treated in Chapter 4. This section examines how the composition of the mixture enters optimization. On the training side, a curriculum may progress from short-horizon skills with limited contact to multistage tasks, perturbed states, and failure recovery, and it should specify success and failure boundaries so that the model learns when to continue execution, acquire new observations, or terminate safely. As action learning progresses, retaining existing multimodal knowledge becomes an additional concern. When an action expert is connected to a pretrained vision-language backbone, gradient updates from action learning may interfere with established semantic representations. Staged unfreezing, parameter-efficient fine-tuning, gradient isolation, and knowledge distillation can help preserve language understanding and visual knowledge while new action capabilities are acquired. The heterogeneous joint training used in $\pi_{0.5}$ and the Knowledge Insulation approach illustrate this principle. Training across diverse data sources expands task and environmental coverage while limiting interference with existing multimodal knowledge.

Pretraining should consequently be evaluated using more than training loss or offline action prediction error. Evaluation should also examine closed-loop performance on unseen objects, novel combinations of instructions, and mildly perturbed conditions. These tests assess whether shared representations support perception, decision-making, and action under the new conditions. Pretraining supplies an initialization for the task- and embodiment-specific adaptation discussed next. Where the prior does not yet support the target task or embodiment, supervised post-training is one route to consistent execution. Outcome-based adaptation provides another, as developed below. Some pipelines also introduce an explicit alignment stage between broad pretraining and task adaptation. Section 6.6.2 examines these choices for dexterous hands through EgoScale on Sharpa Wave.

6.3.2 Supervised Post-Training: From General Priors to Executable Policies

Supervised post-training uses expert demonstrations, teleoperation trajectories, task procedures, and high-quality simulated data to adapt general behavioral priors to specific tasks, embodiments, and environments. Behavioral cloning increases the likelihood of expert actions conditioned on observations, task instructions, and embodiment characteristics, bringing the policy closer to the behavioral distribution represented in the demonstrations. Its effectiveness depends first on whether demonstrations cover the complete task. Training examples should include task initiation, critical contacts, transitions between stages, completion verification, and safe termination, rather than being restricted to locally standardized actions extracted from successful trajectories. For multistep tasks, stage labels, subgoals, and outcome criteria should also be recorded to support learning of temporal relationships among actions.

Adaptation to a new task or embodiment may then use full-parameter fine-tuning, updates to selected modules, or parameter-efficient methods such as low-rank adaptation (LoRA). Full-parameter fine-tuning offers greater scope for parameter adjustment but requires more data and computation. Parameter-efficient methods facilitate rapid iteration, although their effectiveness may be limited by discrepancies between the new task and existing representations. For a VLA initialized from a pretrained vision-language model, parameter-efficient adaptation has a further benefit. The backbone can be kept frozen so that the semantic priors built during pre-training are preserved, while a small number of action-side parameters are learned. Gradient isolation during pretraining addresses the same interference concern. Results reported for OpenVLA and Octo (Kim et al., 2025b; Ghosh et al., 2024) indicate that large-scale multitask pretraining can substantially reduce the data required for downstream adaptation. Performance after adaptation nevertheless depends on appropriate treatment of sensors, action spaces, and embodiment characteristics.

Even with high-quality demonstrations, behavioral cloning faces a discrepancy between training and deployment state distributions. Training typically uses states visited by an expert, whereas deployment states are determined by the model's own actions. A small localization error can cause subsequent observations to depart from the demonstration distribution, allowing errors to compound over time. Interactive dataset aggregation addresses this problem by executing the current policy, requesting expert actions or corrections at the off-distribution states it encounters, and

adding these examples to the training set. DAgger exemplifies this approach to distribution shift in sequential decision-making. For a fine-tuned VLA, the expert is typically a teleoperator who shares the policy's action interface, so corrective labels can be appended to the training set without retargeting, and interactive aggregation becomes a natural extension of demonstration collection rather than a separate pipeline. Demonstration collection thus extends beyond recording ideal execution to include difficult states and recovery behaviors encountered by the model itself.

Human intervention data should be differentiated according to their training function. Valid actions preceding a takeover can serve as positive demonstrations. Human corrections can provide supervised labels for off-distribution states. And failure outcomes can support the training of success classifiers or value models. Erroneous actions, however, should not automatically become imitation targets. This treatment preserves both the conditions under which errors arise and the corresponding recovery strategies, enabling supervised post-training to learn normal execution as well as recovery to feasible states. Nevertheless, supervised post-training primarily learns conditional relationships between states and actions in demonstration data. It does not directly optimize completion time, energy consumption, precision, or long-term success rate. It is well suited to establishing a stable initial policy, particularly when operating procedures are clearly defined and expert demonstrations are of high quality, but reductions in training loss cannot substitute for closed-loop validation on the physical system.

Validation should therefore cover instruction following, action executability, critical state transitions, outcome assessment, and termination under abnormal conditions. For tasks such as liquid transfer and precision assembly, success criteria should also incorporate mass, flow rate, applied forces, leakage, and instrument readings, explicitly distinguishing the execution of action commands from the achievement of task objectives. When a model can reliably follow procedures but remains limited in efficiency, contact precision, or recovery from complex disturbances, feedback from action outcomes is needed to support policy improvement beyond imitation.

6.3.3 Reinforcement Learning-Based Post-Training: Improving Policies Through Action Outcomes

Behavioral cloning can reproduce demonstrator behavior, but its objective does not directly address efficiency gains subject to success-rate requirements, action adjustment under changing contact conditions, recovery from states outside the demonstration distribution, or improvements in action sequences discovered through autonomous interaction. Reinforcement learning evaluates the long-term consequences of actions through environmental feedback, enabling continued policy improvement toward task objectives. For Physical AI, it is particularly relevant to precise contact interactions, long-horizon tasks, disturbance recovery, and efficiency optimization. Reinforcement learning is not, however, necessary for every task. When procedures are stable, demonstrations provide sufficient coverage, and performance already meets requirements, supervised training and deterministic control may be more appropriate. The decision to introduce reinforcement learning-based post-training should depend on the limitations of the existing policy and whether environmental feedback provides an effective signal for addressing them.

Compared with virtual environments, reinforcement learning on physical robots involves slow interactions, costly resets, equipment wear, delayed rewards, and safety risks. States are often partially observable, contact and friction exhibit discontinuities, and outcomes may also depend on material batches, temperature, and tool condition. Large-scale random exploration in the physical world is therefore generally impractical. Under these constraints, policy optimization typically seeks to maximize expected cumulative return while accounting for task rewards, safety costs, and energy consumption, with discounting controlling the relative weight of returns over different time horizons. For physical robot tasks, this objective can be written as

$$J(\pi_\theta) = \mathbb{E}_{\tau_{\text{ep}} \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t \left(\lambda_{\text{g}} r_t^{\text{goal}} + \lambda_{\text{p}} r_t^{\text{progress}} - \lambda_{\text{safe}} c_t^{\text{safety}} - \lambda_{\text{e}} c_t^{\text{energy}} \right) \right], \tag{6.8}$$

where the terms represent task achievement, progress, safety cost, and energy consumption, respectively. Task rewards, progress, safety constraints, and resource consumption should be recorded

separately to determine whether apparent gains reflect improved task performance or exploitation of a particular metric. Safety penalties can help a policy learn risk boundaries, but they cannot replace independent limit enforcement, collision detection, emergency stops, or human takeover mechanisms. Reward design must likewise guide learning while remaining verifiable against physical outcomes. Terminal success signals are simple and reliable but too sparse for many long-horizon tasks. Intermediate rewards can capture distance to the goal, contact quality, trajectory error, completion time, and stage progress, yet may be exploited in ways that diverge from the actual objective. Reward design should therefore prioritize observable, independently verifiable physical quantities, such as changes in mass, position error, peak force, leakage status, and instrument readings, and validate them against independently assessed final outcomes.

Alongside rewards, value functions estimate how current states and actions contribute to future task outcomes, helping the policy identify behaviors with long-term benefits. Vision-language evaluators can infer progress, completion, and stagnation from successive observations, providing denser feedback for physical robots. Such evaluators may nevertheless misclassify outcomes, and policies may exploit evaluation biases through reward hacking. Calibration against physical outcomes is therefore necessary. VLAC (Zhai et al., 2025) illustrates an approach that uses vision-language models for progress estimation and asynchronous reinforcement learning on real robots. These methods broaden the sources of feedback while reinforcing the need to cross-check intermediate evaluations against final outcomes.

For a fine-tuned VLA, a practical pipeline begins offline. The demonstrations, successful and failed trajectories, and human corrections already accumulated during supervised post-training form the dataset from which values are estimated and a policy is derived, reducing the risks of physical exploration. A subsequent online stage can collect a limited amount of autonomous execution data, focusing on states associated with model uncertainty, repeated failures, or high informational value. Training should constrain the policy from deviating excessively from behaviors supported by the data, thereby limiting overly optimistic value estimates for actions absent from the dataset. Human intervention can also be adjusted as the policy improves. Takeovers can prevent hazardous exploration early in training, corrective actions can address difficult states at intermediate stages, and later interventions can be confined to infrequent anomalies. HIL-SERL (Luo et al., 2025) combines demonstrations, reward classifiers, and progressively reduced human intervention to improve the sample efficiency of reinforcement learning on physical robots, illustrating the complementary roles of demonstrations, autonomous interaction, and human correction.

For large VLA models, conventional online policy-gradient methods such as PPO (Schulman et al., 2017) often entail substantial sample requirements and challenges in training stability. An alternative is to collect experience in batches and update the policy between collection rounds. The RECAP method used in $\pi^*_{0.6}$ (Amin et al., 2026) integrates autonomous execution outcomes, human corrections, and existing demonstrations within iterative offline reinforcement learning. It estimates advantages using a value function and then derives an improved policy through advantage conditioning. This approach illustrates how reinforcement learning on physical robots can alternate between batch data collection, offline policy updates, and validation on the robot, without modifying model parameters after every action. The same workflow supports independent validation between updates to determine whether policy improvements persist during physical execution.

To further reduce physical interaction costs, world models can compare the short-term consequences of candidate actions before execution, helping to screen out behaviors likely to cause collisions, slippage, leakage, or task failure. They can also generate supplementary experience in simulation or latent space. However, world models are subject to modeling errors, which a policy may exploit to obtain inflated estimates of return. They are therefore useful for candidate evaluation, risk prediction, and data augmentation, but cannot replace physical outcome measurements, low-level controllers, or independent safety mechanisms. Regardless of the training approach, evaluation should report success rate, completion time, precision, failure recovery rate, the number of physical interactions, human intervention rate, and the number of safety incidents. Improvements beyond demonstration performance can be established only when gains persist on independent

test tasks and under physical disturbances. Retaining and extending those gains during long-term deployment additionally requires continual learning.

6.3.4 Continual Learning: Turning Deployment Experience into Lasting Capabilities

Long-term deployment exposes embodied agents to situations that cannot be fully anticipated during pretraining or post-training. The resulting need for continued adaptation can be understood in light of Sutton's Big World Hypothesis. As Javed and Sutton (2024) articulate, many decision-making environments are far larger and more complex than an agent's perceptual, representational, and computational resources can accommodate. An agent must therefore act on an incomplete understanding of its environment, learning and revising the knowledge most relevant to its current decisions rather than relying on an exhaustive representation of all possible states and optimal actions. For embodied systems, this perspective provides a rationale for continual learning. Capabilities acquired before deployment provide an initial basis for action, but must be evaluated and refined as interaction reveals unfamiliar states, changing task demands, and limitations of the current policy. Such refinement must also preserve previously acquired skills that remain relevant. Supporting this process requires experience records that retain the temporal, spatial, and embodiment context of each interaction, including the model version, task instruction, sensor state, commanded action, observed outcome, and any anomaly or human intervention. Distinguishing successful execution, failure, warnings, takeovers, and recovery within these records allows deployment experience to inform subsequent supervised training, value estimation, and policy improvement.

As experience accumulates, the central challenge in continual learning shifts from increasing data volume to identifying the learning value of individual experiences. Samples can be selected according to failure frequency, model confidence, state novelty, intervention points, and environmental changes, while balancing task value, information gain, collection cost, and risk. Recurrent grasp slippage, for example, calls for additional data on contact states and recovery actions. Occasional instrument anomalies require complete telemetry and procedural context, rather than a final failure label alone. Active sampling should also help identify capability boundaries, allowing the system to distinguish tasks already mastered under current conditions from changes addressable through context updates and changes requiring parameter updates or retraining. This distinction links newly collected data to specific capability gaps and prevents local adaptation from being conflated with improvements in general competence.

Clarifying the form of adaptation is also essential to managing the relationship between new and existing capabilities. Immediate adaptation through context or memory, and runtime calibration through updates to state estimates or embodiment parameters, do not necessarily modify the foundation model. Long-term learning that changes model parameters, by contrast, requires model version control and regression testing. As new tasks arrive and parameters are updated, catastrophic forgetting, interference across tasks, and policy drift may occur. Continual learning therefore needs to optimize new-task adaptation while retaining previously acquired capabilities (Kirkpatrick et al., 2017; Dohare et al., 2024), using an objective of the form

$$\mathcal{L}_{\text{CL}} = \mathcal{L}_{\text{new}} + \lambda_{\text{retain}}\mathcal{L}_{\text{retain}} + \lambda_{\text{reg}}\mathcal{L}_{\text{reg}}, \tag{6.9}$$

where $\mathcal{L}_{\text{new}}$ measures adaptation to new tasks, $\mathcal{L}_{\text{retain}}$ preserves previous capabilities, and $\mathcal{L}_{\text{reg}}$ limits undesirable parameter changes. Replay of representative experiences can preserve coverage of previous task distributions. Knowledge distillation can constrain the new model's behavior on earlier tasks. Parameter regularization can limit changes to parameters important for retained knowledge. And modular architectures can isolate new skills within specific components. Methods such as elastic weight consolidation (Kirkpatrick et al., 2017) mitigate forgetting by restricting changes to important parameters, although their effectiveness in embodied tasks must still be validated through physical interaction. Continual learning should therefore be evaluated in terms of both gains on new tasks and retention of existing skills.

As illustrated in Figure 6.5, continual learning should follow a controlled iterative process to make capability gains verifiable. The process should identify capability gaps, collect targeted experience, update the model offline, jointly validate performance on new and existing tasks together with compliance with safety constraints, and deploy only after predefined performance and

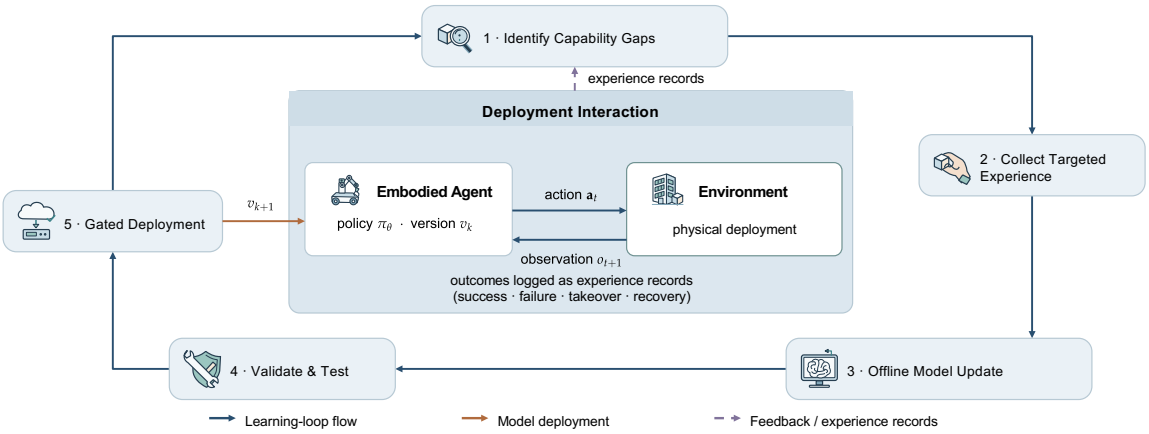


Fig. 6.5 The continual learning loop for embodied agents in long-term deployment. Deployment interactions are logged as experience records (success, failure, takeover, recovery), and each iteration updates the policy version v_k to v_{k+1} through gap identification, targeted collection, offline update, validation, and gated deployment. The process is detailed in Section 6.3.4

safety requirements are met. Previous model versions should remain available for rollback. For high-risk tasks, validation should begin in simulation, hardware-in-the-loop settings, or restricted workspaces before the scope of physical operation is gradually expanded. Each update should establish whether performance on new tasks has improved, whether previous capabilities have been retained, how much data and human effort adaptation required, and whether failure recovery has become more reliable. Deployment experience can be regarded as a source of established system capabilities only when these results are reproduced across independent tasks, different batches of objects, and sustained operation. It can then provide a reliable basis for subsequent task adaptation and policy optimization.

As summarized in Figure 6.4, capability development in foundation models for robot manipulation thus proceeds through a feedback loop of prior acquisition, task adaptation, outcome-based optimization, and experience accumulation, rather than a direct transition from large-scale pretraining to autonomous operation. Pretraining supports shared semantic and behavioral structure. Supervised post-training enables execution of procedures on specific embodiments. Reinforcement learning-based post-training uses physical outcomes to improve precision, efficiency, and recovery. And continual learning supports the long-term retention of these gains and their transfer to new tasks and environments. Each stage has a distinct training emphasis, while demonstrations, interaction feedback, and deployment experience connect them. Together, they support the progression from demonstration-driven behavioral learning toward continual policy optimization grounded in physical feedback.

6.4 Multi-Embodiment Modeling and Model Adaptation

The action interfaces in Section 6.2 specify how a model generates behavior, while the pre-training and post-training procedures in Section 6.3 establish reusable behavioral priors. Realizing these priors on a different physical system requires compatible observation and action interfaces, together with adaptation to its morphology and interaction dynamics. Here, an *embodiment* denotes the robot platform together with its end-effectors, sensing and actuation systems, and control interfaces. Replacing a parallel gripper with a dexterous hand therefore changes the embodiment even if the manipulator remains unchanged. The substitution alters the available contact modes, action dimensionality, sensing geometry, and feasible manipulation strategies. Multi-embodiment modeling seeks to transfer task-relevant knowledge across these configurations while preserving the physical distinctions that determine executable behavior. Cross-robot training in Open X-

Embodiment provides empirical evidence that heterogeneous interaction data can support such transfer, although its reported evaluations do not establish generalization to arbitrary unseen robots (Open X-Embodiment Collaboration, 2024).

Following the partially observable formulation in Chapter 2 and the distinction between physical state and internal representation in Chapter 5, let $e \in \mathcal{E}_{\text{emb}}$ index an embodiment and write

$$\mathcal{M}_e = (\mathcal{S}_e, \mathcal{A}_e, P_e^{\text{tr}}, R_{\ell_{\text{lang}},e}, \mathcal{O}_e, O_e, \gamma), \tag{6.10}$$

where $\mathcal{S}_e$ includes the physical state of the robot and its environment. $\mathcal{A}_e$ is the action space at the declared control interface. P_e^{tr} is the transition kernel. $R_{\ell_{\text{lang}},e}$ specifies the reward for instruction ℓ_{lang}. $\mathcal{O}_e$ is the measurement space. O_e is the observation likelihood, and γ is the discount factor. The measurements comprise exteroceptive observations o_t and proprioceptive measurements $\mathbf{q}_t$, retaining their separate notation from Section 6.1. The kernel P_e^{tr} corresponds to the embodiment-conditioned transition kernel T in Chapter 5, with elapsed time included when required. The measurement pair $(o_t, \mathbf{q}_t)$ belongs to $\mathcal{O}_e$, whereas $O_e(o_{t+1}, \mathbf{q}_{t+1} \mid s_{t+1}, \mathbf{a}_t)$ denotes its conditional observation likelihood. Actuator memory and other dynamic variables must be included in the physical state when needed for the Markov assumption. The subscript on $\mathcal{S}_e$ allows robot configurations with different state dimensions. A shared task representation does not require identical physical state spaces.

Using the history $h_t = (o_{0:t}, \mathbf{q}_{0:t}, \mathbf{a}_{0:t-1})$ and action chunk $\mathbf{A}_t^{\text{seq}} = [\mathbf{a}_t^\top; \ldots; \mathbf{a}_{t+H-1}^\top]$ defined in Eq. (6.1), the policy interface remains

$$\mathbf{A}_t^{\text{seq}} \sim \pi_\Theta(\cdot \mid h_t, \ell_{\text{lang}}, e), \tag{6.11}$$

where Θ denotes the complete set of policy parameters, while θ and ψ below denote representation and action-generation parameters, respectively. The horizon H counts action steps. Its physical duration also depends on the command intervals. A current-observation policy is a restricted instance of this history-conditioned formulation. The modeling objective is to share information about task goals, object relations, and behavioral organization while retaining the embodiment information required for action feasibility and contact regulation.

6.4.1 Embodiment-Aware Data Representation and Alignment

Pooling trajectories across robots requires compatibility of physical meaning, not merely compatibility of tensor shapes. Let $\mathcal{D}_e$ denote the collection of trajectories associated with embodiment e. Each trajectory retains its instruction, ordered observations and actions, outcome annotations, and acquisition metadata. Let m_t^{meta} denote acquisition metadata. Metadata should identify the robot and end-effector configuration, sensor calibration, coordinate frames, units, control mode, timestamps, command duration, latency, and the data-collection policy or operator. These fields determine how a recorded command relates to an observed physical consequence.

Semantic, Geometric, and Dynamic Alignment Semantic alignment concerns task goals, object roles, and stages of behavior. Two robots can share the goal of opening a container while requiring different grasps and motion sequences. Geometric alignment concerns reference frames, pose conventions, kinematics, and correspondences between observation and action representations. A command expressed in a camera frame, tool frame, or joint space requires an explicit interpretation before it can be transferred. Dynamic alignment concerns the embodiment-dependent transition kernel P_e^{tr}, including actuator response, payload, friction, compliance, and execution delay. Geometric correspondence alone does not establish equivalent contact behavior.

These distinctions also delimit what data standardization achieves. The RT-X experiments use a canonical camera view and a coarsely aligned end-effector action interface, with dataset-specific action normalization. The original control conventions are retained, and action coordinate frames are not fully aligned across datasets (Open X-Embodiment Collaboration, 2024). Thus, numerical normalization facilitates joint training without making identical action vectors physically interchangeable. UMI illustrates a complementary interface-level approach in which relative trajectories

and inference-time latency matching support transfer of demonstrations collected with handheld grippers to compatible robot systems (Chi et al., 2024). Neither approach removes the need to verify reachability, tracking, and contact behavior on the receiving embodiment.

Heterogeneous Sensing and Action Interfaces Observation interfaces may differ in camera count, viewpoint, and calibration. They may also differ in whether proprioceptive, force, or tactile measurements are available. Modality-specific tokenizers, sensor identity encodings, and validity masks can expose these differences to a shared model. Octo supports adding or removing observation tokenizers and adapting to new sensory inputs. HPT uses embodiment-specific input stems to map heterogeneous visual and proprioceptive inputs into a common token interface (Ghosh et al., 2024; Wang et al., 2024c). Missing measurements should remain distinguishable from valid zero-valued measurements. For high-rate contact signals, timestamps and observation windows must also preserve the temporal relationship between commands and measured interaction.

Action interfaces require comparable care. A Cartesian displacement, joint target, velocity command, and force target define different interventions even when their numerical dimensions coincide. Padding and normalization can accommodate dimensional heterogeneity, but do not resolve differences in units, control mode, or effective duration. The original π_0 implementation pads state and action vectors to a common dimensionality and masks unused image slots (Black et al., 2025a). Such implementation choices should be specified for the particular model rather than treated as universal conventions of cross-embodiment learning. Likewise, resampling a trajectory does not establish dynamic equivalence between robots operating at different control rates.

Complementary Supervision Sources Human video provides evidence about task structure, object transformations, and behavioral stages, but usually lacks robot-executable action labels. Robot demonstrations and teleoperation provide action-grounded supervision under a specified interface. Simulation supports controlled variation of morphology and dynamics, whereas autonomous rollouts expose the state distribution induced by the executing policy. These sources therefore supervise different aspects of transfer, as discussed in Section 6.3.1 and Chapter 4. HPT combines data from robot teleoperation, deployed robots, simulation, and human video through heterogeneous input and output interfaces (Wang et al., 2024c). For human video without robot action labels, it uses estimated poses and two-dimensional positions as surrogate supervision rather than treating visual motion as a robot command. Failures, interventions, corrections, and recovery segments should remain identifiable because they represent different supervision signals and operating conditions.

Embodiment-aware representation consequently establishes the conditions under which data can be combined or transformed. The desired internal representation z_t should suppress irrelevant variation while preserving information about physical differences that affect the next decision. It is a learned summary of available evidence, following Chapter 5, rather than a physical state or a calibrated belief distribution by definition.

6.4.2 Shared Representations, Execution Interfaces, and Adaptation

The decomposition introduced in Eq. (6.2) can be specialized to heterogeneous observation and action spaces. Let η_e describe the relevant morphology, sensing, actuation, and control properties of embodiment e. A modular policy may be written as

$$\begin{aligned} \bar{h}_t &= E_{\rho_e}(h_t), \\ z_t &= f_\theta(\bar{h}_t, \ell_{\text{lang}}, \eta_e), \\ \mathbf{A}_t^{\text{seq}} &\sim \pi_{\psi_e}(\cdot \mid z_t, \ell_{\text{lang}}, \eta_e), \end{aligned} \tag{6.12}$$

where E_{ρ_e} is an observation-interface encoder, f_θ is a shared representation model, and π_{ψ_e} generates action chunks in $\mathcal{A}_e^H$. This notation retains θ for representation parameters and ψ for policy parameters, as in Section 6.1. The subscripts identify the functional interfaces that may depend on embodiment. They do not require a separate network for every robot. Architectures may

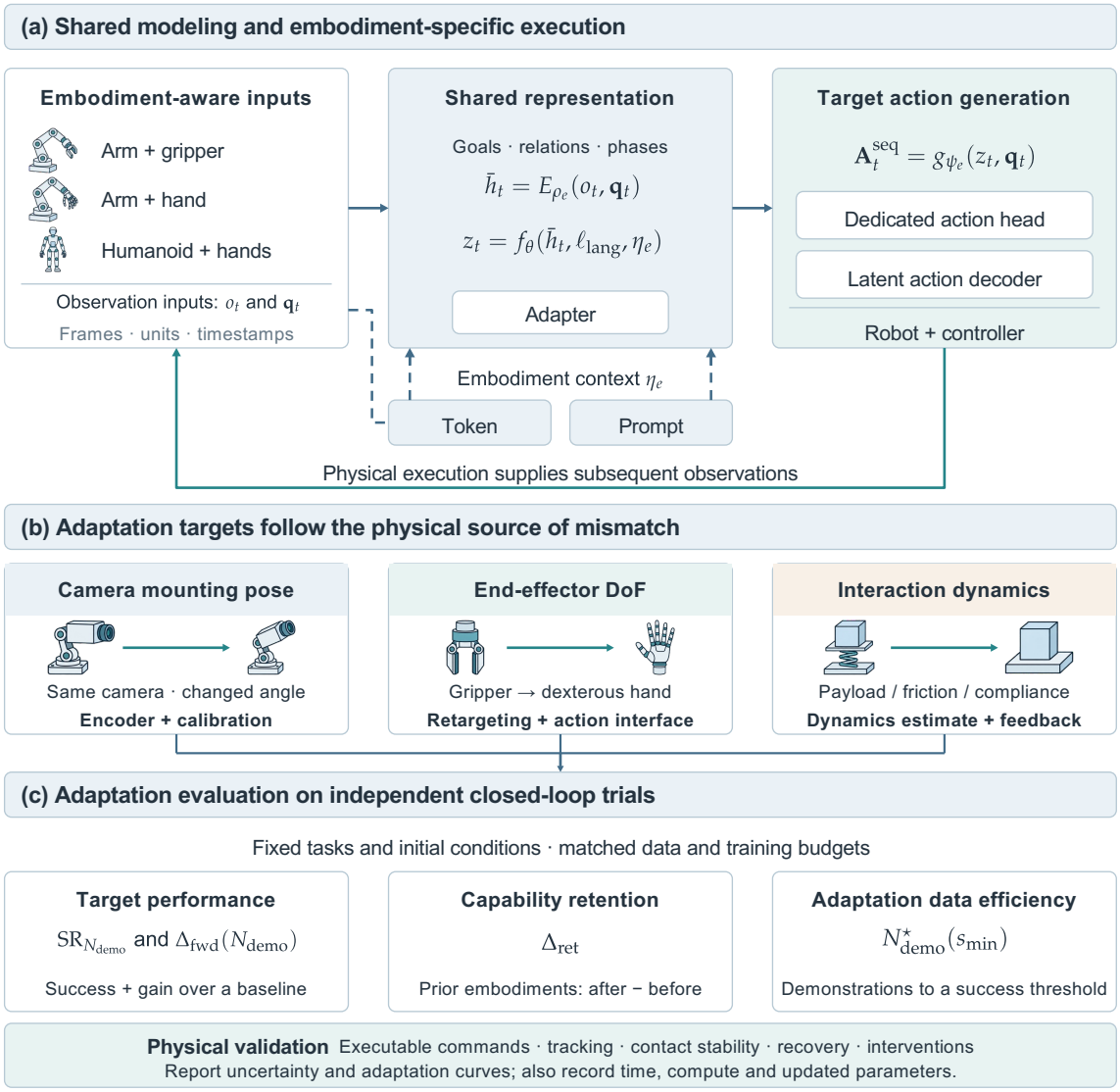


Fig. 6.6 Shared modeling, embodiment-specific adaptation, and adaptation evaluation for manipulation. (a) Observation interfaces, shared task representations, and executable outputs illustrate complementary token, prompt, adapter, action-head, and latent-decoder mechanisms. (b) Changes in camera mounting pose (with camera count unchanged), end-effector degrees of freedom (DoF), and interaction dynamics motivate corresponding adaptation targets. (c) Independent closed-loop trials assess target success and budget-matched transfer gain, retention on previously supported embodiments, adaptation data efficiency, and physical execution. For compactness, panel (a) depicts current-observation inputs and deterministic execution mappings. Eq. (6.12) gives the general history-conditioned policy formulation

share some or all of ρ_e and ψ_e and condition their computation on η_e. A deterministic decoder $\mathbf{A}_t^{\text{seq}} = g_{\psi_e}(z_t, \mathbf{q}_t)$, as depicted in Figure 6.6, is a special case. $\mathbf{q}_t$ is already contained in h_t but may also enter the decoder directly.

The descriptor η_e may comprise measured calibration quantities, a learned embedding, a structured prompt, or an estimate inferred from interaction history. Explicit identity conditioning and

inferred physical parameters serve different purposes. An identity token can index a known configuration, whereas adaptation to unobserved payload or compliance requires evidence about the changed dynamics. Shared features need not be strictly embodiment-invariant. For example, a representation of a feasible grasp should depend on hand geometry even when the object and task are unchanged.

Embodiment Representations and Adaptation Mechanisms The descriptor and interface modules in Eq. (6.12) can be implemented through tokens, prompts, adapters, dedicated action heads, or latent action decoders. These mechanisms differ in where embodiment information enters the model and which components are adapted. They are complementary rather than mutually exclusive. A soft prompt can itself consist of tokens, and a dedicated action head can incorporate a latent decoder.

Embodiment information can be introduced as learned identity embeddings or structured tokens encoding morphology and control properties. Structured tokens expose joint attributes and kinematic relationships to the shared transformer, providing a representation of physical configuration beyond a categorical robot label. Suzuki et al. combine joint-wise kinematic action tokens with topology-aware attention and joint-attribute conditioning (Suzuki et al., 2026). Their kinematic tokens organize action trajectories, while topology and joint descriptors introduce morphological information. The latter instantiate embodiment context associated with η_e. The action tokens themselves should be distinguished from an identity descriptor. An identity embedding indexes a known configuration, whereas generalization to an unseen configuration requires a representation of its relevant physical properties.

An embodiment-specific prompt supplies contextual information that modulates shared computation. A structured textual prompt may describe available sensors or the action interface, while a soft prompt represents such context through trainable continuous embeddings. X-VLA introduces separate learnable soft prompts for heterogeneous data sources, together with domain-specific projections for action-related inputs and outputs (Zheng et al., 2026a). This illustrates how embodiment-dependent context can be incorporated without duplicating the shared backbone. Prompt conditioning should be distinguished from the task instruction ℓ_{lang}. The former describes the execution configuration, while the latter specifies the desired behavior.

Parameter-efficient adapters introduce a restricted set of trainable parameters within a pretrained model, allowing the shared computation to be adjusted for a target configuration. LoRA parameterizes weight updates through low-rank factors, and OpenVLA evaluates this approach for data-efficient robot-policy fine-tuning (Hu et al., 2022a; Kim et al., 2025b). In Eq. (6.13), the adapter parameters can constitute Θ_{upd} while the remaining pretrained weights are fixed. The resulting update changes the model's internal computation. It does not by itself define a compatible observation or action interface.

An embodiment-specific head maps shared features to the dimensionality, units, and control semantics required by the target robot, implementing π_{ψ_e} or its deterministic counterpart g_{ψ_e}. HPT explicitly combines a shared transformer trunk with embodiment-specific input stems and policy heads, reinitializing the head for transfer to a new embodiment (Wang et al., 2024c). Octo similarly permits new observation tokenizers and action readouts when sensory inputs or action spaces change (Ghosh et al., 2024). Interface modularity does not prescribe a head-only update. Input modules, readouts, and shared features can be adapted jointly rather than restricting learning to the action head.

A shared latent action space permits policy learning over a common behavioral representation while an embodiment-specific decoder reconstructs executable actions. Unlike the context representation z_t, which summarizes evidence for decision-making, the latent action represents the behavior proposed by the policy. Latent Action Diffusion learns aligned end-effector action representations using contrastive encoders, trains a diffusion policy in that latent space, and selects the corresponding embodiment-specific decoder during execution (Bauer et al., 2026). The action-generation interface is thereby factorized into shared latent prediction and embodiment-dependent

reconstruction. Its transfer capacity depends on the learned alignment and the target decoder's ability to realize feasible actions.

These mechanisms instantiate different parts of the shared-execution decomposition. Tokens and prompts principally supply embodiment context, adapters modify selected internal computations, and action heads or latent decoders determine how shared information becomes an executable output. The appropriate combination depends on whether the mismatch concerns sensing, morphology, action semantics, or interaction dynamics.

Adaptation Targets and Learning Regimes For a target configuration $e^\star$, adaptation should address the source of mismatch. A change in camera mounting position or orientation, even with the same number of cameras, primarily affects observation encoding and calibration. A change in end-effector degrees of freedom, such as replacing a parallel gripper with a dexterous hand, affects action dimensionality, kinematic retargeting, feasible contact configurations, and action decoding. Changed payload, friction, or actuator response affects dynamics estimation and feedback-dependent execution. Several components may require joint adaptation, but the update scope should follow the identified mismatch rather than being determined solely by a generic fine-tuning recipe.

Zero-shot transfer uses the pretrained policy and a declared target interface without target-specific learning data. Any calibration or robot description available at inference should be reported separately. Few-shot adaptation uses a stated, limited target-data budget. Offline and online adaptation describe when the adaptation evidence is collected and used, and are independent of the number of demonstrations. Moreover, adaptation through context, state estimation, or calibration need not change model weights, whereas fine-tuning does. RoboCat explicitly studies adaptation to new tasks and embodiments using target demonstrations (Bousmalis et al., 2024).

Target Performance and Capability Retention The complete parameter set Θ includes the representation parameters, relevant input and action-interface parameters, and any trainable embodiment descriptors. Let Θ_0 denote their values before target adaptation, including the initialization of newly introduced interface modules, and let Θ_{upd} denote the selected trainable subset. Following Eq. (6.9), parameter adaptation can be formulated as

$$\begin{aligned} \Theta^\star_{\text{upd}} &\in \arg\min_{\Theta_{\text{upd}}} \mathcal{L}_{\text{adapt}}(\Theta), \\ \mathcal{L}_{\text{adapt}}(\Theta) &= \mathcal{L}_{\text{new}}(\Theta; \mathcal{D}_{e^\star}) + \lambda_{\text{retain}} \mathcal{L}_{\text{retain}}(\Theta; \mathcal{D}_{\text{ret}}) + \lambda_{\text{reg}} \mathcal{L}_{\text{reg}}(\Theta, \Theta_0). \end{aligned} \tag{6.13}$$

All parameters outside Θ_{upd} remain fixed. The target loss $\mathcal{L}_{\text{new}}$ uses the action-generation objective appropriate to Section 6.2, such as token likelihood, regression, diffusion, or flow matching. The retention dataset $\mathcal{D}_{\text{ret}}$ represents previously supported embodiments, and its loss can use replay supervision or behavioral distillation. The nonnegative coefficients control the relative weighting of these objectives. Consistent with Section 6.3, $\mathcal{L}_{\text{reg}}$ constrains undesirable parameter changes. Data and computational budgets are reported separately rather than identified with this regularizer. Elastic weight consolidation exemplifies parameter-based protection of previously acquired knowledge (Kirkpatrick et al., 2017), but such a regularizer does not guarantee retained closed-loop manipulation performance.

Negative transfer can result from incompatible action semantics, an imbalanced training mixture, or shared features that omit information essential to one embodiment. Embodiment-aware sampling, modular interfaces, replay, and targeted regression evaluation can reduce these sources of interference. Physical feasibility and interaction limits must additionally be enforced and validated at the relevant execution interface. Minimizing Eq. (6.13) alone does not establish them.

6.4.3 Adaptation Validation and Deployment Handoff

Claims of successful embodiment adaptation require evaluation on independently collected target-embodiment trials. Offline action agreement does not measure the state distribution visited by the

executing policy or its response to contact disturbances. Evaluation should distinguish supported embodiments from held-out embodiments and separately control changes in task, object, scene, operator, and acquisition period. Splits should be made at the trajectory or collection-session level, with paired or retargeted versions of one demonstration grouped together, to prevent correlated samples from inflating generalization estimates.

Target Performance and Transfer Gain Let $\mathcal{E}_{\text{emb}}^{\text{tgt}} \subseteq \mathcal{E}_{\text{emb}}$ be a finite, nonempty set of target embodiments, and let $\mathcal{K}_e^{\text{eval}}$ be the finite, nonempty evaluation task set for embodiment e. Write $\text{SR}_{N_{\text{demo}}}(e,k) \in [0,1]$ for the success rate on task k after adaptation with N_{demo} target demonstrations. Here, N_{demo} is the adaptation-data budget, distinct from the number of evaluation trials. Task completion criteria, initial-state distributions, and aggregation weights must be fixed before comparison. Report absolute target success separately from improvement over a baseline. A budget-matched transfer-gain measure is

$$\begin{aligned} \Delta_{\text{fwd}}(N_{\text{demo}}) = \sum_{e \in \mathcal{E}_{\text{emb}}^{\text{tgt}}} \sum_{k \in \mathcal{K}_e^{\text{eval}}} w_{e,k} \big[\text{SR}_{N_{\text{demo}}}^{\text{adapt}}(e,k) - \text{SR}_{N_{\text{demo}}}^{\text{base}}(e,k) \big], \\ w_{e,k} \geq 0, \qquad \sum_{e \in \mathcal{E}_{\text{emb}}^{\text{tgt}}} \sum_{k \in \mathcal{K}_e^{\text{eval}}} w_{e,k} = 1. \end{aligned} \tag{6.14}$$

For example, the baseline may use the same architecture trained without cross-embodiment pre-training, under the same target-data and target-training budget. This comparison matches the adaptation budget. Pre-training data and computation must be reported separately. A positive difference then supports the benefit of the initialization under that protocol, rather than being inferred from target success alone. Zero-shot comparisons require a separately specified reference policy. This measure is inspired by transfer evaluation in continual learning (Lopez-Paz and Ranzato, 2017), but is defined here for embodiment adaptation and is not identical to the original sequential-task forward-transfer metric. Cross-robot specialist comparisons in Open X-Embodiment likewise motivate stating the reference system explicitly (Open X-Embodiment Collaboration, 2024).

Retention on Previously Supported Embodiments Let $\mathcal{E}_{\text{ret}}$ be a finite, nonempty set of previously supported embodiments on which retention is assessed, excluding the adaptation target when appropriate. Use the same tasks, initial-state distributions, evaluation criteria, and task-aggregation weights before and after adaptation, and define

$$\Delta_{\text{ret}} = \frac{1}{|\mathcal{E}_{\text{ret}}|} \sum_{e \in \mathcal{E}_{\text{ret}}} \big[\text{SR}_{\text{after}}(e) - \text{SR}_{\text{before}}(e) \big]. \tag{6.15}$$

Negative values indicate average degradation under the stated protocol. Retention testing should also report individual embodiment and task results, because an average gain can conceal a substantial loss on one configuration. Any acceptance tolerance should be chosen before adaptation and interpreted with trial counts and uncertainty. This operational measure is related to backward transfer (Lopez-Paz and Ranzato, 2017). It evaluates retained behavior rather than directly measuring the training loss $\mathcal{L}_{\text{retain}}$.

Adaptation Data Efficiency For a target embodiment $e^\star$, a prespecified success threshold $s_{\min} \in [0,1]$, and a set of tested demonstration budgets $\mathcal{N}_{\text{demo}} \subseteq \mathbb{N}_0$, define

$$N_{\text{demo}}^\star(s_{\min}) = \min\big\{ N_{\text{demo}} \in \mathcal{N}_{\text{demo}} : \text{SR}_{N_{\text{demo}}}(e^\star) \geq s_{\min} \big\}, \qquad \min \varnothing := +\infty, \tag{6.16}$$

where $\text{SR}_{N_{\text{demo}}}(e^\star)$ denotes a prespecified weighted average of $\text{SR}_{N_{\text{demo}}}(e^\star, k)$ over the fixed target task set, with nonnegative weights summing to one. The condition $N_{\text{demo}} = 0$ represents zero-shot evaluation when included. This is an empirical summary over tested budgets, not a guarantee of monotonic improvement with additional demonstrations. Report the full adaptation curve, uncertainty across trials and training runs, demonstration duration, and the amount of any additional calibration or interaction data. RoboCat's evaluation across demonstration budgets

illustrates why data requirements should accompany final success rates (Bousmalis et al., 2024). Adaptation time, computational resources, and the number of updated parameters provide complementary cost measures. These evaluation quantities assess distinct aspects of adaptation. They are not in one-to-one correspondence with the three optimization terms in Eq. (6.13).

Physical Validation and Deployment Interface Target trials should include the failure modes that motivated adaptation, such as grasp loss after an end-effector change, tracking error after a control-interface change, or unstable contact under altered compliance. Alongside task success, relevant measurements include executable-command rate, trajectory deviation, contact stability, recovery success, and intervention frequency. Proposed model outputs and applied controller commands should be recorded separately, so that success obtained through controller corrections can be distinguished from intrinsic policy performance.

The deployment handoff comprises more than a neural checkpoint. It should specify the embodiment and controller configuration, observation and action contracts, normalization statistics, calibration and timing information, descriptor and adapter versions, validated operating conditions, and behavior when required inputs or model outputs are unavailable. Unsupported conditions should remain identifiable, and updates must preserve compatibility among these components. Multi-embodiment modeling thus connects reusable task representations with verified, embodiment-specific physical behavior.

6.5 Model Deployment and Efficiency Optimization

A trained policy becomes a useful embodied capability only when its decisions arrive in time to influence the physical process. Deployment therefore concerns more than fitting a checkpoint into accelerator memory. It connects sensor acquisition, preprocessing, model execution, action interpretation, and feedback control into an operating system with finite computation and variable delay. A policy with higher offline accuracy can perform worse on a robot if it acts on older observations, pauses between action chunks, or loses contact information during preprocessing. The objective is to improve task performance under a specified resource and timing budget.

6.5.1 Deployment Architectures and the Execution Pipeline

Where the Model Runs Onboard deployment keeps sensing and inference close to the robot and avoids dependence on a network for every policy update. Its limits are accelerator capacity, memory bandwidth, power, and heat dissipation. A nearby workstation or local server can support a larger model and several robots, but introduces transmission and scheduling delays. Cloud deployment provides elastic computation and centralized model management, while adding a wider-area communication path whose delay and availability may be difficult to bound. These are engineering trade-offs rather than a hierarchy in which a more remote or larger computer is necessarily better.

The computation can also be split. A remote process may interpret instructions or generate task context, while an onboard policy produces actions and a local controller regulates motion. The partition should follow decision time scales and communication dependencies. A task goal can remain useful for seconds, while contact feedback may lose value within a small fraction of that interval. Sending every force sample through a remote language model can consequently increase bandwidth and delay without improving the decision that requires the sample. Conversely, splitting a transformer across machines can create repeated inter-device dependencies and be slower than moving the whole policy to one machine.

Table 6.3 compares the deployment choices. The appropriate configuration depends on the task's tolerance for delayed decisions, the expected concurrency, and the behavior available when an inference request is late. For a manipulation system, task reasoning and learning can often be remote even when action validation, control, and timeout handling must remain local.

The Online Data Contract Every inference request needs a reference time, an instruction, an embodiment identifier, the observations it requires, and a clear record of what is missing or delayed.

Table 6.3 Deployment placement and its principal performance constraints

Placement	Useful property	Main constraint	Typical allocation
Robot onboard	Short communication path, local autonomy	Memory, power, cooling	Compact policy and local feedback
Local server	Larger shared accelerator capacity	Network and queueing variability	VLA inference with local execution
Cloud	Centralized resources and fleet-level services	Wide-area delay and outages	Task reasoning, training, and non-urgent analysis
Split deployment	Different compute budgets for different time scales	Context consistency and interface design	Slow reasoning with fast local action updates

The preprocessing pipeline must reproduce the training convention for image crops, color channels, camera ordering, state normalization, action scaling, and coordinate frames. A model trained on relative tool-frame displacement cannot be deployed by interpreting its output as an absolute base-frame pose. Likewise, reversing the sign of a gripper command or applying normalization statistics from another embodiment can invalidate an otherwise correct prediction.

To align modalities sampled at different times, retain each modality's acquisition time and validity mask and construct the input history according to the model's convention. Interpolation can work well for smoothly varying joint positions, but averaging across a contact onset may erase the very event the policy needs. Training augmentations such as random cropping should also be distinguished from the deterministic preprocessing used for evaluation. These details link deployment directly to the data representation of Section 6.4.

For a sequential execution path, a useful latency decomposition is

$$L_{\text{e2e}} = L_{\text{acq}} + L_{\text{pre}} + L_{\text{net}} + L_{\text{queue}} + L_{\text{model}} + L_{\text{decode}} + L_{\text{dispatch}}, \tag{6.17}$$

where the terms describe acquisition, preprocessing, communication, queueing, model execution, action conversion, and dispatch to the control interface. Overlapped implementations should instead measure the critical path without double counting concurrent work. The interval ends once a command is available to the controller. When measuring response to a disturbance, the physical actuator response adds further delay. Even with fast accelerator kernels, waiting for camera frames, host-device copies, and server queues can account for substantial delay.

The deployed controller should retain both the requested and the applied command. Saturation, rate limiting, inverse kinematics, or a lower-level safety filter can alter an action after model inference. Subsequent history and learned dynamics should reflect what actually took effect. Recording only the unmodified output makes later failures appear to be modeling errors even when their cause lies in the execution interface.

6.5.2 Model Compression and Inference Acceleration

Optimization begins with profiling the actual execution path. A large VLM backbone may be bottlenecked by weight movement and visual-token processing. An autoregressive policy may be slowed by repeated sequential decoding. A generative action expert may spend most of its time on repeated denoising evaluations. For a small policy, preprocessing, device transfers, and Python-level dispatch can become the dominant cost. The benefit of the same optimization therefore varies across architectures and hardware.

Compression of Parameters and Activations Quantization stores selected weights or activations at reduced numerical precision. It can lower memory use and data movement, but speed gains require supported kernels and favorable tensor shapes. OpenVLA studies quantized deployment as part of its efficiency analysis (Kim et al., 2025b). Calibration for a robot policy should include the states where precision matters, such as small displacements near contact, rare sensor values,

and transitions between action modes. Keeping sensitive output projections or normalization operations at higher precision may be preferable to applying the same precision everywhere.

Structured pruning removes channels, heads, layers, or entire processing branches. Unstructured sparsity does not necessarily accelerate dense hardware kernels, and removing a rarely used feature can damage a rare but important recovery behavior. Knowledge distillation trains a smaller student to reproduce useful teacher behavior or representations (Hinton et al., 2015). The student can be trained on actions, subgoals, intermediate features, or generated trajectories. For closed-loop control, training should cover perturbed states and the teacher's corrections, and recovery should be tested after the student's own errors.

A lightweight action expert provides another form of computation allocation. The expensive backbone constructs semantic context, while a smaller module performs repeated action-generation steps. This division is exemplified by π_0 and studied through knowledge insulation (Black et al., 2025a; Driess et al., 2025). Parameter-efficient methods like LoRA mainly cut the cost of adapting a model. They do not, however, make the original backbone any cheaper to run. An adapter merged into supported weights can avoid extra adapter operations, but the main model still runs.

Reducing Work Per Decision For autoregressive policies, action tokenization determines the number of sequential backbone evaluations. FAST compresses temporal action structure and thereby changes this trade-off (Pertsch et al., 2025). Parallel prediction can remove token-by-token dependencies, but must be supported by the training objective and attention structure. OpenVLA-OFT combines parallel decoding, action chunking, and continuous regression (Kim et al., 2025a). Its result should not be described as merely running the original autoregressive policy faster.

For diffusion and flow-based policies, fewer sampling steps reduce the number of action-expert evaluations. An excessively coarse solver can change action quality, especially at sharp transitions or for narrow contact tolerances. Few-step distillation, combined with suitable training objectives, can improve this trade-off. Reducing the number of inference steps requires a new closed-loop evaluation. The number of steps generated, how many are actually executed, and how often new observations arrive should all be tuned together.

Visual token reduction can lower the cost of processing several camera views. Cropping, token pooling, selective view use, and compact history encoders are possible choices, but the retained information must match the task. A small contact region can occupy few pixels while determining success. Compression that works for reaching may therefore fail during insertion or when distinguishing two similar tools.

Execution-Level Optimization and Cache Validity Operator fusion, compilation, fixed-shape execution, preallocated buffers, and reduced host-device synchronization can remove overhead without intentionally changing the policy function. IO-aware attention implementations such as FlashAttention reduce attention memory traffic (Dao et al., 2022). Their end-to-end benefit still depends on whether attention dominates the deployment workload. Warm-up and compilation cost should be separated from steady-state inference, while occasional recompilation and shape changes should remain visible in tail-latency measurements.

Caching is useful only when the dependency being cached is unchanged. Within a flow or diffusion solve, the observation-language prefix can be reused if it cannot attend to changing action tokens. Across different robot observations, cached visual features generally become stale. Even an unchanged instruction does not imply that every text hidden state can be reused if the backbone mixes text and images bidirectionally. A deployment should distinguish reusable tokenization or independent encodings from context-dependent transformer states. This distinction prevents apparent speed from silently reducing responsiveness.

Efficiency should be reported at a consistent boundary. If the policy generates H actions in L_{model} seconds, its nominal action-generation throughput is

$$R_{\text{gen}} = \frac{H}{L_{\text{model}}}. \tag{6.18}$$

This quantity describes generated commands. It does not count independent decisions made from fresh measurements. Comparisons must hold the action duration, prediction horizon, execution horizon, input views, sampling settings, and hardware conditions sufficiently explicit to interpret the number.

6.5.3 Asynchronous Inference and Real-Time Closed-Loop Control

Four Distinct Frequencies The sensor frequency determines how often measurements become available. The policy frequency determines how often a new observation-conditioned prediction is produced. The command frequency determines how often targets are delivered to the lower-level controller. The servo frequency determines how often that controller updates actuator inputs. These frequencies can differ by orders of magnitude. Interpolating a predicted trajectory or streaming a long action chunk raises the command frequency without raising the policy's feedback frequency.

Let Δt be the interval between action targets, H the predicted chunk length, and $N_{\text{exec}} \leq H$ the number of actions selected for execution before replacing the remaining plan. The prediction horizon is $H\Delta t$, whereas the nominal replanning interval is $N_{\text{exec}}\Delta t$. Action chunking captures temporal correlations and amortizes inference. Receding-horizon execution retains only a prefix and uses new evidence to reconsider the rest. Longer N_{exec} reduces inference demand but delays responses to unexpected changes. The appropriate value depends on the physical event, not only on available compute.

Observation age gives a more direct measure of responsiveness. If a command applied at physical time t_{apply} is based on a reference observation acquired at t_{obs}, define

$$\Delta_{\text{age}} = t_{\text{apply}} - t_{\text{obs}}. \tag{6.19}$$

For multimodal policies, ages may also be reported separately by modality. A small model runtime does not guarantee small observation age when old frames are queued or a long action prefix is committed. Dropping obsolete requests can improve feedback freshness even when it reduces the total number of processed frames.

Overlapping Prediction and Execution In synchronous execution, a robot may finish a chunk and wait for the next inference. Naively removing this pause creates a different problem. The next chunk was generated from a state that no longer coincides with the state at which it will begin. The remaining trajectory must account for commands executed during inference. Real-time chunking addresses this issue for flow and diffusion policies by conditioning the new chunk on a committed prefix while regenerating the uncommitted future (Black et al., 2025b).

If a request requires L_{req} seconds from input availability to usable output, the approximate number of command intervals that elapse during the request is

$$d_{\text{delay}} = \left\lceil \frac{L_{\text{req}}}{\Delta t} \right\rceil. \tag{6.20}$$

The overlap procedure should preserve the commands that are already committed during those d_{delay} intervals and use current timing information to select the replacement suffix. For example, at a command rate of 50 Hz, a 100 ms request spans five action intervals. A 50-action chunk has a one-second prediction horizon, but neither that horizon nor the command rate implies 50 fresh policy decisions per second. This is an illustrative timing calculation, not a benchmark result.

A simple buffer feasibility condition is

$$N_{\text{buf}}\Delta t \geq L_{\text{req}}^{(p_{\text{quant}})} + L_{\text{margin}}, \tag{6.21}$$

where N_{buf} is the number of validated buffered commands, $L_{\text{req}}^{(p_{\text{quant}})}$ is a selected latency percentile, and L_{margin} accounts for dispatch uncertainty. This condition cuts down on buffer underruns under the measured latency distribution. It does not guarantee anything in the worst case. A deeper

buffer also commits the robot further into an old plan. Responsiveness requires replacing the uncommitted portion when new evidence arrives and defining task-appropriate behavior when the buffer is exhausted.

Chunk transitions require semantic as well as numerical consistency. Averaging two trajectories may smooth a position jump while mixing incompatible modes, such as opening and closing a gripper or maintaining and releasing contact. Continuity should be checked for the commanded quantity and its physical role, including pose, velocity, force targets, and discrete events. A late result associated with an earlier instruction or aborted task should be rejected rather than appended to the current action stream.

Slow Context and Fast Correction A multi-rate system can update task context or predictive features less frequently than local action decisions through the following decomposition

$$c_k^{\text{ctx}} = F_\theta^{\text{ctx}}(h_{t_k}, \ell_{\text{lang}}, e), \qquad \mathbf{A}_t^{\text{seq}} = G_\psi(c_{k(t)}^{\text{ctx}}, o_t^{\text{recent}}, \mathbf{q}_t, e), \tag{6.22}$$

where $k(t)$ selects the latest valid context. The fast module uses recent evidence, including contact history when available, while the slow module revises task interpretation or predicted futures. For this separation to work, the action module must be trained to operate in context of varying age and must receive enough current information to correct behavior. Merely caching an old full-policy output does not create such a capability.

Replanning can combine periodic updates with event triggers. Target motion, grasp loss, unexpected force, a completed subtask, or a change in instruction may invalidate the current context. Thresholds and hysteresis can prevent excessive switching around noisy events. The event path itself needs a delay bound. Also, a task-level replan should not interrupt a local stabilization response that is physically necessary. The control principles for this fast feedback layer are developed further in Chapter 7.

6.5.4 Deployment Evaluation and Operational Maintenance

Deployment evaluation should compare complete operating configurations. Alongside model weights, record numerical precision, input resolution and views, action horizon, execution horizon, sampling steps, device, concurrent workload, and communication conditions. Otherwise, a higher throughput may reflect a larger chunk or a less responsive execution schedule rather than an improvement in the same task interface.

The evaluation should connect computation to behavior. Table 6.4 links deployment measurements to their interpretations and test conditions. Relevant computational quantities include end-to-end latency, latency percentiles and variability, observation age, missed deadlines, peak memory, and power. Relevant physical quantities include task success, completion time, trajectory and contact accuracy, recovery after disturbances, and human intervention. Energy per completed task is often more informative than accelerator power alone. A faster inference configuration may consume more instantaneous power but finish the task sooner, while a slower configuration may increase robot holding time.

A staged evaluation can first compare numerical outputs on recorded observations. Next, test timing under replay or hardware-in-the-loop execution, and finally measure closed-loop behavior. Recorded-data agreement is a diagnostic rather than a substitute for robot trials. Dynamic-target tasks expose delayed feedback, contact tasks expose precision loss, and long-horizon tasks expose accumulation of small errors. Comparisons should use matched task conditions and report trial counts and uncertainty. A speedup measured on one accelerator does not establish a hardware-independent property.

Operational logs should connect each input snapshot to its model version, preprocessing version, inference timestamps, proposed action, applied command, and observed outcome. This makes it possible to distinguish recognition errors from stale observations, action conversion errors, or controller overrides. Model weights, tokenizer or action decoder, normalization statistics, embod-

Table 6.4 Deployment measurements and the questions they answer

Measurement	Interpretation	Useful test condition
Median and tail latency	Typical response and occasional late decisions	Concurrent requests, warm-up, sustained load
Observation age, deadline misses	Freshness of feedback used in execution	Moving targets, delayed frames, network jitter
Memory, power, task energy	Resource feasibility and operating cost	Long runs at the intended thermal limit
Success and completion time	Whether speed changes useful performance	Matched tasks and initial-state distributions
Contact error and recovery	Precision and response beyond nominal motion	Insertion, slippage, unexpected contact
Interventions and fault recovery	Dependence on operator support	Timeouts, sensor loss, server interruption

iment adapters, and runtime configuration form one deployment version. Rolling back weights while retaining incompatible normalization or action decoding is not a complete rollback.

Timeout behavior must be appropriate to the task. Holding position may be reasonable in free space but undesirable during a forceful contact. Stopping a pouring motion can require a controlled change in orientation. Such responses should be implemented and validated at the execution layer, with a clear path to human takeover where needed. After compression, compiler changes, or hardware migration, a fixed regression suite should test both nominal skills and the disturbances that previously caused failures.

Deployment is where this chapter's modeling loop closes. Training creates behavioral priors. Adaptation makes them an embodiment-specific interpretation. The execution system then decides when those priors can actually affect the world. Efficiency is established when reduced computation or communication produces timely, accurate behavior under the intended operating conditions. The resulting execution traces then supply the evidence needed for the next round of capability improvement.

6.6 Dexterous Manipulation Foundation Models

Most of the models discussed in this chapter act through a parallel gripper, whose action is a wrist pose and a single opening width. A multi-fingered hand breaks three assumptions behind that design. The action space grows to 20 or more finger joints, and whether an object stays under control depends on how those joints move together. Many decisive events, such as a finger beginning to slide or load shifting onto the palm, happen at contacts the cameras cannot see. Robot data are scarcer still. Teleoperating 20 joints is much harder than teleoperating one, and every hand defines its own action space. Human hands, meanwhile, manipulate objects constantly and on camera, which makes human data the obvious place to look for scale.

The subsections examine how to represent finger actions (Section 6.6.1) and how to learn from human hands, with EgoScale as the worked example of the stages in Section 6.3 (Section 6.6.2). They then examine how much of a policy can be shared across hands, extending Section 6.4 (Section 6.6.3), where touch helps (Section 6.6.4), and what current results establish (Section 6.6.5). The hands themselves are described in Section 9.6, and the low-level hand controllers that sit beneath these models in Sections 7.4.4 and 7.4.5.

6.6.1 Action Representations for Multi-Fingered Hands

A dexterous policy has to fix the meaning of its action vector before data collection begins. Current systems use one of three options. Joint targets for a specific hand are directly executable but bind the model to that mechanism. Fingertip or keypoint targets are closer to what human data measure, but

need a solver to turn them into joint commands. Functional coordinates give each entry a meaning shared across hands, such as index flexion or thumb opposition, and keep a separate map from those entries to each hand's joints. Most systems also command the wrist separately, as a relative change in hand pose. The choice decides which data can supervise the model and how much of it can be shared between hands.

Human data enter through retargeting, the geometric alignment of Section 6.4.1, which for hands can be written as

$$\min_{\mathbf{q},\mathbf{T}} \sum_i w_i \|\mathbf{p}_i^{\text{robot}}(\mathbf{q},\mathbf{T}) - \widehat{\mathbf{p}}_i^{\text{human}}\|^2 + \lambda\|\mathbf{q}-\mathbf{q}_{\text{prev}}\|^2, \quad \mathbf{q}_{\min} \le \mathbf{q} \le \mathbf{q}_{\max}, \tag{6.23}$$

where $\mathbf{q}$ is the robot hand configuration, $\mathbf{T}$ aligns the wrist frames, $\mathbf{p}_i$ is a fingertip position, and w_i weights its importance. The second term penalizes abrupt changes. The objective is illustrative, not the loss of a particular study, and practical versions add collision and contact constraints. As Figure 9.15 shows, a MANO pose vector and a robot joint vector do not correspond one to one. Retargeted human motion is useful supervision only insofar as the mapped action keeps the relevant interaction. More data widen coverage but do not remove a systematic geometric mismatch.

6.6.2 Learning from Human Hands: Pretraining, Alignment, and Post-Training

EgoScale is the clearest evaluated example of the training stages of Section 6.3 applied to a dexterous hand. It is evaluated on a Galaxea R1Pro with two 22-DoF Sharpa Wave hands and three RGB cameras (Zheng et al., 2026b). The hands are described in Section 9.6.3. Figure 6.7 lays out the stages.

Pretraining: Learning Reusable Wrist-Finger Coordination EgoScale represents human-video motion as relative wrist displacement plus retargeted hand actions (NVIDIA, 2026). Predicting articulated actions pushes the model to encode how the wrist and fingers move together. Carrying a bottle without losing the grasp is a different coordination pattern from twisting its cap. Patterns like these recur across tasks and can be learned before any target-robot data are collected. The representation also commits early to an execution interface. Relative wrist motion transfers easily, but the finger component has to be realizable in the target hand's joint space, so the retargeting of Eq. (6.23) is already present during pretraining.

Alignment: Connecting Human Observations to Robot Actions EgoScale's aligned mid-training uses about 50 hours of human and 4 hours of robot play data. The human capture matches the robot's camera viewpoints and intrinsics, with Vive wrist tracking and MANUS hand capture synchronized to video (Zheng et al., 2026b). The question at this stage is which differences in appearance and motion come from the embodiment and which signal a different task state. A human finger and a robot link may occupy different parts of the image while doing the same job of supporting a cup, and similar silhouettes may hide different reachable contact surfaces. In the terms of Section 6.4.1, matched viewpoints and retargeting reduce observation and geometric mismatch, while robot play exposes dynamic mismatch under load. The capture hardware and its calibration are described in Section 9.6.4.

Post-Training: Specializing the Prior to a Physical Task Task-specific robot data then tie the prior to the actual arm, hand, objects, and cameras. For a handle grasp, the prior may already organize approach, closure, and transport, but only the target setup determines how far the fingers must close to engage the handle and which wrist path leaves room for the next step. In EgoScale's scaling experiment, direct post-training without aligned mid-training raises the average task-completion score from 0.30 to 0.71 as human pretraining grows from 1,000 to 20,000 hours (Zheng et al., 2026b). This experiment varies the amount of human data, unlike the comparison with and without pretraining summarized in Section 2.3. A separate one-shot protocol uses 1 robot demonstration plus 100 aligned human demonstrations per object, after the earlier stages (NVIDIA, 2026). The two experiments answer different questions concerning whether more prior experience helps, and how

little robot data is then needed to select a behavior the model already supports. Section 4.5 discusses what such scaling variables can and cannot establish.

Two other studies on Sharpa hands use human data in narrower roles. CAIP aligns images with hand-action proxies to pretrain a visual encoder and evaluates policies without tactile input (Sharma et al., 2026b). Do as I Do reconstructs and retargets human hand-object motion into robot trajectories on a dual-UR3e Sharpa system (Paliwal et al., 2026). Each improves one ingredient, the representation or the execution data. EgoScale shows how the ingredients fit into one training sequence.

6.6.3 Sharing Actions Across Hands

UniDex asks which finger-level action roles can be shared when joint structures differ. Its dataset spans eight hand types and is built by retargeting human demonstrations, with interactive correction and visual alignment. Its Function-Actuator-Aligned Space (FAAS) assigns joints with the same function to common coordinates, the functional-coordinate option of Section 6.6.1, and keeps a map to each hand. A policy conditioned on point clouds, language, and proprioception predicts action chunks (UniDex Team, 2026).

A spray bottle shows what such a mapping has to preserve. The thumb must reach the trigger while the other fingers resist its reaction force. Moving the wrist for better trigger access can weaken the

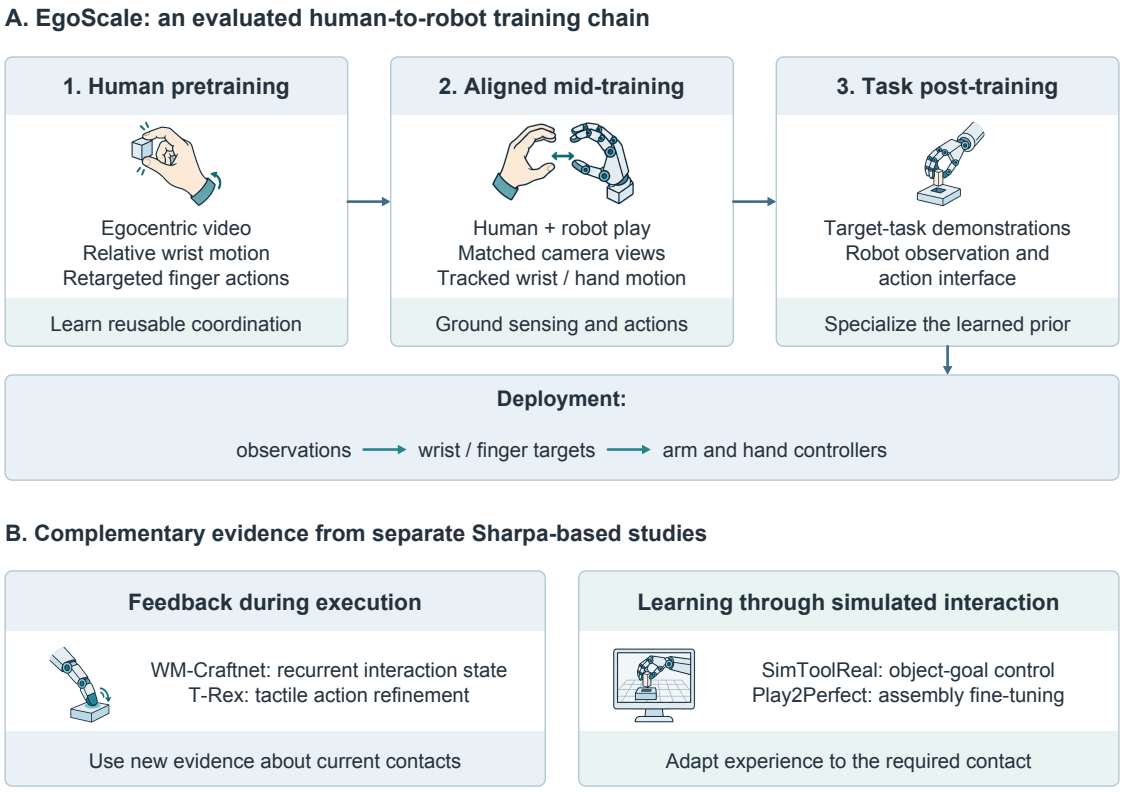


Fig. 6.7 The learning chain on Sharpa Wave. The upper path follows EgoScale's training stages, with human and robot observations connected through wrist motion and hand-action representations (Zheng et al., 2026b; NVIDIA, 2026). The lower panels place separate studies of execution feedback and simulation-based adaptation, discussed in Sections 7.4.4 and 7.4.5. The grouping is conceptual. The studies share no weights or training with EgoScale

Table 6.5 UniDex real-robot results across 5 tasks using Wuji and Inspire hands, with 20 trials per task and method. Task progress averages stage outcomes. Final success measures task completion. Aggregate means reproduced from Fig. 11 of Zhang et al. (2026a). Values are percentages

Method	Task progress	Final success
π_0	38.0	35.0
UniDex-VLA without pretraining	32.5	23.0
UniDex-VLA	81.0	76.0

support or misaim the nozzle. And once the trigger moves, the fingers must let it return without dropping the bottle. Minimizing each joint error separately preserves none of these relations.

The Wuji experiments in UniDex use the 20-DoF hand on a Franka arm with RGB-D sensing, on tasks that include a spray bottle, scissors, and a computer mouse, with 50 robot demonstrations per task for adaptation (Zhang et al., 2026a). The robot demonstrations correct systematic differences in grasp placement, response, and observation that human data cannot reveal. Table 6.5 reports both outcome measures. Pretraining improves intermediate progress and final success, but because the aggregate pools Wuji and Inspire tasks, it does not isolate the contribution of the Wuji hardware.

Without further training, an Inspire-trained policy transferred to Wuji reaches 40% average task progress over 10 trials (Zhang et al., 2026a). A shared flexion coordinate lets the parameters be shared, but the resulting fingertip motion still depends on link lengths, couplings, and joint ranges, so task structure transfers while contact execution is only partly preserved. SimToolReal (Section 7.4.4) takes the opposite approach. It places the shared interface at the object and leaves finger coordination to the controller.

6.6.4 Where Tactile Input Helps a Dexterous Policy

Section 6.2.3 describes three ways contact information can enter a VLA. For dexterous hands the narrower question is at which decision a tactile signal changes the outcome. Two XHAND studies give opposite answers.

DexUMI collects demonstrations with an exoskeleton fitted to the target hand and replaces the human hand in the images with the robot hand (Section 9.6.4). Its policy predicts wrist and finger actions from visual and tactile input (Xu et al., 2025c). In an XHAND kitchen task of turning a knob, moving a pan, and adding salt, touch matters in the salt step because the fingers are hidden from view, and closing the hand to a plausible posture does not show that they reached the salt.[6] With relative actions and visual adaptation fixed, accumulated success through the salt stage is 75% with tactile input and 15% without, over 20 episodes per task (Xu et al., 2025c).

In the shared-autonomy system of Cui et al. (2025), DexGrasp-VLA controls the 12-DoF hand from vision and touch during data collection, and the resulting trajectories, supplemented by corrective demonstrations, train an arm-hand VLA with separate arm and hand feature branches. The full system reaches 88.7% pick-and-place success across 50 objects with 3 trials each. In an ablation, adding tactile features directly to the arm-hand policy lowers success from 95% to 82%, although the same features help the hand-only controller (Cui et al., 2025).

The difference lies in the decision the signal feeds. Touch helps where the next action depends on a contact that vision cannot resolve, and hurts where it becomes one more input to a policy that primarily selects where to reach and place. A third option keeps touch out of the foundation model and uses it in a faster refinement loop beneath it, as T-Rex does (Section 7.4.5).

[6] DexUMI Team, *DexUMI: Using Human Hand as the Universal Manipulation Interface for Dexterous Manipulation*, project page and demonstrations, 2025.

6.6.5 Evaluating Dexterous Foundation Models

The studies above report different endpoints (task-completion score, stage-wise progress, final success, and accumulated success through a sequence) on different hands, arms, cameras, objects, and trial counts, and some pool results over several hands. They cannot be ranked against each other. Each supports a narrower claim. EgoScale shows that a larger human prior improves post-trained performance under a fixed protocol. UniDex, that a functional action space allows pretraining across hands and some zero-shot transfer. DexUMI and DexGrasp-VLA, that tactile input pays off only at decisions it can change.

Three questions remain open. Does human pretraining keep scaling once tasks require sustained force, not just reaching the right configuration? Can a shared action space carry contact strategies across hands, or only task structure? And how should a model with slow visual updates hand authority to faster contact-driven control, the interface studied in Chapter 7? Answering them requires the closed-loop protocols of Chapter 11, applied to whole tasks from grasping through recovery on declared hardware.

7 Foundation Models for Whole-Body Motor Control

Humanoid and other whole-body robots are an important class of physical embodiments. With improvements in hardware, more recent work focuses on controlling the entire body as a coordinated system. This chapter discusses how to extend current foundation-model algorithms to whole-body motor control. The whole-body control connects task-level decisions to coordinated physical action: it uses control interfaces to regulate motion and contact within the robot's dynamic limits, while adjusting in real time to external forces, terrain changes, and contact transitions. Each of them is discussed in detail as follows.

7.1 Scope, Taxonomy, and Control Interfaces

Task-level objectives specify what a robot should accomplish, but generally leave open how its joints and contacts should be coordinated under the body's dynamics and actuation limits. Motor control addresses these execution requirements through feedback from the body and its environment, as Figure 7.1 illustrates. This section defines the role of whole-body motor control in this process, traces the control principles underlying it, and identifies the interfaces needed to reuse motor capabilities across tasks. In humanoid tasks that combine locomotion and manipulation, body support must be coordinated with arm motion and hand-object contacts.

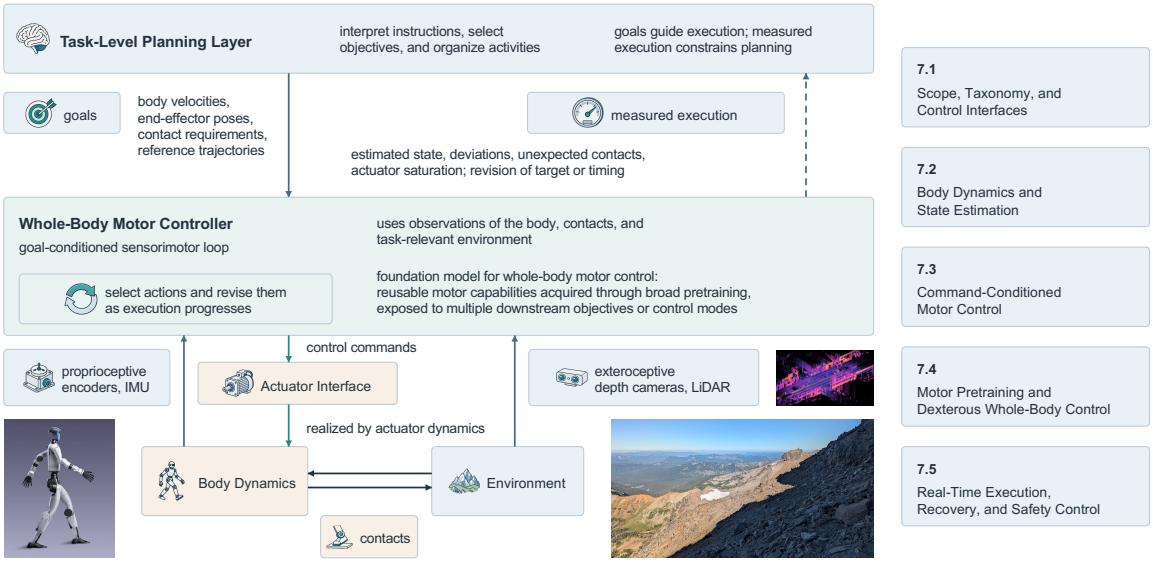


Fig. 7.1 The whole-body motor controller in the robot stack. A task-level planning layer issues goals to a goal-conditioned sensorimotor closed loop, which reads proprioceptive and exteroceptive measurements and issues control commands through the actuator interface. Contacts couple the body to the environment, and measured execution returns to constrain planning. Section markers indicate where Sections 7.1 to 7.5 develop this loop

7.1.1 Functional Scope and Foundation Models

In this chapter, a *whole-body motor controller* is a goal-conditioned sensorimotor model that coordinates robot motion and physical interaction through closed-loop control. It uses available observations of the body, contacts, and task-relevant environment to select and revise actions at high frequency as forces and contact conditions change. Building on this feedback capability, a *foundation model for whole-body motor control* acquires reusable motor capabilities through broad pretraining and exposes them to multiple downstream objectives or control modes (Zeng et al., 2025; 2026).

This role complements task-level planning. Task-level models interpret instructions, select objectives, and organize activities. Motion or behavior generators express those objectives as body velocities, end-effector poses, contact requirements, or reference trajectories. The motor controller executes these targets using the available state information. A specified hand position, for instance, may admit different arm configurations, trunk postures, and support arrangements. Selecting a realization requires accounting for its effects on the rest of the body (Khatib, 1987).

A motor objective remains subject to execution feedback after it is issued. Small disturbances may be corrected locally. Persistent tracking errors, unexpected contacts, or sustained actuator saturation from torque demands beyond hardware limits may require the upstream planner to revise the target or its timing. Information must therefore flow in both directions. Goals guide execution, while measured execution constrains subsequent planning. A planner must account for estimates of the state reached during execution, including deviations from the preceding reference.

Realizing this feedback loop also requires observation processing, state estimation, target interpretation, actuator interfaces, and mechanisms for enforcing execution limits. Some components may be learned jointly. Others remain explicit. A pretrained tracking policy can provide the core motor capability while relying on external estimation and low-level servos. Such architectures combine explicit control components with learned motor capabilities, drawing on complementary developments in model-based control and reinforcement learning.

7.1.2 From Control Principles to Reusable Motor Capabilities

For articulated robots, the main difficulty is that control objectives are coupled through mechanics. Operational-space control expresses motion and force objectives in task coordinates and relates them to joint actuation, making end-effector behavior an explicit object of control (Khatib, 1987). Impedance control further regulates the dynamic relationship between motion and interaction force, rather than requiring exact position tracking in all circumstances (Hogan, 1985). Classical work on robot hands similarly analyzes grasping through fingertip geometry, force transmission, and frictional interaction (Mason and Salisbury Jr, 1985). Together, these methods establish that an appropriate motor interface can specify a physical interaction rather than a complete sequence of joint configurations. In legged robots, the same mechanical coupling must be coordinated across repeated contact transitions. Raibert et al. (1984) decompose hopping into regulation of forward velocity, body attitude, and hopping height.

To account for future motion and support requirements, MPC makes the consequences of candidate actions explicit over a finite horizon. At each update, it optimizes a control sequence from the current state estimate, applies the initial input, and repeats the optimization after receiving new feedback. State and input constraints can be included directly, although feasibility and stability depend on the model, formulation, and assumptions (Mayne et al., 2000). In 2003, zero-moment-point preview control provides a related predictive formulation for biped walking, using future support references to shape the current motion through optimal linear feedback (Kajita et al., 2003). By 2018, the MIT Cheetah 3 controller uses convex MPC to optimize ground-reaction forces over a finite horizon, while a gait state machine and swing controller supply contact timing and swing-leg motion (Di Carlo et al., 2018).

In the late 2010s, deep reinforcement learning (RL) expands the use of simulated interaction to acquire feedback policies. DeepMimic combines reference-motion imitation with task rewards to learn coordinated skills for physics-based characters (Peng et al., 2018a). Hwangbo et al. (2019) train policies for ANYmal using a simulator that combines rigid-body dynamics with an actuator model learned from robot data, then deploy those policies on the physical robot. Lee et al. (2020) extend locomotion to challenging terrain by training a privileged teacher and distilling its behavior into a student that uses proprioceptive history. History-conditioned adaptation further uses recent observations and actions to infer information about unobserved dynamics, including changes in load or ground interaction. Rapid Motor Adaptation (RMA) makes this inference a dedicated learned module that conditions a locomotion policy (Kumar et al., 2021). The estimation methods supporting such adjustment are developed in Section 7.2.

A parallel line develops motor knowledge that downstream tasks can reuse. Neural probabilistic motor primitives (NPMP) compress expert skills into a shared latent representation (Merel et al., 2019). An adversarial motion prior (AMP) learns a distributional motion prior that shapes policy optimization (Peng et al., 2021). Adversarial Skill Embeddings (ASE) learns a skill-conditioned policy whose latent space can serve as the action space of a downstream controller (Peng et al., 2022). A distributional prior shapes policy learning, while a skill-conditioned policy generates executable actions. Their shared contribution is to separate the acquisition of coordinated movement from the specification of a particular downstream task.

Modern systems develop both hybrid execution architectures and more reusable command interfaces. Deep Tracking Control combines an online trajectory-optimizing planner with a learned tracking policy that executes its references under model mismatch and disturbances (Jenelten et al., 2024). This combination retains explicit predictive structure while learning the feedback needed during execution. More recent humanoid controllers extend reuse to the control interface. HOVER distills motion-tracking capability into a policy supporting multiple command modes (He et al., 2025b). A behavior foundation model (BFM) uses motion imitation and masked online distillation to pretrain a shared behavioral repertoire, while ScaleBFM studies direct goal-conditioned RL pretraining with varied motion data and model capacity (Zeng et al., 2025; 2026). These humanoid approaches allow downstream applications to invoke learned movement through different target specifications, as developed in Section 7.4.

These approaches reflect two complementary design choices concerning the *execution mechanism* and *how motor knowledge is acquired and reused*. Table 7.1 shows representative combinations.

Table 7.1 Selected control approaches compared by execution mechanism and the acquisition and reuse of motor knowledge

Approach	Execution mechanism	Acquisition and reuse	Representative examples
Analytical feedback	Task-space motion and force feedback	Explicit models and designed interaction objectives	Operational-space and impedance control (Khatib, 1987; Hogan, 1985)
Predictive optimization	Online finite-horizon optimization with feedback	Dynamics and constraints used to recompute motion or force plans	Cheetah 3 convex MPC (Di Carlo et al., 2018)
RL feedback policies	Observation- and command- or reference-conditioned actions	Task rewards or motion imitation History-based inference can support adaptation	DeepMimic, ANYmal, RMA (Peng et al., 2018a; Hwangbo et al., 2019; Kumar et al., 2021)
Motor priors and skills	Feedback policy, optionally conditioned on a skill latent	Demonstration distributions or distilled experts reused in downstream learning	NPMP, AMP, ASE (Merel et al., 2019; Peng et al., 2021; 2022)
Multi-mode motor pretraining	Shared goal- or command-conditioned policy	Distillation or broad RL pretraining exposes a reusable repertoire	HOVER, BFM, ScaleBFM (He et al., 2025b; Zeng et al., 2025; 2026)
Hybrid planning and tracking	Online optimization with a learned feedback tracker	Planned references executed by a policy trained for tracking	Deep Tracking Control (Jenelten et al., 2024)

7.1.3 Architecture and Interfaces

A motor-control architecture connects state representation, goal interpretation, and action generation, as Figure 7.2 illustrates. In a humanoid, moving the arms changes whole-body dynamics, while stepping changes the support configuration and the position from which the hands act. A

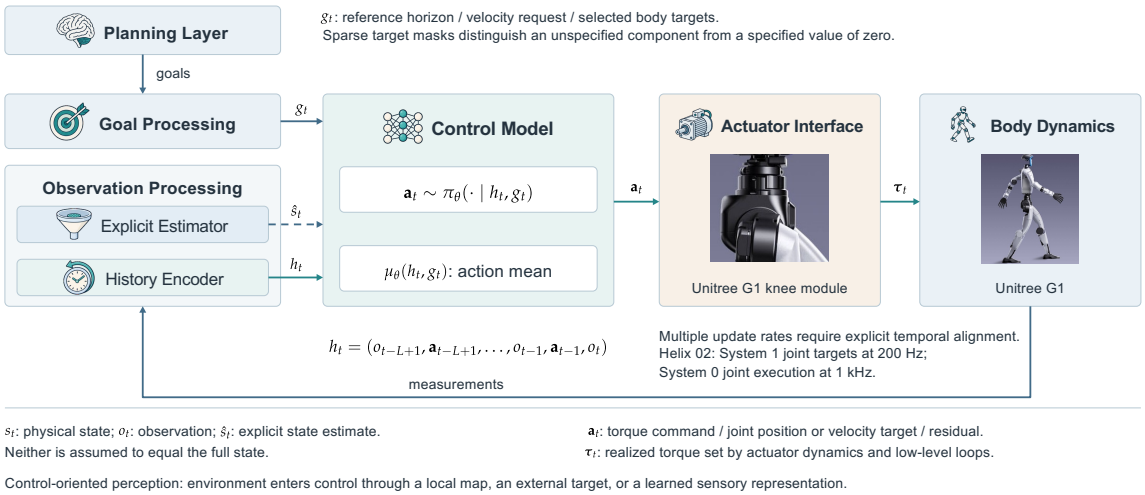


Fig. 7.2 Closed-loop architecture and interfaces of a whole-body motor controller. Goal processing supplies g_t, observation processing supplies h_t and $\hat{s}_t$, and the control model outputs $\mathbf{a}_t$, which the actuator interface realizes as joint torques $\boldsymbol{\tau}_t$ through actuator dynamics and low-level loops. Measurements close the loop. Operation at multiple update rates requires explicit temporal alignment

requested behavior therefore has to be realized through coordinated body motion and corrected as the resulting motion and contacts evolve.

Figure's Helix 02 provides a concrete example through its publicly described System 2-System 1-System 0 hierarchy.[7] System 2 interprets scenes and language, sequences behaviors, and supplies latent goals to System 1. Conditioned on these latents, System 1 combines head-camera and palm-camera images, fingertip tactile sensing, and full-body proprioception to generate full-body joint targets at 200 Hz. System 0 is a learned whole-body controller that tracks these targets, using full-body joint state and base motion to produce joint-level actuator commands at 1 kHz. This division connects semantic goals to visuomotor targets and then to feedback execution that handles balance, contact, and whole-body coordination. The additional sensing also serves specific feedback needs. Palm cameras provide in-hand visual feedback when an object is occluded from the head camera, and fingertip tactile sensors detect contact forces.

Throughout this chapter, s_t denotes the physical state at control step t, including the robot and any environment variables needed to describe its evolution. The observation o_t contains information available during execution. An explicit state estimate is denoted by $\hat{s}_t$. Neither is assumed to equal the full state. A finite observation-action history is

$$h_t = (o_{t-L+1}, \mathbf{a}_{t-L+1}, \ldots, o_{t-1}, \mathbf{a}_{t-1}, o_t), \qquad \mathbf{a}_t \sim \pi_\theta(\cdot \mid h_t, g_t), \tag{7.1}$$

where $L \geq 1$ is the history length, θ denotes policy parameters, and g_t is the command representation. The policy $\pi_\theta(\cdot \mid h_t, g_t)$ is a conditional action distribution, and $\sim$ denotes sampling from it. For $L = 1$, $h_t = o_t$. A recurrent state or learned history encoding can summarize longer histories.

Privileged observations provide training-time access to variables that can be read directly in simulation but are difficult to obtain reliably from the robot's sensors. HUGWBC uses base linear velocity, ground friction, and foot contact forces as such variables (Xue et al., 2025). In its asymmetric actor-critic training, the critic combines these quantities with proprioception and commands to predict returns. Its value estimates then enter the advantage estimates used to update the actor with PPO (Schulman et al., 2017; HugWBC Authors, 2025). This allows the simulator's richer information to contribute to policy optimization through the critic. The actor's execution inputs

[7] Figure, *Introducing Helix 02: Full-Body Autonomy*, January 27, 2026.

include recent proprioceptive history, commands, and estimates of selected hidden quantities obtained from that history (Xue et al., 2025; HugWBC Authors, 2025).

Command granularity determines how much of the movement the controller must choose. A desired body velocity leaves freedom in foothold placement and posture, as in the locomotion commands of Section 7.3. A full reference motion constrains the coordinated body trajectory more closely, while targets for only selected body links leave the remaining movement free to accommodate support and interaction constraints. Desired reference quantities are denoted by s_t^{ref}, with the hat reserved for estimates. The policy receives their command representation g_t, which can encode a velocity request, a reference horizon, or selected body targets. For partial targets, a mask identifies the specified components and distinguishes an unspecified target from a specified value of zero, as in the masked-target representation of Section 7.4.

Executing these goals requires translating the policy's action $\mathbf{a}_t$ into physical joint actuation. When $\mathbf{a}_t$ contains joint-position targets, a low-level servo converts position error and joint feedback into torque commands. Actuator dynamics then determine the actual joint torque $\boldsymbol{\tau}_t$, including the effects of saturation and delay (Hwangbo et al., 2019). Policies may instead output joint-velocity targets, direct torque commands, or a residual added to an existing controller, changing what the downstream control interface must realize. For a stochastic training policy, $\boldsymbol{\mu}_\theta(h_t, g_t)$ denotes its action mean, which can also be used for deterministic execution.

Consider a hand target fixed in the world while the humanoid turns or takes a step. The target stays fixed, but its pose relative to the body changes, so a body-frame encoding must be updated using the current body pose. An object-relative target instead follows the object's motion. Timing matters for the same physical reason. A reference computed for one support configuration may need revision after touchdown or lift-off. When perception, goal generation, and actuation update at different rates, a controller can receive observations or references describing an earlier body or contact state. Feedback correction between target updates must therefore relate these inputs to the current motion. A measurement timestamp indicates the time of the observed state, and a reference's validity interval indicates when it is intended to apply. Delayed information can increase tracking error, motivating the latency analysis in Section 7.5.

For a humanoid traversing uneven ground through a low passage, successful motion requires both placing the feet and keeping the torso clear of overhead obstacles. Proprioceptive history reflects body motion and the response to contact, supporting correction of ongoing execution. Exteroceptive observations supply the geometry ahead of the robot so that motion can be adapted before contact. Visual servoing uses the discrepancy between current and desired visual features to correct robot motion (Chaumette and Hutchinson, 2006). For perceptive locomotion, Miki et al. combine proprioception with terrain-height observations from a robot-centric elevation map, allowing the controller to adapt its motion before contact while retaining feedback from body motion (Miki et al., 2022). A ground-height map, however, does not preserve all geometry relevant to a humanoid's whole body. Gallant instead uses LiDAR-derived voxel grids together with proprioception to represent lateral obstacles and overhead constraints as well as ground-level terrain (Ben et al., 2026). These examples illustrate *control-oriented perception*, in which the system retains the information needed to select a physical action and correct it as the body interacts with its surroundings.

7.1.4 Generality, Dexterity, and Evaluation

Generality concerns reuse under changes in objectives and physical conditions. Reproducing many training motions establishes behavioral coverage. A more demanding test is whether the controller can combine familiar capabilities to meet new task requirements. Goal flexibility allows an existing capability to be invoked through different target representations, while task transfer asks whether it remains useful in a new activity. A controller may track new references yet need substantial retraining for new interaction dynamics. Reuse therefore involves both executing new goal combinations and adapting coordinated behavior to changes in load, terrain, or contact.

These demands become tightly coupled when locomotion and manipulation share the same body. Maintaining an object pose during locomotion imposes constraints that are absent from either un-

constrained walking or stationary manipulation (Fu et al., 2023). For a humanoid carrying an object with both hands, body acceleration changes the forces required to support the object, while arm configuration affects both reach and whole-body balance. Dexterous execution additionally requires finger configurations that maintain the grasp. An end-effector pose specifies a hand's position and orientation but leaves its internal contact configuration unresolved. In bimanual manipulation, the two hands interact through the shared object. Their contact forces jointly determine object motion, but different force distributions can produce the same resultant wrench while imposing different grasping loads. Coordinated execution must therefore account for support, object motion, and contact maintenance together, especially during load transfer or changes in the grasp (Bicchi, 1994). The learned hand-control and force-adaptive balance examples in Sections 7.4.4 and 7.4.5 provide concrete examples of contact execution and load-dependent body support.

Extending this reuse to another embodiment introduces changes in the body itself. Robots differ in topology, geometry, actuation, and sensing, so a shared controller must represent these differences or accommodate them through embodiment-specific components. Morphology-conditioned transformers provide one approach to sharing control representations across bodies (Gupta et al., 2022). Task-space and contact-based goals can reduce dependence on a particular joint layout, but the mapping from those goals to actions remains constrained by each robot's capabilities. Transfer to a physical platform additionally depends on its actuator and sensor characteristics, beyond the morphological variation represented in simulation.

The carrying example makes these forms of generality directly testable. Changing walking speed and object-pose targets tests the composition of movement requirements. Varying the carried load tests adaptation. Using a robot with a different morphology tests embodiment transfer. Tracking error, object slip, and loss of balance assess target execution, grasp maintenance, and body support, respectively. For goals or conditions that require further learning, the benefit of pretraining can be measured by the data and optimization needed to reach a given task performance, compared with learning from scratch (Merel et al., 2019; Zeng et al., 2025).

7.2 Body Dynamics and State Estimation

The motor-execution layer is a high-frequency control loop around a physical system. Figure 7.3 depicts floating-base dynamics, contact constraints, and the actuation interface. It receives task commands from the planning layer, reads proprioceptive measurements (joint encoders and an inertial measurement unit, IMU) together with exteroceptive ones such as depth cameras and LiDAR, and issues control commands realized through the actuator interface. Three questions organize this loop, asking how the body moves, where it is now and what it touches, and what the model is getting wrong. This section develops the three answers in turn. Throughout, s_t denotes the actual state, $\hat{s}_t$ its estimate, and s_t^{ref} a commanded reference.

7.2.1 Motion-Control Foundations: Floating-Base Dynamics, Contacts, and Actuators

A legged robot is a *floating-base* system. Six base degrees of freedom carry no actuators, and only the joints are driven. Throughout this section, $\mathcal{W}$ denotes the inertial world frame and $\mathcal{B}$ the base frame, and a left superscript marks the frame in which a quantity is expressed. The world vertical is opposite the gravity direction, and ${}^{\mathcal{W}}\mathbf{R}_{\mathcal{B}}$ maps base-frame vectors to world coordinates. With n actuated joints, the configuration combines the base pose with the joint angles, and the generalized velocity stacks the base spatial velocity (linear and angular), expressed in $\mathcal{B}$, with the joint rates, giving

$$\begin{aligned} \mathbf{q} &= \left({}^{\mathcal{W}}\mathbf{p}_{\mathcal{B}}, {}^{\mathcal{W}}\mathbf{R}_{\mathcal{B}}, \mathbf{q}_j\right) \in \mathrm{SE}(3) \times \mathbb{R}^n, \\ \boldsymbol{\nu}_{\mathcal{B}} &= \begin{bmatrix} {}^{\mathcal{B}}\mathbf{v}_{\mathcal{B}} \\ {}^{\mathcal{B}}\boldsymbol{\omega}_{\mathcal{B}} \end{bmatrix} \in \mathbb{R}^6, \qquad \mathbf{v} = \begin{bmatrix} \boldsymbol{\nu}_{\mathcal{B}} \\ \dot{\mathbf{q}}_j \end{bmatrix} \in \mathbb{R}^{6+n}. \end{aligned} \tag{7.2}$$

The base twist $\boldsymbol{\nu}_{\mathcal{B}}$ contains the linear velocity of the base origin and its angular velocity, both expressed in $\mathcal{B}$. Using this velocity convention consistently is essential. Configuration derivatives

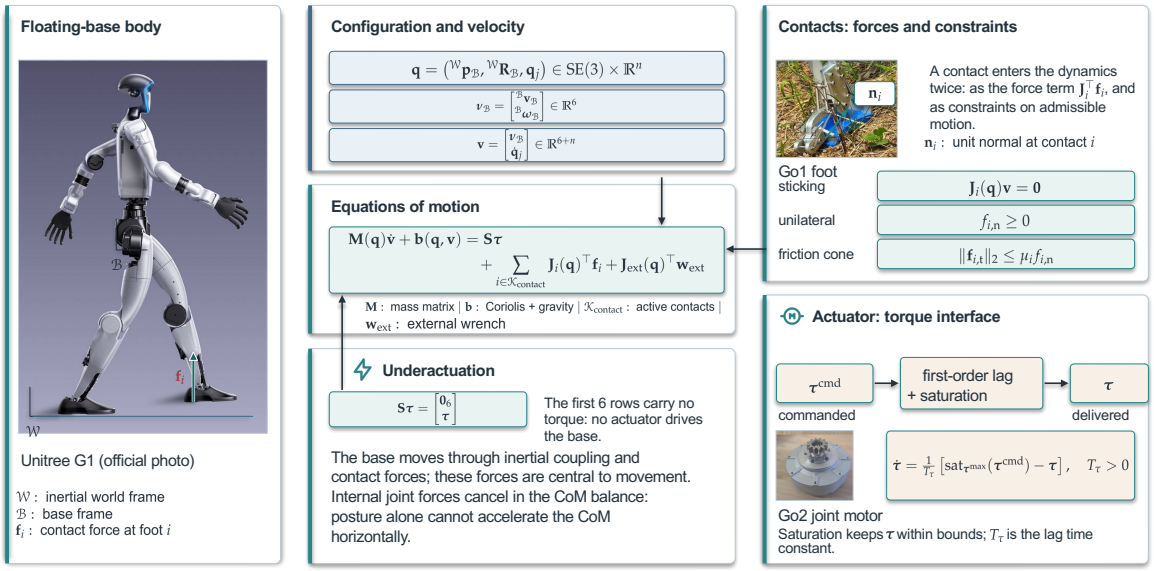


Fig. 7.3 Floating-base dynamics, contacts, and actuation of a legged robot. Joint torques act through the selection matrix on the joint rows only, so contact forces are central to movement. Contacts contribute forces subject to sticking, unilateral, and friction-cone constraints, and the delivered torque follows the commanded torque through a first-order lag with saturation

are not simply $\dot{\mathbf{q}} = \mathbf{v}$. In particular, ${}^{\mathcal{W}}\dot{\mathbf{p}}_{\mathcal{B}} = {}^{\mathcal{W}}\mathbf{R}_{\mathcal{B}}\,{}^{\mathcal{B}}\mathbf{v}_{\mathcal{B}}$, and rotation evolves through the base angular velocity. The equations of motion are

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{b}(\mathbf{q}, \mathbf{v}) = \mathbf{S}\boldsymbol{\tau} + \sum_{i \in \mathcal{K}_{\text{contact}}} \mathbf{J}_i(\mathbf{q})^\top \mathbf{f}_i + \mathbf{J}_{\text{ext}}(\mathbf{q})^\top \mathbf{w}_{\text{ext}}, \tag{7.3}$$

where $\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{(6+n)\times(6+n)}$ is the symmetric positive definite mass matrix. The bias term $\mathbf{b} = \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} + \mathbf{g}(\mathbf{q})$ collects Coriolis, centripetal, and gravity forces. The vector $\boldsymbol{\tau} \in \mathbb{R}^n$ contains the joint torques. The selection matrix

$$\mathbf{S} = \begin{bmatrix} \mathbf{0}_{6\times n} \\ \mathbf{I}_n \end{bmatrix} \in \mathbb{R}^{(6+n)\times n} \tag{7.4}$$

places torques on the joint rows only. $\mathcal{K}_{\text{contact}}$ is the set of active point contacts, with world-frame forces $\mathbf{f}_i \in \mathbb{R}^3$. $\mathbf{J}_i \in \mathbb{R}^{3\times(6+n)}$ maps the generalized velocity to the world-frame velocity of contact point i. A contact patch can be represented by multiple such points. The six-dimensional external wrench $\mathbf{w}_{\text{ext}}$ and its spatial Jacobian use matching world-aligned coordinates and the same moment reference point. Additional external interactions contribute analogous terms. Written blockwise, the dynamics are

$$\begin{bmatrix} \mathbf{M}_{\mathcal{B}\mathcal{B}} & \mathbf{M}_{\mathcal{B}j} \\ \mathbf{M}_{j\mathcal{B}} & \mathbf{M}_{jj} \end{bmatrix} \begin{bmatrix} \dot{\boldsymbol{\nu}}_{\mathcal{B}} \\ \ddot{\mathbf{q}}_j \end{bmatrix} + \begin{bmatrix} \mathbf{b}_{\mathcal{B}} \\ \mathbf{b}_j \end{bmatrix} = \begin{bmatrix} \mathbf{0}_6 \\ \boldsymbol{\tau} \end{bmatrix} + \mathbf{J}_{\mathcal{K}_{\text{contact}}}^\top \mathbf{f}_{\mathcal{K}_{\text{contact}}} + \mathbf{J}_{\text{ext}}^\top \mathbf{w}_{\text{ext}}, \tag{7.5}$$

where $\mathbf{J}_{\mathcal{K}_{\text{contact}}}$ stacks the Jacobians of the active contacts and $\mathbf{f}_{\mathcal{K}_{\text{contact}}}$ their forces. The equations expose the central structural fact of legged robotics. The first six rows contain no joint torque, so no actuator acts directly on the base. Joint torque moves the base through inertial coupling (an airborne robot can pitch its trunk by swinging its legs) and through contact with the environment. Internal joint forces cancel in the center-of-mass (CoM) balance, so changing limb or trunk posture cannot create horizontal CoM acceleration when the net external horizontal force is zero. Foot friction, hand contacts, and other external interactions supply such forces. This underactuation makes contact forces central to movement (Wensing et al., 2024).

Contacts enter the equations both as forces and as constraints on admissible motion. A foot in sticking contact with stationary ground satisfies $\mathbf{J}_i(\mathbf{q})\mathbf{v} = \mathbf{0}$. Each contact is unilateral. The ground pushes but does not pull, and friction bounds the tangential force. In world coordinates, let $\mathbf{n}_i$ be the unit surface normal and μ_i the friction coefficient, and split $\mathbf{f}_i$ into a normal part $f_{i,\mathrm{n}} = \mathbf{f}_i \cdot \mathbf{n}_i$ and a tangential part $\mathbf{f}_{i,\mathrm{t}} = \mathbf{f}_i - f_{i,\mathrm{n}}\mathbf{n}_i$. Unilateral contact and Coulomb friction require

$$f_{i,\mathrm{n}} \geq 0, \qquad \|\mathbf{f}_{i,\mathrm{t}}\|_2 \leq \mu_i f_{i,\mathrm{n}}. \tag{7.6}$$

The pinned foot turns the stance leg into a reconfigurable strut that couples the base to the ground, and the same coupling is what lets the base state be inferred from joint encoders and an IMU, as the next subsection shows. Contact forces must also be compatible with the available joint torques. Contacts are ultimately hybrid. A contact is open, sticking, or sliding, and impact is discontinuous. Time-stepping complementarity formulations capture this without pre-specifying a mode sequence (Stewart and Trinkle, 1996; Posa et al., 2014), while deployed controllers often fix a contact sequence from a gait or footstep plan and optimize the force distribution within each mode (Di Carlo et al., 2018).

Two further properties of the dynamics recur throughout control and identification. The Coriolis matrix can be chosen with an energy property that underlies passivity-based control arguments, and the dynamics are linear in the inertial parameters, with ten parameters per rigid link. This linearity forms the basis of classical identification (Khalil and Dombre, 2004; Featherstone, 2008). Standard recursive algorithms evaluate inverse dynamics and unconstrained forward dynamics in time linear in the number of joints. Forming a dense mass matrix, resolving contacts, and solving control optimizations have additional costs (Featherstone, 2008). Any gap between the simulated body and the real one is a model error the deployed controller must absorb.

So far the joint torque is a free input. On hardware it is not. The actuator sits between the commanded torque and the torque that reaches the joint. Three common designs illustrate different trade-offs. Hydraulic actuation provides high force density, as on HyQ (Semini et al., 2011). Series elastic actuation introduces compliance that protects the drivetrain and makes torque measurable from spring deflection at the cost of bandwidth (Pratt and Williamson, 1995). And quasi-direct drive uses high-torque motors behind a low-ratio gearbox, as on the MIT Cheetah and Mini Cheetah (Seok et al., 2015; Katz et al., 2019). A simplified torque-interface model uses a first-order lag with saturation,

$$\dot{\boldsymbol{\tau}} = \frac{1}{T_\tau}\left[\mathrm{sat}_{\boldsymbol{\tau}^{\max}}\left(\boldsymbol{\tau}^{\mathrm{cmd}}\right) - \boldsymbol{\tau}\right], \qquad T_\tau > 0, \tag{7.7}$$

where saturation clips each commanded torque to its admissible interval, and the time constant T_τ is measured in seconds. Initialized within those intervals, this model preserves the torque bounds. Position-target actions additionally require a model of the intervening servo. Some actuator physics is difficult to capture with a hand-written model. Hwangbo et al. train an actuator network on physical-robot data, mapping a history of joint position errors and velocities to delivered torque, and policies trained with this model transfer to hardware (Hwangbo et al., 2019).

7.2.2 Explicit State Estimation: Attitude, Velocity, Contact, and Local Terrain

In an explicit state-estimation architecture, the controller acts on physical quantities reconstructed from sensor measurements. Figure 7.4 contrasts explicit estimation with implicit estimation from observation history. On a legged robot the fused body state includes the base pose and velocity and the joint states, while the estimation pipeline additionally determines the contact set and, on perceptive platforms, maps the local terrain. Joint encoders observe joint configuration, with velocity obtained through sensor processing. An IMU provides angular velocity and specific force. Neither directly measures base position or translational velocity. The estimator must account for sensor orientation, lever arms, timestamps, and coordinate transforms.

A torso-mounted IMU, taken to sit at the base origin, provides two high-rate signals. The gyroscope measures angular velocity up to a slowly drifting bias. Integrating it gives short-term attitude that accumulates drift. The accelerometer measures *specific force*, physical acceleration minus gravity expressed in the sensor frame. At rest it reads not zero but a vector opposite gravity. Aligning the

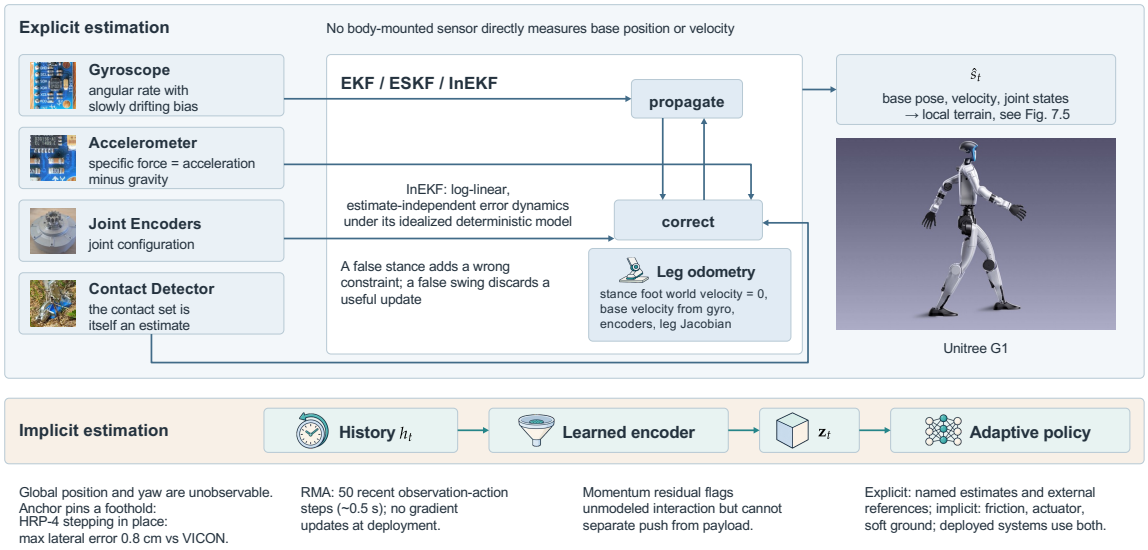


Fig. 7.4 Explicit and implicit state estimation on a legged robot. Explicit estimation fuses inertial and kinematic measurements, joint encoders, and an estimated contact set into a body state estimate through a propagate-correct loop. Implicit estimation maps recent observation-action history to a task-relevant latent. Deployed systems combine the two routes

predicted gravity direction with the measured one corrects roll and pitch when other accelerations are modeled or sufficiently small, while yaw requires an external reference such as vision or a magnetometer. Complementary fusion trusts gyro integration at high frequency and the gravity reference at low frequency, implemented on the HRP-4 humanoid as an extended Kalman filter (EKF) (Caron, n.d). The EKF loop propagates the state and its covariance with the model and then corrects both with each measurement. On the rotation manifold, the error-state Kalman filter (ESKF) propagates a nominal orientation and estimates a small rotation-vector error (Solà, 2017). The contact-aided invariant extended Kalman filter (InEKF) has log-linear, estimate-independent error dynamics under its idealized deterministic model and improves convergence and consistency in the reported Cassie comparisons. IMU-bias augmentation weakens some theoretical guarantees, and covariance accuracy still depends on the model and noise assumptions (Hartley et al., 2020).

The contact set is itself an estimate. Simple detectors threshold contact-force estimates obtained from joint-torque measurements using the leg Jacobian and a dynamics model. Classification is particularly sensitive during touchdown and lift-off transients. Probabilistic estimators maintain a per-foot contact probability from proprioceptive signals, with explicit impact detection, on HyQ (Camurri et al., 2017), and filter-based estimators fold contact uncertainty into the measurement update (Bloesch et al., 2012). A false stance introduces an incorrect constraint, while a false swing discards a useful update. With the contact set decided, a foot in firm stance on stationary ground provides a velocity constraint. Its world-frame velocity is zero, so the base velocity follows from the gyroscope, the encoders, and the leg Jacobian. This inference is the legged analog of wheel odometry. Bloesch et al. carry the foothold positions and IMU biases in the filter state and fuse these measurements with IMU propagation in one EKF (Bloesch et al., 2012). Leg odometry is only as good as its premise. Slipping feet and misclassified swings bias the estimate. During flight phases, the estimator propagates inertially without contact corrections, and uncertainty grows according to sensor quality and the assumed dynamics.

Even perfect fusion leaves global translation and rotation about gravity unobservable when only inertial and relative kinematic measurements are available (Bloesch et al., 2012; Hartley et al., 2020). Anchor schemes reduce accumulated drift by assuming that a selected stance-foot point occupies its planned world location and combining this with orientation and encoder-based forward kinemat-

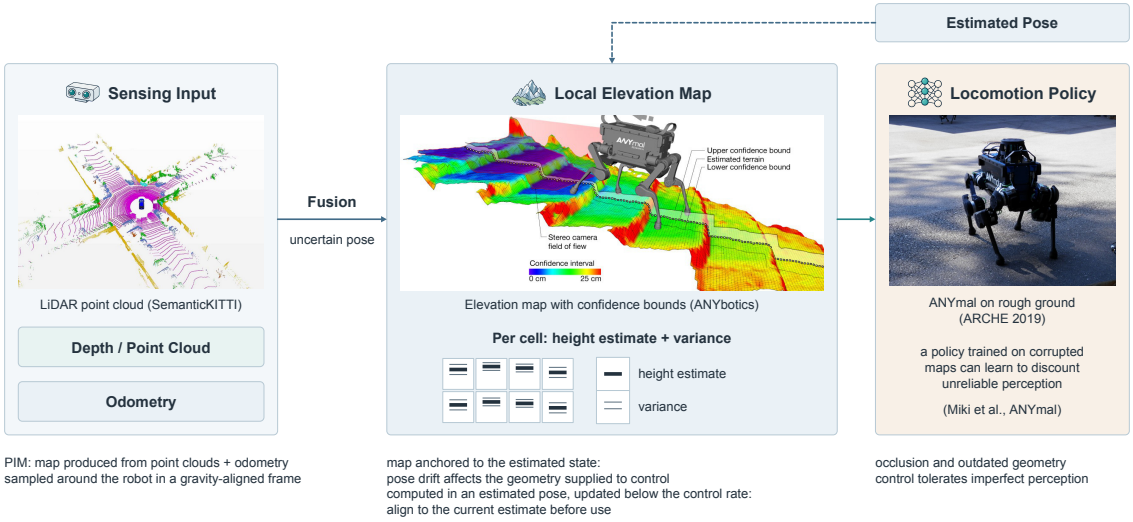


Fig. 7.5 Local terrain estimation for perceptive locomotion. Depth measurements are fused, under an uncertain estimated pose, into a local elevation map in which each cell retains a height estimate and a variance. The map is anchored to the estimated state and updated below the control rate, so it must be aligned to the current estimate before use

ics. In Caron's reported HRP-4 *stepping-in-place example*, the maximum lateral base-position error relative to VICON is 0.8 cm. The anchor assumption can fail with sliding, misplaced footholds, or foot flexibility (Caron, n.d).

In addition to the body state, locomotion over unstructured ground needs a local terrain model, commonly an elevation map that fuses depth measurements under an uncertain robot pose and retains a height estimate and variance for each cell, as illustrated in Figure 7.5 (Fankhauser et al., 2018). Because the map is anchored to the estimated state, pose drift affects the geometry supplied to control. The perceptive internal model (PIM), for example, uses an elevation map produced from point clouds and odometry and sampled around the robot in a gravity-aligned frame (Long et al., 2025a). Miki et al. show on ANYmal that a policy trained over corrupted maps can learn to discount unreliable perception (Miki et al., 2022). Maps are computed in an estimated pose and updated at lower rates than the control loop, so they must be aligned to the current estimate before use. Occlusion and outdated geometry further motivate control that tolerates imperfect perception.

7.2.3 Implicit Estimation from Observation History

Explicit estimators rely on nominal dynamics, measurement models, and uncertainty assumptions. Real deployments introduce effects outside these models. Scree is slipperier than the simulated friction coefficient assumes, a carried payload changes the inertial parameters, actuators wear, and encoders are miscalibrated. Classical identification estimates such parameters offline from designed excitation, while a deployed robot can also infer its current response from recent observation-action history h_t. In *implicit estimation*, a learned encoder recovers a compact, task-relevant representation $\mathbf{z}_t$ without naming every hidden physical parameter. The history must be temporally aligned and sufficiently informative. Actuator saturation, for example, can make the requested action differ from the input actually experienced by the body.

Training-time randomization varies the physics (masses, friction, and actuation) so that deployment conditions appear as further variations (Tobin et al., 2017; Peng et al., 2018b). Randomization alone does not require a dedicated estimate of the current dynamics, although a history-conditioned policy may learn adaptation. A complementary line trains with a targeted adversary

in which a learnable disturber selects destabilizing pushes, with an H-infinity-style constraint relating control cost to disturbance intensity (Long et al., 2025b). Privileged teacher-student distillation explicitly trains a student to reproduce a privileged teacher's behavior from proprioceptive history (Lee et al., 2020). The history thereby supplies information about hidden conditions that the teacher observes directly.

RMA makes the estimator a dedicated module (Kumar et al., 2021). A base policy acts on the current observation and a latent vector, and an adaptation module produces the latent from 50 recent observation-action steps, corresponding to 0.5 s in the reported implementation. During training the latent is supervised to match a privileged encoder. At deployment the module updates it with no gradient computation. The latent encodes how behavior should change without requiring accurate recovery of friction or payload parameters. The hybrid internal model (HIM) combines an explicitly estimated velocity with an implicit response representation trained against successor observations (Long et al., 2024). Its humanoid extension, the PIM, adds terrain sampled from the robot-centered elevation map described above (Long et al., 2025a). Learned estimation can therefore expose physical state, latent response information, or both. The pattern spans platforms. A history-conditioned humanoid transformer combines sequence-model pretraining with reinforcement learning (Radosavovic et al., 2024). Deep whole-body control unifies manipulation and locomotion on a quadruped with an arm through regularized online adaptation (Fu et al., 2023). And Deep Tracking Control (DTC) pairs a trajectory-optimizing planner with a learned tracking controller (Jenelten et al., 2024).

Generalized momentum, the product of the mass matrix and velocity, provides a dynamics residual through a momentum observer (De Luca and Mattone, 2003). This residual can indicate unmodeled interaction, but also contains torque or model mismatch, so it cannot uniquely separate a push from a payload change. It complements learned response representations with a physics-derived diagnostic.

Several limits shape the division of labor. Distinct physical conditions can produce indistinguishable histories, and conditions outside the training distribution can invalidate a learned latent. A conventional latent vector carries no calibrated confidence measure. An EKF covariance also remains conditional on its model and noise assumptions. History can recover local motion and dynamic response, but inertial and relative kinematic inputs cannot supply a missing absolute position or yaw reference. Implicit representations cover model-resistant effects such as friction variation, actuator response, and soft-ground compliance, while explicit tools provide named physical estimates and external references. Deployed systems combine the two (Miki et al., 2022; Jenelten et al., 2024). A learned policy can consume the latent directly, while a model-based controller uses physical estimates, parameter corrections, or a momentum residual.

Table 7.2 summarizes sensor inputs, estimated quantities, controller uses, and characteristic failure modes. The control interface specifies the components supplying $\hat{s}_t$, h_t, and $\mathbf{z}_t$, together with their frames and timestamps. Persistent estimation or tracking errors may require a slower command, a different contact plan, or upstream replanning, connecting local estimation to the task-level feedback in Section 7.1.

7.3 Command-Conditioned Motor Control

A motion controller converts an abstract request such as "walk forward at a given speed" into a time sequence of coordinated joint actions. For a humanoid, this conversion is difficult because the request is multi-dimensional, the body is underactuated during foot contact, and the same joint motion simultaneously affects translation, turning, balance, foot clearance, and upper-body posture. A useful controller must therefore expose commands at a semantic level while learning (or solving) the coordination needed at the joint level. A velocity command leaves joint configurations and foot placement largely unspecified, while posture and phase commands specify body configuration and support timing. Reference-conditioned tracking, developed in Section 7.4, supplies more detailed time-varying motion targets.

Table 7.2 Representative sensor-estimator-controller interfaces and their characteristic failure modes

Sensor / input	Estimator	Output	Controller / use	Failure / limit
IMU, encoders, contact cues	Contact-aided EKF / InEKF	Base pose and velocity, footholds, biases, covariance	Legged feedback or trajectory tracking	Slip and contact errors, global position and yaw drift
Orientation, encoders, planned foothold	Anchor point and forward kinematics	Base translation and CoM	HRP-4 walking stabilization	Incorrect foothold assumption or foot flexibility
Point clouds, odometry, gravity alignment	Local elevation mapping	Terrain heights and available uncertainty	Footstep planning or perceptive policy	Pose drift, occlusion, outdated geometry
Proprioceptive and action history	RMA adaptation module	Task-relevant latent $\mathbf{z}_t$	Adaptive joint-target policy	Ambiguous history, conditions outside training
History, terrain observations in PIM	HIM / PIM hybrid estimator	Velocity estimate and response latent	Command-conditioned locomotion	Unseen dynamics, erroneous terrain input

7.3.1 Command Interfaces and Controller Families

Framework and Modules A model-based controller separates prediction over a future horizon from instantaneous enforcement of whole-body dynamics. A reference generator converts velocity, posture, and gait commands into trajectories and a contact schedule. Nonlinear model predictive control (NMPC) plans the motion and support wrenches. And a faster whole-body controller (WBC) realizes them through joint commands. State estimation combines IMU and encoder measurements with contact kinematics, while a robot-model module supplies the dynamics and Jacobians. Figure 7.6 adapts this organization to a humanoid robot.

NMPC: Anticipating Bipedal Support Use the configuration $\mathbf{q}$ and generalized velocity $\mathbf{v}$ of Section 7.2.1, with n actuated joints. A centroidal formulation (Sleiman et al., 2021) uses the state $\mathbf{x} = (\mathbf{h}_{\mathrm{G}}, \mathbf{q})$ and input $\mathbf{u} = (\mathbf{w}_{\mathrm{L}}, \mathbf{w}_{\mathrm{R}}, \dot{\mathbf{q}}_j)$, where $\mathbf{h}_{\mathrm{G}} = (\mathbf{p}_{\mathrm{G}}, \mathbf{k}_{\mathrm{G}})$ contains linear and angular momentum about the center of mass, and $\mathbf{w}_i = (\mathbf{f}_i, \boldsymbol{\mu}_i)$ is a six-dimensional foot wrench. Here $\mathbf{p}_{\mathrm{G}}$ is linear momentum in $\mathrm{kg\,m/s}$, $\mathbf{k}_{\mathrm{G}}$ is angular momentum in $\mathrm{kg\,m^2/s}$, and m_{robot} in the centroidal equations is robot mass in kg. Here momentum is unnormalized and each foot-wrench moment is taken about its foot-frame origin. With all terms expressed in world coordinates,

$$\dot{\mathbf{p}}_{\mathrm{G}} = m_{\mathrm{robot}}\mathbf{g}_{\mathrm{acc}} + \sum_{i\in\{\mathrm{L,R}\}} \mathbf{f}_i, \qquad \dot{\mathbf{k}}_{\mathrm{G}} = \sum_{i\in\{\mathrm{L,R}\}} \left[(\mathbf{r}_i - \mathbf{c}_{\mathrm{G}}) \times \mathbf{f}_i + \boldsymbol{\mu}_i\right], \tag{7.8}$$

where $\mathbf{c}_{\mathrm{G}}$ is the CoM position and $\mathbf{r}_i$ the foot origin. External loads are omitted here. The centroidal momentum matrix $\mathbf{A}_{\mathrm{G}}(\mathbf{q})$ relates momentum to velocity by $\mathbf{h}_{\mathrm{G}} = \mathbf{A}_{\mathrm{G}}\mathbf{v}$. Together with joint rates and configuration kinematics, this determines the state flow $\dot{\mathbf{x}} = F_{\sigma_{\mathrm{contact}}}(\mathbf{x}, \mathbf{u})$ for a prescribed contact schedule $\sigma_{\mathrm{contact}}$.

At each update, NMPC minimizes

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \quad & \Phi_{\mathrm{term}}(\mathbf{x}(t+T)) + \int_t^{t+T} \ell(\mathbf{x}(\tau), \mathbf{u}(\tau), \tau)\, \mathrm{d}\tau, \\ \ell = \; & \|\mathbf{v}_{\mathrm{G,xy}} - \mathbf{v}_{\mathrm{xy}}^{\mathrm{cmd}}\|^2_{\mathbf{Q}_v} + \|\omega_{\mathrm{B,z}} - \omega_{\mathrm{z}}^{\mathrm{cmd}}\|^2_{Q_\omega} + \|\mathbf{e}_{\mathrm{torso}}\|^2_{\mathbf{Q}_{\mathrm{R}}} \\ & + \|c_{\mathrm{G,z}} - c_{\mathrm{G,z}}^{\mathrm{ref}}\|^2_{Q_c} + \|\mathbf{q}_{\mathrm{upper}} - \mathbf{q}_{\mathrm{upper}}^{\mathrm{ref}}\|^2_{\mathbf{Q}_a} + \|\mathbf{u} - \mathbf{u}^{\mathrm{ref}}\|^2_{\mathbf{R}_{\mathrm{u}}}, \end{aligned} \tag{7.9}$$

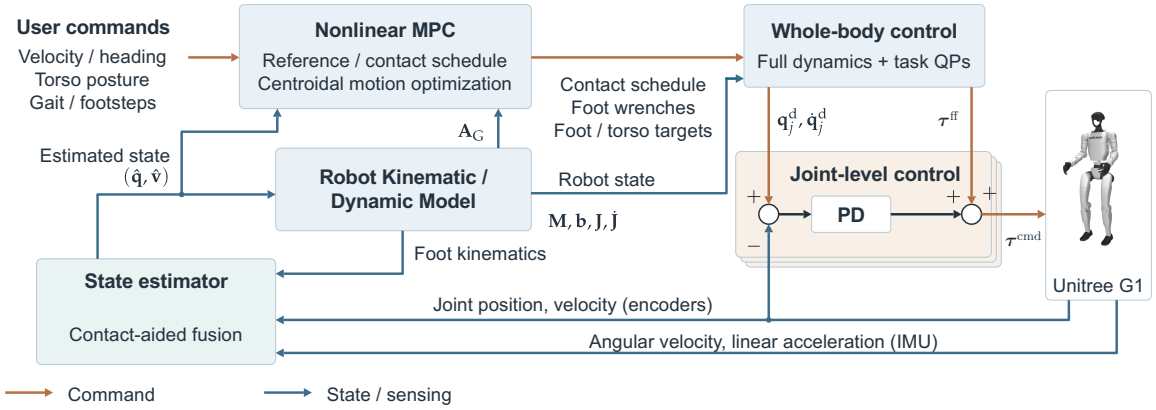


Fig. 7.6 Model-based humanoid motion control, illustrated with Unitree G1. Command paths connect reference generation, NMPC, WBC, and joint control. Feedback paths supply estimated state, robot-model quantities, and local joint measurements

subject to the estimated initial state, centroidal dynamics, joint limits, and mode-dependent contact constraints. Here $\|\mathbf{e}\|_{\mathbf{Q}}^2 = \mathbf{e}^\top \mathbf{Q}\mathbf{e}$, Φ_{term} is a terminal tracking cost, $\mathbf{v}_G = \mathbf{p}_G / m_{\text{robot}}$, and $\mathbf{e}_{\text{torso}}$ is a torso-orientation error. The nonnegative scalars Q_ω and Q_c weight the angular-velocity and CoM-height errors, respectively. Their weighted squares are $\|e\|_{Q_\omega}^2 = Q_\omega e^2$ and $\|e\|_{Q_c}^2 = Q_c e^2$ for a scalar error e. Velocity commands are transformed into the same frames as the predicted velocities before comparison. The upper-body target specifies waist and arm posture. Desired support wrenches and joint rates define $\mathbf{u}^{\text{ref}}$. The humanoid objective therefore couples commanded travel to upright posture, CoM height, and upper-body motion.

A stance sole remains stationary. A swing foot carries zero wrench and follows a clearance trajectory. Each supporting foot must satisfy unilateral contact, friction, center-of-pressure bounds inside its sole, and admissible yaw moment. These surface-wrench constraints are essential when transferring load between one and two supporting feet. A prescribed schedule fixes contact order and timing. Optimizing those discrete choices requires an additional planner. Multiple shooting and sequential quadratic programming provide one implementation route. Only the current portion of the optimized trajectory is passed to WBC before replanning.

WBC: Realizing the Plan on the Full Body Between contact switches, WBC solves at the current estimated configuration for $\mathbf{z}_{\text{WBC}} = (\dot{\mathbf{v}}, \mathbf{w}_L, \mathbf{w}_R, \boldsymbol{\tau})$. A representative task objective is

$$\min_{\mathbf{z}_{\text{WBC}}} \quad \sum_{k \in \mathcal{T}_{\text{WBC}}} \|\mathbf{J}_k \dot{\mathbf{v}} + \dot{\mathbf{J}}_k \mathbf{v} - \mathbf{a}_k^{\text{des}}\|_{\mathbf{W}_k}^2 + \|\mathbf{w} - \mathbf{w}_{\text{NMPC}}^\star\|_{\mathbf{W}_f}^2 + \|\boldsymbol{\tau}\|_{\mathbf{W}_\tau}^2, \tag{7.10}$$

where $\mathbf{W}_k \in \mathbb{R}^{d_{\text{task},k} \times d_{\text{task},k}}$, $\mathbf{W}_f \in \mathbb{R}^{12\times 12}$, and $\mathbf{W}_\tau \in \mathbb{R}^{n\times n}$ are symmetric positive-semidefinite weight matrices, with $\|\mathbf{e}\|_{\mathbf{W}}^2 = \mathbf{e}^\top \mathbf{W}\mathbf{e}$. The positive integer $d_{\text{task},k}$ is the dimension of task k's acceleration residual. The vector $\mathbf{w} = (\mathbf{w}_L, \mathbf{w}_R) \in \mathbb{R}^{12}$ stacks the two six-dimensional foot wrenches, and $\boldsymbol{\tau} \in \mathbb{R}^n$ contains the torques of the n actuated joints. The optimization is subject to the hard constraints

$$\begin{aligned} \mathbf{M}\dot{\mathbf{v}} + \mathbf{b} &= \mathbf{S}\boldsymbol{\tau} + \sum_i \mathbf{J}_{w,i}^\top \mathbf{w}_i, \\ \mathbf{J}_{w,i}\dot{\mathbf{v}} + \dot{\mathbf{J}}_{w,i}\mathbf{v} &= \mathbf{0} \qquad (i \text{ in stance}), \\ \mathbf{w}_i \in \mathcal{W}_i^{\text{wrench}}(\sigma_{\text{contact}}), &\qquad \boldsymbol{\tau}_{\min} \le \boldsymbol{\tau} \le \boldsymbol{\tau}_{\max}. \end{aligned} \tag{7.11}$$

The six-row Jacobian $\mathbf{J}_{w,i}$ maps generalized velocity to foot twist, extending the point-contact model of Section 7.2.1. $\mathcal{W}_i^{\text{wrench}}$ is the admissible wrench set, reduced to $\{\mathbf{0}\}$ in swing. The torque bounds

in Eq. (7.11) apply componentwise. Tasks $\mathcal{T}_{\text{WBC}}$ track CoM motion, torso orientation, swing-foot pose, and waist/arm posture, with feedback embedded in $\mathbf{a}_k^{\text{des}}$. Linearized friction bounds give a quadratic program. Weighted tasks express trade-offs. Strict priorities instead require hierarchical quadratic programs (QPs) that preserve higher-priority optima (Bellicoso et al., 2016). Joint position and velocity limits can be enforced through bounds on predicted motion over the control step.

The resulting torque is combined with local joint feedback,

$$\boldsymbol{\tau}^{\text{cmd}} = \boldsymbol{\tau}^{\text{ff}} + \mathbf{K}_{\text{p}}(\mathbf{q}_j^{\text{d}} - \mathbf{q}_j) + \mathbf{K}_{\text{d}}(\dot{\mathbf{q}}_j^{\text{d}} - \dot{\mathbf{q}}_j). \tag{7.12}$$

Desired joint motion is obtained from the whole-body tracking solution, with the feedforward and feedback terms sharing the actuator torque budget. Predictive planning anticipates support changes. WBC enforces instantaneous feasibility. Their performance remains dependent on state estimates, contact assumptions, and timely optimization. A learned controller moves much of this coordination into training, while retaining a comparable command interface.

7.3.2 A Unified Command Space

The HUGWBC controller provides a compact example (Xue et al., 2025). Its policy receives task commands, behavior commands, proprioceptive history, and an intervention indicator, and emits the whole-body action at 50 Hz. Its design brings command following, disturbance rejection, and motion quality into a common training framework. In the released implementation, the task command contains desired base-frame longitudinal and lateral linear velocity and the base-frame angular-velocity z component, which need not equal the Euler yaw-angle derivative,

$$\mathbf{v}_t^{\text{cmd}} = [v_{\text{x},t}^{\text{cmd}}, v_{\text{y},t}^{\text{cmd}}, \omega_{z,t}^{\text{cmd}}]^\top. \tag{7.13}$$

This representation allows the policy to select footholds and body motion while being judged by the achieved velocity. The command is augmented with behavior variables including gait frequency f_t^{gait}, maximum foot clearance ℓ_t^{clear}, body-height command z_t^{cmd}, body pitch β_t^{pitch}, waist yaw χ_t, phase offset ψ_t^{phase}, leg phases $\phi_{i,t}$, and stance duty factor d_t^{stance}, forming

$$\mathbf{c}_t^{\text{beh}} = [f_t^{\text{gait}}, \ell_t^{\text{clear}}, z_t^{\text{cmd}}, \beta_t^{\text{pitch}}, \chi_t, \psi_t^{\text{phase}}, \phi_{1,t}, \phi_{2,t}, d_t^{\text{stance}}]^\top. \tag{7.14}$$

The clearance command is measured relative to a contact-height reference, and the duty factor specifies the fraction of a periodic cycle allocated to stance. The phase variables provide a low-dimensional clock shared by the legs and the reward terms. They are internal clocks advanced from frequency and phase offset, rather than independent operator inputs. The periodic mapping below uses $0 < d_t^{\text{stance}} < 1$. Standing instead fixes the phases. For a control interval Δt,

$$\phi_{1,t+1} = (\phi_{1,t} + f_t^{\text{gait}}\Delta t) \bmod 1, \qquad \phi_{2,t+1} = (\phi_{1,t+1} + \psi_t^{\text{phase}}) \bmod 1. \tag{7.15}$$

A walking gait uses half-cycle phase separation, whereas a jumping gait synchronizes both legs. In the paper's nominal definitions, standing fixes both phases at 0.25, and hopping fixes the airborne leg at 0.75 while the other leg cycles. HUGWBC uses one policy for standing, walking, and jumping, and a separate policy for hopping. Both the reported settings and the public default configuration fix $d_t^{\text{stance}} = 0.5$.

The raw phase is normalized so that stance and swing occupy equal phase intervals according to

$$\bar{\phi}_{i,t} = \begin{cases} \dfrac{\phi_{i,t}}{2d_t^{\text{stance}}}, & 0 \le \phi_{i,t} < d_t^{\text{stance}}, \\ \dfrac{1}{2} + \dfrac{\phi_{i,t} - d_t^{\text{stance}}}{2(1 - d_t^{\text{stance}})}, & d_t^{\text{stance}} \le \phi_{i,t} < 1. \end{cases} \tag{7.16}$$

Thus stance maps to $[0, \frac{1}{2})$ and swing to $[\frac{1}{2}, 1)$. The actor receives an encoded command $\mathbf{c}_t = [\mathbf{v}_t^{\text{cmd}}; \tilde{\mathbf{c}}_t^{\text{beh}}]$, where $\tilde{\mathbf{c}}_t^{\text{beh}}$ replaces the two phase entries by $\sin(2\pi\bar{\phi}_{i,t})$. The complete command separates *what* the robot should accomplish from *how* the motion should look, while leaving the policy free to discover the joint-level realization.

7.3.3 *From Commands to Coordinated Actions*

The policy observes a history of joint positions and velocities, base angular velocity and gravity direction, and its previous action, using h_t defined in Eq. (7.1). Write $g_t = (\mathbf{c}_t, I_t^{\text{ext}})$ for the command input, where I_t^{ext} indicates whether an external upper-body controller is active. The deployed action mean is

$$\mathbf{a}_t = \boldsymbol{\mu}_\theta(h_t, g_t). \tag{7.17}$$

During training, exploration uses a stochastic policy around this mean. A history encoder and state estimator supply estimated translational velocity, body height, and foot clearance. The privileged critic additionally sees linear velocity, body-height error, foot clearance, contact forces, friction, terrain samples, and collision indicators during training. The history allows the policy to infer quantities that are not directly measured or are delayed, a practical form of implicit state estimation.

The action is interpreted as a joint target (or a residual around a nominal posture) and passed through the robot's low-level actuator loop. Consequently, a velocity command is not converted to a fixed stride table. The policy coordinates stance-leg actuation, swing-leg trajectories, trunk inclination, and arm motion to reduce the velocity error. Contact forces are controlled indirectly through body motion and joint actuation.

To describe a smooth contact target, let Φ be the standard normal cumulative distribution function and $\sigma_\phi > 0$ a dimensionless smoothing width. For normalized phase $z \in [0, 1)$, define

$$\begin{aligned} C_{\text{contact}}(z) = \Phi(z/\sigma_\phi)[1 - \Phi((z - \tfrac{1}{2})/\sigma_\phi)] \\ + \Phi((z-1)/\sigma_\phi)[1 - \Phi((z - \tfrac{3}{2})/\sigma_\phi)], \qquad C_{i,t}^{\text{contact}} = C_{\text{contact}}(\bar{\phi}_{i,t}). \end{aligned} \tag{7.18}$$

$C_{i,t}^{\text{contact}}$ is high in the intended stance interval and low in swing, with smooth transitions near phase boundaries. This command-derived expected contact weight is distinct from the sensor-based contact estimates of Section 7.2. The smooth weighting gives the contact objective a finite transition interval between stance and swing.

7.3.4 *Contact Penalties and Swing Targets*

Command following alone can admit motions that track velocity on average while making unnecessary contacts or moving the support foot. HUGWBC adds a periodic contact-swing objective. The public implementation uses nonnegative saturating functions (HugWBC Authors, 2025) given by

$$g_f(x) = 1 - \exp(-x^2/s_{\text{F}}), \qquad g_v(x) = 1 - \exp(-x^2/s_{\text{V}}), \qquad s_{\text{F}}, s_{\text{V}} > 0. \tag{7.19}$$

The denominators have units compatible with squared force and squared speed. With positive effective weights $w_{\text{F}}, w_{\text{V}}$ that absorb control-interval scaling by Δt, the combined penalty can be written as

$$r_t^{\text{contact}} = -\frac{1}{2} \sum_{i=1}^{2} \left[w_{\text{F}}(1 - C_{i,t}^{\text{contact}}) g_f(\|\mathbf{f}_{i,t}^{\text{foot}}\|_2) + w_{\text{V}} C_{i,t}^{\text{contact}} g_v(\|\mathbf{v}_{i,t}^{\text{foot}}\|_2) \right], \tag{7.20}$$

where $\mathbf{f}_{i,t}^{\text{foot}}$ is the three-dimensional contact force and $\mathbf{v}_{i,t}^{\text{foot}}$ the three-dimensional world-frame foot velocity. This follows the released implementation. The paper expresses its velocity term using horizontal components. The code defaults are $s_{\text{F}} = 50$, $s_{\text{V}} = 5$, and reward weights 2 and 4, respectively, before control-interval scaling. During stance the penalty encourages a planted support foot. During swing it discourages premature contact. The phase-dependent weight smooths the transition between these soft objectives.

In the paper, the vertical swing target is generated with a piecewise quintic polynomial whose coefficients satisfy position, velocity, and acceleration boundary conditions. A convenient mathematical expression uses

$$q_{\text{interp}}(u_{\text{interp}}) = 10u_{\text{interp}}^3 - 15u_{\text{interp}}^4 + 6u_{\text{interp}}^5, \qquad 0 \le u_{\text{interp}} \le 1, \tag{7.21}$$

which satisfies $q_{\text{interp}}(0) = 0$, $q_{\text{interp}}(1) = 1$, and zero first and second derivatives at both endpoints. For equal starting and ending contact heights, the relative clearance is

$$\ell_{i,t}^{\text{target}} = \ell_t^{\text{clear}} \begin{cases} 0, & 0 \le \bar{\phi}_{i,t} \le \frac{1}{2}, \\ q_{\text{interp}}(4\bar{\phi}_{i,t} - 2), & \frac{1}{2} < \bar{\phi}_{i,t} \le \frac{3}{4}, \\ q_{\text{interp}}(4 - 4\bar{\phi}_{i,t}), & \frac{3}{4} < \bar{\phi}_{i,t} < 1. \end{cases} \tag{7.22}$$

The target is zero in stance, reaches the commanded height ℓ_t^{clear} in mid-swing, and returns to the contact reference before touchdown. With fixed clearance and duty commands and constant phase rate during a cycle, height, velocity, and acceleration match at lift-off, the apex, and touchdown. For different endpoint heights, each half is written as $p_0 + (p_1 - p_0)q_{\text{interp}}(u_{\text{interp}})$ using its time-ordered endpoints and a local coordinate u_{interp} increasing from zero to one.

The paper's folded variable $u_{\text{fold}} = \frac{1}{4} - |\bar{\phi}_{i,t} - \frac{3}{4}|$ gives the equivalent swing expression $\ell_t^{\text{clear}} \sum_{k=0}^{5} a_k^{\text{swing}} u_{\text{fold}}^k = \ell_t^{\text{clear}} q_{\text{interp}}(4u_{\text{fold}})$, with $(a_0^{\text{swing}}, \ldots, a_5^{\text{swing}}) = (0, 0, 0, 640, -3840, 6144)$. This form makes the boundary conditions explicit. A swing penalty is

$$r_t^{\text{swing}} = -w_\ell \sum_{i=1}^{2} (1 - C_{i,t}^{\text{contact}})(\ell_{i,t}^{\text{target}} - \ell_{i,t}^{\text{foot}})^2, \qquad w_\ell > 0, \tag{7.23}$$

where $\ell_{i,t}^{\text{foot}}$ is actual clearance in the same convention. The polynomial supplies a smooth vertical reference, while the policy learns the coordinated foothold and body motion. The released implementation includes both linear and polynomial clearance rewards, with the default configuration enabling the linear term and assigning zero weight to the polynomial term (HugWBC Authors, 2025).

7.3.5 Environment Rewards and Mirror Regularization

The environment reward contains task tracking, behavior tracking, and regularization terms, combined as

$$r_t = r_t^{\text{task}} + r_t^{\text{behavior}} + r_t^{\text{reg}}. \tag{7.24}$$

Task terms track the three velocity components with dense exponential errors. Behavior terms track height, pitch, and waist yaw, and include the contact and swing terms above. Regularization penalizes torque, joint-speed limit violations, joint acceleration, and action increments. Collision-based termination discourages failed execution. In particular, an action-rate term $-\|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2$ discourages frame-to-frame jitter. These rewards define the rollout return and advantages used by PPO.

Mirror consistency is a separate training loss. Let $\mathcal{F}_h$ mirror body observations and past actions at each history step, $\mathcal{F}_g$ mirror the associated commands, and $\mathcal{F}_a$ mirror actions across the robot's sagittal plane. The intervention indicator is unchanged. For rollout minibatches $\mathcal{D}_{\text{roll}}$, the released implementation uses

$$\mathcal{L}_{\text{sym}} = \lambda_{\text{sym}} \mathbb{E}_{(h_t, g_t) \sim \mathcal{D}_{\text{roll}}} \left[\frac{1}{n_a} \left\| \text{sg}[\boldsymbol{\mu}_\theta(h_t, g_t)] - \mathcal{F}_a\big(\boldsymbol{\mu}_\theta(\mathcal{F}_h(h_t), \mathcal{F}_g(g_t))\big) \right\|_2^2 \right], \tag{7.25}$$

where n_a is the action dimension and sg denotes stop-gradient. The default $\lambda_{\text{sym}} = 0.5$. Symmetry is applied to observations, past actions, and commands as well as the action output, so it remains useful when the same policy is asked to produce left-right variants of an otherwise identical behavior (Yu et al., 2018).

The optimizer minimizes the PPO actor and value losses minus the entropy bonus, plus $\mathcal{L}_{\text{sym}}$ and the state-estimation loss when jointly trained. The mirror loss does not enter each-step reward, discounted return, or advantage. An independent foot-position symmetry reward used for non-walking gaits can enter the environment return. It is distinct from policy mirror consistency (HugWBC Authors, 2025).

7.3.6 *Couplings Between Commands*

The learned controller exposes several useful couplings. At approximately steady forward speed, the pair (v_x, f^{gait}) controls displacement per cycle. At fixed speed, raising frequency shortens each cycle's displacement. At fixed frequency, raising speed requires greater displacement. The pair $(f^{\text{gait}}, \ell^{\text{clear}})$ trades clearance against swing speed and work. Lateral velocity and turning interact with foot placement, phase offset, and waist orientation. Body height z^{cmd} changes leg configuration and effective workspace, while pitch β^{pitch} changes body geometry and the relation between base-frame and world-frame velocity. Finally, phase offset ψ^{phase} and duty factor d^{stance} specify the intended contact rhythm. A unified policy can learn these trade-offs from the return, while coordinated actuation and contact still limit the command combinations it can realize.

The reward weights also interact with these commands. For example, a high-frequency, high-clearance target increases the acceleration required by the swing trajectory, so an action-rate penalty suitable for walking may over-damp hopping. Curriculum sampling gradually expands command ranges and intervention magnitude. Massive parallel simulation makes such coverage practical (Rudin et al., 2022). Periodic reward composition and multiplicity of behavior provide related examples of diverse gait learning through explicit command spaces (Siekmann et al., 2021; Margolis and Agrawal, 2023).

A useful diagnostic is therefore a command-interaction map rather than a list of single-command errors. Evaluate tracking error and smoothness on a grid of $(v_x, v_y, \omega_z, f^{\text{gait}}, \ell^{\text{clear}}, z^{\text{cmd}}, \beta^{\text{pitch}}, \chi)$, then inspect where one command degrades another. Such tests assess behavior between command combinations as well as at isolated settings. They also expose saturation. If increasing f^{gait} no longer changes measured cadence, actuator or contact limits may be responsible, but further tests are needed to distinguish these limits from inadequate command conditioning.

7.3.7 *Disturbance Rejection and External Control*

Robustness is trained as part of motion generation. Upper-body intervention replaces the policy's arm actions with sampled joint targets or bounded noise, while the lower-body command remains active. The intervention indicator tells the policy which control mode is present, and the intervention is interpolated over a finite interval rather than switched instantaneously. Regularization rewards for the intervened joints are masked, while velocity tracking, contact objectives, collision-based termination, and lower-body regularization remain active. A curriculum gradually increases the contribution of external actions once baseline velocity tracking is reliable. This trains trunk and leg compensation for arm motion while preserving the commanded gait, supporting loco-manipulation.

The resulting architecture closes the feedback loop at each control step. A semantic command is encoded as velocity, posture, and phase variables. Temporal proprioception supports state estimation. A command-conditioned policy maps the combined input to joint targets. And contact and motion-quality objectives shape the solution. Low-level torque limits and deployment checks complement this learned control. For deployment, the most important tests jointly assess command tracking under phase changes, recovery after external impulses, action-rate and contact behavior, and retention of lower-body stability during upper-body intervention. Maintaining this coordination on hardware also depends on the delays and actuator limitations examined in Section 7.5.

7.4 Motor Pretraining and Dexterous Whole-Body Control

Command-conditioned control exposes a specified behavior domain. Motor pretraining seeks capabilities that can be reused across downstream objectives. Three approaches serve different roles. Reference tracking learns to execute supplied motions. Learned priors and skill representations support downstream policy learning. And multi-mode pretraining or distillation exposes a shared repertoire through varied commands (Peng et al., 2018a; 2021; 2022; He et al., 2025b; Zeng et al., 2025). These approaches can be combined. This section first develops tracking as a common learn-

ing objective, then examines behavior foundation models, and finally shows that the same dynamics, estimation, tracking objectives, and command interfaces carry over to the multi-fingered hand, where the held object rather than the robot's base is the unactuated body.

7.4.1 Motion Tracking

Motion tracking formulates humanoid whole-body control as the reproduction of a reference motion under the robot's dynamics. Reference trajectories provide a time-varying description of the desired behavior, capturing the coordinated movement of the body across different skills. DeepMimic (Peng et al., 2018a) establishes a reinforcement-learning formulation of this approach in physics-based character animation. Recent humanoid systems, including BeyondMimic (Liao et al., 2026) and SONIC (Luo et al., 2026), build on this principle to acquire physically executable skills and broad behavior repertoires. The reference specifies how the robot should move, while reinforcement learning discovers the feedback control needed to reproduce that movement under physical dynamics.

Formally, consider a reference motion sequence

$$\mathcal{M}_{\text{motion}} = (s_0^{\text{ref}}, s_1^{\text{ref}}, \dots, s_T^{\text{ref}}), \tag{7.26}$$

where s_t^{ref} specifies the target kinematic quantities at timestep t, such as joint configurations, body poses, and velocities. At each timestep, the policy selects an action conditioned on the available history and reference command, and the system evolves according to

$$\mathbf{a}_t \sim \pi_\theta(\cdot \mid h_t, g_t), \qquad s_{t+1} \sim P(\cdot \mid s_t, \mathbf{a}_t), \tag{7.27}$$

where π_θ is the policy parameterized by θ, $\mathbf{a}_t$ is the action, h_t is the observation-action history defined in Eq. (7.1), g_t is the reference-derived command, and P denotes the transition dynamics. The full physical state s_t may include privileged information accessible in simulation but unavailable to the deployed policy.

The discrepancy between the executed and reference motions provides a dense learning signal. A generic tracking reward takes the form

$$r_t^{\text{track}} = \sum_{k \in \mathcal{I}_{\text{track}}} w_k \exp\left[-\frac{d_k(s_{t+1}, s_{t+1}^{\text{ref}})}{{\sigma_k^{\text{track}}}^2} \right], \qquad w_k \geq 0, \tag{7.28}$$

where $\mathcal{I}_{\text{track}}$ indexes the tracked quantities, d_k measures the corresponding squared discrepancy, w_k determines its relative contribution, and $\sigma_k^{\text{track}} > 0$ controls the sensitivity to tracking errors. Additional costs can regularize physical execution through a reward of the form

$$r_t = r_t^{\text{track}} - \sum_j \lambda_j c_j(s_t, \mathbf{a}_{t-1}, \mathbf{a}_t, s_{t+1}), \qquad \lambda_j \geq 0, \tag{7.29}$$

where c_j denotes a nonnegative regularization cost and λ_j its weight. The preceding action is included to allow action-rate penalties. Such costs can penalize undesirable behavior, including abrupt action changes or joint-limit violations. Motion tracking therefore provides a shared behavioral objective while retaining regularization appropriate for physical execution.

The policy is trained to maximize the expected discounted return according to the objective

$$\max_\theta \ \mathbb{E}_{\mathcal{M}_{\text{motion}} \sim \mathcal{D}, \zeta_{\text{traj}} \sim p_\theta(\zeta_{\text{traj}} | \mathcal{M}_{\text{motion}})} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \tag{7.30}$$

where $\gamma \in [0, 1)$ is the discount factor, $\mathcal{D}$ is the reference-motion distribution, and ζ_{traj} is an executed state-action trajectory. The trajectory distribution p_θ is induced by the policy, transition dynamics, and initialization distribution. For a single-clip policy, $\mathcal{D}$ concentrates on one reference motion. For a controller trained on a motion corpus, it samples across multiple clips. In both

settings, reference motions specify the desired outcomes, while interaction with the environment supplies the experience used for policy optimization. PPO (Schulman et al., 2017) is commonly used to optimize this objective, as exemplified by SONIC (Luo et al., 2026).

Several practical strategies support effective learning. *Reference state initialization* starts episodes from sampled reference configurations and velocities, exposing the policy to later phases of a motion before it can reliably reach them from the beginning. *Early termination* ends episodes when predefined failure conditions are met, such as excessive tracking errors, avoiding prolonged rollouts in unsuccessful states. *Adaptive sampling* further directs training effort toward challenging motions or segments. For example, BeyondMimic (Liao et al., 2026) prioritizes temporal regions with high estimated failure rates while retaining a uniform sampling component to preserve coverage. Together, these strategies determine where rollouts begin, when they end, and how experience is distributed across the reference corpus.

7.4.2 Behavior Foundation Model

Motion tracking provides a shared learning objective, but a general-purpose controller must also accommodate the diverse commands used in downstream applications. A complete reference sequence may be suitable for motion replay, whereas interactive control may specify targets for only a few body links. This motivates behavior foundation models, humanoid controllers pretrained on large-scale motion data to acquire a broad repertoire of reusable whole-body behaviors and execute them in response to diverse behavioral specifications (Zeng et al., 2025; 2026). By combining broad behavioral coverage with a versatile control interface, BFMs enable learned motor capabilities to be shared across multiple control modes and downstream tasks.

In this formulation, a behavior is defined as a trajectory of body states and actions, where s_t^{p} denotes the proprioceptive body variables selected from s_t. The trajectory is written as

$$\mathbf{B}^{\mathrm{seq}} \triangleq (s_0^{\mathrm{p}}, \mathbf{a}_0, s_1^{\mathrm{p}}, \mathbf{a}_1, \ldots, s_{T-1}^{\mathrm{p}}, \mathbf{a}_{T-1}, s_T^{\mathrm{p}}). \tag{7.31}$$

Goal specifications are treated as external conditions that guide or constrain the execution of a behavior. A single behavior may satisfy different specifications, while a given specification may admit multiple physically valid behaviors. For example, specifying a hand target leaves considerable freedom in the robot's stance and the movement of its other arm. Separating goals from the definition of $\mathbf{B}^{\mathrm{seq}}$ therefore distinguishes the executed behavior from the conditions used to specify it.

From this perspective, BFM pretraining requires both a shared objective for acquiring diverse behaviors and a versatile interface for specifying their execution. Motion tracking provides the former through coordinated demonstrations and dense tracking rewards. The control interface provides the latter by allowing downstream applications to specify only the aspects of movement relevant to their objectives. Motion tracking thus serves as a proxy task for learning reusable motor capabilities, which can subsequently be invoked through commands that need not describe a complete whole-body trajectory.

One practical realization of this interface uses masked Cartesian body targets. Let $\mathbf{x}_t^{\mathrm{target}}$ contain target poses and tracking errors expressed in the robot's current root frame, and let $\mathbf{m}_t$ be a binary mask selecting the commanded body links. The command and policy are then

$$g_t = (\mathbf{m}_t \odot \mathbf{x}_t^{\mathrm{target}}, \mathbf{m}_t), \qquad \mathbf{a}_t \sim \pi_\theta(\cdot \mid h_t, g_t), \tag{7.32}$$

where h_t retains the history definition in Section 7.1.3, and the mask is applied to the features associated with each link. An entry $m_t^\ell = 1$ indicates that link ℓ is specified. An all-one mask requests full-body tracking, while a sparse mask specifies selected links and allows the controller to coordinate the remaining body movements. Including the mask explicitly distinguishes an unspecified target from a valid target whose encoded value is zero. Sampling different masks during pretraining enables a single controller to learn from multiple forms of behavioral specification. For globally specified targets, root-relative encoding uses the robot pose relative to the target.

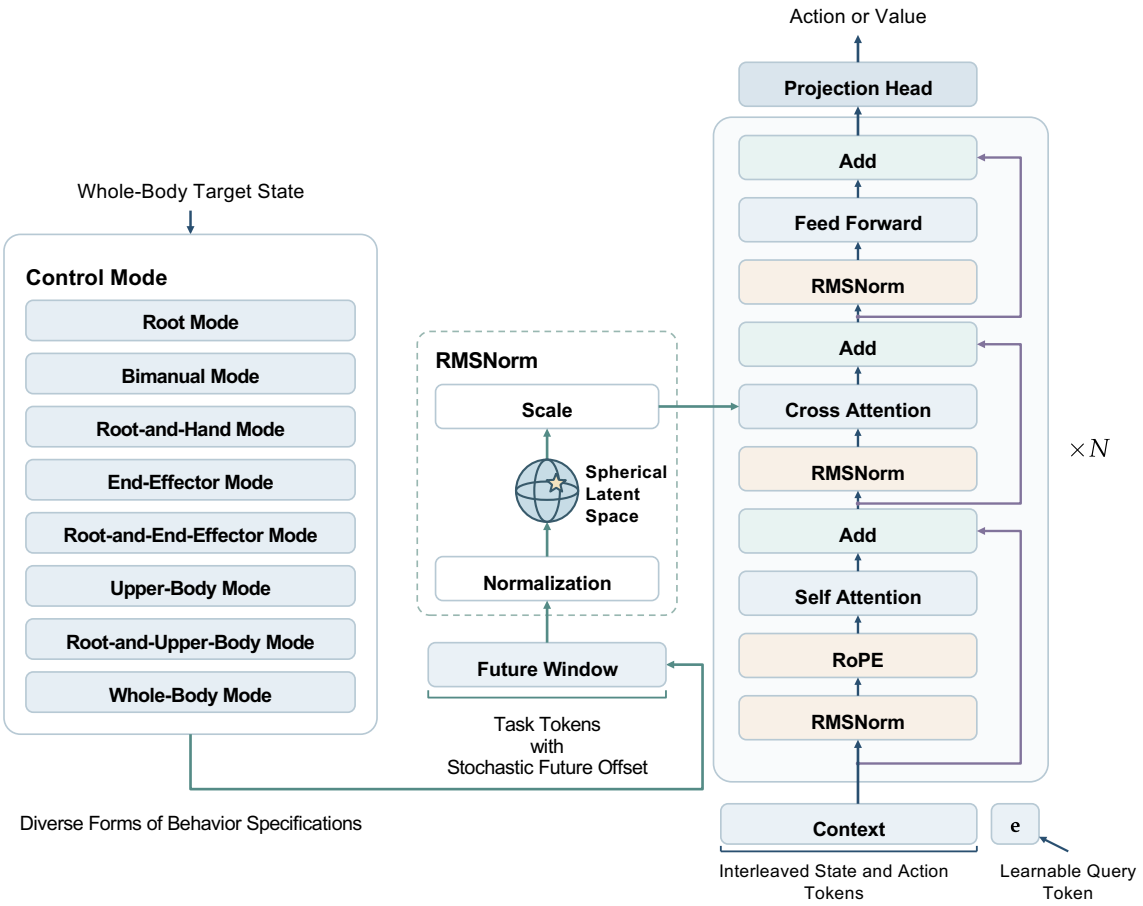


Fig. 7.7 Visualization of the Humanoid Transformer architecture applied in ScaleBFM (Zeng et al., 2026)

The original BFM implementation trains a motion-imitation policy and transfers its behavior through masked online distillation. It also evaluates learning new behaviors through a residual decoder with the BFM frozen (Zeng et al., 2025). ScaleBFM studies direct goal-conditioned reinforcement-learning pretraining, examining the joint effects of training data and model capacity (Zeng et al., 2026). Its findings highlight the complementary roles of reference-motion diversity and on-policy experience. Diverse references expand the range of behaviors available for learning, while rollouts supply the interaction experience needed to acquire them. Effective scaling therefore calls for coordinating these two resources with an architecture capable of translating diverse behavioral specifications into coordinated actions.

As its model backbone, ScaleBFM introduces the *Humanoid Transformer*, which combines temporal context with goal-conditioned control, as illustrated in Figure 7.7. Proprioceptive states and past actions are encoded as interleaved tokens and processed through self-attention to capture the recent execution history. A learnable query token aggregates this information for action or value prediction. Future goal states, together with their temporal offsets, are encoded into goal tokens and incorporated through cross-attention. This design allows the controller to interpret the requested movement in the context of its ongoing behavior, directly accommodating the history and commands in Eq. (7.32).

ScaleBFM also studies the representations learned through goal conditioning. It applies root mean square normalization (RMSNorm) (Zhang and Sennrich, 2019) to goal embeddings and analyzes

their normalized directions. For the selected motion sequences, neighboring targets yield nearby representations, while behaviors such as forward and backward walking occupy distinguishable regions. Perturbation experiments test how changes in embedding direction affect control performance. These findings provide evidence of local organization and robustness in the learned goal representation.

Together, these components illustrate how the general BFM framework can be instantiated in practice. Motion tracking supplies a shared learning objective, masked targets provide a versatile control interface, and the Humanoid Transformer relates behavioral specifications to the robot's execution history. ScaleBFM thus offers a concrete example of learning coordinated motor skills and structured goal representations within a single pretrained controller, supporting the reuse of its behavioral repertoire across multiple control modes.

7.4.3 Evaluation Metrics

Evaluating a BFM requires measuring how reliably and accurately it realizes the behavioral specifications supplied through its control interface. Motion tracking provides a direct setting for this assessment because the executed behavior can be compared with a time-aligned reference. Using the tracking metrics reported by ScaleBFM (Zeng et al., 2026), this chapter considers sequence-level success rate together with mean per-keypoint position and rotation errors in global and local coordinates. These metrics describe complementary aspects of control, including persistent compliance with the command, accuracy in the world frame, and fidelity of the body configuration after horizontal root displacement and heading differences are removed.

Consider a reference motion $\mathcal{M}_{\text{motion}}$ and the corresponding rollout of $\pi_\theta(\cdot \mid h_t, g_t)$. With the initial state indexed by 0 as above, let $t = 1, \ldots, T$ index the frames actually executed and evaluated. For a fixed control mode, the command mask satisfies $\mathbf{m}_t = \mathbf{m}$, and the set of evaluated links is

$$\mathcal{L}_{\mathbf{m}}^{\text{links}} = \{\ell \mid m^\ell = 1\}. \tag{7.33}$$

Let $\mathbf{p}_{t,\ell} \in \mathbb{R}^3$ and $\mathbf{R}_{t,\ell} \in \mathrm{SO}(3)$ denote the world-frame position and orientation of link ℓ, extracted from s_t, and let $\mathbf{p}_{t,\ell}^{\text{ref}}$ and $\mathbf{R}_{t,\ell}^{\text{ref}}$ denote their counterparts in s_t^{ref}. In SI units, these position quantities are measured in m. Each rotation matrix maps vectors from the corresponding link frame to the world frame. The reference and executed states refer to the same robot embodiment and the same timestep. Only links in $\mathcal{L}_{\mathbf{m}}^{\text{links}}$ contribute to the tracking metrics. Sequences with an all-zero command mask are excluded from the tracking-error means below, and the number excluded is reported. An unspecified link may move differently from the reference while helping the robot satisfy the commanded targets.

Success Rate For an evaluation set containing $N_{\text{eval}} > 0$ reference sequences, let T_i denote the full reference length of sequence i in frames, with $T_i > 0$ and a nonempty selected-link set. Given a positional tolerance $\delta > 0$, the success rate is

$$\mathrm{Succ}(\delta) = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} \mathbf{1}\left[\max_{\substack{1 \le t \le T_i \\ \ell \in \mathcal{L}_{\mathbf{m}}^{\text{links}}}} \left\| \mathbf{p}_{t,\ell}^{(i)} - \mathbf{p}_{t,\ell}^{\text{ref},(i)} \right\|_2 \le \delta \right], \tag{7.34}$$

where $\mathbf{1}[\cdot]$ is the indicator function. A sequence succeeds only if every selected link remains within the tolerance throughout its execution. A rollout terminated by failure before completing the reference is counted as unsuccessful. ScaleBFM uses $\delta = 0.5\,\text{m}$. The maximum over links and time makes this criterion sensitive to even a single large deviation. Success rate therefore complements average tracking errors, which can conceal brief failures. Since the criterion constrains position alone, orientation accuracy must be assessed separately. The score lies in $[0, 1]$ and may be multiplied by 100 when reported as a percentage.

Global Tracking Errors For $T > 0$ and a nonempty selected-link set, the global mean per-keypoint position error averages the Euclidean distance between corresponding link positions. The

error is computed as

$$\text{G-MPKPE} = \frac{1}{T|\mathcal{L}_{\mathbf{m}}^{\text{links}}|} \sum_{t=1}^{T} \sum_{\ell \in \mathcal{L}_{\mathbf{m}}^{\text{links}}} \left\| \mathbf{p}_{t,\ell} - \mathbf{p}_{t,\ell}^{\text{ref}} \right\|_2 . \tag{7.35}$$

This error measures positional agreement in the world frame and is affected by both root motion and body articulation. For example, a controller may reproduce the reference limb configuration while gradually drifting away from the intended path, resulting in a large global position error.

For orientations, define the angular distance between two rotation matrices as

$$d_{\text{R}}(\mathbf{R}_1, \mathbf{R}_2) = \arccos\left(\text{clip}\left(\frac{\text{tr}(\mathbf{R}_1^\top \mathbf{R}_2) - 1}{2}, -1, 1 \right) \right), \tag{7.36}$$

where tr denotes the matrix trace and clipping accommodates numerical roundoff. This distance is the magnitude of the shortest relative rotation, taking values in $[0, \pi]$. It gives a geometric orientation error without depending on the choice or wrapping of Euler angles. The global mean per-keypoint rotation error is then

$$\text{G-MPKRE} = \frac{1}{T|\mathcal{L}_{\mathbf{m}}^{\text{links}}|} \sum_{t=1}^{T} \sum_{\ell \in \mathcal{L}_{\mathbf{m}}^{\text{links}}} d_{\text{R}}\left(\mathbf{R}_{t,\ell}, \mathbf{R}_{t,\ell}^{\text{ref}} \right) . \tag{7.37}$$

Position errors are measured in meters and rotation errors in radians. Both are means of per-link distances, without squaring the errors.

Local Tracking Errors Local evaluation removes the horizontal root position and heading of each trajectory at each timestep. To make this convention explicit, let

$$\mathbf{c}_t^{\text{root}} = \begin{bmatrix} p_{t,\text{x}}^{\text{root}} & p_{t,\text{y}}^{\text{root}} & 0 \end{bmatrix}^\top , \qquad \mathbf{H}_t = \mathbf{R}_{\text{z}}(\psi_t^{\text{head}}), \tag{7.38}$$

where $\mathbf{p}_t^{\text{root}}$ is the robot root position, ψ_t^{head} is its heading angle, and $\mathbf{R}_{\text{z}}(\psi_t^{\text{head}})$ is a rotation about the vertical axis. Define $\mathbf{c}_t^{\text{root,ref}}$ and $\mathbf{H}_t^{\text{ref}}$ analogously from the reference root state. The aligned link poses are

$$\begin{aligned} \bar{\mathbf{p}}_{t,\ell} &= \mathbf{H}_t^\top (\mathbf{p}_{t,\ell} - \mathbf{c}_t^{\text{root}}), & \bar{\mathbf{R}}_{t,\ell} &= \mathbf{H}_t^\top \mathbf{R}_{t,\ell}, \\ \bar{\mathbf{p}}_{t,\ell}^{\text{ref}} &= (\mathbf{H}_t^{\text{ref}})^\top (\mathbf{p}_{t,\ell}^{\text{ref}} - \mathbf{c}_t^{\text{root,ref}}), & \bar{\mathbf{R}}_{t,\ell}^{\text{ref}} &= (\mathbf{H}_t^{\text{ref}})^\top \mathbf{R}_{t,\ell}^{\text{ref}}. \end{aligned} \tag{7.39}$$

The executed and reference poses are thus expressed in their respective heading frames, with each frame origin placed at the root's horizontal projection onto the ground plane. Because this transformation removes only horizontal translation and yaw, discrepancies in height, roll, and pitch remain measurable. Applying the same position and rotation distances to these aligned quantities yields

$$\begin{aligned} \text{L-MPKPE} &= \frac{1}{T|\mathcal{L}_{\mathbf{m}}^{\text{links}}|} \sum_{t=1}^{T} \sum_{\ell \in \mathcal{L}_{\mathbf{m}}^{\text{links}}} \left\| \bar{\mathbf{p}}_{t,\ell} - \bar{\mathbf{p}}_{t,\ell}^{\text{ref}} \right\|_2 , \\ \text{L-MPKRE} &= \frac{1}{T|\mathcal{L}_{\mathbf{m}}^{\text{links}}|} \sum_{t=1}^{T} \sum_{\ell \in \mathcal{L}_{\mathbf{m}}^{\text{links}}} d_{\text{R}}\left(\bar{\mathbf{R}}_{t,\ell}, \bar{\mathbf{R}}_{t,\ell}^{\text{ref}} \right) . \end{aligned} \tag{7.40}$$

A low local error indicates accurate reconstruction of the commanded body configuration after this alignment. A substantial global position error together with a small local position error can therefore indicate difficulty following the desired root trajectory despite accurate local pose reproduction. Global and local scores should consequently be interpreted together. Their coordinate conventions describe how a rollout is scored and are separate from the coordinate encoding used to provide g_t to the policy.

For meaningful comparisons, results should be reported separately for each control mode using matched reference motions, selected links, initialization, and sampling frequency. The formulas

above define errors for one sequence. The reported mean is the arithmetic mean of the per-sequence errors, with each included sequence receiving equal weight. The error statistics retain the evaluated frames from failed and truncated rollouts. Because early termination can omit harder parts of a reference, mean errors should be interpreted together with success rate and rollout duration. Applying the same protocol to held-out motions from familiar sources and motions from unseen sources further distinguishes tracking accuracy from cross-source generalization. These measures characterize the realization of behavioral specifications. Downstream scene-interaction tasks additionally require evaluation of their own task outcomes.

7.4.4 Dexterous Hands: Motor Control at the Finger Level

A Cartesian hand target in a body controller specifies where the hand should move, but leaves its finger configuration and object contacts unresolved. Extending motor pretraining to dexterity therefore requires examples of contact transitions, together with an interface that can request useful movement without prescribing every contact.

The Held Object as a Floating Base A multi-fingered hand needs the same layers of low-level control as the humanoid body, because its control problem has the same structure. Recall from Eq. (7.5) that the first six rows of a legged robot's dynamics contain no joint torque. The base moves only through inertial coupling and contact. In in-hand manipulation the unactuated body is the held object. With the wrist pose given by the arm, the object twist $\boldsymbol{\nu}_\mathrm{o} \in \mathbb{R}^6$ and the finger joints $\mathbf{q}_\mathrm{h}$ evolve as

$$\mathbf{M}_\mathrm{o}\dot{\boldsymbol{\nu}}_\mathrm{o} + \mathbf{b}_\mathrm{o} = \mathbf{G}(\mathbf{q}_\mathrm{h}, \mathbf{x}_\mathrm{o})\, \mathbf{f}_{\mathcal{K}_\mathrm{hand}} + \mathbf{w}_\mathrm{ext}, \qquad \mathbf{M}_\mathrm{h}(\mathbf{q}_\mathrm{h})\ddot{\mathbf{q}}_\mathrm{h} + \mathbf{b}_\mathrm{h} = \boldsymbol{\tau} - \mathbf{J}_{\mathrm{h},\mathcal{K}_\mathrm{hand}}^\top \mathbf{f}_{\mathcal{K}_\mathrm{hand}}, \tag{7.41}$$

where $\mathcal{K}_\mathrm{hand}$ is the set of active finger-object contacts, $\mathbf{x}_\mathrm{o}$ is the object pose, $\mathbf{M}_\mathrm{o}$ and $\mathbf{b}_\mathrm{o}$ its spatial inertia and bias wrench, $\mathbf{f}_{\mathcal{K}_\mathrm{hand}}$ stacks the forces that the fingers apply at the active contacts, and the grasp matrix $\mathbf{G}$ maps them to the resultant wrench on the object (Lynch and Park, 2017). Each contact force must lie in the friction cone of Eq. (7.6). As for the floating base, no actuator acts on the object directly. Joint torque $\boldsymbol{\tau}$ reaches it only through $\mathbf{J}_{\mathrm{h},\mathcal{K}_\mathrm{hand}}$ and $\mathbf{G}$, and only while the corresponding contacts are closed. Contact forces in the null space of $\mathbf{G}$ squeeze the object without moving it. They set the friction margins that keep the grasp stable (Bicchi, 1994).

Table 7.3 extends the correspondence to the rest of the chapter. Two points go beyond the table. First, rolling or sliding fingertips change $\mathbf{G}$ and $\mathbf{J}_{\mathrm{h},\mathcal{K}_\mathrm{hand}}$ during a manipulation, much as foot place-

Table 7.3 Correspondence between humanoid whole-body control and dexterous in-hand control. References in the humanoid-body column point to the relevant formulations and sections of this chapter. The dexterous-hand column shows analogous quantities for a multi-fingered hand holding an object

Concept	Humanoid body	Dexterous hand
Unactuated body	Floating base, Eq. (7.5)	Held object, Eq. (7.41)
Contacts	Feet or hands on the environment	Fingertips, phalanges, and palm on the object
	Friction cone	Same cone
Contact schedule	Gait phase, stance and swing (Section 7.3)	Finger gaiting (release, reposition, re-contact)
Stability objective	Balance of the center of mass over the support	Grasp stability (resultant wrench and friction margins)
Hidden state	Base velocity, foot contact, terrain	Object pose, contact locations, incipient slip
Command interface	Velocity commands, masked link targets, Eq. (7.32)	Fingertip targets, object-pose goals, functional finger coordinates
Reference data	Retargeted human motion capture	Retargeted human hand motion from gloves or video
Failure and recovery	Fall and get-up (Section 7.5)	Slip, drop, and regrasp

ment changes the stance Jacobian, so the contact mode is part of the state to be estimated. Second, the hand differs in scale and instrumentation. Finger actuators are small, often heavily geared, and work close to their force limits, so the gap between commanded and realized torque in Eq. (7.7) matters more than it does for a leg. Available sensing ranges from motor current to fingertip tactile images (Section 9.6.3). And the hand's "floating base" has a different mass and shape in every task.

The three routes to reusable motor skills named at the start of this section, namely reference tracking, learned priors, and goal-conditioned interfaces, carry over to the hand with the object in place of the base.

Tracking Human Hand References TeleDexter makes the desired object motion explicit (Li et al., 2026c). The operator specifies fingertip positions and object pose in the wrist frame. A recurrent hand policy maps these goals and the observed hand-object state to residual joint-position commands, while independent inverse kinematics tracks the operator's wrist with a fixed-base Franka FR3 arm. Training uses consecutive hand-object subgoals, with sparse goal-reaching rewards supplemented by dense tracking rewards. The hand can discover contact strategies between subgoals rather than reproduce every reference frame. Deployment runs at 30 Hz. Reaching and initial grasping use kinematic retargeting before the operator switches to the learned in-hand controller. Its training signal follows Eq. (7.28), but the reference covers the object and fingertips instead of every joint, much like the sparse masked targets of Eq. (7.32).

On a SharpaWave hand, TeleDexter achieves 75.2% mean success over seven reorientation and tool-use tasks, with 15 trials per task. Reusing the human reference data after retargeting and training a separate LeapHand controller yields 60% and 73.3% success on two reorientation tasks. This is reuse of data and a training procedure across hand morphologies. The current controller remains object-specific. A new object requires reference collection and policy training.

A Learned Motion Prior for Fingers DexterityGen gives a concrete pretraining implementation (Yin et al., 2025). Simulation reinforcement learning generates transitions between nearby sampled grasps, supplemented by free-finger and precision-manipulation behaviors. Varying object geometry, wrist motion, and physical parameters broadens the resulting dataset. A diffusion model then learns a distribution of future finger-keypoint motion conditioned on proprioceptive history and a mode indicator. An inverse dynamics model maps sampled motion to joint-position targets. At deployment, an external motion command guides diffusion sampling toward the requested finger movement while the learned prior favors motions represented in the training data. Thus, the command enters through sampling guidance. It is not simply copied to the joints. As in a behavior foundation model, a prior learned from broad simulated experience supplies coordinated motion, and a sparse command selects among behaviors it already supports.

The experiments demonstrate reuse on an Allegro hand attached to a separately controlled Franka Panda arm, with human teleoperation supplying motion commands. Test objects are absent from the pretraining dataset. Assisted teleoperation improves object retention and completes reorientation and regrasping tasks, but complete screwdriver and syringe tasks succeed in only 3/20 and 4/20 trials, respectively. These results support a reusable hand-motion prior while identifying the difficulty of composing skills into long tasks. The low-level controller uses proprioception rather than visual or tactile input. Control errors provide only an indirect contact cue.

Object-Goal Control Learned in Simulation SimToolReal asks whether simulated experience can produce a controller that follows object goals taken from a human demonstration. It uses a 22-DoF Sharpa hand on a KUKA iiwa arm and trains a recurrent goal-reaching policy on procedurally generated tool-like objects. At deployment, a human RGB-D demonstration supplies a sequence of object poses. The policy observes proprioception, the current object pose, a grasp-region bounding box, and the current goal, and commands arm and hand joint targets (Kedia et al., 2026). A demonstration can thus specify how a brush should be oriented without saying which finger releases first, and the controller is free to rotate the brush in the hand when moving the whole arm would hit the table. This is the object-centered goal representation of Section 8.3.1. The study evaluates 120 real rollouts over 24 trajectories, 12 tool instances, and six categories. Its Task Progress metric counts

the fraction of demonstrated poses reached within a distance threshold and does not measure each tool's effect (Kedia et al., 2026). Reaching the intended hammer trajectory is not the same as driving a nail to depth. In the terms of Table 7.3, SimToolReal tracks a reference for the unactuated body itself and leaves the contact schedule to the policy, as a velocity command leaves footstep placement to a locomotion controller.

Adapting a Prior to Tight Contact Play2Perfect, also on a Sharpa hand and KUKA arm, pretrains object manipulation in free space and then fine-tunes in CAD-derived assembly environments with sparse goal rewards. In 10 real trials at 0.5 mm insertion clearance, the fine-tuned policy succeeds 6 times and the play-only policy never (Lum et al., 2026). The deployment is zero-shot sim-to-real, but the policy for the assembly task itself was trained in simulation. Free-space reorientation can reach the right pose through many contact arrangements. Insertion cannot, since a small angular error produces an edge contact that jams the part, and the grasp must still allow corrective motion.

This is the hand counterpart of adapting a pretrained body controller to a new behavior, as the residual decoder of the original BFM does. The prior makes the approach reachable, and task-specific training concentrates on the final contact.

The interfaces exposed by the four controllers to the layer above are fingertip and object goals (TeleDexter), guided finger-keypoint motion (DexterityGen), object-pose sequences (SimToolReal), and a task reward in simulation (Play2Perfect). Any of them could be where a manipulation foundation model (Section 6.6) hands a goal to a hand controller, as a semantic planner hands targets to a humanoid controller (Section 7.1.3). None yet combines broad object coverage with contact sensing, the subject of the next subsection.

7.4.5 Contact Feedback and Force-Adaptive Body-Hand Coordination

Contact execution requires feedback that distinguishes mechanically different states. Similar finger positions can occur during pressing, sliding, or a failed regrasp. TeleDexter excludes contact-force and tactile observations. Its failure analysis links tracking stalls to the inability to distinguish pressing against an object from sliding past it. Tool impacts also perturb grasps beyond its free-space hand-object training distribution. Motion targets specify intended interaction, while contact feedback helps determine whether it is being realized.

One response is to give the controller a state that keeps evidence the current image has lost. WM-Craftnet combines wrist depth, binary tactile contact, proprioception, and previous actions in a recurrent World Synesthesia Model, whose state conditions a PPO policy that outputs relative targets for the 22 joints of a Sharpa Wave hand. Simulation supplies clean-depth reconstruction targets during training. At deployment the state is updated from noisy sensor data (Yin et al., 2026a). The model gives the policy context and does not generate imagined rollouts, so it is a history-based state estimator in the sense of Section 5.1.2, serving as the hand counterpart of the RMA-style estimators of Section 7.2.3, with the object and its contacts as the hidden state. On an unseen double-notched block, WM-Craftnet meets the rotation criterion (more than half a revolution within 20 seconds) in 8 of 10 real trials, while a policy using WSM-denoised depth succeeds in none (Yin et al., 2026a). This favors recurrent interaction context over image denoising alone, although matched comparisons of history, capacity, and objectives would be needed to pin down the source of the gain.

Touch can also correct execution between visual updates. T-Rex, evaluated on a fixed-base dual-arm system with two Sharpa Wave hands, refines actions asynchronously from recent fingertip deformation and force history, reusing the visual-language context instead of recomputing it for every refinement (Niu et al., 2026a). A separate 300 Hz thread tracks the updated targets. Recent contact evidence can therefore change finger motion while the action chunk remains valid at the task level, the multirate interface of Section 7.1.3. In a six-task ablation, the average score is 65 for the full system, 42 without touch, and 60 without asynchronous refinement, under the paper's task-specific scoring (Niu et al., 2026a). Both the tactile signal and the way it is integrated contribute. The split between slow refinement and a fast servo is the hand version of the latency budget of Section 7.5. The tactile rates that make it necessary are given in Section 9.6.3.

Body-hand coordination introduces a further mechanical constraint. When a held object accelerates with the body, its required net contact force changes. Arm configuration can also change the moment transmitted to the base. With both hands on one object, their forces jointly determine its resultant wrench. Force components that cancel at the object can still change internal grasp loading and friction margins (Bicchi, 1994). Coordinating a feasible object trajectory therefore also requires maintaining contacts and body support.

FAME provides direct evidence for force-adaptive standing (Pudasaini et al., 2026). An encoder jointly maps 15 torso-and-arm joint positions and the two three-dimensional hand forces to an eight-dimensional latent, which conditions a lower-body policy. Training combines an upper-body pose curriculum with sampled hand loads. Deployment estimates each hand force from measured joint torques $\boldsymbol{\tau}_t$, rigid-body inverse dynamics, and the hand Jacobian, without wrist force/torque sensors. The policy outputs 12 leg-joint position offsets at 50 Hz. Upper-body motion remains externally specified. This force estimate describes the load transmitted through an arm.

In simulation, FAME is evaluated on 100 upper-body configurations sampled from the training pose distribution, with 25 load draws per configuration. Success requires remaining upright, preserving the stance, and limiting force-induced hand displacement. At a 150 mm tolerance, success is 38.9%, compared with 16.6% when the same force information is supplied without jointly encoding force and upper-body configuration. On a Unitree H1-2, FAME maintains stance in all three asymmetric single-arm and all three symmetric bimanual loading trials. The pose-only baseline falls in all corresponding trials. These limited hardware tests establish load-dependent leg compensation. Together, the cases identify complementary interfaces. Object-and-finger goals request contact execution, tactile observations refine local actions, and hand-load estimates inform body support. An integrated system would need to be evaluated on simultaneous grasp maintenance, object motion, and balance.

7.5 Real-Time Execution, Recovery, and Safety Control

Deploying learned control policies from simulation to real humanoid robots involves challenges that extend beyond the performance of the policy itself. In practice, the system must operate under limited computational resources, perception and communication delays, actuator dynamics, and unexpected external disturbances while maintaining stable, continuous, and safe control. This is particularly challenging for humanoid robots due to their high-dimensional, strongly coupled, and partially underactuated dynamics, where local errors can propagate into whole-body deviations or even loss of balance. Therefore, real-world deployment requires jointly addressing real-time inference, motion smoothing, sim-to-real adaptation, anomaly monitoring, disturbance rejection, and recovery, ultimately forming a closed-loop safety framework from normal execution to failure handling and task recovery. Standing up from a fallen, seated, or leaning posture brings several of these requirements together. The robot must reorganize its contacts, control the resulting motion, and settle into stable support. HoST, introduced in *Learning Humanoid Standing-up Control across Diverse Postures*, provides a concrete example of learning this recovery capability (Huang et al., 2025b).

7.5.1 Real-Time Execution and the Sim-to-Real Gap

Deploying learned control policies on physical humanoid robots requires bridging a substantial gap between simulation and the real world. This gap arises from multiple mismatches between the simulated and physical systems across dynamics, actuation, sensing, computation, and environmental interaction. An implementation with explicit state estimation can be summarized as

$$o_t \longrightarrow \hat{s}_t \longrightarrow \pi_\theta(\cdot \mid \hat{s}_t, g_t) \longrightarrow \mathbf{a}_t \longrightarrow \text{actuator and physical system} \longrightarrow o_{t+1}, \tag{7.42}$$

where o_t denotes the observation at time t, $\hat{s}_t$ is the estimated robot state, π_θ is the learned policy, g_t is the command, and $\mathbf{a}_t$ is the resulting action. The schematic suppresses the estimator's memory and any additional policy history. A history-conditioned policy can instead use h_t directly or combine it with explicit estimates. In practice, the discrepancy between the simulated and real

execution of this loop can progressively degrade motion tracking, stability, and task performance. The major sources of the sim-to-real gap are discussed below.

Dynamics and Physical Parameter Mismatch The physical dynamics of a humanoid robot are difficult to reproduce exactly in simulation. Differences in mass distribution, center of mass, inertia, joint friction, damping, actuator strength, gear transmission, and joint limits can cause the simulated robot to respond differently from its physical counterpart under the same control command. Contact-related parameters, such as ground friction, restitution, and collision geometry, further introduce discrepancies during locomotion, jumping, and other highly dynamic motions. Although system identification and domain randomization can reduce these differences, the real robot inevitably exhibits unmodeled dynamics and parameter variations that are difficult to capture completely during training.

Actuation and Low-Level Control Mismatch The action produced by a learned policy is not directly realized as an idealized control input on a physical robot. Instead, it is typically converted through a low-level controller, actuator dynamics, communication interfaces, and hardware-specific constraints. Differences between simulated and real actuator response, including torque tracking errors, motor bandwidth, saturation, dead zones, and control frequency, can therefore introduce discrepancies between the commanded and realized motion. These errors become particularly significant for fast motions, where accurate timing and high-frequency torque responses are essential.

Perception and State-Estimation Errors In simulation, the policy can often access accurate robot states, object positions, or environment information, whereas real systems must estimate these quantities from noisy and delayed sensor measurements. Camera noise, depth errors, occlusions, motion blur, imperfect calibration, and tracking failures can lead to discrepancies between the true physical state and the state provided to the policy. Such errors can alter both the magnitude and timing of the resulting control actions, especially for tasks involving fast-moving objects or rapidly changing contacts.

Control Latency and Asynchronous Execution Real-time execution also introduces computational and communication delays across perception, state estimation, policy inference, communication, and actuation. For a sequential processing path, the total latency can be approximated as

$$d_{\text{total}} = d_{\text{perception}} + d_{\text{estimation}} + d_{\text{policy}} + d_{\text{communication}} + d_{\text{actuation}}, \tag{7.43}$$

where each d is a time duration. In asynchronous or overlapping pipelines, the observation timestamp and actual execution time determine information age. The sum alone is insufficient. Given a control period Δt, a large d_{total} means that the action is effectively computed from an outdated state, introducing a phase lag into the control loop. This effect is particularly detrimental to fast dynamic motions and balance control, where even a small temporal mismatch can result in noticeable tracking errors. In addition, asynchronous execution between perception, policy inference, and low-level control can introduce variable delays, making the effective control dynamics different from those assumed during simulation.

HoST illustrates how policy updates and joint feedback operate at different rates. On the 23-DoF Unitree G1, a three-layer MLP uses a short proprioceptive history, including body roll and pitch, angular velocity, and joint states, together with the previous action and action scale. The policy updates joint-position targets at 50 Hz, while the physical robot's proportional-derivative (PD) servos run at 500 Hz (Huang et al., 2025b). Joint feedback thus continues between policy updates. The simulation uses a 200 Hz PD loop, illustrating why transfer depends on the response of the complete policy-servo system as well as the policy update rate.

Contact and Environmental Mismatch Physical interaction with the environment is another important source of discrepancy. Contact locations, collision geometry, ground compliance, friction conditions, and external disturbances are often simplified in simulation but can vary substantially

in the real world. For humanoid robots, small differences in foot-ground contact or object interaction can change the resulting contact forces and consequently alter the entire-body motion. Such discrepancies are particularly challenging for behaviors involving impacts, rapid changes of support, or interaction with non-rigid or unstructured environments.

Reference-Tracking Error For systems that generate reference motions and subsequently track them with a learned controller, the sim-to-real gap can be further amplified by tracking errors. Even if a kinematic trajectory generated by a planner is physically plausible, the real robot may not reproduce it exactly due to actuator limitations, modeling errors, latency, and state-estimation noise. The resulting deviation between the reference trajectory and the executed motion can accumulate over time, causing the robot to gradually drift away from the intended state. This issue is especially important in hierarchical architectures, where the output of one module becomes the input or conditioning signal of another.

Error Accumulation in Hierarchical and Autoregressive Systems The effect becomes more pronounced when motion generation and control are coupled through multiple sequential modules. Consider a hierarchical system consisting of a high-level motion planner and a low-level tracking controller. If the planner generates a reference trajectory $\mathbf{q}^{\text{ref}}_{t:t+H}$ spanning the current step and H future control steps and the tracker executes it with tracking error, the actual state can deviate from the planned trajectory. When subsequent planning decisions use execution feedback, this deviation can propagate to future references. Errors in previously generated references can also propagate through the history used by an autoregressive planner.

This problem is particularly evident in autoregressive kinematic planners. Instead of generating an entire trajectory independently, an autoregressive planner predicts future motion conditioned on previously generated reference states and a planning condition c^{plan}_t according to

$$\mathbf{q}^{\text{ref}}_{t+1} = F_{\text{plan}}(\mathbf{q}^{\text{ref}}_{0:t}, c^{\text{plan}}_t), \tag{7.44}$$

where F_{plan} is the motion generator. Consequently, an error in an earlier prediction can become part of the input to subsequent predictions, resulting in accumulated prediction error over a long horizon. In a hierarchical planner-tracker architecture, tracking errors introduce another source of deviation between the planned and physically executed states. When execution feedback enters c^{plan}_t or another planning input, these state deviations can change future references. This coupling is particularly important for long-horizon and highly dynamic behaviors, where small local discrepancies can progressively amplify over time. Deep Tracking Control combines an optimization-based planner with a learned tracker (Jenelten et al., 2024). AdaPT conditions a tennis planner on robot and ball information and adapts the reference execution speed (Huang et al., 2026a). These examples illustrate how planner-tracker interfaces connect motion generation with physical execution. Figure 7.8 summarizes sources of sim-to-real mismatch and their propagation through a planner-tracker system.

7.5.2 Motion Smoothing and Continuous Execution

In addition to inference speed, the temporal continuity of policy outputs is critical for stable real-world execution. Directly executing the actions produced by a learned policy at each timestep may result in high-frequency oscillations, particularly under observation noise, out-of-distribution states, or discrepancies between simulated and real dynamics. Such oscillations can introduce unnecessary joint acceleration, actuator load, and mechanical wear.

Locally Lipschitz Continuous Constraint (L2C2) regularization encourages local smoothness of policy and value functions during training (Kobayashi, 2022). It penalizes changes under nearby input perturbations, helping reduce sensitivity to small observation changes. For highly dynamic behaviors, the resulting action continuity must be considered together with the responsiveness needed to preserve task-relevant motion components.

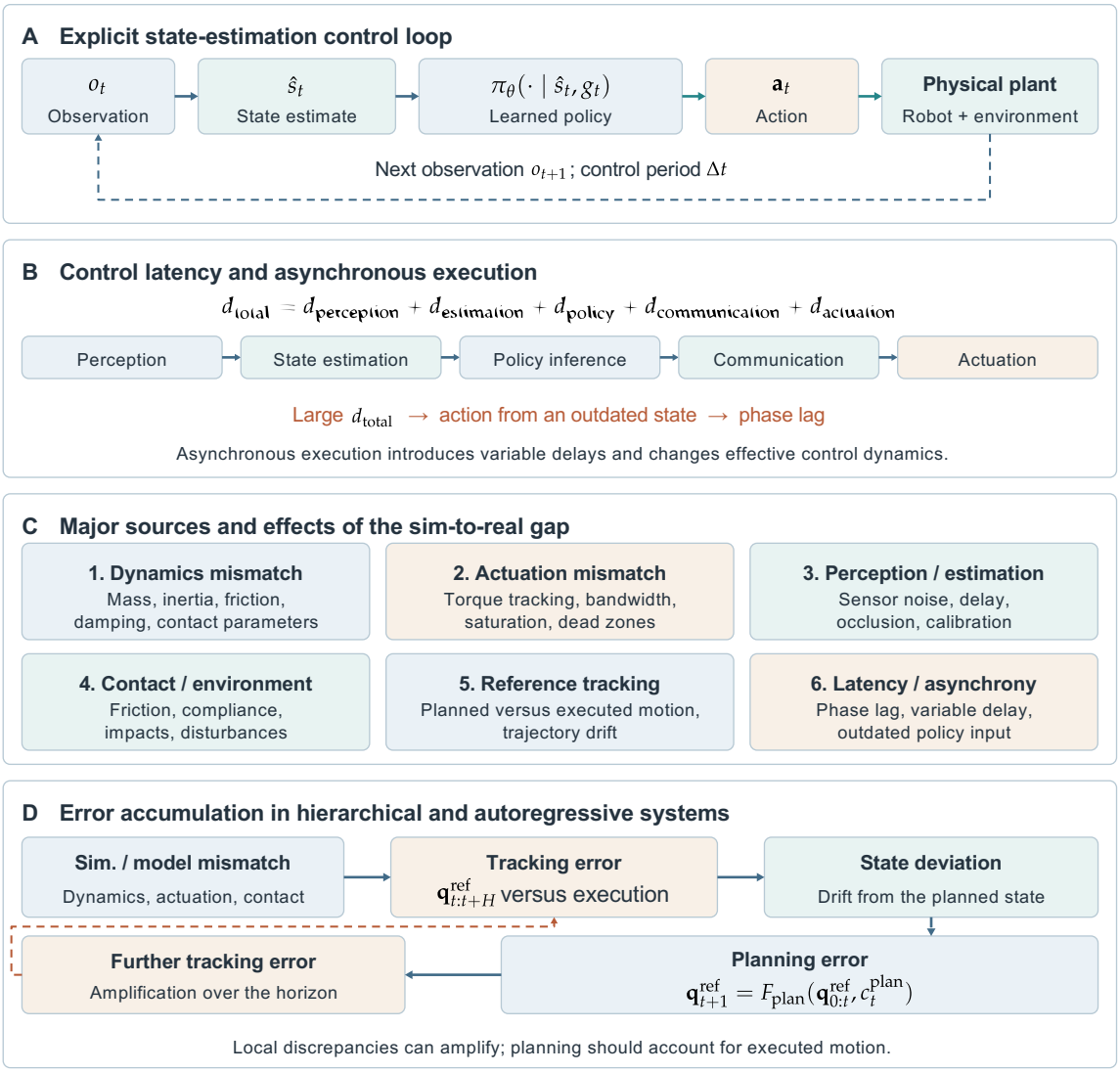


Fig. 7.8 Sources of sim-to-real mismatch and their propagation through a planner-tracker system. The latency sum depicts a sequential processing path. The dashed state-to-planning connection applies when execution feedback conditions the planner

HoST applies L2C2 to the actor and each critic, constructing nearby inputs from differences between successive observed states. This training objective encourages smoother actor and value outputs. In hardware experiments with 5 trials in each of 4 settings (ground, platform, wall, and slope), HoST succeeds in 20/20 trials, compared with 11/20 after removing L2C2 (Huang et al., 2025b). These results connect local function smoothness to the reliability of physical execution under the tested conditions.

Motion amplitude introduces a complementary trade-off. Large actions aid exploration but can produce violent transitions. HoST bounds the policy output and constructs each joint target relative to the current joint position according to

$$\mathbf{q}^{\text{d}}_{j,t} = \mathbf{q}_{j,t} + \alpha_{\text{act}}\mathbf{a}_t, \qquad \mathbf{a}_t \in [-1,1]^n, \tag{7.45}$$

where $\mathbf{q}_{j,t}$ and $\mathbf{q}_{j,t}^{\mathrm{d}}$ are the measured and desired positions of the n actuated joints, and α_{act} scales the target increment in radians. Meeting an episode-end head-height criterion advances an action-scale curriculum that reduces α_{act} from 1 toward 0.25 (Huang et al., 2025b). This bounds the requested displacement at each update and indirectly moderates motion through the PD response. Gradual tightening retains early exploration while encouraging less abrupt execution as the skill develops.

For tracking a reference motion with specified timing, smoothing must also preserve the required movement speed. The objective is to suppress high-frequency noise while preserving the temporal structure and key dynamic characteristics of the original motion. Excessive smoothing may, for example, reduce the speed of a tennis swing or alter the timing of a dynamic motion. Therefore, motion smoothing should be designed jointly with motion phase, robot dynamic capabilities, and task constraints. The planner-tracker decomposition in AdaPT converts generated motions into executable commands through a dedicated tracking controller.

7.5.3 Sim-to-Real Adaptation

The deployment mismatches described above motivate training and adaptation across variations in dynamics, sensing, and latency.

A widely used approach is domain randomization, where physical and environmental parameters are randomized during training by sampling from a distribution

$$\xi_{\mathrm{sim}} \sim p(\xi_{\mathrm{sim}}), \tag{7.46}$$

where ξ_{sim} may include robot mass, friction coefficients, joint damping, actuator strength, observation noise, control latency, and external disturbances. By exposing the policy to a distribution of possible environments, the learned controller can become less sensitive to modeling errors.

Another approach is to calibrate the simulation using real-robot data through system identification, thereby reducing the discrepancy between simulated and physical dynamics. In practice, these approaches can be combined. Real-robot data are first used to estimate the dominant physical parameters, while domain randomization is then applied to cover the remaining uncertainty.

HoST combines coverage of initial support conditions with randomization of physical parameters. Ground, platform, wall, and slope settings expose training to different starting postures and contacts. Within these settings, randomization includes base center-of-mass offsets of up to ± 3 cm and control delays from 0 to 100 ms, alongside friction, actuator strength, and PD gains (Huang et al., 2025b). Hardware deployment also adjusts hip and knee PD stiffness to compensate for differences in torque response. The example shows how broad simulation coverage and servo-gain adjustment contribute jointly to transfer.

For humanoid motion control, an important observation is that a motion that is kinematically feasible in simulation is not necessarily dynamically executable on the real robot. Consequently, the training pipeline should account not only for task-level objectives but also for the actual controller, state estimation process, and execution constraints used during deployment. For systems that generate high-level motion references, the generated motion must ultimately be converted into executable behavior by a tracking controller that accounts for these constraints.

7.5.4 Balance Protection and Fall Recovery

For humanoid robots, a particularly important aspect of safety control is the possibility of losing balance and falling. A complete safety framework should therefore not assume that the robot never falls, but should also provide mechanisms for controlled falling, post-fall recovery, and subsequent task resumption.

Balance protection can be organized into several stages according to the severity of the instability. The first stage is rapid disturbance rejection, such as adjusting foot placement, torso orientation, and motion speed. If the robot can no longer recover its standing state and a validated protective controller is available, the system can enter a controlled fall mode and modify the body configura-

tion to reduce impact. After falling, a dedicated, trained and validated get-up controller can bring the robot back to a stable standing configuration.

Learning such a controller requires objectives that guide the robot through changing support configurations. HoST learns standing-up motion without reference trajectories, using height-dependent reward activation to organize righting, rising, and stable standing (Huang et al., 2025b). Its four reward groups (task, style, regularization, and post-task) describe progress, motion quality, and the stability of the achieved posture. Each group has a separate critic. Its generalized advantage estimates are normalized within the training batch, then combined in a weighted sum to update a shared actor with PPO. This separates return estimation for signals with different scales and temporal structure while learning a policy that coordinates their requirements.

Exploration is especially difficult before the robot discovers how to raise its body into a supported posture. HoST therefore applies an upward force to the base during training when the trunk is nearly vertical. As the episode-end head-height criterion is met, the assistance decreases from 200 N in 20 N steps until it reaches zero. Together with the action-scale curriculum, this lets training first discover the recovery motion and then refine its execution with reduced assistance and smaller target increments. Post-task rewards favor low base linear and angular velocities and the maintenance of body height and posture. Consistently, simulation success requires the base to reach the target height and remain above it for the rest of the episode (Huang et al., 2025b). Recovery thus includes stabilizing the terminal posture.

Executing a learned recovery skill also depends on the available space around the body. HoST reports collisions with the surroundings in some seated-start trials using proprioception alone (Huang et al., 2025b). Such cases motivate environmental checks when selecting and initiating a recovery motion. Incorporating the skill into an autonomous system further requires fall detection, controller handover, and task resumption. The HoST study leaves this integration for future work.

Task recovery should go beyond simply returning the robot to a standing posture. The system should also restore the task context. For example, if the robot loses balance during a long-horizon motion imitation task, the recovery module should determine the current task phase and decide whether to restart the current motion, resume from an intermediate state, or replan a new executable trajectory. Integrating these decisions with balance protection makes recovery part of the task-level control process.

Future humanoid control systems should account for dynamic feasibility while integrating motion generation, real-time control, failure detection, risk mitigation, recovery, and task resumption. These functions need validation within the intended operating conditions.

8 Physical Agents

For a long time, research on the interaction between AI and the physical world has focused primarily on short-term actions and isolated skills, such as executing a single grasp or traversing a short stretch of terrain. Yet a true embodied agent must be able to operate continuously in real environments over extended periods of time. Achieving this requires more than low-level perception, motor skills, and future prediction. Instead, it demands a capable cognitive core that can perform long-horizon planning and adapt as conditions change.

This chapter discusses how to build such an integrated system, which is a physical agent, that can interact with the physical world continuously in a closed loop. To this end, the chapter first defines the scope of physical agents and clarifies how they differ from purely digital agents and conventional robots. It then discusses how physical agents scale to adapt to entity-level, composite, and system-level constraints. Building on this foundation, it explains how physical goals become assessable task specifications, how available skills are organized into feasible behavior, and how execution feedback supports verification and recovery. Finally, it examines collaboration among heterogeneous agents and the emergence of distributed capabilities.

8.1 Definition of Physical Agent

As artificial intelligence extends from digital information spaces into the physical world, the notion of an intelligent agent acquires additional requirements. A digital agent may pursue goals, maintain memory, reason over observations, and invoke software tools, while its actions primarily alter informational states. A physical agent must additionally operate through an embodied system whose actions are constrained by geometry, dynamics, contact, sensing limitations, actuation uncertainty, time, energy, and safety (Russell and Norvig, 2020; Pfeifer and Bongard, 2006; Siciliano and Khatib, 2016). Its decisions can change not only the state of the task, but also its own physical configuration, the surrounding environment, and the observations and actions that become available afterward.

A physical agent should therefore not be understood simply as a robot equipped with an artificial intelligence model. The relevant distinction lies in how the system organizes perception and action around an objective under physical constraints. In this chapter, the term refers to systems that are physically situated, possess some degree of autonomy in selecting their behavior, and continuously use the consequences of physical interaction to guide subsequent decisions. This perspective provides a common basis for discussing agents implemented as mobile robots, manipulators, legged systems, aerial vehicles, autonomous machines, or more distributed physical systems (Brooks, 1991; Liu et al., 2025d).

8.1.1 Concept and Boundary of Physical Agent

A Working Definition In this chapter, *a physical agent is defined as a goal-directed, relatively autonomous unit that perceives physical conditions, selects and executes feasible actions through its embodiment, and uses feedback from their physical effects to adjust subsequent behavior.* Its embodiment determines what it can sense and how it can act, while environmental conditions, resource limits, and assigned authority constrain admissible behavior. Feedback connects intended outcomes to observed changes in the agent and its surroundings, enabling it to continue execution, correct its actions, or revise its plan.

This definition treats physical agency as a relationship among objective, embodiment, environment, action, and feedback. The agent is *situated* because its behavior takes place in an environment governed by spatial and physical relationships rather than solely by symbolic or computational rules. It is *goal-directed* because its behavior is organized around a desired outcome rather than a fully prescribed sequence of actions. It is autonomous or semi-autonomous because at least part of the mapping from a task objective to concrete actions is determined during operation rather than completely specified in advance.

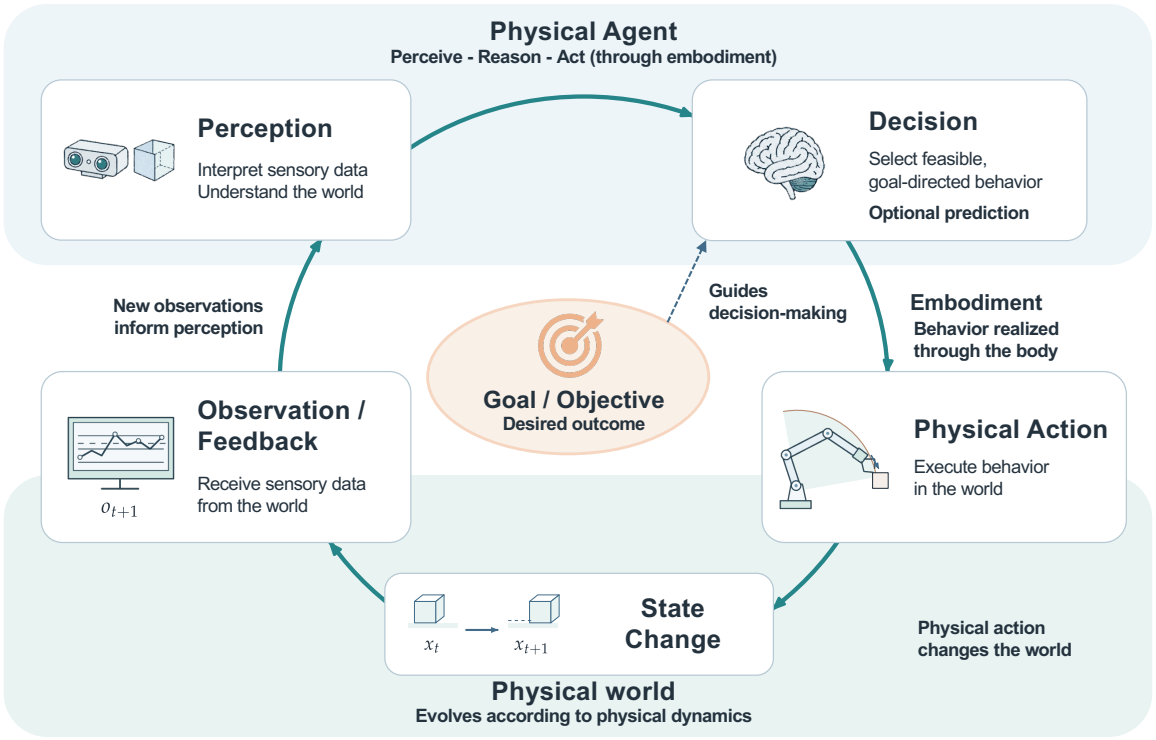


Fig. 8.1 Closed-loop interaction between a physical agent and the physical world. Perception and task-relevant representations inform decision-making, which may include optional prediction and scenario simulation. Decisions are realized through embodied physical actions, and these actions change the physical state of the agent and/or environment. The resulting state gives rise to new observations that inform subsequent decisions. Here x_t is the joint physical state at step t, x_{t+1} its next state, and o_{t+1} the resulting observation

Physical embodiment is essential to this definition. The embodiment includes the system's mechanical structure together with its sensors, actuators, onboard computation, and the interfaces through which it exchanges information and physical influence with the environment. It determines what the agent can observe, which actions are feasible, and what physical consequences those actions may produce. The embodiment is therefore not merely a carrier on which an intelligent algorithm is deployed. It constitutes part of the agent's sensing and action capabilities and thereby defines part of its operational boundary (Pfeifer and Bongard, 2006; Siciliano and Khatib, 2016).

The definition also requires closed-loop interaction (Brooks, 1991; Åström and Murray, 2008). An action produces a physical consequence, that consequence gives rise to new observations, and those observations may alter subsequent behavior. This interaction is illustrated in Figure 8.1.

Perception and action are coupled in this loop. Actions change the world from which future observations are obtained, and new observations change which actions are subsequently selected.

Physical Agents and Digital Agents A physical agent and a digital agent share several fundamental properties. Both may be goal-directed, maintain internal state, use memory, reason over uncertain information, invoke tools, and revise their behavior according to feedback. The distinction therefore does not lie in whether the system performs reasoning or planning.

The more consequential difference concerns the domain in which actions take effect. A digital agent primarily changes informational states, for example by generating text, editing files, querying a

database, executing code, or invoking a software service. These actions may indirectly lead to physical consequences when mediated by an external system, but the agent's immediate action space is digital.

A physical agent acts through a physical embodiment. Its actions may change its position, orientation, velocity, configuration, contact state, or energy state, as well as the location, configuration, or physical condition of external objects. Such actions are subject to feasibility constraints and may produce delayed, uncertain, or irreversible consequences. A manipulator cannot instantaneously move its end effector to an arbitrary pose. An aerial vehicle cannot change velocity without satisfying its dynamics. A mobile robot cannot traverse an occupied region merely because a planner identifies it as geometrically desirable.

The distinction can therefore be expressed in terms of the state directly affected by action. Digital agents act primarily on informational states, whereas a physical agent directly participates in the evolution of physical states. This difference introduces requirements that are less prominent in purely digital settings, including dynamic feasibility, physical safety, execution uncertainty, limited sensing, resource constraints, and the need to verify the actual outcome of an action rather than only whether a command was issued.

Physical Agents and Robots The terms *robot* and *physical agent* also overlap without being synonymous. A robot primarily identifies a class of embodied machines capable of sensing, computation, and actuation. The concept of a physical agent places greater emphasis on how such capabilities are organized around an objective and how much of the resulting behavior is selected during interaction with the environment.

Consider an industrial manipulator that repeatedly follows a predetermined welding trajectory. The system is clearly a robot, and it may employ sophisticated feedback control to achieve high precision. Nevertheless, if the task, sequence, and responses to relevant situations are almost entirely specified in advance, its behavior exhibits relatively little agent-level autonomy. By contrast, a manipulation system that is given a desired outcome, selects an appropriate grasp and motion strategy from the observed scene, detects unsuccessful execution, and chooses an alternative course of action has stronger agentic properties even if the underlying physical platform is similar.

The distinction is therefore one of emphasis rather than a strict separation of system categories. A robot characterizes the physical means through which sensing and action are realized, whereas a physical agent characterizes a mode of operation in which perception, decision-making, physical action, and feedback are organized in pursuit of a goal. Many modern robotic systems satisfy both descriptions, while some robots remain primarily automated machines with narrowly prescribed behavior.

8.1.2 Functional Organization and Interaction Loop

Building on the four elements in Chapter 1 (Section 1.3), this section distinguishes five functional roles by discussing world representation and physical prediction separately. These roles comprise physical state perception and estimation; physical representation and world models; physical prediction and simulation; physical decision-making, planning, control, and execution; and closed-loop feedback and multiscale adaptation. A physical agent realizes this same loop within an identifiable operational boundary. The following account applies those elements to agent-level behavior, without prescribing separate software modules. Their functions may be explicitly implemented, learned jointly, or distributed across computational and physical subsystems (Siciliano and Khatib, 2016; Liu et al., 2025d).

Embodiment, objectives, and operating constraints provide the conditions under which all five functional roles function. The embodiment includes mechanical structure, sensors, actuators, computation, and environment interfaces. It determines which observations and physical actions are available. A wheeled platform, manipulator, and aerial robot may therefore use different strategies for the same objective. Goals specify desired outcomes, while kinematic and dynamic limits, collision avoidance, energy, time, permissions, and safety requirements delimit acceptable behavior.

These conditions guide the entire loop. Memory supports continuity across its elements, and safety mechanisms constrain and monitor their operation.

Physical State Perception and Estimation A physical agent generally lacks direct access to the complete state of the world. Cameras, depth sensors, LiDAR, tactile and force sensors, inertial units, encoders, and other instruments provide incomplete measurements affected by noise, occlusion, calibration errors, and latency (Siciliano and Khatib, 2016). Perception and estimation construct task-relevant descriptions of the body's state and surroundings, including pose, motion, object configurations, contact, free space, or process quantities such as temperature. The required information and its accuracy depend on the decisions and control actions that follow.

Physical Representation and World Models Representations organize estimated information into spatial, semantic, relational, or physical descriptions that support action. Memory retains earlier observations, explored regions, attempted actions, and outcomes, enabling the physical agent to reason beyond its current view. A world model may encode object relationships, affordances, and transition regularities. Such information can be explicit, implicit in a learned policy, or distributed across components. A comprehensive internal simulator is therefore unnecessary. The relevant requirement is a representation adequate for the task and operating conditions.

Physical Prediction and Simulation Prediction uses available representations to anticipate how candidate actions, disturbances, or other participants may change physical conditions. Scenario simulation can compare alternative sequences before committing to execution, using analytical dynamics, learned models, or other approximations. Its usefulness depends on the prediction horizon, uncertainty, and computational cost. This element need not be a separate predictive module. A reactive controller or learned policy may incorporate local expectations implicitly, while explicit prediction supports longer horizons and more complex dependencies. Forecasts remain conditional on their assumptions and must be checked against interaction outcomes.

Physical Decision-Making, Planning, Control, and Execution This element connects estimated conditions and the objective to feasible physical intervention (Russell and Norvig, 2020; Kober et al., 2013). A complex objective can be decomposed into intermediate goals and realized through reusable skills, such as approaching, grasping, transporting, or inspecting. Planning coordinates these skills and their dependencies. Control converts the selected behavior into trajectories, velocities, forces, torques, or actuator commands compatible with the embodiment. An instruction to grasp a container may thus require approach motion, alignment, contact establishment, closure, and force regulation. Modular architectures may expose these stages individually, while learned policies may combine them. In either case, action selection remains conditioned on the task, current information, and admissible operating conditions.

Closed-Loop Feedback and Multiscale Adaptation Execution changes the physical state and produces new evidence. Feedback compares intended and observed outcomes and informs corrections at different scales (Åström and Murray, 2008). A controller can regulate contact force, a skill can adjust its motion, and a task planner can replace an unsuccessful operation. Over repeated operation, calibration and retained experience can support adaptation to equipment drift or changing conditions. Adaptation may update estimates, representations, predictions, or behavior without necessarily changing learned parameters online. Safety mechanisms operate throughout the loop by restricting candidate actions, monitoring execution, and triggering recovery, fallback, or intervention when limits are approached. Feedback therefore connects local regulation, task-level revision, and longer-term adaptation, sustaining the closed-loop organization described in Chapter 1 and its application loop (Section 1.5.1).

A Formal View of Physical Interaction The interaction of a physical agent with its environment can be represented as a partially observable closed-loop decision process (Kaelbling et al., 1998). Let t be a nonnegative integer indexing interaction steps, B_{emb} describe the embodiment and its sensing

and actuation capabilities, and $\mathcal{X}$ be the space of joint physical states of the body and environment. Then

$$x_t \in \mathcal{X} \tag{8.1}$$

denotes the physical state at step t. The physical agent receives an observation o_t in an observation space $\mathcal{O}$ according to

$$o_t \sim p(o_t \mid x_t, B_{\text{emb}}), \tag{8.2}$$

where $p(o_t \mid x_t, B_{\text{emb}})$ is the conditional observation distribution and $\sim$ denotes sampling from that distribution. It accounts for the sensing capabilities of the embodiment and measurement uncertainty.

Let m_t^{mem} denote retained information from earlier interactions, g the task objective, c_t^{constr} the currently relevant constraints, and $o_{\leq t} = (o_0, \ldots, o_t)$ the observation history through step t. The policy π is a conditional action distribution that selects an action a_t from the embodiment's action space $\mathcal{A}_{B_{\text{emb}}}$, subject to c_t^{constr}:

$$a_t \sim \pi\left(a_t \mid o_{\leq t}, m_t^{\text{mem}}, g, c_t^{\text{constr}}, B_{\text{emb}}\right). \tag{8.3}$$

The physical state then evolves according to

$$x_{t+1} \sim p(x_{t+1} \mid x_t, a_t, B_{\text{emb}}), \tag{8.4}$$

where $p(x_{t+1} \mid x_t, a_t, B_{\text{emb}})$ is the conditional state-transition distribution, including environmental uncertainty and the effects of embodied actuation. The notation p denotes the appropriate conditional distribution in each equation. Deterministic sensing, policies, or dynamics are special cases of these distributions.

Eqs. (8.2) to (8.4) specify the observation, action-selection, and physical-transition relationships, rather than a particular control or learning architecture. Estimation, representation, and any explicit prediction support the policy. Feedback supplies subsequent observations and updates. The resulting interaction is

$$x_t \to o_t \to a_t \to x_{t+1} \to o_{t+1}, \tag{8.5}$$

where arrows indicate the causal and informational progression of one cycle. Actions affect both the physical state and the evidence available for later decisions, coupling sensing and acting throughout the loop.

8.1.3 Autonomy and the Transition from Automation to Agency

Autonomy as a Continuum The distinction between a physical agent and a conventional automated system is not binary. Physical systems exhibit different degrees of autonomy, and many useful machines occupy intermediate positions between fixed automation and general goal-directed agency (Parasuraman et al., 2000).

In fixed automation, task structure and action sequences are largely specified before deployment. A production system may repeatedly execute the same sequence with high precision, while conventional feedback control compensates for deviations in variables such as position, velocity, force, or temperature. Such feedback is essential to reliable physical operation, but local error correction alone does not imply that the system selects its behavior at the task level.

As sensing, computation, and adaptive control become more capable, physical systems can respond to a wider range of environmental variations. A mobile robot may choose a different route when an aisle is blocked, and a manipulator may adjust its motion when an object is displaced. In such cases, some aspects of behavior are determined online, although the range of possible tasks and responses may remain strongly bounded by predefined representations and skills.

A more agentic system shifts the specification from actions toward outcomes. Rather than being provided with a complete trajectory or procedure, it may receive a target state or higher-level objective and determine an appropriate sequence of actions from the current situation. The progression can be represented schematically as

$$\text{Fixed Automation} \to \text{Feedback Control} \to \text{Adaptive Robotics} \to \text{Goal-Directed Agency}. \tag{8.6}$$

This sequence should be interpreted as an organizing continuum rather than a strict taxonomy. Real systems may combine characteristics from several stages and may exhibit different levels of autonomy in different parts of the same task.

Goal-Conditioned Action Selection A central feature of agency is that the same objective can lead to different actions depending on the current physical situation. If a direct route becomes obstructed, a mobile agent may select an alternative path. If an intended grasp is infeasible, a manipulation system may choose another grasp configuration or reposition itself before trying again. The objective remains unchanged while the means of achieving it are revised.

This distinguishes goal-conditioned action selection from a fixed input-output response. The agent's action depends jointly on the current state, the desired outcome, the available capabilities, and the constraints under which it operates.

Adaptation Under Uncertainty Physical environments are only partially observable and are rarely perfectly predictable. Objects may move, materials may behave differently from expected models, sensing may degrade, and the effect of an action may differ from its nominal prediction. A physical agent must therefore operate despite uncertainty about both the current state and the consequences of its own actions (Kober et al., 2013; Brunke et al., 2022).

Adaptation in this context does not necessarily imply online learning of model parameters. It more generally refers to the ability to revise behavior as new evidence becomes available. The agent may update a state estimate, change a planned path, select another skill, reduce speed, request additional observations, or transfer control to a human operator. These mechanisms differ in implementation but serve the same role by preventing execution from being tied irreversibly to assumptions made before interaction.

Outcome-Driven Correction and Recovery An important consequence of closed-loop autonomy is that issuing an action is not equivalent to completing it. Physical execution must be assessed in terms of outcomes. A command to close a door is not successful merely because the corresponding motor command was sent. The agent must establish, to the extent required by the task, that the resulting physical state satisfies the intended condition.

When execution does not produce the intended outcome, an agent may need to distinguish among transient error, action failure, environmental change, or an infeasible objective. Recovery can then involve repeating the action, selecting an alternative action, revising the plan, modifying the objective within permitted limits, or requesting external intervention. The ability to make such distinctions becomes increasingly important as tasks extend over longer horizons and involve larger numbers of dependent actions.

Autonomy Does Not Imply Unrestricted Operation Autonomy should not be equated with the absence of human control. In practical physical systems, autonomous behavior is typically bounded by authority, safety, and operational constraints. A robot may plan its own route while requiring authorization for a safety-critical manipulation, or it may execute a task independently while requesting human assistance when uncertainty exceeds an acceptable threshold.

For this reason, a physical agent is more appropriately described as an autonomous or semi-autonomous system than as a necessarily fully autonomous machine. Its autonomy concerns the degree to which action selection and adaptation are delegated to the system within a specified operating envelope.

Autonomy should also not be treated as synonymous with intelligence. A narrowly specialized machine may operate with little human intervention while possessing limited ability to generalize beyond a predefined task. Conversely, a system with sophisticated perception, reasoning, or learning capabilities may intentionally operate under substantial human supervision. The development of Physical AI should therefore be considered along multiple dimensions, including autonomy, task

diversity, environmental openness, uncertainty, temporal horizon, and the complexity of adaptive behavior.

From this perspective, the transition from automation to physical agency is characterized less by the addition of any single algorithm than by a change in how behavior is organized. The system moves from executing actions specified in advance toward selecting and revising actions in response to goals, physical constraints, and interaction outcomes. This concept provides the foundation for the subsequent discussion of goal representation, perception, world modeling, planning, learning, control, and other capabilities required for a physical agent.

8.2 Physical Forms of Physical Agent

A physical agent can take different forms, distinguished by how its body provides access to the world. A manipulator changes an object's pose through contact, a mobile robot changes position and viewpoint, and a laboratory installation transforms and measures samples. These differences arise from both bodily morphology and the arrangement of sensing, actuation, computation, and resources. Accordingly, *physical form* denotes the organization through which an agent acts, including systems whose action interfaces are distributed across several devices. Situated robotics and embodied intelligence provide the conceptual foundations for relating behavior to the coupling of controller, body, and environment (Brooks, 1991; Pfeifer et al., 2007).

Under the working definition adopted in this chapter, a physical agent is a relatively autonomous, goal-directed unit that perceives physical conditions, selects feasible actions, changes those conditions, and adapts through feedback. A robot is one possible realization of its body. Agency is a property of the complete operational loop, including the executive and its authority over physical interfaces. Software that recommends an action remains a digital agent at that boundary. Attributing physical agency to a software-equipment installation requires including the equipment that executes the action and the measurements used to evaluate it.

Figure 8.2 introduces three complementary views. An *entity-level* view treats a coherent acting body as one unit. A *composite* view resolves functionally separable action subsystems that fulfill a joint mission. A *system-level* view encompasses distributed resources coordinated across an operational domain. This is the chapter's analytical framework, rather than a standardized taxonomy or an autonomy ranking. For example, a mobile manipulator can be one entity in a transport study, a

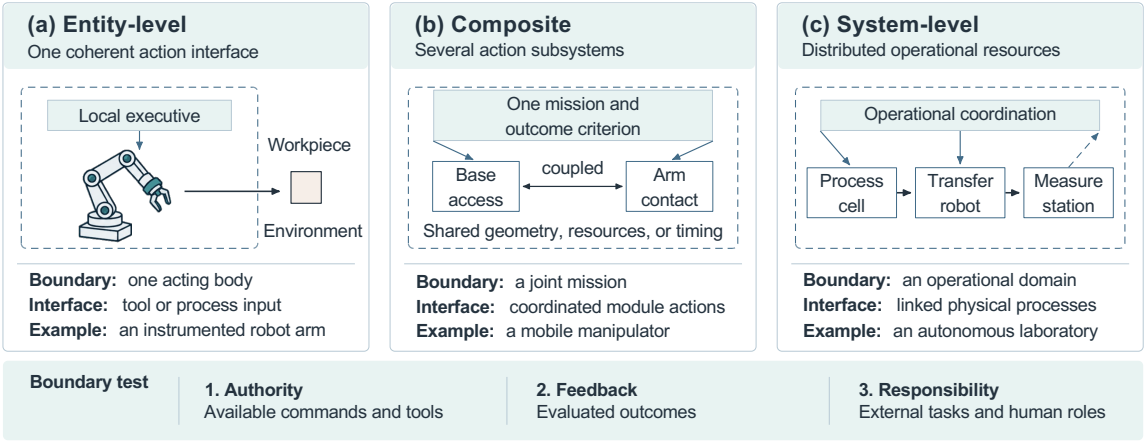


Fig. 8.2 Physical forms distinguished by the boundary of attribution, the organization of action, and the scope of the task. Dashed outlines delimit the selected agent. Solid dark arrows denote physical action or flow, green arrows commands, and dashed blue arrows information. The three views can be nested. Original analytical synthesis informed by Pfeifer et al. (2007); Yim et al. (2007); Monostori et al. (2016)

base-arm composite in a manipulation study, and a resource within an autonomous laboratory. The chosen view should follow the capability being explained.

Three criteria make that choice explicit, namely command authority, observable outcomes, and the allocation of responsibility to people or other systems. They distinguish an operational boundary from a building perimeter or a network membership list. Task objects usually remain part of the environment, even while grasped. Materials and mechanisms can themselves support adaptive behavior, as emphasized in physical intelligence research (Sitti, 2021). The working definition here additionally requires a goal-directed action loop. With this boundary established, physical forms can be examined through the interfaces and constraints that make particular actions possible.

8.2.1 Entity-Level and Composite Forms of Physical Agent

Entity-Level Forms and Their Action Interfaces An entity-level physical agent has a coherent body and an identifiable action interface. Mobility, contact geometry, compliance, workspace, payload, and sensing arrangement describe how that body interacts with its surroundings. Figure 8.3 pairs representative morphologies with the physical variables through which they act.

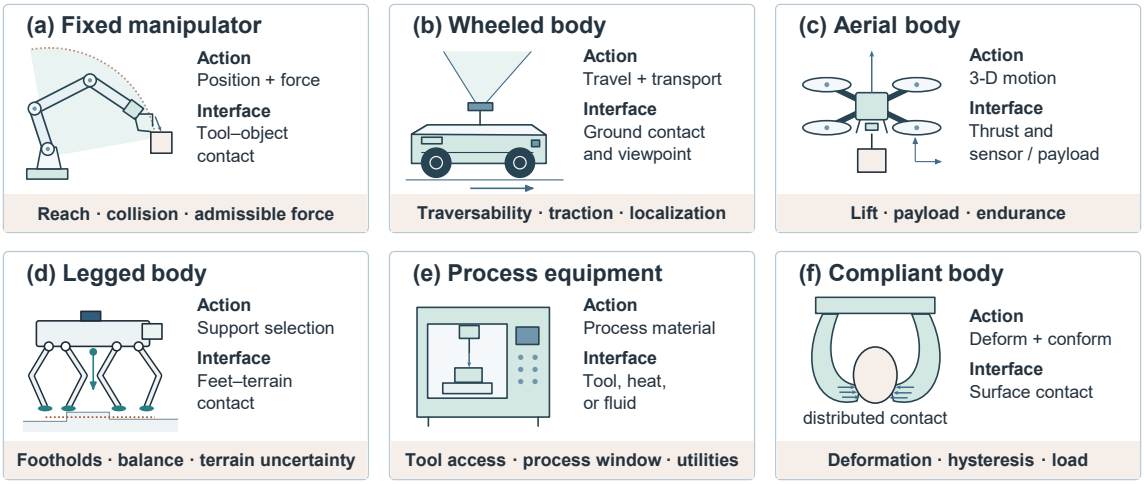


Fig. 8.3 Morphology, action interface, and limiting conditions for six representative forms. Shaded reach and sensing regions are qualitative. Arrows indicate motion or interaction directions. The compliant gripper contrasts nominal straight fingers with deformed contact. These original schematics describe physical affordances, not the autonomy of a particular product

Fixed Manipulators An arm controls end-effector pose and interaction forces within a bounded workspace. Operational-space control formalizes the relationship between joint actuation and end-effector motion or force (Khatib, 1987). Geometric reach is only one requirement. Tool shape, friction, collision clearance, and admissible contact loads also determine whether an operation is feasible. Pairing two arms adds complementary contact roles, such as stabilizing and inserting. ALOHA demonstrates learned fine manipulation using such a bimanual arrangement and human demonstrations (Zhao et al., 2023). At the installation scale the pair can be treated as one manipulation unit. A composite view makes its coordination requirements explicit.

Ground-Mobile Robots Wheeled bodies provide efficient transport where supporting surfaces are traversable. Legged bodies can select footholds across uneven terrain, with balance and contact transitions forming part of the action problem. Miki et al. (2022) integrate terrain information with proprioception for quadrupedal locomotion under unreliable perception. Terrain estimates support action through the footholds and dynamics available to the body. Humanoids extend this family

with human-like proportions and manipulation interfaces. Resemblance to people is not a criterion for physical agency.

Aerial Robots Flight provides three-dimensional access and changes the sensor's viewpoint without continuous ground support. This capability couples navigation to lift, attitude stabilization, obstacle clearance, and energy availability. Loquercio et al. (2021) demonstrate high-speed autonomous flight using onboard sensing and computation. Inspection and delivery build on aerial access but additionally depend on task-specific sensors, payload interfaces, and operating conditions.

Process Equipment and Compliant Bodies Stationary equipment can change geometry, temperature, pressure, or composition through tools and process inputs. Cyber-physical manufacturing architectures connect such equipment to state assessment and adaptive responses (Lee et al., 2015). Soft structures, meanwhile, support conformal interaction by distributing deformation over a body (Rus and Tolley, 2015). Passive dynamic walking offers a complementary demonstration of mechanics contributing to organized motion (McGeer, 1990).

Composite Forms: Coordinated Action Across Modules A composite physical agent combines functionally separable action subsystems under one mission and outcome criterion. Typical combinations join locomotion with manipulation, ground transport with flight, or sample handling with experimental processing. The analytical emphasis lies on the interfaces between these capabilities. Mechanical attachment is one coupling mechanism. Shared timing, resources, and responsibility can also bind modules that separate in space.

The schematic in Figure 8.4 uses x_B and z_B for base-frame axes. For frame labels F and H, the homogeneous transform ${}^{F}\mathbf{T}_{H}$ maps coordinates from H to F, so ${}^{W}\mathbf{T}_{E} = {}^{W}\mathbf{T}_{B}(\mathbf{q}_B)\,{}^{B}\mathbf{T}_{E}(\mathbf{q}_A)$ composes the base and arm transformations. In the routing example, L, D, and R label launch, delivery, and recovery sites. The arrival times t_C, t_D and nonnegative waiting times w_C, w_D belong to the carrier

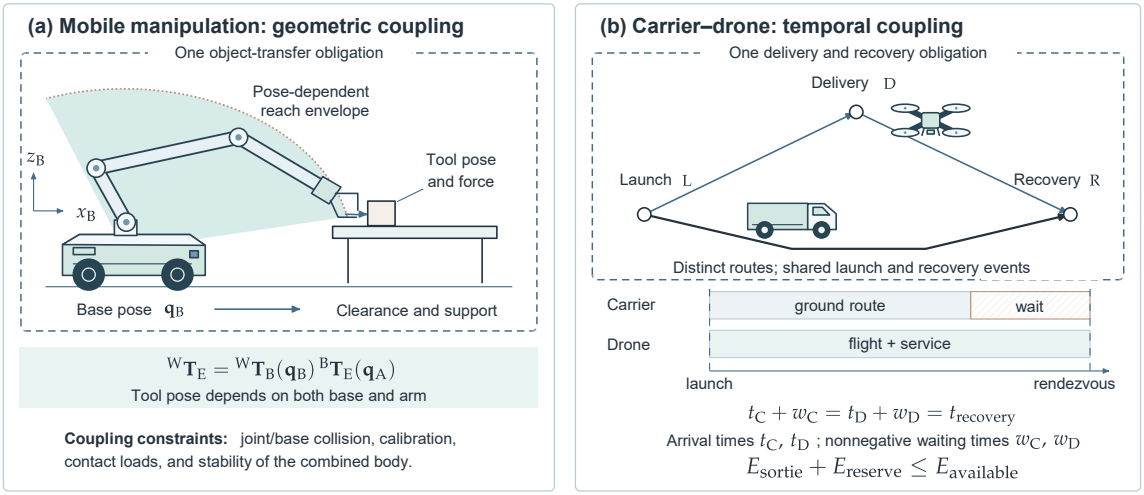


Fig. 8.4 Two forms of inter-module dependence. (a) Base configuration $\mathbf{q}_B$ and arm configuration $\mathbf{q}_A$ jointly determine tool pose. W, B, and E denote world, base, and end-effector frames. (b) Separate routes require compatible recovery times and sufficient flight energy. Symbols within the schematic are defined in the accompanying text. The reach boundary and waiting timeline are qualitative. Original schematics informed by mobile manipulation (Fu et al., 2025) and truck-drone routing (Murray and Chu, 2015)

and drone, respectively. t_{recovery} is their common recovery time. The terms E_{sortie}, E_{reserve}, and $E_{\text{available}}$ denote required sortie energy, required reserve, and available drone energy.

Mobile Manipulation Base motion extends access beyond a fixed arm's workspace while changing its reference frame, sensing viewpoint, and collision envelope. The pose relation in Figure 8.4(a) expresses this geometric dependence. Contact forces and stability introduce additional dynamic constraints. Mobile ALOHA combines a mobile base, two arms, and whole-body demonstration collection to learn coordinated tasks, including cabinet operation and use of a kitchen faucet (Fu et al., 2025). Locomotion establishes access to a work location, while manipulation determines where the base can operate. The cited study evaluates learned execution after demonstration collection, so its teleoperation apparatus and its autonomous policy serve distinct roles.

Vehicle-Drone Systems A carrier can move a drone between launch sites while the drone serves locations through complementary aerial routes. The flying sidekick traveling salesman problem models the resulting route, endurance, and rendezvous dependencies (Murray and Chu, 2015). Figure 8.4(b) makes one consequence visible. Arrival at the recovery site must be synchronized, potentially through waiting. This is an optimization formulation, with human driving of the truck retained in the original problem setting. A composite interpretation becomes useful when one mission executive is responsible for both delivery and recovery. Separate vehicles with independently assigned goals invite a multi-agent interpretation.

Multi-Device Experimental Platforms Dispensing, processing, transfer, and measurement modules can form a composite when they execute a bounded protocol. Burger et al. (2020) connect laboratory stations through a dexterous mobile robot in a photocatalyst search. Transporting one vial can be attributed to the robot, whereas conducting the experimental search also requires the stations and experiment-selection software. The example shows how changing the attributed outcome changes the necessary physical boundary.

Composite form can itself be variable. Modular self-reconfigurable robots alter the connectivity and arrangement of their modules (Yim et al., 2007). Self-modeling robots can revise their behavior after bodily damage (Bongard et al., 2006). Capability consequently depends on current connectivity, calibration, module condition, and coordinated skills. This dependence becomes still more prominent when the acting resources form a persistent, distributed installation.

8.2.2 System-Level Physical Agents

A Distributed Body with an Operational Boundary A system-level physical agent acts through distributed sensors, robots, process equipment, computing nodes, and control networks. Its operational unity comes from an objective and the coordinated use of resources across a domain. Physical distribution and controller organization are separate design choices. A distributed body may use centralized, distributed, or hybrid coordination. Holonic manufacturing provides an early account of autonomous units participating in a larger production organization (Van Brussel et al., 1998). Cyber-physical production connects computational coordination with physical processes (Monostori et al., 2016).

Two related graphs clarify the organization of such a body:

$$\mathcal{G}_{\text{phys}} = (V_{\text{phys}}, E_{\text{phys}}), \qquad \mathcal{G}_{\text{info}} = (V_{\text{phys}} \cup V_{\text{comp}}, E_{\text{info}}), \tag{8.7}$$

where $\mathcal{G}_{\text{phys}}$ and $\mathcal{G}_{\text{info}}$ are the physical-dependency and information-exchange graphs. The resource vertex sets V_{phys} and V_{comp} contain sensing/acting and logical computation resources, respectively. Computation may be hosted on the same devices. The edge set E_{phys} encodes mechanical, material-transfer, or process dependencies. The edge set E_{info} encodes sensing, coordination, or command exchange. The union $V_{\text{phys}} \cup V_{\text{comp}}$ comprises both resource types. Figure 8.5 illustrates the organization of a system-level physical agent. This descriptive model distinguishes the existence of a communication path from the existence of an executable physical operation. For example, rerouting a part requires a compatible transport path as well as a revised assignment.

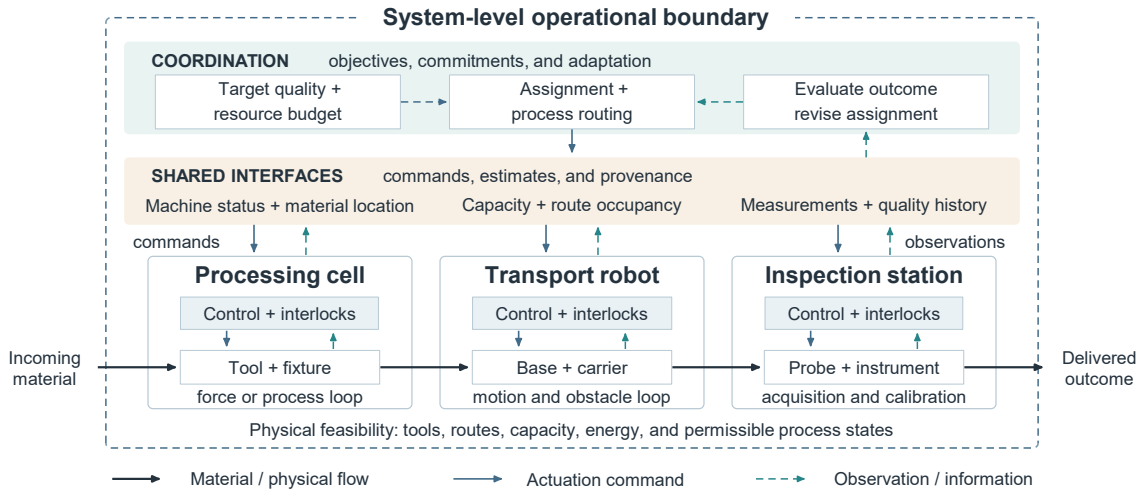


Fig. 8.5 Physical and informational organization of a system-level physical agent. Coordination functions operate through shared interfaces, while individual resources retain local feedback and interlocks. Commands, observations, and material flows are distinguished explicitly. The layers describe functional responsibilities and do not prescribe a centralized network architecture. Original synthesis based on holonic and cyber-physical manufacturing (Van Brussel et al., 1998; Lee et al., 2015; Monostori et al., 2016)

Attribution of agency at this scale requires an actionable system objective, authority over the necessary equipment, and measured outcomes that change subsequent interventions. A production objective, for example, becomes operational through coordinated transformation, transport, and quality evaluation. Sensor aggregation alone supplies only part of that loop. Similarly, a fixed command sequence can automate execution while leaving adaptation to an operator. The relevant distinction concerns who selects and revises physical actions when conditions change.

The operational boundary also spans different timescales. Fast contact and motion loops coexist with slower production and experiment cycles. Local stabilization and protective responses must remain effective within the coordination architecture. Human supervisors can set objectives, replenish resources, and authorize exceptional operations, provided these responsibilities are stated. A system-level physical agent can therefore contain local agents and depend on specified human support. What justifies the higher-level description is the additional feedback loop that coordinates resources to achieve an installation-wide outcome. Smart factories, intelligent mines, and autonomous laboratories are representative realizations of a system-level physical agent.

Protocols and Standards for a Physical Agent For a system-level physical agent, interoperability determines whether independently developed resources can participate in a common course of action. Robots, instruments, and process equipment need shared ways to describe their capabilities, accept delegated work, and report what occurred. Protocols specify these exchanges, while standards establish common information models and requirements against which implementations can be checked. Together, they make the relationships between reasoning, coordination, and physical execution explicit enough to support integration across manufacturers and applications.

A useful organization separates objective formation and revision, task coordination, and local execution. Protocols connect these responsibilities by carrying capability descriptions, task requests, and execution feedback. This separation allows planning methods and equipment implementations to evolve while preserving agreed interfaces. The Model Context Protocol (MCP), for example, specifies how AI applications discover and invoke tools with structured input and output descriptions (Model Context Protocol, 2025). Applying such interfaces to physical resources re-

quires their descriptions and results to carry the operational meaning needed by the task, including which objects are affected, which conditions permit execution, and what observations establish completion. Anthropic's Model Hardware Standard (MHS), introduced as a research preview in collaboration with HHMI Janelia Research Campus in 2026, addresses the discovery and operation of programmable physical devices.[8] MHS combines standardized drivers with descriptions of device characteristics and operating limits, and supports access through MCP, command-line interfaces, and code. It thus provides a concrete example of connecting general agent interfaces to laboratory and manufacturing equipment.

The central abstraction is the capability that a resource makes available to the wider system. This may be a measurement, a material-processing operation, a robotic skill, or an established procedure involving several devices. A common description helps an agent determine what can be requested and what information it can expect in return. The Web of Things Thing Description provides one foundation for describing interactions with heterogeneous devices (W3C, 2023). At the system level, this suggests a way to organize access around useful functions while retaining the specialized knowledge and control methods within each resource. The same planning system could then work with different combinations of equipment through compatible descriptions.

An autonomous laboratory illustrates how this organization connects reasoning with diverse physical capabilities. In an illustrative materials study, a scientific agent might propose candidate compositions and the properties to be measured. A laboratory coordination system would relate this plan to available preparation, processing, and characterization capabilities. Dedicated instruments would perform established procedures, while robots could contribute manipulation or transport where needed. The protocol would connect these contributions to the same experimental objective and return their results in an interpretable form. The OPC UA Laboratory and Analytical Device Standard supports this kind of integration through common models for device functions, program execution, and results (OPC Foundation, 2023). Here OPC UA denotes OPC Unified Architecture.

The significance of this connection extends to what the agent can learn from an intervention. In the laboratory example, measurements become useful feedback when they remain associated with the candidate material, preparation history, and conditions under which they were obtained. The agent can then compare outcomes, revise an explanation, or select a subsequent experiment. Similar relationships arise when a production system uses inspection results to adjust processing conditions, or an environmental monitoring system directs further measurements toward an uncertain region. Across these settings, a protocol should preserve the context that makes returned observations relevant to the objective. This connects access to physical resources with the continuing development of the agent's understanding.

Shared descriptions also create opportunities for reuse across tasks and facilities. A preparation-and-measurement procedure could be offered as a composite capability for later studies, while its constituent functions remain available for other combinations. Another facility could contribute a complementary measurement or a compatible processing operation. Industrial asset representations such as the Asset Administration Shell provide standardized structures for accessing asset information and services (IEC, 2023). Research linking these representations with capability models and OPC UA communication explores how such descriptions can support reconfigurable systems (Justmann et al., 2024). Reuse still depends on whether the available methods, materials, and operating conditions fit the new task.

From this perspective, protocols and standards provide shared infrastructure through which physical capabilities can accumulate and be used by different agents. A common core can organize capability discovery, task interaction, and contextualized feedback, while domain-specific descriptions express the knowledge needed in manufacturing, laboratories, or other settings. As models, instruments, and procedures improve, compatible interfaces could allow those advances to benefit multiple workflows. The broader opportunity is to connect independently developed intelligence and physical resources into systems that can pursue objectives, evaluate their interventions, and

[8] Anthropic, *Previewing the Model Hardware Standard*, research preview, August 27, 2026.

draw on a growing range of capabilities. Autonomous laboratories offer a concrete setting in which to examine these relationships.

8.2.3 Correspondence Between Form, Capability, and Application

Reaching an object, exerting a controlled force, and checking the result require different physical provisions. A long reach does not establish that an operation can be performed at every reachable position. A large payload does not establish precise control of contact. Connecting form to application therefore requires examining how the body produces the actions a task needs, and under what conditions it can produce them reliably.

Geometry and Motion Link lengths, joint arrangements, and movement limits determine which positions and orientations a mechanism can reach. Its current posture also matters. Joint motion can produce end-effector motion more readily in some directions than in others, and this relationship changes as the mechanism moves. This posture dependence is captured by manipulability measures (Yoshikawa, 1985). For capability assessment, the implication is that access to a position and freedom to act there should be examined separately. The relevant description includes the required working posture and motion direction, alongside collision clearance and load conditions. Manipulability supplies one local kinematic measure. The other requirements need their own checks.

Contact and Material Response Surface geometry, friction, and compliance affect how a body establishes contact and transmits force. Compliance allows a body to deform under contact, sometimes accommodating differences in object shape through its mechanics. A deformable gripping interface, for instance, can conform to an object before stiffening to hold it. Its effectiveness still depends on obtaining suitable contact (Brown et al., 2010). Changing the object or contact conditions can therefore change the result even when the controller remains the same. Thus, describing a body as compliant or adaptable should be accompanied by the object properties and loads for which that behavior is useful.

Sensing Geometry The placement and mobility of sensors determine which information the body can acquire. A camera may have adequate resolution while lacking a useful view of the interaction. Moving the sensor or the body can expose an occluded region. Active perception explicitly connects sensing to such purposeful actions (Bajcsy et al., 2018). Physical form consequently affects both the execution of an action and the possibility of obtaining the feedback needed to guide it. Relevant capabilities include reaching an informative viewpoint, maintaining visibility during movement, and accessing the measurements required to verify the result.

Relating Task Requirements to Physical Arrangements These mechanisms give the form-capability-task-constraint relationship a practical meaning. *Form* specifies the available physical means, *capability* describes the actions and performance those means support, *task* specifies the required effect, and *constraints* establish the conditions that must hold during execution. A useful comparison starts with requirements such as a working region, permissible contact force, positional tolerance, or completion time, and then examines whether a proposed arrangement can satisfy them together.

Table 8.1 organizes this comparison into five dimensions. It is an analytical synthesis of the mechanisms and performance-assessment approaches discussed here, rather than a standardized classification. The task determines which dimensions and criteria need evaluation.

The requirements interact. Extending a mechanism's reach can alter the actuator effort needed for a motion. Adding mobility changes the relationship between the working tool, its support, and the surrounding space. This interaction motivates the joint design of morphology and control. Research on morphology-control co-design makes this relationship explicit by optimizing mechanical dimensions and actuator placement together with motion trajectories and forces, subject to physical and task constraints (Ha et al., 2018). This connects design choices to the behavior needed

Table 8.1 Connecting task requirements to physical form and evidence of capability. The dimensions provide a framework for comparison, not a single score of intelligence

Task requirement	Physical properties to examine	Evidence of a suitable match
Spatial access and motion	Dimensions, degrees of freedom, workspace, support and clearance	Required positions and orientations can be reached and the intended motion can be executed
Physical action and load	Tool or process interface, actuation, stiffness, friction and contact geometry	Required loads or process conditions are maintained within the specified tolerances
Observation and verification	Sensor placement, field of view, measurement access and calibration	Task-relevant states and outcomes can be measured with sufficient accuracy
Sustained operation	Energy supply, thermal limits, replenishment and available capacity	Performance can be maintained for the required duration or rate of work
Handover between operations	Physical connections, transfer paths, interface compatibility and shared resources	The output of one operation meets the input conditions of the next

for a specified task. It also explains why improvements in one hardware specification need to be evaluated in the context of the complete operation.

For a composite or system-level physical agent, the same reasoning extends to the physical connections between resources. Cyber-physical production research situates computational coordination within ongoing physical processes (Monostori et al., 2016). Applying that perspective here, compatible connections between local operations are a prerequisite for system-level capability. A transfer must deliver an object in a state and location that the receiving resource can accept, and shared equipment must be available when required. These conditions must be combined with effective coordination and feedback to achieve the overall task. Communication enables coordination, while physical interfaces and material routes determine which coordinated operations can actually occur. The relevant scale of embodiment follows these dependencies and may take the form of one coherent body, several coupled modules, or a distributed arrangement of resources.

Comparing Forms Through Demonstrated Performance Capability descriptions should distinguish the physical means available, the performance of the implemented agent, and the consistency of that performance. End-effector benchmarking, for example, measures grasp strength, cycle time, and repeatability separately because they answer different questions about application suitability (Falco et al., 2020). At the task level, the relevant evidence may include success rate, completion time, error, and required human intervention. Hardware specifications and task results consequently provide complementary evidence when comparing forms.

The conditions of comparison must also be explicit. The YCB benchmarking framework records the task, environment and object states, hardware, prior information, procedure, and execution constraints (Calli et al., 2015). Following this approach, alternative embodiments should be compared against stated outcome criteria and documented conditions, while allowing each an appropriate means of execution. Repeated trials establish how consistently the implemented system meets those criteria. They also make clear the operating range within which a capability has been demonstrated, rather than assuming that one successful demonstration establishes performance across settings.

The correspondence between form and application is therefore established through a traceable connection. Task requirements identify the necessary physical interactions, bodily organization makes those interactions possible, and evaluation establishes the conditions under which they are achieved. This provides a basis for selecting among entity-level, composite, and system-level forms, and for expressing the physical goals developed in the next section.

8.3 Physical Goal Representation and Task Grounding

A physical agent realizes intended changes in the world through actions constrained by its embodiment and environment. A request such as "store the sample safely" must therefore identify an acceptable outcome, admissible interactions, and evidence of completion. This connection applies both to individual robots and to system-level implementations of a physical agent acting through machines, instruments, and control interfaces.

Task realization involves specifying the required outcome, organizing available capabilities to achieve it, and using feedback to assess and revise execution. These functions are examined through the instruction "Place the sealed sample container in cabinet B, keep it upright, and close the cabinet." The example connects goal semantics and task-relevant state representations to feasible behavior and observed results.

8.3.1 Physical Goal Representation and Semantic Grounding

From an Instruction to a Physical Specification A goal specifies a desired outcome. A plan describes how available actions may achieve it. The container-storage goal should remain meaningful whether executed by a mobile manipulator or by a fixed manipulator supplied through a conveyor. Separating outcomes from procedures supports alternative plans and transfer across embodiments.

For exposition, a physical task can be summarized as

$$\mathcal{T}_{\text{task}} = \left(\mathcal{G}_{\text{goal}}, \mathcal{C}_{\text{exec}}, \mathcal{P}_{\text{pref}}, \mathcal{V}_{\text{verify}}\right), \tag{8.8}$$

where $\mathcal{T}_{\text{task}}$ is the task specification. Here $\mathcal{G}_{\text{goal}}$ specifies acceptable outcomes, $\mathcal{C}_{\text{exec}}$ constrains execution, $\mathcal{P}_{\text{pref}}$ expresses preferences among acceptable solutions, and $\mathcal{V}_{\text{verify}}$ specifies how outcomes and constraints are assessed. This is an organizing abstraction rather than a prescribed data structure. Reaching the storage region is an outcome. Preserving the seal is a constraint. Minimizing travel time is a preference. Checking placement and closure belongs to verification. Timing and resource requirements may be constraints or preferences, depending on the application.

Let x_t denote the joint physical state of the physical agent and environment at decision index t. In this example, c_{cont} labels the container and B_{cab} the cabinet. An illustrative Boolean terminal-goal predicate g is

$$\begin{aligned} g(x_T) = \ & \text{Inside}(c_{\text{cont}}, B_{\text{cab}}) \wedge \text{Supported}(c_{\text{cont}}) \wedge \text{Released}(c_{\text{cont}}) \\ & \wedge \text{Upright}(c_{\text{cont}}) \wedge \text{Closed}(B_{\text{cab}}), \end{aligned} \tag{8.9}$$

where T is the terminal decision index, x_T the corresponding physical state, and $\wedge$ logical conjunction. Evaluated on x_T, the predicates require the container to be inside the intended storage region, physically supported, released by the manipulator, and upright. They also require the cabinet to be closed. Keeping the container upright throughout transport additionally constrains the intervening trajectory. Operational definitions must specify tolerances and evidence appropriate to the task and sensing capabilities. Support and release distinguish completed placement from holding the container at the target.

Complementary Representations and Task Abstractions Goals may originate from language, images, demonstrations, geometric designs, or numerical process specifications. They may require a terminal state, sustained conditions, or repeated service. Symbolic descriptions express objects, relations, and logical requirements. Geometric representations specify poses, regions, and clearances. Physical variables describe forces, temperatures, or velocities. Temporal specifications distinguish eventual achievement from requirements that must persist throughout execution (Kress-Gazit et al., 2009). These descriptions serve different decisions within the same task. A target image communicates arrangement but may omit hidden contact conditions. A learning reward guides optimization but need not fully specify acceptable behavior.

Task-relevant abstractions connect these descriptions to observations. Object properties and relations indicate which conditions currently hold, while action models describe how those conditions

may change. Such abstractions can be designed or proposed from language, video, and interaction records, then checked for consistency and against available data. A learned detector may assess whether a cabinet is open, while an action model associates opening with access to its interior. The representation provides an adjustable interface for reasoning, learning, and verification. It need not impose a fixed symbolic vocabulary or a particular architecture. Learning this interface must preserve the task's specified requirements.

Grounding Entities and Conditions in the Current Scene Semantic grounding identifies the intended container and cabinet and estimates their task-relevant properties and relations. Recognition alone does not establish that the cabinet is accessible or has sufficient storage space. Even a small observational difference, such as an open versus closed door, changes the conditions from which action must begin. Grounding therefore relates instruction semantics to the current scene and the agent's capabilities.

Language and vision-language models can supply useful priors. SayCan combines language-based skill relevance with skill value estimates grounded in the robot's situation (Ichter et al., 2023). VoxPoser constructs observation-linked spatial value maps to guide motion generation (Huang et al., 2023a). They illustrate different interfaces between semantic interpretation and physical behavior, each relying on appropriate perception and capability estimates. Figure 8.6 summarizes the grounding process. Action organization is developed in the next subsection.

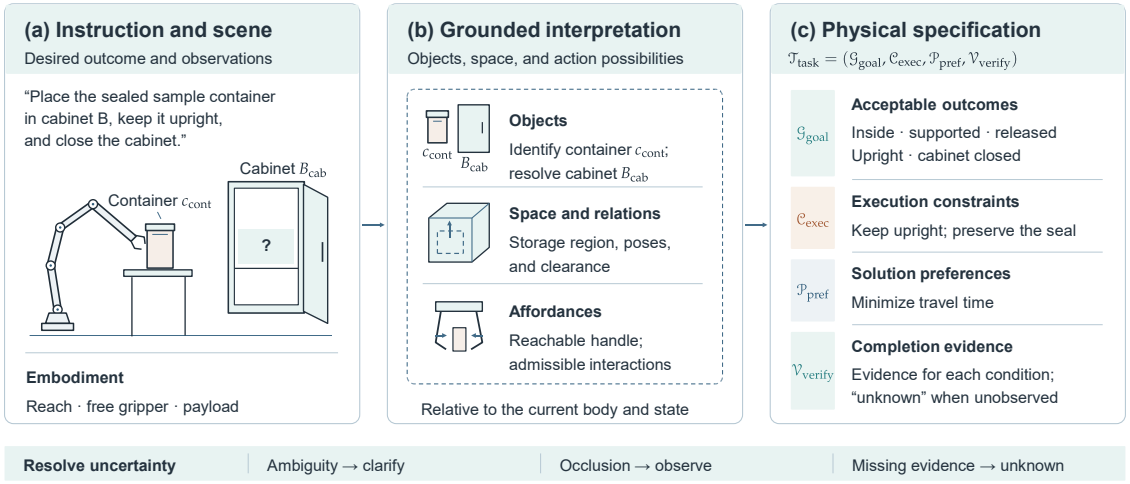


Fig. 8.6 Grounding an instruction in task-relevant objects, state conditions, constraints, and completion evidence. Observations and embodiment capabilities inform interpretation. Unresolved uncertainty may require further observation or clarification

Uncertainty and Evidence Requirements Physical states are partially observed. The belief state $b_t(x) = p(x_t = x \mid o_{0:t}, u_{0:t-1})$ represents the conditional probability mass or density of a candidate physical state x at decision index t. Here p is the conditional state distribution, $o_{0:t}$ denotes observations from indices 0 through t, and $u_{0:t-1}$ denotes the executed control inputs over preceding transitions. u_t is the control input implementing the selected action a_t. This representation distinguishes measured information from inferred conditions and supports information-seeking actions (Kaelbling et al., 1998). Implementations may instead use confidence-tagged records, alternative hypotheses, or other uncertainty estimates.

If two containers match the instruction, further evidence may be needed to resolve their identity. An occluded cabinet interior may require inspection before placement. Clarification is appropriate when ambiguity materially changes the intended outcome or acceptable risk. Observation should resolve the uncertainty relevant to the next decision.

Verification requirements must reflect what can be established from evidence. A visually closed door does not establish latching without suitable observations. Conditions should therefore admit an "unknown" assessment when evidence is insufficient. The specification must identify both the required conditions and the evidence that would justify claiming their satisfaction.

8.3.2 Task Decomposition and Action Organization

A Functional Hierarchy from Goals to Control Task realization connects decisions at different abstractions and timescales. A useful functional hierarchy is

$$\begin{aligned}\text{goal specification} &\longrightarrow \text{subgoals and dependencies} \longrightarrow \text{skills} \\ &\longrightarrow \text{motion or process references} \longrightarrow \text{control inputs.}\end{aligned} \tag{8.10}$$

Upper levels determine desired outcomes and their dependencies. Intermediate levels select skills and parameters. Lower levels generate movement or process evolution. These are functional responsibilities. A learned policy may combine several levels, while a system-level physical agent may refine tasks into instrument commands rather than joint trajectories.

For the storage example, decomposition includes preparing access, acquiring and transporting the container, placing it, and closing the cabinet. The order depends on capability and resource availability. A single-gripper robot may need to open the cabinet before acquiring the container or use an intermediate support surface. An additional manipulator permits other arrangements. Decomposition therefore accounts for resources occupied by each action and conditions needed by subsequent actions.

Task Planning and Dependency Structure Task planning uses action conditions and predicted effects to identify dependencies and construct plans, as exemplified by the Stanford Research Institute Problem Solver (STRIPS) (Fikes and Nilsson, 1971). Hierarchical decomposition organizes recurring procedures into reusable task structures. The planning model should reflect the agent's available skills and the current environment, so that task steps correspond to capabilities the agent can invoke.

A plan need not impose a total order. Access must precede insertion, and the manipulator must leave the door's swept region before closure, whereas independent operations may be reordered or run concurrently when resources permit. Logically independent steps may still compete for the same manipulator or instrument. Conditional branches accommodate initially unknown states, such as whether the cabinet is open. The resulting structure separates persistent dependencies from ordering choices that vary with initial conditions and resources.

Coupling Task Planning with Physical Feasibility Grasp choice, approach configuration, and placement pose affect which task sequences are feasible. A grasp may obstruct passage through the opening, or a placement may leave insufficient room for withdrawal. Task and motion planning (TAMP) addresses these interactions by coupling discrete action and contact choices with continuous configurations, grasps, and trajectories (Garrett et al., 2021).

The coupling is bidirectional. A task planner proposes actions and required conditions, while geometric and physical reasoning supplies feasible parameters or revises those choices. PDDLStream connects symbolic planning to black-box procedures that generate values satisfying continuous constraints (Garrett et al., 2020). Failure to find suitable parameters can prompt a different grasp, approach, or task sequence. These parameters must support the entire relevant interaction, including approach, contact, transport, and release, rather than merely reaching an isolated pose.

Relevant constraints extend beyond collision avoidance. Container transport requires stability and acceleration limits. Door manipulation involves contact and changing geometry. Manufacturing and laboratory workflows similarly couple operation ordering to machine availability, instrument access, and admissible process or sample states. These conditions must be represented where they affect planning. Geometric feasibility alone cannot capture them all. Figure 8.7 summarizes this organization.

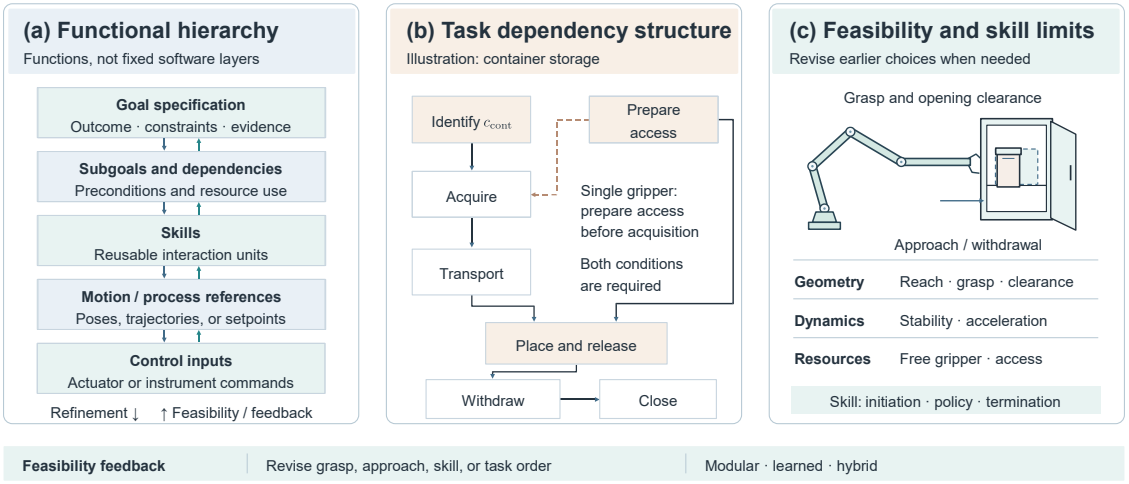


Fig. 8.7 Hierarchical action organization with downward task refinement and upward feasibility feedback. Geometric, dynamic, and resource constraints can revise higher-level choices within modular, learned, or hybrid implementations

Skills as Reusable Units of Interaction Skills such as grasping, opening, docking, and measuring encapsulate temporally extended interactions. In the options framework, a skill comprises an initiation set, an internal policy, and a termination rule (Sutton et al., 1999):

$$\sigma_k^{\text{opt}} = (\mathcal{I}_k^{\text{init}}, \pi_k^{\text{skill}}, \beta_k^{\text{term}}), \tag{8.11}$$

where σ_k^{opt} denotes skill k in the skill library, $\mathcal{I}_k^{\text{init}}$ is its set of admissible starting states, π_k^{skill} its internal action policy, and β_k^{term} its state-dependent termination probability in $[0, 1]$. Practical interfaces may expose parameters, expected effects, operating limits, and completion or failure statuses. Such interfaces support composition independently of whether a skill is programmed or learned.

Composition requires compatible boundary conditions. Grasping must leave the container suitable for transport. Placement must preserve access for withdrawal and closure. Skill quality includes its contribution to subsequent actions. Behavior trees organize conditions, skills, and alternative branches into a modular execution structure (Colledanchise and Ögren, 2018), providing one way to assemble reusable capabilities into longer tasks.

Learning, Adaptation, and Hybrid Organization Learning can acquire skills from demonstrations or interaction, estimate skill parameters, and support action selection. It can also be used to adapt a planner to a particular task domain and skill library. Learning a planner concerns the selection and organization of actions, whereas learning a skill concerns how an interaction is executed. The two can share experience while retaining distinct objectives. Such adaptation should preserve the relationship between proposed steps, available capabilities, and task constraints.

Generative policies offer expressive representations for skill execution. Diffusion Policy produces observation-conditioned action sequences with receding-horizon updates (Chi et al., 2023). OpenVLA connects visual observations and language instructions to learned robotic actions (Kim et al., 2025b). Their use in new settings depends on data coverage and compatibility with the agent's embodiment and operating conditions. Adaptation may therefore involve updating selected components rather than redesigning the entire hierarchy.

Hybrid organization assigns complementary functions to planning and learned behavior. A planner can select subgoals and skills, while a learned policy handles contact-rich interaction. Code as Policies illustrates another interface by generating programs that compose perception and control

functions (Liang et al., 2023a). The allocation of functions should follow task demands and the evidence available for validating each component.

8.3.3 Closed-Loop Execution and Failure Recovery

Execution as Repeated State-Dependent Commitment Execution tests a plan's predictions against a changing world. At each decision point, the physical agent updates its state estimate and task context, then decides whether to continue, obtain information, adjust behavior, or replan. A compact representation is

$$\begin{aligned} b_{t+1} &= \text{Update}(b_t, u_t, o_{t+1}), \\ d_{t+1}^{\text{skill}} &= \text{Supervise}(b_{t+1}, \mathcal{T}_{\text{task}}, \zeta_t^{\text{exec}}), \end{aligned} \tag{8.12}$$

where t indexes decision steps, b_t is the task-relevant belief state, u_t is the control applied during step t, and o_{t+1} is the resulting observation. The function Update revises that belief, while Supervise selects the next supervisory decision d_{t+1}^{skill} using the task specification $\mathcal{T}_{\text{task}}$ and execution context ζ_t^{exec}. This context records the active skill, verified subgoals, and relevant constraint history. The decision may authorize continuation, information gathering, recovery, replanning, or termination. History matters because terminal observations cannot reveal every earlier constraint violation. If another actor closes the cabinet during transport, the remaining plan must accommodate the changed access condition.

Feedback operates at several timescales. Controllers regulate physical variables, skills assess progress and termination, and task-level supervision revises dependencies. Model predictive control repeatedly solves a finite-horizon problem and executes an initial portion before updating it (Mayne et al., 2000). Inner Monologue uses success feedback, scene descriptions, and human feedback to inform language-based task planning (Huang et al., 2023b). These levels provide complementary corrections. Timely control feedback regulates execution, while task supervision assesses its relevance to the goal.

Monitoring Effects and Diagnosing Discrepancies Monitoring tests whether prerequisites remain valid, expected changes occur, constraints hold, and goals are achieved. Evidence may include images, joint states, contact measurements, instrument readouts, and elapsed time. Gripper closure does not establish acquisition, a placement pose does not establish support, and a door-close command does not establish closure. The relevant checks concern physical effects.

Diagnosis distinguishes semantic mistakes, state-estimation errors, infeasible plans, skill failures, and external changes. A small alignment error may be corrected locally. Persistent lack of progress may indicate a blocked mechanism or an incorrect condition estimate. Repeated discrepancies also warrant checking the state abstractions and action models used for planning. Execution records can support revisions to initiation conditions or effect predictions, which should be validated before reuse.

Monitoring itself is uncertain. Confirmation over time, complementary sensors, and explicit "unknown" assessments can reduce false alarms and missed detections. Confirmation must nevertheless remain compatible with timely protective action. Figure 8.8 relates these checks to subsequent decisions.

Recovery as Constrained Continuation Recovery addresses the diagnosed cause while preserving progress that remains valid. Responses range from local correction and renewed observation to another grasp, repositioning, or task-level replanning. Each response starts from the current physical state. A failed placement may leave the container tilted or partially supported, so the conditions for retrying must be reassessed. Previously verified subgoals must also be rechecked when subsequent actions may have disturbed them.

Physical actions can consume resources or cause irreversible changes. Retries should therefore depend on remaining resources, acceptable risk, and evidence that another attempt can address

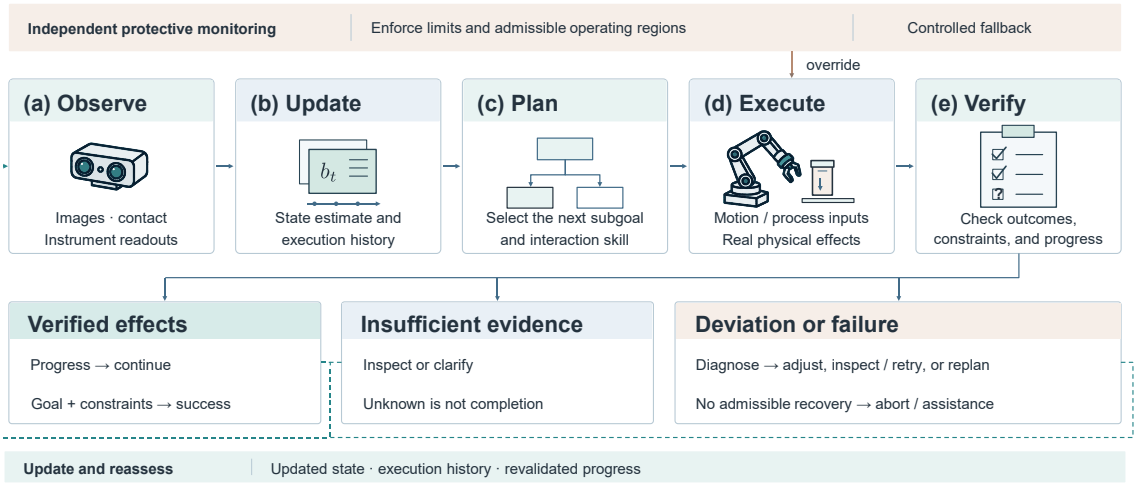


Fig. 8.8 Observed effects update state estimates and execution context. Verified progress supports continuation. Discrepancies trigger diagnosis and recovery. Insufficient evidence prompts further observation. Protective monitoring can interrupt the loop independently of task planning

the cause. If no admissible recovery exists, controlled termination or assistance may be required. Changing the destination or relaxing upright transport changes the task specification and requires authorization unless such alternatives were already permitted.

Safety and Bounded Autonomy Protective mechanisms constrain commands and monitor operating limits independently of unrestricted task-level decisions. Control barrier functions provide one approach to enforcing safety-related state constraints under modeling and feasibility assumptions (Ames et al., 2019). Their guarantees depend on those assumptions, including valid state estimates and available corrective control. Safety requirements therefore need explicit physical conditions and appropriate protection.

Fallback behavior must suit the embodiment and situation. A loaded manipulator may need to maintain support. A moving platform may require controlled deceleration. The scope of autonomy should specify when task execution yields to protective control or an operator, as well as which actions may be initiated.

Execution Evidence and Broader Evaluation Recorded observations, verified subgoals, constraint violations, and interventions preserve the evidence behind execution decisions. Chapter 11 develops their use in systematic task evaluation, capability comparison, and benchmarking. Here, verification supplies the evidence needed to continue, recover, or terminate the current task.

The connection between task specifications, capability-dependent plans, and execution evidence also underpins the coordination among multiple locally autonomous participants discussed next.

8.4 Multi-Agent Collaboration and Distributed Physical AI

Section 8.2.2 describes a *system-level physical agent* through its operational boundary. Distributed sensors, robots, equipment, and computing resources act toward a system objective, with measured outcomes informing subsequent interventions. This section examines the *collaboration among multiple physical agent instances* that retain local sensing, decision-making, and action capabilities. The distinction concerns the level at which agency and coordination are analyzed. System-level agency does not require every component to be autonomous, whereas multi-agent collaboration concerns the relationships among locally autonomous participants. A laboratory or production

installation may therefore be a system-level physical agent as a whole and a multi-agent organization internally. Centralized task allocation is compatible with local agency. An instrument accessible through an application programming interface (API) that only executes commands can instead remain a supporting resource.

Distributed Physical AI denotes the collective ability of agents to combine observations, complementary capabilities, and coordinated actions across a shared physical environment (Parker, 2007; Rizk et al., 2019; Wu et al., 2025a). Coordination must reconcile local autonomy with collective commitments when decisions are coupled through space, time, resources, and the effects of physical action. Communication supports the exchange of observations and coordination information (Zhou et al., 2023). Shared infrastructure and observed actions also inform coordination. Humans may contribute objectives, expertise, supervision, and intervention. Building on the individual-agent mechanisms in Sections 8.1 and 8.3, the following discussion focuses on the distribution of decision authority, inter-agent coordination, and the capabilities and limitations of the resulting teams.

8.4.1 Organizational Architectures for Distributed Physical AI

Consider a partially collapsed tunnel in which a heterogeneous robot team searches for trapped workers and delivers supplies. An aerial vehicle surveys accessible openings. A quadruped traverses rubble and carries relay equipment. A mobile manipulator transports supplies and removes small obstacles where feasible. Their tasks depend on shared access routes, communication coverage, and complementary capabilities. Organizing the team requires specifying where coordination state, computation, and decision authority reside (Yan et al., 2013). Four representative architectures are centralized, hierarchical, decentralized, and hybrid.

Centralized Organization A central node aggregates reported state and performs team-level allocation, resource coordination, or joint planning. It may combine automated planning with human supervision. In the tunnel example, it assigns search, relay, and delivery responsibilities using the available map and robot status. Agents retain local execution authority within these assignments. The center's decisions depend on the coverage and freshness of the information it receives.

Hierarchical Organization Coordination responsibilities are divided across mission, team, and agent levels. A mission coordinator sets rescue priorities. Team coordinators organize operations within assigned regions. Agents execute their responsibilities locally. Decisions that affect several teams are escalated, while regional details remain within the relevant team. Effective hierarchy requires explicit interfaces between levels and sufficient information to recognize cross-team dependencies.

Decentralized Organization Agents coordinate without a single team-level decision center, using local observations and information from other agents. After discovering a trapped worker, an explorer can request assistance. Candidates exchange capabilities and costs and agree on responsibility. Coordination may use negotiation, consensus, or distributed optimization. Agents must track the validity of agreements and revise or suspend them when information or connectivity changes.

Hybrid Organization Different functions or operating conditions use different arrangements. A team may build maps cooperatively while a mission center assigns priorities, or negotiate passage access locally while following centralized scheduling. If communication is interrupted, agents may continue within agreed limits. Such changes require rules for transferring authority and reconciling outstanding commitments when coordination is restored. Figure 8.9 summarizes the four forms.

Table 8.2 compares their typical tradeoffs. Actual performance depends on task coupling, network conditions, and the implementation of coordination protocols.

These architectures describe runtime coordination authority. When coordination policies are learned, centralized training with decentralized execution (CTDE) describes a separate choice

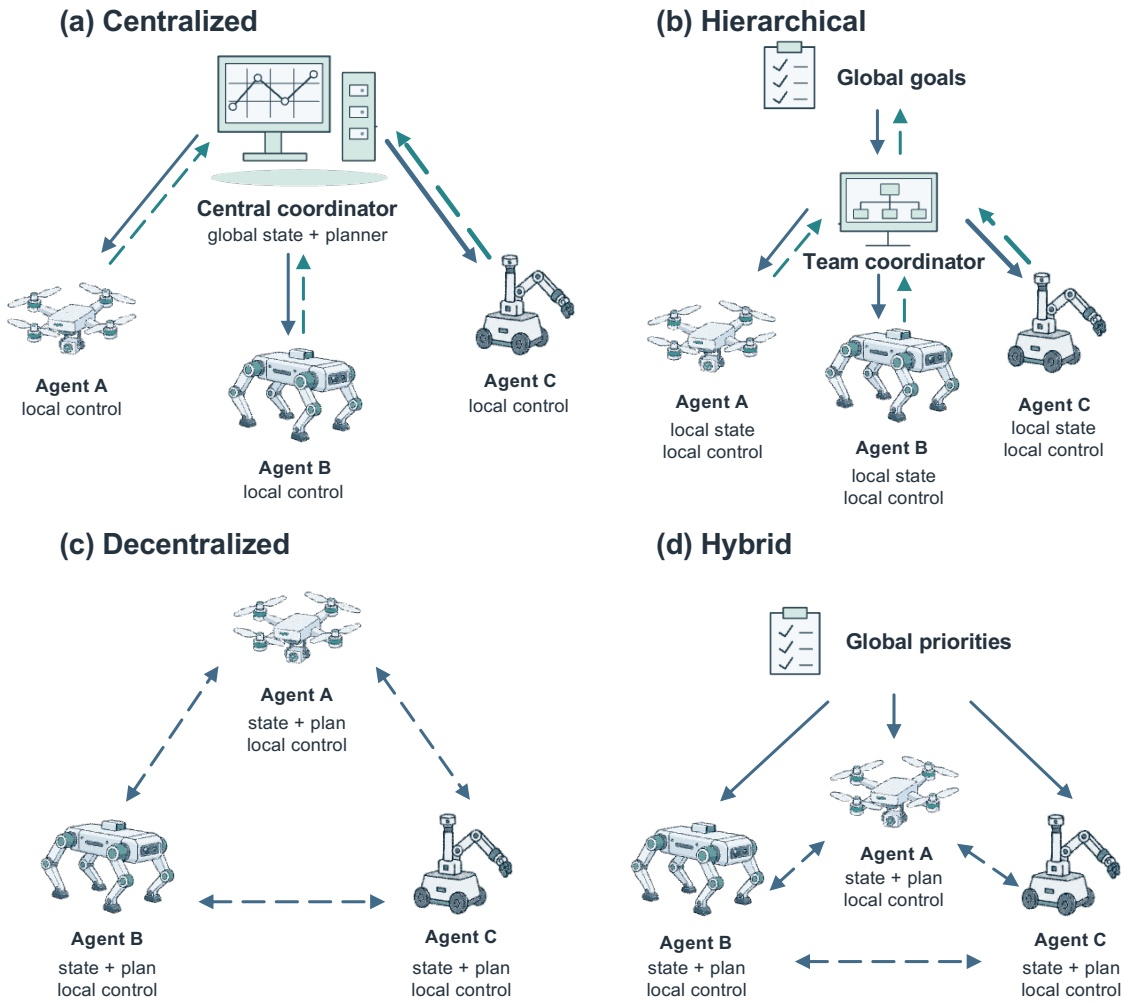


Fig. 8.9 Four organizational architectures for distributed Physical AI, comprising centralized, hierarchical, decentralized, and hybrid

about information available during learning and deployment (Amato, 2024). Training may use global state or centralized value estimates, while deployed policies act on locally available information, potentially including communicated messages. CTDE alone does not determine the team's runtime allocation or coordination architecture.

8.4.2 Coordination Mechanisms for Distributed Physical AI

Organizational architecture determines where coordination decisions are made. Coordination mechanisms determine how agents align their objectives, information, and actions. The central issue is how decisions made by different agents remain compatible when they depend on shared resources or on one another's behavior.

Eight interdependent mechanisms organize this discussion, namely establishing shared goals, capability and role modeling, task allocation, collaborative perception, shared state, joint planning, conflict resolution, and execution and reallocation (Rizk et al., 2019; Wu et al., 2025a). They operate concurrently and revise one another as conditions change. Their ordering in the figure is schematic. Figure 8.10 summarizes their relationships.

Table 8.2 Typical tradeoffs among organizational architectures for distributed Physical AI

Dimension	Centralized	Hierarchical	Decentralized	Hybrid
Global coordination	Joint decisions from reported state, with sensitivity to missing or stale information.	Cross-team coordination through upper levels, where abstraction can hide dependencies.	Global constraints require agreements across local views.	Coordination concentrated on selected global dependencies.
Communication dependence	Requires access to the center for team-level decisions.	Depends on links within teams and between levels.	Depends on the connectivity required by local protocols.	Varies by function and operating mode.
Scalability	Central computation and reporting can become bottlenecks.	Grouping reduces load, while cross-team coupling adds overhead.	Sparse interactions scale more readily than strongly coupled tasks.	Flexible grouping adds management complexity.
Response time	Team decisions incur center communication and processing delays.	Local decisions are fast, while escalation adds delay.	Local response is fast, while negotiation may delay agreement.	Depends on authority transfer and coordination mode.
Fault tolerance	Center failure requires redundancy or local fallback.	Fault isolation depends on backup paths and coordinators.	Avoids a sole center, though partitions can disrupt commitments.	Alternative modes help when transitions are reliable.

Establishing Shared Goals Agents need a common understanding of overall objectives, priorities, completion criteria, and collaboration protocols (Kannan et al., 2024; Obata et al., 2025). Their assigned subgoals can differ while contributing to the same outcome. In tunnel rescue, searching for trapped people, maintaining communication, and delivering supplies must be coordinated

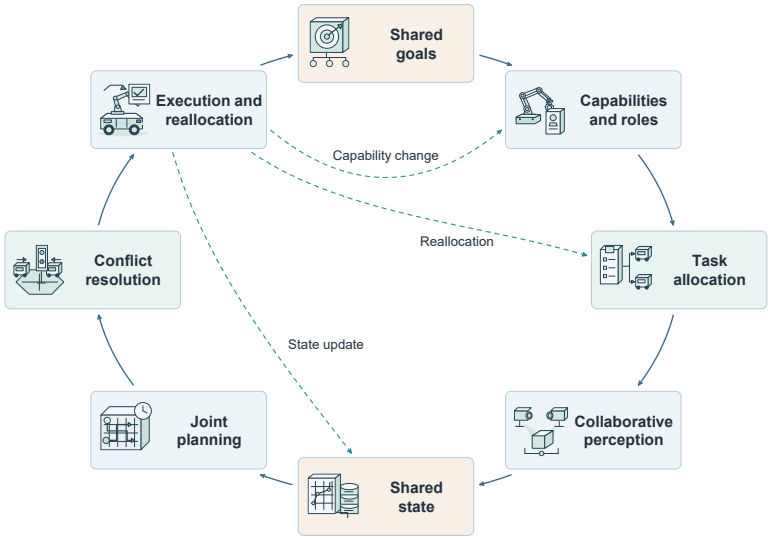


Fig. 8.10 Eight interdependent mechanisms of multi-agent collaboration. The mechanisms may operate concurrently, exchanging information and revising decisions as task conditions and agent capabilities change

according to agreed priorities. A human decision to prioritize confirmation of vital status over delivery should propagate to affected agents, so that their local objectives remain consistent with the revised mission.

Capability and Role Modeling Coordination requires descriptions of what each agent can currently contribute. These combine configuration-dependent capabilities, such as locomotion, sensing range, and payload, with changing conditions, including battery energy, communication connectivity, equipment, and health (Chen et al., 2025a). Roles express how those capabilities serve the mission. A quadruped carrying relay equipment may maintain communication rather than search, while another takes over exploration. Distinguishing capability from role allows responsibilities to change without assuming that robots of the same category are interchangeable.

Task Allocation Allocation assigns tasks to agents or temporary groups, accounting for capability matching, dependencies, energy, risk, and completion time (Gerkey and Matarić, 2004). A delivery task may require both a carrier and a relay, making the assignment a joint choice rather than two independent selections. Centralized optimization, auctions, negotiation, and distributed optimization offer different ways to establish these assignments. Allocation also determines which agents remain available for subsequent demands. Minimizing the immediate travel distance can therefore conflict with preserving scarce capabilities for later tasks.

Collaborative Perception Collaborative perception combines complementary observations and coordinates sensing effort (Xu et al., 2024). An aerial vehicle can survey the tunnel structure, while ground robots inspect occluded regions and assess traversability. Agents can divide observation responsibilities according to uncertainty and information value, reducing unnecessary duplication. Time synchronization, coordinate transformations, and target association support observation fusion across viewpoints (Hu et al., 2022b). Fusion should preserve relevant uncertainty and identify conflicting observations.

Shared State Shared state maintains information needed for joint decisions, including maps, targets, agent locations, resource occupancy, and task progress. Unlike collaborative perception, which concerns acquiring and combining observations, this mechanism concerns distributing and updating the resulting operational information. State may reside centrally or in local copies. Limited bandwidth favors task-relevant abstractions over continuous transmission of raw sensor data (Chang et al., 2023). Agents need sufficiently current and consistent information about the resources and dependencies on which their decisions rely. Identical knowledge of the entire environment is unnecessary. The required consistency depends on how tightly the agents' actions are coupled.

Joint Planning Joint planning addresses dependencies among agents' actions in space, time, and resource use. Two robots sharing a narrow passage must coordinate traversal order. A delivery robot may depend on a relay reaching its coverage position first. Plans can be generated centrally or reconciled through iterative negotiation (Mandi et al., 2024). Coordination can concentrate on strongly coupled near-term actions while leaving independent actions locally planned. This limits the need to fix every agent's full trajectory while retaining explicit commitments where another agent's progress depends on them.

Conflict Resolution Conflicts concern spatial occupancy, schedules, power, bandwidth, or incompatible task assignments. They may arise from competing objectives, outdated information, or deviations from agreed plans. Priorities, reservations, negotiation, and replanning can resolve resource contention. Protective intervention may be required for imminent physical conflicts (Zhang et al., 2025c; Lee et al., 2025). Resolution must propagate changes to affected commitments so that other agents can revise their dependent actions.

Execution and Reallocation Execution reports reveal whether assignments and inter-agent commitments remain viable. Agents share progress, exceptions, and remaining resources when these

affect collaborators. A blocked route, lost connection, or unavailable executor may require transferring a task or reorganizing a group (Liu et al., 2025b). Reallocation should account for work already completed, current resource reservations, and dependencies on other agents. Updating roles and shared state alongside the assignment prevents a replacement agent from duplicating completed work or continuing under obsolete assumptions.

8.4.3 Open Problems and Trends in Distributed Physical AI

Coordination can extend the coverage, throughput, and range of tasks achievable by a team. These gains depend on how sensing, task dependencies, resources, and failures are managed across its members.

Collective Capabilities Complementary sensing and coordinated revisits extend spatial coverage and the frequency of environmental updates (Francos and Bruckstein, 2023). Independent subtasks can proceed concurrently, while ordered operations can be pipelined. Under the idealized assumption of equal per-participant execution rates, let W_{work} denote total workload measured as the time required by one physical agent, $N_{\text{agent}} \geq 1$ the number of participating instances, L_{crit} the duration of the critical path induced by task dependencies, and T the team's completion duration. In this bound, W_{work}, L_{crit}, and T use the same time unit, and

$$T \geq \max\left(\frac{W_{\text{work}}}{N_{\text{agent}}}, L_{\text{crit}}\right). \tag{8.13}$$

This lower bound exposes the limits of parallelism (Brent, 1974). Physical contention and coordination overhead can further restrict the benefit of additional agents. Heterogeneous capabilities also expand the feasible task set. Aerial reconnaissance, ground access, manipulation, and infrastructure support can form task chains unavailable to any member alone. Alternative executors and routes can preserve service after failures (Bossens et al., 2022), but resilience requires substitutes for critical tools, capabilities, and communication links. Realizing these gains raises the following open problems.

Communication, Information Freshness, and Consistency Communication range changes with robot motion and environmental conditions, while bandwidth, delay, and packet loss constrain information exchange (Gielis et al., 2022). Collaboration requires deciding which observations, task commitments, and resource states to share, and when their expected benefit justifies transmission. Agents must also track whether information remains valid when making coupled decisions. A delayed passage-occupancy update can be more consequential than an incomplete distant map. Maintaining sufficient consistency requires task-specific priorities and rules for conflicting or outdated updates. Because motion changes network topology, communication and physical planning may need to be optimized jointly.

Cross-Embodiment Knowledge and Semantic Interfaces Different observation spaces, action spaces, and dynamics prevent direct transfer of many policies between embodiments. Cross-embodiment learning provides useful experience (Open X-Embodiment Collaboration, 2024), but collaboration additionally requires alignment of object identities, task states, capability descriptions, and expected skill outcomes. An aerial vehicle and a mobile manipulator may interpret the same destination while requiring different access conditions and control policies. Shared interfaces should expose these differences so that task assignment and composition remain meaningful. Compatible task semantics must therefore support coordination while preserving embodiment-specific actions and requirements.

Collaborative Safety and Runtime Assurance Individually acceptable actions can conflict through shared space, resource occupancy, or coupled task dependencies. Updating one learned policy can also change the operating conditions encountered by other agents (Garg et al., 2024). Assurance must therefore address joint behavior, including the validity of reservations and commitments when communication is delayed or an agent fails. Local protective responses need to respect

team interactions. Stopping one robot may obstruct another or interrupt a jointly supported load. Coordination protocols should specify how such responses are communicated and how affected commitments are revised, including the respective authority of agents, coordinators, and human supervisors.

System-Level Evaluation Embodied multi-agent benchmarks provide a basis for assessing collaborative task performance (Chang et al., 2025). Evaluation should also measure the benefit and cost of coordination, including team completion time, duplicated work, workload distribution, communication demand, inter-agent conflicts, and recovery after member failures. Tests should vary team size, embodiment combinations, connectivity, and capability availability. Comparisons with suitable single-agent or independently operating baselines help identify gains attributable to coordination. Recovery results should distinguish loss of a replaceable member from loss of a unique capability. These protocols reveal whether performance reflects effective distributed collaboration and whether it persists as the team changes.

The following directions seek to address these limitations.

Shared World Models Shared world models extend exchanged state records toward predictions of how one agent's actions affect other agents and the environment (Zhang et al., 2025a). For example, moving a relay robot changes the communication conditions under which other agents can proceed. The challenge is to coordinate local models with different spatial resolutions and prediction horizons without requiring a single central model. Their predictions should expose coupled constraints and support comparison of alternative joint actions. Learning these interactions from team experience could improve planning when capability, resource, and communication dependencies cannot be handled independently.

Multi-Agent Foundation Models Learned coordination components, including collaborative perception (Blumenkamp et al., 2025), suggest a route toward broader models of team behavior. Models trained across tasks and team configurations could acquire priors about capability differences, task dependencies, and other agents' likely actions. Transfer to new teams would require representations that accommodate variable membership and embodiment-specific capabilities. Such models should provide reusable coordination knowledge for allocation and joint planning, assessed through physical team performance.

Self-Organizing Collaboration in Open Environments Long-running teams must accommodate agents joining or leaving, newly arising tasks, changing connectivity, and human intervention. Self-organization can form temporary teams, select coordinators, and revise roles according to available capabilities (Zhu et al., 2024). Such changes must preserve task ownership and outstanding commitments across organizational transitions. The resulting direction is adaptive coordination that maintains core services while membership and architecture evolve, extending distributed Physical AI beyond fixed teams and predefined operating conditions.

9 Hardware and Systems

At last, every Physical AI program ultimately depends on an actual machine to execute proposed actions in the real world. Therefore, this chapter discusses the actual hardware and system through five questions: what physical interactions a body's morphology can support (Section 9.1), how sensing and actuation close the feedback loop (Section 9.2), where computation can run without missing the time to act (Section 9.3), how software organizes these capabilities (Section 9.4), and how they remain reliable through repeated use and updates (Section 9.5). For the ease of understanding, there are several case studies: a simple component case in Section 9.2, a dexterous-hand case study in Section 9.6 to illustrate how hardware shapes interaction, and a more complex humanoid case that stitches all components together in Section 9.7.

9.1 Hardware and Embodiment Taxonomy for Physical AI

A robot opening an unfamiliar drawer must do more than recognize a handle. It must reach it, establish a grip, and discover how the drawer responds to a pull. Resistance may mean that the drawer is heavy, jammed, or still locked. Continuing the same movement could damage the handle. The robot needs evidence that distinguishes these possibilities and a body capable of changing its action. This connection between physical interaction, observation, and correction is central to Physical AI.

9.1.1 From Hardware Components to Embodiment

Cameras, motors, encoders, transmissions, and processors are hardware components. Their arrangement gives a robot its *embodiment*, comprising its shape and mass distribution, available movements, sensing locations, and ways of making contact. The platform adds power, computation, communication, and the controllers that regulate those movements. Together, these correspond to the broader notion of the embodiment B_{emb} in Section 8.1. A change anywhere in this system can change what an action means. Replacing a rigid fingertip with a soft pad alters both the grip and the measurements produced during contact.

The drawer example makes these dependencies visible. An external camera may locate the handle but lose sight of it behind the gripper. Joint encoders describe arm movement, while force or tactile measurements can reveal resistance or slip. A small initial pull then serves two purposes by moving the drawer and testing the robot's prediction about its motion. The outcome can justify continuing, changing direction, or releasing the handle. Such exploratory actions require suitable force limits and feedback. Trial and error is useful only when the body can tolerate the trial and the system can interpret its outcome.

The same mechanics constrain possible corrections. Reach does not guarantee that an arm can sustain a load, and a grip that supports lifting may fail when twisting a cap. Grasp analysis captures this distinction through the forces and torques a set of contacts can resist (Ferrari and Canny, 1992). A mobile robot may reach the handle yet lack room to reposition or sufficient support to resist the pull. Planning must therefore consider how to remain controllable after the first movement, as well as whether that movement is possible.

Useful feedback must also arrive before the opportunity to correct is lost. A detailed image delivered after an object has slipped provides evidence for later learning but cannot rescue that grasp. Sensor placement and task-state observability (Section 9.2.3) must be considered together with control delay and actuator response (Section 9.2.2). Better inference can extract more from available signals. Sustained performance still depends on obtaining the right evidence and acting within the body's limits.

Table 9.1 compares embodiments through the interactions they support. A liquid handler is an example of a fixed instrument. An arm on a wheeled base is a mobile manipulator. A biped with arms belongs to the legged and humanoid category. These categories describe different physical

Table 9.1 Common embodiments and the feedback their tasks require. Categories overlap. A hand belongs to a carrier, and a laboratory may combine several kinds of machine

Embodiment	Physical advantage	What the feedback must resolve
Fixed instruments	Fixtures and specialized tools support repeatable operations	Whether a process achieved its target volume, temperature, or other measured result
Fixed-base arms	Stable support for manipulation within reach	Tool position and contact loading, including errors hidden by visual occlusion
Mobile manipulators	Access to several workstations	Base location and support as well as arm and object state
Legged and humanoid robots	Movement across steps and uneven ground	Balance and changing support contacts while moving or manipulating
Dexterous hands	Multiple contacts permit adjustment within the grasp	Finger configuration, contact distribution, and incipient slip
Soft and continuum robots	Deformation accommodates confined spaces or delicate objects	Changing shape and loading despite material hysteresis and distributed contact
Aerial and underwater robots	Access beyond ground routes	Body motion under fluid disturbances, with constrained energy and communication
Robot-instrument systems	Complementary machines complete experimental workflows	Sample identity, transfers, and measured outcomes across machine interfaces

advantages, rather than successive levels of intelligence. Suitability depends on the task and on what each body can measure while performing it.

Autonomous laboratories illustrate why capability often comes from a combination of bodies. Instruments provide controlled processes and measurements, while arms or mobile robots transfer samples and operate equipment (Burger et al., 2020; Szymanski et al., 2023; Dai et al., 2024). The resulting system must track whether each transfer and operation actually succeeded. These interfaces connect embodiment to the system-level agents in Section 8.2.

9.1.2 Embodied Model-Hardware Co-Design and Multi-Embodiment Adaptation

Model-hardware co-design starts from a coupled choice. The body determines which states can be observed and which actions can be realized, while the model and controller determine how much of that capability is used. A camera added to expose a contact changes the policy's inputs and training data. A softer fingertip changes contact dynamics and may reduce the precision the policy must achieve. A larger model can improve inference yet lose that advantage if its delay exceeds the time available to correct a slip. These choices must therefore be evaluated as a system, against the same physical task and operating limits.

A model influences the world through a control interface. It may request a tool movement, joint positions, or forces. Local controllers translate those targets into actuator behavior. An *embodiment adapter* handles the robot's command conventions, coordinate frames, and limits. Figure 9.1 shows three arrangements of these responsibilities. All require a return path from the physical outcome to the next decision.

A modular pipeline estimates an object's pose, plans a motion, and tracks it with a controller. Its intermediate quantities make some failures easier to locate. The object may have been misidentified, the path may have been infeasible, or contact may have differed from the prediction. An end-to-end vision-language-action policy learns a more direct mapping from images and instructions to actions, as in OpenVLA (Kim et al., 2025b). It can exploit visual cues without requiring an explicit object pose at every step, but its output still has the units and physical consequences defined by the controller. Without incorporating new observations, neither arrangement can revise its task-level response to an unexpected event.

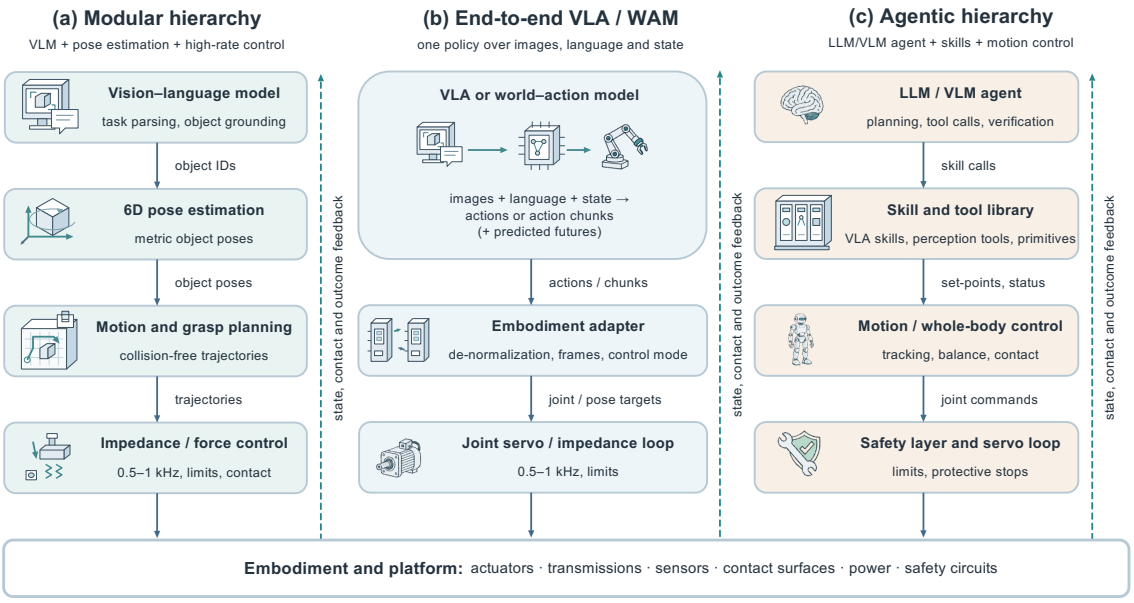


Fig. 9.1 Models reach the world through different control arrangements. A modular hierarchy exposes intermediate estimates and plans. An end-to-end policy learns a more direct observation-to-action mapping. An agent calls skills and checks their outcomes. Local feedback supports timely motion and contact regulation in each arrangement

An agentic hierarchy adds task-level decisions, such as selecting a tool or deciding to retry. Systems that compose skills from language illustrate this separation (Ichter et al., 2023; Liang et al., 2023a). During drawer opening, a fast controller may limit contact force while the higher-level system decides whether resistance calls for another strategy. A completion message from a skill is useful only insofar as it is supported by evidence about the drawer. Execution checking in Section 9.4 and reliability assessment in Section 9.5 develop this requirement further.

Transfer to another robot changes the relationship between observation and action. Knowledge that a drawer translates along a guide may remain useful. Camera appearance, reachable poses, finger contacts, motor limits, and the forces generated by a command may all change. Expressing a desired movement relative to the tool can reduce dependence on a laboratory's coordinates, but does not make the new robot mechanically equivalent. The receiving body must still support the intended motion and contact.

Training across robots can reveal useful shared regularities. The Open X-Embodiment study demonstrates benefits from combining data across robots (Open X-Embodiment Collaboration, 2024). Octo supports adaptation to new observation and action spaces through fine-tuning (Ghosh et al., 2024), while HPT combines a shared model with embodiment-specific input and output modules (Wang et al., 2024c). These approaches organize what can be shared and what must be adapted. Their physical value must be established on the receiving robot, including its contact behavior and recovery from errors (Section 6.4). Success on familiar images alone does not establish that the transferred commands produce the intended forces.

Hardware design can make this adaptation easier. ALOHA's matched leader and follower arms give demonstrations a direct relationship to robot joint movements (Zhao et al., 2023). UMI coordinates a handheld gripper, camera geometry, and action representation to support transfer from portable demonstrations (Chi et al., 2024) (Section 9.2.1). A compliant fingertip can also tolerate small placement errors, while tactile sensing reveals how the contact develops. Joint optimization can extend to the mechanism itself. Chen et al. (2021) optimize mechanical and control parame-

ters together and build an underactuated hand from the resulting design. Such optimization still depends on whether the modeled contacts and actuator response describe the physical machine.

For the drawer task, a useful co-design cycle begins by identifying the failed correction. If the handle disappears behind the fingers, changing the viewpoint or adding touch may resolve missing evidence. If resistance is measured but force cannot be reduced promptly, transmission, compliance, and controller bandwidth become the relevant design variables. If the state is observed and the action is feasible but the policy chooses poorly, additional training may be the appropriate change. Each candidate requires data collected or transformed for its own observation and action interface, followed by evaluation on held-out physical trials. Comparing an unchanged policy across bodies measures transfer. Allowing each body the same adaptation budget tests the quality of the co-designed system. Reporting both prevents a favorable match to one policy from being mistaken for a universal hardware advantage. The final comparison is cost per valid output, including collection, computation, interventions, and maintenance, as developed below and in Section 9.5.

9.1.3 Task-Oriented Embodiment Selection, Validation, and Evaluation

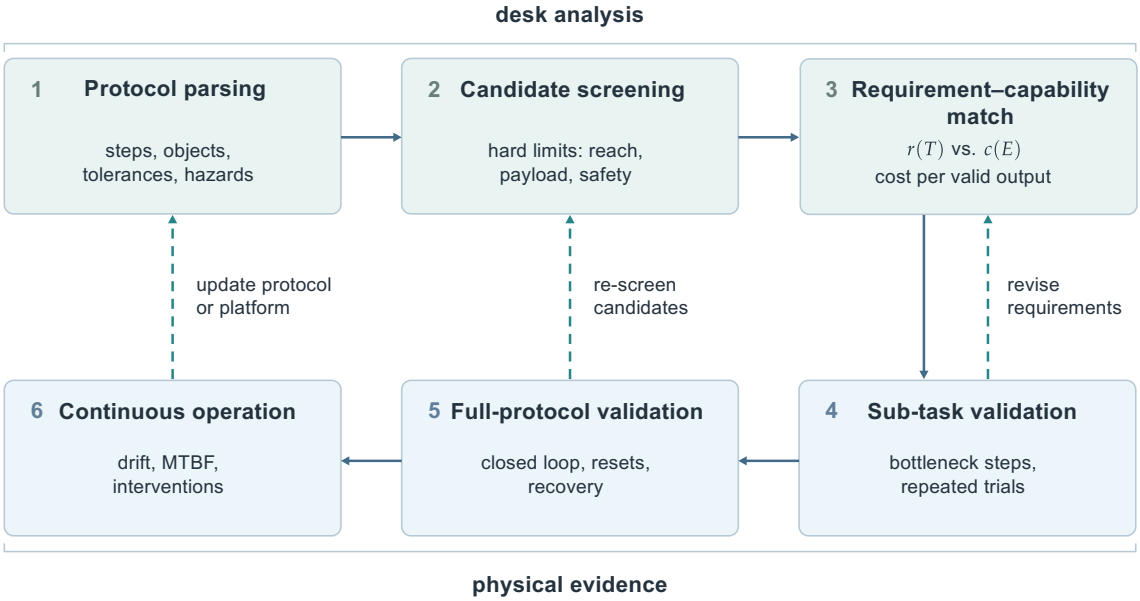


Fig. 9.2 Embodiment selection develops through increasingly realistic evidence. Task analysis and specifications identify candidates. Operation-level tests, complete workflows, and continued use reveal whether the proposed body and controller meet the requirements

Selecting a robot begins with a complete task, including the measurements that establish success. Reach and payload can exclude an unsuitable arm, but do not establish whether it can thread a cap without damaging a vial. Figure 9.2 moves from requirements to tests of individual operations, complete workflows, and continued use. Failures can motivate a change in the policy, sensing, mechanics, or protocol.

Candidates should meet the same requirements for workspace, load, precision, contact force, time, and recovery. Among feasible options, the cost per valid output introduced in Section 1.5 gives a useful comparison. Divide the fully allocated average cost of an attempt by the measured fraction that produces a valid result. The cost includes failed attempts, human interventions, and recovery. Early estimates remain provisional until tested under the intended workload.

A laboratory preparing metal-organic frameworks (MOFs), porous materials built from metal-containing units and organic linkers (Furukawa et al., 2013), provides a concrete example. A syn-

thesis workflow may involve opening vials, transferring solids and liquids, heating, separation, and loading instruments (Tom et al., 2024). Successful motion is insufficient. The sample must retain its identity, composition, and integrity. A balance may verify a transfer that the arm's camera and encoders cannot establish by themselves.

Consider opening a sealed vial, dispensing a specified quantity, and seating the vial in an instrument. Table 9.2 compares four possible configurations through the questions these operations raise. Their exact arms, grippers, sensors, and controller settings must be specified before testing. Additional fingers or lower transmission ratios change the possibilities for interaction. Their practical benefit depends on how well that interaction is observed and controlled.

Tests should use the same protocol, object batches, starting-condition distribution, and definition of success. Repeated trials distinguish a reliable advantage from a fortunate grasp (Section 9.5.1). Recording observations, commands, and measured outcomes helps explain failures by distinguishing whether the robot lacked information, chose an unsuitable action, or could not physically realize it. The diagnosis then guides a concrete change and another test.

The preferred solution may combine specialized machines, provided their transfers and coordination remain reliable. Continued operation must then test calibration drift, worn contact surfaces, thermal limits, and recovery after interruptions. This closes the design loop. Experience changes the model and, where necessary, the body that supplies its evidence and carries out its decisions.

9.2 Sensors, Actuators, and Intelligent Materials

To lift a glass vial, a robot must first find it, then recognize contact, and finally hold it firmly without breaking it. A camera helps with the approach. Joint encoders report how the fingers and arm have moved. Touch and force measurements reveal what happens when the fingers meet the glass. Electric motors supply the motion, but their response depends on the transmission, the controller, and the load. Physical AI emerges through this interaction. The robot acts, measures the consequences, and adjusts its behavior. A model can infer an object's properties from experience only when the hardware exposes useful evidence and can carry out the resulting decisions.

The relevant hardware therefore extends from visible cameras and grippers to the electronics inside a joint, including encoders, current sensors, power transistors, and local control processors. These components determine both what a learning system can observe and how its actions affect the world. Figure 9.3 connects the measurements collected during learning to the feedback used during execution. Section 3.2 discusses sensor physics and fusion more generally. Here the emphasis is on their consequences for robot learning and physical interaction.

Table 9.2 Questions for evaluating four candidate configurations in a MOF workflow. These are mechanisms to test, not measured rankings. The first three configurations use parallel grippers

Configuration	Interaction to evaluate	Evidence needed
Torque-sensing dual arm with impedance control	Yielding during cap engagement and vial seating	Contact loads, alignment errors, damage, and completion under matched tuning
Position-controlled dual arm without added force sensing	Precise free motion followed by constrained contact	Contact-force peaks, effects of passive compliance, and recovery from misalignment
Low-reduction dual arm with motor-current feedback	Responsive motion and current-based loading estimates	Accuracy under load, effects of friction and acceleration on force estimates
Arms carrying dexterous hands	Reorientation and cap handling through multiple contacts	Slip, successful in-hand adjustments, and whether fewer regrasps offset coordination costs

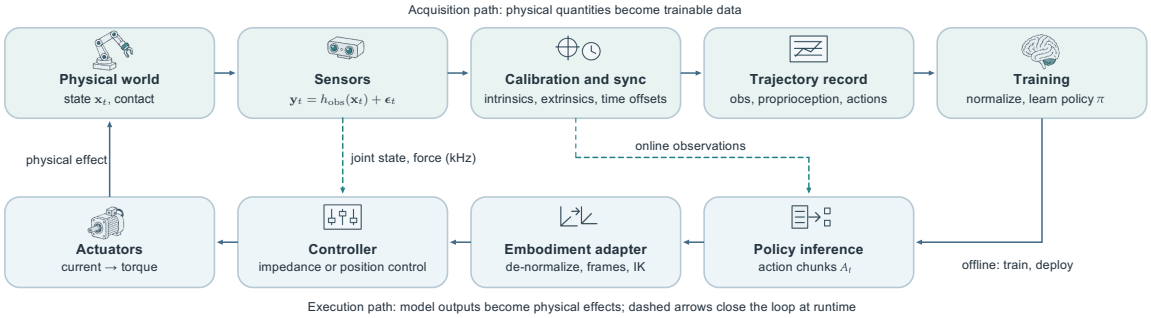


Fig. 9.3 Measurements connect learning to physical action. Calibrated, synchronized observations become training trajectories. During execution, the policy supplies targets, while local controllers use fresh body and contact measurements to drive the actuators. The commanded movement and the movement actually achieved need not be identical

9.2.1 Acquiring Physical Experience for Learning

Before a robot can learn from a recorded movement, its sensors must agree on where and when that movement occurred. Camera calibration relates image coordinates to the robot. Joint positions and a kinematic model locate the tool. For a wrist camera, hand-eye calibration identifies the fixed relationship between camera and gripper (Zhang, 2000; Tsai and Lenz, 1989; Park and Martin, 1994). Timing is equally important. A 20 ms mismatch for a tool moving at 0.5 m/s corresponds to 1 cm of displacement. Hardware triggering, synchronized clocks, and measured delays help align images, joint states, and forces. A receipt timestamp is not necessarily the acquisition time. These details can determine whether a demonstration teaches the robot to react to contact or, mistakenly, to a state that occurred afterward.

A useful trajectory records observations, measured body state, and the commands sent to the controller together. It also preserves their units, coordinate frames, calibration, and control mode. A command to advance the tool by a centimeter means something different from a command to exert a force. Even identical position targets can produce different contact behavior under different controller gains. Changes to the camera, gripper, or controller consequently change the learning problem, not just the equipment inventory. If actions are normalized numbers or relative tool motions, deployment must preserve the same scaling and coordinate conventions when converting them into controller targets.

Different collection methods preserve different parts of this physical experience. Robot teleoperation, as in ALOHA and DROID (Zhao et al., 2023; Khazatsky et al., 2024), records demonstrations through a robot's own body and controller. Handheld devices such as UMI make collection more portable by combining a gripper with a camera and motion estimation (Chi et al., 2024). Their trajectories can describe real contact, but the human holding the device does not have the robot's inertia, reach, or torque limits. Human video supplies broader examples of activities and object use, while motion-tracking gloves and exoskeletons add hand or arm kinematics (Grauman et al., 2022; Wang et al., 2024a; Fang et al., 2024a).

Surface electromyography (sEMG) adds a different kind of observation. Electrodes on the skin record the electrical activity associated with muscle recruitment (Farina et al., 2014). This is the modality denoted EMG (electromyography) in Figure 9.4. It can reveal motor intent when movement is slight or visually occluded, making it a useful complement to video during demonstration collection (Kaifosh et al., 2025). Its interpretation nevertheless depends on the person, posture, and electrode placement. Neither visible motion nor muscle electrical activity directly specifies the contact forces a robot must reproduce. Recordings should therefore synchronize EMG with kinematics and video, preserve the electrode configuration and calibration, and include force or tactile mea-

surements when contact-force supervision is required. These sources complement robot recordings rather than serving as mechanically interchangeable substitutes.

The data pyramid in Figure 9.4, motivated by GR00T N1 (NVIDIA et al., 2025c), places abundant human video below synthetic and robot-free demonstrations, with robot execution at the top. Its distinction is the kind of evidence available, as well as its volume. Simulation can supply complete robot actions while approximating friction and contact. Human recordings contain real interactions without the robot's execution dynamics. A weighted training objective retains these differences through

$$\min_{\theta} \sum_{j} w_j \, \mathbb{E}_{\zeta_{\text{sample}} \sim \mathcal{D}_j}[\mathcal{L}_j(\theta; \zeta_{\text{sample}})], \qquad w_j \geq 0, \quad \sum_{j} w_j = 1. \tag{9.1}$$

Here each source $\mathcal{D}_j$ supports its own learning objective $\mathcal{L}_j$. Robot actions can supervise action prediction, whereas human video can support visual prediction or activity understanding. Reweighting cannot turn an unmeasured quantity into direct supervision (Section 4.1).

9.2.2 Closing the Loop from Measurements to Motion

When a finger touches a hard surface, force may rise before a large model has processed another image. The robot must therefore respond at several timescales. A learned policy can choose a grasp, a movement, or a desired compliance. A local controller follows that target. The motor drive regulates electrical current. Fast joint feedback remains useful even when the policy is learned end to end. Playing a predicted sequence of actions rapidly is also different from reacting rapidly to new evidence.

The electronics make this distinction concrete. A drive switches power transistors to apply voltages to the motor windings. Current-sensing circuits and analog-to-digital converters report the resulting currents. An encoder or other rotor-position sensor tells the controller how the rotor is oriented. The drive adjusts these voltages to produce the desired torque. Filtering suppresses noise but introduces delay, and electrical switching can contaminate small sensor signals. Consequently, sensing, computation, and power delivery must be designed together (Muthalagu et al., 2025; Kollmorgen, 2020).

The total delay includes sensing, communication, inference, and actuation. In a linearized feedback loop, a delay Δ adds a phase lag $\omega\Delta$ at angular frequency ω. The controller increasingly acts on an outdated state. This explains why contact regulation usually stays close to the hardware, while slower reasoning can run elsewhere (Section 9.3). Each interface must specify reference frames, units, timestamps, control modes, and limits. The return path should report the measured motion and available forces or currents, together with faults and temperature. Without this feedback, a learning system can confuse a saturated actuator or a delayed measurement with an incorrect plan.

9.2.3 Task-State Observability: Cameras, Encoders, and Contact Sensing

The camera is often the most recognizable sensor on a physically intelligent robot. An RGB camera measures light and provides the appearance cues used to recognize objects, interpret a scene, and connect language to action. Depth cameras add geometric information. Stereo systems compare two viewpoints. Structured-light systems observe a projected pattern. Time-of-flight systems estimate distance from the travel time or phase of returned light. These approaches have different sensitivities to distance, illumination, and surface appearance. An integrated RGB-D sensor makes color and depth available together, as illustrated by the RealSense architecture (Keselman et al., 2017). However, transparent glass, shiny metal, and low-texture surfaces can leave missing or misleading depth. ClearGrasp addresses one such problem by inferring geometry for transparent objects (Sajjan et al., 2020). Inferred geometry still requires validation when an action can damage the object.

Placement determines what the camera can contribute. A head or external camera sees the overall workspace but may lose the contact behind the hand. A wrist camera follows the approach and

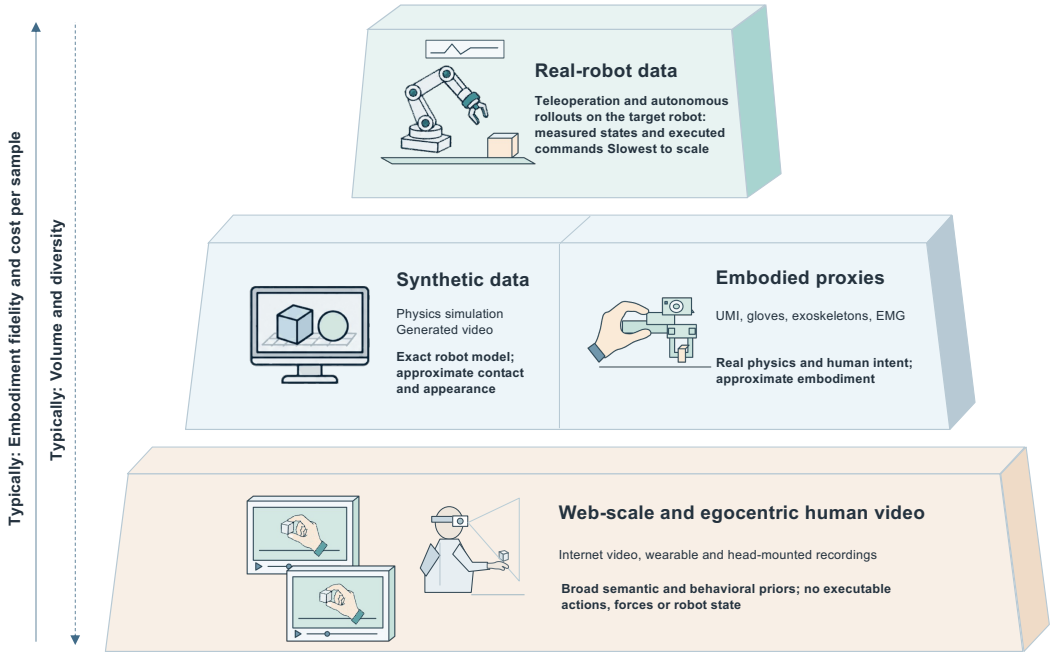


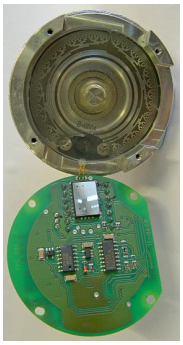
Fig. 9.4 Sources of experience for Physical AI. Human video offers broad coverage. Simulation and handheld devices address different parts of the gap between observation and execution. Robot recordings capture the target body's behavior. The levels describe complementary evidence, not a universal ranking of dataset quality. EMG denotes electromyography. It records muscle activity rather than robot contact forces

reveals small features, although its view moves with the arm and may become obstructed at the final instant. Exposure, motion blur, field of view, and timing can matter more than the advertised pixel count. For a vision-language-action model, moving the camera also changes the relationship between an image and a successful action. Calibration aligns geometry. Training must still account for the change in appearance.

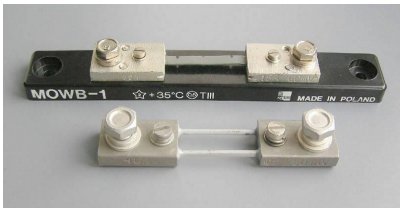
While cameras look outward, encoders measure the body's own movement. A rotary encoder converts shaft rotation into an electrical position reading. Optical encoders read a patterned disk. Magnetic encoders sense the orientation of a magnet. Inductive designs use electromagnetic coupling. Incremental encoders count changes and generally need a reference after startup, whereas absolute encoders identify position within their specified measurement range. More digital counts do not by themselves guarantee more accurate position. Mounting error, alignment, mechanical deformation, and calibration also matter (HEIDENHAIN, 2023). Velocities are often estimated from successive position measurements, which introduces a trade-off between noise and delay.

A motor encoder and a joint-output encoder observe different shafts. With an ideal rigid gearbox, motor angle divided by the reduction ratio gives joint angle. Real transmissions have friction, lost motion, and elastic deformation. An output encoder can reveal movement errors that the motor encoder misses, particularly during reversals and under load. Joint measurements and the robot's geometry then estimate the tool's pose, but do not directly reveal whether an object is slipping between the fingers. On a walking or mobile robot, an IMU adds angular velocity and specific force from gyroscopes and accelerometers. Combined with joint and contact measurements, it helps estimate body motion. Integration alone accumulates error, and gravity must be accounted for when interpreting acceleration. Figure 9.5 presents examples of cameras, encoders, and current-sensing shunts.

(a) RGB-D camera



(b) Optical encoder

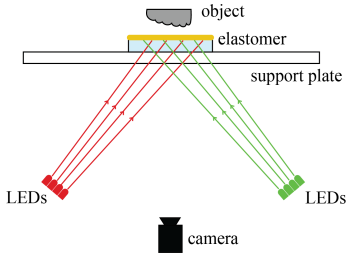


(c) Current-sensing shunts

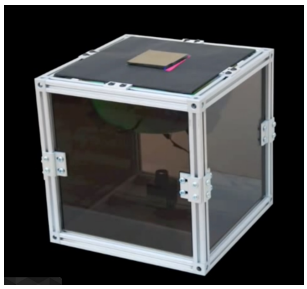
Fig. 9.5 Different measurements support different parts of an action. A camera observes the scene, an encoder reads shaft angle, and a shunt converts electrical current into a small voltage. The opened encoder shows its coded disk. These examples illustrate principles rather than a prescribed robot sensor suite. Photos: (a) Marc Auledas, CC BY-SA 4.0; (b) Mike1024/Dicklyon, public domain; (c) Marek bydg, CC BY-SA 3.0

Touch becomes especially valuable once the hand hides the contact. In a vision-based tactile sensor, a small camera looks inward at a soft surface rather than outward at the scene. Pressing an object against that surface changes its shape and, in some designs, moves printed markers. GelSight uses controlled illumination to recover fine contact geometry. DIGIT packages a related optical approach into a compact fingertip (Yuan et al., 2017; Lambeta et al., 2020). Figure 9.6 shows why optics and material design are inseparable here. The camera observes the deformation of a purpose-built skin. Images can support estimates of local shape, shear deformation, and incipient slip. They are not direct force maps. Recovering force requires calibration or a learned model of the skin and its loading.

This information changes what a policy can learn. A gripper can tighten when the contact patch starts to slip, or adjust the direction of insertion when an edge presses unevenly. A compliant skin may also wear, creep, or change with temperature, shifting the relationship between image and force. Other tactile technologies measure changes in resistance, capacitance, or magnetic field. ReSkin, for example, uses a deformable magnetic skin (Bhirangi et al., 2022). The choice is therefore between useful measurements, spatial coverage, durability, and processing demands, rather than between a sensor that "has touch" and one that does not.



(a) Optical principle



(b) Sensor assembly

(c) Camera and LEDs

Fig. 9.6 Optical touch converts contact into an image of a deformable surface. The reflective coating and illumination allow surface shape to be recovered independently of the contacted object's optical appearance. Reproduced from Figure 2 of Yuan et al. (2017), under CC BY 4.0. The illustration describes the sensing principle, not the packaging of every modern fingertip

Force sensing complements this local picture. A six-axis wrist force-torque sensor measures three force components and three moments transmitted through the wrist. Strain gauges detect tiny deformations of an elastic structure. Calibrated combinations of their signals recover the load. The reading includes the tool's weight and inertial loading, which must be compensated when estimating contact force (ATI Industrial Automation, n.d.). A joint torque sensor measures twisting load about one joint axis, often on the output side of the transmission. Its location matters. A sensor upstream of a lossy gearbox does not measure the same load as one downstream. Multiple joint measurements can support estimates of external contact, but these depend on the robot's dynamics and contact assumptions. Specialized joint force-torque sensing has even supported touch localization over a robot's body (Iskandar et al., 2024). An ordinary single-axis torque sensor does not automatically provide that capability.

Current sensors offer another source of evidence already needed by the drive. A small shunt resistor produces a voltage proportional to current. Magnetic current sensors infer it from the surrounding magnetic field. For a suitable motor model, torque-producing current estimates motor torque. It does not directly measure force at the fingertips. Friction, acceleration, gravity, and transmission losses contribute to the same signal. Current remains valuable for control, overload detection, and learning about loading, provided these ambiguities are recognized (Muthalagu et al., 2025; Wensing et al., 2017).

The purpose of combining sensors is to resolve uncertainty that matters for an action. In the Jenga experiments of Fazeli et al. (2019), vision locates blocks and force measurements during interaction help infer how they can move. The contact supplied information unavailable from an image alone. A similar principle applies when opening a drawer or seating a connector. The robot must distinguish free motion from resistance, then decide whether to continue, change direction, or retreat. This is task-state observability in practical terms. The most useful sensor is the one that reveals the distinction in time to change the action, at a precision appropriate to the task.

9.2.4 Electric Motors and the Mechanics of Actuation

An actuator converts supplied energy into mechanical work. Electric motors are a practical default for many robot arms, grippers, mobile bases, and legged platforms because electrical power is readily distributed, motor drives are mature, and feedback can regulate motion and torque. These advantages depend on scale and duty. Hydraulic, pneumatic, and material-based actuators remain useful where their force capacity or mechanical behavior better fits the task.

Inside an electromagnetic motor, current in windings interacts with a magnetic field to turn a rotor. The stationary part is the stator. Sustained rotation requires the winding currents to change as the rotor turns, a process called commutation. In a brushed direct-current (DC) motor, physical brushes and a segmented commutator perform this switching. Such motors are straightforward to drive and remain useful in compact or inexpensive mechanisms, but the brushes wear and create electrical noise. A coreless brushed motor removes the iron core from the rotating winding, reducing rotor inertia and cogging. It still needs careful thermal design. "Coreless" describes construction, rather than a separate feedback control method (FAULHABER, n.d.).

Brushless permanent-magnet motors place commutation in the electronics. The rotor carries magnets and the stator carries windings. The drive energizes the phases according to rotor position. Eliminating brushes removes a wear mechanism and enables efficient, controllable drives, at the cost of more complex electronics. The labels brushless direct-current (BLDC) motor and permanent-magnet synchronous motor (PMSM) describe closely related motor families. In a common convention, BLDC designs are associated with trapezoidal back-electromotive force and block commutation, whereas PMSMs are associated with sinusoidal waveforms. Products and control methods overlap, so the label alone is insufficient to predict torque quality. Field-oriented control regulates current components relative to the rotor field, allowing smooth torque control over a useful operating range (Sidelsky, 2020; Muthalagu et al., 2025).

Packaging creates further options within these families. An inrunner places the rotor inside the stator. An outrunner rotates an outer shell, often providing a larger torque-producing radius. A

frameless motor supplies the rotor and stator for integration into the robot's own bearings and housing. A hollow center can accommodate cables, while different winding and rotor designs trade speed, torque, inertia, cooling, and space. These are engineering choices rather than a universal ranking. An easily accelerated rotor can benefit a responsive finger, whereas a large joint must also carry sustained load and reject heat. Figure 9.7 illustrates motor construction and feedback integration.

A stepper motor also uses electronically switched windings, but is designed to advance through discrete angular equilibria. Counting commanded steps can provide simple positioning in lightly loaded fixtures, dispensers, and laboratory mechanisms. The count is a command history, however, not proof of actual position. An overloaded open-loop stepper can lose steps. Microstepping makes the commanded motion smoother without guaranteeing correspondingly finer absolute accuracy. Closed-loop steppers add feedback, and the best choice depends on speed, load, efficiency, and required disturbance response (Oriental Motor, n.d.). For Physical AI, this distinction affects the data as well as the motion. A dataset should not silently treat requested steps as measured joint states.

The word servo describes feedback-controlled operation, not a unique electromagnetic motor type. A servo system combines a motor, a drive, a position or velocity sensor, and a controller. It may also contain a gearbox (Kollmorgen, 2020). A small hobby servo and an industrial brushless servo share this principle while differing greatly in precision, torque control, response, and access to measurements. For a learning system, the interface matters. Position-only commands offer different possibilities from current or torque commands, and a nominally identical target can behave differently under another drive's limits and gains.

The transmission then converts motor motion into joint motion. Planetary, strain-wave, and cycloidal gearboxes are transmission mechanisms, not types of motor. They allow a relatively fast motor to turn a slower joint with greater torque. Direct drive couples the motor to the load without reduction. Quasi-direct drive combines a high-torque motor with a modest reduction. Neither approach is free of trade-offs. Removing reduction requires the motor itself to supply more torque, often increasing its size or thermal burden. Under an idealized constant-efficiency approximation,

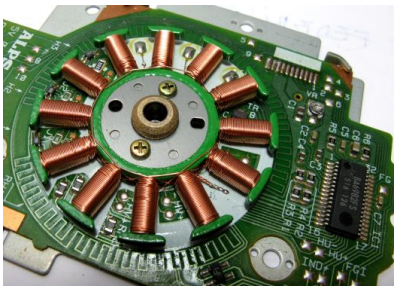
$$\tau_{\mathrm{j}} \simeq \eta_{\mathrm{gear}} N_{\mathrm{gear}} k_{\mathrm{t}} i_{\mathrm{q}}, \qquad \omega_{\mathrm{j}} = \frac{\omega_{\mathrm{m}}}{N_{\mathrm{gear}}}, \qquad I_{\mathrm{refl}} = N_{\mathrm{gear}}^2 I_{\mathrm{m}}, \tag{9.2}$$

where N_{gear} is motor speed divided by joint speed, η_{gear} is transmission efficiency, and $k_{\mathrm{t}} i_{\mathrm{q}}$ is electromagnetic torque in an appropriate torque-current convention. For a simple brushed motor, armature current replaces i_{q}. The last relation shows why a large reduction can make the rotor's inertia strongly felt at the joint. Actual output also depends on acceleration, friction, elasticity, and direction of power flow, so this equation is not an external-force measurement.

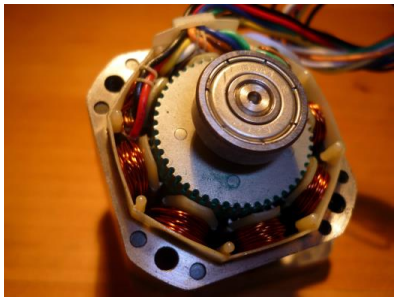
These effects become apparent when a foot lands or a hand meets an unexpected obstacle. A transmission that can be driven readily from the joint side is called backdrivable. Low reduction and low friction can improve this response and make motor current more informative about joint loading, as explored in the MIT Cheetah's proprioceptive actuators (Wensing et al., 2017). High-reduction joints can instead use output torque sensing and feedback to yield under external load, as in the DLR lightweight robot (Albu-Schäffer et al., 2007). A series elastic actuator places a spring in the force path. Its calibrated deflection estimates transmitted torque and cushions disturbances, while introducing additional dynamics that the controller must handle (Pratt and Williamson, 1995).

Peak torque is only part of the operating envelope. Winding heat grows approximately with current squared times resistance, so a joint may produce a large torque briefly but be unable to sustain it. At higher speed, the motor's back-electromotive force and the supply voltage limit the current the drive can establish. A policy that repeatedly asks for unavailable torque experiences delayed or incomplete movement. Temperature-dependent limits can make an identical command behave differently later in a task. Training and evaluation should therefore include the relevant torque-speed limits, thermal duty, friction, and control delay, rather than treating an actuator as an unlimited source of prescribed motion (Kafader, 2024; Gealy et al., 2019). In the ANYmal experiments of Hwangbo et al. (2019), a learned actuator model helps bridge this gap by representing the real

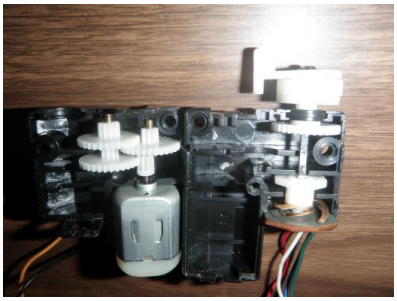
(a) Coreless DC motor winding



(b) Exposed brushless motor stator



(c) Stepper motor



(d) Opened hobby servo

Fig. 9.7 Motor construction and feedback integration. (a) A lightweight winding removed from a coreless DC motor. (b) Stationary windings of a brushless motor with its rotor removed. (c) A stepper motor, whose motion follows commanded magnetic alignments. (d) A hobby servo showing the motor, reduction gears, and output-position sensor. The servo is an integrated feedback system, not a separate electromagnetic motor family. Photos: (a) Avsararas, (b) DnaX, and (d) Jstapko, CC BY-SA 3.0; (c) Dolly1010, CC BY 3.0

response between commands and torque. The result shows why actuator and controller dynamics must be accounted for when transferring behavior from simulation to the physical robot.

For contact, the controller must also decide how strongly to resist position error. Impedance control makes a robot respond like a chosen spring and damper (Hogan, 1985). A softer response can absorb a small alignment error. A stiffer response may be needed to support a load. In a simple, slowly varying contact model,

$$F_{\text{contact}} \approx k_{\text{eff}} e_{\text{contact}}, \qquad k_{\text{eff}} = \left(\frac{1}{k_{\text{r}}} + \frac{1}{k_{\text{e}}} \right)^{-1}, \tag{9.3}$$

where e_{contact} is the position error along the contact direction, and the robot and environment stiffnesses, k_{r} and k_{e}, act in series. An effective stiffness of 10^5 N/m turns a 1 mm error into about 100 N. Lower stiffness reduces this quasi-static force, although collision severity also depends on mass and speed. Learning can select compliance together with motion (Martín-Martín et al., 2019; Hou et al., 2025). Fast feedback must still realize that choice within the actuator's limits. A position-controlled robot can instead use force feedback to adjust its position target, an approach called admittance control. This is how a learned intention to "insert gently" acquires a measurable physical meaning.

9.2.5 Fluid Power, Compliance, and Functional Materials

Electric motors do not cover every useful form of actuation. Hydraulic systems use pressurized liquid to move pistons or hydraulic motors. Large force can be generated in a compact actuator, making fluid power attractive for demanding loads, but the complete machine also needs pumps,

valves, seals, and fluid routing. Valve dynamics, leakage, friction, and fluid temperature affect the response a controller must learn or model. Electrically powered pumps and local hydraulic circuits can combine electrical distribution with hydraulic force generation. "Hydraulic" does not necessarily mean a remote central pump.

Pneumatic systems use compressed gas. Air can inflate a soft finger that wraps around an object with relatively simple mechanics, but its compressibility, flow restrictions, and material hysteresis complicate precise positioning. Pressure is useful feedback, yet does not uniquely determine the fingertip force when shape and contact change. Both fluid technologies therefore need sensing appropriate to their dynamics, rather than relying on pressure commands alone (Rus and Tolley, 2015; Laschi et al., 2016).

Functional materials offer an emerging route to combining actuation, sensing, and compliance within the body. They remain under active investigation alongside the motor-driven platforms discussed above. Thermal and electrochemical artificial muscles offer distinctive combinations of strain, force, and response time (Mirvakili and Hunter, 2018), but their value for physical intelligence depends on the interaction they enable. Hydraulically amplified self-healing electrostatic (HASEL) actuators, for example, use electrostatic forces to redistribute a liquid dielectric inside flexible pouches, producing motion. Changes in capacitance also reveal deformation (Acome et al., 2018). By integrating driving electronics, capacitive self-sensing, and computation, Ly et al. (2021) demonstrate feedback control of a Peano-HASEL-powered arm. The actuator therefore supplies both mechanical work and a measurement that can guide the next action.

Figure 9.8 connects this component behavior to task feedback. A compliant finger may accommodate an object's shape mechanically, while its deformation provides evidence for adjusting the grasp. Yet compliance alone does not reveal whether the object is secure, slipping, or being compressed excessively. That distinction requires calibrated sensing and a controller whose response remains useful under changing loads. High-voltage electronics, nonlinear mechanics, material drift, and fatigue must be considered together with control bandwidth and replacement needs. Progress should therefore be measured through repeatable task performance, including sensing, power delivery, and maintenance, as well as the properties of an isolated material sample.

Even without an unusual actuator, the body's structure can simplify control. A compliant insertion fixture can guide a misaligned peg, and an underactuated hand can conform to an object using fewer motors than moving joints (Whitney, 1982; Dollar and Howe, 2010). This is a practical form of sharing the work between mechanics and computation. The same softness that accommodates contact can make shape harder to estimate. Electronic skins and learned sensing models help address that problem (Shih et al., 2020). For physical intelligence, the useful question is which part of an interaction the body can handle reliably, and which changing circumstances still require observation, inference, and a new action.

The following cases examine a hand and a humanoid as complete sensing and actuation systems (Figure 9.9). Section 9.6 then compares three dexterous hands at the level of actuation, tactile sensing, and demonstration hardware.

9.2.6 Case Study: RobotEra XHAND 1

RobotEra's XHAND 1 illustrates the measurement-motion loop at the level of one component. Its host sends joint targets, controller gains, and torque limits. The hand returns joint positions, motor-current information, temperature, and error status, together with contact measurements when the optional fingertip sensors are fitted (RobotEra, n.d.; 2024). Section 9.6.3 compares the mechanism, loading capacity, and communication timing with other hands. The question here is how these signals support correction during a grasp.

Vision can locate an object before contact, while joint feedback records how the fingers move toward it. Once a finger meets the object, target position alone no longer describes the interaction. Controller gains, effort limits, and transmission mechanics determine the load. Tactile measurements, when available, can reveal changes in contact that require a revised finger target. Motor

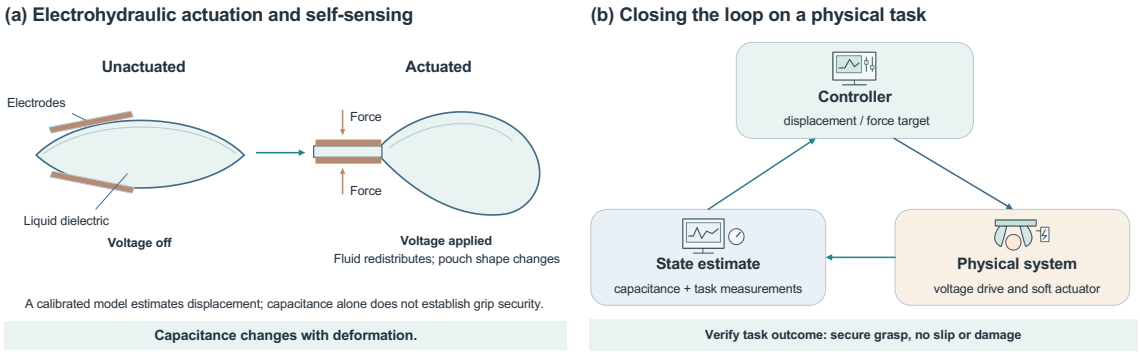


Fig. 9.8 Functional materials become useful to physical intelligence through measured interaction. (a) In a conceptual HASEL cross-section, electrostatic attraction redistributes liquid and deforms the pouch. (b) Self-sensing contributes to feedback, while task measurements establish whether the interaction achieved its purpose (Acome et al., 2018; Ly et al., 2021). Shapes are illustrative, and the diagram does not imply that capacitance alone measures contact force

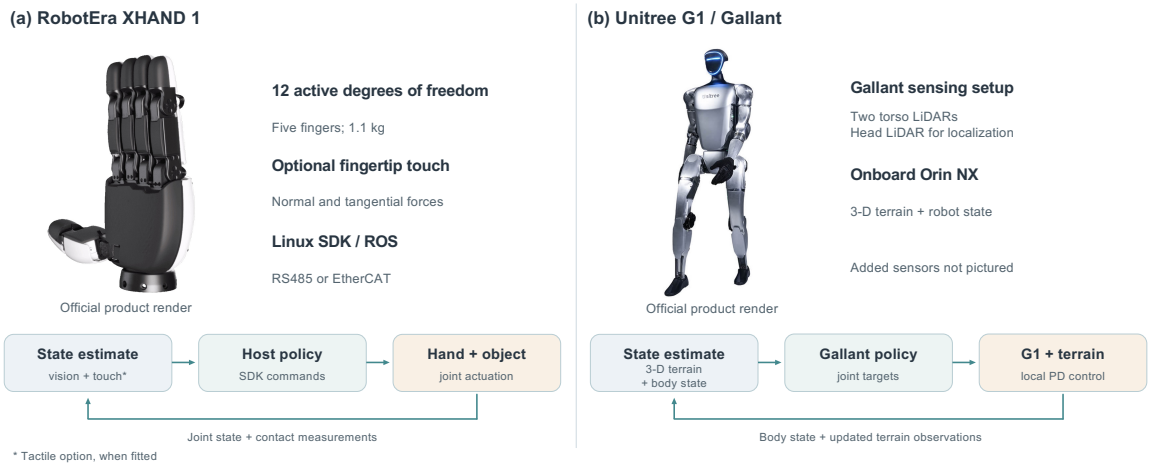


Fig. 9.9 Product hardware and the feedback needed for Physical AI. (a) RobotEra XHAND 1 links visual and optional tactile observations to commanded finger motion and renewed contact measurements. (b) Gallant on Unitree G1 links 3D terrain perception to learned joint targets and local control (Ben et al., 2026). Official product renders from RobotEra and Unitree Robotics. The G1 image shows the base platform, without Gallant's additional LiDARs. The control diagrams summarize information flow, and the products are not shown to scale

current supplies useful actuator telemetry but does not locate that contact. The host must align these observations with the commands that produce them and respond while the grasp is still recoverable.

The same boundary applies to transferred demonstrations. ManipTrans uses Realman arms and Inspire hands, fitting 12 simulated hand coordinates to six hardware coordinates before smoothing and replay (Li et al., 2025a). Applying such motion to XHAND requires a new mapping through its joint axes, ranges, and contact surfaces. Matching a pose vector is only one step. The deployed hand must sustain the intended contact under load. Section 9.6.4 examines how demonstration hardware and collection interfaces address this correspondence.

9.2.7 Case Study: Unitree G1 and Perceptive Locomotion

Gallant illustrates how a humanoid's useful workspace depends on the sensing and control system built around its body (Ben et al., 2026). The task is to move a 29-degree-of-freedom Unitree G1 through stairs, gaps, narrow passages and overhead obstacles. Two torso-mounted LiDARs provide complementary views of the surrounding geometry. A head-mounted Livox Mid360 supports localization. A voxel representation preserves the vertical structure that a ground-height map discards, allowing the controller to account for both footholds and body clearance.

The software connects point-cloud processing, proprioception and learned control on an onboard NVIDIA Orin NX. A convolutional encoder extracts spatial features, which are combined with the robot state to generate joint commands. Sensor placement and the representation used during training must therefore remain consistent at deployment. Changing the visible space changes the information available for each action. In the initial deployment, LiDAR traffic and robot-state messages share a software pipeline and incur substantial communication delay. Restricting transmitted point clouds and using shared memory reduce this overhead (Ben et al., 2026). These changes concern the age of the feedback reaching the controller, rather than the size of the policy alone.

The real-robot evaluation uses 15 trials per terrain and compares the full system with perception and training ablations (Ben et al., 2026). This provides evidence for the integrated locomotion loop within the tested environments. On approach to a low ceiling, observed clearance and current body configuration must jointly determine posture. A correct map delivered too late cannot support that correction. The useful capability belongs to the complete perception-action loop.

9.3 Device-Edge-Cloud Collaboration and Real-Time Communication

9.3.1 Distributed Architecture

The purpose of Physical AI is to enable intelligent agents, under finite computational resources, to interact continuously with the physical environment through perception, decision-making, and action and to accomplish a given physical objective in real time, effectively and safely. Resources in computation, energy, storage, communication, and control cycles must be allocated across perception, state estimation, prediction, planning, policy generation, and execution control at different nodes. These functions have different temporal scales, resource requirements, and spatial distances. This creates a central challenge for Physical AI. High-level computation tends to be low-frequency, computationally intensive, and spatially remote, whereas physical perception and interaction must be high-frequency, computationally limited, physically proximate, and temporally continuous.

The device-edge-cloud distributed architecture provides a way to balance these requirements (Figure 9.10). It decomposes a unified perception-decision-action loop according to distinct *timescales, spatial scopes, and responsibility boundaries* (Lee et al., 2026), organized around *validity deadlines, distributed networks, and responsibility horizons*. The *device* connects directly to the body, sensors, and actuators. It handles the fastest state perception, servo control, contact responses, and safety constraints so that the minimum safety-critical loop does not depend on the network. The *edge* operates at the physical site with greater computational resources. It performs multimodal fusion, local world modeling, policy inference, trajectory generation, action-chunk generation, and coordination of multiple bodies, transforming site-level state into executable commitments that remain valid. The *cloud* operates over longer timescales across tasks, devices, and sites. It supports goal decomposition, experimental design, long-term memory, experience accumulation, model updates, and cross-site orchestration. Computation can migrate according to task requirements, but physical responsibility cannot migrate arbitrarily. The closer a responsibility is to a physical consequence, the more it must be assumed by a trusted node close to the body.

Accordingly, device-edge-cloud collaboration is better understood in terms of *migratable responsibility boundaries* rather than as a fixed three-layer computational hierarchy. When real-time communication is healthy, prediction, planning, and policy generation can migrate toward the edge or cloud in exchange for stronger model capabilities. When real-time communication becomes abnormal, responsibility must contract toward the physical object and switch to validated degraded modes.

Furthermore, device-edge-cloud collaboration exchanges node state and commitments with physical semantics. At least three categories of state should be distinguished. *Physical state* includes pose, joint state, force, contact, limits, environmental conditions, and execution mode. It must carry sampling time, coordinate frame, units, and effective range. *Action state* includes action chunks, task phase, resource occupancy, execution progress, and version information. It must distinguish among "generated," "sent," "received," "acknowledged," and "physically acted upon." *Evidence state* includes credit, measurement identity, experimental conditions, execution confirmation, anomalies, and recovery records. It determines whether a result is sufficiently grounded to be treated as a fact by subsequent reasoning. Confusing these categories produces characteristic failures in Physical AI closed loops, including treating a plan as a fact, transmission as execution, or an old observation as the current state.

The following sections address four questions within this Physical AI framework. What information must the device maintain, and what must it guarantee? How does the edge transform site-level state into valid action? How does the cloud provide low-frequency, high-level intelligence? How can real-time communication ensure that information transferred across these responsibility boundaries remains physically valid?

9.3.2 Device-Side Computing

The device is the computational and control node closest to the point of physical action in the Physical AI loop. It typically includes sensor interfaces, actuator interfaces, body-state estimation, safety control, local policies, and an adaptation layer for real-time communication with higher-level systems (Ortega-Kral et al., 2026). Device-side computing must ensure that the physical system does not enter an unacceptable state simply because it is waiting for remote information, even under the worst real-time communication conditions. Hosting the largest model is secondary to this requirement.

The device must first establish a *time-aligned multimodal state*. Cameras, encoders, force sensors, tactile sensors, and motor feedback do not inherently share the same sampling instant. A visual-frame timestamp may correspond to exposure or acquisition time. Joint state may originate from a controller cycle. Force signals may contain additional delay introduced by filtering, transmission, and

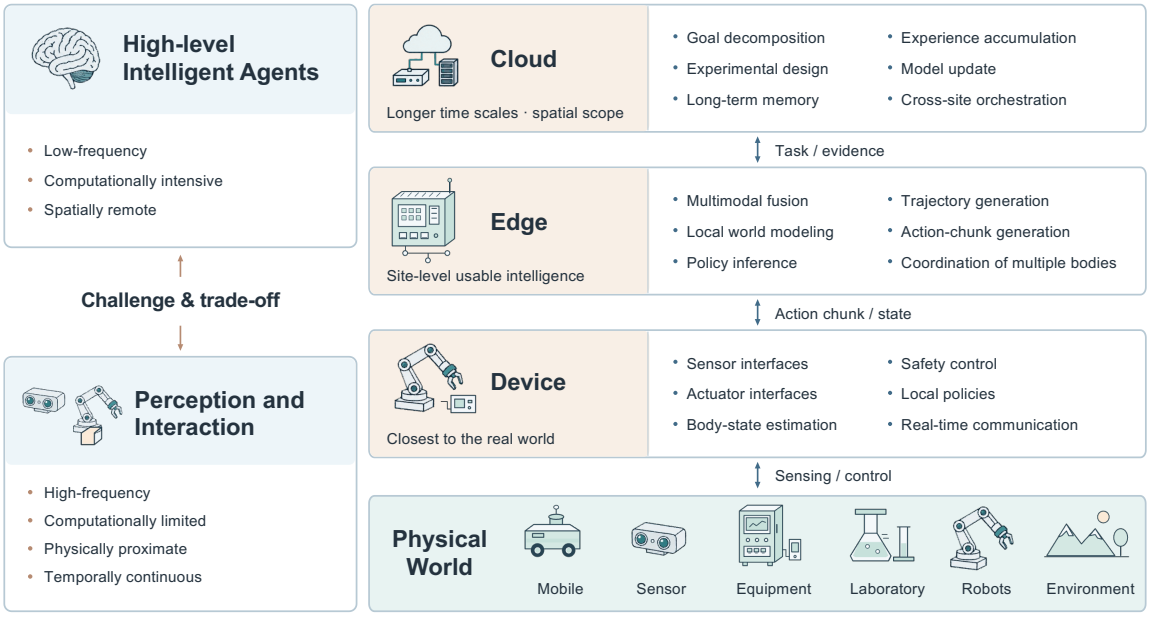


Fig. 9.10 The challenge of physical AI and device-edge-cloud collaborative Architecture

mechanical response. Device state should therefore be represented by states carrying timestamps, source identities, safety attributes, spatial coordinate frames, units, and quality indicators, rather than simply by a collection of "current values." Only then can the system determine whether different trusted observations describe the same contact event and, when a sensor is missing, represent the state as "unknown" rather than incorrectly reusing the previous frame.

The device must also maintain a *local execution state machine*. The state machine should at least distinguish standby, preparation, execution, constrained execution, pause, fault, and recovery, while specifying which actuator actions are permitted in each state. Its purpose is to make device-native safety enforcement independent of interpretation by a high-level model. A policy model may recommend "continue grasping," but if a joint is approaching its limit, force exceeds a threshold, or the real-time communication lease has expired, the device-side state machine must be able to reject the action. Policy output and execution permission are distinct. The policy proposes an action, whereas the safety layer determines whether it is currently allowed.

For real-time control, the device typically uses a fixed-period or event-triggered local closed loop. The real-time communication layer provides references, targets, parameters, or action chunks. It should not require every servo cycle to wait for a remote system to generate the next joint command. This prevents network jitter from entering the innermost control loop and turns network failure into an explicit operating state rather than an accident handled only after it occurs. Based on the remaining validity of an action chunk, local state, and safety rules, the controller can continue execution, execute in a degraded mode, perform a controlled stop, or stop immediately, thereby implementing device-native safety.

Device computation is also constrained by physical considerations. Larger models imply higher memory consumption, power draw, thermal load, and computational latency, and may also alter the weight distribution, power capacity, and onboard compute budget of a mobile system. Model size therefore cannot be discussed independently of the control cycle. For tasks requiring rapid contact feedback, a more complex but slower model may be unsuitable for direct participation in the execution loop. It can instead support slower action generation, candidate selection, or anomaly interpretation, while the fast loop remains the responsibility of lightweight models, controllers, and safety logic.

9.3.3 Edge-Side Computing

Edge-side computing provides site-level computation and coordination. It is closer to the physical process than the cloud while offering substantially more computational and coordination capability than the device. Edge nodes are well suited to policy inference, local world models, trajectory generation, resource coordination, and multimodal fusion (XPolicyLab Community et al., 2026). These computations, however, have physical control value only when their outputs still correspond to the current state of the site.

In this architecture, an *action chunk* serves as a time-bounded execution commitment with explicit semantics. An action chunk should include, at minimum, the target device or body identity, reference time, validity interval, sequence number, control units, execution mode, and necessary constraints. The device can then determine whether the action still belongs to the current task, whether it has expired, and whether it has been superseded by a newer version. The key metric for an edge closed loop is not average inference time, but the *end-to-end interval between the observation reference time and the instant at which the action begins to take effect*. This interval must be shorter than the validity deadline permitted by the physical action, rather than merely satisfying a requirement that model inference itself be fast enough. Loss of an action chunk must not be interpreted by default as "continue executing the previous chunk," because doing so hides communication failure inside the control state. A more appropriate execution rule uses a *lease*. When the lease expires without a new valid commitment, the device enters a predefined safe state.

The edge should also keep the *world model decoupled from the controller*. A world model may predict "what might happen if this grasp continues," but that prediction does not thereby acquire direct control authority. Its output can condition action generation, candidate-action selection, or risk es-

timation, while final execution must still pass through the device-control interface and safety state machine. This allows independent checks before execution and lets the world model be upgraded independently.

In site-level distributed collaboration, the edge is better suited to maintaining *resource semantics* than to replicating all internal device data. For example, when multiple robots compete for the same tool, the edge needs information about tool occupancy, reservation expiry, task priority, and current execution phase. It does not need to continuously aggregate high-frequency joint data from every robot. Resource state can be encoded using leases, version numbers, or monotonic sequence numbers, making "who owns the resource" a state that can be checked rather than a conclusion inferred from whichever message arrives last.

9.3.4 Cloud-Side Agents

Cloud-based agents operate on the longest time scale and are suited to processing information across experiments, devices, and sites. They can maintain long-term memory, compare historical results, generate scientific hypotheses, plan resources, and select subsequent tasks, but should not directly assume millisecond-scale contact control. Communication between the cloud and the physical site therefore primarily concerns *task state and evidence state*, rather than the next joint command.

The cloud must distinguish *plan, request, confirmation, and fact*. A plan describes what is intended to happen. A request specifies what the site is asked to execute. A confirmation indicates that the site has accepted the request. A fact originates from an identified, timestamped physical measurement. These four concepts cannot substitute for one another. For example, when the cloud generates a task to "close the valve," this does not mean that the valve has closed. Even if the actuator reports "completed," sensor evidence may still be required to confirm the final physical state. Only states supported by site-level measurements should enter the evidence set used for subsequent experimental reasoning.

Accordingly, communication between cloud and site is better implemented through *event-driven state synchronization* than through continuous replication of all raw signals. High-frequency data can be aggregated, compressed, or selectively uploaded on site. The cloud receives task-relevant measurements, summaries, anomalies, and evidence. For experiments that must be reproducible, uploaded records must also carry identities for samples, devices, experimental conditions, and software versions. Otherwise, historical results become difficult to interpret reliably.

When the cloud becomes unavailable, an action that has already started should not be extended indefinitely while waiting for a cloud response. The site should complete, pause, or stop according to device- and edge-level state. After recovery, the cloud should first re-read site-level facts and only then select the next action. In particular, "the last transmission succeeded" must never be interpreted as "the experiment has completed."

9.3.5 Real-Time Communication

The central metric for real-time communication in Physical AI is the physical validity of information when it arrives relative to the physical process, rather than throughput alone. Average latency, average bandwidth, and average packet-loss rate are therefore insufficient to characterize a physical closed loop. At minimum, one must also consider latency bounds, jitter, queueing time, observation age, action validity, reordering, duplication, and the duration of disconnection (Sajid et al., 2026).

A simple serial end-to-end real-time constraint (Diao et al., 2025) can be written as

$$T_{\text{e2e}} = T_{\text{sensor}} + T_{\text{encoder}} + T_{\text{uplink}} + T_{\text{compute}} + T_{\text{downlink}} + T_{\text{decoder}} + T_{\text{actuator}}. \tag{9.4}$$

The terms correspond, in order, to the stages of one remote closed loop, namely on-device sensing, observation encoding, uplink transport, computation at the edge or in the cloud, downlink

transport, command decoding, and the onset of actuation. For one observation and its resulting action, the stage intervals cover sensing start to actuation onset without gaps or overlaps, including any queueing time. Their sum is T_{e2e}, with stage boundaries specified and timestamps mapped to a common clock domain. T_{deadline} is not an arbitrary parameter chosen by the real-time communication system, but a deadline determined by the physical process, and T_{e2e} must be strictly less than it. Contact, balance, and high-speed motion may require deadlines close to the control cycle, whereas experimental steps and resource scheduling may tolerate delays of seconds, minutes, or even longer. A distributed physical AI can therefore contain multiple real-time classes simultaneously. A single "real-time/non-real-time" label cannot adequately describe all of its communication.

In device-edge-cloud collaboration, asynchronous execution and queueing affect which observation an action uses. The *age of information (AoI)* (Yates et al., 2021) complements end-to-end latency by measuring information freshness. At an evaluation time, standard AoI is the interval since generation of the freshest state update available at the receiver. The observation age of an action measures the interval from generation of the observation actually used for that action to the action's execution. Both age calculations use timestamps mapped to a common clock domain. For the same update stream and receiver, the two ages coincide at action execution if the action uses the freshest available update. The action's observation age is greater if it uses an older update. These quantities should not be conflated with the simple serial latency estimate above. A backlog of action chunks can make nominal network latency appear normal while delaying execution and increasing the age of the observation used by the current action, potentially beyond its validity period. For real-time action generation, skipping queued observations that are no longer useful to the current control computation can prevent queue buildup. Older observations may still be retained for state estimation, sensor fusion, or historical analysis. For example, a model requiring only 5 ms for inference can still produce an unusable action if its input waits 100 ms in a queue. Conversely, a slower model may be entirely acceptable for second-scale task planning.

The action side requires complementary execution rules. An action prefix that has already begun execution generally cannot be arbitrarily rolled back, whereas an unexecuted suffix can be regenerated according to new observations. This distinction avoids excessive action oscillation while allowing new information to modify future behavior. When an action chunk expires, the system should stop extending the commitment rather than repeatedly copying the last action.

Real-time communication must also handle *semantic loss*. Undetected message loss can leave the receiver with an incorrect view of task or safety state. Losing a frame used only for visualization may be harmless, whereas silently losing an event indicating that a device has entered a safe state could cause the cloud to schedule the next experiment incorrectly. Communication protocols must therefore distinguish discardable information from information that requires acknowledgment, and provide the latter with confirmation, sequence numbers, timeouts, and idempotent semantics (Zhang et al., 2024; Klüner et al., 2025; Chovet et al., 2024; Liang et al., 2023b).

In multi-body collaboration, real-time communication should likewise not force all bodies to share a single high-frequency state. When two robots compete for the same tool, the edge must make tool-occupancy and release events visible *before a conflict occurs*. If occupancy information arrives late, its effect is equivalent to having no information. Force feedback within a body should remain in a local closed loop, because elevating it to an inter-body network only increases real-time communication latency and the risk of failure.

A real-time communication system with explicit responsibility boundaries should provide at least four capabilities, including *timestamping, state versioning, validity control, and explicit fault representation*. Timestamping identifies when a state was observed, versioning identifies the plan or policy that produced it, and validity control determines whether it remains usable. Explicit fault representation identifies who is responsible for entering the safe state when the information is no longer usable. Together, these capabilities establish state and responsibility semantics for maintaining the Physical AI closed loop in addition to transporting messages.

9.3.6 Self-Driving Laboratories

A self-driving laboratory (SDL) provides a validation setting for device-edge-cloud collaboration (Gao et al., 2025). A typical dry/wet experimental SDL simultaneously contains millisecond-scale device control, second-scale experimental operations, and minute-to-hour-scale scientific decision-making. Instruments and robots must execute rapidly. The site must continuously maintain the state of samples, devices, and the environment, while higher-level scientific-discovery agents typically operate in a relatively *predefined experimental world*, running a "hypothesis, execution, measurement, and next selection" loop. The objectives, workflow, instruments, and basic interpretive framework for measurements are specified before an experimental campaign begins, and the system primarily optimizes parameters and actions within those predefined boundaries.

An open-governance architecture for SDLs emphasizes a transition from *closed-loop optimization in a predefined world* to *open-world scientific discovery*. This motivates designing the InternLab platform (Li et al., 2026e) to advance Physical AI from a single-laboratory closed loop to a governable capability network across laboratories and institutions.

InternLab's device-edge-cloud distributed architecture organizes physical capabilities into a governed network without centralizing all laboratories into a single cloud. Scientific agents can use it to discover and compose experimental capabilities dynamically from nodes across the devices, sites, and cloud systems of multiple laboratories and institutions. Its three mutually coupled layers are *Physical Intelligence & Distributed Network* (the capability layer), *Lab OS Runtime* (the execution and orchestration runtime connecting the capability and governance layers), and *Native Governance* (the governance foundation).

The *Capabilities Network Layer* transforms "equipment owned by a laboratory" into "capabilities callable by a network." Robotic manipulation, scientific instruments, analytical models, and experimental skills are no longer merely private components within a site. They can be described as capabilities with explicit applicability conditions. The *Capability Graph* describes which nodes can provide which capabilities under which conditions. *Scheduling & Routing* selects qualified nodes according to sample state, temporal constraints, instrument availability, and governance conditions. The *Composition Engine* combines multiple capabilities serially or in parallel into an experimental route.

The *Lab OS Runtime Layer* transforms continuously changing scientific intent into executable physical actions and brings the resulting outcomes back into the scientific loop. Its core can be abstracted into four runtime functions, namely *Task Routing, Live State, Authorization Checks, and Trace Return*. Task Routing delivers tasks or action chunks to bodies, instruments, or sites with the required capabilities, rather than requiring every servo cycle to wait for a remote command. Live State maintains site state with timestamps, coordinate frames, and validity intervals, enabling higher-level decisions to determine whether observations remain fresh. Authorization Checks verify the necessary governance prerequisites before an action enters the physical system. Trace Return brings execution results, anomalies, and recovery information back into the next round of scientific reasoning.

The *Native Governance Layer* is the foundation that enables InternLab to move from a conventional automation network to an open experimental network. InternLab explicitly contains at least five inseparable conditions, including *Permission, Safety, Calibration, Provenance, and Credit*. Permission specifies who may invoke a capability, for what purpose, over which sample scope, and within which risk boundary. Safety is not a local property of a single action, but an end-to-end property spanning hardware, environmental monitoring, action validation, and fail-safe recovery. Calibration determines whether an instrument or sensor is currently in a measurement state that can be trusted. Provenance records sample identity, protocol version, data source, and operational chain so that cross-site evidence can be audited and reproduced. Credit preserves attribution for contributions across institutions. Most importantly, the design goal is not merely "post hoc auditing," but governance-native execution. Governance conditions are part of the interface when a capability is defined and invoked, and they propagate together with the execution context. When any prerequisite fails, the capability should be rejected, degraded, re-authorized, or escalated to human handling.

The two InternLab pilot deployments (Li et al., 2026e) demonstrate that the device-edge-cloud collaboration and responsibility model extends to an *open spatial scale*. The local safety loop at the device, site-level state and resource coordination at the edge, and long-horizon scientific reasoning in the cloud therefore do not disappear when collaboration becomes cross-site. They become embedded within a network of capability discovery, task routing, governance checks, and evidence return. The evolution from SDL to the InternLab design extends Physical AI from a *single, predefined, closed experimental loop* to an *open-world loop that spans bodies, sites, and institutions and supports re-planning, re-composition, and governance-native execution*. This makes device-edge-cloud collaboration and real-time communication operational infrastructure for open-world, distributed Physical AI.

9.4 Software Stacks and Development Infrastructure

A robot carrying a vial cannot wait for a language model to finish reasoning before responding to a slipping grasp. Nor does a completed arm trajectory establish that the vial reached the correct rack. These two observations define the software problem. Physical regulation must continue at the rate required by the body, while task execution must remain accountable to measured outcomes. Software connects these timescales through interfaces that specify what a component may command, what evidence it consumes, and what happens when that evidence or the component itself becomes unavailable.

Section 9.3 establishes where computation can run across the device, edge, and cloud. This section examines the software boundaries that make that distribution workable. The architecture comprises nested feedback loops, hardware and data interfaces, a runtime for controllers and learned policies, and a task execution framework. This chapter uses *Embodied Intelligence Harness* as its organizing term for the last of these. It brings together established mechanisms for skill composition, state management, execution monitoring, and recovery. The term denotes their integration for this chapter rather than a new middleware standard.

9.4.1 Layered Software Architecture and Physical Interaction Loops

Robotics software separates perception, estimation, planning, and control because these functions have different computational and physical requirements. Robot Operating System 2 (ROS 2) provides a widely used framework for composing such components through explicit interfaces (Macenski et al., 2022). Learning changes what can be implemented inside a component, and foundation models expand how tasks can be specified and decomposed. Neither change removes the need to assign ownership of state, control authority, and failure responses. A hierarchy is useful only if its feedback paths and failure boundaries are equally explicit.

Figure 9.11 distinguishes three loops. The *task loop* interprets instructions, selects skills, checks their applicability, and evaluates their effects. It can revise a plan when a rack is occupied or an object's identity is uncertain. The *local control loop* regulates motion or contact using measurements available on the device. Its execution period follows the plant dynamics and the controller design. The *protection path* detects conditions requiring intervention and can inhibit or limit actuation within its specified response time. It may use drive electronics, a separate controller, or other independently provisioned hardware. These functions can exchange evidence without sharing a single execution path.

The task loop admits a request such as "transfer vial A to slot B" while the local controller processes encoder samples and motor updates. At admission, the executor assigns a skill instance a lease, which grants authority to issue task commands until a deadline on the executor's monotonic clock. Each command or action chunk also has an expiry determined by its intended execution interval and the validity of its observations. The executor applies task commands only while both the lease and the command remain valid. A new policy output cannot renew the lease. An admitted segment must fit within this joint interval. The executor begins its preconfigured fallback no later than expiry, allowing for detection and switching delay. That fallback has separate local authority and a specified completion or holding limit.

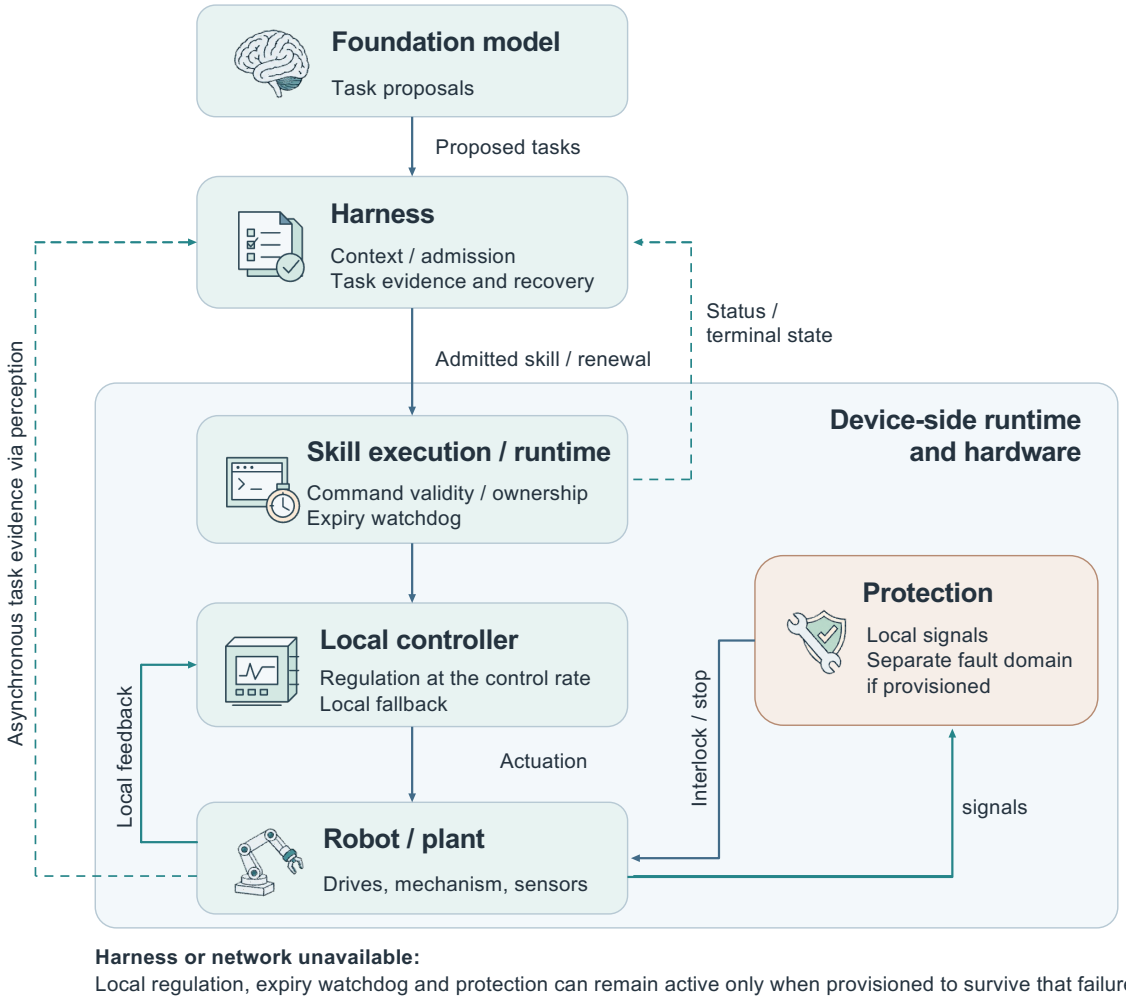


Fig. 9.11 Nested feedback loops with distinct control authority. Task proposals pass through the Harness and a skill interface before reaching local execution. Sensor feedback closes the control loop directly on the device. A separate protection path can inhibit actuation. Dashed paths carry perception evidence and executor status to the Harness. Protection-event reporting to the supervisor is omitted for clarity. Intervention uses the local path

Crash, stall, and network partition exercise this boundary differently. A crashed Harness stops renewing its lease. A stalled task monitor must also stop renewal, even if a separate heartbeat thread is alive. Renewal follows a completed monitoring cycle with valid evidence. A partition can leave both sides running with divergent state. The executor therefore checks skill identity, supervisor epoch, sequence, and freshness before accepting a renewal. One implementation uses an executor-issued challenge with a local response deadline. Delayed responses cannot start a fresh lease merely by arriving. Expired or superseded authority requires re-admission. The lease duration must accommodate the admitted renewal delay while leaving time for the tested fallback. Until expiry, locally available regulation and protection continue within the existing authority.

The correct fallback depends on the physical state. A moving base may brake. A loaded gripper may need to maintain a limited holding force. A gravity-loaded joint may require controlled braking rather than abrupt removal of torque. Holding has thermal and energy limits and needs an escalation or unloading procedure. If sensing or the controller fails, an independent protec-

tion channel handles the faults assigned to it. Merely placing two processes on the same computer does not establish independence from kernel failure, power loss, or resource exhaustion. A task-level cancellation request is not an emergency-stop mechanism. Industrial implementations specify and validate the safety-related control and drive functions under applicable standards, including ISO 13849-1 and IEC 61800-5-2 (International Organization for Standardization, 2023a; International Electrotechnical Commission, 2016). The required functions follow the machine's risk assessment.

Runtime assurance provides a related logical architecture. An advanced controller operates while a monitor retains authority to transfer control to a verified fallback under stated assumptions (Desai et al., 2019). Its guarantees depend on the plant model, measurements, recovery region, and timely execution of the monitor and fallback. Implementing that logic in a separate fault domain is an additional engineering decision. The local sensing, compute, power, and actuation paths must survive the faults for which continued operation is claimed. This separation preserves the autonomous device loop emphasized in Section 9.3 while slower layers improve task selection and recovery.

9.4.2 Hardware Abstraction, Sensor Data, and Robotics Middleware

A hardware interface should hide transport details while exposing the physical properties on which correct action depends. The `ros2_control` framework, for example, separates controllers from hardware components through state and command interfaces, with a controller manager coordinating the read-update-write cycle (ros2_control Development Team, n.d.). An interface named "position" is still incomplete without joint units, coordinate conventions, limits, supported modes, calibration state, and the conditions for switching modes. A simulated actuator and a physical drive can satisfy the same software signature while differing in delay, saturation, compliance, and failure behavior. Closed-loop substitution consequently requires testing the dynamics as well as the API.

Sensor interfaces need similar precision. An observation should carry its acquisition time, clock domain, frame, calibration identity, and validity information alongside the measurement. Covariance or another uncertainty representation is needed when the estimator relies on it. Receipt time alone conceals camera exposure, transport, and queueing delays. Combining a recent force sample with an old image can create a plausible state that never existed. The estimator must either align the measurements to a common time, propagate them using an explicit motion model, or mark the combination as unsuitable for the intended decision. Missing and invalid measurements must remain distinguishable from a measured zero.

Sensor validity can be expressed on the executor's clock. Let $\hat{t}_s$ be the mapped acquisition time, with a clock-mapping error bounded by ϵ_{sync}. The estimated age of this particular observation at use time t, rather than standard AoI for the latest available update, is

$$\Delta_{\text{age}}(t) = t - \hat{t}_s, \qquad \Delta_{\text{age}}(t) + \epsilon_{\text{sync}} \leq \Delta_{\text{age}}^{\max}, \tag{9.5}$$

where $\Delta_{\text{age}}^{\max}$ is a configured age limit for the particular observation and use. It can differ for a cached object label and a force estimate. The test applies at admission and at each intended actuation time, so age continues to grow across an action chunk. For commands that depend on this observation, it limits expiry to $\hat{t}_s + \Delta_{\text{age}}^{\max} - \epsilon_{\text{sync}}$. Additional constraints may shorten the interval. If the clock mapping or its error bound is unavailable, the receiver needs another validated freshness mechanism or must reject the observation for that use. ROS 2 distinguishes ROS time, which can follow simulation, from steady time (Foote, 2018). Physical watchdogs and leases use the latter. Simulation tests must state how their virtual timing maps to that mechanism.

Middleware carries these interfaces but does not supply their physical meaning. ROS 2 offers topics for streams, services for request-response interactions, and actions for longer operations with feedback and cancellation (Macenski et al., 2022; Biggs et al., 2020). The Data Distribution Service (DDS) specifies data-centric publish-subscribe communication and quality-of-service (QoS) policies, including reliability, history, deadline, and liveliness (Object Management Group, 2015). Zenoh combines publish-subscribe with query-based data access, and `rmw_zenoh` integrates it be-

neath ROS 2's middleware interface (Eclipse Zenoh, n.d.; ROS 2 Project, n.d.). These technologies occupy different layers. ROS 2 defines the action protocol above the underlying DDS or Zenoh transport.

QoS choices should follow the consequences of delay and loss. A high-rate state stream may benefit from a shallow queue that favors recent samples. A configuration transaction may require acknowledged delivery. Retransmission can improve delivery while increasing age, and a growing queue can turn a responsive estimator into a consumer of historical data. DDS deadline and liveliness events make some communication failures observable. They do not stop a motor or prove that a task is progressing. Backend support also differs. The Jazzy implementation of `rmw_zenoh` does not implement deadline or lifespan policies. A design that depends on those policies must verify the selected release and enforce the necessary command-expiry behavior locally (ROS 2 Project, n.d.).

The resulting boundary is practical. Middleware transports typed information. The application establishes freshness, ownership, and permissible use. A command should identify its producing skill and sequence, its coordinate and control modes, and its validity interval. The receiver rejects incompatible or expired commands before they enter the actuator path. Configuration changes are versioned and checked before activation. These conventions allow a developer to distinguish a transport failure from a stale observation, a conflicting controller, or an invalid request, each of which calls for a different response.

9.4.3 Real-Time Runtimes and Learned Policy Integration

A controller needs its next input before a deadline, whereas a learned policy may have variable inference time. Running both in one callback chain makes physical timing depend on model execution, memory allocation, and accelerator contention. ROS 2's real-time guidance identifies page faults, dynamic allocation, and indefinitely blocking synchronization as hazards in a time-critical path (Kay, 2016). Deployment therefore separates the bounded control cycle from inference, logging, visualization, and network-facing task services. Separation may require dedicated threads, priorities, memory discipline, processor allocation, or a separate controller. The necessary arrangement depends on the faults and interference the system must tolerate.

For a periodic local update, a useful timing budget is

$$C_{\max} + B_{\max} + J_{\max} < T_{\text{deadline}} \leq T_c, \tag{9.6}$$

where $C_{\max}$ bounds computation and input/output (I/O) on the selected path, $B_{\max}$ bounds blocking and scheduling interference, $J_{\max}$ bounds release jitter, T_{deadline} is the deadline, and T_c is the control period. This is an accounting condition for that path, not a complete schedulability or stability proof. The bounds require evidence for the actual workload and hardware. A finite benchmark's largest observed latency is not automatically a worst-case bound. A complete assessment also includes sensing delay, communication, actuator response, and the controller's tolerance to missed updates.

Learned policies fit this structure through explicit adapters. OpenVLA illustrates a vision-language-action model that maps observations and instructions to robot actions (Kim et al., 2025b). ACT predicts sequences of actions for fine-grained manipulation (Zhao et al., 2023). Their outputs may be joint targets, end-effector increments, or other embodiment-specific commands. The adapter must preserve the model's normalization, reference frames, action period, and control mode, and establish compatibility with the receiving controller. Position, velocity, and force checks constrain admission. Collision avoidance and contact stability additionally depend on the controller and its operating envelope.

Figure 9.12 shows an asynchronous arrangement. An inference worker consumes a timestamped observation and produces a candidate action sequence with a sequence identity and validity horizon. A local executor continues its fixed-rate cycle from an already admitted buffer while the candidate is computed. Before a buffer switch, the receiver checks observation age at the scheduled use times, command order, frame, operating mode, lease, and relevant limits. It rejects late results whose assumptions are no longer valid. The gate also applies a controller-specific continuity rule

before the buffer is switched atomically. Otherwise the handover can introduce a discontinuity in velocity, force, or target pose. To avoid fallback, a replacement must finish inference, transport, and admission before the current buffer expires. When inference runs at the edge, the freshness budget includes both outward observation transport and returning commands. A small learned controller with a demonstrated execution bound can instead run inside the periodic path and is subject to Eq. (9.6).

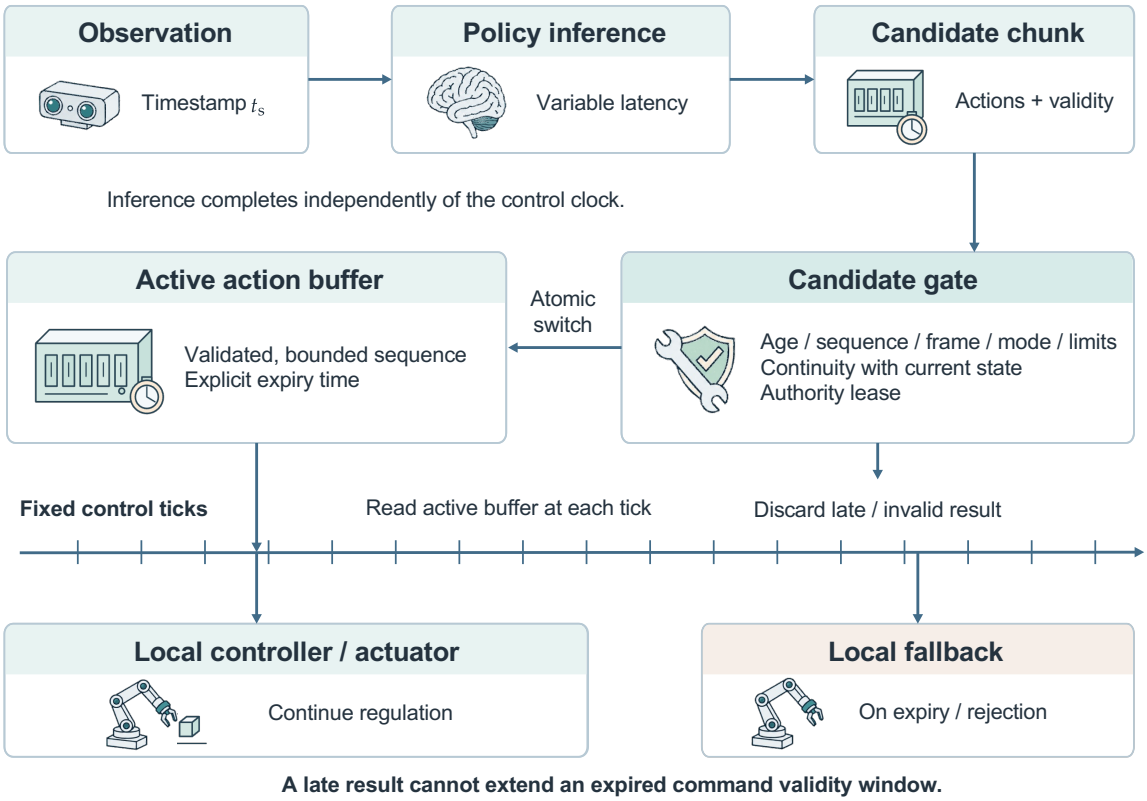


Fig. 9.12 Asynchronous inference coupled to local execution. Variable-duration policy computation produces candidate action chunks, while the local loop consumes a validated buffer at its own period. Admission and a controlled buffer switch connect the two. Expired results are discarded. Loss of valid command authority activates a local fallback. The control ticks are schematic. The gate checks continuity as well as validity before the atomic buffer switch

Action chunks reduce the frequency at which a model must be invoked, but they also commit the robot to predictions made from an earlier observation. A longer chunk can absorb inference delay while delaying the response to a changed scene. Real-time chunking addresses this interface by generating a new chunk asynchronously while accounting for actions committed during inference (Black et al., 2025b). It coordinates policy execution under delay through a continuation consistent with the committed prefix. Deployment still requires measurement and validation of the complete runtime. Contact-sensitive control and protection still require timely local measurements, even when the high-level action sequence remains valid.

The runtime should make buffer expiry, inference overrun, and observation dropout first-class events. Repeating the last action indefinitely can be especially dangerous when the action is a velocity or torque. At the earlier of lease and command expiry, the executor begins the plant-specific fallback described above, records the transition, and prevents a delayed response from restoring authority. Resumption requires fresh state and an explicit re-admission. Thus learning

can change the motion strategy without making the availability of a large model a prerequisite for every control update.

9.4.4 The Embodied Intelligence Harness and Skill Contracts

The Harness governs the task loop across these interfaces. It maintains the current goal and evidence, resolves requests against available skills, arbitrates resources, and tracks execution to an observable result. Hierarchical reinforcement learning formalizes temporally extended actions through initiation, policy, and termination components (Sutton et al., 1999). Behavior trees provide established mechanisms for composing and monitoring robot behaviors (Iovino et al., 2022). These precedents explain much of the required machinery. Language-conditioned skill selection with affordance scoring, as in SayCan, provides a concrete precedent for connecting model proposals to robot capabilities (Ichter et al., 2023). In the framework used here, the model proposes skills and arguments. Admission, monitoring, and completion decisions follow explicit contract checks over timestamped evidence.

A skill contract gives the supervisor something more precise than an action name. Table 9.3 develops one example for moving a capped vial between known fixtures. Its parameters and conditions are illustrative design choices, to be set from the actual robot, payload, and procedure. The example separates request acceptance, physical progress, and completion evidence. A ROS 2 action can carry the goal, feedback, result, and cancellation protocol, but resource ownership, physical preconditions, command leases, and evidence requirements remain application responsibilities (Biggs et al., 2020).

Admission checks establish that a skill can start. Execution monitoring determines whether its assumptions continue to hold (Pettersson, 2005). A fresh target pose may become stale during an approach. Another device may occupy a reserved region. A grasp may be established and then lost. The monitor should therefore associate each predicate with evidence and a validity interval, and distinguish false from unknown. Unknown identity or destination occupancy cannot be converted to success merely because the motion controller reports completion. Learned perception can misidentify a vial or occupancy state. Completion thresholds need calibration, and consequential decisions may require complementary sensing or repeated observation. Conflicting evidence triggers re-observation or a recovery whose own conditions are checked.

Cancellation also has physical semantics. A ROS 2 action server may accept a cancellation request while the underlying mechanism still needs time to decelerate or secure its load. The Harness should wait for the executor's terminal state and physical evidence before allocating its resources to another skill. If communication fails after a vial has been placed but before the result arrives, a timeout leaves the outcome uncertain. Retrying the transfer immediately could act on an empty fixture or move a second object. Persistent request identities, execution records, and observation of the fixtures allow the supervisor to reconcile state after reconnection. Reliable message delivery alone cannot provide exactly-once physical effects.

Restarting the Harness reconstructs task state from the executor and fresh observations before any new authority is issued. The application-level "outcome unknown" state describes missing physical evidence or a missing result. It is distinct from the unset status represented by ROS 2's `STATUS_UNKNOWN`.

9.4.5 Development Infrastructure and Evidence for Deployment

Development infrastructure should test the boundaries just described. A reproducible episode records acquisition and actuation times, frame and calibration versions, model and controller identities, skill requests, authority changes, and the evidence used to declare completion. Tracing the path from a sensor event through callbacks to a command helps distinguish inference delay from scheduling or transport delay. `ros2_tracing` provides instrumentation for such analysis in ROS 2 (Bédard et al., 2022). High-rate recording itself consumes resources, so its effect on the control workload must be measured or isolated.

Table 9.3 An illustrative `transfer_vial` skill contract. Symbols denote configured bounds, not universal thresholds. The local executor enforces command validity and fallback independently of the task supervisor

Contract field	Concrete interface and obligation
Input	`request_id`, vial identity, source and destination fixture IDs, pose and frame, calibration version, payload class, and maximum task duration T_{task}. Duplicate IDs query the recorded execution state rather than starting another transfer.
Preconditions	Identity and source occupancy confirmed, destination empty, pose age satisfying Eq. (9.5), compatible gripper and payload, local controller ready, required interlocks valid. Unknown conditions trigger inspection or rejection.
Resources	Exclusive arm and gripper authority, reservation of the destination slot and shared motion region. Acquisition is bounded. Partial acquisition is released on rejection.
Timing	Task duration T_{task} starts at local admission. Each phase has its own timeout. Task commands expire at the earlier of the local lease deadline and their observation-limited expiry. Fallback begins no later than that point, independently of Harness cancellation.
Progress	Timestamped phase, controller status, grasp/contact evidence, observation age, and remaining authority. "Goal accepted" and "target pose reached" are intermediate states.
Completion evidence	After release and retreat, destination occupancy and vial identity are observed, the source is empty, and the gripper no longer holds the vial. Return evidence timestamps, confidence or validity flags, and the relevant log references.
Failure and recovery	The executor reports `aborted` or `canceled` with the last verified state. A missing result leaves the Harness with an application-level `outcome_unknown`. Apply the local fallback and re-observe before retrying or releasing resources.

Recorded-data replay is valuable for comparing estimators and diagnosing failures, but an unchanged sensor log cannot establish the closed-loop outcome of a changed policy. Different actions would have produced different observations. Software-in-the-loop (SIL) testing runs the relevant software against a simulated plant to examine logic, interfaces, and responses to modeled disturbances. Hardware-in-the-loop (HIL) testing runs target firmware on target control electronics against a simulated plant, exposing processor and scheduling effects. Sensor and actuator I/O may still be emulated through a different link, as in PX4's hardware-in-the-loop (HITL) arrangement (PX4 Development Team, n.d.). The test report must identify which physical interfaces were exercised. Gripper friction, camera exposure, and the complete machine's stopping response require physical trials.

Figure 9.13 organizes these stages around explicit claims. For the vial-transfer skill, SIL can inject a stale pose, a duplicated request, or a lost acknowledgment. HIL can expose queue growth, inference contention, and watchdog delay. Bounded hardware trials can test grip loss and verify a loaded stop. Terminating the Harness, suspending its monitoring loop while a heartbeat remains alive, and partitioning the network are particularly informative. Local regulation and assigned protection must remain available, command authority must expire as specified, and reconnection must reject queued commands and renewals from obsolete epochs. Tests should also challenge sensor loss and the shared resources that could defeat the intended isolation.

The evidence should include the operating conditions, observed latency distribution and misses, age at actuation, time to fallback, recovery outcome, and the physical criterion for success. A release bundles the policy weights, preprocessing and action adapter, controller settings, middleware configuration, firmware, calibration, and skill schema used to obtain that evidence. Changing one of these can invalidate a previously established interface or timing assumption. Regression tests and staged physical trials determine whether the affected claims still hold. Binding evidence to this configuration provides the software basis for the sustained reliability and fleet deployment considered in Section 9.5.

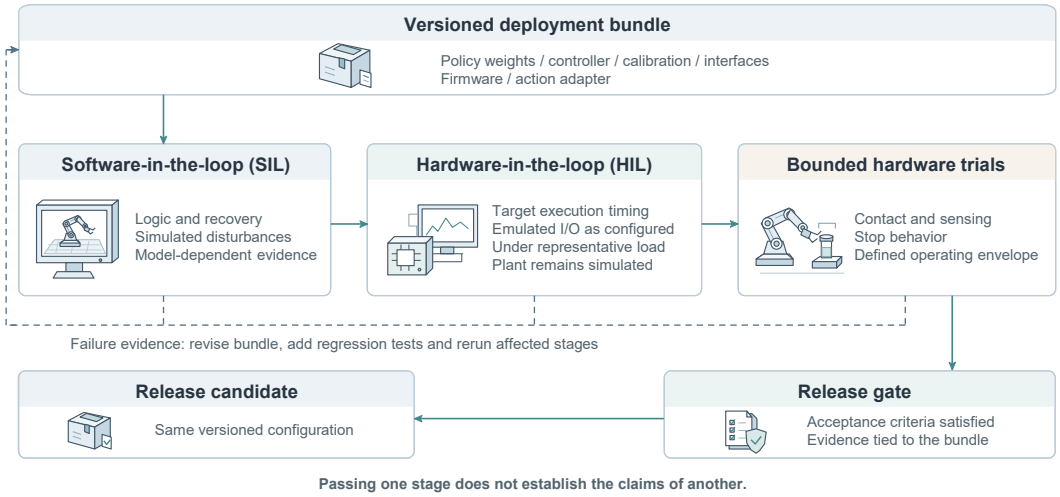


Fig. 9.13 Validation stages answer different questions. SIL tests execution logic against a modeled plant. HIL adds target electronics and timing. Bounded physical trials expose sensing, contact, and actuation effects absent from the model. Release decisions apply to a versioned combination of model, firmware, controller, action adapter, calibration, and interfaces. Failure evidence drives component revisions and regression tests

9.5 Reliability and Deployment at Scale

A vial-handling robot may complete a demonstration and still fail to produce usable samples over a working day. A cap slips after the fingers warm up, a dispenser delivers less liquid than commanded, or a restart repeats a transfer already completed. Each failure breaks a different connection between an intended action and its physical result. Reliable Physical AI requires that connection to remain observable and correctable. The system must detect a deviation, determine what can still be recovered, and verify the result before proceeding. Experience from these events can improve subsequent behavior, provided the cause is identified and the change is tested. This section follows that loop from one task to a fleet of machines.

9.5.1 From Successful Attempts to Sustained Physical Output

Reliability concerns continuity of correct service. Availability concerns whether a system is ready to work (Avizienis et al., 2004). For a physical task, correct service needs an observable endpoint. A sample is valid only if its identity, quantity, and required process conditions are preserved. Command delivery, completed trajectories, and apparent grasps are intermediate events. Their success matters only through the result they enable. A robot that remains powered throughout a shift but repeatedly requires a person to rescue its samples has poor autonomous productivity.

The measurements should therefore follow the task. Encoders establish arm configuration, touch can reveal loss of contact, and a balance can verify a transfer. Agreement among several signals is useful when they provide independent evidence. Several estimates derived from the same incorrect calibration may agree while the sample is wrong. The task-state observability of Section 9.2.3 thus becomes a reliability requirement. The system must distinguish states that call for different actions while there is still time to intervene. Table 9.4 links this evidence to the scale of correction.

Small errors accumulate over long procedures. Fifty independent steps that each succeed 98% of the time give only about 36% success for the whole sequence ($0.98^{50} \approx 0.364$). Detection and recovery can change this outcome. Let p_k be the initial success probability at stage k, d_k^{detect} the probability

Table 9.4 Reliability depends on matching physical evidence to the scope of correction. A failure observed at one level may originate at another

Level	Example deviation	Evidence needed	Scope of correction
Component	Gripper response changes after warm-up	Current, temperature, measured motion and load	Reduce duty and inspect the drive or transmission
Skill	Cap slips during opening	Contact change, cap motion, applied torque	Re-establish support and revise the grasp
Workflow	Vial contents disagree with the task record	Sample identity, measured quantity, verified checkpoints	Reconcile progress and recover or reject the sample
Fleet	Failures rise after a policy release	Outcomes by hardware, calibration and software version	Isolate affected configurations and validate restoration

of detecting an initial failure, and r_k^{rec} the probability that one recovery attempt succeeds given that failure was detected. A simplified model gives

$$P_{\text{task}} = \prod_{k=1}^{N_{\text{stage}}} \left[p_k + (1 - p_k) d_k^{\text{detect}} r_k^{\text{rec}} \right]. \tag{9.7}$$

Figure 9.14 shows the effect of this additional path to completion. The product assumes independent stages and successful recovery without hidden damage. Shared calibration errors, accumulated wear, and contamination violate those assumptions. The equation also omits recovery time. A system can eventually succeed yet produce little usable work per hour. Full-workflow tests must therefore measure both outcome and time.

Finite trials also limit what can be claimed. Twenty successes without a failure leave an exact one-sided 95% upper confidence bound of about 14% on failure probability, under independent trials with a fixed protocol (Hanley and Lippman-Hand, 1983). Evaluation should state the workload, initial conditions, trial count, allowed assistance, and stopping rules. All attempted tasks, including aborted ones, belong in the record.

9.5.2 Detecting and Diagnosing Failures from Physical Evidence

A monitor must recognize both an abnormal interaction and a missing task result. Rising torque with little movement may indicate a jam. Ordinary joint motion with an unchanged balance reading may indicate an empty transfer. The first can require a rapid local response, while the second requires knowledge of the process. The sensor and actuator interfaces in Section 9.2 provide the measurements. The runtime in Section 9.4 must associate them with the action being attempted and its expected outcome. A successful command acknowledgment closes neither loop.

Detection is easier to interpret when it compares a prediction with a measurement tied to the task. A dynamics observer can compare expected and measured joint loading, subject to its calibration and model assumptions (Haddadin et al., 2017). A task monitor can check whether a cap rotated, a vial moved with the gripper, or mass changed by the required amount. These checks should be conditional on the current phase. Resistance is expected when tightening a cap but may indicate a fault before the threads engage. A fixed threshold applied without that context can either stop useful contact or overlook an abnormal one.

Learned monitors can supplement these checks. Sentinel combines action consistency tests with vision-language assessment of task progress (Agia et al., 2025b). It provides evidence for detecting failures during execution. Detection alone does not establish a correct recovery. Likewise, a plausible explanation from a model remains a hypothesis. A stopped finger may reflect an obstructed object, a current limit, overheating, or loss of communication. Synchronized images, encoder readings, currents, temperatures, and command records help separate these causes. Their timestamps matter because evidence collected after the robot has withdrawn may no longer describe the failed contact.

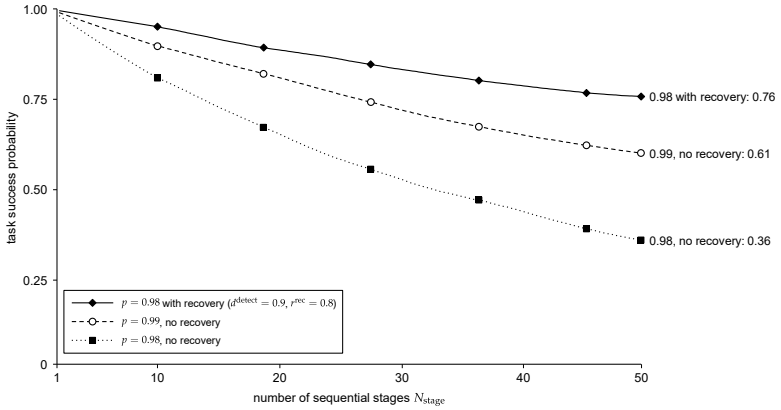


Fig. 9.14 Detection and recovery change the probability of completing a 50-stage task. In Eq. (9.7), $p = 0.98$, $d^{\text{detect}} = 0.9$, and $r^{\text{rec}} = 0.8$ give an effective stage success of 0.9944 and about 76% task success, compared with 36% for $p = 0.98$ and 61% for $p = 0.99$ without recovery. These are illustrative calculations under the stated assumptions, not measured robot performance

Detection quality should be measured where it changes the physical outcome. Report missed failures, false stops per scheduled operating hour, and the time from a detectable deviation to the protective response. Delays include sensing, transport, inference, and actuation, as discussed in Section 9.3. An alarm raised after a fragile vial breaks has a different value from one that allows the grasp to be corrected. Evaluate monitors on representative faults as well as normal contact transitions. High accuracy on a dataset dominated by uneventful motion is insufficient.

9.5.3 Recovering Progress and Handling Irreversible Actions

Recovery begins by establishing what is still true. After a slipping cap, the vial may remain supported, the cap may have moved, and the threads may already be partly disengaged. Returning the arm to a previous pose does not restore that state. The system should first preserve support and limit further loading, then acquire enough evidence to choose a feasible continuation. A new view, a gentle probing action, or a quantity check can be more useful than another full attempt. These actions use interaction to reduce uncertainty, which is a central capability of Physical AI.

A recovery should change the conditions responsible for failure. A missed grasp may permit a revised approach. A jam may require controlled withdrawal and realignment. A thermal limit may require reduced duty or cooling. Force, travel, retries, and elapsed time need task-specific bounds. If the cause remains unresolved or those bounds are reached, the robot should stop or request assistance. The physical checks required to resume belong to the recovery procedure itself.

Long tasks need checkpoints defined by verified physical outcomes. After a transfer, the system records sample identity, measured quantity, and completion status outside the policy's transient state (Section 8.3). On restart, it reconciles that record with the scene before issuing a new command. Otherwise, replaying a command whose acknowledgment was lost can double a dose. An operator may also have moved the container, or a sample may have exceeded its usable holding time. The checkpoint preserves what was verified. Renewed observation establishes what remains valid now.

Some actions admit no useful rollback. Mixing the wrong reagent cannot be undone by restoring a software version. Before an irreversible step, the system should verify its identity and process preconditions and resolve uncertainty while correction is still possible. Action-conditioned world models can simulate candidate interactions. UniSim uses predicted visual outcomes to train planners and control policies (Yang et al., 2024b). These predictions may still omit contamination, weak

contact, or damage invisible in an image. They should guide investigation, while the actual process supplies the evidence for committing the action and accepting its result.

9.5.4 Learning from Operation Without Concealing Hardware Drift

The same command can produce a different interaction as pads wear, a camera mount shifts, or actuator temperature rises. Changes also arise from new objects and from states reached after the policy's own mistakes. These forms of distribution shift demand different remedies. A loose camera mount requires repair and recalibration. Repeated failure on a correctly observed new cap geometry may justify additional demonstrations or a revised control policy. Retraining on every deviation risks teaching the model to compensate for a deteriorating machine.

The operating record should retain the observation, intended action, measured response, hardware and software configuration, intervention, and verified outcome. A successful human rescue is especially informative when it records which state prompted the intervention and what changed. DAgger addresses learning from states visited by the learner through expert feedback (Ross et al., 2011). Sirius studies learning from robot execution with human monitoring and interventions (Liu et al., 2023b). These approaches support improvement from experience. Intervention frequency and operator time remain part of the deployment result, including the monitoring Sirius requires.

Simulation can test a proposed correction across variations that are costly to reproduce physically. RialTo reconstructs real environments for simulation-based policy improvement (Villasevil et al., 2024), while dynamics randomization varies physical parameters to reduce dependence on one nominal model (Peng et al., 2018b). The relevant variation should follow the diagnosed mechanism. Friction is relevant to slip, delay to late correction, and camera placement to an observation mismatch. Randomizing unrelated parameters cannot establish that the observed failure has been addressed.

SIMPLER evaluates agreement between simulated and real policy performance within tested manipulation settings (Li et al., 2025b). Such agreement makes simulation useful for screening candidates, but a favorable ranking does not establish adequate absolute reliability. A revised policy needs fresh physical trials on the failure conditions, earlier successful tasks, and representative operating variation. Development episodes and acceptance trials must be separated. Fast control feedback corrects the current interaction. Learning changes future behavior and therefore requires this additional validation loop.

9.5.5 Deploying Compatible Configurations Across a Fleet

Scaling a policy means preserving its observation-action relationship on multiple physical machines. Differences in camera placement, fingertip wear, controller gains, and firmware can change that relationship even when robots share a model name. Common collection procedures, as used in DROID, help make experience comparable across sites (Khazatsky et al., 2024). Deployment additionally requires evidence that each supported configuration still produces the intended physical outcome under its assigned workload.

Consider a new cap-opening policy for a fleet of vial-handling robots. The release manifest identifies model weights, observation preprocessing, action units and scaling, controller code and gains, drive firmware, and supported hardware revisions. It also binds each robot to its own checked camera, tool, and force-sensor calibration, using a specified calibration procedure and tolerances. Calibration values cannot be copied between machines merely because their parts have the same names. The validated unit is the compatible configuration. Installing new weights with an old action convention can turn a useful learned correction into an unintended movement.

After interface, replay, simulation, and hardware-in-the-loop checks, a small subset runs supervised trials spanning the intended hardware and workload. Robots on the previous release provide a concurrent comparison. Acceptance criteria are fixed beforehand and cover sample quality, completion, contact loads, temperature, interventions, and recovery time. The trials include repeated cycles and warm actuators. Expansion depends on adequate exposure to these conditions,

rather than a few successful picks. Results should be separated by configuration. An improved fleet average can conceal a regression on one gripper revision.

Suppose cap slip increases after the update. Further rollout stops and affected robots are removed from the scheduler. Each preserves its load and reaches a validated stopping state where possible. Uncertain samples are held for inspection. Logs retain the complete configuration and the physical evidence around the event. Recovery restores the previously validated compatible configuration where supported, checks calibration, and runs a physical acceptance sequence before returning the robot to service. Firmware that cannot be reversed needs a tested recovery image or maintenance procedure. Software restoration and sample recovery are separate obligations. A spilled sample remains lost after rollback.

Table 9.5 connects these trials to sustained productivity. Faults still unresolved at the reporting cutoff must remain visible, rather than disappearing from recovery-time statistics. Over the same reporting period, the cost per valid output introduced in Section 9.1 is

$$C_{\text{valid}} = \frac{C_{\text{equipment}} + C_{\text{operation}} + C_{\text{assistance}} + C_{\text{maintenance}} + C_{\text{failed material}}}{N_{\text{valid}}}, \qquad N_{\text{valid}} > 0, \quad (9.8)$$

where costs are allocated once to that period. With no valid outputs, there is no finite observed unit cost. Better task success can therefore coexist with worse deployment performance if recovery consumes more time or assistance. The release decision should follow the measured physical work, including its interruptions. Those records then identify whether the next improvement belongs in the policy, the sensing and actuation hardware, or the operating procedure.

The cases in Section 9.6 examine this requirement at the contact level. A demonstrated manipulation skill becomes useful within a larger task only when its entry conditions, physical result, and recovery requirements are preserved.

9.6 Case Studies in Dexterous Hand Hardware

A hand holding a spray bottle must support the bottle while another finger moves the trigger and the wrist maintains the spray direction. Rotating an object within the palm instead requires changing the contacts that keep it under control. In both cases, what a learned policy can achieve depends on which contacts the hardware can make, sense, and adjust.

Applying Sections 9.1 and 9.2 at the finger level, the following cases examine RobotEra XHAND 1, the first-generation Wuji Hand, and Sharpa Wave W01 through their task requirements, mechanisms, sensing and control interfaces, and demonstrated capabilities. The emphasis is on the hardware conditions under which a result holds. Learning methods are covered in Section 6.6, and low-level hand controllers in Sections 7.4.4 and 7.4.5.

Table 9.5 Operational measures for comparing releases under the same workload. Scheduled time includes stops and recovery. Aborted attempts remain in the task record. Report trial counts and uncertainty with success rates, and distributions for intervention and recovery times

Measure	Definition and interpretation
Task success	Valid completed tasks divided by all attempted tasks, with assistance reported separately
Effective throughput	Valid outputs per scheduled operating hour. A faster motion is useful only if it increases accepted output
Human intervention	Interventions per scheduled operating hour, together with operator minutes consumed and the reason for assistance
Downtime	Time unavailable for assigned work divided by scheduled time. Separate planned maintenance from unplanned stops
Recovery time	Elapsed time from fault detection to verified return to service. Include waiting for assistance and physical rechecks

9.6.1 *Manipulation Tasks and Contact Requirements*

Dexterous tasks place different demands on the hand. Grasp acquisition establishes contact with an object. Transport preserves its relation to the hand. In-hand manipulation changes that relation. Tool operation additionally changes another object or process. Their evaluation boundaries matter. A policy tested with an object already in its palm has not thereby been tested on acquiring it from a table, and reaching a tool pose does not necessarily establish that the tool performed its intended operation.

To move the index finger around a cylinder, the remaining fingers must first sustain the load. Excessive squeezing can prevent rotation, while premature release can lose support. Rolling, sliding, and breaking contact permit different motions. The controller must coordinate the transition between them, as well as the finger positions (Lynch and Park, 2017). In tool use, it must also resist the reaction to pressing, cutting, or twisting. A grasp that supports the tool's weight may be inadequate once the tool meets its target.

Each of these demands is partly a hardware question. Holding a load while another finger moves takes enough independent joints and enough sustained fingertip force. Regulating squeeze instead of position takes a drive that accepts force or impedance commands, or at least torque limits. Catching the onset of sliding takes a contact sensor fast enough to act before the grasp is lost. Resisting a tool's reaction takes force capacity, and an arm that can carry the hand's own mass.

9.6.2 *Hand Design and the Observation-Action Interface*

The hands differ in their independently controlled coordinates (Table 9.6), excluding the arm and wrist (RobotEra, n.d.; Wuji Technology, n.d.c; Sharpa Robotics, n.d.a). Thumb opposition, finger spread, joint limits, and available contact surfaces determine what those coordinates can accomplish. More coordinates enlarge both the possible contact arrangements and the coordination problem. They do not alone rank task performance.

Figure 9.15 compares a MANO human-hand mesh with the actual robot geometries and locates their rotational coordinates. MANO represents each of its 15 finger joints by a three-parameter rotation. Its 45-dimensional hand pose is a model parameterization, not a count of independently actuated anatomical motions (Romero et al., 2017). Reconstructed human landmarks and robot joints are different representations. A tracked fingertip need not correspond to one motor, and equal joint-vector dimensions need not imply equal fingertip motion. For a thumb-index pinch, the useful correspondence concerns fingertip separation and opposition relative to the object. Reproducing them may require different combinations of finger flexion, lateral motion, and wrist placement. Moving the wrist can compensate for limited finger reach, but also moves the other contacts.

The retargeting objective in Eq. (6.23) must be constrained by collisions and contact. A small fingertip error does not guarantee that a tool can be held under load.

The observation interface requires the same care. A fingertip contact flag can detect contact without resolving shear load or object pose, while an external camera can locate an object without seeing its supporting contacts. Coverage also matters. A dense fingertip sensor does not observe a load borne by the palm. *Rotating without Seeing* illustrates the alternative of distributing binary contact sensing over the fingers and palm, supporting learned reorientation on real objects (Yin et al., 2023). Sensors should therefore be compared by the contact decisions they support, not by resolution alone.

Policy targets also differ from actuator response. A finger stopped by an object cannot track every requested position. Its loading depends on gains, transmission, and effort limits. A study therefore has to report both the policy's targets and the observations it actually used. A tactile sensor listed in a hand's specification may be absent from a particular policy input. Hardware revisions and additional software development kit (SDK) capabilities should likewise remain separate from the experimental configuration being discussed.

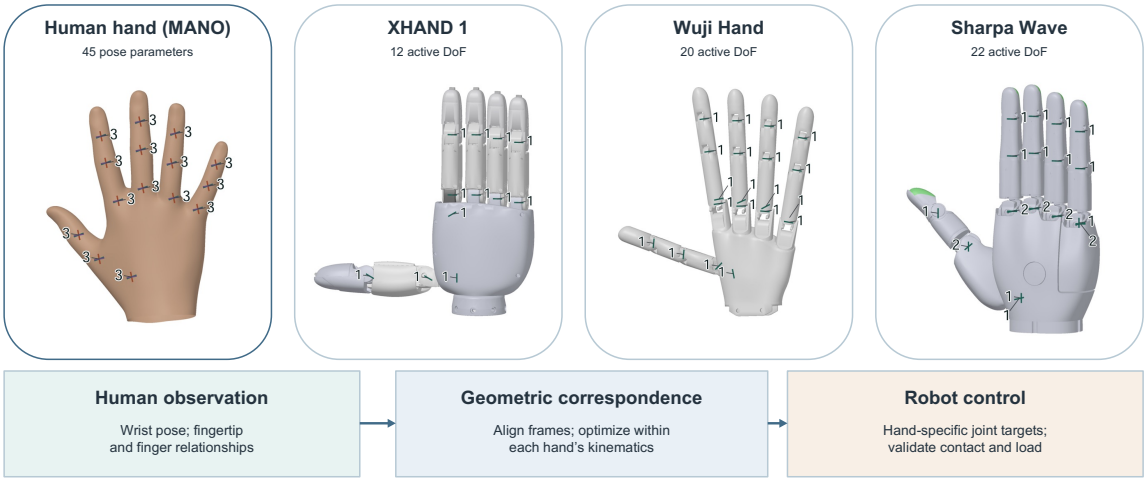


Fig. 9.15 Hand geometry and local rotational coordinates. The surface-smoothed MANO template and imported XHAND, Wuji, and Sharpa meshes are shown at a common orthographic scale after hand-frame alignment (Romero et al., 2017; SPIDER Contributors, 2026; Wuji Technology, 2026; Sharpa Robotics, n.d.b). Axis marks locate model joints. Adjacent numerals give the local rotational dimension, with coincident robot axes grouped. MANO uses three pose parameters at each finger joint. Robot labels count active Unified Robot Description Format (URDF) coordinates. Wrist and arm motion are excluded. Correspondence therefore requires retargeting through the robot's kinematics, rather than copying the human pose vector

9.6.3 Actuation, Sensing, and Electrical Interfaces

Table 9.6 lists manufacturer specifications for the cited versions, rather than measurements from the learning studies. Their implications depend on the load, contact geometry, and feedback required by the task.

Mass, Size, and Force Capacity All three hands approach human hand size, but their masses differ by more than a factor of two (Wuji Technology, n.d.c; Sharpa Robotics, n.d.a). Hand mass consumes arm payload before an object is lifted and adds to wrist inertia. It must therefore enter the arm's dynamics model. Maximum fingertip-force ratings do not specify how long a force can be sustained. For a stiff trigger, the relevant test is continuous force within the motor's thermal limits, evaluated in the geometry of the actual grasp.

Drive and Compliance The three drives give a policy's joint target different physical meanings. Wuji's rotary direct-drive joints operate under position control and are specified as self-locking (Wuji Technology, n.d.c). Active yielding to an external load requires the controller to adjust the position target from feedback. Sharpa Wave instead exposes position, admittance, impedance, and zero-force modes, the last being backdrivable for manual guidance (Sharpa Robotics, n.d.a). Under impedance control, a blocked finger produces a force that grows with its position error and the stiffness selected, as in the series-compliance relation of Eq. (9.3). Under zero-force mode an operator can move the fingers directly, which is one route to kinesthetic demonstration. XHAND accepts joint targets together with controller gains and torque limits from the host (Section 9.2.6). Joint targets alone therefore underspecify an action. The control mode, gains, and effort limits used during collection must be logged and reproduced at deployment.

Sensing Load at the Joints Sharpa Wave estimates joint torque from motor current by default and also provides five torque sensors at the thumb carpometacarpal and four metacarpophalangeal joints. Their full-scale ratings are 5 and 3 N m, respectively, with a stated total error of $\pm$(1% of full

Table 9.6 Manufacturer specifications of the three dexterous hands used in the case studies of Sections 6.6, 7.4.4, and 9.6 (RobotEra, n.d.; 2024; Wuji Technology, n.d.c; Sharpa Robotics, n.d.a). Values are as stated by each manufacturer for the cited version and were not independently verified. A dash means that the cited document does not state the value. Ratings are not directly comparable, because test conditions are not specified uniformly

	XHAND 1	Wuji Hand (first gen.)	Sharpa Wave W01
Active DoF	12 (thumb 3, index 3, other fingers 2 each)	20 (four per finger)	22 (thumb 5, fingers 4 each, plus little-finger carpometacarpal (CMC) joint)
Drive (as described)	Geared motor at each driven joint	Self-locking rotary direct-drive joints, field-oriented control (FOC) drive	—; zero-force mode is backdrivable
Joint control	Joint targets with host-set gains and torque limits	Position control	Position, admittance, impedance (MIT), zero-force
Mass and size (mm)	1.1 kg, $190 \times 94 \times 47$	0.58 kg excl. cables, $201 \times 75 \times 50$	1.3 kg, $208 \times 90 \times 50$
Max. fingertip force	15 N (whole hand 80 N)	15 N	20 N (active)
Grasp-diameter limit	Minimum 16 mm	Maximum 100 mm	Minimum 10 mm
Joint load sensing	Motor current	—	Current-based estimate 5 torque sensors at thumb CMC and finger metacarpophalangeal joints
Tactile sensing	Optional fingertip arrays, >100 points per finger, 3D force and temperature	None on the hand	Vision-based fingertip sensors (5), with 6-axis force, deformation map, and contact point 20 mN, 1 mm resolution
Tactile rate and latency	—	—	30 Hz, 50 ms (processed on board) up to 180 Hz, 20 ms (raw images, host GPU)
Interface and joint rate	RS485 or EtherCAT (Ethernet for Control Automation Technology), $\approx$12 ms whole-hand update	Universal Serial Bus (USB) 2.0, 1 kHz $\times$ 20 axes	Ethernet (100/1000 Mb/s), 500 Hz
Supply and power	—	12 to 20 V DC, 12 W static	18 to 28 V DC 15 W static, 100 W average, 180 W peak
Rated life and protection	—	$\geq 3 \times 10^5$ unloaded grips; IP40	$> 10^6$ unloaded cycles, tactile skin $> 10^5$ cycles at 40 N; IP20

scale + 5% of reading) after calibration (Sharpa Robotics, n.d.a). XHAND also returns motor-current information. Current-based estimates require calibration for transmission friction and motor temperature. A dedicated torque sensor measures the load through its joint but still does not locate the contact. Section 7.4.5 examines control using such joint-torque force estimates.

Tactile Sensing: Coverage, Rate, and Latency Each Sharpa Wave fingertip uses an internal camera to image a compliant surface. It reports force, surface deformation, and contact location, with a 30 N force measurement range (Sharpa Robotics, n.d.a). For control, their timing is critical. In the standard mode these outputs are computed on board at 30 Hz with a reported nominal latency of about 50 ms. In the high-performance mode raw images stream at up to 180 Hz with a reported nominal latency of about 20 ms and are processed on a host GPU. The joint loop, by comparison, runs at 500 Hz. The reported nominal tactile latencies of 20 to 50 ms correspond to 10 to 25 joint-control periods at 500 Hz. Tactile updates at 30 to 180 Hz arrive approximately 3 to 17 times less frequently than joint-control updates. Systems such as T-Rex therefore refine finger targets asynchronously from tactile history while a separate servo thread tracks them (Section 7.4.5). Sensing latency remains part of the end-to-end budget in Eq. (9.4). XHAND's optional arrays report fingertip force and temperature (RobotEra, n.d.). The first-generation Wuji Hand has no tactile sensing. Pressure measured by its companion glove belongs to the operator's contacts, not the robot's. In all three cases the palm and the inner finger segments are uninstrumented, so loads that the policy routes there remain unobserved.

Electrical Interfaces, Bandwidth, and Power The joint interfaces in Table 9.6 impose different update periods (Wuji Technology, n.d.c; RobotEra, 2024). Sensor transport can impose a separate constraint. Sharpa Wave's hand control and 30 Hz tactile outputs fit within 100 Mb/s Ethernet, whereas its 180 Hz raw tactile stream requires gigabit Ethernet (Sharpa Robotics, n.d.a). Choosing raw images thus moves computation to the host and raises network load, linking sensing to the device-edge placement decisions of Section 9.3. Power follows the same logic. Sharpa Wave specifies 100 W average and 180 W peak per hand, and 200 W average for a pair. Wuji's supplied adapter is rated for 420 W 10 ms transients (Sharpa Robotics, n.d.a; Wuji Technology, n.d.c). On a mobile manipulator, two hands draw from the same battery as the base and arms, and peak currents during simultaneous grasps must be within the supply's transient capacity.

Durability and Maintenance The rated hand lifetimes concern unloaded motion, whereas the tactile-skin rating specifies loaded cycles (Sharpa Robotics, n.d.a; Wuji Technology, n.d.c). These conditions matter in contact-rich learning. Fingertip skins become consumables, and pressure-induced tactile drift calls for recalibration within the maintenance loop of Section 9.5, separate from model updates. Neither stated ingress rating specifies liquid-ingress protection. Covers or gloves needed for spills in kitchen or laboratory tasks also alter friction and tactile response.

The specifications alone already limit what can transfer between hands, whatever the learning method. A policy trained on Sharpa fingertip deformation images cannot run on a hand without tactile sensing. A controller that relies on impedance mode cannot be transferred unchanged to a position-only interface. A refinement loop designed for a 500 Hz joint interface must be retimed for a 12 ms bus update. Section 6.6.3 asks the complementary question of what a shared action representation can transfer.

9.6.4 Demonstration and Data-Collection Hardware

For dexterous hands, data collection must capture wrist motion, articulated finger motion, and contact together. The same tracker and glove can record a person directly manipulating an object or provide commands to a robot. The resulting records describe different bodies and interactions.

Human pretraining data include egocentric video with estimated hand motion and instrumented demonstrations with measured tracking signals. CAIP uses both pose-estimated video and laboratory human recordings with MANUS Metagloves Pro and Vive Ultimate Trackers (Sharma et al., 2026b). Instrumented capture reduces dependence on reconstructing occluded fingers, but still requires the hand model and the retargeting of Section 6.6.1. Robot teleoperation additionally records how the deployed arm and hand respond to the mapped commands. This response is the extra evidence needed when a geometrically plausible demonstration does not produce the intended grasp.

A representative teleoperation arrangement pairs a wrist tracker with a data glove. The tracker estimates the hand's position and orientation in the tracking space. The glove resolves the fingers

relative to the hand. MANUS Core supports external tracker poses, including HTC Vive devices, with calibrated offsets between tracker and glove (MANUS, n.d.b). In CAIP's Sharpa setup, Vive wrist motion drives relative arm targets, while MANUS finger keypoints are retargeted into hand-joint targets (Sharma et al., 2026b). The two measurements solve separate parts of the arm-hand coordination problem. Finger tracking alone cannot tell the arm where to place the hand in the workspace.

Wuji's official teleoperation interface likewise supports Vive tracking with Wuji Glove or MANUS as the hand input (Wuji Technology, n.d.b). Wuji Glove combines electromagnetic fingertip tracking, a fitted hand model, and pressure sensing on the palmar side (Wuji Technology, n.d.a). MANUS also distinguishes raw sensor measurements from its solved hand skeleton and retargeted output (MANUS, n.d.a). For supervision, this means that a model-derived joint angle is not an independently measured robot joint angle. Likewise, pressure recorded while the wearer holds an object describes human-side contact. During free-hand teleoperation, the glove does not acquire the remote robot's contact forces. Those require robot-side sensing. Measuring pressure and rendering force feedback to the operator are separate capabilities.

Calibration has two distinct roles. Tracker-to-hand and tracking-to-robot transforms establish wrist targets, while hand-shape calibration and retargeting establish finger targets. A misplaced wrist tracker shifts every fingertip even when finger articulation is estimated accurately. Recording issued targets alongside measured robot motion helps distinguish this error from failure to reach a target under load.

Exoskeleton interfaces move part of this correspondence into the mechanism. Linkages and encoders relate the operator's motion to the demonstration hand, and mechanically matched designs can make collected contacts closer to those available during deployment. DexEXO illustrates this approach with a wearable linkage driving a passive hand whose appearance and kinematics match the deployed hand. Aligned wrist-camera images can then be used directly for policy training (Zhu et al., 2026). The benefit must be weighed against fit, range of motion, and access around a tool. A bulky finger attachment can prevent a valid demonstration even when the robot itself fits the tool. Passive coupling also differs from an actuated interface that returns robot forces to the wearer. An exoskeleton does not, by itself, imply bidirectional force feedback.

XHAND: Shaping the Collection Interface to the Mechanism XHAND approaches the correspondence problem from the collection side. A lower-dimensional hand changes which demonstrated finger motions can be reproduced independently, as Figure 9.15 shows. DexUMI investigates this problem on XHAND and Inspire hands. It combines a wearable exoskeleton fitted to the target hand's kinematics, measured joint motion, matched tactile sensing, and visual replacement of the human hand and device with the robot hand (Xu et al., 2025c). Mechanical alignment constrains the motion before it becomes training data. Observation alignment makes the same interaction more recognizable after deployment. Neither step alone ensures that the robot can sustain the demonstrated load. The paper also reports tactile drift after high pressure and an inconsistent motor-to-fingertip mapping (Xu et al., 2025c). Collection and deployment may use nominally identical devices that differ in loading history, so recalibration and loaded tests are part of keeping the demonstration interface valid.

A second XHAND study addresses the difficulty of coordinating the arm and fingers during collection. In the shared-autonomy framework of Cui et al. (2025), a person guides a UR3e arm through virtual reality (VR) while DexGrasp-VLA uses vision and touch to control the 12-DoF hand. This division reduces the number of decisions the operator must make simultaneously. The person chooses the approach and places the wrist, and the hand controller handles contact. The benefit depends on that controller. If it closes too early, the demonstration records the error unless the operator corrects it. A useful collection system must therefore expose who controlled each interval, how an intervention changed the action, and whether the resulting grasp served the task. Section 6.6.4 discusses the learning results of both studies, including where tactile input helped.

Figure 9.16 organizes these routes by where the physical interaction occurs. This determines whether the recorded contact belongs to a person, a passive demonstration hand, or the controlled

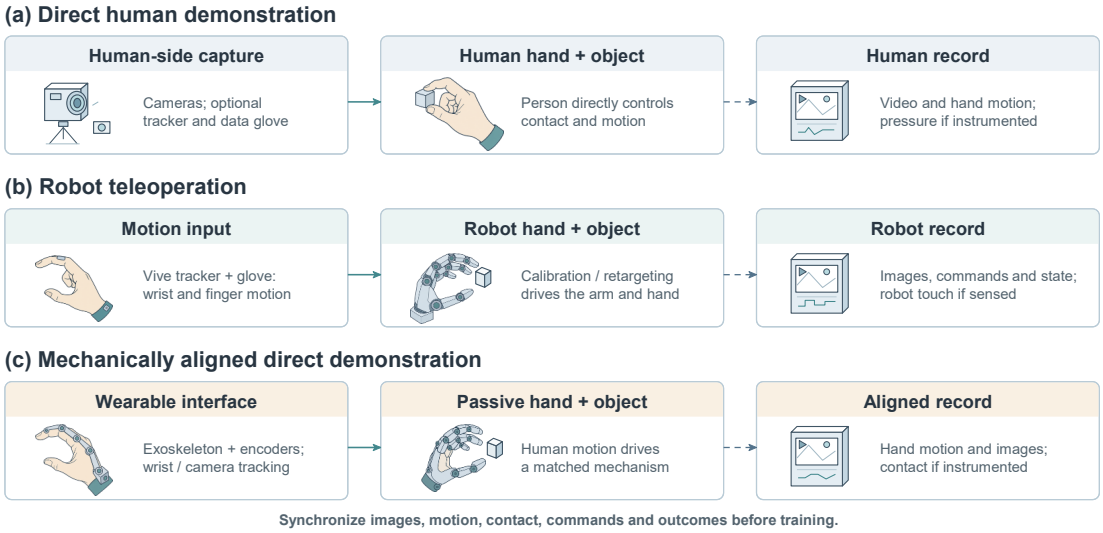


Fig. 9.16 Three routes from demonstration to learning data. A tracker and glove can record direct human manipulation or drive a robot through retargeting. A mechanically coupled passive hand supplies another form of direct demonstration. Solid arrows show motion or control. Dashed arrows show recording. The exoskeleton row illustrates passive collection, as in DexEXO. An actuated or remotely controlling exoskeleton requires a different feedback path. These are collection routes, not a fixed assignment of devices to training stages (MANUS, n.d.b; Wuji Technology, n.d.b; Zhu et al., 2026)

robot. The distinction remains important even when all three routes use similar cameras and motion trackers.

Contact transitions make timestamp alignment consequential. Pairing finger closure with an image acquired after contact changes the state on which the action appears to depend. Records must distinguish pre-contact intent, measured contact, and subsequent correction.

9.6.5 From Demonstrated Skills to Integrated Tasks

Each learning result built on these hands covers one piece of a task. EgoScale learns a target-robot policy from human data (Section 6.6.2), WM-Craftnet rotates a held object, SimToolReal follows object-pose goals, and Play2Perfect adapts to assembly (Sections 7.4.4 and 7.4.5). Sharing a hand does not make them composable. The skill interfaces of Section 8.3.2 require the state produced by one behavior to support the next. An acquired grasp must permit the required finger movement, the object must remain observable, and the transition must preserve its support.

Consider using a tool after picking it up. A grasp that permits lifting may occupy the finger needed to operate the handle. Integration then requires reorientation, a changed grasp, or placement on a support surface before operation can begin. An in-hand rotation result supplies evidence about one of these transitions under its tested initial conditions. It does not establish the complete acquisition-operation sequence. This is where the runtime interfaces of Section 9.4 must connect policy outputs to measured contact and task state. Finishing a motion command is not by itself permission to begin the next skill.

Feedback also has to reach the right level. Recent touch can correct finger targets while the visual goal stays valid, but losing a usable grasp calls for a new task-level decision, and the runtime must route the event accordingly (Section 9.3). With tactile latency spanning ten or more joint-control

periods in the reported modes (Section 9.6.3), a faster servo cannot make up for a policy that never receives the evidence of slip.

The reported endpoints (rotation completion, progress along a tool trajectory, insertion success, and accumulated success in a kitchen sequence) do not support a controlled comparison between hands (Section 6.6.5). Complete-task evidence requires grasping, contact transitions, operation, and recovery to succeed together under a declared protocol, with assistance costs reported alongside success.

Across the cases, reusable motor knowledge becomes useful only through a specific sensing and control interface. What transfers may be human coordination, an object-level goal, or a functional finger action. What has to be checked is the contact behavior on the deployed hand. Transfer to another hand, adaptation to assembly, and continued operation through wear are separate achievements. On a humanoid, this interface also depends on the moving body that supports the hand. Section 9.7 extends the analysis to that integrated system.

9.7 Case Studies in Humanoid System Integration

Carrying an object across a room connects almost every component considered in this chapter. The hands maintain the grasp, arm motion redistributes the load, stepping changes the support contacts, and the moving cameras must still provide useful observations of the destination. Recent systems built on the Unitree G1 show how these components can be organized for teleoperation, data collection, and autonomous whole-body interaction. They make the co-design problem in Section 9.1 concrete. The task model must express the required movements, and the robot's mechanics, sensing, and computation must support their execution in closed-loop interaction. The comparison follows a common set of questions concerning what the task demands from the body, how observations and commands cross the interfaces, what was demonstrated, and which operating conditions bound that evidence. G1 provides a common platform for comparing these integration choices.

9.7.1 Embodiment and Whole-Body Coordination

A humanoid's manipulation workspace depends on how it positions and supports its body. In G1's 29-DoF configuration, six joints per leg, three at the waist, and seven per arm provide multiple ways to reach and orient an object. Hand joints are counted separately (Unitree Robotics, 2026). Waist and arm motion can extend a reach, but the resulting posture and load must remain compatible with the supporting contacts. The hand's contact capabilities therefore need to be considered together with the body motions required to use them, extending the hand-design problem in Section 9.6 to the complete robot.

This coupling shapes the division of control between a task model and a motor controller. A task model may specify hand and arm motion while leaving the supporting legs to a local feedback policy. When a foot also manipulates an object, the task interface must convey that contact objective as well. HOMIE and SONIC implement these divisions of control on G1 (Ben et al., 2025; Luo et al., 2026). Both depend on a motor controller that produces the expected physical response from the assembled robot.

9.7.2 From a Learned Policy to Physical Execution

For a joint-target policy, motion depends on the interaction between its outputs, feedback gains, and actuator mechanics. As the actuation analysis in Section 9.2 explains, matching target angles alone does not match the robot's response. ASAP addresses this problem on G1 by learning a correction to the simulated action-response relationship from measured robot trajectories, with a focus on the ankle linkage. The correction is used during policy fine-tuning in simulation. The resulting policy executes on the robot without that additional correction model (He et al., 2025a). Physical measurements thus change the dynamics against which the policy is trained, improving transfer without adding that model to the online control loop.

Policy execution also depends on the state estimates and timing of its feedback path. BeyondMimic's G1 motion-tracking stack combines a 500 Hz state estimator with a 50 Hz policy, whose onboard CPU inference takes less than 1 ms (Liao et al., 2026). The estimator updates the body-state estimate between policy evaluations, while the policy uses that estimate to update joint targets. Policy frequency, inference latency, and estimation frequency therefore describe different parts of the physical loop. This separation implements the multirate execution discussed in Section 9.4. Timely movement depends on the complete sensing-computation-actuation path, including the stages outside neural-network inference.

9.7.3 Connecting Body Control, Hands, and Task Models

In whole-body manipulation, the division of control determines which aspects of motion a person or task model specifies and which the local controller supplies. HOMIE and SONIC provide two concrete answers (Figure 9.17). Both connect a task-command source to body and hand execution, but they expose different levels of control over the supporting legs.

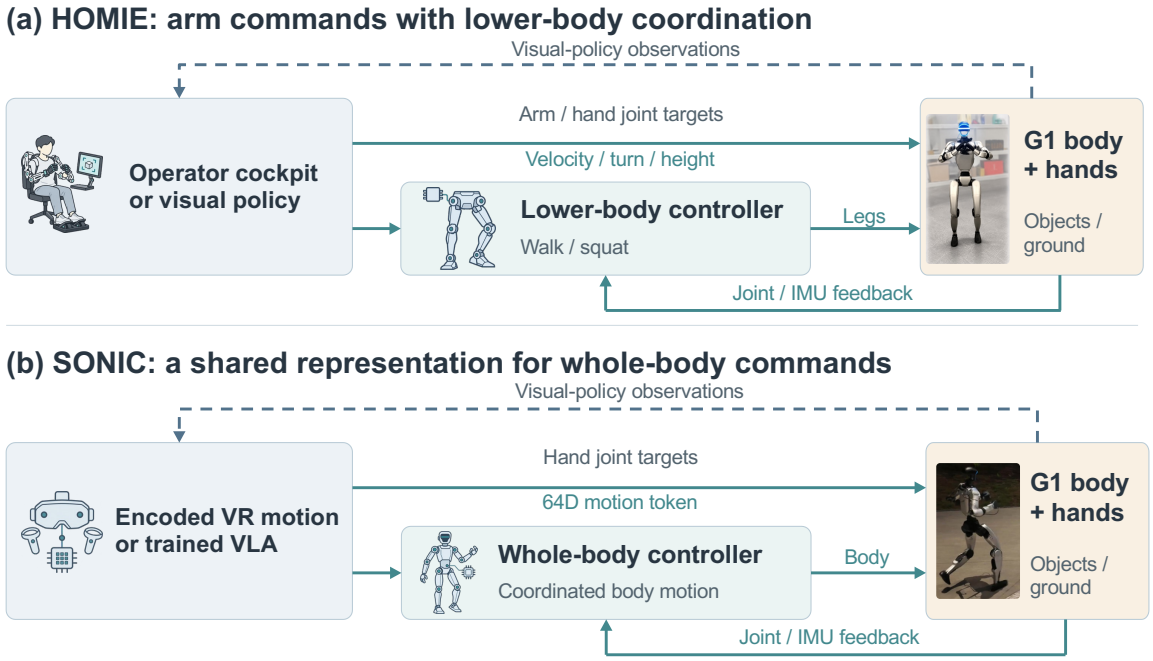


Fig. 9.17 Two G1 integration routes. HOMIE's teleoperation interface combines arm and hand targets with velocity, turning, and torso-height commands to a learned lower-body controller. Its autonomous visual policy predicts arm, hand, and height targets. In SONIC's whole-body VLA route, a shared motion token conditions the body controller, while separate joint targets command the hands. In both systems, local proprioceptive feedback supports body control, and visual observations inform task commands. VR motions in SONIC are encoded into the shared motion representation (Ben et al., 2025; Luo et al., 2026)

HOMIE: A Shared Interface for Demonstration and Autonomous Execution HOMIE connects an isomorphic arm exoskeleton, motion-sensing gloves, and foot pedals to a G1 equipped with Dex3-1 hands (Ben et al., 2025). The exoskeleton supplies arm-joint targets. The gloves command the hands. The pedals specify forward or backward velocity, turning rate, and torso height. A learned lower-body controller uses robot feedback to walk or squat as the upper-body posture changes. This allocation reduces the operator's burden. The person directs the reach and grasp while the lower-body policy supplies the compensating motion discussed in Section 7.3. The op-

erator receives no force feedback, however, limiting direct assessment of contact loads through the teleoperation interface.

The same allocation connects data collection to autonomous execution. HOMIE records images, joint states, and task commands during teleoperation, then trains a visual policy to produce arm, hand, and height targets. Its local locomotion policy runs at 50 Hz on the robot's Jetson Orin. For autonomous manipulation, head- and arm-mounted cameras and joint feedback supply a visual policy on an external RTX 4080 computer through wired Ethernet. This upper-level loop runs at 10 Hz. The task model can therefore replace the operator while retaining the controller that regulates body motion. After collecting 50 demonstrations per task, the system succeeds in 11 of 15 trials of squatting to pick a tomato from a low sofa and 12 of 15 trials of tabletop pick-and-place (Ben et al., 2025).

SONIC: Exposing Coordinated Body Motion to a VLA Some tasks require the task model to choose a foot contact as deliberately as a grasp. Opening a pedal-operated bin while holding an object, for example, requires placing one foot on the pedal while the other supports the body. SONIC connects a fine-tuned GR00T N1.5 VLA model to a reusable whole-body controller through a learned motion representation (Luo et al., 2026). In its whole-body mode, the VLA predicts a 64-dimensional motion token and 14 hand-joint targets. The token conditions coordinated body motion. The hand targets command finger configurations. Teleoperated demonstrations populate this same action representation, connecting the motor pretraining discussed in Section 7.4 to task learning.

The G1 motion stack controls 29 body joints and runs onboard a Jetson Orin. Its learned controller updates at 50 Hz, while an independent 500 Hz command publisher streams the latest joint targets to the low-level interface (Luo et al., 2026). Separating these operations sustains command delivery between policy evaluations. At the task level, the VLA-driven system completes the sequence of picking up a can, walking to a bin, pressing its pedal, and disposing of the can in 6 of 10 trials after training on 1,000 multi-object demonstrations. It also places a drill in a box and carries the box to a shelf with both hands in 7 of 10 trials after 300 demonstrations. These are complete-task results. Grasping, locomotion, and changes in support must succeed within the same execution.

The two interfaces support different task demands. HOMIE assigns the operator or visual policy direct control of the arms and hands and leaves leg coordination to a command-conditioned controller. SONIC's motion representation also lets the task model request coordinated leg motions, including a foot's interaction with an object. In both cases, reusing a motor controller becomes useful through an interface that expresses the movements needed to establish and maintain task contacts.

9.7.4 Physical and Perceptual Conditions for Task Execution

A task interface connects learned models to coordinated movement, but the range of executable tasks also depends on the robot's end effectors, the states it can observe, and the demands of repeated operation. AdaPT's G1 tennis experiments expose the coupling between the first two. Serving requires both a body capable of the swing and timely observations of the ball (Huang et al., 2026a).

For professional-style serving, AdaPT combines a task-specific racket mount with a lightweight parallel gripper that holds and tosses the ball. The heavier Unitree hand, termed Dex-3 in the AdaPT study, is evaluated with simpler amateur motions because the tested configuration cannot maintain balance with that hand during the professional serving motions (Huang et al., 2026a). The choice of hand thus changes which whole-body movements are feasible, even though its immediate role is to hold and release the ball. End-effector selection must account for the dynamic task performed by the complete body as well as the local grasp.

Executing the swing also requires observations that arrive early enough to coordinate racket-ball contact. AdaPT's principal experiments use a 35-camera motion-capture system to track the robot and ball. In a separate field deployment on the Dobot Atom humanoid, stereo vision and HTC

VIVE Ultimate Trackers provide ball observations and robot localization, respectively (Huang et al., 2026a). The motor skill is therefore evaluated together with a particular source of task-state information. The same dependence appears in locomotion. Gallant's confined-terrain case in Section 9.2.7 requires three-dimensional geometry over the space occupied by the body. In both settings, sensing must resolve the geometry relevant to making or avoiding contact. Intercepting a moving ball additionally requires predicting where contact occurs.

Sustaining these motions over repeated trials adds thermal and mechanical constraints to those of a single execution. ASAP reports motor overheating and partial damage to two G1 robots during its high-dynamic-motion data collection, limiting the real-world data available for adaptation (He et al., 2025a). This connects the actuator limits in Section 9.2 to the operating-life considerations in Section 9.5. Physical learning depends on how often the assembled robot can repeat the contacts and accelerations needed to collect experience.

9.7.5 From Integrated Demonstrations to Sustained Operation

HOMIE and SONIC demonstrate that a shared command interface can connect data collection, task models, and local body control. Their reported task trials test whether grasping, movement, and changes in support can succeed in one execution. The different tasks, demonstration sets, and trial protocols do not provide a controlled comparison between the two systems. Nor do those trial counts describe how either system performs over a working shift. The measures in Section 9.5 extend the evaluation to valid output, assistance, downtime, and recovery under a declared workload.

Table 9.7 relates the evidence discussed here to those deployment questions. Its final column states further checks for an intended deployment. It does not attribute unreported tests or protective functions to the cited systems. Hardware specifications, measured loop rates, and task success each answer a different part of the system question.

Table 9.7 From humanoid integration evidence to deployment questions. The middle column summarizes the cases discussed in this section. The final column identifies additional evidence needed for the intended operating conditions, following Sections 9.4 and 9.5

System question	Reported evidence	Further deployment question
Body and task compatibility	AdaPT couples the serving motion to end-effector choice and a specified observation setup.	Do payload, mounting, visibility, and sensing delay remain within the tested envelope at the intended site?
Feedback and execution	ASAP adapts the simulated response from robot measurements, while BeyondMimic separates estimation and policy updates.	Does the complete feedback path remain timely under the intended load, and what happens when an observation or command expires?
Task composition	HOMIE and SONIC report trials connecting hand actions to body motion through different command interfaces.	Which physical evidence permits each transition, and how are failed grasps, interrupted tasks, and operator interventions handled?
Repeated operation	ASAP reports overheating and robot damage during high-dynamic-motion data collection.	What duty cycle, maintenance, and recovery procedures preserve valid output over the scheduled operating period?

For a humanoid carrying an object, this distinction becomes concrete when the task computer or its communication link becomes unavailable. A local body controller may still receive proprioceptive feedback, but that alone does not specify whether the last task command remains authorized or how the object is secured. The runtime must define command expiry and the locally executable response for the relevant physical state, as in Section 9.4. Trials then need to establish the response under the faults assigned to that design, including the conditions for resuming the task. These

fault-response tests form part of deployment validation. The task trials summarized above do not establish their outcomes.

The hand and humanoid cases bring the chapter's five questions together. The body determines which contacts and loads are feasible. Sensing makes their consequences observable. Computation must finish while that evidence remains useful. Software assigns authority and connects commands to measured outcomes. Deployment preserves these relationships through repetition, wear, and updates. A change in fingertip material, payload, camera placement, or command timing can alter several of these relationships at once. The unit to validate is therefore the assembled observation-action system under its intended workload. Hardware and software become a dependable physical capability when they repeatedly produce the required result and retain a tested means to detect, contain, and recover from deviations.

Part III
Future Outlook

"The real problem of humanity is the following: we have paleolithic emotions; medieval institutions; and god-like technology."

— Edward O. Wilson

10 Physical AGI

The preceding chapters examine the basic concepts of Physical AI together with perception, data, world models, foundation models for robot manipulation, foundation models for whole-body motor control, agents, and hardware systems. These technologies provide the foundations for systems that perceive, reason, and act in the physical world. The question for this chapter is whether these foundations can support more than a growing repertoire of predefined physical tasks. This monograph focuses on intelligence that can interpret unfamiliar situations, develop new ways of acting, and take part in creating and maintaining the material conditions of human life. It calls the ultimate goal of this line of research *physical artificial general intelligence (Physical AGI)*. This term denotes an artificial system that integrates broad understanding, learning, and reasoning with reliable action in the physical world and can solve unfamiliar real-world problems across many domains at a level comparable to capable humans.

The qualifier *physical* gives this goal its specific meaning. An agent must connect its task-relevant knowledge to what its body can actually do, reason about the consequences of intervention, and revise its understanding when the world responds unexpectedly. Recognizing an object or describing a plausible plan is only the beginning. The agent must also find out how the object is supported, what changes once contact is made, and which uncertainties matter for execution. The physical world is thus more than the place where an intelligent system is tested. It supplies evidence, constrains reasoning, and is the medium through which the system achieves its goals.

Physical grounding, a complete technical stack, a larger task repertoire, and greater model scale do not by themselves establish generality. Chollet defines intelligence as skill-acquisition efficiency relative to priors, experience, and generalization difficulty, and thereby separates it from achieved skill (Chollet, 2019). High task performance may reflect effective learning, extensive prior training, or both. The relevant contrast is between a fixed repertoire and the ability to meet unfamiliar demands by reusing knowledge and acquiring what is missing. Scale can improve both, and Sutton identifies search and learning as general methods that keep benefiting from increasing computation (Sutton, 2019). For Physical AI, what must be shown is that these gains translate into better understanding, transfer, execution, and adaptation.

Physical AGI must therefore act effectively across tasks, environments, and embodiments, and acquire further competence when existing knowledge falls short, without being redesigned for each task. Section 10.1 makes this requirement precise. It examines what separates such a system from today's Physical AI and characterizes competence along four dimensions, namely breadth, depth, plasticity, and continuity. As Figure 10.1 shows, the subsections that follow trace four pathways toward Physical AGI, namely systematic generalization, human-robot collaboration, collective Physical AI, and self-evolution, each of which relaxes an assumption of current Physical AI. A concluding subsection considers the proposed purposes of Physical AGI in scientific discovery and in engineering beyond human limits.

10.1 Core Capabilities of Physical AGI

Most Physical AI systems today are built and evaluated for a stated set of tasks, settings, and platforms. Within that set they can be highly capable, but their competence is bounded by the scope anticipated by training and engineering, and a new demand typically calls for new data, retraining, or redesign. Physical AGI differs from such systems in generality, the "G" in AGI, rather than in higher performance on the same tasks. A Physical AI system is characterized by the skills it has. Physical AGI is characterized by how far it can carry those skills, and extend them, beyond the conditions covered by its preparation.

This chapter takes *generality* in this sense to mean the capacity to reuse knowledge and problem-solving processes across diverse tasks, environments, and embodiments, and to acquire or revise competence efficiently when existing knowledge is insufficient. Generality is always relative to a stated scope, performance standard, and resource budget, not to all conceivable tasks.

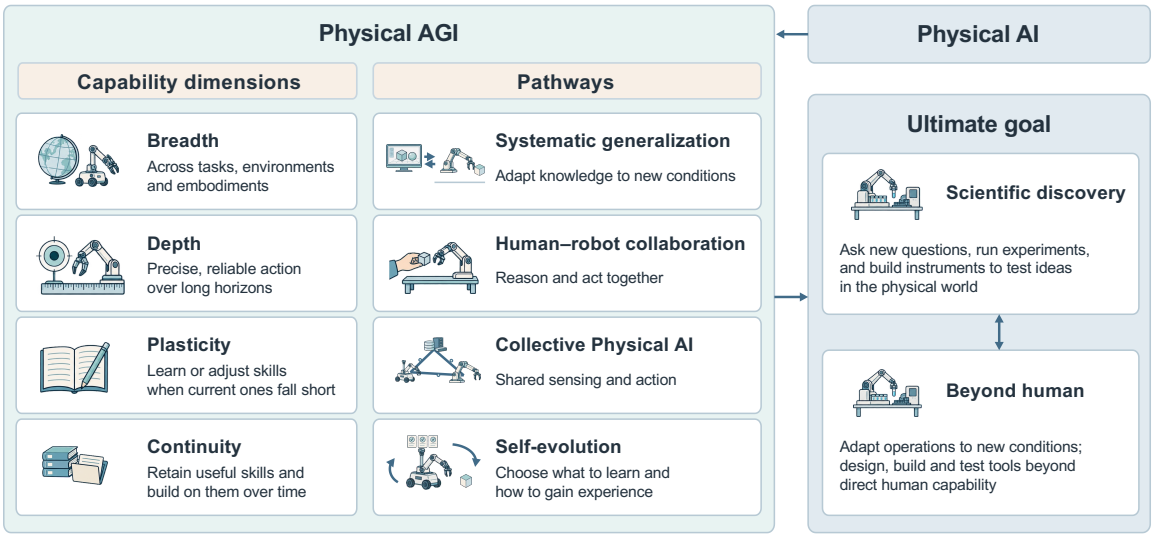


Fig. 10.1 Overview of physical artificial general intelligence (Physical AGI). Building on Physical AI, Physical AGI is described by the four capability dimensions of breadth, depth, plasticity, and continuity. It is pursued through four pathways, which are systematic generalization, collaboration between humans and robots, collective Physical AI, and self-evolution. Its ultimate goal is to advance scientific discovery and to enable engineering beyond the reach of human capability

What makes this definition physical is the relationship between knowledge and intervention. An agent must reason about objects, materials, spatial structure, forces, and its own body well enough to act on them, and it must link an abstract objective, such as repairing an object or assembling a mechanism, to the physical conditions under which that objective can be met. The link runs both ways, since interaction can expose unknown constraints and change how the agent understands the problem. Physical AGI thus acts with available knowledge and also uses action to acquire knowledge for later tasks.

The definition covers both the use of existing competence and the acquisition of new competence, and many unfamiliar conditions call only for the former. A robot may grasp an object with an unseen surface texture using a policy it already has. A narrower container opening may call for a new grasp or a contact-guided insertion, which planning with existing knowledge may still find. New learning becomes necessary only when knowledge and available skills both fall short. The deciding question is whether existing competence still suffices, not whether the situation is unfamiliar.

For the same reason, a large but fixed repertoire is weak evidence of adaptability, and fast learning within a narrow task family is weak evidence of breadth. Generality also does not mean freedom from human help. Demonstrations, feedback, and manual adjustments needed for a new task count toward its adaptation cost. The criterion is whether the system's own adaptation and learning can meet new demands within its physical, computational, and safety limits, without task-by-task redesign.

To describe both the competence a system has and its capacity to develop further, this chapter uses four dimensions. Breadth and depth describe present capabilities, following the distinction between generality and performance (Morris et al., 2024). Plasticity and continuity describe how competence is acquired and accumulated through experience.

Breadth is the range of physical conditions over which a system acts effectively, including variation in tasks, objects, scenes, tools, dynamics, and embodiments. Counting task names is a poor proxy for this range. Many differently named tasks rely on the same grasp and motion, while a small change in friction can demand quite different control. Breadth should therefore be described by

the physical variation over which competence holds. Open X-Embodiment shows that training across robotic platforms can improve performance by transferring experience between them (Open X-Embodiment Collaboration, 2024), and $\pi_{0.5}$ performs household manipulation in homes absent from its training data (Physical Intelligence et al., 2025). Both results demonstrate breadth under their evaluation conditions, but neither addresses arbitrary changes in dynamics or morphology. Sharing parameters across robots makes transfer possible, but whether transfer actually occurs is an empirical question.

Depth is how well the system performs within that range, including its accuracy, compliance, efficiency, reliability, and coordination over long horizons. In physical tasks these qualities are tightly linked. A grasp must support transport, transport must leave the object in a state from which it can be placed, and a small slip can alter the conditions for every later action. Deep competence therefore includes detecting disturbances, maintaining the conditions that later actions depend on, and recovering when those conditions are lost. Hierarchical planning supports long-horizon coordination by decomposing an objective into subgoals and reusing existing policies for familiar low-level actions (Sutton et al., 1999; Kaelbling and Lozano-Pérez, 2011). A system with many fragile skills and one with a few reliable skills are limited in different ways, and describing either requires both dimensions.

Plasticity is the capacity to acquire or revise competence when existing behavior is inadequate. A change in payload or friction may force the agent to revise its model, adjust its control, or learn a different interaction. Plasticity is measured by the experience, time, computation, supervision, and risk that adaptation requires. Two systems may reach the same success rate after very different numbers of physical trials, and their plasticity then differs even though their final performance does not. Plasticity is also distinct from retention. Dohare and colleagues show that standard deep-learning methods can gradually lose the ability to learn in continual-learning settings (Dohare et al., 2024). Forgetting concerns what a system can no longer do, whereas loss of plasticity concerns what it can no longer learn.

This chapter treats plasticity as a property of the whole embodied system, not only of its parameter updates. Memory, model-based replanning, and newly generated procedures can all support adaptation (Hafner et al., 2019; Wang et al., 2024b). Replanning with unchanged knowledge may produce an unfamiliar solution without any new competence being acquired, while new knowledge can be stored in memory or executable procedures without changing network weights. This chapter therefore judges plasticity by whether new experience leads to effective behavior, whatever mechanism carries the change.

Learning accumulates only if the results of adaptation outlast the episode and can be recalled when needed. This chapter calls this accumulation of useful competence across encounters *continuity*. It involves keeping skills that remain valid, revising assumptions that no longer hold, and preserving the ability to keep learning. Elastic Weight Consolidation, for instance, restricts changes to parameters that matter for previously learned tasks (Kirkpatrick et al., 2017). This can reduce forgetting, but it leaves open when a physical failure calls for learning, which part of the system should change, and when a retained skill should be used. These are system-level decisions, and this chapter measures continuity by the lasting benefit of experience rather than by how long parameters persist.

None of the four dimensions is sufficient alone. Zero-shot success can show breadth without plasticity, and rapid adaptation can occur without continuity. Assessing them jointly is therefore a longitudinal matter. It asks whether experience lastingly extends the conditions under which the system acts effectively, without demanding new learning in every successful encounter.

To make this criterion operational, this chapter introduces the *capability boundary*, the limit of the conditions under which a system meets a specified performance standard, with computational resources, interaction budgets, human assistance, and safety constraints held fixed. Prior training and accumulated experience should be reported separately from any resources added during evaluation. Success on a task the evaluator has not tried before may reveal competence that was already present, so novelty to the evaluator is not, on its own, evidence of capability expansion.

This chapter distinguishes three outcomes. *Direct transfer* applies existing competence without acquiring task-specific knowledge, although it may involve substantial search or planning. *Episodic adaptation* resolves the current difficulty but does not retain the useful change. *Persistent capability expansion* retains changes that let the system meet the standard under previously unsupported conditions. Claiming it requires evidence of better performance, or of lower interaction, time, or assistance costs, under comparable constraints, together with evidence that valid skills and the ability to keep learning have been preserved. Gains within conditions the system already handled improve performance or efficiency but do not expand the boundary.

Current Physical AI typically assumes that operating conditions resemble those covered by training, that a person specifies the task and the robot merely executes it, that the robot acts alone in a single body, and that people design and run its learning process and fix its physical configuration. Each of these four assumptions confines the capability boundary in a different direction. In the case of collaboration, the boundary in question is that of joint human-robot action rather than of the robot alone. Relaxing each assumption defines a pathway toward Physical AGI, a proposed system independent of all four assumptions. The next four subsections take the pathways in turn, covering systematic generalization (Section 10.2), human-robot collaboration (Section 10.3), collective Physical AI (Section 10.4), and self-evolution (Section 10.5). The four dimensions and the distinction among direct transfer, episodic adaptation, and persistent capability expansion serve as a common yardstick for all four.

10.2 Systematic Generalization in Open-World Settings

The first pathway relaxes the assumption that operating conditions resemble those covered by training. Here the distinction between transfer and capability expansion shapes how generalization tests should be read. Many out-of-distribution evaluations vary objects, positions, or task combinations within a predefined family while the basic form of the problem stays the same (Koh et al., 2021). Success shows transfer within the family but says little about what happens when the sources of change are themselves unknown.

Under open conditions, an agent may first need to work out which factors have changed and whether the knowledge behind its current policy still applies, and only then decide what to learn. A drawer that does not open may be latched, jammed, or poorly grasped, and each explanation calls for a different response. The lack of motion alone does not distinguish these cases, whereas a constrained probe can provide additional evidence. Evaluations of open-world generalization should therefore state the changes introduced, the feedback available, and the adaptation permitted.

The same issue arises when systematic generalization is framed, as it usually is, through combinations of familiar constituents. Lake and Baroni show that meta-learning can support such generalization in controlled sequence-learning tasks (Lake and Baroni, 2023). Recombination works when the constituent operations keep their meaning and effectiveness. In physical tasks the operations themselves can fail. A change in friction may break a grasp that sits in exactly the right place in the task sequence, and reordering the sequence does not repair it.

This chapter therefore defines *system-level systematic generalization* as the capacity to identify changes, preserve reusable structure, and find effective solutions when the conditions supporting existing policies no longer hold. It extends the compositional notion to perception, internal models, decision processes, and control without replacing it. A local failure may require changing only part of the system while the rest of its knowledge continues to serve the task. The agent must decide whether and where revision is needed, acquire or repair the missing competence, and test the result.

The structure retained for this purpose need not reproduce every observable detail. World models can predict in an abstract representation space without reconstructing future observations in full (Garrido et al., 2025; Assran et al., 2025), and temporal abstractions support prediction over extended actions (Sutton et al., 1999). Such a representation must preserve the information needed to evaluate possible outcomes across tasks and changing conditions. Learned largely from observation, it can supply reusable physical knowledge before a new task arrives, so that not all useful

structure has to be acquired through reward-driven trial and error (Hafner et al., 2019). Abstraction of this kind is one plausible route to generalization, but it is not part of the definition.

Representations alone do not explain how a system responds when its current solution fails. This chapter distinguishes five functions in that response, namely *competence assessment, change characterization, composition and acquisition, interactive validation, and knowledge consolidation*. Together they lead from recognizing a difficulty to building, physically testing, and retaining a revised solution. They are roles rather than software modules or a fixed sequence, and they may share representations and recur as new evidence arrives. Not all five are always needed, since existing competence may suffice.

Competence Assessment The agent judges whether continued execution is likely to remain effective and safe. Unexpected contact forces, mismatches between predicted and observed motion, or uncertain outcomes can warrant further observation, constrained probing, targeted assistance, or a safe stop. Prediction error rises, for instance, when a video shows an event that violates learned physical expectations (Garrido et al., 2025). Such a discrepancy is a reason to reassess but offers no safety guarantee, and visual familiarity does not show that a policy remains reliable, because friction can change with no visible sign. An assessment is useful only if it changes behavior early enough, which can happen before the cause of the difficulty is known.

Change Characterization Detecting a difficulty does not identify its cause. The agent must determine which assumptions need revision and which knowledge remains usable, since holding on to invalid assumptions invites repeated failure and discarding valid knowledge makes adaptation more expensive. The principle of Independent Causal Mechanisms supports selective revision, because under suitable assumptions a change in one mechanism need not require revising the others (Schölkopf et al., 2021). Whether a learned representation has captured the relevant factors is a separate question. The agent may need informative actions to tell competing explanations apart, and it can act on a partial causal account as long as the remaining uncertainty constrains what it does. Learning can then concentrate on the affected parts of the system.

Composition and Acquisition The agent builds a candidate solution from the knowledge that remains valid and identifies any competence still missing. A world model, drawing on memory and current perception, can predict the outcomes of candidate action sequences and score them against a task objective. Searching over these sequences at inference time can yield an unfamiliar solution without changing the model or learning a new skill (Hafner et al., 2019; Assran et al., 2025). A new objective may still need a learned readout from the representation, as reward prediction does in latent model-based control (Hafner et al., 2019), and such training counts toward the adaptation cost. SayCan combines language-model proposals with value functions that estimate whether learned robotic skills can be executed (Ichter et al., 2023), so its choices are limited to the skills it already has. When a required interaction is missing or no longer works, the task structure can isolate it as a local learning problem, and the agent can acquire or revise that interaction without relearning the whole task. The new competence then becomes available for other combinations, although any solution built from it still requires physical validation.

Interactive Validation The agent tests the candidate solution under the physical conditions in which it must work. Limited probes, which must themselves respect force, motion, and safety limits, can reveal resistance, contact constraints, or errors in the state estimate. Predicted trajectories can be checked against task goals and safety constraints before execution (Wabersich and Zeilinger, 2021), but only real feedback shows whether the predictions and constraint estimates hold. *Inner Monologue* uses success detection, scene descriptions, and human feedback to revise robotic plans (Huang et al., 2023b). Feedback may also show that the diagnosis itself was wrong and send the agent back to assessment or characterization. A successful recovery resolves the immediate difficulty but remains episodic adaptation unless its benefit is consolidated.

Knowledge Consolidation The system retains useful experience in a form that can inform later decisions, whether in policies, world models, memory, or executable procedures. Verified constraints and failed strategies are worth retaining alongside successful actions, together with the conditions under which each applies, so that retrieval can judge whether a skill fits the current situation. Consolidation need not take place in model weights. Voyager keeps executable skills in a retrievable library without fine-tuning its underlying language model (Wang et al., 2024b), although its Minecraft setting leaves open how the approach would fare on physical robots. In robotics, RECAP improves $\pi^*_{0.6}$ over successive rounds of data collection and training that use autonomous experience and human corrections (Amin et al., 2026). Neither system constitutes a fully autonomous lifelong-learning process, and establishing persistent capability expansion for either would require the longitudinal evidence specified in Section 10.1.

This account extends generalization from unfamiliar samples and task combinations to the whole perception, cognition, decision, and action system. The four capability dimensions specify which improvements to assess, and the five functions describe how experience can produce and preserve them. Because actions can obtain the evidence needed to revise a model or establish a new solution, the functions also connect systematic generalization to physical inquiry. Collaboration with people, the second pathway, supplies another source of such evidence, as well as a setting in which the resulting competence must work.

10.3 Human-AI Collaboration and Social Intelligence

The second pathway relaxes the assumption that a person specifies the task and the robot merely executes it. When people and robots share a task, both act on the world, and each changes the conditions under which the other can proceed. One partner's operation may change access to the workspace, the state of an object, or the support needed for a later action, so a robot can carry out its own step correctly and still leave its partner unable to continue. Joint execution therefore requires coordination between two active participants, and the robot must consider how its actions affect the person as well as the objects and the environment.

Coactive Design distinguishes individual *taskwork* from the *teamwork* needed to manage interdependence (Johnson et al., 2014). Its requirements of observability, predictability, and directability concern access to relevant information, expectations about the other participant's actions, and the ability to influence the interaction. For a robot, this means making its progress and limitations visible enough for the person to respond as the task unfolds. The participants need the information and physical conditions required to act, not identical internal models or continuous verbal communication.

How an action is performed also matters for coordination. Dragan and colleagues distinguish *predictable* motion, which matches what an observer expects given a known goal, from *legible* motion, which helps the observer infer the goal (Dragan et al., 2013). Their experiments show that motion design affects collaboration even when the movements accomplish the robot's immediate task (Dragan et al., 2015). A trajectory both changes the physical state and informs the partner's next action, and a robot should choose how to move with both effects in mind.

At the task level, Chaski selects and schedules robot actions within a shared plan, accounting for the human's progress and idle time (Shah et al., 2011). The person need not command each step, yet the person's own choices still constrain when the robot can act and which operations can follow. Scheduling should therefore aim at joint progress rather than at finishing the robot's own work as quickly as possible.

In direct physical interaction, the participants' control behaviors are coupled as well. Li and colleagues combine estimation of the human's control strategy with a differential-game controller (Li et al., 2019b), treating human behavior as part of the coupled dynamics rather than as a fixed trajectory or an external disturbance. Many collaborations involve no direct contact, however, and do not need this form of control.

Coordination can also change with repeated interaction. Nikolaidis and colleagues model uncertainty about how willing a person is to adapt and incorporate it into the robot's decision-making (Nikolaidis et al., 2016). In cross-training, participants swap roles to improve coordination within a shared plan (Nikolaidis and Shah, 2013). Both methods help the partners' actions fit together during a task, but improvement within one encounter remains episodic unless it is shown to persist or transfer.

Evaluating these interactions therefore requires attention to how the work is coordinated as well as whether it is completed. Hoffman's notion of collaborative fluency describes how well the participants' activities mesh and treats it as distinct from task efficiency (Hoffman, 2019). Useful measures include task outcomes, coordination delays, and the effort needed to keep the interaction working. The person's physical work is part of the joint task and should be reported separately from effort spent compensating for avoidable system shortcomings.

Tests of generality should vary the collaborators as well as tasks and environments, since performing well with a familiar partner does not demonstrate the same ability with a new one. Evaluations should report prior joint training and the additional coordination needed when the pairing changes, separate team competence from the robot's own competence, and identify gains that come from changes in human behavior. The resulting evidence characterizes the capability boundary of the team and the cost of forming new coordination, which a count of predefined collaborative routines cannot capture.

The longer-term goal proposed here is a physical collaborator that takes part in developing a project instead of merely executing delegated movements. Such a system would help clarify an underspecified goal, explain what it has learned about the environment, propose feasible alternatives, and acquire new methods through joint work. In a repair or construction project, the person and the robot might diagnose a problem together, redesign part of the procedure, and verify the result. Collaboration in this sense concerns participation in shared reasoning and action and does not require a human-like appearance. It also makes the preservation of human initiative, understanding, and authority a design requirement for future Physical AI systems.

Such projects can run for weeks under changing conditions. The system would need to remember why earlier decisions were made, distinguish temporary workarounds from completed repairs, and revise its plans when new evidence undermines an earlier assumption. Shared memory would hold agreements, open questions, and causal context in addition to observations, so that later actions build on earlier decisions instead of treating each encounter as a new task. Such projects rarely involve a single robot, and the same need for shared context arises when competence is distributed across several bodies.

10.4 Multi-Agent Systems and Collective Physical AI

The third pathway relaxes the assumption that the robot acts alone in a single body. Once several embodied agents are involved, the organization of physical competence becomes a question about the group. This chapter defines *collective Physical AI* as Physical AI in which a group achieves physical objectives through coordinated sensing, action, and interaction. Capability is then assessed at the level of the group and depends on how individual skills are organized, what information is exchanged, and how the bodies interact. Control may be centralized or distributed, and coordination may partly emerge from local self-organization. The number of robots and the control architecture do not by themselves indicate generality.

Members need not each possess everything the group can do. Prorok's mixture-of-robots perspective argues for specialized, interdependent components instead of the same generalist model in every robot (Prorok, 2025), which makes the distribution of competence across bodies a design choice. Shared capabilities let one member substitute for another, while specialized capabilities allow contributions tied to a particular embodiment or role (Gerkey and Matarić, 2004). Bettini and colleagues report benefits from heterogeneous policies and communication in multi-robot reinforcement learning on simulated and physical tasks with heterogeneous requirements (Bettini

et al., 2023), although their results do not show that diversity by itself improves performance. Specialization enables complementary action but also makes the group depend on particular members.

Pooling robot experience to train a policy is a different use of multiple bodies. Shared data can improve a policy that each robot later runs on its own, without any coordinated behavior at deployment. Collective competence instead rests on what members achieve by acting together. Assessing it means measuring how much the observation, manipulation, or control conditions provided by their organization contribute to the outcome.

Such organization depends on coordinating information, since a single robot's observations may not suffice for the group's decisions. Tolstaya and colleagues train local controllers by imitating a centralized controller that has global information during training, and at execution time the learned controllers rely on local observations and communicate over graphs that change as the robots move (Tolstaya et al., 2020). CoViS-Net pools visual information across robots for relative pose estimation and has been evaluated in real-world multi-robot formation control (Blumenkamp et al., 2025). As robots move, the observations available for later decisions and the coordinated actions that remain feasible change with them (Olfati-Saber et al., 2007). The system must decide what to communicate, when local information is sufficient, and how missing or inconsistent information affects group behavior. Members' internal representations need to support the task but need not agree in every detail. Evaluations should distinguish the information available during training from that available at deployment.

Sensing networks extend this organization beyond the sensors on individual robots. The term *sensing network* here denotes connected sensing resources distributed across agents, objects, and environmental infrastructure, building on research on cooperative observation through distributed sensing, local processing, and communication (Akyildiz et al., 2002). A sensing node need not be an autonomous agent. It contributes information about physical conditions that another component needs to understand or act on. The research opportunity for collective Physical AI is to organize such networks around the decisions and interventions they support. In an instrumented workshop, a fixed sensor might detect a change outside a robot's field of view, a mobile agent might approach to resolve the uncertainty, and a manipulator might then test or repair the affected mechanism, which in turn changes what the network needs to observe next. This requires tracking the origin, timing, and uncertainty of each observation, maintaining local operation when communication or sensing fails, and keeping access to observations of people and shared spaces subject to explicit authorization.

Bodies and environmental structure can also coordinate collective behavior without any explicit exchange of plans. In TERMES, Werfel and colleagues derive locally executable rules whose execution by robots with local sensing produces a specified global construction, with coordination passing through the changes the robots make to their shared environment (Werfel et al., 2014). In particle robotics, Li and colleagues build loosely coupled particles that cannot move individually but move collectively through local oscillations modulated by a global signal (Li et al., 2019a). No single component executes a complete locomotion policy, so describing such a system requires an account of its contact relationships and spatial organization as well as its controllers.

Understanding collective behavior thus involves both predicting the outcomes of local interactions and designing interactions that achieve a given objective. Stable behavior under one set of conditions shows only that a particular mechanism works. Generality requires further evidence that the relationship between local behavior and global outcome can be maintained or adjusted when objectives, component capabilities, or physical conditions change. Physical coupling belongs in this analysis as a mechanism whose contribution to task competence needs to be measured.

That relationship also depends on the membership of the group. Stone and colleagues formulate *ad hoc teamwork* as collaboration without prior coordination, distinguishing cooperation within an established team from cooperation with unfamiliar partners (Stone et al., 2010). In a physical group, a member that leaves or is impaired may remove an observation, a manipulation capability, or a source of support on which the task relies, and reassigning task labels cannot restore these condi-

tions. Reconfiguration must find a feasible organization of the remaining capabilities or recognize that none exists.

Scalability, resilience, and generality therefore need separate evidence. Scalability concerns effective operation as group size changes, resilience concerns preserving or restoring function after disruption, and generality requires testing unfamiliar combinations of members, capabilities, and task dependencies. Comparisons should account for hardware, computation, communication, and preparation costs, because adding resources can raise performance without any improvement in how they are organized. Collective generality is ultimately judged by the group's capability boundary and by the cost of establishing the relationships it requires.

Looking further ahead, collective Physical AI could operate at the scale of a home, a laboratory, or an infrastructure network, with sensing resources tracking local conditions while heterogeneous agents investigate, maintain, and repair different parts of the environment. An observation made by one component could guide an experiment carried out by another, and the resulting knowledge could inform work elsewhere. Its physical reach would depend on the relationships among models, bodies, people, and infrastructure, without every component being generally intelligent or every decision being made centrally. The research challenge is to keep evidence, decisions, and physical consequences consistent as sensing and action pass between components. A related question is how the participating systems might help improve the knowledge, skills, and physical organization on which their collective competence rests.

10.5 Autonomous Evolution

The fourth pathway relaxes the assumption that people design and run the learning process and fix the system's physical configuration. In both individual and collective settings, the question is then how a system can take part in developing its own competence over time. Here, *self-evolution of Physical AI* denotes the sustained development of physical understanding, prediction, and action through experience that the system increasingly helps to select, acquire, and use. Its objectives, authorization, and resource limits remain explicit. The definition includes learning with a fixed body and with ongoing human support, and it requires neither biological evolution nor unrestricted self-modification. Progress is measured by lasting changes in physical competence.

Tegmark's *Life 3.0* offers a broader frame for this possibility (Tegmark, 2017). In this conceptual classification, *Life 1.0*, exemplified by bacteria, has its physical organization and behavioral programs shaped largely by biological evolution. *Life 2.0*, exemplified by humans, retains an evolved body but acquires much of its behavioral repertoire through learning and culture. *Life 3.0* is a prospective technological form of life that can redesign both its software and its hardware. The classification concerns how behavior and physical organization can change, not levels of task performance.

Within a physical system, change can occur in perceptual representations, predictions of action outcomes, knowledge of the body's capabilities, control policies, or reusable skills, and these components may change at different rates. Bongard, Zykov, and Lipson demonstrate a robot that builds a predictive model of its own body through interactions chosen to discriminate between competing models, and that uses the model to generate locomotion before and after damage (Bongard et al., 2006). Cully and colleagues instead use a repertoire generated before deployment to guide trial-and-error adaptation after damage (Cully et al., 2015). Both studies recover a lost function, which differs from acquiring a broader repertoire. Gupta and colleagues evolve morphology jointly with learned control and examine how evolved bodies affect the learnability of new tasks (Gupta et al., 2021), but only in simulation, so their results do not show that deployed robots can redesign their own hardware.

From the standpoint of Life 3.0, learning a new policy on fixed hardware addresses only the behavioral side. A more ambitious system would use experience to identify a physical limitation, propose a change to its sensing or actuation, and test whether the revised configuration enables new competence. Experience with deformable objects might show, for example, that a system cannot distinguish contact states with its current sensors. Instead of collecting more of the same data, it could propose an additional sensing modality, design an experiment to measure its value, and

adapt its control to the new information. The studies above each supply one ingredient of such a process, whether learning, self-modeling, or body-controller co-design. Their integration would be an early technical step toward the co-development of intelligence and its physical substrate, but none of the studies achieves it, let alone artificial Life 3.0. Within authorized limits, such a process might begin with selecting tools or replacing modules and later extend to designing and validating new physical configurations.

A system that helps organize its own learning must also decide which uncertainties, interactions, and skills are worth investigating. Oudeyer, Kaplan, and Hafner use learning progress to direct exploration toward situations in which a robot's predictive competence can improve (Oudeyer et al., 2007). High prediction error alone is ambiguous, since it may reflect a learnable regularity, noise, or missing information that the available interactions cannot supply. The value of an exploratory action lies in the learning it makes possible, and novelty is only one factor in that judgment. Observation can further reduce how much must be learned through direct intervention. V-JEPA 2 learns first from video by self-supervision and then from robot trajectories with action conditioning, a combination that supports physical planning (Assran et al., 2025). A self-directed learner can likewise use accumulated physical knowledge to select targeted interactions and keep task-specific trial and error to a minimum.

Every physical attempt changes the environment, and automatic resets are not always available. Eysenbach and colleagues learn task and reset policies jointly and use estimates of recoverability to avoid states from which recovery is difficult (Eysenbach et al., 2018). Resets, recovery, and requests for help belong in the reported cost of learning, and the acquisition of experience must preserve the conditions under which further learning is possible.

Within these limits, current policies can also collect data for their own later training. RoboCat uses a trained multi-task robotic agent to gather experience for subsequent training iterations and evaluates adaptation across tasks and embodiments (Bousmalis et al., 2024). RECAP combines demonstrations, autonomous rollouts, and expert corrections to improve a vision-language-action policy (Amin et al., 2026). In both systems, tasks, data, or supervision are still supplied externally, so claims of autonomy should be tied to specific stages of the process. As the system takes on a larger role in acquiring and using experience, reliable external criteria for judging improvement need to remain in place.

Whether these learning processes help must be checked against later physical behavior, using the criteria of Section 10.1. Changes in prediction loss, policies, or memory count only if they improve later task outcomes, reduce the experience needed for further adaptation, or make previously unsupported conditions achievable, while valid skills and the ability to keep learning are retained (Kirkpatrick et al., 2017; Dohare et al., 2024). Tests confined to the most recently practiced task cannot determine whether development is cumulative. Persistence, transfer, and autonomy each require separate evidence, since an improvement that persists may be specific to the conditions in which it was learned. Progress need not be monotonic on every task, but gains and regressions should be reported over a stated range and time horizon, together with the costs of data collection, computation, resetting, correction, and physical recovery.

The same assessments decide which updates to accept. Updates selected with success detectors, reward models, or predictive models must be checked against the physical objectives those signals stand in for, including under conditions not used to select them. A verified fallback and the option to intervene or suspend can limit the damage of an unsuccessful update, but any safety claim still rests on the assumptions under which the fallback is valid (Wabersich and Zeilinger, 2021). Long-term development should be judged by retained physical competence and by the system's measured role in organizing its own learning, and continued operation or repeated retraining is not evidence of either.

The horizon extends beyond continual policy improvement. A developing Physical AI system would take a growing part in deciding what to investigate, which capabilities to acquire, and which tools or bodily configurations would enable further progress, and each change to its means of

interaction would open new opportunities for understanding. This reciprocal development, more than learning alone, gives the Life 3.0 analogy its concrete content.

10.6 Toward the Ultimate Goal of Physical AGI

The preceding subsections examine how Physical AI might extend its capability boundary. This subsection considers what such an extension could enable, with Physical AGI as a long-term research goal. Two prospects illustrate why extending this capability boundary could matter. The first is scientific discovery, where claims about the physical world will ultimately be tested against empirical evidence. The second is engineering at scales, levels of precision, and speeds beyond direct human capability, or in environments inaccessible to people. In both domains, the examples discussed below illustrate progress within problem settings and infrastructures largely defined by people. The prospect envisioned here is a shift toward more open-ended spaces, where systems can formulate new problems, revise their approaches, and test solutions through physical interaction. Progress toward generality would also require transferring these abilities across unfamiliar tasks and environments, supported by coordinated advances across the four pathways discussed above.

Experimental science provides a concrete setting for acquiring knowledge through action. Its cycle of hypothesis, experiment, and revision links reasoning to empirical validation. Parts of this cycle are already automated. The Robot Scientist Adam generated and tested hypotheses in yeast functional genomics (King et al., 2009). A mobile robot chemist searched for photocatalysts using instruments built for human researchers (Burger et al., 2020). Coscientist demonstrated the use of large language models to plan and run chemical experiments (Boiko et al., 2023). Kitano frames the long-term aim as an AI Scientist that conducts research at the level of the best human scientists (Kitano, 2021). These experimental demonstrations establish components of autonomous inquiry, while leaving research goals and much of the experimental infrastructure under human specification.

Physical AGI points toward inquiry in which that space is itself open. Such inquiry offers no standard answer to imitate, its feedback may take months, and failure is the normal outcome. This chapter therefore identifies three necessary shifts, from data-driven learning to world interaction, from immediate feedback to long-horizon reasoning, and from imitating correct results to learning from failure. This chapter also regards embodied autonomous exploration as a necessary route to Physical AGI. Attaching a large model to a robot is not enough, because embodied intelligence requires physical feedback that keeps revising the model. Physical Recursive Self-Improvement (Physical RSI) offers a framework for investigating how experience from physical interaction can produce persistent changes to both task capabilities and the procedures that improve them (Zhou et al., 2026). Its recursive objective is to make subsequent improvement more effective across tasks. In the vision considered here, this could contribute to Physical AGI by enabling systems to refine how they design experiments, acquire skills, and develop new tools. An artificial experimentalist must decide which uncertainty justifies an experiment, design probes that discriminate between explanations, and retain negative results with the conditions that produced them. When existing instruments cannot resolve a question, it may need to build and calibrate a new one, so that the instruments themselves become part of the space of possible designs.

Engineering illustrates how specialized robotics and control can extend human physical reach. A reinforcement-learning controller has demonstrated control over diverse plasma configurations in a tokamak (Degrave et al., 2022). Research on untethered microrobots explores the potential for procedures inside the body (Nelson et al., 2010). The underwater robot Ocean One has extended an operator's visual and tactile access through teleoperation in challenging underwater environments (Khatib et al., 2016). These examples illustrate specialized control, prospective medical applications, and human-robot collaboration. Extending such capabilities to unfamiliar tasks and operating conditions remains a challenge.

Physical AGI would open this space in two ways. The first is generality in operation, where a Physical AGI system running a fusion device or a planetary outpost would recognize when conditions have moved outside the scope of its controllers' design and adapt, sometimes faster than a

human engineering cycle. The second is open-ended design. When no tool or procedure solves a problem, the system would translate the goal into functional requirements, design and fabricate a candidate, and verify it under the conditions in which it must work. Such work is collective over years of operation, and people unable to act on site still set goals, weigh trade-offs, and authorize interventions. Human intent thus reaches regimes that human bodies cannot.

The two prospects share one arc, from operating within a predefined space to opening it, through new instruments in science and new tools and mechanisms in engineering. They also reinforce each other, since new instruments raise new questions and new understanding enables new engineering. Their value depends on reliable results and on actions that remain safe when they are hard to reverse. This monograph adopts the principle it calls the AI-45° Law (Yang et al., 2024a). Over the long term, capability and safety should advance together, with short-term fluctuation permitted but neither persistently outpacing the other. For systems that operate in laboratories, power plants, and natural environments, meeting these requirements is an operational necessity as well as a responsibility of governance. AI may accelerate the expansion of knowledge, but this monograph holds that the compass of scientific exploration should remain human curiosity and human values. Physical AGI, as proposed here, would enlarge what people can investigate and build without displacing their judgment about what is worth pursuing. These capabilities have not yet been demonstrated. Today's Physical AI systems lay the foundations, and progress toward the goal is measured by the physical results they achieve.

11 Evaluation, Safety, and Governance for Physical AI

Physical AI turns decisions into changes in the world, making reliable goal pursuit, bounded consequences, effective human control, and accountability essential to its practical value. *This chapter develops an evidence-based framework linking physical-agent evaluation, action and system safety, and lifecycle governance.* These concerns are inseparable because competence and risk emerge from interaction among models, software, hardware, embodiment, and the environment. Task completion alone cannot establish safe or autonomous operation. Evaluation and safeguards must therefore address the complete agent under specified operating conditions, with requirements proportionate to its authority and potential physical effects.

Figure 11.1 illustrates this framework around the agent's perception-decision-action-outcome loop. Physical-world evaluators (Section 11.1) establish evidence for capability assessment, benchmarking, and application readiness (Section 11.2). These findings inform operating limits and action alignment (Section 11.3), while system safety and risk governance (Section 11.4) sustain safeguards through fault containment, monitoring, updates, and maintenance. Law, ethics, standards, and global governance (Section 11.5) provide overarching responsibilities, ethical boundaries, and oversight. Field evidence feeds back into evaluation and governance, supporting reassessment as the agent and its operating conditions change.

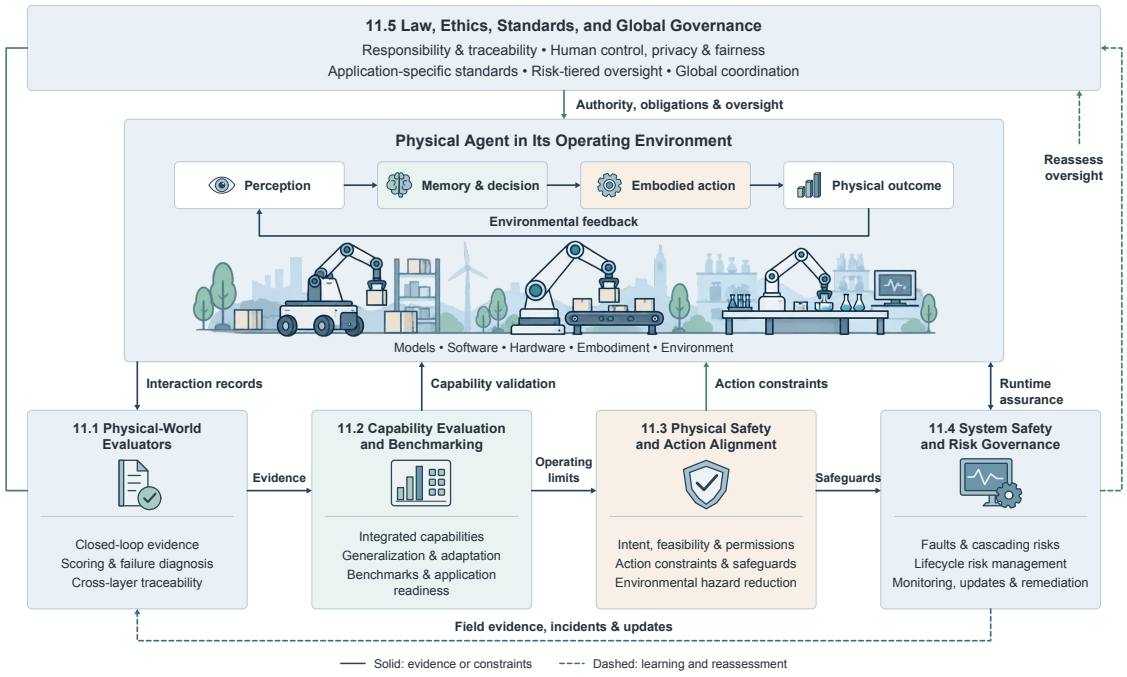


Fig. 11.1 A prospective framework for evaluating and governing physical agents connects interaction evidence, capability assessment, action alignment, and continuous system safety assurance. Law, ethics, standards, and global coordination provide overarching obligations and oversight. Solid arrows show evidence and constraints, while dashed paths feed operational experience back into evaluation and governance

11.1 Physical-World Evaluators

The chapter's approach begins with evidence about what a physical agent actually does. A physical-world evaluator observes goal-directed, autonomous or semi-autonomous interaction, compares behavior and outcomes with stated requirements, and produces judgments grounded

in recorded evidence. Its evaluation unit is the complete agent acting through its embodiment and responding to environmental feedback. The physical significance of model outputs depends on their role in this closed-loop interaction. Intended behavior, issued commands, measured execution, and final physical outcomes may not align.

This section establishes a common framework for the chapter. It first organizes evaluation around reliability, safety, operational controllability, and traceability, linking these concerns to Sections 11.2 to 11.5. It then explains how evidence from the physical agent and its environment can be collected, interpreted, and combined to support assessment, diagnosis, and improvement. A cup-transfer task illustrates the connection between general evaluation objectives and concrete physical evidence.

11.1.1 Evaluation Objectives and the Chapter Framework

The Complete Physical Agent as the Evaluation Unit Component tests remain useful within physical-agent evaluation. For example, visual recognition benchmarks such as ImageNet measure classification and detection performance (Russakovsky et al., 2015). These measurements characterize a perceptual capability under specified inputs. They do not establish whether a physical agent can select a feasible grasp, execute it reliably, detect slipping, and recover without damage. RLBench extends evaluation to interactive manipulation tasks with explicit success conditions (James et al., 2020). Nevertheless, completion alone can conceal retries, human assistance, excessive contact force, or delayed damage. Achieving a goal and avoiding unwanted consequences are distinct requirements (Amodei et al., 2016).

The evaluation unit is therefore a complete interaction episode. Perception informs decisions, embodied action changes physical states, and the resulting observations guide subsequent behavior. The sequence *perception-decision-action-outcome* describes recurring stages within this feedback loop. Evaluation should examine whether the physical agent recognizes discrepancies and appropriately continues, replans, requests assistance, or stops. This perspective applies to modular and end-to-end architectures without prescribing their internal organization.

Claims about autonomy must also match the responsibilities assigned to the physical agent. A trial in which an operator selects every grasp examines a different capability from one in which the agent receives only a goal and selects its own strategy. Evaluation must state which decisions are autonomous, which information is supplied externally, and when assistance is permitted.

Four Objectives Connecting Evaluation, Safety, and Governance The chapter develops four connected concerns introduced here. Each provides a guiding question and a principal connection to one of the following sections.

- *Reliability (Capability Evaluation and Benchmarking (Section 11.2))* Can the physical agent accomplish its goals repeatedly, with a declared degree of autonomy, under specified operating conditions? This question motivates assessment of perception, reasoning, manipulation, mobility, long-horizon execution, and generalization. Section 11.2 develops the capability tests, benchmarks, and application evaluations needed to establish what the agent can do and where that competence remains dependable.
- *Safety (Physical Safety and Action Alignment (Section 11.3))* Does the physical agent pursue goals within authorized and physically acceptable limits? Successful completion may coexist with unsafe contact, excessive force, object damage, or harmful environmental changes. Section 11.3 examines how security, environmental safety, and action alignment constrain the perception-decision-execution loop so that useful behavior remains consistent with intended and permitted consequences.
- *Operational Controllability (System Safety and Risk Governance (Section 11.4))* Can protective mechanisms and authorized supervision keep the physical agent within operating limits, intervene in time, and establish an appropriate fallback state under faults or changing conditions? Section 11.4 develops this concern through cross-layer failure analysis,

fault containment, runtime protection, and lifecycle risk management. Here, controllability concerns effective operational intervention, including automatic protection and human takeover, rather than only the availability of a stop command.

- *Traceability (Law, Ethics, Standards, and Global Governance (Section 11.5))* Can system configurations, decisions, actions, interventions, and updates be reconstructed and connected to responsible actors? Section 11.5 relates this evidence to accountability, meaningful human control, privacy, fairness, standards, and international coordination. Traceable records support investigation and responsibility reviews. They do not independently determine legal liability or acceptable use.

These connections identify the main emphasis of each section, while the concerns remain interdependent. Reliable capability informs operating limits. Action-safety requirements guide system protection. Traceability connects evidence from all three to oversight and accountability. Human control, for example, requires both effective intervention mechanisms and legitimate authority. Section 11.1 supplies the common evidence framework, which Sections 11.2 to 11.5 develop into capability assessment, safe action, sustained system protection, and accountable deployment.

Conditions and Limits of Evaluation Claims Every conclusion should identify the tested embodiment, task, degree of autonomy, permitted assistance, and operating conditions, including lighting, payload, and human proximity. The evaluator's own limitations, such as camera occlusion or missing force measurements, must also be reported. Detecting a hazard does not itself establish that the physical agent can respond safely. Communication, intervention authority, and the physical response must be assessed as well.

11.1.2 Evidence-Based Evaluation of Physical Agents

The four objectives become assessable through a connected process of specifying requirements, collecting evidence across the interaction, applying appropriate assessment methods, and using the results for diagnosis and renewed testing. This process connects component measurements to judgments about the complete physical agent.

Specify the Task and Record the Interaction In a cup-transfer task, requirements may specify the correct cup, target region, permitted spillage, completion time, and stability period after release. Safety limits, termination rules, and permitted human assistance should be fixed before testing. Records should preserve timestamps for observations, task-relevant state estimates, selected actions, issued commands, measured motion, and interventions. Commands document intended behavior. Physical measurements establish what actually occurred.

Table 11.1 links the interaction stages to evaluation evidence. Records should also capture feedback-driven changes, such as a revised grasp, an additional observation, or a protective stop. External tracking or independently reviewed video can help check the physical agent's own estimates. Controlled variations should test its response to disturbances. A shifted cup tests state updating and grasp adjustment, a blocked route tests replanning, and slipping tests detection and recovery.

Link Evidence Across Components, Embodiment, and Environment A common timeline and configuration record should connect the physical agent's models and decision functions, software, hardware, and integrated embodiment with its environmental conditions. Decision records reveal perception, task-relevant memory, and action selection. Software records reveal message delays and data freshness. Hardware measurements reveal sensing and actuator response. Complete-task observations reveal coordinated motion, grasping, stopping, and supervisory response. Environmental records identify changes in lighting, friction, object properties, and human activity.

These are complementary diagnostic perspectives, not a prescribed architecture. The environment remains the agent's interaction context, while its embodiment provides the physical means

Table 11.1 Evaluation evidence across a physical agent's interaction episode

Stage	Evaluation question	Evidence and possible output
Perception	Does the physical agent identify and locate the target?	Reference images or tracking, selection accuracy and pose error.
Decision	Are its chosen steps feasible and responsive to feedback?	Task rules and decision logs, satisfied preconditions and appropriate replanning.
Action	Does its embodiment execute commands within constraints?	Joint, force, and tactile records, together with tracking error, slipping, and interventions.
Outcome	Is the required physical state reached and maintained?	Post-action observations, placement, stability, damage, and spillage.

of sensing and action. Dependencies beyond model code are important in machine-learning systems (Sculley et al., 2015). For physical agents, their effects must also be traced through contact, motion, and feedback. Linking these records helps explain why apparently correct component outputs can still lead to an unsatisfactory physical outcome.

Apply Complementary Assessment Methods Rules, sensor analysis, multimodal assessment, and optional world-model predictions interpret the shared record for different purposes. Models used by the evaluator are assessment tools and should be distinguished from components of the physical agent being evaluated.

Executable rules test target-region membership, speed or force limits, and prerequisites such as grasp confirmation before lifting. RLBench illustrates task-specific, machine-checkable success conditions (James et al., 2020). Physical trials require corresponding measured inputs. Each rule should specify thresholds, units, temporal scope, and the treatment of missing or uncertain measurements.

Images and depth measurements reveal position and motion, force and tactile signals reveal contact or slipping, and joint measurements reveal execution. Calibration, coordinate frames, timestamp alignment, and data loss must be checked before scoring. Estimates derived from the same sensor should not automatically be treated as independent confirmation.

Instructions and video can support judgments about object selection, action order, and task progress. RoboVQA demonstrates video-based reasoning about success, previous events, and possible future actions (Sermanet et al., 2024). Such judgments require explicit rubrics, supporting frames or timestamps, and an "uncertain" option. They should be validated against human-reviewed cases and should not override calibrated measurements that establish a physical constraint violation.

A world model may estimate the consequences of a candidate action as

$$\hat{x}_{t+1} = f_\theta(\hat{x}_t, a_t; B_{\text{emb}}), \tag{11.1}$$

where $\hat{x}_t$ estimates the joint state of the physical agent and its environment, a_t is a candidate action, and B_{emb} denotes embodiment. DayDreamer demonstrates world-model-based behavior learning on physical robots (Wu et al., 2023). Related predictions may support risk assessment when their quality is validated for the relevant conditions. Predicted consequences must remain distinct from observed events. Alternative rollouts suggest hypotheses rather than independently establishing causation. World models are optional tools, not a defining requirement of physical agency.

Produce Scores with Explicit Safety and Evidence Status Scoring rules should be fixed before testing. Binary requirements may receive 0 or 1, while continuous measurements can be normalized to $[0, 1]$, with higher values consistently indicating better performance. If required evidence is

sufficient and no predefined critical safety violation is confirmed, an optional aggregate is

$$S_{\text{eval}} = 100 \sum_{k=1}^{K_{\text{eval}}} w_k s_k^{\text{eval}},$$
$$0 \le s_k^{\text{eval}} \le 1, \qquad w_k \ge 0, \qquad \sum_{k=1}^{K_{\text{eval}}} w_k = 1, \tag{11.2}$$

where s_k^{eval} is a criterion score and w_k its preselected weight. Weights apply to criteria, not to the methods supplying evidence. Missing required scores must not silently trigger weight redistribution.

A confirmed critical violation gives a failed safety assessment even if the task is completed. Missing or conflicting critical evidence gives a "review required" result unless a violation is already confirmed. In either case, withhold the aggregate while retaining individual measurements. Report completion, safety status, interventions, and evidence quality separately. Across repeated trials, state the trial count and whether completion times include only successful trials. Autonomous and assisted completion must remain distinguishable, as illustrated by RoboVQA's treatment of cognitive and physical interventions (Sermanet et al., 2024). A high score does not establish safety outside the tested scope.

Diagnose Failures and Verify Improvements Suppose the physical agent identifies the correct cup and selects a feasible route, but the cup slips during transport. Force and tactile records locate the onset of slipping. Video identifies the task stage. Command timestamps and actuator measurements reveal timing and grip response. Environmental observations indicate whether the cup was wet. The evaluation should also examine whether the agent detected the discrepancy, adjusted its grip, placed the cup down, requested help, or continued despite evidence of failure.

Diagnosis links the failed requirement and earliest abnormal event to these records, while separating confirmed events from possible causes. If delay is suspected, a controlled repeat can correct it while keeping the cup, route, and other conditions as consistent as practical. Persistent slipping motivates further checks of grip response and surface conditions. A language model may summarize the evidence, but its explanation requires validation. The evaluator itself should be checked for missed failures, false alarms, disagreement, and incomplete evidence.

After changes to learned components, software, or embodiment, repeat relevant component tests and complete interaction episodes. Simulation can broaden scenario coverage, while physical trials address contact and execution effects it does not reproduce. This connects evaluation to improvement of the complete physical agent and supplies evidence for the capability, safety, system-protection, and governance questions developed in the following sections.

11.2 Capability Evaluation and Benchmarking

Building on the evidence framework of Section 11.1, capability evaluation asks what a physical agent can accomplish, how independently it can act, and under which operating conditions its competence holds. Such competence emerges through coordinated perception, reasoning, manipulation, mobility, memory, and adaptation. Benchmarking makes these claims comparable by specifying tasks, embodiments, assistance, and resource budgets. *Simulation supports scalable capability diagnosis, while real-robot evaluation provides indispensable evidence of executed behavior and physical consequences.* Together, they connect capability profiles to claims about autonomous completion, execution quality, recovery, and deployment readiness.

This section develops a layered capability framework, compares simulation, real-robot, and hybrid benchmarks, and connects their evidence to application evaluation and governance. Figure 11.2 relates benchmark coverage to progressively demanding physical test settings, illustrated by autonomous laboratories, industrial production lines, and agricultural work. The discussion explains how simulation guides diagnosis, why deployment claims require physical trials, and how proto-

cols should account for assistance, uncertainty, and operational variation. These capability boundaries provide the basis for Section 11.3's treatment of physically safe and appropriately authorized action.

11.2.1 Layered Capability Evaluation of Physical Agents

A physical agent must convert observations into feasible decisions and coordinated actions, preserve progress over time, and respond to changing conditions. The following capability layers organize these observable responsibilities without prescribing a particular internal architecture. Their integration is the central evaluation problem. An agent must select, combine, and revise its skills as a task unfolds. Generalization and adaptation apply across the layers, while sustained closed-loop autonomy requires evidence that their coordination remains reliable during operation.

The Physical Agent as the Evaluation Unit The evaluation unit determines what a result can establish. Model-level tests assess a policy under specified observations, actions, and computational resources. System-level tests assess the coupled sensing, decision-making, control, hardware, and protective functions of the physical agent. Application-level tests add sustained workflows and the human effort needed to maintain service. Fixed-platform comparisons help isolate policy differences. Cross-embodiment comparisons assess combined model and hardware capabilities. Reports should identify which responsibilities are autonomous and which depend on external software or operators. Where accessible, both proposed and executed actions should be recorded to reveal interventions by protective controllers. Capabilities outside an embodiment's scope should be marked as not applicable, with the supported operating conditions stated.

Perception and Active State Understanding Perception evaluation concerns task-relevant geometry, affordances, and interaction states under partial observability. Recognizing a container, for example, does not establish whether its handle is obstructed or its grasp is stable. Tests should examine whether the agent changes viewpoint or performs a permitted diagnostic action when observations are insufficient. Controlled changes in occlusion, lighting, calibration, and latency can reveal state-estimation accuracy, responses to uncertainty, and the cost of acquiring useful information. Required precision should be determined by the subsequent physical operation.

Reasoning and Physical Consequence Prediction Reasoning evaluation examines goal interpretation, prerequisite recognition, feasible action sequences, and revision after unexpected outcomes. Embodied Agent Interface distinguishes goal interpretation, subgoal decomposition, action sequencing, and transition modeling (Li et al., 2024). Agent-level claims require testing these decisions through execution. Paired trials can preserve visual appearance while changing mass, friction, or physical constraints, revealing whether decisions respond to their consequences. Verbal explanations alone provide limited evidence of physical understanding. Behavioral tests establish whether an agent acts appropriately without assuming a particular internal representation.

Manipulation and Adaptive Physical Interaction Manipulation evaluation spans grasping, placement, assembly, tool use, compliant contact, and deformable-object handling. A capable agent should adjust its interaction strategy as physical properties become observable. Tests can vary mass, friction, stiffness, geometry, tolerance, and pose while measuring placement accuracy, contact forces, damage, and output conformance. Unfamiliar tools and altered contact conditions further test whether strategies transfer beyond practiced object-action combinations.

Mobility and Embodied Coordination Mobility evaluation covers navigation, obstacle avoidance, transport, balance, and whole-body coordination under changes in payload, floor conditions, and clearance. Successful navigation must deliver a usable configuration for the next operation. Reaching a room while remaining beyond grasping range leaves the workflow incomplete. Tests should therefore assess route choice and final base pose together with subsequent interaction, including revisions when access or stability changes.

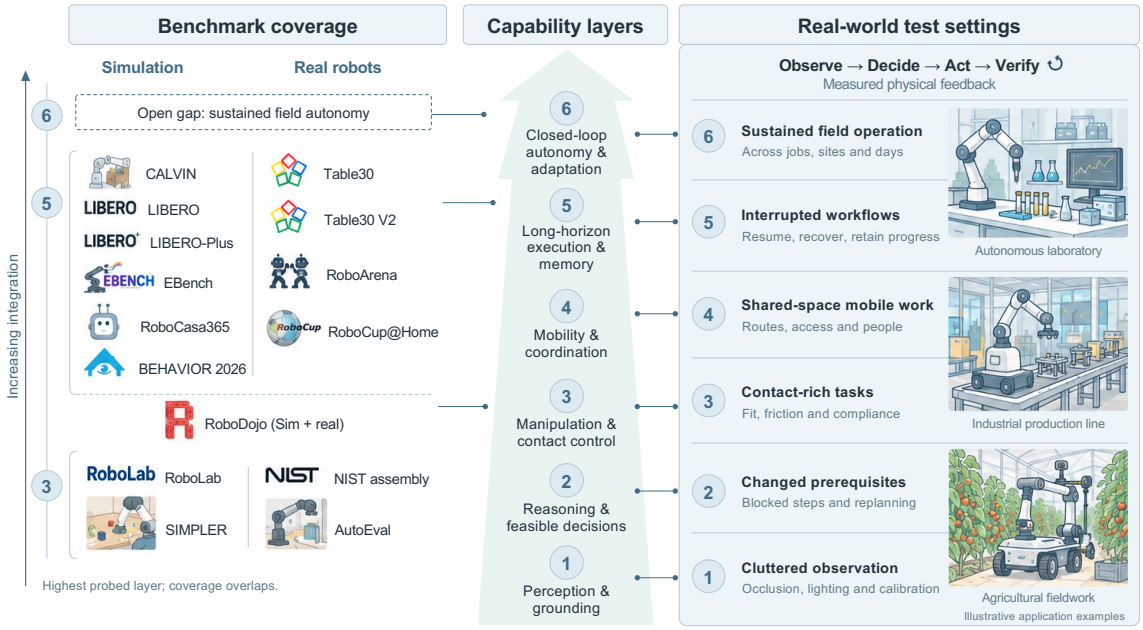


Fig. 11.2 A proposed layered framework connects benchmark coverage with real-robot evaluation settings. Each benchmark is shown once at its highest represented layer, indicating partial and overlapping coverage rather than a universal difficulty ranking. Illustrative laboratory, industrial, and agricultural settings show where actions, feedback, and physical consequences must be measured. Claims of sustained deployment require closed-loop evidence from the intended working environments

Long-Horizon Goal Pursuit and Memory Long-horizon evaluation examines subgoal formation, progress verification, memory, and transitions between operations. CALVIN evaluates supplied chains of language-conditioned manipulation tasks (Mees et al., 2022). Autonomous decomposition of a high-level objective imposes additional responsibilities. Tests with interruptions, delayed observations, or revised goals can assess whether the agent preserves valid achievements, updates its task state, and avoids unnecessary repetition. Complete-workflow success and dependence on assistance remain essential measures. Multiplying isolated skill success rates does not generally predict mission reliability, because sensing errors, state changes, and recovery decisions couple successive operations.

Generalization and Learning Through Interaction Generality must be evaluated under declared shifts in objects, sites, physical properties, task compositions, and embodiments. LIBERO studies transfer and retention during sequential learning (Liu et al., 2023a). Frozen-policy transfer, budgeted adaptation, and task-specific training should be reported as distinct evaluation conditions. Learning through interaction additionally requires measuring how efficiently experience improves later decisions and whether prior competence is retained. Data, compute, interaction, and memory budgets make these improvements interpretable. Held-out task compositions test whether familiar capabilities can be reorganized without demonstrations of the complete workflow.

Capability Integration and Diagnostic Tests Table 11.2 connects capability claims to controlled tests and observable evidence. Isolated trials help locate weaknesses. Integrated missions test coordination under dependencies and uncertainty. A failed retrieval, for example, may originate in

perception, planning, navigation, contact control, or their interaction. Diagnosis requires interventions that distinguish these explanations.

Controlled substitutions and matched interventions can support diagnosis, provided any privileged information or external assistance is declared. Systematic variation in visibility, precision, contact complexity, task dependencies, and disturbance magnitude helps establish an operating envelope. The resulting profile should distinguish limitations in individual skills from failures of coordination, rather than infer general physical competence from an aggregate success score.

11.2.2 Benchmarking Physical Agents: Current Paradigms and Future Directions

Benchmarks operationalize capability claims through specified tasks, embodiments, environments, and scoring procedures. Existing suites provide complementary evidence, from controlled skill diagnosis to physical execution and multi-stage workflows. Comparing their scope clarifies both what current scores establish and what remains to be tested for agents that pursue unfamiliar goals, adapt through interaction, and operate over extended periods.

Simulation Benchmarks: Controlled Diagnosis and Coverage Tables 11.3 and 11.4 compare representative benchmarks by task scope, setting, metrics, and training conditions. CALVIN evaluates supplied skill chains, while LIBERO studies transfer and retention during sequential learning (Mees et al., 2022; Liu et al., 2023a). SIMPLER examines correspondence between aligned simulated evaluation and real-world policy performance in validated settings (Li et al., 2025b). These protocols support controlled diagnosis, but their task definitions and observation interfaces determine which agent responsibilities are exercised.

Recent simulation suites broaden this coverage. LIBERO-Plus introduces seven perturbation families to expose vulnerabilities hidden by nominal success (Fei et al., 2026). RoboCasa365 expands household mobile-manipulation tasks and supports multiple learning protocols (Nasiriany et al., 2026). The 2026 BEHAVIOR Challenge evaluates 100 household workflows using goal-predicate completion and efficiency measures under onboard-observation restrictions (BEHAVIOR Team, 2026). Comparisons should distinguish the full task catalog, the evaluated task panel, and the available training resources, since these quantities describe different aspects of coverage.

RoboLab is a robot- and policy-agnostic simulation platform built on Isaac Lab. Its RoboLab-120 suite comprises 120 tasks organized around visual, relational, and procedural competencies (Yang et al., 2026b). Its analysis extends binary success with graded completion, execution errors, trajectory quality, and sensitivity to environmental variation. The reported evaluation emphasizes policies trained on real-world data without task-specific simulation co-training. These results char-

Table 11.2 A proposed capability-to-test matrix for physical agents. Generalization and adaptation cross all capability layers

Capability	Controlled test	Observable evidence
Perception	Occlude a handle, allow a viewpoint change	State accuracy, response to ambiguity, information-gathering cost
Reasoning	Change a prerequisite or block a planned step	Feasible alternatives, consequence-sensitive decisions, replanning
Manipulation	Vary mass, friction, tolerance, or compliance	Adaptive contact, force excursions, placement quality and damage
Mobility	Change access, floor conditions, or payload	Reachability, balance, usable final pose, route revision
Long-horizon tasks	Interrupt execution or revise an intermediate goal	Workflow success, memory, progress retention, intervention frequency
Generalization	Hold out objects, sites, properties, or embodiments	Transfer by shift type, adaptation cost, retained competence

Table 11.3 Simulation benchmarks for physical agents. Task scope, setting, metrics, and training conditions follow the cited protocols

Benchmark	Task scope	Setting	Main metrics	Training conditions
CALVIN (Mees et al., 2022)	34 skills, five-instruction chains.	PyBullet simulation, Panda arm.	Chain success, mean completed chain length.	Play demonstrations, D→D, ABCD→D, or ABC→D, declared sensors.
LIBERO (Liu et al., 2023a)	130 tasks across four manipulation suites.	Tabletop simulation.	Success, transfer and retention during lifelong learning.	Demonstrations, sequential or joint training, optional LIBERO-90 pretraining.
LIBERO-Plus (Fei et al., 2026)	LIBERO tasks with seven perturbation families.	LIBERO simulation, controlled variations.	Success by perturbation type and severity.	Existing checkpoints, distinguish additional robustness fine-tuning.
SIMPLER (Li et al., 2025b)	Manipulation with controlled visual and scene variations.	Simulation aligned to Google Robot and WidowX.	Success, sim-real ranking consistency in validated settings.	Real-world policy checkpoints, declare simulation adaptation.
RoboCasa365 (Nasiriany et al., 2026)	365-task catalog, varied kitchen scenes and task compositions.	Simulated household mobile manipulation.	Task success by split and learning protocol.	Specified demonstration mixtures, pretraining, adaptation, or lifelong learning.
RoboLab-120 (Yang et al., 2026b)	120 tasks Visual, relational, procedural competencies.	Isaac Lab, robot-agnostic task interface.	Success, graded completion, errors, trajectory quality, sensitivity.	Reported real-data-trained policies. Declare any simulation training or adaptation.
EBench (Gao et al., 2026)	26 tasks, 5 capability axes, 4 generalization shifts.	Isaac Sim, mobile and fixed-base dexterous manipulation.	Success and progress, capability profiles and held-out retention.	Simulation demonstrations for post-training, asset-separated evaluation splits.
BEHAVIOR 2026 (BEHAVIOR Team, 2026)	100 household workflows, seven scenes.	OmniGibson simulation, onboard observations.	Goal-predicate completion, time, navigation and hand-motion efficiency.	Public demonstrations, additional data allowed, no privileged evaluation inputs.

acterize transfer into a controlled simulated domain. Deployment performance still requires physical validation.

EBench diagnoses generalist manipulation policies through 26 tasks covering mobile pick-and-place, mobile long-horizon execution, and fixed-base dexterous manipulation (Gao et al., 2026). It decomposes results by scene, atomic skill, temporal horizon, precision, and operating mode, and evaluates controlled shifts in backgrounds, objects, instructions, and their combinations. Success and partial-progress scores reveal capability differences concealed by aggregate performance. The authors do not claim that these simulation scores predict real-robot performance. The suite provides reproducible diagnosis before physical evaluation.

Real-Robot Benchmarks: Direct Physical Evidence Real-robot benchmarks address physical validity while improving diversity, repetition, and access. RoboArena aggregates distributed double-blind comparisons, using the DROID platform in its reported implementation (Atreya et al., 2025;

Table 11.4 Real, hybrid, and application-oriented physical agent benchmarks. Comparison dimensions match Table 11.3, scores are protocol-specific

Benchmark	Task scope	Setting	Main metrics	Training conditions
RoboArena (Atreya et al., 2025)	Evaluator-chosen tasks across institutions.	Distributed real robots, DROID platform.	Double-blind preferences, relative ranking.	Submitter-selected training, compatible policy interface.
AutoEval (Zhou et al., 2025)	4 tasks in original prepared scenes.	Real WidowX, automated scoring and resets.	Task success from a validated detector.	Policy training varies, evaluator/reset training is separate.
Table30 (Yakefu et al., 2025)	30 multi-stage manipulation tasks.	Shared remote real robots, fixed-table setups.	Success, stage progress with retry penalties.	Provided demonstrations, task or generalist adaptation.
Table30 V2 (Ma et al., 2026b)	30 tasks, revised coverage and held-out tracks.	Shared real robots, revised protocols.	Success by track, progress at timeout.	Multi-task training, object/environment holdouts, task training may be allowed.
RoboDojo (Chen et al., 2026a)	42 simulated and 18 physical manipulation tasks.	Isaac Sim, three real robot embodiments.	Success, partial-progress score, capability-wise averages.	Provided demonstrations, Open tasks lack task-specific simulation demonstrations.
NIST assembly (Kimble et al., 2020)	Insertion, fastening, connectors and flexible parts.	Physical task boards, declared hardware.	Completion, time and assembly quality.	No common training corpus, declare programming and tuning.
RoboCup@ Home (Iocchi et al., 2015)	Domestic navigation, interaction and manipulation.	Real robots in competition home settings.	Edition-specific task and subtask scores.	Team-developed systems, league-specific preparation rules.

Khazatsky et al., 2024). AutoEval automates trial scheduling, success detection, and resets (Zhou et al., 2025). RoboChallenge provides shared remote robots, while Table30 V2 revises task coverage and held-out evaluation factors (Yakefu et al., 2025; Ma et al., 2026b). NIST assembly tests and RoboCup@Home relate measurements to industrial and domestic requirements (Kimble et al., 2020; Iocchi et al., 2015). Their results remain conditional on the selected physical tasks and protocols.

RoboDojo combines simulation and physical evaluation in one infrastructure, with 42 simulated tasks and 18 physical tasks across three real robot embodiments (Chen et al., 2026a). Its simulation dimensions cover generalization, memory, precision, long-horizon execution, and open-vocabulary instruction following. Success and partial-progress scores expose different failure patterns, while Open tasks exclude task-specific simulation demonstrations. The simulated and physical suites provide complementary coverage rather than one-to-one matched task pairs. A shared interface therefore improves evaluation access and consistency without, by itself, measuring direct sim-to-real transfer.

Why Real-Robot Evaluation Is Indispensable Simulation permits controlled variation, inexpensive repetition, and targeted diagnosis. Even high-fidelity simulation evaluates a computational representation whose contact parameters, sensing model, actuator response, and timing require physical validation. SIMPLER demonstrates useful simulation-real correspondence within validated settings (Li et al., 2025b). Such correspondence supports screening within that regime rather than unrestricted extrapolation. EBench similarly reserves deployment claims for subsequent phys-

ical evaluation (Gao et al., 2026). The value of simulation lies in exposing hypotheses and weaknesses that can then be examined on the deployed system.

Real-robot evaluation is indispensable because it observes the coupled physical system whose deployment performance is being claimed. A simulated placement may succeed while a physical cup slips because surface properties, grip response, or command timing differ from the model. Physical trials can measure contact forces, object damage, sensing errors, stopping behavior, and recovery effort, including consequences that persist after the last commanded motion. They also reveal the operational costs of resets, interventions, wear, and downtime. This evidence records what the hardware executed and what happened in the world.

Physical evidence remains bounded by the tested conditions and exposure. A laboratory demonstration does not establish unrestricted field autonomy, and incident-free trials do not establish universal safety. A credible evaluation strategy combines simulation screening, standardized physical trials, and external or distributed sites, with uncertainty and coverage reported explicitly. Relative rankings, complete-task success, and partial-progress scores answer different questions and should be interpreted within their respective protocols.

Capability Coverage and Progressively Demanding Physical Settings Figure 11.2 organizes evaluation from perception and feasible decisions to coordinated manipulation, mobility, long-horizon execution, and sustained autonomy. This ordering is a guide to increasing integration and responsibility, not a universal difficulty scale. A precision assembly task may be harder than a particular long-horizon mission. The physical settings on the right show how each responsibility can be tested through observable actions and consequences.

Each benchmark is displayed once at the highest layer probed by its represented task scope, with RoboDojo identified as a hybrid suite. Placement indicates evidence for dimensions within a layer, not comprehensive coverage or mastery by evaluated agents. Suites shown at higher layers may also test underlying skills, while generalization cuts across the framework. The open gap at the top concerns sustained field autonomy, which episode-level success and robustness tests cannot establish on their own.

Physical settings make these responsibilities observable through controlled changes and integrated work. Cluttered observation tests sensitivity to occlusion, lighting, and calibration. Changed prerequisites test feasibility and replanning. Contact-rich tasks vary fit, friction, payload, and compliance, while shared-space work varies routes, access, and human activity. Interrupted workflows probe memory, resumption, and recovery. Sustained operation across jobs, sites, and days adds evidence about adaptation, wear, service quality, and human effort. The application illustrations show possible working environments rather than results from a particular benchmark.

Simulation can test reasoning, memory, generalization, and long-horizon behavior within its modeled domain. Physical trials establish whether those capabilities remain effective under the coupled sensing, contact, timing, and operational conditions of deployment. Resuming a simulated sequence, for example, does not establish recovery after a fixture moves, a sensor drifts, or an operator changes the job. *Sustained autonomy, useful adaptation, and dependable recovery require closed-loop validation on real robots in the working environments covered by the claim.* Simplified scenes provide initial measurements. Integrated and longitudinal trials examine whether these measurements support dependable service.

A Real-Robot Evaluation Workflow The following workflow synthesizes standardized, automated, and distributed evaluation approaches (Zhou et al., 2025; Atreya et al., 2025; Ma et al., 2026b). It is a proposed procedure for this section, rather than the official protocol of every listed benchmark.

1. **Declare the protocol.** Fix goals, quality requirements, operating conditions, model and system versions, training exposure, trial budgets, permitted retries, assistance, and termination rules. Specify whether the claim concerns frozen-policy transfer, adaptation, or a complete product system.

2. **Calibrate and prepare.** Check sensors, coordinate frames, timestamps, control interfaces, end effectors, payloads, and protective functions. Prepare reproducible initial states and record unavoidable variation. Keep calibration and operator preparation distinct from autonomous task execution.

3. **Execute and record.** Run repeated closed-loop trials under the stated protection and supervision. Log observations, proposed and executed actions where accessible, physical outcomes, stops, failures, retries, and all interventions. Preserve unsuccessful episodes and verify the state after each reset.

4. **Adjudicate and measure.** Validate success detectors against independently reviewed physical evidence. Report autonomous completion, execution quality, forces or damage where relevant, intervention burden, recovery, and resource use. Automated scoring and reset errors require their own assessment (Zhou et al., 2025).

5. **Replicate and report.** Randomize or counterbalance policy order and repeat across days, task variations, embodiments, or sites as required by the claim. Report uncertainty, exposure, exclusions, and per-condition results. Use discrepancies and field failures to refine scenarios, models, and subsequent physical tests.

Challenges of Authentic Physical Evaluation Authentic physical evaluation faces variation that a nominal reset cannot eliminate. Object placement, friction, lighting, battery state, temperature, calibration, and wear may change during a campaign. Reset success must therefore be verified, and comparisons should control or record these changes. Distributed evaluation increases environmental coverage, but site and operator differences require stratified reporting (Atreya et al., 2025). Shared hardware and remote access improve comparability without removing dependence on the tested task distribution.

Protective functions and human intervention also affect what trials reveal. A stop may prevent harm while leaving the outcome of the unmodified policy unknown. Reports should distinguish proposed actions, applied safeguards, observed outcomes, autonomous recovery, appropriate handover, and assisted completion. Cost and safety exposure constrain sample size, especially for long-horizon and contact-rich tasks. These constraints motivate targeted scenario selection, validated measurements, transparent trial budgets, and uncertainty estimates. They do not make simulation a substitute for deployment evidence.

Limitations of Current Physical Agent Benchmarks The first limitation is incomplete assignment of autonomous responsibility. Supplied instruction chains, fixed catalogs, prepared initial states, and external resets may leave goal decomposition, preparation, and sustained recovery outside the test. RoboDojo adds memory and open-task evaluation, while BEHAVIOR extends household workflows, but no listed result alone establishes broad, continuing autonomy. Reports must identify which responsibilities the agent actually performs.

The second limitation is restricted evidence about generalization and physical understanding. High success in familiar settings may depend on narrow scene or instruction distributions. LIBERO-Plus exposes fragility under controlled perturbations (Fei et al., 2026). Visual variation, altered physical properties, novel task compositions, and new embodiments test distinct forms of transfer. Evaluation should separate these factors, examine their interactions, and establish whether observations and instructions influence behavior when they change the required action.

The third limitation is incomplete comparability and consequence measurement. Training exposure, sensor access, retry budgets, evaluators, and hardware can alter scores independently of policy competence. Repetition improves precision within tested conditions but does not automatically broaden coverage. Success and progress measures also leave execution quality, efficiency, safety, recovery burden, and uncertainty only partly characterized. Episodic trials with fresh resets provide limited evidence about accumulated failures, retained memory, and adaptation during prolonged operation.

A Framework for Future Physical Agent Benchmarks Future benchmarks should link capability claims to a declared coverage matrix spanning goals, physical conditions, embodiments, and consequences. Three complementary test levels are needed: diagnostic skill trials, integrated missions in intended working environments, and longitudinal sequences of changing tasks. High-level goals, partial observations, interruptions, and recoverable failures should be introduced under specified initial conditions, assistance rules, and acceptance criteria. Coverage can then be assessed against the intended application domain.

Stable anchor tasks support comparisons over time, while sequestered missions and controlled interventions test generalization and adaptation. Holdouts should cover the sites, object families, physical properties, and task compositions relevant to the claim. Frozen-policy, budgeted-adaptation, and continual-learning tracks need distinct rules for exposure and memory retention. Matched interventions can probe consequence-sensitive decisions and active information acquisition. Adaptive challenge selection requires reproducible rules and comparable resource budgets.

Evidence should connect simulation, genuinely matched physical trials, and independent sites, identifying where correspondence holds and where rankings change. Hybrid suites with different simulated and physical tasks provide complementary coverage rather than matched transfer measurements. Consequential weaknesses found in simulation should guide physical tests, which should be repeated after changes to sensing, contact, timing, or protection. Reports should combine autonomous task completion with efficiency, quality, robustness, safety, recovery, and adaptation cost. Success detectors and resets need independent validation, and partial progress remains diagnostic when mandatory conditions fail. Eq. (11.5) specifies the protocol fields needed to document these comparisons.

Maintained benchmarks also require declared roles, exposure records, controlled feedback, adjudication procedures, uncertainty estimates, and reviewable trial evidence. Changes to tasks, hardware, or scoring should be versioned and connected through anchor comparisons. Operational failures should inform revision. These proposed requirements extend a benchmark from a static score to a maintained account of what an agent can do, how efficiently it adapts, and where its reliability has been established.

11.2.3 Application Evaluation and Governance for Physical Agents

Application evaluation turns capability evidence into a decision about a defined product role. Evaluation governance makes that evidence traceable and comparable across scenarios, embodiments, and tasks. The connection depends on explicit responsibilities, measurable physical outcomes, and clearly bounded deployment claims.

Application Evaluation in Product Scenarios Product evaluation begins with the intended workflow, users, operating environment, and accepted output. It specifies autonomous responsibilities, permitted human involvement, quality tolerances, resource budgets, and conditions for stopping or handing over control. Acceptance concerns the integrated model-software-hardware-embodiment-environment system. A successful motion matters only if it delivers the required physical result under those conditions. Simulation can guide stress-case selection and development, while acceptance requires trials on the physical configuration and operating conditions covered by the claim.

Success, quality, efficiency, and reliability should be assessed together. For $N_{\text{eval}} > 0$ protocol-valid trials, including unsuccessful executions, let $Y_i^{\text{aut}} = 1$ when the agent meets all goal and quality requirements within budget, without assistance in its assigned autonomous responsibilities, and let $Y_i^{\text{aut}} = 0$ otherwise. The observed autonomous completion rate is

$$\widehat{p}_{\text{aut}} = \frac{1}{N_{\text{eval}}} \sum_{i=1}^{N_{\text{eval}}} Y_i^{\text{aut}}. \tag{11.3}$$

First-attempt success, completion after autonomous retries, and assisted completion should be reported separately. Appropriate handover should be scored according to the product workflow.

Complementary measures include accepted outputs per scheduled operating hour, energy and human effort per accepted output, availability, and execution quality. Resource accounting must include failed attempts, resets, and recovery, with external evaluation overhead identified separately. Mandatory quality and safety conditions remain acceptance requirements rather than tradeable additions to a success score.

Product trials should vary objects, illumination, access, payload, and equipment availability according to the intended operating distribution. Immediate robustness should be distinguished from improvement after permitted adaptation. For $N_{\mathrm{rec}} > 0$ prescribed recoverable-failure trials, let $Y_j^{\mathrm{rec}} = 1$ when the agent detects the failure and restores acceptable progress within budget, and let $Y_j^{\mathrm{rec}} = 0$ otherwise. The observed recovery rate is

$$\widehat{p}_{\mathrm{rec}} = \frac{1}{N_{\mathrm{rec}}} \sum_{j=1}^{N_{\mathrm{rec}}} Y_j^{\mathrm{rec}}. \tag{11.4}$$

Undetected failures remain in the denominator. Required safe termination should be assessed separately from recovery. Recovery time, additional effort, naturally occurring failure frequency, and intervention-free operating time help establish whether repeated task completion can support dependable service.

Industrial evaluation emphasizes assembly quality, cycle time, and tolerance variation. NIST task boards provide reproducible test artifacts (Kimble et al., 2020). Household evaluation combines navigation, manipulation, and interaction under unfamiliar layouts and changing requests, drawing on RoboCup@Home (Iocchi et al., 2015). Autonomous laboratories additionally require procedure ordering, sample identity, precision, contamination control, and traceability. Agricultural work adds changing crop geometry, access, and handling requirements. Across these domains, integrated trials should test whether the agent diagnoses disruptions, preserves valid progress, revises its workflow, and seeks appropriate assistance while meeting product requirements.

Unified Evaluation Governance Across Diverse Settings A common evidence framework can support scenario-specific evaluation modules. Shared definitions establish task terminology, metric meanings, protocol fields, assistance rules, and evidence requirements. Each module specifies its goals, difficulty, operating distribution, and acceptance thresholds. NIST's test-method program illustrates the separation of repeatable capability measurements from mission-specific performance requirements.[9] This principle supports comparability while preserving the physical requirements of different applications.

A task protocol can be represented as

$$\mathcal{B}_k^{\mathrm{bench}} = (\mathcal{G}_k, \rho_k^{\mathrm{init}}, \mathcal{I}_k^{\mathrm{io}}, \mathcal{H}_k, \mathcal{C}_k^{\mathrm{ctrl}}, \mathcal{U}_k^{\mathrm{budget}}, \mathcal{M}_k^{\mathrm{meas}}, \mathcal{L}_k^{\mathrm{log}}), \tag{11.5}$$

where k indexes a specified task-scenario-embodiment condition. The components are, respectively, goals and quality requirements $\mathcal{G}_k$, the initial-state distribution ρ_k^{init}, observation-action interfaces $\mathcal{I}_k^{\mathrm{io}}$, hardware and calibration $\mathcal{H}_k$, controllers and protective functions $\mathcal{C}_k^{\mathrm{ctrl}}$, resource budgets $\mathcal{U}_k^{\mathrm{budget}}$, measurements $\mathcal{M}_k^{\mathrm{meas}}$, and evidence records $\mathcal{L}_k^{\mathrm{log}}$. Common identifiers should connect scenarios, embodiments, tasks, and protocol versions. Sensors, end effectors, payloads, communication delays, training exposure, and adaptation allowances must be recorded so that the evaluated agent and its external support are identifiable.

Cross-embodiment comparisons should begin with shared functional outcomes. A fixed arm, mobile manipulator, and humanoid can be compared on an assembly operation when workspace access, quality requirements, and difficulty are matched. Embodiment-adapted fixtures require validation that these conditions remain comparable. Absolute physical measurements should accompany any baseline-normalized indicators. Capability profiles should declare coverage and nonapplicable tests. Product assessment must still count a workflow failure when an unavailable capability is required by the claimed role.

[9] NIST, *Performance of Emergency Response Robots*, program overview.

Results should be stratified by goal type, difficulty, physical conditions, and embodiment before matched groups are compared. Per-group performance, uncertainty, and coverage should precede aggregation. Equal-task averages characterize breadth, whereas deployment-weighted averages characterize a specified use distribution. Both require a common task panel and declared weights. Denominators, exclusions, and adaptation budgets should be published, with consequential low-frequency conditions retained. Cross-platform policy transfer and independently optimized product systems also require distinct comparison tracks.

Governance assigns stewards to maintain protocols, operators to prepare trials, evaluators to adjudicate outcomes, and reviewers to examine disputes. Combined roles and conflicts should be disclosed. Automated judgments and resets need validation against independently checked trials. Randomization or counterbalancing, declared stopping rules, and treatment of clustering across sites and days make comparisons interpretable. Incident reports should state severity and exposure. For $N_{\text{eval}} > 0$ independent Bernoulli trials with a common incident probability, zero observed incidents yields the one-sided 95% upper bound

$$p_{\text{upper}} = 1 - 0.05^{1/N_{\text{eval}}}. \tag{11.6}$$

Correlated trials require an uncertainty analysis that accounts for their dependence. Limited incident-free exposure therefore supports a bounded statement about the tested regime, not a general safety guarantee.

Evidence records should link configurations, observations, actions, interventions, and judgments, with appropriate access controls for private information. Permitted learning experience must remain separate from hidden scoring feedback. Material changes to software, hardware, tasks, or scoring require versioning and targeted re-evaluation, while shared anchor tasks connect successive assessments. This structure supports both a comparable capability profile and an application-specific acceptance decision, keeping each claim tied to observed physical behavior under declared conditions.

11.3 Physical Safety and Action Alignment

The capability evidence developed in Section 11.2 must be complemented by constraints on how goals are pursued. In Physical AI, an erroneous decision can become motion, contact, or an environmental change before correction is possible. Safety therefore depends on the evolving relation among human intent, the agent's decisions, physical feasibility, and permitted consequences. This includes both preventing hazards created by the agent and ensuring that actions intended to reduce existing environmental hazards do not introduce new ones. Detecting danger, possessing the means to intervene, and having authority to act are separate requirements.

This section first explains the transition from content safety to physical action safety, examining separation, contact force, energy, object integrity, persistent environmental effects, and hazardous task interpretations. It then develops multi-layer action alignment from task reasoning through planning, real-time control, and actuator safeguards. Shared action contracts connect intended effects, preconditions, operating bounds, and termination conditions across these layers. The discussion shows why successful task completion can coexist with unsafe or unauthorized behavior, and why warning, clarification, assistance, or safe termination may be appropriate outcomes. These action-level requirements provide the basis for the integrated-system and lifecycle assurance developed in Section 11.4.

11.3.1 Physical Agent Security

The security of a physical agent concerns its ability to maintain the integrity of perception, decision-making, and action under malicious interference. As defined in Chapter 8, a physical agent is a goal-directed system that integrates sensing, computation, physical embodiment, execution, and feedback. Its security cannot therefore be reduced to the robustness of an individual foundation model. An adversary may influence the agent through its observations, instructions, learned components, communication interfaces, or control commands. Unlike attacks on purely digital appli-

cations, successful interference with a physical agent may propagate into unauthorized motion, inappropriate contact, or other physical consequences. Security must consequently be considered across the complete perception-decision-execution loop.

Threat Models and Attack Surfaces Security threats should first be distinguished from ordinary uncertainty and physical failure. For example, a sensor may produce an incorrect measurement because of noise or occlusion, whereas an adversary may deliberately manipulate the observed scene to induce a particular decision. Similarly, a policy may fail on an unfamiliar object without being attacked, while a compromised model may exhibit malicious behavior when a specific trigger appears. The distinction matters because defenses against distribution shift do not necessarily protect against an adaptive adversary. A useful threat model should specify the attacker's objectives, accessible interfaces, permitted modifications, and intended effects on the agent. Relevant security properties include the integrity of observations and commands, authenticity of instructions, availability of essential functions, and preservation of authorized decision-making. SafeClawArena evaluates system-level security in persistent software agents, including failures arising from interactions among their components (Niu et al., 2026b).

Adversarial Perception Attacks Perception provides an important attack surface because physical agents depend on observations that may originate outside their trusted computational boundary. Adversarial perturbations can influence learned perception without requiring access to the model's parameters. Eykholt et al. (2018) demonstrate that physical modifications to traffic signs can induce incorrect visual classifications under varying viewpoints and environmental conditions. Such results illustrate how deliberately constructed physical inputs may alter downstream decisions. For a physical agent, the consequences depend on the role of the compromised perception output. An incorrectly identified object may affect grasp selection, while a manipulated environmental feature may influence navigation or task planning. Possible defenses include multimodal consistency checks, temporal verification, detection of abnormal observations, and conservative responses when critical state estimates are unreliable. Their effectiveness must be established against specified attack conditions rather than assumed from improved nominal perception accuracy.

Prompt Injection and Instruction Manipulation Physical agents increasingly use language models to interpret commands, decompose tasks, and select actions. This creates a security boundary between authorized instructions and information obtained from less trusted sources. Text detected in the environment, retrieved documents, external messages, or tool responses may contain instructions intended to redirect the agent. RoboPAIR (Robey et al., 2024) and BadRobot (Zhang et al., 2025b) investigate adversarial attacks against robotic systems controlled by large language models (LLMs), demonstrating that manipulated instructions can induce harmful physical-action decisions under the evaluated conditions. These findings motivate explicit separation between task authority and contextual information. An agent may use external content to understand its surroundings, but such content should not independently authorize new objectives, override operational restrictions, or disable protective mechanisms. Memory adds a temporal dimension to safety risks in LLM agents, as information accumulated during benign interactions can contribute to violations on later, unrelated tasks through temporal memory contamination, even without adversarial input (Al-Tawaha et al., 2026).

Model Backdoors and Supply-Chain Attacks Security risks also arise before deployment. Physical agents may incorporate pretrained models, third-party skill libraries, training datasets, and remotely maintained software components. An attacker with influence over these resources may introduce hidden behaviors that are difficult to identify through ordinary testing. BadNets (Gu et al., 2017) demonstrates how malicious training procedures can insert trigger-dependent behavior into learned models while preserving apparently normal performance. This attack mechanism provides a threat model for learning-enabled components in physical agents. Practical safeguards include verifying model and data provenance, restricting component permissions, validating software updates, and testing for suspicious behavior under controlled trigger conditions. These measures should extend to the interfaces connecting learned models with executable robot skills.

Security vulnerabilities may also occur outside the learned models. Sensor spoofing, unauthorized remote commands, communication interference, and compromised middleware can modify the information or control signals exchanged among agent components. A perception model may function correctly while receiving falsified measurements, and a planner may generate valid commands that are subsequently altered before reaching the controller. Protecting the foundation model alone is therefore insufficient. Authentication, integrity checks, access control, interface isolation, and monitoring of command consistency provide complementary safeguards.

The key challenge is that attacks can propagate across layers. A manipulated observation may change a high-level objective, which leads to a plausible plan and is then faithfully executed by an otherwise functioning controller. Security evaluation should therefore examine complete attack-to-action pathways, including whether the agent detects suspicious inputs, preserves authorized objectives, rejects unauthorized commands, and prevents compromised decisions from producing unacceptable actions. Independent physical constraints provide an additional boundary when upstream components fail, but they do not replace protection against unauthorized control. Physical agent security is ultimately concerned with maintaining trustworthy agency. The agent should act according to authorized objectives despite malicious attempts to alter its perception, decisions, or execution.

11.3.2 Layered Environmental Safety and Action Alignment

Physical agents can contribute to environmental safety by perceiving hazardous conditions, selecting appropriate responses, and executing interventions that reduce physical risk. This role extends beyond ensuring that the agent itself is not attacked or compromised. The surrounding environment may contain unstable objects, moving machinery, excessive temperatures, leakage, obstructed pathways, or unsafe interactions among people and equipment. Such hazards can arise independently of the agent's behavior. Environmental safety therefore concerns whether the agent can recognize relevant physical conditions and influence their evolution without introducing additional risks.

A physical agent's contribution to environmental safety depends on the coordination of several capabilities. Perception provides evidence about the current state. Foundation models and task reasoning connect observations with safety objectives. Planning identifies feasible interventions. Control and actuation realize those interventions under physical constraints. Feedback determines whether the intended safety improvement has occurred. These functions may be implemented through separate modules or integrated learned policies, but their responsibilities remain distinguishable. The relevant alignment problem is to maintain consistency among human intent, agent decisions, physical feasibility, and environmental safety requirements throughout execution.

Environmental Perception and Hazard Identification Environmental protection begins with identifying physical states that may lead to unacceptable consequences. Relevant observations include geometry, object stability, contact relationships, human proximity, equipment motion, temperature, pressure, vibration, and other process-specific variables. A single visual observation may reveal an obstruction but provide insufficient evidence about its mass or stability. Likewise, an abnormal equipment image may not establish whether a mechanical failure is developing. Multimodal sensing and temporal state estimation can provide complementary evidence, while uncertainty estimates help determine whether additional observations are required.

AnomalyGPT (Gu et al., 2024) illustrates how vision-language models can support industrial anomaly detection and localization. Such methods provide a possible perceptual foundation for environmental safety agents, particularly when hazardous conditions have recognizable visual manifestations. However, anomaly recognition is not equivalent to hazard assessment. The system must relate the detected condition to physical consequences, operating requirements, and the potential development of the environment. A physical agent should therefore distinguish observed abnormalities from evidence-supported risks and unresolved conditions that require further inspection.

Foundation-Model Reasoning and Safety Objectives After identifying a potential hazard, the agent must determine what response is appropriate. This requires connecting physical observations with task objectives, human intentions, operating permissions, and safety priorities. A household agent may detect a spill near electrical equipment, while an industrial agent may observe that a person has entered a machinery workspace. In both cases, the desired response depends on context rather than object recognition alone. Possible responses include warning nearby people, restricting access, delaying the current task, requesting assistance, or initiating an authorized physical intervention.

Foundation models can assist by interpreting safety-related observations and decomposing high-level objectives into executable steps. SayCan (Ichter et al., 2023) illustrates a mechanism for combining language-based action selection with estimates of robotic skill feasibility. For environmental protection, a related approach can associate safety-oriented skills with explicit preconditions, permitted interactions, and expected outcomes. An agent may identify that an obstruction should be removed while determining that its available manipulation skills are insufficient to move the object safely. In such cases, requesting assistance can be more appropriate than executing a semantically plausible but physically unsupported action.

Human intent and environmental safety requirements must also remain aligned. An instruction to transport an object does not authorize passing dangerously close to a person, exceeding contact-force limits, or disturbing hazardous equipment. Conversely, detecting a hazard does not automatically grant the agent permission to perform unrestricted intervention. A practical implementation can distinguish the requested objective, the agent's authorized actions, and mandatory environmental constraints. These elements define the permissible scope within which autonomous decisions can be made.

Safety-Constrained Planning Planning converts safety objectives into physically realizable actions. Candidate interventions should be evaluated for reachability, collision, stability, contact conditions, workspace restrictions, and possible effects on surrounding objects or people. VoxPoser (Huang et al., 2023a) illustrates how language-level constraints can be grounded in three-dimensional spatial representations to support manipulation planning. Such representations provide one possible mechanism for translating environmental safety requirements into motion-related objectives, although their use does not independently guarantee safe execution. Safe reinforcement learning complements planning by incorporating safety requirements into policy optimization. Constrained Policy Optimization optimizes expected return under bounds on expected discounted cumulative costs (Achiam et al., 2017), with a multi-agent extension for multi-robot control (Gu et al., 2023).

Environmental protection requires considering both intervention and non-intervention. Moving an unstable container may reduce one hazard while increasing the probability of spillage. Similarly, an agent approaching malfunctioning machinery may expose itself or others to additional risk. A safety-aware planner should therefore compare alternative responses according to their expected consequences and uncertainty. Observation, warning, isolation, or human assistance may be preferable to direct manipulation when the available evidence is insufficient.

The admissibility of an intervention can be expressed through constraints on the predicted physical trajectory. Let b_t denote the agent's belief about the joint state, $\mathbf{a}_{t:t+H-1}$ a candidate action sequence, and C_i^{safe} a task-dependent safety constraint. Let $X_{t:t+H}$ denote the predicted random state trajectory under the belief and candidate actions, and let N_{constr} be the number of safety constraints. One possible joint chance-constrained formulation is

$$\Pr\left(\bigcap_{i=1}^{N_{\text{constr}}} \left\{C_i^{\text{safe}}(X_{t:t+H}, \mathbf{a}_{t:t+H-1}) \leq 0\right\} \,\middle|\, b_t, \mathbf{a}_{t:t+H-1}\right) \geq 1 - \varepsilon_{\text{risk}}, \tag{11.7}$$

where H is the prediction horizon and $\varepsilon_{\text{risk}}$ bounds the probability that at least one constraint is violated. This formulation is conditional on the validity of state estimation, dynamics prediction, and uncertainty characterization. It provides a possible planning criterion rather than a universal safety guarantee.

Control and Physical Execution Even an admissible plan may become unsafe when environmental conditions change during execution. A person may enter the workspace, an object may slip, or the contact dynamics may differ from those assumed during planning. Low-level control must therefore monitor the actual physical state and adjust commands when motion, force, or proximity approaches a safety boundary. Control barrier functions provide one established mechanism for enforcing such constraints. Ames et al. (2017) formulate safety-critical control through barrier functions and quadratic programming, allowing nominal commands to be modified when necessary to preserve a specified safe set. Their guarantees depend on the assumptions concerning system dynamics, state information, and available control authority.

The actuator and hardware layer supplies further physical protection through joint limits, torque limits, compliant mechanisms, braking systems, and protective stops. These mechanisms bound the physical consequences of commands that pass through higher-level decision-making. They also determine which environmental hazards the agent can address. A robot may recognize a high-temperature or heavy object without possessing the mechanical protection, payload capacity, or sensing required for safe manipulation. The ability to identify a hazard should therefore not be confused with the capability to remove it.

Cross-Layer Action Alignment Environmental safety requires these layers to operate under consistent conditions. One possible mechanism is an action contract specifying the intended intervention, required preconditions, authorized contacts, physical limits, termination criteria, and evidence needed to verify the outcome. Foundation-model reasoning identifies the safety objective, planning establishes a feasible intervention, control constrains its execution, and feedback determines whether the relevant hazard has been reduced.

This organization also clarifies why task completion alone does not establish safe behavior. An agent may remove an obstruction while damaging nearby equipment or exposing a person to another hazard. Conversely, an agent that declines direct intervention because the available evidence is insufficient may be acting appropriately. The objective is not unrestricted autonomous intervention, but the reliable realization of authorized safety objectives under physical and operational constraints.

11.3.3 Predictive Safety and Degradation

Conventional safety mechanisms often respond after a hazardous condition has been detected, for example when a collision occurs, a contact-force threshold is exceeded, or an emergency stop is activated. Such mechanisms remain necessary, but their effectiveness may be limited when physical consequences develop faster than corrective action can be applied. A more capable physical agent should anticipate risks before execution, monitor whether its assumptions remain valid, and modify its behavior while safer alternatives are still available. This progression applies both to preventing unsafe agent behavior and to protecting the surrounding environment from developing hazards.

Risk Prediction Predictive safety begins with estimating how the physical state may evolve under candidate actions or existing environmental conditions. Geometric checks, dynamic models, contact prediction, and learned representations can assess collision, instability, excessive force, or inadequate stopping distance. The prediction horizon should reflect the relevant physical process rather than only the next control step. A grasp may remain stable during initial lifting but become unsafe during subsequent acceleration, while a navigation trajectory may become hazardous as nearby agents move. Evaluating these possibilities before execution allows the physical agent to compare actions according to their expected consequences.

For environmental protection, prediction also considers how a hazard may develop without intervention. An inspection agent may observe increasing equipment temperature or vibration, while a mobile agent may estimate whether moving objects are approaching a dangerous configuration. The agent can then compare continued observation, warning, physical intervention, and human escalation. Prediction should include uncertainty because physical environments are partially ob-

servable and their dynamics are not always known. When uncertainty can be reduced through additional sensing, the agent may change its viewpoint or acquire another measurement. When further observation is insufficient or delay is dangerous, conservative constraints or restricted operation may be more appropriate.

Self-Checking and Execution Verification Prediction alone cannot establish safety because the predictor may itself be incorrect or compromised. A physical agent should therefore verify critical conditions before, during, and after execution. Before acting, it checks task preconditions, relevant permissions, and the current physical state. During execution, it compares observed motion, contact, and environmental changes with expected behavior. After execution, it determines whether the intended outcome occurred and whether unintended effects remain. Inner Monologue (Huang et al., 2023b) demonstrates how environmental and task feedback can inform the revision of language-based robotic plans. This provides one possible route toward closed-loop decision revision, although safety-critical conclusions still require reliable physical evidence.

Self-checking should use evidence that is sufficiently independent of the mechanism generating the action. A policy that proposes a trajectory and then evaluates its own safety using the same potentially incorrect representation provides limited protection against shared errors. Independent geometric checks, force or tactile measurements, redundant state estimates, and separately implemented monitors can expose discrepancies. For security-sensitive agents, these mechanisms can also help detect inconsistencies between authorized instructions, received observations, and executed commands.

A practical runtime process can therefore follow a predict-execute-verify cycle. The agent evaluates a candidate action, executes it while monitoring relevant constraints, and verifies the resulting state before proceeding. For environmental protection, the corresponding process extends to an observe-predict-intervene-verify sequence that identifies a potential hazard, estimates its development, selects a justified intervention, and confirms that the hazard has been reduced. If observations differ substantially from prediction, the agent should revise its plan rather than continue an obsolete action sequence.

Safe Degradation and Fallback Control When a physical agent cannot maintain its nominal operating conditions, it should reduce its capabilities in a controlled manner instead of continuing unrestricted operation. Possible degraded modes include reducing speed, restricting workspace access, disabling contact-intensive skills, increasing human supervision, or transferring control to a simpler policy. SOTER (Desai et al., 2019) illustrates a runtime-assurance architecture combining an advanced controller with a safety-oriented fallback controller, with switching governed by specified conditions. Such mechanisms provide a possible implementation of controlled degradation, but their guarantees remain conditional on the validity of the safety specifications and controller assumptions.

Degradation must also apply to agents performing environmental safety functions. An inspection agent with unreliable localization may continue collecting local measurements while suspending autonomous navigation near hazardous equipment. An agent that detects a dangerous condition but cannot determine a safe manipulation strategy may issue a warning or request assistance instead of intervening directly. Likewise, when the integrity of critical observations or commands cannot be established, the agent should restrict actions that depend on the affected components. The available operating modes should reflect the information and control capabilities that remain trustworthy.

Importantly, entering a degraded mode must not create a new physical hazard. Immediate stopping may be inappropriate when a manipulator is carrying a heavy load or a mobile system is maintaining dynamic stability. The agent may first need to secure the object, reduce stored energy, or reach a stable configuration. Similarly, an intervention intended to protect the environment should not be abandoned in a state that increases the original risk. Returning to normal operation should require evidence that the conditions responsible for degradation have been resolved.

Experience from near misses, rejected actions, unexpected environmental changes, and unsuccessful interventions can provide evidence about declining safety margins. These observations may reveal weaknesses that are difficult to identify through ordinary task-success measurements. They can inform improvements to risk prediction, sensing, action constraints, and regression testing. The objective is not to assume that a physical agent can prevent every failure, but to recognize emerging hazards before they produce unacceptable consequences, preserve appropriate control under uncertainty, and reduce autonomous capability when safe continuation cannot be established.

11.4 System Safety and Risk Governance

Action alignment for a highly intelligent physical agent depends on an integrated physical system whose sensing, computation, communication, control, actuation, and power remain adequate for the task. Even correct AI-model decisions can be undermined by delayed commands, unreliable observations, degraded actuators, or interacting failures across these layers, compromising the constraints developed in Section 11.3. Conversely, an incorrect model output need not cause physical harm if independent safeguards can constrain or prevent unsafe execution. Safety assurance must therefore examine how the complete agent detects faults, contains their propagation, and preserves an appropriate physical state. It must also account for changes in hardware condition, software, task permissions, and operating environments, because evidence from initial testing may no longer describe the deployed agent.

As illustrated in Figure 11.3, system safety and risk governance follow the three connected stages of cross-layer risk analysis, risk-driven assessment, and lifecycle safeguards. Cross-layer failure analysis first investigates how local faults propagate into hazardous physical outcomes and how independent safeguards can interrupt this process. These failure pathways then motivate system-level safety assessment, which examines observable degradation, evaluation indicators, and protective responses across the hardware and software layers of a physical agent. Beyond these immediate risks, lifecycle risk governance connects hazard analysis and pre-release verification with deployment, maintenance, and operational reassessment. Continuous safety assurance is integrated into this lifecycle perspective through runtime monitoring, protective intervention, user-facing fault communication, field feedback, controlled software updates, and physical remediation. As physical agents move toward increasingly autonomous operation in homes, factories, and public environments, these mechanisms shift safety evaluation from a one-time test result toward a continuously maintained body of evidence throughout the product life cycle. They also establish the technical records and responsibilities needed for the broader legal, ethical, and institutional governance discussed in Section 11.5.

11.4.1 Cross-Layer Failures and Cascading Risks

Perception and Timing Cascades A local fault becomes a physical hazard when it propagates through the perception-planning-control-actuation chain. A mobile robot moving from a dim room into strong illumination, for example, may receive temporarily saturated camera frames while automatic exposure adjusts. Industrial-camera documentation confirms that exposure changes may take effect only after subsequent frames have been acquired (Basler AG, n.d.). Benchmarks of visual depth estimation further document sensitivity to lighting, sensor, and data-processing corruptions (Kong et al., 2023). If these observations are treated as reliable, obstacle detection may fail and a seemingly coherent navigation decision may increase collision risk. A different cascade begins in real-time scheduling. As a hypothetical example, a controller nominally updated every 10 ms may experience a delay of 100 ms because of thread contention or blocking. Meanwhile, momentum, gravity, and contact continue to change the robot state. Research on control timing explains why execution delays must be analyzed with closed-loop dynamics rather than average processing speed alone (Cervin et al., 2003).

Shared Infrastructure and Physical Actuation Processor overheating may activate thermal throttling, protecting a chip while reducing the computation available for time-critical decisions (NVIDIA Corporation, n.d.). Communication timeouts and inconsistent timestamps can

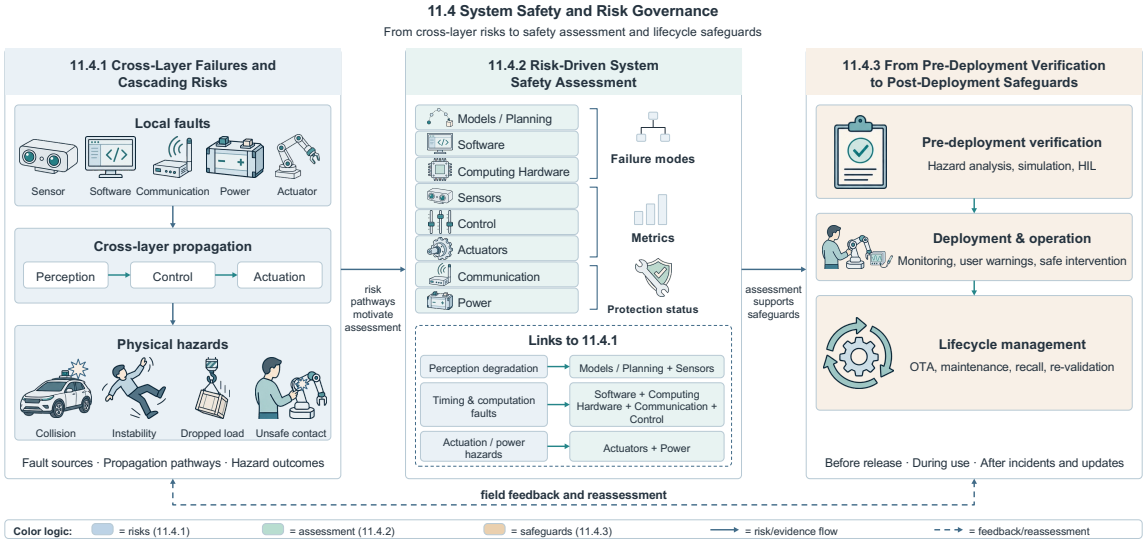


Fig. 11.3 Risk-driven framework for system safety and risk governance. (a) Faults in sensing, software, communication, power, and actuation can propagate through perception, control, and execution into physical hazards. (b) The corresponding safety assessment maps perception degradation to sensor-validity and estimator checks. It maps timing and computation faults to latency, jitter, and command-age tests. It maps actuation or power hazards to current, thermal, and power-loss verification. (c) Safety evidence supports pre-deployment verification and post-deployment monitoring, user warnings, protective intervention, updates, maintenance, and reassessment. Solid arrows indicate the progression from identified risks to assessment and safeguards. The dashed arrow denotes field feedback

make feedback stale. The need to detect communication anomalies is also well established in networked control systems (Baglietto et al., 2020). Undervoltage or power interruption may simultaneously disable multiple subsystems, making the robot's power-loss behavior a distinct safety-verification concern. At the joint level, an unsafe position target is not identical to a motor current. A controller translates tracking error into drive demand, which a motor driver regulates under the actual mechanical load. If a joint is obstructed, persistent error may sustain high demand. Without suitable protection, overheating, transmission damage, or loss of support may follow. Existing drivers demonstrate current regulation, overcurrent protection, and thermal shutdown (Texas Instruments, 2019), while surveys of robotic fault-tolerant control document the broader need to detect and accommodate actuator faults (Milecki and Nowak, 2023). These functions must be verified for the specific actuator rather than assumed universally.

Containment Under Degraded Conditions The appropriate response depends on the remaining physical capabilities. Command-age checks, watchdogs, current and torque limits, thermal protections, and independent safety supervision can interrupt some cascades, but a generic emergency shutdown may create another hazard. Abrupt removal of joint power can destabilize a balancing humanoid or cause a load-bearing manipulator to drop its payload. Safe intervention may instead require controlled deceleration, support retention, a designed braking mechanism, or an energy-independent mechanical fallback. The appropriate stopping function and residual hazards must be specified for the application, rather than inferred from the existence of an emergency-stop device (International Organization for Standardization, 2015; Bi et al., 2021). Loss-of-power behavior must be tested before deployment. Fault analysis should therefore record the initiating event, propagation path, physical consequence, and intended protective barrier. System-theoretic accident analysis complements component-oriented methods by identifying hazardous interactions that need not begin with a defective part (Leveson, 2004). The resulting failure pathways provide

the basis for the risk-driven safety assessment that follows. Each path must be associated with an observable precursor, a protective requirement, and a physically meaningful verification outcome.

11.4.2 Risk-Driven System Safety Assessment

From Failure Pathways to Safety Evidence Task-success rates, perception accuracy, and tracking errors quantify useful capabilities but do not establish what happens after corrupted observations, missed deadlines, or degraded actuators. A defensible safety claim concerns the complete physical agent, its authorized tasks, the exposure of people and equipment, and the conditions under which protective functions remain effective. It should distinguish an action proposed by the model, the command actually admitted and applied, and the resulting physical state. Surveys of safety-critical robotics and safeguarding of autonomous systems identify the need to state assumptions, adopt suitable safety mechanisms, and examine evidence across heterogeneous application domains (Guiochet et al., 2017; Müller et al., 2022). Research on testing standards for integrated robots likewise identifies limitations of component-centered performance tests when systems combine multiple functional modules (Bi et al., 2020). Assessment should consequently be derived from the fault pathways described above, with explicit acceptance criteria for both component degradation and end-to-end physical outcomes.

Observable Indicators Across System Layers Perception cascades motivate tests of image saturation, dropped measurements, timestamp consistency, estimator residuals, and behavior when observations are unreliable. Robust-perception benchmarks explicitly test lighting, sensor, and processing corruptions, although they do not by themselves establish the safety of the complete robot (Kong et al., 2023). Fault-tolerant robotics research also addresses failures in sensor information and state estimation (Milecki and Nowak, 2023). Timing cascades require tail latency, deadline-miss rate, control jitter, and command freshness. Real-time control research relates timing to closed-loop performance, while ROS 2 guidance identifies blocking, memory allocation, and priority inversion as sources of execution-time nondeterminism (Cervin et al., 2003; Kay, 2016). Processor and supply degradation motivate thermal, clock-frequency, voltage, and interruption tests. Hardware thermal-management documentation establishes why computation may degrade under overheating (NVIDIA Corporation, n.d.). Communication assessment should measure latency, loss, and timeout behavior, drawing on networked-control fault-detection research (Baglietto et al., 2020). Actuator assessment must examine current, temperature, joint and torque limits, stall detection, and the resulting physical response. Existing driver protections and robotic fault-tolerance studies provide corresponding engineering foundations (Texas Instruments, 2019; Milecki and Nowak, 2023). Table 11.5 synthesizes these candidate measurements and associated protective responses. Their thresholds and applicability should follow the robot's risk assessment. Even published safety-related test guidance for personal care robots does not treat every test as universal (International Organization for Standardization, 2020; Müller et al., 2022).

End-to-End Protection Verification Measurements become safety evidence only when they are tied to the physical response of the integrated system. Fault injection should introduce exposure faults, delayed commands, packet loss, thermal or computational degradation, actuator restrictions, and loss of power under representative tasks. Systematic reviews of robotic testing, validation, and verification, together with research on integrated-robot testing, provide a foundation for selecting complementary test methods and reporting their coverage (Araujo et al., 2023; Bi et al., 2020). Evaluation should measure detection and intervention latency, the ability to isolate a failing component, and whether the robot can enter a state whose residual risk is acceptable. Work on interpretable runtime monitoring demonstrates how predicted unsafe states can trigger corrective replanning, while safety-critical control research develops explicit constraints on admissible system behavior (Di Franco and Bezzo, 2020; Ames et al., 2017). Safety-related control design supplies further principles for protective functions whose effectiveness must be validated independently of the nominal policy (International Organization for Standardization, 2023b; Bi et al., 2021). The goal is a traceable relationship between an identified hazard, a tested safety requirement, and the observed outcome, not a context-free aggregate safety score.

Table 11.5 Representative failure modes, safety consequences, and evaluation dimensions across robotic system layers. Indicators are illustrative and platform-dependent

System layer	Representative failure modes	Potential safety consequences	Monitoring / evaluation dimensions
Models and planning	Unsafe decisions Infeasible joint targets	Hazardous motions Unintended contact	Constraint violations Safety-filter interventions
Real-time software	Missed deadlines Scheduling blocks, stale commands	Delayed corrective action Loss of stability	Deadline-miss rate Control jitter, command age
Computing hardware	Thermal throttling Processor faults	Increased control latency Interrupted execution	Chip temperature Clock rate, execution latency
Sensors	Image saturation IMU drift, frame loss	Missed obstacles Incorrect measurements	Saturated-pixel ratio Bias, frame-drop rate
Estimation and control	Estimator divergence Feedback delay, oscillation	Unstable motion Excessive contact	Estimation residuals Feedback age, oscillation amplitude
Actuators and transmissions	Motor stall, overcurrent Overheating, transmission wear	Joint overload or damage Loss of support	Motor current, temperature Torque-limit events, tracking error
Communication	Packet loss, bus timeout Timestamp mismatch	Stale feedback Uncoordinated execution	Packet-loss rate, bus errors Link latency, timestamp skew
Power and battery	Undervoltage Power interruption, aging	Unexpected shutdown Fall or dropped load	Bus voltage, battery health Power-loss test outcome

11.4.3 From Pre-Deployment Verification to Post-Deployment Safeguards

Safety-by-Design Before Delivery Risk governance starts before a product is placed into use. Developers should identify foreseeable operating environments, human proximity, loads, task permissions, misuse, and failure combinations, then translate them into safety requirements and independent protection functions. Recent robotics safety reviews emphasize risk assessment across both design and deployment (Adesiji et al., 2026). Simulation, hardware-in-the-loop testing, cross-module fault injection, environmental stress testing, and controlled physical trials should verify both nominal performance and behavior under degraded conditions, using complementary techniques whose limitations have been surveyed in the robotic testing literature (Araujo et al., 2023). The corresponding evidence must record hardware and software versions, model or policy configuration, operating limits, residual risk, and what was not tested. ISO 12100 and ISO 13849-1 provide established foundations for machinery risk assessment and safety-related control design. ISO 13482 and ISO 10218-1 illustrate additional requirements for particular robot applications, not universal certification rules for every physical agent (International Organization for Standardization, 2010; 2023b; 2014; 2025).

Release as an Evidence-Based Transition Acceptance testing should validate not only whether a robot can complete its assigned tasks but whether its safety monitors, limits, fallback controllers, brakes, and any emergency energy provisions work together when a fault occurs. The need to test the integrated configuration and select risk-dependent test methods is reflected in integrated-robot testing research and application-specific ISO guidance (Bi et al., 2020; International Organization for Standardization, 2020). A release record should link the tested product configuration to its operating envelope, protective functions, acceptance criteria, and outstanding hazards. Wear, new payloads, changed environments, repaired components, and updated policies may invalidate the assumptions documented in the release record. Post-deployment safeguards must therefore maintain and revise the evidence created before delivery rather than replace it with an unrelated monitoring process (Guiochet et al., 2017; Müller et al., 2022).

Runtime Detection and Protective Action After delivery to the user or operator, the robot should monitor whether observations remain trustworthy, commands are fresh, and control deadlines are met. It should also monitor whether actuator, computing, communication, and power subsystems retain adequate capacity. Where justified by the hazard analysis, an independent safety supervisor should restrict commands or transfer control when the nominal policy is unavailable or wrong. Runtime monitoring and corrective replanning have been demonstrated for autonomous robotic operation, while control-barrier methods illustrate how physical safety constraints can be imposed on control inputs when their modeling assumptions hold (Di Franco and Bezzo, 2020; Ames et al., 2017). Predictive checking and measured feedback should support graduated responses, including limiting speed or workspace, disabling contact-rich actions, reducing payload, requesting confirmation, or moving into a validated protective mode. In an emergency, local protection must act within the required response time rather than wait for a user decision. The chosen safe state must respect the actual mechanics. A dynamically balancing robot or one supporting a heavy load may need controlled motion or braking before torque is removed. Protection must also recognize when insufficient power or sensing makes autonomous recovery impossible.

User-Facing Warnings and Operational Responsibility A deployed safety system must communicate consequential faults, not merely store engineering logs. Research on human-robot collaboration identifies operator awareness and effective visual, acoustic, and other notification channels as important interface design concerns (Grushko et al., 2021; Bi et al., 2021). Operators or household users should receive an intelligible account of the affected capability, the protective action already taken, and the appropriate next step. For example, the account should explain that a joint has overheated, forceful manipulation has been disabled, and servicing is required before that function resumes. Notifications should distinguish informational maintenance advice from an urgent restriction on continued use, without exposing users to raw diagnostics they cannot interpret. In higher-risk settings, a warning may need to be paired with physical isolation, a lockout, or escalation to an authorized operator. Such interaction supports accountable operation but must not substitute for automatic, timely safety intervention.

Validated Updates and Physical Remediation Some defects can be corrected by over-the-air (OTA) software or firmware updates, but changing perception, scheduling, control logic, or model capabilities may invalidate safety assumptions. Future update processes should verify provenance and integrity, assess hardware-software compatibility, test affected safety functions, stage deployment, and provide recovery or rollback where feasible. The software-update-management principles of UN Regulation No. 156 offer an automotive precedent, not a law automatically applicable to robots (United Nations Economic Commission for Europe, 2021). Other defects require inspection or replacement. Motors, gearboxes, brakes, sensors, batteries, and power electronics can wear or fail in ways software cannot correct. When a common defect affects a product batch, the manufacturer may need to identify affected configurations, contact users, restrict risky operation, provide repairs, or initiate a recall. Automotive recall practice offers a relevant governance analog.[10] OTA maintenance is not a substitute for on-device protection during an unfolding hazard.

Continuous Improvement Across the Product Life Cycle Incident and near-miss reports, safety-filter interventions, maintenance histories, and configuration records should connect field observations to the original hazard analysis and qualification tests. Comparing evidence across deployed robots may reveal emerging combinations of temperature, wear, environmental conditions, and task behavior that were not covered before release. Data collection should remain proportional to the safety purpose, particularly in private environments. Updated risks must result in reviewed corrective decisions, re-validation of affected configurations, and, where necessary, revised operating permissions or withdrawal from service. Linking risk analysis to subsequent tests is consistent with existing safety-assessment and robotic verification research (Adesiji et al., 2026; Araujo et al., 2023). The objective is not to guarantee the absence of all faults, but to maintain a credible and revis-

[10] NHTSA, *Check for Recalls: Vehicle, Car Seat, Tire, Equipment*.

able basis for safe operation as physical agents and the environments in which they work continue to evolve.

11.5 Law, Ethics, Standards, and Global Governance

The preceding sections have examined how to evaluate the behavior and competence of physical agents, constrain their actions, and maintain safety evidence throughout operation. These technical foundations inform deployment decisions, but they do not by themselves determine who is responsible, which uses of autonomy are acceptable, or what oversight is required. Because Physical AI connects perception and decision-making to actions in the physical world, governance must address the deployed system rather than the model in isolation. Law, ethics, standards, and international coordination connect technical evidence to obligations and acceptable uses, extending the chapter's focus from measurable behavior and safety assurance to accountable deployment.

This section first addresses responsibility allocation and traceability, then examines human control, privacy, fairness, and ethical boundaries. It next considers legal and regulatory frameworks from technical, ethical, and application perspectives, connecting these issues to safety standards and the book's illustrative risk-tiered governance framework. Finally, it explores global cooperation through compatible evaluation evidence, incident records, and investigation procedures.

11.5.1 Responsibility Allocation and Traceability

Learning-based systems may behave in ways that designers and operators cannot fully anticipate, creating difficulties associated with the *responsibility gap* (Matthias, 2004). Responsibility consequently extends beyond the person exercising immediate control. Model developers should document capabilities, limitations, and versions. Hardware manufacturers should address mechanical and electrical hazards. Integrators should validate interfaces and operating limits. Operators should maintain safeguards and intervention procedures. Maintenance providers must track safety-relevant updates and configuration changes. These engineering duties support accountability but do not themselves determine legal liability.

This allocation depends on what each party supplies or changes. For example, replacing a model, increasing permitted speed, or changing an end-effector can alter different parts of a safety argument. Documentation should identify who approved the change, which assumptions were reconsidered, and what evidence supports continued operation.

Product-liability law provides a complementary perspective. Directive (EU) 2024/2853 includes software and addresses safety-relevant updates and upgrades. Its rules apply to products placed on the market or put into service after December 9, 2026 (European Parliament and Council of the European Union, 2024b). The significance for Physical AI is that assessment cannot stop at initial delivery. Subsequent changes may alter the behavior of the deployed system.

Accountability also depends on reconstructing what occurred. A safety-relevant event record should connect system and model identity with perception states, decisions, executed actions, safety settings, and human interventions, together with update, maintenance, and cybersecurity histories. Recording should serve investigation and safety improvement rather than unrestricted surveillance. Data minimization, access controls, encryption, and proportionate retention are therefore part of traceability design, not exceptions to it.

11.5.2 Human Control, Privacy, Fairness, and Ethical Boundaries

Meaningful human control requires more than the presence of a supervisor. A person must understand relevant capabilities and limitations, receive usable information, possess intervention authority, and have sufficient time to act. An override that cannot bring the system into a safe state within the available response time offers little practical control. Human oversight and accountability are also central to the UNESCO and OECD AI ethics frameworks (UNESCO, 2021; Organisation for Economic Co-operation and Development, 2019).

The appropriate form of oversight depends on the task. A supervisor may authorize a sequence in advance, monitor execution, or intervene when an exception occurs. These roles require different information and response times. Where physical dynamics are faster than human reaction, protective functions must constrain the system without waiting for an operator to diagnose the problem.

Physical sensing extends privacy concerns beyond direct users. A domestic robot may observe visitors or neighbors who have neither purchased nor authorized it. *Privacy by Embodiment*, used here as a design principle, connects privacy protection to sensing and actuation. Under this principle, systems should process data locally where feasible, minimize raw-data retention, protect sensitive information, and communicate active sensing and remote access. User authorization does not by itself resolve bystander exposure.

Fairness likewise concerns physical outcomes, not only balanced prediction accuracy. Differences in mobility, body characteristics, or speech may change whether users can interact safely or obtain timely assistance. Evaluation should therefore examine accessibility and failure consequences across intended users, particularly children, older people, and people requiring care. Greater autonomy does not remove the need for human judgment where actions threaten safety, dignity, or essential services. Physical AI may also displace workers, while newly created jobs may not benefit those affected. Fair deployment should therefore consider worker participation, retraining opportunities, and the equitable distribution of productivity gains (UNESCO, 2021; Organisation for Economic Co-operation and Development, 2019).

11.5.3 Legal and Regulatory Frameworks for Physical AI

Technique Technical regulation connects responsibility allocation, change management, and traceability to legal accountability. The EU AI Act's high-risk framework addresses risk management, technical documentation, event logging, and cybersecurity, subject to its scope and application dates (European Parliament and Council of the European Union, 2024a; 2026). Directive (EU) 2024/2853 complements this framework by covering software and addressing safety-relevant updates in product liability (European Parliament and Council of the European Union, 2024b). As discussed in Section 11.5.1, model replacements or changes in speed and tooling should prompt reassessment, with records identifying who approved each change and what evidence supports continued operation. Event records should connect deployed versions, actions, interventions, and maintenance histories. These engineering duties support accountability but do not themselves establish legal liability.

Ethics Meaningful human control, Privacy by Embodiment, and fairness require safeguards for people affected by deployment. The EU AI Act's high-risk framework addresses effective oversight, while practical intervention must remain feasible within the available response time (European Parliament and Council of the European Union, 2024a; 2026). The GDPR addresses lawful processing, data minimization, and protection by design. China's Personal Information Protection Law addresses sensitive information and fairness in automated decision-making (European Parliament and Council of the European Union, 2016; Standing Committee of the National People's Congress, 2021). Article 5 of the AI Act prohibits specified manipulative practices and exploitation of vulnerabilities where its statutory conditions are met (European Parliament and Council of the European Union, 2024a; 2026). Legal compliance does not exhaust the ethical responsibilities identified in Section 11.5.2, including equitable access to safe assistance and support for workers affected by automation (UNESCO, 2021; Organisation for Economic Co-operation and Development, 2019).

Application Application-level regulation depends on intended use, system boundaries, and jurisdiction. Industrial robot installations fall under applicable machinery rules, while qualifying medical robots are subject to medical-device regulation (European Parliament and Council of the European Union, 2006; 2023; 2017). The EU AI Act separately classifies high-risk systems through Article 6 and Annex III, subject to their statutory conditions (European Parliament and Council of the European Union, 2024a; 2026). Table 11.6 presents the book's illustrative five-tier frame-

work, considering harm severity, human exposure, tooling, autonomy, safeguard reliability, and user vulnerability (International Organization for Standardization, 2010; Tabassi, 2023). These tiers are neither legal categories nor an international standard and do not map directly onto the AI Act. Classification must record operating assumptions and be reassessed when conditions change. A guarded robot's Low placement reflects controlled exposure, not harmless machinery. Lower tiers do not remove legal duties, and higher tiers cannot authorize prohibited uses.

Table 11.6 Illustrative risk tiers proposed in this monograph. Placements depend on task, exposure, and validated safeguards

Tier	Illustrative conditions and harm	Governance focus
Minimal	Supervised, low-energy desktop sorting without sharp, hot, or heavy tooling. Contact harm is at most trivial and is reversible.	Basic product safety, use limits, instructions, and controlled updates.
Low	Industrial operation behind validated guarding and interlocks, without routine human access during automatic operation. Exposure is mainly during setup or maintenance, but contact remains potentially severe.	Machinery assessment, protective-function validation, access control, and safe maintenance with lockout procedures.
Moderate	Speed-limited goods transport in worker-accessible areas, repeated encounters with reversible injury or localized damage foreseeable.	Route and speed limits, detection and stopping tests, monitoring, and incident analysis.
High	Close-contact collaboration, hazardous household manipulation, or elder-care transfer assistance. Human exposure is frequent, and crushing, burns, cuts, falls, or incorrect assistance are foreseeable.	Independent evaluation, meaningful oversight, runtime safeguards, incident reporting, and privacy protection.
Very high	Traffic-speed autonomous driving, invasive surgical motion, or critical-infrastructure control. Death, multiple casualties, or cascading disruption are possible.	Rigorous assurance, independent review, continued supervision, appropriate fail-safe or fail-operational design, and sector-specific compliance.

11.5.4 Global Cooperation and Governance

Models, components, integrators, and operators may span jurisdictions, while responsibility for an incident must remain traceable across their interfaces. Building on the obligations and safeguards discussed in Section 11.5.3, international cooperation should seek compatible safety evidence and investigation procedures rather than identical legal systems. The practical objective is to preserve the responsibility chain described in Section 11.5.1 when development, integration, deployment, and maintenance take place in different countries.

The UNESCO and OECD frameworks provide shared ethical principles, while the Council of Europe AI Framework Convention connects AI governance with human rights, democracy, and the rule of law (UNESCO, 2021; Organisation for Economic Co-operation and Development, 2019; Council of Europe, 2024). The 2023 Bletchley and 2024 Seoul declarations emphasize safety cooperation alongside broader development goals (Countries Attending the AI Safety Summit, 2023; Participants Attending the Leaders' Session of the AI Seoul Summit, 2024). China's 2024 Shanghai Declaration and 2025 Global AI Governance Action Plan likewise advocate safe, inclusive development and international cooperation (World AI Conference and High-Level Meeting on Global AI Governance, 2024; World Artificial Intelligence Conference and High-Level Meeting on Global AI Governance, 2025). These declarations and policy initiatives are not binding international safety

regulations. Their value for Physical AI lies in supporting cooperation on the shared concerns of human control, privacy, fairness, and accountability identified in Section 11.5.2.

Practical cooperation would include common incident categories, interoperable event records, reliable model and system identification, compatible evaluation methods, cross-border safety notifications, and coordinated corrective updates. Evaluation evidence should preserve the operating assumptions, system configuration, and safeguards under which it was obtained, so that investigators can determine whether it remains relevant in another deployment. Shared investigation procedures should make it possible to connect a reported event with the organizations responsible for the affected model, hardware, integration, and subsequent modifications. These mechanisms would support investigation and improvement across borders without presuming agreement on every regulatory choice or automatic recognition of another jurisdiction's approval.

Interoperability does not require publishing every operational record. Shared formats can describe the system, event, and applicable safeguards while access to personal data, security-sensitive information, and other protected evidence remains controlled. Consistent with Sections 11.5.1 and 11.5.2, cooperation should preserve both investigative usefulness and proportionate information protection. The practical test of global governance is whether an incident can be investigated across organizational and national boundaries without losing the chain of responsibility or exposing sensitive information unnecessarily.

12 Strategy and Civilizational Outlook

The preceding chapters have examined the concepts, technical foundations, models, and systems of Physical AI. This final chapter considers how these advances might develop into capabilities that shape the longer-term course of human civilization. The central question is how intelligence grounded in physical interaction can expand humanity's ability to discover, produce, build, and sustain the systems on which it depends.

The chapter first identifies development priorities for the coming decade and beyond, including the convergence of digital and physical foundation models, self-evolving agents, hardware-software co-evolution, an Internet of Physical Agents, and the scientific and technological ecosystem supporting their development. It then examines how local achievements can become shared physical services, allowing verified capabilities to accumulate across teams, locations, and generations. From this foundation, the discussion extends to planetary energy and infrastructure associated with a Type I civilization, and to the autonomous construction, resource use, and long-duration operation that proposed stellar-scale development requires.

These possibilities depend on progress in both intelligence and its physical foundations. Energy, materials, equipment, and maintenance remain essential constraints. The civilizational significance of Physical AI therefore rests on its ability to turn knowledge into reliable action, preserve useful capabilities over time, and support further discovery under increasingly demanding conditions.

12.1 Development Strategies for Physical AI

The development of Physical AI requires advances in models, learning systems, hardware, and the infrastructure through which capabilities are shared. Over the next five to ten years and beyond, these advances should support a common objective of systems that can undertake increasingly varied physical tasks, learn from their operation, and transfer useful capabilities with less additional engineering. Figure 12.1 summarizes five priorities that follow from this objective, including the convergence of digital and physical foundation models, self-evolving agents, hardware-software co-evolution, an Internet of Physical Agents, and a scientific and technological ecosystem that allows progress to accumulate.

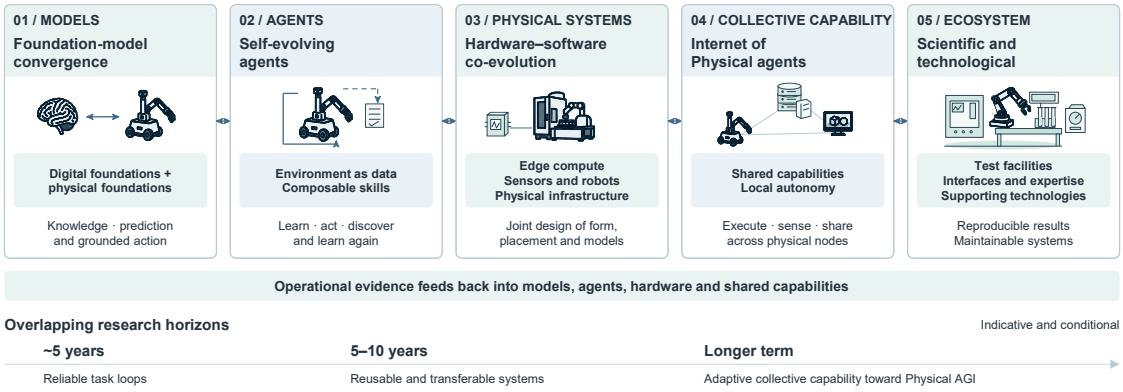


Fig. 12.1 Five development priorities for Physical AI, including foundation-model convergence, self-evolving agents, hardware-software co-evolution, the Internet of Physical Agents, and a supporting scientific and technological ecosystem. Operational evidence informs all five priorities. The research horizons are indicative and conditional, rather than fixed delivery commitments

12.1.1 *Converging Digital and Physical Foundation Models*

Digital foundation models and physical foundation models should develop through increasingly close integration. Digital foundation models learn broadly reusable representations and capabilities from language, code, images, and other digitally represented information. Physical foundation models seek reusable capabilities grounded in physical states, dynamics, interventions, and interaction.

The strategic opportunity lies in connecting the breadth of digital knowledge and reasoning with the grounding required for physical action. A system working in the real world must interpret objectives, draw on prior knowledge, recognize physical constraints, anticipate the consequences of its actions, and revise its decisions after observation. Digital and physical capabilities need to inform one another throughout this process. Physical evidence can correct an assumption made during reasoning, while semantic knowledge can help identify which variables to measure or which intervention to attempt.

The integration of large multimodal models and world models is one important example. Multimodal models can connect observations to concepts and task descriptions, while world models can predict how a physical state may change under an intervention. Other combinations include code-generating models coupled with executable control interfaces, reasoning models linked to scientific simulators, and general-purpose planners working with embodied action models. These examples suggest a broader research direction involving foundation-model systems that connect knowledge, prediction, and action across digital and physical domains.

Integration can proceed through explicit interfaces before requiring a unified architecture. Models can exchange task objectives, state estimates, candidate actions, predicted outcomes, and uncertainty. Shared representations and joint training may subsequently reduce inconsistencies and improve transfer. Analytical models, numerical solvers, and simulators should remain available where they provide useful structure or more reliable estimates. The choice between a single model and a coordinated set of models should depend on performance, adaptability, latency, and cost.

The longer-term goal is a foundation-model system that participates actively in acquiring knowledge. It should recognize when available evidence is insufficient, select a useful observation or intervention, and incorporate the result into subsequent decisions. Progress should be assessed through the quality of those decisions, the physical trials required to reach them, and the ability to reuse knowledge under new conditions.

12.1.2 *Self-Evolving Agents for Open-Ended Tasks*

Physical AI should support agents that extend their capabilities through experience. Here, *self-evolution* refers to a controlled process of identifying capability gaps, acquiring evidence, and improving memory, skills, models, or policies. Objectives and operating constraints remain explicit, and changes to deployed behavior require evaluation.

This approach treats the *environment as data*. The operating environment is a continuing source of learning opportunities, rather than merely a destination for a pretrained model. An agent's actions influence which observations become available. A failed grasp may reveal an incorrect estimate of friction. An unexpected measurement may expose a missing variable. A successful recovery may suggest a reusable procedure. Such experience becomes useful training evidence only when it is recorded with sufficient context, checked for quality, and interpreted with appropriate attention to uncertainty and alternative explanations.

The resulting cycle (Figure 12.2) is *learning-action-discovery-relearning*. Existing knowledge guides action. Action exposes the agent to physical outcomes. Discrepancies and new observations reveal gaps or regularities. Subsequent learning revises the relevant capability. Discovery in this cycle may be modest, such as a changed equipment response, a previously unrecognized task constraint, or a more effective action sequence. The value of the cycle depends on whether these findings improve later performance.

Adaptation during execution should be distinguished from persistent learning. Updating a state estimate, revising a plan, or retrieving another skill may be sufficient to complete the current task. Changes intended to improve future tasks require retained evidence, failure analysis, candidate updates, and evaluation against established competencies. Physical exploration must remain within defined safety and resource limits, and updates should be introduced in stages with a means of rollback.

Composable skills make this process more economical. Capabilities for movement, manipulation, measurement, inspection, and recovery should describe their preconditions, expected effects, and failure modes. Agents can then assemble workflows from existing components and learn the missing parts. Composition still requires testing, since the timing and physical effects of one skill can invalidate the assumptions of another.

A useful measure of progress is the cost of acquiring the next capability. Learning across tasks, environments, and embodiments should reduce the additional demonstrations, trials, engineering changes, and human assistance needed in a new setting. Comparisons with frozen systems should account for these costs, as well as retained performance and the reliability of long-horizon execution.

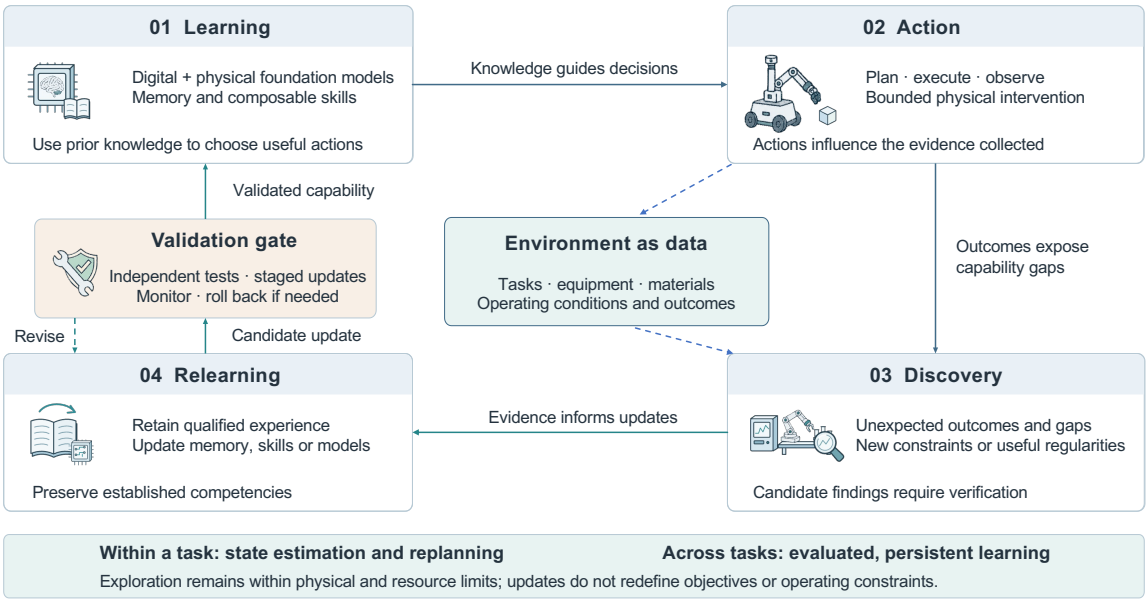


Fig. 12.2 The learning-action-discovery-relearning cycle. Physical interaction generates observations that reveal capability gaps and candidate discoveries. Qualified experience informs updates to memory, skills, or models, subject to evaluation and controlled deployment. Adaptation within a task is distinguished from persistent learning across tasks

12.1.3 Hardware-Software Co-Evolution

Hardware-software co-evolution in Physical AI spans computing resources, physical terminals, and the infrastructure in which they operate. Hardware includes edge processors and communication devices, sensors and actuators, robots and scientific instruments, and installed systems such as cameras and traffic signals. Their design determines what algorithms can observe, how quickly they can respond, and which actions are physically possible.

Edge computing should be developed alongside model architectures and execution requirements. Memory capacity, energy consumption, thermal limits, and latency constrain the models that can run near a device. Model compression, task allocation, and the division of computation between

local and remote resources should reflect these constraints. Time-critical control and essential protective functions need dependable local execution when communication is delayed or unavailable, while less urgent reasoning and learning may use shared computing resources.

For robots and other terminals, morphology and function should be considered together with models and control algorithms. Joint arrangement, actuator characteristics, mechanical compliance, sensor placement, and end-effector design affect both task performance and learning difficulty. A hardware change may simplify a control problem or make a previously hidden state observable. Conversely, improvements in perception or control may allow a simpler mechanical design. Co-design should evaluate these trade-offs against task coverage, reliability, energy use, and maintenance requirements.

The same reasoning extends to physical infrastructure. Camera placement can be optimized jointly with perception and state-estimation methods, taking account of coverage, occlusion, calibration, and privacy constraints. The layout and configuration of traffic signals can be studied together with sensing, prediction, and coordination algorithms, subject to engineering and safety requirements. In such settings, the environment's instrumentation and configuration become part of system design. Software performance should be assessed against the infrastructure it requires, including installation and operating costs.

Over approximately five years, priorities should include reliable edge deployment, instrumented platforms, and reproducible integration and testing. Over five to ten years, modular terminals, reusable interfaces, and coordinated infrastructure design could reduce the cost of supporting new tasks. More distant research may investigate reconfigurable bodies, adaptive sensing systems, and brain-computer interfaces that connect human neural signals with computational and physical systems. The latter remains an exploratory direction, with substantial questions concerning signal stability, biocompatibility, privacy, and safe operation. It should not be treated as a prerequisite for near-term progress.

Across these horizons, access to physical testing remains essential. Computing resources need complementary support from equipment, calibration, maintenance, consumables, and technical expertise. Co-evolution is useful when changes in hardware and software jointly improve sustained performance or reduce total resource requirements.

12.1.4 The Internet of Physical Agents: Architecture and Sustained Operation

The *Internet of Physical Agents* is a proposed federated infrastructure through which agents, models, data, computing resources, physical equipment, and human participants cooperate. Its architecture (Figure 12.3) should support the circulation of observations, knowledge, task requests, actions, and verified outcomes among heterogeneous participants. A node may consume one capability while providing another, and its role can change from one task to the next.

Three interacting components provide a useful architectural starting point. Physical nodes combine local perception, computation, control, and equipment interfaces. Shared capability services provide models, skills, datasets, simulation, computing, and evaluation. Coordination services support discovery, resource matching, task decomposition, scheduling, authorization, and evidence exchange. These are logical functions that may be distributed or replicated across sites, rather than fixed levels in a centralized hierarchy.

Information and capability should move in multiple directions. A robot may receive a task and invoke a shared model, while also collecting measurements, identifying an unexpected condition, and supplying evidence that helps another agent. An instrument can execute an experimental procedure and return observations that change the procedure itself. One node's findings may update a shared resource, inform a peer directly, or trigger a request for further measurement elsewhere. The network therefore links execution with sensing, interpretation, and learning through several interacting loops.

Local autonomy remains necessary within this distributed architecture. Physical nodes retain time-critical control, equipment protection, and bounded execution. Network services coordinate activ-

ities whose timing and dependencies permit wider cooperation. Capability interfaces should specify operating conditions, uncertainty, latency, resource requirements, and failure handling. Provenance and version information are needed so that recipients can determine whether an observation, model, or procedure is suitable for their own setting.

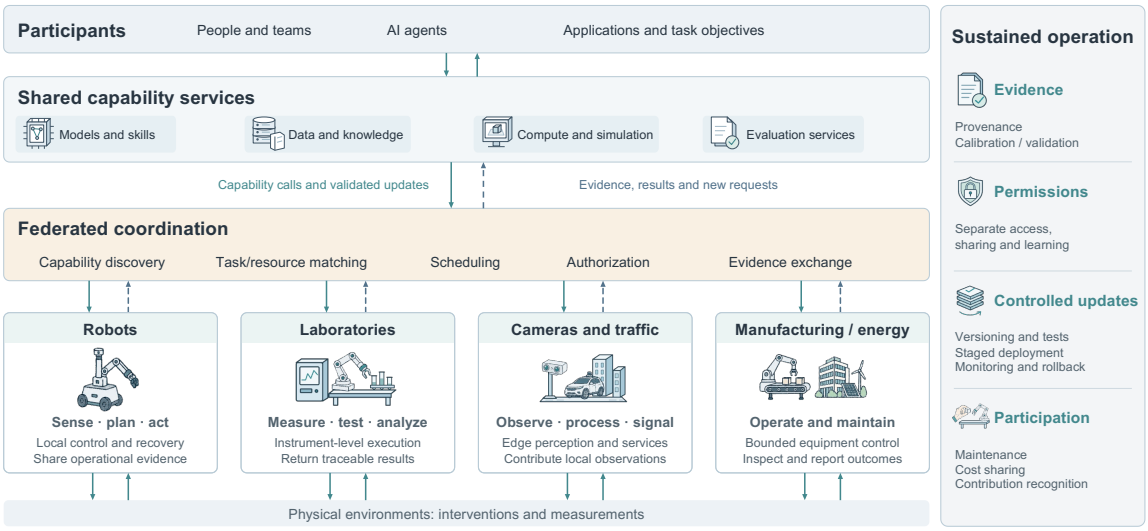


Fig. 12.3 Conceptual architecture of the Internet of Physical Agents. Shared capability services, federated coordination, and locally autonomous physical nodes support multidirectional exchange of requests, observations, results, and validated updates. Nodes also cooperate with peers and interact with their local environments. Logical services may be distributed across sites, while time-critical control remains local

The *Scientific Internet* is one proposed implementation case. It would connect scientists, scientific literature, datasets, models, laboratories, instruments, and computing resources across disciplines and locations to support interdisciplinary autonomous scientific discovery. A workflow could move from literature analysis to hypothesis generation, simulation, physical experimentation, and revised investigation. Experimental results would feed back into hypotheses and models, while unresolved questions could redirect work to other facilities. Scientists would define objectives and assess significance, and scientific claims would remain subject to independent verification.

Sustained operation requires clear permissions and controlled updates. Equipment access, result sharing, and permission to learn from operational data should be specified separately. Changes to shared models or services need compatibility tests, staged deployment, monitoring, and rollback, since an update can affect dependent nodes. Participation also depends on practical arrangements for maintenance, cost sharing, and contribution recognition. Expansion should follow evidence that capabilities can be reused by other teams, coordination produces a net benefit, and service quality persists beyond the support of the original developers.

12.1.5 Building the Scientific and Technological Ecosystem for Physical AI

The reach of Physical AI depends on the maturity of the technologies and practices around it. Models require suitable instruments, reliable equipment, manufacturing capacity, energy supplies, communication infrastructure, and people who can integrate and maintain them. Development strategies should identify these complementary requirements and the bottlenecks that limit useful deployment. The limiting factor may be a model in one setting and measurement quality, equipment reliability, or integration effort in another.

A productive ecosystem makes results easier to reproduce and capabilities easier to extend. Shared test facilities, documented datasets, reference implementations, and interoperable interfaces can

reduce repeated engineering work. Evaluation methods should support meaningful comparison across systems while leaving room for different architectures. Training and collaboration should connect AI research with control, equipment engineering, domain expertise, and operations. These connections are particularly important when moving from a successful prototype to a system that another team can use and maintain.

Scientific research provides one example of how Physical AI could strengthen this broader technological base. Better instruments and experimental methods can produce more informative evidence for learning. Models and agents can, in turn, help design experiments, investigate materials, improve fabrication processes, and diagnose equipment failures. Improvements can thus propagate between research tools, scientific knowledge, and engineering capability. Their value depends on reproducible findings and usable technologies, including cases where experiments reject a hypothesis or reveal a method's limits.

Similar feedback can occur in manufacturing, logistics, maintenance, and energy operations. Deployment reveals drift, wear, resource constraints, and interactions that are difficult to reproduce in isolated tests. When this experience is documented and returned to development, it can improve both the intelligent system and the processes that support it. Longer-term investment should preserve the ability to investigate unresolved problems while also developing the interfaces, equipment, and operating practices needed for reliable reuse.

Technological maturity should therefore be assessed beyond model-level benchmarks. Relevant measures include the cost of adaptation, intervention frequency, operating lifetime, recovery time, and useful output relative to total resource consumption. Reproducibility across sites and continued operation after the original developers leave are important tests of whether a capability has become broadly usable.

At a larger scale, Physical AI could extend the ability to investigate, construct, operate, and maintain physical systems. The significance of that extension depends on what can be accomplished, across how much space, for how long, and with what expenditure of energy, materials, and human effort. A capability acquires enduring value when it delivers useful work reliably, survives changing conditions, and can be recovered and passed on to others.

12.2 Physical AI and Human Civilization

Physical AI connects computation, perception, and action. Questions about civilizational scale therefore depend on energy, materials, and infrastructure. The level a civilization can reach depends on how much energy it can obtain and organize, how effectively it can turn that energy into stable production and services, and whether those capabilities can continue to operate, be repaired, and be inherited across locations and generations. Larger models and more computation become civilizational capabilities only when they enter equipment, energy systems, and real environments and are tested through physical feedback.

Physical AI as a Service (PAIaaS) is used here to describe how such capabilities can be organized. Model as a service separates model inference from individual applications and lowers the cost of using digital intelligence. PAIaaS extends this idea to experiments, manufacturing, inspection, energy operations, and maintenance. Users can call on real equipment and operating procedures without rebuilding the full facility at every site. Physical capabilities can then be shared across locations and combined into more complex tasks.

The development of Physical AI is therefore connected to the growth of civilization. Reaching Type I requires organizing energy, production, and infrastructure across Earth. Reaching Type II requires infrastructure beyond Earth that can obtain resources, manufacture components, and operate for long periods. The discussion below first considers how physical capability can be measured and then examines the role of physical AI in progress toward Type I and Type II civilizations.

12.2.1 *Civilizational Scale and the Physical Basis of Intelligence*

Civilizational scale requires energy measures to be interpreted together with practical capability. The Kardashev index provides a power scale, but power alone says how much energy a civilization continuously directs, not what that energy enables. A physical measure must also account for the area a system can serve, the duration for which it can operate, and its ability to recover from failures and changing conditions.

Civilization as a Physical Measure In the continuous convention, the Kardashev index is written as

$$K_{\text{civ}} = \frac{\log_{10}(P_{\text{power}}/\text{W}) - 6}{10}, \tag{12.1}$$

where P_{power} is the power continuously used by a civilization. Under this convention, Type I, Type II, and Type III correspond to approximately 10^{16}, 10^{26}, and 10^{36} W, respectively. Using 2019 global total energy supply, Jiang et al. (2022) estimate humanity's index at about $K_{\text{civ}} = 0.73$. Using this rounded index in the continuous convention gives a Type I power level about $10^{2.7} \approx 501$ times the baseline (Kardashev, 1964; Sagan, 1973). A small change in the index can therefore correspond to a large change in the energy base.

Power is an entry point for measuring civilizational capability. The same power can support different levels of production, transport, computation, and public service, or it can be consumed by inefficient conversion and maintenance. A fuller account should combine power with duration, spatial coverage, reliability, and recovery. A remote facility that operates for several hours has a different physical capability from one that can work for years without a local human team.

Energy Use and Energy Acquisition Overall energy efficiency is determined by two efficiencies working together. Energy-use efficiency describes how much of the energy already obtained becomes useful service. Energy-acquisition efficiency describes the equipment, materials, land, and maintenance required to bring new energy into the usable energy system. Higher use efficiency allows the same energy supply to support more service. Higher acquisition efficiency allows more resources to become available. A limitation at either stage constrains the expansion of a civilization's energy base.

Both efficiencies are physical. Use efficiency depends on equipment, materials, conversion processes, transmission networks, and operating conditions. Acquisition efficiency depends on resource location, environmental conditions, tool lifetime, operational safety, and resupply. Information processing alone cannot establish either one. Both have to be tested through equipment, field operations, and long-term maintenance. Energy systems should therefore be evaluated by the useful service delivered over a defined period, together with the full requirements for equipment, computation, materials, and personnel.

Civilization develops in two directions at once. It must make better use of energy already available and continue to obtain or create new energy sources. Digital systems rest on the same physical base. Computation requires electricity, chips, materials, cooling, and equipment maintenance. The expansion of digital capability is therefore supported and limited by physical conditions.

From Digital Information to Physical Action Digital systems process information, while civilizational capability is realized through physical processes. A model can propose a design, predict demand, or generate a control policy. Such results become testable changes only when they enter equipment and an environment. More information, model parameters, or computation cannot replace energy conversion, material processing, equipment assembly, or field maintenance.

Physical processes also change the meaning of a computation. Equipment may respond differently from the model's prediction, materials may degrade, and environmental conditions may shift. A system must update its judgment from observations and check whether the result meets requirements for safety, performance, and lifetime. Physical AI concerns this continuing interaction. Com-

putation proposes an action, equipment carries it out, the environment provides feedback, and the system adjusts its next action.

The basic unit of Physical AI is therefore a physical system that combines perception, computation, execution, verification, and maintenance. An isolated model cannot provide this capability by itself. A computational capability acquires civilizational significance when it can reliably change a real process and remain interpretable and recoverable under different conditions.

From Local Capability to Shared Physical Services Physical capability usually begins with one set of equipment at one site. It performs a task under particular material, environmental, and operating conditions, while also generating calibration, failure, and maintenance experience. To move the capability to another site, its dependencies must be identified, transferable steps must be separated from site-specific ones, and the new setting must provide its own evidence.

When the experience, models, interfaces, and equipment can be used by another team, a local capability becomes a shared capability. Sharing includes equipment access, task execution, result verification, and maintenance support, not only a model file. A user specifies a physical task, a provider performs it under agreed conditions and returns evidence, and the parties define permissions, responsibilities, and failure handling.

PAIaaS is the service form of this shared capability. It makes experiments, measurements, manufacturing, energy operations, and maintenance available on demand. This lowers the threshold for using specialized facilities, improves equipment utilization, and allows services at different locations to be combined. A local success that has been validated can then be inherited by more teams and later generations.

Physical services remain constrained by resources, equipment, and operating environments, so they cannot be copied without limit like digital interfaces. A service must be usable by a new team, its results must withstand verification, and its facility must continue operating after the original developers leave. When these conditions are met, local experience can become a civilizational capability that remains available over time.

12.2.2 Toward a Type I Civilization, Planetary-Scale Physical Systems

Moving from this estimated level toward Type I begins on Earth. The first task is to organize energy and infrastructure at planetary scale. Physical AI supports continuous improvement of the energy system, beginning with more effective use and acquisition of existing energy, continuing with new energy processes, and extending to controlled fusion as a stable facility. Every stage depends on perception, control, diagnosis, maintenance, and iteration in real environments.

Existing Energy Systems Fossil fuels, wind, solar, hydropower, and nuclear fission already form the energy base of modern society. Here, the focus is on operating and maintaining facilities already in service. Physical AI can support coordinated dispatch, equipment inspection, fault diagnosis, and maintenance across the chain from resource acquisition to final use. For nuclear fission facilities, these tasks also involve monitoring material degradation and planning maintenance within independently verified safety limits.

Large energy networks depend on connected equipment. Wind and solar output varies with weather and location, while transmission and storage constrain dispatch. A local improvement can create congestion elsewhere. Physical AI can adjust generation, transmission, and demand using equipment state, weather, and changing loads. The relevant measures are reliable supply, recovery from failure, and sustained service.

Energy extraction is part of the same system. Oil, gas, coal, minerals, deep-sea resources, and nuclear fuels require exploration, collection, transport, processing, and maintenance. Wind, solar, and hydropower require dispersed equipment that must also be built and maintained. Local conditions in deep-sea, underground, and high-temperature or high-pressure facilities often exceed what de-

sign documents describe. Physical AI can use field observations to adjust an action, conduct a limited probe, or suspend a task when the risk becomes too high.

Records of extraction and maintenance should inform later construction. Repeated failures under particular conditions can expose problems in materials, site selection, or maintenance plans. Physical AI can identify such patterns and use them to improve equipment design and operation. Each operating cycle can then inform the next round of infrastructure.

Developing Advanced Energy Technologies The focus now shifts from operating existing facilities to developing and validating new materials, devices, and processes, including advances in photovoltaics, wind power, storage, and nuclear fission.

Physical AI can support this development by connecting design proposals to physical experiments. In photovoltaic research, a system could select material compositions and fabrication conditions, prepare samples, and compare measured performance with model predictions. Similar workflows could examine storage materials, turbine components, or materials intended for nuclear systems. The experimental objective is to establish how a proposed change affects performance, durability, and operating limits.

Results from these experiments should guide subsequent tests and revisions to materials, structures, and control plans. Progress toward deployment requires evidence from prototypes and extended operation, together with independent verification of relevant safety requirements. Physical AI can help carry knowledge from one development stage to the next, while controlled fusion presents a particularly demanding example of this process.

Controlled Fusion on Earth Controlled fusion is a key energy process on the path to Type I. The Sun continuously produces energy through fusion. On Earth, the challenge is to create and maintain an extreme plasma state while managing magnetic fields, heating, wall materials, energy output, and component lifetime.

Tokamak plasmas change rapidly and involve many control variables, so fixed rules cannot cover every operating state. Degrave et al. (2022) show that reinforcement-learning controllers can move from simulation to a real TCV tokamak and control several plasma configurations magnetically. The experiment shows how Physical AI can turn a theoretical configuration into an operable and measurable experiment.

Fusion development must extend from research into engineering and operation. Models should be updated from experimental observations, follow-up experiments should be selected, equipment should be improved, and long-term stability should be tested. Sustained generation and safe maintenance require evidence from real devices. Physical AI can shorten the path from physical discovery to engineering iteration and carry the knowledge from one experiment into the next device.

Planetary Coordination and Long-Term Maintenance When energy systems expand to planetary scale, one facility can affect other regions and infrastructures. Reservoir operation can change downstream water and ecological conditions, while large energy facilities can alter land use, water use, and regional industry. Models must represent these connections and be tested against observations at the relevant scale.

Physical AI can help identify missing data, compare operating plans, and schedule monitoring and maintenance. Models cannot replace public decisions or remove the physical consequences of intervention. Climate intervention in particular requires a clear distinction between mechanism studies and large-scale deployment, together with explicit treatment of uncertainty, risk, and responsibility.

Type I civilization requires energy and infrastructure services to continue over long periods. Inspection, repair, and replacement should occur before failures spread, and efficiency should be evaluated across the full service cycle. More complex control can increase sensor and computation requirements, while equipment that cannot be repaired locally can increase dependence on remote

supply. The maturity of Physical AI is assessed by stable energy supply, continuing maintenance, and rapid recovery across the planet.

12.2.3 Toward a Type II Civilization, Space Infrastructure and Stellar-Scale Energy

Type II civilization extends physical activity to stellar scale. One reason for this transition is that energy development and large infrastructure on Earth face increasing constraints from space, environmental effects, and social coordination. New computing, energy, and manufacturing facilities occupy public space and can conflict with local communities and other industries. Moving parts of energy production, computation, resource extraction, and manufacturing to orbit, the Moon, Mars, and other bodies may provide another way to expand civilizational capability.

Space facilities must do more than complete a single mission. They need to build themselves, use local resources, produce replacement parts, manage failures, and operate with little direct human presence. Physical AI allows these activities to continue despite communication delay, extreme environments, and incomplete information.

Deep-Space Autonomy and Continual Learning Earth-Mars one-way light time varies with orbital configuration from roughly 3 to 22 min (National Aeronautics and Space Administration, 2023). Urgent actions must be performed locally, equipment diagnosis can wait longer, and changes to mission objectives generally require ground review. Perseverance's AutoNav illustrates this arrangement by supporting onboard navigation and hazard avoidance while the ground team plans routes and mission activities (Jet Propulsion Laboratory, 2021). Remote autonomy can begin with defined local tasks, while validated control, local adaptation, and ground supervision handle decisions at different timescales.

Deep-space missions encounter environments that cannot be fully covered during preparation on Earth. Javed and Sutton's Big World Hypothesis describes agents whose capacity is limited relative to the environments they encounter (Javed and Sutton, 2024). Such agents cannot maintain a complete world description and must understand the parts relevant to their current goals. A model trained on Earth therefore omits features that matter at a particular site. New observations may require a revised state estimate, a different existing plan, or a change to the model and policy.

Continual learning needs evidence from local experiments. A rover can first probe low-risk ground and then decide whether to enter more difficult terrain. The probe should change the next decision, while the risks of immobilization, energy use, and tool damage remain acceptable. Some unknowns cannot be resolved safely, so the system must be able to postpone an objective. Long-duration construction also requires constrained physical self-improvement in tools, experiments, and local control. Such changes need field tests, regression checks, and rollback procedures.

Adaptation Under Unfamiliar Physical Conditions Deep-space autonomy must also address physical conditions that differ from those encountered during development on Earth. The broad gravitational conditions of the Moon and Mars are already characterized, but local terrain, contact properties, and equipment responses may remain incompletely known. The challenge is to determine how transferable physical models apply at a particular site and when their assumptions cease to support reliable action.

Reduced gravity changes the relationship between equipment weight, available traction, and the reaction forces generated by tools. An excavator or assembly robot may move or lose stable contact when applying forces that a terrestrial platform could resist. NASA's RASSOR excavator illustrates the resulting need for different mechanical arrangements. Opposing, counterrotating bucket drums reduce the net reaction forces associated with excavation.[11] Physical AI would need to estimate support conditions and coordinate tool motion with vehicle posture, anchoring, or other means of stabilization.

[11] NASA Kennedy Space Center, *Regolith Advanced Surface Systems Operations Robot (RASSOR) Excavator*, technology-transfer page.

Illumination creates a different set of coupled constraints. Low solar angles near the lunar south pole produce long shadows and strong contrasts that complicate terrain interpretation (Litaker et al., 2025). A route that appears geometrically accessible may offer insufficient visibility or solar energy for continued operation. Physical AI should therefore combine terrain estimates, sensing confidence, illumination forecasts, and energy reserves when selecting routes and work periods.

Surface interaction introduces further uncertainty. Friction alone is insufficient to describe whether a vehicle slips, sinks, or remains supported, or whether a tool can penetrate the ground. Local estimates must also account for bearing capacity, cohesion, and the response of material to repeated contact. Limited traverses or shallow tool interactions could provide evidence for revising these estimates, provided that the system retains sufficient resources and a feasible recovery option.

Future research should examine how Physical AI can identify these conditions from limited observations and interactions, adapt control without losing previously validated capabilities, and recognize when further probing would be too costly. Evaluation should include the physical interactions required before reliable operation, the energy and equipment wear incurred during adaptation, and recovery when an environmental estimate proves incorrect.

Planetary Construction and Underground Habitats Space construction is harder than a single measurement because each action changes the setting for the next one. Lunar and Martian facilities may benefit from underground spaces, which can provide radiation protection and stable temperatures while reducing exposure to surface conditions. Construction must proceed from local observations and repeatedly adjust excavation, support, and inspection.

A lunar energy facility would pass through transport, positioning, installation, inspection, and connection. Each installed component must be checked before later work can rely on it. The system cannot simply execute a design prepared in advance. It must establish step by step that the emerging structure can support further construction and operation.

Space-based solar power provides a concrete object for studying these dependencies. NASA's 2024 assessment identifies major gaps in orbital assembly and maintenance, autonomous operation, and power transmission (Rodgers et al., 2024). An orbital collector becomes an energy service only when its associated equipment can be deployed, repaired, and operated continuously. Physical AI must connect construction planning, state inspection, failure handling, and later task scheduling.

Local production can reduce resupply, but raw material alone does not create manufacturing capability. A remote facility must repeatedly produce components that meet performance requirements and verify that they are fit for their intended roles. Materials processing, quality verification, and equipment maintenance must form a continuous local process.

Long-Duration Operation with Limited Resources Decision autonomy and resource independence are different properties. A facility may schedule months of work while still depending on Earth for a critical component. Better planning can extend limited supplies, but it cannot replace missing materials or tools. The autonomy of a remote facility must therefore be described together with its remaining dependence on resupply, communication, and maintenance.

Long-duration operation requires choices between current tasks and future capability. When resources are scarce, equipment status is uncertain, or external support is interrupted, postponing an experiment may preserve more useful work. A remote facility should leave later missions with usable energy, tools, data, and maintenance conditions rather than forcing every mission to start again from Earth.

Stellar-Scale Infrastructure and Dyson Swarms A Dyson swarm is a hypothetical arrangement of independently orbiting collectors for harvesting stellar radiation (Wright, 2020). Its relevance to Physical AI lies in the construction and maintenance tasks that such an arrangement would impose. Components would have to be manufactured, transported, assembled, inspected, and replaced across widely separated sites, with limited opportunities for direct human intervention.

Physical AI could support these tasks by connecting local observations to physical action. During assembly, a robotic system would need to estimate component pose and deformation, adjust contact forces, and verify each completed connection before further construction proceeds. During operation, each facility would need to diagnose degradation and adjust energy collection and use within equipment and thermal limits. Repair would require identifying a fault, selecting an available intervention, and checking that the repaired equipment can safely resume its role.

Coordination across the swarm would also require Physical AI at multiple scales. Local systems would handle immediate control and fault response, while broader coordination would allocate transport, manufacturing, and maintenance resources using delayed information. Construction and repair records should preserve the conditions under which earlier designs were validated, allowing later systems to assess compatibility when replacing components or extending facilities.

These requirements define a speculative research setting for autonomous construction, distributed coordination, and maintenance across successive generations of equipment. Physical AI could help identify and operate workable arrangements, but those arrangements would still require suitable materials, manufacturing capacity, and sufficient heat rejection. Their capability would be demonstrated through verified construction, recovery from failures, and sustained useful operation.

12.3 Summary and Outlook

This chapter has considered Physical AI over the long time horizon of human development and civilizational change. Physical AI requires stable infrastructure and systems in which equipment, data, computation, and people can continue to work together. The value of a project lies in its results and in the models, interfaces, methods, and operating experience that later teams can verify and reuse.

The development of Physical AI can begin in scientific research and engineering practice. Shared models and data, connected experimental facilities, cross-institutional service interfaces, and deployment of validated capabilities all contribute to the accumulation of physical capability. PAIaaS offers an organizing form for this process. Experiments, manufacturing, inspection, energy operations, and maintenance can become services that other teams call when needed. Physical capability can then grow from a single site into a broader network of cooperation.

At the scale of civilization, humanity must first improve the acquisition and use of energy on Earth, develop new energy and nuclear systems, build planetary infrastructure, and advance controlled fusion. A more distant stage requires deep-space infrastructure that can operate with communication delay, continual learning, autonomous construction, local manufacturing, and long-duration activity. Civilizational level is ultimately reflected in whether energy, equipment, materials, and maintenance can continue to work across larger ranges and longer periods.

This monograph has examined the foundations, methods, systems, and applications of Physical AI, and has considered the perception, action, learning, and adaptation required by Physical AGI. These discussions show that Physical AI requires systems to understand environments, act in them, bear the consequences of action, and change their later behavior from real experience. A digital model becomes a component of a Physical AI system only when it is integrated into this closed-loop process.

Future progress must be tested in concrete equipment and real tasks. Laboratories, factories, energy facilities, and deep-space systems impose different constraints and expose unresolved weaknesses in current models and methods. Progress toward Physical AGI depends on whether systems can gain reliable experience under these constraints, turn local experience into transferable capability, and remain safe, interpretable, and maintainable over long operation. Physical AI may become an important technical basis for humanity's transition to a higher civilization, but reaching that future requires sustained work by researchers, engineers, and infrastructure operators.

Contributors

Monograph leading authors: Weinan Zhang, Chunhua Shen, Tianfan Xue, Bin Zhao, Bowen Zhou

Overall coordination group: Weinan Zhang, Bingqi Jiang, Zihan Guo, Zicai Cui

- Chapter 1: Weinan Zhang, Yang Li, Ming Zhou, Bingqi Jiang.
- Chapter 2: Tianfan Xue, Xinyuan Chen, Lihe Ding, Xiao Chen.
- Chapter 3: Tianfan Xue, Yujin Wang, Shi Guo.
- Chapter 4: Chunhua Shen, Junting Dong, Tao Lu, Xing Shen.
- Chapter 5: Chunhua Shen, Haoyu Guo.
- Chapter 6: Weinan Zhang, Yao Mu, Haiyong Yu, Haoxiang Ma, Fuxian Huang, Qiaojun Yu, Yunsong Zhou.
- Chapter 7: Weinan Zhang, Kailin Li, Zirui Wang, Yufei Xue, Weishuai Zeng, Tao Huang.
- Chapter 8: Bin Zhao, Zhigang Wang, Junli Liu, Pengyuan Wu, Pingrui Zhang, Haotian Wang, Dongzhan Zhou.
- Chapter 9: Bin Zhao, Jiafei Cao, Feng Yuan, Xinyi Ye, Quanli Xuan, Yunsong Zhou, Kailin Li.
- Chapter 10: Chunhua Shen, Mulin Yu, Shangding Gu, Kaiwen Song.
- Chapter 11: Bin Zhao, Dong Wang, Jiarui Li, Kehui Liu, Zhongjie Jia, Dongjiang Li, Jiacheng Bao, Haoran Yang, Zhaoqing Chen, Shangding Gu.
- Chapter 12: Weinan Zhang, Bingqi Jiang, Zihan Guo, Yiming Sun.

Suggested citation (BibTeX)

```
@misc{zhang2026physicalai,
  author = {Zhang, Weinan and Shen, Chunhua and Xue, Tianfan and Zhao, Bin and Jiang,
  Bingqi and Guo, Zihan and Cui, Zicai and Li, Yang and Zhou, Ming and Chen, Xinyuan and
  Ding, Lihe and Chen, Xiao and Wang, Yujin and Guo, Shi and Dong, Junting and Lu, Tao and
  Shen, Xing and Guo, Haoyu and Mu, Yao and Yu, Haiyong and Ma, Haoxiang and Huang, Fuxian
  and Yu, Qiaojun and Zhou, Yunsong and Li, Kailin and Wang, Zirui and Xue, Yufei and
  Zeng, Weishuai and Huang, Tao and Wang, Zhigang and Liu, Junli and Wu, Pengyuan and
  Zhang, Pingrui and Wang, Haotian and Zhou, Dongzhan and Cao, Jiafei and Yuan, Feng and
  Ye, Xinyi and Xuan, Quanli and Yu, Mulin and Gu, Shangding and Song, Kaiwen and Wang,
  Dong and Li, Jiarui and Liu, Kehui and Jia, Zhongjie and Li, Dongjiang and Bao, Jiacheng
  and Yang, Haoran and Chen, Zhaoqing and Sun, Yiming and Zhou, Bowen},
  title = {{Physical AI}: Concepts, Techniques, and Challenges},
  year = {2026},
}
```

Terminology and Abbreviations

Chapter links point to uses of each term in the sense defined here, including its full form and equivalent expressions.

Term	Meaning in this monograph	Chapters
2D	*Two-Dimensional*. Representation in a plane or another two-dimensional space.	3, 5, 6
3D	*Three-Dimensional*. Representation of spatial position, shape, and orientation in three dimensions.	1, 2, 3, 4, 5, 6, 7, 8, 9, 11
3DGS	*3D Gaussian Splatting*. Scene representation and rendering using anisotropic three-dimensional Gaussian primitives.	3, 4
4D	*Four-Dimensional*. Three-dimensional spatial structure evolving over time in the perception context used here.	3, 5
Action alignment	Consistency of intentions, plans, and execution with human authorization, constraints, and acceptable outcomes.	11
Action chunk	A contiguous sequence of actions generated or processed together.	4, 5, 6, 7, 9
Action expert	A specialized module that generates continuous actions from semantic and robot-state inputs.	5, 6
Action proxy	An action representation inferred from video, poses, trajectories, or latent variables. It is not necessarily an executable robot command.	4, 6
ACT	*Action Chunking with Transformers*. A conditional-variational-autoencoder policy that predicts action chunks, using an L_1 reconstruction term and a KL-divergence regularizer during training.	6, 9
Affordance	An action opportunity relative to an agent's body, current conditions, and constraints.	1, 2, 3, 8, 9, 11
AGI	*Artificial General Intelligence*. The goal of general intelligence that learns and transfers across tasks and domains.	1, 10, 12
AI	*Artificial Intelligence*. Machine capabilities for perception, learning, reasoning, and decision-making.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
AoI	*Age of Information*. Time elapsed since generation of the latest update available at a receiver. The age of the observation actually used by an action is a distinct quantity.	9
API	*Application Programming Interface*. An interface through which software invokes capabilities and exchanges data.	8, 9
AR	*Autoregressive*. Sequential prediction conditioned on preceding variables or tokens.	5, 6, 7
Asset Administration Shell	A standardized representation of industrial-asset information and access to services.	8
Autonomy	The degree of independent action selection, execution, monitoring, and correction within specified objectives and operating limits.	1, 2, 3, 4, 6, 7, 8, 9, 10, 11, 12
BC	*Behavior Cloning*. Supervised policy learning from demonstrated actions conditioned on observations, histories, or task context.	4, 5, 6
Behavior tree	A tree-structured execution model organizing conditions, actions, sequences, and fallback behavior.	2, 8, 9
Belief state	A probability distribution over states conditioned on observation and action history.	2, 5, 8, 11
BFM	*Behavior Foundation Model*. A humanoid-control model that learns reusable whole-body behavior from broad motion data and interaction.	7

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
BLDC	*Brushless Direct Current Motor*. An electronically commutated permanent-magnet motor. The convention used here associates BLDC operation with trapezoidal back electromotive force and block commutation.	9
BS	*Brier Score*. The mean squared difference between predicted event probabilities and binary outcomes.	5
CAD	*Computer-Aided Design*. Editable geometric and engineering representations of objects or systems.	3, 7
Calibration	Adjustment or assessment against reference measurements or outcomes. Sensor, simulator, probability, and evaluator calibration have distinct objects.	1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12
Capability graph	A representation of available capabilities, their applicability conditions, and their composition relationships.	9
Causal Imprint	A research-specific mechanism in InternW0-Δ that learns action-relevant scene-change representations from observed context using future-derived training supervision. Realized future observations are not inputs to the action-prediction path.	5
CDF	*Cumulative Distribution Function*. A function giving the probability that a random variable does not exceed a specified value.	5, 7
CL	*Continual Learning*. Learning from new data or experience while maintaining previously acquired capabilities.	1, 6, 10, 11, 12
Closed loop	*Closed-Loop Feedback*. A feedback process in which measured outcomes inform subsequent decisions and control.	1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12
CMC	*Carpometacarpal Joint*. A joint between the wrist and metacarpal bones, used here to describe hand kinematics and sensor locations.	9
CNN	*Convolutional Neural Network*. A neural network using locally shared convolutional operations.	3
Collective Physical AI	System-level physical capability arising from complementary agents, shared information, or local coordination rules.	10
CoM	*Center of Mass*. The position determined by the mass-weighted distribution of a body.	4, 6, 7, 8, 9
Compliance	*Mechanical Compliance*. Displacement or deformation under an applied force, affecting contact and impact response.	1, 2, 3, 4, 6, 7, 8, 9, 11, 12
Control barrier function	A function specifying a safe set and associated constraints on admissible control under stated assumptions.	2, 8, 11
Controllability	For a specified dynamical model, the ability of admissible inputs to steer the state between specified states over a finite interval.	2
Counterfactual	*Counterfactual Prediction*. Prediction of consequences under alternative actions or interventions.	1, 4, 5, 6
CPS	*Cyber-Physical Systems*. Systems integrating computation, communication, and physical processes, including their timing and interface constraints.	1
CPU	*Central Processing Unit*. A processor for general-purpose computation and software execution.	4, 9
CRPS	*Continuous Ranked Probability Score*. A proper scoring rule comparing a predictive cumulative distribution with a realized scalar observation.	5
CTDE	*Centralized Training with Decentralized Execution*. Training with broader information while deployed agents act on local inputs and available messages.	8

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
Curriculum learning	Training organized progressively by difficulty, horizon, or task structure.	2, 4, 5, 6, 7
DAgger	*Dataset Aggregation*. An iterative imitation-learning method that aggregates expert labels on states visited during rollouts that may mix expert and learner policies. Its task-cost guarantees depend on the stated learning and analysis assumptions.	4, 6, 9
Data flywheel	An improvement cycle linking deployment, evaluation, correction, data selection, and retraining.	1, 4
Data pyramid	An organization of data sources by scale, diversity, collection cost, and relevance to target physical execution.	4, 9
DC	*Direct Current*. Electric current with a fixed direction, used here in motor-drive discussions.	9
DDS	*Data Distribution Service*. A data-centric publish-subscribe communication standard with configurable quality-of-service policies.	9
Diffusion policy	A policy generating continuous action sequences through conditional denoising.	4, 5, 6, 8
Digital AI	AI whose direct operating interfaces act on digital representations, regardless of whether the represented information originates in the physical world.	1
Digital cousin	A generated environment preserving relevant interaction properties without exactly replicating a physical scene.	4
Digital twin	A digital representation associated with a particular physical object or process and updated using observations.	1, 4, 5
Distillation	Transfer of knowledge from a teacher, privileged information, or auxiliary modalities to a deployment model.	4, 5, 6, 7
Distributed Physical AI	Coordinated physical capability across nodes retaining local sensing, decision-making, and action.	8, 9
DiT	*Diffusion Transformer*. A Transformer-based denoising architecture for image, video, or continuous-action generation.	6
DoF	*Degrees of Freedom*. The number of independent coordinates describing a system's motion.	2, 4, 6, 7, 8, 9
Domain randomization	Variation of simulated parameters, appearance, or disturbances to cover uncertain deployment conditions.	1, 4, 7
EKF	*Extended Kalman Filter*. A nonlinear state estimator using local linearization for prediction and measurement correction.	7
ELBO	*Evidence Lower Bound*. A variational lower bound on log marginal likelihood. Its negative is used as a training objective.	5
EMA	*Exponential Moving Average*. A recursively weighted average used here to update target-encoder parameters from student-encoder parameters.	5
Embodied AI	*Embodied Artificial Intelligence*. AI studied through the coupled relationships among an agent's body, perception, action, and environment, including simulated embodiments.	1, 2
Embodied cognition	Theories explaining cognition through interactions among an agent, its body, and its environment.	1, 2
Embodiment	The body structure, sensors, actuators, and physical interfaces through which a system interacts with its environment.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
EMG	*Electromyography*. Recording of muscle electrical activity for movement analysis or human-machine interfaces.	9
End effector	A tool, gripper, or hand through which a robot directly interacts with a task object.	1, 2, 3, 4, 6, 7, 8, 9, 11, 12
Equivariance	A property whereby a transformation of the input induces the corresponding transformation of the output.	2
ESKF	*Error-State Kalman Filter*. A Kalman-filter formulation propagating a nominal state and estimating local state errors.	7
EtherCAT	*Ethernet for Control Automation Technology*. An industrial Ethernet technology for real-time automation and control.	9
Ethernet	A networking technology used to connect devices, task computers, and control systems.	9
FDM	*Forward Dynamics Model*. A model predicting future observations or representations from observed context and specified actions.	5
Fine-tuning	Adaptation by updating parameters of an existing model using new data.	4, 6, 7, 9, 10, 11
FM	*Flow Matching*. Generative learning of a flow or velocity field connecting noise and data distributions.	5, 6
FOC	*Field-Oriented Control*. Motor control using a rotating reference frame to regulate flux-related and torque-related current components.	9
Forward dynamics	Computation of acceleration or state evolution from a mechanical state and input forces. Learned formulations may instead predict states or observations from context and actions.	5, 7
Foundation model	A model pretrained on broad data that can be adapted to a range of downstream tasks.	1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12
Foundation models for robot manipulation	Pretrained models providing reusable representations and policies for manipulation across tasks, objects, environments, or robot embodiments.	1, 4, 6, 10
Foundation models for whole-body motor control	Broadly pretrained models providing reusable motor capabilities for coordinated whole-body motion under dynamics, contact, and actuator constraints.	1, 4, 6, 7, 10
Friction cone	The set of admissible contact-force vectors with a nonnegative normal component and a tangential magnitude bounded by the friction coefficient times the normal component.	2, 7
FSM	*Finite-State Machine*. An execution model with discrete modes and event-triggered transitions.	2
FVD	*Fréchet Video Distance*. A video-generation metric comparing distributions of learned video features.	5
G-MPKPE	World-frame position error averaged over evaluated timesteps and selected keypoints.	7
G-MPKRE	World-frame orientation error averaged using angular distance on the rotation group.	7
GPU	*Graphics Processing Unit*. A processor supporting parallel rendering, simulation, training, and inference.	4, 9
Grounding	Association of symbols, instructions, or goals with observable entities, action capabilities, and physical consequences.	1, 2, 3, 4, 5, 6, 8, 10, 12
Harness	*Embodied Intelligence Harness*. A runtime organization connecting model proposals, skill contracts, actuation, feedback, and authorization boundaries.	9

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
HASEL	*Hydraulically Amplified Self-Healing Electrostatic Actuator*. A soft actuator using electrostatic forces to redistribute a dielectric liquid and produce deformation.	9
HIL	*Hardware-in-the-Loop*. Testing that connects target control hardware and firmware to a simulated plant.	1, 6, 9, 11
HITL	*Hardware-in-the-Loop*. The PX4 spelling of hardware-in-the-loop testing in this monograph. It does not denote human-in-the-loop operation here.	9
I/O	*Input/Output*. Sensor and actuator interfaces or computational data transfer and access.	9
IDM	*Inverse Dynamics Model*. A model inferring compatible actions from observed state or observation transitions. Using realized future observations is a training or retrospective inference interface.	4, 5, 7
Imitation learning	Learning action selection or movement from demonstrations.	2, 4, 6, 7
Impedance control	Control of interaction through a specified dynamic relationship between motion and force.	7, 9
IMU	*Inertial Measurement Unit*. An inertial sensor assembly providing angular-rate and specific-force measurements.	3, 7, 9, 11
InEKF	*Invariant Extended Kalman Filter*. An extended Kalman filter using system symmetries to formulate estimation errors.	7
Information freshness	The temporal relevance of available observations to a current decision or action.	8, 9
Inverse dynamics	Computation of required forces or torques from a mechanical state and desired acceleration. Learned formulations may infer actions from transitions.	2, 4, 5, 6, 7
ISP	*Image Signal Processor/Processing*. The processor or processing pipeline converting raw camera samples into usable images.	3
JEPA	*Joint-Embedding Predictive Architecture*. An architecture predicting target representations from context representations without requiring reconstruction of every raw observation.	2, 5, 10
Kardashev index	An index expressing a civilization's energy scale in terms of sustained power use.	12
KL	*Kullback-Leibler Divergence*. An asymmetric divergence between probability distributions.	5, 6
Knowledge insulation	A VLA training scheme that blocks gradients from the continuous action expert into the vision-language backbone while training the backbone with discrete action and other available vision-language objectives.	6
LiDAR	*Light Detection and Ranging*. Laser-based distance sensing for range measurements or point clouds.	2, 3, 7, 8, 9
LLM	*Large Language Model*. A large language model used for language processing, reasoning, or task decomposition.	1, 10, 11
L-MPKPE	Mean per-keypoint position error after removing horizontal root translation and heading at each timestep.	7
L-MPKRE	Mean per-keypoint orientation error in the local frame defined by root heading at each timestep.	7
LoRA	*Low-Rank Adaptation*. Adaptation of a pretrained model using trainable low-rank parameter updates.	6
LPIPS	*Learned Perceptual Image Patch Similarity*. An image-difference metric computed from learned visual features.	5

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
LQR	*Linear-Quadratic Regulator*. Optimal state feedback for a linear model with a quadratic cost.	2
Lyapunov function	A scalar function whose evolution along trajectories supports a stability analysis.	2
MCP	*Model Context Protocol*. A protocol supporting structured discovery and invocation of tools by AI applications.	8
MDP	*Markov Decision Process*. A sequential-decision model specified by states, actions, transitions, rewards, and a planning or discounting convention.	2
MHS	*Model Hardware Standard*. The physical-device discovery, description, and runtime-interface specification discussed in this monograph.	8
MLP	*Multilayer Perceptron*. A feedforward network composed of successive nonlinear transformations.	6, 7
MMRV	*Mean Maximum Rank Violation*. A metric quantifying real-performance differences associated with simulator-induced policy-ranking errors.	4
Model-hardware co-design	Joint design of mechanisms, sensing, action interfaces, and models under task and resource constraints.	9
MOF	*Metal-Organic Framework*. A class of porous materials formed from metal nodes and organic linkers.	9
Morphological computation	Use of body geometry and dynamics to distribute or reduce the computation required for control.	2
MoT	*Mixture of Transformers*. An architecture in which separately parameterized Transformer experts exchange or jointly process modality information.	5, 6
MPC	*Model Predictive Control*. Repeated finite-horizon optimization based on the current state estimate, followed by execution of the initial control segment.	1, 2, 5, 7, 8
Native governance	Governance that embeds authorization, responsibility, constraints, and traceability within capability networks and execution.	9
NeRF	*Neural Radiance Field*. A continuous neural representation mapping position and viewing direction to density and radiance or appearance.	2, 3, 4
Neural ODE	*Neural Ordinary Differential Equation*. A continuous-time dynamical model with a neural-network-parameterized state derivative.	2
NMPC	*Nonlinear Model Predictive Control*. Receding-horizon optimization with nonlinear dynamics and relevant motion or contact constraints.	7
Observability	The ability to infer relevant states from measurements under a specified dynamics and sensing model.	2, 9, 11
Operational controllability	The ability of protective mechanisms and authorized supervision to intervene in time, maintain operating limits, and establish an appropriate fallback state. This differs from dynamical controllability.	11
OPC UA	*OPC Unified Architecture*. An industrial communication and information-modeling architecture for interoperable services and devices.	8
OTA	*Over-the-Air Update*. Network delivery of software, model, or firmware updates.	11
Outcome unknown	An execution state in which an action may have affected the world but its outcome or acknowledgment is unavailable.	9

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
PAIaaS	*Physical AI as a Service*. The service organization described in this monograph, through which users access physical capabilities such as experiments, manufacturing, inspection, or maintenance over shared interfaces.	12
Passivity	A system property expressed through input-output energy inequalities.	7
PD	*Proportional-Derivative Control*. Feedback using proportional and derivative error terms.	7
PDE	*Partial Differential Equation*. An equation involving partial derivatives of an unknown field with respect to multiple independent variables.	2, 5
Physical agent	An agent that senses through physical interfaces, selects interventions, and adjusts behavior using measured outcomes.	1, 2, 5, 8, 11, 12
Physical AGI	*Physical Artificial General Intelligence*. The goal of general intelligence for an open physical world, requiring systematic generalization, long-term adaptation, and reliable action.	1, 10, 12
Physical AI	*Physical Artificial Intelligence*. Artificial-system capability to perceive physical states, represent relevant relationships and action consequences, and select and execute interventions through feedback under physical, operational, and computational constraints.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
Physical intelligence	Intelligent or adaptive capability arising through physical interaction or the organization of bodies and materials. Its scope follows the source and context.	1, 8, 9
Physical RSI	*Physical Recursive Self-Improvement*. The research framework cited in Chapter 10 for using physical interaction to persistently update both a robot's task capabilities and its improvement procedure, with the aim of making subsequent improvement more effective across tasks.	10
PID	*Proportional-Integral-Derivative*. Feedback using proportional, integral, and derivative error terms.	2
PINN	*Physics-Informed Neural Network*. A neural model trained using data together with physical equations or residual constraints.	2, 5
PMSM	*Permanent-Magnet Synchronous Motor*. A permanent-magnet motor operating synchronously with its rotating magnetic field, commonly using sinusoidal currents and vector control.	9
Policy	A rule or model selecting actions from observations, states, histories, or goals.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
POMDP	*Partially Observable Markov Decision Process*. A sequential-decision model in which states are inferred indirectly from observations.	2, 5, 6
Post-training	Training that adapts general priors using task data, interaction outcomes, or corrections.	4, 5, 6, 11
PPO	*Proximal Policy Optimization*. A policy-gradient method whose clipped surrogate objective discourages excessively large updates. Clipping is not a hard bound on the realized policy change.	2, 6, 7
Pre-training	Training on broad data to acquire reusable representations or behavioral priors.	1, 2, 4, 5, 6, 7, 9, 11
Privacy by Embodiment	A design principle used here to incorporate privacy into sensing and actuation, including effects on bystanders.	11

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
Proprioception	Observation of the robot's own configuration, motion, and related internal states.	1, 2, 3, 5, 6, 7, 8, 9
QoS	*Quality of Service*. Communication policies specifying properties such as reliability, history, deadlines, and liveliness.	9
Quasi-direct drive	Actuation combining high-torque motors with low-ratio transmission to balance torque, backdrivability, and impact response.	7, 9
Real-time	*Real-Time Execution*. Execution within specified sensing, computation, communication, and control deadlines.	1, 2, 4, 5, 6, 7, 9, 11
Recovery	*Failure Recovery*. Correction of behavior after failure to preserve objectives and restore task progress. Physical consequences are not necessarily reversible.	1, 2, 4, 5, 6, 7, 8, 9, 10, 11, 12
Reliability	Provision of specified physical outcomes over a stated operating period and set of conditions.	1, 3, 4, 5, 7, 9, 10, 11, 12
Retargeting	*Motion Retargeting*. Mapping motion from a person or another embodiment to a target robot's kinematic and movement spaces.	4, 6, 7, 9
Reward hacking	Exploitation of reward or evaluator errors to improve a score without achieving the intended physical outcome.	6
RFT	*Reinforcement Fine-Tuning*. Policy fine-tuning using execution outcomes and rewards. It is a method within the broader reinforcement-learning-based post-training stage.	6
RGB	*Red, Green, Blue*. The three color channels used here to represent color-image observations.	3, 4, 5, 6, 7, 9
RGB-D	*Red, Green, Blue plus Depth*. Color-image observations with associated depth measurements.	3, 4, 6, 7, 9
RL	*Reinforcement Learning*. Learning sequential decisions from interaction-dependent rewards or returns.	1, 2, 4, 5, 6, 7, 9, 10, 12
RLDS	*Reinforcement Learning Datasets*. A data format organizing interaction into episodes and steps, with explicit boundary and termination semantics.	4
RMSE	*Root Mean Square Error*. The square root of mean squared prediction error.	5
RMSNorm	*Root Mean Square Layer Normalization*. Feature normalization by root mean square without subtracting the feature mean.	7
Robustness	Maintenance of specified performance under modeling errors, disturbances, or changing conditions.	1, 2, 5, 7, 11
ROS 2	*Robot Operating System 2*. A robotics software framework providing modular components, communication interfaces, and tools for composing robot systems.	9, 11
RS485	*RS-485*. A differential serial electrical-interface standard. It does not specify robot-command semantics.	9
Runtime assurance	Runtime monitoring and intervention that maintain specified operating conditions under stated assumptions.	8, 9, 11
RVQ	*Residual Vector Quantization*. Multi-stage quantization of successive residuals using vector codebooks.	6
Safety	*Physical Safety*. Control of harm and unacceptable physical consequences within specified limits.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
Scaling law	An empirical relationship between training scale, model capacity, data, and performance under stated conditions.	4
Scene graph	A representation of entities and their spatial, semantic, or interaction relationships.	3, 5

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
SDF	*Signed Distance Function/Field*. A geometric representation whose signed distance is zero at the represented surface. The sign convention is defined locally.	2, 3
SDK	*Software Development Kit*. A collection of software libraries, interfaces, and examples for development.	9
SDL	*Self-Driving Laboratory*. A laboratory closing the loop between experimental design, execution, measurement, and subsequent decisions.	9
SE(3)	*Special Euclidean Group in Three Dimensions*. The group of three-dimensional rigid transformations combining rotation and translation.	2, 3, 7
Sensor fusion	*Multimodal Sensor Fusion*. Combination of sensor information using compatible temporal and spatial reference conventions.	1, 3, 7, 9
Series elastic actuation	Actuation with an elastic element in the transmission path, supporting torque estimation and impact absorption.	7, 9
SFT	*Supervised Fine-Tuning*. Adaptation of a model or policy using supervised data. It is a method within the broader supervised post-training stage.	6
Shared autonomy	Allocation of task or action responsibilities between human guidance and autonomous robot control.	6, 9
SIL	*Software-in-the-Loop*. Testing that connects executable software to a simulated plant without target control hardware.	9
Sim-to-real	*Simulation-to-Real Transfer*. Transfer of policies or capabilities learned or evaluated in simulation to physical systems.	1, 4, 7, 11
Skill contract	An interface specification for a skill's inputs, preconditions, authority, limits, timing, feedback, and termination.	9
SLAM	*Simultaneous Localization and Mapping*. Joint estimation of an agent's pose and a map of an initially unknown environment.	2
SO(3)	*Special Orthogonal Group in Three Dimensions*. The matrix group representing three-dimensional rotations.	2, 7
SR	*Success Rate*. The fraction of trials satisfying a specified success criterion. Evaluation trials and adaptation demonstrations are different counts.	1, 4, 5, 6, 7, 8, 9, 10, 11
SSM	*State-Space Model*. A temporal model defined through latent-state evolution and observation generation.	2, 5
Stability	A property of behavior near an equilibrium or reference trajectory under specified initial conditions and disturbances.	1, 2, 4, 6, 7, 8, 9, 11, 12
State estimation	Inference of unobserved physical states using measurements, history, and models.	1, 2, 3, 5, 6, 7, 8, 9, 11, 12
STRIPS	*Stanford Research Institute Problem Solver*. A classical planning system and representation based on action preconditions and effects.	8
System identification	Estimation of a system's model, parameters, or dynamics from inputs and observations.	4, 5, 7, 11
Systematic generalization	Recombination, transfer, and application of capabilities across task, environment, object, and embodiment changes.	10
Tactile sensing	Sensing of contact deformation, pressure, shear, or slip to support contact-state inference.	1, 3, 4, 6, 7, 9
TAMP	*Task and Motion Planning*. Joint reasoning about discrete task actions and continuous geometric and physical feasibility.	8

Continued on the next page

Term	Meaning in this monograph (continued)	Chapters
Taxel	A discrete tactile sensing element with a specified sensitive region and a measured response to local contact.	3
Teacher forcing	Training with ground-truth preceding inputs rather than exclusively with a model's own preceding predictions.	5
Teleoperation	Human operation of a robot through remote or mapped control interfaces with feedback.	1, 4, 6, 7, 8, 9, 10
Thing Description	*Web of Things Thing Description*. A standardized description of a device's interaction affordances within the Web of Things.	8
Twist	A six-dimensional representation of rigid-body velocity comprising linear and angular velocity, with a specified coordinate frame and reference point.	7
UMI	*Universal Manipulation Interface*. A handheld-gripper demonstration interface combining visual tracking, relative action trajectories, and latency matching for transfer to compatible robot systems.	4, 6, 9
Uncertainty	*Predictive Uncertainty*. Representation of uncertainty in predicted outcomes, states, or models, distinguishing observational variability from limited knowledge where applicable.	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12
Underactuation	A condition in which not all degrees of freedom are directly driven by independent actuators.	7
URDF	*Unified Robot Description Format*. A robot-description format representing links, joints, geometry, and associated properties.	9
USB	*Universal Serial Bus*. An interface standard for connecting devices and transferring data.	9
VAE	*Variational Autoencoder*. A probabilistic encoder-decoder model with latent variables.	5, 6
ViT	*Vision Transformer*. A Transformer architecture processing image patches as tokens.	6
VLA	*Vision-Language-Action Model*. A model combining visual, language, and action processing to generate robot actions from observations and instructions.	2, 3, 4, 6, 9
VLM	*Vision-Language Model*. A model jointly processing visual and textual information. A visual-language evaluator does not thereby acquire physical action capability.	3, 4, 5, 6, 8, 11
VR	*Virtual Reality*. An immersive visual and tracked-input interface used here for teleoperation.	9
WAM	*World-Action Model*. A model coupling action generation with prediction of future observations or states. The category can overlap with VLA models.	2, 5, 6
WBC	*Whole-Body Control*. Coordination of whole-body motion and contact forces under robot dynamics and task constraints.	7
World model	An internal model of task-relevant physical state and action consequences. Visual realism alone does not establish predictive physical validity.	1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 12
Wrench	A six-dimensional representation of force and moment, expressed in a specified coordinate frame with the moment taken about a specified reference point.	2, 3, 7
Zero-shot	*Zero-Shot Transfer/Execution*. Execution under stated target conditions without new target demonstrations or parameter updates.	4, 6, 7, 10

Notation

Semantic subscripts and superscripts distinguish different roles. Bold type denotes explicitly defined vectors or matrices. Abstract states, histories, functions, and model parameters retain their specified types. Descriptive labels, coordinate-frame labels, and units are upright. Variable indices are italic. Reused dummy indices and generic function or parameter notation are defined at their point of use. Chapter references identify the relevant definitions.

Symbol	Meaning	Chapters
$A_t^{\text{seq}}, \mathbf{A}_t^{\text{seq}}$	Action sequence or candidate plan. In Chapter 6, the explicitly matrix-valued action chunk has shape $H \times d$, with one d-dimensional action per row.	5, 6
$A^{\text{adv},\pi}(h,a), \widehat{A}_t^{\text{adv}}$	Policy advantage conditioned on a history (or a state in the fully observed case), and its estimate.	2
$\mathbf{A}$	Linear state-transition matrix.	2
$A_i, \mathcal{A}_i$	Taxel area and sensitive region, with $A_i = \int_{\mathcal{A}_i} \mathrm{d}A$.	3
$\mathbf{A}_{\text{G}}$	Centroidal momentum matrix mapping generalized velocity to momentum.	7
$a_t, \mathbf{a}_t$	Action. Bold type denotes an explicitly vector-valued action. Superscripts cmd and exec distinguish commands and execution.	2, 4, 5, 6, 7, 8, 11
a_i^{lidar}	LiDAR return intensity.	3
a_{scale}	Positive coefficient in a scaling relationship.	4
$\Delta_{\text{age}}, \Delta_{\text{age}}(t), \Delta_{\text{age}}^{\max}$	Age of the observation used at a specified decision or actuation event, its time-indexed form, and the maximum permitted age. Distinct from standard AoI.	6, 9
$\mathbf{a}_{\text{imu}}$	IMU specific-force vector.	3
$\mathbf{a}_k^{\text{des}}$	Desired task-space acceleration.	7
$a_0^{\text{swing}}, \ldots, a_5^{\text{swing}}$	Swing-trajectory polynomial coefficients.	7
$\mathbf{B}$	Linear control-input matrix.	2
B_{stereo}	Stereo-camera baseline length.	3
N_{buf}	Number of actions in an action buffer.	6
N_{roll}	Number of candidate prediction rollouts.	5
$N_{\text{point}}, H_{\text{MPC}}$	Point-cloud point count and discrete MPC prediction horizon, respectively.	2
B_{emb}	An embodiment or physical platform.	8, 9, 11
B_{cab}	Cabinet object in the placement example.	8
$B_{\max}$	Worst-case blocking time.	9
$\mathcal{B}, \mathrm{B}$	Base coordinate frame, using the local frame convention.	7, 8
$\mathbf{B}^{\text{seq}}$	An interleaved body-state and action sequence.	7
$b_t, \widetilde{b}_t$	State belief and predictive belief, respectively.	2, 5, 8, 11
ζ_t^{state}	Optional state-information field in an interaction record. It may store a state estimate or a belief, with its representation declared locally.	4
b_{scale}	Positive coefficient in a scaling relationship.	4
b	Candidate-rollout index.	5
$\mathbf{b}$	Generalized dynamics bias, including Coriolis, centrifugal, and gravitational terms.	7
$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$	Coriolis and centrifugal matrix.	2
$C(s)$	Controller transfer function.	2
$\mathbf{C}_{\text{obs}}$	Linear observation matrix.	2
$\mathbf{C}_{\text{cal}}$	Force/torque sensor calibration matrix.	3
C_{train}	Training compute, with the local approximation $C_{\text{train}} \approx 6 N_{\text{par}} N_{\text{data}}$.	4

Continued on the next page

Symbol	Meaning (continued)	Chapters
$C_{\text{contact}}(z), C_{i,t}^{\text{contact}}$	Smooth gait-contact weighting function and its evaluated value.	7
C_t^{run}	Nonnegative per-interval resource cost, such as time or energy, in the planning utility.	5
C_i^{safe}	Physical constraint residual depending on a predicted state trajectory and candidate actions. Feasibility is $C_i^{\text{safe}} \leq 0$.	11
$C_{\max}$	Worst-case computation time.	9
C_{valid}	Cost per valid success. Semantic subscripts identify constituent costs.	9
c, c_t	Scene configuration or task conditions.	4, 5
c_{light}	Speed of light under the stated propagation convention.	3
$\mathbf{c}_{\text{rgb}}, i_{\text{rgb}}$	Radiance-field color vector and RGB channel index, respectively.	2, 3
$c_{\text{acq}}(x)$	Data-acquisition cost.	4
c_{lim}	Fitted limiting length tolerance in precision scaling. The fitted regime has $P_{\text{tol}} > c_{\text{lim}}$.	4
c_t^{constr}	Execution-constraint context available at time t.	8
c_{cont}	Container object in the cabinet-placement example.	8
$\mathbf{c}_t, \mathbf{c}_t^{\text{beh}}$	Motor-command vector and its behavior-command component.	7
c_k^{ctx}	Cached slow-context features.	6
$c_j, c_t^{\text{safety}}, c_t^{\text{energy}}$	Regularization, safety, or energy cost terms under their local objectives.	6, 7
$\mathbf{c}_t^{\text{root}}$	Horizontal root translation defining a local motion-evaluation origin.	7
$\mathbf{c}_{\text{G}}$	Center-of-mass position.	7
$\mathbf{d}, \mathbf{d}_i$	Viewing direction or LiDAR unit-ray direction.	2, 3
$d_{\text{node}}, d_{\text{edge}}, d_{\text{msg}}$	Positive integer dimensions of graph-node, graph-edge, and message vectors, respectively. These dimensions are not required to be equal.	2
$d_{\text{state}}, d_{\text{noise}}$	Positive integer state and Wiener-noise dimensions in neural differential equations. These dimensions are not required to be equal.	2
$d_{\text{task},k}$	Positive integer dimension of task k's acceleration residual in WBC.	7
$d_{\text{stereo}}(\mathbf{u}_{\text{pix}})$	Stereo disparity at a pixel-coordinate vector.	3
$c_{\text{x}}, c_{\text{y}}$	Horizontal and vertical principal-point coordinates in the calibrated camera image.	3
d_{delay}	Control delay measured in discrete steps.	6
d_{feat}, d_k	Feature-distance function and indexed squared tracking discrepancy, respectively.	5, 7
$d_{\text{R}}(\mathbf{R}_1, \mathbf{R}_2)$	Geodesic angular distance between two rotation matrices, used for orientation-tracking error.	7
i_{src}	Training-data-source index.	5
$d, d_{\text{D}}, d_{\text{r}}$	Feature, action, or target dimensionality.	5, 6
d_{t+1}^{skill}	Skill-selection or execution-supervision decision.	8
d_k^{detect}	Failure-detection probability.	9
d_t^{stance}	Gait stance duty factor, with $0 < d_t^{\text{stance}} < 1$.	7
d_{total}	Perception-to-actuation delay budget with explicitly defined stage boundaries.	7
N_{data}	Training-data count, distinct from a dataset object $\mathcal{D}$.	4
N_{src}	Number of training-data sources.	5

Continued on the next page

Symbol	Meaning (continued)	Chapters
$D_t, D_{t,i}$	Stacked clean-latent change targets and the target for future slice i, respectively. Each slice retains the declared latent representation. Here i does not index an individual scalar coordinate.	5
T_{deadline}	Exclusive deadline for the timing path or periodic local update defined in context. The remote closed-loop model requires $T_{\text{e2e}} < T_{\text{deadline}}$; the local update budget must also finish strictly before its stated deadline. The values are specified for each timing model.	9
D_{KL}	Kullback-Leibler divergence.	5
$\mathbf{E}$	Disturbance-input matrix in a state-space model.	2
E_ϕ	Encoder. Similarly named encoders in different chapters are distinct learned functions.	5
$E_\lambda(\mathbf{u}_{\text{pix}}, \tau)$	Spectral irradiance.	3
$\mathcal{E}, E_{\text{phys}}, E_{\text{info}}$	Graph edge sets, including physical and information edges.	2, 8
E	End-effector coordinate-frame label.	8
$e_{\text{track}}(t), \mathbf{e}, e_{\text{ss}}, e_{\text{contact}}$	Scalar tracking-error signal, vector tracking or pose error, steady-state error, and contact-direction position error, respectively.	2, 7, 9
$e, e^\star$	Embodiment identifier and target embodiment.	6
e_t^{event}	Recorded trajectory event.	4
e_i^{obs}	Binary observed event outcome used in the Brier score.	5
$\mathbf{e}_{ij}^{\text{edge}}$	Graph-edge feature vector in $\mathbb{R}^{d_{\text{edge}}}$, with node indices i, j.	2
e_t^{enc}	Encoded observation features in $e_t^{\text{enc}} = f_\theta(o_t)$, an abstract representation unless vector coordinates are explicitly specified.	2
$f, f_\theta, f_{\text{c}}, f_{\text{d}}$	Locally defined mappings, dynamics, or encoders. The subscripts c and d distinguish continuous- and discrete-time dynamics; θ denotes model parameters.	2, 6, 11
$\mathbf{f}_\theta^{\text{drift}}(\mathbf{x}, t)$	Vector-valued drift mapping from $\mathbb{R}^{d_{\text{state}}} \times [0, \infty)$ to $\mathbb{R}^{d_{\text{state}}}$ in neural differential equations.	2
$\mathbf{f}_i, \mathbf{f}_{\text{n}}, \mathbf{f}_{\text{t}}, f_{i,\text{n}}, \mathbf{f}_{i,\text{t}}, \mathbf{f}_{\mathcal{K}_{\text{hand}}}$	Contact-force vectors and their normal or tangential vector components. In the point-contact decomposition, $f_{i,\text{n}}$ is the scalar normal projection and $\mathbf{f}_{i,\text{t}}$ is the tangential force vector. $\mathbf{f}_{\mathcal{K}_{\text{hand}}}$ stacks the active finger-object contact forces.	2, 7
f_j^{eq}	Physical equality-constraint residual, with feasibility $f_j^{\text{eq}} = 0$.	5
f_t^{gait}	Gait frequency.	7
$\mathbf{f}_{t,i_{\text{tok}}}^{+,\ell_{\text{T}}}$	Aligned target feature at time, position, and target-layer indices.	5
$f_{\text{x}}, f_{\text{y}}$	Camera focal lengths in image-axis units.	3
F, F_θ	State-transition or neural prediction mapping.	2
F_θ^{NeRF}	Radiance-field mapping to color and density.	2, 3
F_{ISP}	Image-signal-processing mapping.	3
F_t^{future}	Future-observation sequence in joint world-action modeling.	5
F_{CDF}	Predictive cumulative distribution used in CRPS.	5
F_{contact}	Scalar contact force.	9
F_θ^{ctx}	Slow-context module in a multirate policy.	6
$F_{\text{plan}}, c_t^{\text{plan}}$	Autoregressive motion generator and its planning condition, respectively. The condition may include execution feedback and is not assumed to be a motor-command vector.	7
$G, \mathcal{G}_{\text{phys}}, \mathcal{G}_{\text{info}}$	Graph structures, including physical and information graphs.	2, 8
G_{tac}	Optical-tactile mapping from an image to estimated contact state.	3
G_ψ	Fast action-decoding module in a multirate policy.	6
$\mathbf{G}$	Grasp matrix mapping contact forces to an object wrench.	7

Continued on the next page

Symbol	Meaning (continued)	Chapters
g, g_t	Task goal or command condition. Chapters 4, 5, and 8 use g for a goal. Chapter 6 may use g_t for a subgoal. In Chapter 7, g_t contains behavior or reference-derived motor commands.	4, 5, 6, 7, 8
$\mathbf{g}(\mathbf{q})$	Generalized gravitational force in the robot dynamics.	2, 7
$\mathbf{g}_{\text{acc}}$	Gravitational acceleration expressed in world coordinates in the centroidal momentum balance, measured in $\mathrm{m\,s^{-2}}$.	7
$g_{\text{in}}(x)$	Input vector field in the locally defined control-affine model.	2
$\mathbf{G}_\theta^{\text{diff}}(\mathbf{x}, t)$	Diffusion mapping with values in $\mathbb{R}^{d_{\text{state}} \times d_{\text{noise}}}$, multiplying the Wiener increment.	2
g_{group}	A group element acting on an input or output.	2
g_i^{taxel}	Taxel transduction or calibration function.	3
g_j^{ineq}	Physical inequality-constraint residual, with feasibility $g_j^{\text{ineq}} \leq 0$.	5
g_{ψ_e}	Embodiment-specific action decoder.	6
$g(x_T), g_f, g_v$	Terminal success predicate or contact-related reward gates, according to context.	7, 8
g_ϕ	JEPA feature-prediction mapping.	2
$H, H_{\text{a}}, H_{\text{o}}$	Prediction horizon in steps. The world-model discussion also uses H_{a} and H_{o} for action and observation horizons.	5, 6, 7, 11
H_∞	The robust-control or function-space designation, distinct from a prediction horizon.	2
H_θ	Latent-state update mapping.	2
$\mathbf{H}_t$	Heading rotation used in local motion evaluation.	7
$\mathbf{h}_i^{\text{node},t}$	Graph-node feature vector in $\mathbb{R}^{d_{\text{node}}}$, distinct from observation-action history h_t and centroidal momentum $\mathbf{h}_{\text{G}}$.	2
h_t	Observation-action history, including locally specified robot-state inputs.	2, 5, 6, 7
$\mathbf{H}_t^{\text{VLM}}$	Vision-language backbone token-feature matrix supplied to the action expert, distinct from the interaction history h_t.	6
$h_{\text{obs}}(x)$	Observation function.	2
$h_{\text{safe}}(x)$	Control-barrier function defining a safe set.	2
$\mathbf{h}_{\text{G}}$	Total centroidal momentum, comprising linear and angular momentum.	7
$\mathbf{I}, \mathbf{I}_p$	Identity matrix, with order specified by context or subscript.	2, 5
$I_t, I^{\text{RGB}}, I_{\text{tac}}$	Image observations, including color and tactile images.	3
I_t^{ext}	Indicator of upper-body external-controller activation.	7
$I_{\text{m}}, I_{\text{refl}}$	Motor inertia and reflected inertia, with $I_{\text{refl}} = N_{\text{gear}}^2 I_{\text{m}}$ under the stated transmission convention.	9
i	Local sample, node, contact, or constraint index.	2, 3
$\mathbf{i}, i_{\text{q}}$	Motor-current vector and quadrature-axis current.	3, 9
$i : j$	Index interval or segment endpoints.	5
$J_{\text{cost}}, J(\pi), J_{\text{real}}$	LQR control cost, expected policy return, and measured real-task utility, respectively. Cost is minimized. Return and utility use their stated maximization convention.	2, 4, 5, 6
$\mathbf{J}, \mathbf{J}_i, \mathbf{J}_{\mathcal{K}_{\text{contact}}}, \mathbf{J}_{w,i}, \mathbf{J}_{\text{ext}}, \mathbf{J}_{\text{h},\mathcal{K}_{\text{hand}}}$	Robot Jacobians under the stated task, frame, and reference-point convention. $\mathbf{J}_{\mathcal{K}_{\text{contact}}}$ stacks active point-contact Jacobians, $\mathbf{J}_{w,i}$ is the six-dimensional foot-wrench Jacobian used in WBC, and $\mathbf{J}_{\text{h},\mathcal{K}_{\text{hand}}}$ maps finger joint rates to velocities at the active hand-object contacts. In Chapter 2, $\mathbf{J} \in \mathbb{R}^{6 \times n}$ matches the reference point and coordinates of $\mathbf{w}_{\text{ext}}$.	2, 7
$J_{\max}$	Worst-case scheduling jitter.	9
$J_{\text{eq}}, J_{\text{ineq}}$	Numbers of equality and inequality constraints.	5
j	Local diffusion-step, constraint, target, or source index.	5

Continued on the next page

Symbol	Meaning (continued)	Chapters
j	Imaginary unit in frequency-domain control expressions.	2
k	Local timestep, prediction offset, iteration, or task-step index.	4, 5
$k(t)$	Index of the latest available slow-context update.	6
k_{t}	Motor torque constant.	9
$k_{\mathrm{r}}, k_{\mathrm{e}}, k_{\mathrm{eff}}$	Robot, environment, and effective stiffness.	9
$\mathbf{k}_{\mathrm{G}}$	Centroidal angular momentum.	7
$K_{\mathrm{P}}, K_{\mathrm{I}}, K_{\mathrm{D}}, \mathbf{K}$	Scalar or matrix feedback gains according to the specified controller.	2
$\mathbf{K}_{\mathrm{cam}}$	Camera intrinsic matrix.	3
$H_{\mathrm{roll}}, K_b, N_{\mathrm{slice}}, N_{\mathrm{acttok}}, N_{\mathrm{exec}}$	Deterministic model-rollout horizon, candidate-branch interval count, future-latent slice count, action-token count, and number of actions executed before replanning, respectively.	2, 5, 6
N_{stage}	Number of stages in the task-success product with at most one modeled recovery attempt per stage.	9
K_{civ}	Kardashev index, $K_{\mathrm{civ}} = (\log_{10}(P_{\mathrm{power}}/\mathrm{W}) - 6)/10$.	12
K_{eval}	Number of evaluation criteria or scoring dimensions.	11
L_{mech}	Mechanical Lagrangian, $L_{\mathrm{mech}} = E_{\mathrm{kin}} - E_{\mathrm{pot}}$.	2
$L, L(s)$	Number of observations in the finite interaction history in Chapter 7; open-loop transfer function in Chapter 2, respectively. The argument s is the Laplace-domain variable.	2, 7
$L_{\mathrm{scale}}, L_{\mathrm{floor}}$	Validation loss and irreducible loss floor in a scaling relationship.	4
L_{e2e}	Latency until the command becomes available at the control interface. Actuator response is counted separately.	6
L_{crit}	Critical dependency-chain length or duration in parallel scheduling.	8
$L_f h_{\mathrm{safe}}, L_{g_{\mathrm{in}}} h_{\mathrm{safe}}$	Lie derivatives of the safety-barrier function along the drift and input vector fields.	2
L_1, L_2	In the action-regression discussion, L_1 denotes an elementwise absolute-error loss. In robust control, L_2 denotes the finite-energy signal space. Its norm is listed separately.	2, 6
ℓ_{lang}	Language task instruction.	6
$\ell(\mathbf{x}, \mathbf{u}), \ell_j, \ell(\widehat{y}, y)$	MPC stage cost in Chapters 2 and 7; per-example objective loss or prediction loss in Chapter 5. Arguments follow the local definitions.	2, 5, 7
$\ell_t^{\mathrm{clear}}, \ell_{i,t}^{\mathrm{target}}, \ell_{i,t}^{\mathrm{foot}}$	Maximum foot clearance, target swing height, or realized foot height.	7
$\ell, \ell_{\mathrm{s}}, \ell_{\mathrm{T}}$	Network-layer or command-mask component index.	5, 7
m_t^{mem}	Memory state or retained summary of prior interaction.	8
$m_j(\xi), \mathbf{m}_t, m_t^{\ell}$	Supervision-validity mask, command-mask vector, and its scalar components.	5, 7
m_{EMA}	Exponential-moving-average mixing coefficient.	5
m	Control-input dimension in the linear state-space model in Chapter 2; ensemble-member index in Chapter 5.	2, 5
$m_t^{\mathrm{multi}}, m_t^{\mathrm{multi}\prime}$	Additional multimodal inputs to prefix and suffix processing, respectively.	6
N_{constr}	Number of constraints in a joint safety event.	11
m_{robot}	Robot mass.	7
m_{fit}	Positive coefficient in the empirical precision fit $\log N_{\mathrm{demo}} = m_{\mathrm{fit}}/(P_{\mathrm{tol}} - c_{\mathrm{lim}}) + n_{\mathrm{fit}}$.	4
$\mathbf{M}(\mathbf{q})$	Rigid-body mass or inertia matrix.	2, 7
M	Number of models in an uncertainty ensemble.	5
N_{par}	Number of model parameters in a scaling analysis.	4

Continued on the next page

Symbol	Meaning (continued)	Chapters
$N_{\text{demo}}, N_{\text{iter}}, N_{\text{tok}}, N_{\text{step}}, N_{\text{eval}}$	Demonstration count or adaptation budget, DAgger iteration count, sequence-token count, state-transition step count, and evaluation-trial count, respectively. Each count refers to the locally declared dataset or protocol.	4, 5, 6, 7, 11
N_{agent}	Number of cooperating agents or parallel workers.	8
N_{gear}	Transmission reduction ratio under the stated input/output convention.	9
$\mathcal{N}(i)$	Neighbor set of graph node i.	2
N_{valid}	Number of valid successful trials.	9
$N_{\text{rec}}, N^{\star}_{\text{demo}}(s_{\min})$	Number of recovery-eligible evaluation trials, and the minimum tested demonstration budget meeting the prescribed adaptation-success threshold, respectively.	6, 11
n, n_a	Locally defined dimensions. In Chapter 2, n is the state dimension in the linear state-space model and the joint-coordinate dimension in the robot model. In Chapter 7, n counts actuated joints, so generalized velocity has dimension $6+n$; n_a is the action dimension in mirror-consistency regularization.	2, 7
$\mathbf{n}_i$	Unit contact-surface normal.	7
n_{fit}	Intercept in the empirical precision fit $\log N_{\text{demo}} = m_{\text{fit}}/(P_{\text{tol}} - c_{\text{lim}}) + n_{\text{fit}}$.	4
$O_{\psi_{\text{render}}}, O_{\omega_{\text{obs}}}, O_e$	Observation model or likelihood.	4, 5, 6
O_t^+	Future-observation sequence in the manipulation-model discussion.	6
o_t	Observation, with sensor content defined locally.	2, 4, 5, 6, 7, 8
$\mathbf{o}$	Sensor ranging origin.	3
$p, p_\theta, p_{\xi_{\text{sim}}}, p_{\eta_{\text{sim}}}$	Probability density or mass function under the specified measure, or a scalar event probability where explicitly defined. Parameters identify the local probability model.	2, 4, 5, 6, 7, 8, 9
p_i, p_k	Predicted event or step-success probabilities in the stated evaluation or reliability protocol.	5, 9
$\mathbf{p}, \mathbf{p}_i$	Spatial position vector. Frame labels specify its coordinates.	2, 3, 6
p_{press}	Normal contact pressure.	3
i_{tok}	Spatial-token position index.	5
d_{reg}	Regressor dimension or identity-matrix order in persistent-excitation analysis.	2
p_{quant}	Latency-quantile level.	6
$\mathbf{p}_{\text{G}}$	Centroidal linear momentum, $\mathbf{p}_{\text{G}} = m_{\text{robot}}\mathbf{v}_{\text{G}}$, measured in $\mathrm{kg\,m\,s^{-1}}$.	7
$\widehat{p}_{\text{aut}}, \widehat{p}_{\text{rec}}, p_{\text{upper}}$	Estimated autonomous success, estimated recovery success, or an upper bound on incident probability.	11
$P^{\text{tr}}, P^{\text{tr}}_{\xi_{\text{sim}}}, P^{\text{tr}}_e, P(\cdot \mid s, \mathbf{a})$	State-transition probability kernel. Chapter 7 writes the motion-tracking kernel as $P(\cdot \mid s, \mathbf{a})$. Its conditional arguments distinguish it from the plant transfer function $P(s)$.	2, 4, 6, 7
$P(s)$	Plant transfer function.	2
$P_\theta, P^{\text{q}}_\theta$	Feature predictor or future-token readout. The upright superscript q labels the predictive-token readout and is distinct from a query-type index.	5
P_{tol}	Required length tolerance in precision scaling. Smaller values impose tighter requirements.	4
P_{power}	Total power used in civilization-scale analysis.	12
P_{task}	Long-task success probability under the stated detection and recovery model.	9

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\mathbf{Q}, \mathbf{Q}_v, \mathbf{Q}_{\mathrm{R}}, \mathbf{Q}_a$	Explicitly matrix-valued state or tracking weights in LQR and NMPC.	2, 7
Q_ω, Q_c	Nonnegative scalar NMPC weights for angular-velocity and CoM-height errors, respectively. The definition is $\|e\|_Q^2 = Qe^2$ for a scalar error and either scalar weight.	7
$Q_{i_{\mathrm{rgb}}}^{\mathrm{quant}}$	Sensor-channel quantization function.	3
Q_ψ	World-model task readout.	5
$Q^\pi(h, a)$	Action-value function of policy π.	2
$\mathbf{q}_t$	Proprioceptive measurement vector at time t in the manipulation-model interface. Its contents are specified by the embodiment and are not assumed to be configuration coordinates alone.	6
$\mathbf{q}, \mathbf{q}_j$	Robot configuration and the joint-coordinate subvector, respectively. Configuration derivatives and generalized velocities need not have identical representations.	2, 3, 6, 7
q_{query}	Query-type index.	5
q_ϕ	Variational posterior or approximate latent-state inference density.	5
$q_t, \mathbf{q}_{t,i_{\mathrm{tok}}}^{\ell_{\mathrm{s}}}$	Predicted tokens or auxiliary feature readouts, according to the local model.	5
q	Motor quadrature-axis label.	9
$\mathbf{q}_{\mathrm{B}}, \mathbf{q}_{\mathrm{A}}$	Base and arm configuration vectors.	8
$q_{\mathrm{interp}}(u_{\mathrm{interp}})$	Quintic smooth interpolation, $q_{\mathrm{interp}}(u_{\mathrm{interp}}) = 10u_{\mathrm{interp}}^3 - 15u_{\mathrm{interp}}^4 + 6u_{\mathrm{interp}}^5$.	7
$\mathbf{R}$	Rotation matrix, with frame labels specifying its mapping direction.	2, 3, 7
$\mathbf{R}_{\mathrm{u}}$	Control-input weighting matrix in LQR and the NMPC objective, positive definite in the stated LQR formulation.	2, 7
$R, R_{\ell_{\mathrm{lang}}, e}$	MDP reward function or embodiment-specific reward.	2, 6
$R_{\mathrm{T}}, R_\theta^{\mathrm{S}}$	Frozen teacher and learned student geometry/motion readouts.	5
R_{gen}	Action-generation rate, H/L_{model}.	6
Y_j^{rec}	Binary recovery-trial outcome.	11
r_t	Realized reward.	2, 5, 6, 7
$r(t)$	Control reference input.	2
r_i	Measured ray range, distinct from optical-axis depth away from the optical axis.	3
r	Correlation coefficient.	4
r_t^{reason}	Reasoning tokens or text.	6
$\mathbf{r}_t^{\mathrm{T}}, \mathbf{r}_t^{\mathrm{S}}$	Teacher and student geometry/motion descriptors.	5
r_k^{rec}	Failure-recovery success probability.	9
$\mathcal{S}_{\mathrm{surf}}$	Implicit geometric surface.	2
$S(s)$	Closed-loop sensitivity function.	2
$\widehat{S}$	Predicted physical-state sequence.	5
$\mathbf{S}$	Floating-base actuator-selection matrix.	7
$S_{i_{\mathrm{rgb}}}(\lambda)$	Sensor-channel spectral response.	3
$\mathsf{S}, \mathsf{S}_m, \mathsf{S}_0$	Sensor-frame labels.	3
S_{eval}	Optional weighted aggregate evaluation score on a 0 to 100 scale.	11
$s_t, \hat{s}_t$	Physical or task state and its estimate. Vector notation applies when coordinates are explicitly specified.	2, 4, 5, 6, 7
s	Complex Laplace-domain frequency variable.	2
$s_1^{\mathrm{shear}}, s_2^{\mathrm{shear}}$	Tangential contact-stress components.	3

Continued on the next page

Symbol	Meaning (continued)	Chapters
s_k^{eval}	Normalized score for one evaluation criterion.	11
$s_{\text{F}}, s_{\text{V}}$	Positive scale parameters in contact penalties.	7
s_t^{ref}	Reference motion state.	7
$s_{\min}$	Minimum acceptable adaptation success rate.	6
$\mathrm{SO}(3)$	Three-dimensional rotation group.	2, 7
$\mathrm{SE}(3)$	Three-dimensional rigid-transformation group.	2, 3, 7
$se(3)$	Lie algebra of $\mathrm{SE}(3)$.	2
$t, t_{\text{obs}}, t_{\text{apply}}$	Time, discrete timestep, or explicitly defined observation and actuation timestamps. Event boundaries and clock domains are stated locally.	2, 5, 6, 9
$\mathbf{t}$	Translation vector.	2, 3
t	Tangential-direction label on a contact-force component.	2
t_m, t_0	Local sensor-clock reading and corresponding reference-clock reading in the affine clock-mapping model.	3
$\mathbf{T}, {}^{\text{F}}\mathbf{T}_{\text{H}}$	Rigid-pose or homogeneous-transform matrix mapping coordinates from frame H to frame F.	2, 3, 6, 8
E_{kin}	Kinetic energy.	2
$T_{\text{temp}}(t)$	Temperature.	3
$T(s), \mathbf{T}_{zw}$	Complementary-sensitivity or disturbance-to-performance transfer function.	2
T, T_θ	State-transition probability kernel in a world model.	5
$T, T_i, T_c, T_{\text{task}}$	Episode duration or step count as defined locally. In Chapter 7 mean tracking errors, T counts actually executed evaluated frames, and T_i is the full reference length of sequence i in frames. T_c and T_{task} denote control period and task-completion time.	4, 7, 9
T_τ	Actuator torque-response time constant.	7
T_0	Positive integration-window duration in the persistent-excitation condition.	2
T_{e2e}	Latency from sensing start to actuation onset for one observation-action path in the serial model. Stage intervals cover this path without gaps or overlaps and include queueing time.	9
$u_t, \mathbf{u}_t$	Control input. Vector notation follows the local control space.	2, 8
$u_{\text{pix}}, \mathbf{u}_{\text{pix}}, \tilde{\mathbf{u}}_{\text{pix}}$	Pixel coordinate, pixel-coordinate vector, or homogeneous image coordinates.	3
$u_\theta^{\text{field}}(x,t)$	Neural approximation to a partial-differential-equation solution field.	2
u	Dummy variable of integration or a scalar argument of the positive-part function.	5
u_{dev}	Upper bound on the increase in the expert's remaining cost after one deviating action in DAgger analysis.	4
u_t^{vid}	Future-video latent representation.	5
$u_{\text{interp}}, u_{\text{fold}}$	Normalized swing-interpolation or folded-phase parameter.	7
U	Utility of a candidate action plan.	5
U_t^+	Future-video latent sequence.	5
E_{pot}	Potential energy.	2
$V(x)$	Lyapunov candidate function.	2
V_{f}	MPC terminal cost.	2
$V_{\omega_{\text{critic}}}$	Learned value evaluator.	6
$\mathcal{V}, V_{\text{phys}}, V_{\text{comp}}$	Graph vertex sets.	2, 8
N_{view}	Number of input camera views.	6
$V^\pi(h)$	State- or history-value function of policy π.	2

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\mathbf{v}$	Generalized, base, or joint velocity under the specified coordinate convention.	7
$v^{\text{noise}}, \mathbf{v}^{\text{noise}}$	Measurement noise, with scalar or vector form defined by the observation model.	2
$\mathbf{v}_{\text{sens}}, \mathbf{v}_{\text{sens},0}$	Raw force/torque sensor signals and their offsets.	3
v_{pix}	Vertical pixel coordinate.	3
$\mathbf{v}_\theta^{\text{flow}}$	Learned flow-matching velocity field.	5, 6
v	Video-target label in a loss term.	5
$\mathbf{W}_k, \mathbf{W}_{\text{f}}, \mathbf{W}_\tau$	Symmetric positive-semidefinite WBC weights for task-acceleration, stacked foot-wrench, and joint-torque residuals, with dimensions $d_{\text{task},k} \times d_{\text{task},k}$, 12×12, and $n \times n$, respectively. The symbol n is the number of actuated joints.	7
W_k^{sim}	Simulator or world model at iteration k.	4
$\mathbf{W}_t^{\text{Wiener}}$	Standard d_{noise}-dimensional Wiener process in the stochastic state model, distinct from WBC weight matrices and simulator W_k^{sim}.	2
W_{work}	Total parallel-task work.	8
$\mathcal{W}$	World coordinate frame. Upright W is used as its frame label in coordinate expressions such as $\mathbf{p}_{\text{W}}$, not as a standalone physical variable.	3, 7, 8
W	Watt, the unit of power.	12
$w_t, \mathbf{w}_t$	Process disturbance or process noise.	2
$\mathbf{w}_{\text{S}}$	Six-dimensional wrench expressed in a sensor frame.	3
$\mathbf{w}_{\text{ext}}$	Six-dimensional external wrench, ordered as force then moment, at the stated reference point and in the same coordinates as its matching Jacobian or body dynamics. Force components are in N and moment components in N m.	2, 7
$\mathbf{w}_i, \mathbf{w}_{\text{L}}, \mathbf{w}_{\text{R}}$	Six-dimensional foot wrench $\mathbf{w}_i = (\mathbf{f}_i, \boldsymbol{\mu}_i)$, expressed in world coordinates, with its moment taken about the corresponding foot-frame origin. Labels L and R denote the left and right feet. Force components are in N and moment components in N m.	7
w_j, w_k, w_i	Scalar weights for losses, tracking, evaluation criteria, or retargeting.	5, 6, 7, 9, 11
$X_{\text{C}}, Y_{\text{C}}, Z_{\text{C}}$	Scalar Cartesian coordinates in the camera frame.	3
$X_{t:t+H}$	Predicted random state trajectory conditioned on the belief and candidate actions.	11
$\mathbf{x}_{\text{pos}}$	Spatial position vector.	2, 3
$x_t, \mathbf{x}_t$	Physical or task state. Bold type denotes an explicitly vector-valued state. The neural differential equations use $\mathbf{x}_t \in \mathbb{R}^{d_{\text{state}}}$. Its dimension need not equal the noise dimension.	2, 8
$x_i, \mathbf{x}, \mathbf{x}^{(j)}, \mathbf{x}^{\sigma_{\text{flow}}}$	Sequence token, clean continuous target, diffusion-corrupted target, and flow-interpolated target, respectively. The continuous targets retain the shape declared by the local model.	5
$x^\star$	Data-acquisition decision.	4
$\mathbf{x}_t^{\text{target}}$	Target command in masked motion imitation.	7
$y_t, \mathbf{y}_t, y^{q_{\text{query}}}$	Measurement, control output, or query-specific prediction target.	2, 3, 5
y	Recorded terminal physical outcome.	4
Y, Y'	Random outcomes or independent draws under the specified predictive distribution.	5
$Y_{\text{C}}^{\text{raw}}$	Raw mosaic image supplied to the image-signal-processing pipeline.	3
Y_i^{aut}	Binary autonomous-success trial outcome.	11

Continued on the next page

Symbol	Meaning (continued)	Chapters
$Z_{\text{depth}}(u_{\text{pix}}, v_{\text{pix}})$	Optical-axis depth image. Its values correspond to the camera-coordinate component Z_{C} under the stated calibration convention.	3
$z_t, \widehat{z}_t, \mathbf{z}_t, \hat{\mathbf{z}}_t$	Latent state and its prediction or estimate. Bold notation is used only when a vector representation is explicitly declared.	2, 5, 6, 7
$z_{\text{perf}}, \mathbf{z}_{\text{perf}}$	Performance output in robust control.	2
$\widehat{\mathbf{z}}_t^{\text{tac}}$	Estimated tactile contact state.	3
$z_{1:N_{\text{acttok}}}^{\text{act}}$	Autoregressive action-token sequence.	6
z_t^{cmd}	Body-height command.	7
$\mathbf{z}_{\text{WBC}}$	Whole-body-control optimization vector.	7
z	Dummy normalized-phase argument in a gait-contact function.	7
α_{safe}	Locally Lipschitz extended-class-$\mathcal{K}$ function in a barrier condition.	2
α_m^{clk}	Relative sensor-clock rate.	3
α_{scale}	Model-size exponent in a scaling relationship.	4
$\alpha_j, \bar{\alpha}_j$	Diffusion retention coefficient and its cumulative product.	5
α_j^{eq}	Nonnegative physical equality-residual weight.	5
α_{mix}	Mixing coefficient for autoregressive and flow-matching losses.	6
α_{act}	Scaling from normalized action to joint-position target.	7
β_m^{clk}	Sensor-clock offset.	3
β_{scale}	Data-size exponent in a scaling relationship.	4
β_{KL}	Variational KL-divergence weight. Changing it from one changes the ordinary ELBO objective.	5
β_j^{ineq}	Nonnegative inequality-constraint penalty weight.	5
β_k^{term}	Option termination condition or termination probability.	8
β_t^{pitch}	Body-pitch command.	7
χ_t	Waist-yaw command.	7
$\boldsymbol{\delta}(\mathbf{x}, t)$	Optical-tactile surface-deformation field.	3
δ	Positive position-error tolerance for whole-sequence motion success.	7
$\Delta t, \Delta t_i, \Delta t_t, \Delta_1, \Delta_2$	Sampling, propagation, control, or transition time intervals under their local event definitions.	3, 4, 5, 6, 7
ΔJ	Change in closed-loop performance or utility.	4, 5
Δ	Physical feedback delay producing phase lag $\omega\Delta$.	9
$\Delta_{\text{fwd}}(N_{\text{demo}}), \Delta_{\text{ret}}$	Forward-transfer gain and retained-capability change, measured as success-rate differences.	6
$\boldsymbol{\epsilon}_t, \varepsilon_{i_{\text{rgb}}}, \varepsilon_I, \boldsymbol{\varepsilon}_i$	Measurement noise: a vector in the state-observation model, scalar noise in a raw color channel or optical-tactile image value, and vector taxel-measurement noise, respectively.	2, 3
$\boldsymbol{\epsilon}, \boldsymbol{\epsilon}_\theta$	Gaussian corruption noise and predicted noise in generative models.	5
$\boldsymbol{\varepsilon}, \varepsilon_{ij}$	The $H \times d$ standard-Gaussian noise matrix used in action flow matching and its independent scalar entries.	6
ϵ_{clip}	PPO clipping half-width.	2
$\epsilon_{\text{BC}}, \epsilon_{\text{DAgger},N_{\text{iter}}}$	Behavior-cloning error under the expert distribution, and the best comparison-class average surrogate loss over the state distributions induced by the rollout policies across N_{iter} iterations, respectively.	4
ϵ_{sync}	Clock-synchronization error bound.	9
$\varepsilon_{\text{risk}}$	Permitted probability of violating at least one constraint in the joint safety event.	11

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\eta_{\mathrm{I}}(t)$	Integrated tracking error in PID control.	2
η_{sim}	Parameters of a distribution over simulator parameters.	4
η_e	Embodiment description, which may comprise calibration, an embedding, a structured prompt, or inferred information.	6
η_{pen}	Nonnegative violation-penalty coefficient in model-based planning.	5
η_{gear}	Transmission efficiency.	9
γ	Reward discount factor, with admissible values determined by the horizon and return definition.	2, 5, 6, 7
γ_{gain}	Disturbance-to-output gain bound in robust control.	2
γ_{cam}	Camera intrinsic skew parameter.	3
κ	Conditioning input to an autoregressive, diffusion, flow-matching, or probabilistic prediction model.	5
Λ_{vis}	Wavelength integration domain.	3
λ	Optical wavelength in Chapter 3. Under local definitions, it is a physical-loss weight in Chapter 2, resource-cost coefficient in Chapter 5, and a retargeting smoothness weight in Chapter 6.	2, 3, 5, 6
$\lambda_j, \lambda_{\mathrm{g}}, \lambda_{\mathrm{p}}, \lambda_{\mathrm{safe}}, \lambda_{\mathrm{e}}$	Locally defined objective weights, including goal, progress, safety, and energy weights.	2, 5, 6, 7
μ, μ_i	Friction coefficient.	2, 7
μ_{PE}	Positive lower bound in a persistent-excitation matrix inequality.	2
$\mu_\theta, \mu_m, \bar{\mu}$	Scalar predictive means or ensemble mean. Vector means use bold type.	5
$\boldsymbol{\mu}_\theta$	Action mean or deterministic policy output.	7
m_t^{meta}	Data-collection metadata.	6
$\boldsymbol{\mu}_i$	Moment component of a contact wrench.	7
$\boldsymbol{\nu}_{\mathcal{B}}, \boldsymbol{\nu}_{\mathrm{o}}$	Six-dimensional base or object twist.	7
ω	Physical angular frequency.	2, 9
$\boldsymbol{\omega}, \omega_{\mathrm{z}}^{\mathrm{cmd}}, \omega_{\mathrm{m}}, \omega_{\mathrm{j}}$	Angular-velocity vector, commanded base-frame angular-velocity z component, or scalar motor/joint angular velocity.	3, 7, 9
$\omega_{\mathrm{obs}}, \omega_{\mathrm{critic}}$	Observation-model or critic parameters.	5, 6
Φ_{term}	NMPC terminal cost.	7
Φ	Standard normal cumulative distribution function in gait-contact smoothing.	7
$\phi, \bar{\phi}$	Encoder or variational-model parameters and moving-average target parameters.	5
$\boldsymbol{\phi}_{\mathrm{PE}}(t)$	Persistent-excitation regressor.	2
ϕ_{node}	Graph node-update mapping $\mathbb{R}^{d_{\mathrm{node}}} \times \mathbb{R}^{d_{\mathrm{msg}}} \to \mathbb{R}^{d_{\mathrm{node}}}$.	2
$\phi_i, \bar{\phi}_i$	Gait phase and its remapped or normalized form.	7
$\mathbf{\Pi}$	Algebraic Riccati solution matrix in continuous-time LQR.	2
π, π_θ, π_k	Policy or conditional action distribution. An iteration subscript identifies a learned version.	2, 4, 5, 6, 7, 8
π_k^{skill}	Internal policy of option or skill k.	8
ψ_{msg}	Graph message mapping $\mathbb{R}^{d_{\mathrm{node}}} \times \mathbb{R}^{d_{\mathrm{node}}} \times \mathbb{R}^{d_{\mathrm{edge}}} \to \mathbb{R}^{d_{\mathrm{msg}}}$.	2
ψ_{render}	Simulator observation or rendering parameters.	4
ψ, ψ_e	Task-readout or action-decoder parameters.	5, 6
ψ_t^{phase}	Left-right leg phase offset.	7
ψ_t^{head}	Root heading angle used in local motion evaluation.	7
$\rho_{i_{\mathrm{src}}}$	Nonnegative data-source mixture weight, with weights summing to one.	5
$\rho_t^{\mathrm{PPO}}(\theta)$	New-to-old policy probability ratio in PPO.	2
ρ_e	Embodiment-specific observation-encoder parameters.	6

Continued on the next page

Symbol	Meaning (continued)	Chapters
ρ_k^{init}	Initial-state distribution of a benchmark protocol.	11
σ_{rad}	Volumetric density in a radiance field.	2, 3
$\boldsymbol{\sigma}_{\text{c}}, \overline{\sigma}_i$	Contact-stress vector and its taxel-area average.	3
$\sigma, \bar{\sigma}$	Singular value and largest-singular-value operator.	2
σ_{flow}	Flow interpolation time increasing from clean data to noise.	5
$\sigma_\theta, \sigma_m^2$	Predictive standard deviation or variance.	5
σ_ϕ	Positive, dimensionless smoothing width in the gait-contact function built from the standard normal cumulative distribution.	7
σ_k^{track}	Positive motion-tracking reward scale.	7
σ_k^{opt}	Option tuple comprising initiation set, internal policy, and termination condition.	8
σ_{contact}	Contact mode or support schedule.	7
$\boldsymbol{\tau}, \tau_{\text{j}}, \boldsymbol{\tau}_{\text{S}}$	Joint-torque vector, scalar joint-output torque in the transmission model, and moment vector in a sensor-frame wrench, respectively.	2, 3, 7, 9
$\boldsymbol{\tau}^{\text{cmd}}, \boldsymbol{\tau}^{\text{ff}}$	Commanded joint-torque vector and feedforward joint-torque vector, respectively. The commanded torque combines feedforward and local feedback terms.	7
τ_{ep}	Recorded interaction episode or trajectory.	2, 4, 6, 7
τ	Dummy time variable in a time integral. Distinct from an interaction trajectory τ_{ep}.	2, 3, 7
τ_{flow}	Flow interpolation time increasing from noise to clean data.	6
τ_{stride}	Future-video feature-sampling stride.	5
θ	Trainable neural-model, policy, or dynamics parameters.	2, 4, 5, 6, 7, 9, 11
$\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}$	Unknown adaptive-control parameters and their estimates.	2
$\boldsymbol{\theta}_{\text{tac}}$	Physical or geometric parameters of an optical-tactile model.	3
Θ	Complete set of model or policy parameter groups under the local formulation.	5, 6
$\xi_{\text{sim}}, \boldsymbol{\xi}_{\text{sim}}$	Physical or numerical simulator parameters under the stated parameterization.	4, 7
ξ_{sample}	Joint-training record or sample.	5
ζ_t^{exec}	Execution-supervision record of skill state, verified subgoals, and constraint history.	8
ζ_{sample}	Weighted training sample in the hardware-software discussion.	9
ζ_{traj}	Sampled interaction trajectory in motion-tracking reinforcement learning.	7
$\mathcal{A}, \mathcal{A}_e$	Policy action space or embodiment-specific action space.	2, 6
$\mathcal{A}_H$	Declared set of input or action sequences over horizon H.	5
$\mathcal{A}_k^{\text{acq}}$	Data-acquisition plan at iteration k.	4
$\mathcal{B}_k^{\text{data}}$	Batch of newly collected physical interaction data.	4
$\mathcal{B}_k^{\text{bench}}$	Task-scenario-embodiment benchmark-protocol tuple.	11
$\mathcal{C}_{\text{safe}}$	Barrier-defined safe state set.	2
$\mathcal{C}_{\text{exec}}$	Execution-constraint set in a task specification.	8
$\mathcal{C}_{\text{cand}}$	Set of candidate world-model rollouts.	5
$\mathcal{C}_k^{\text{ctrl}}$	Benchmark controller and protection configuration.	11
$\mathcal{D}, \mathcal{D}_{\text{r}}, \mathcal{D}_e, \mathcal{D}_{i_{\text{src}}}$	Dataset or empirical distribution with locally defined source, task, or embodiment subscripts.	4, 5, 6, 7, 9
$\mathcal{D}_{\text{Lyap}}$	Domain of a Lyapunov analysis.	2
$\mathcal{E}_{\text{eval}}$	Isolated held-out evaluation set in a data-improvement loop.	4

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\mathcal{E}_{\text{emb}}, \mathcal{E}_{\text{emb}}^{\text{tgt}}, \mathcal{E}_{\text{ret}}$	Embodiment-identifier set, target-embodiment set used in transfer evaluation, and finite nonempty set of previously supported embodiments used for retention evaluation, respectively.	6
$\mathcal{F}_{\text{opt}}$	Optical-tactile image-formation function.	3
$\mathcal{F}_h, \mathcal{F}_g, \mathcal{F}_a$	Mirror transformations of history, goals, and actions.	7
$\mathcal{G}_{\text{goal}}$	Permitted goal or outcome set in a task specification.	8
$\mathcal{H}_k$	Benchmark hardware and calibration configuration.	11
$\mathcal{I}_k^{\text{init}}$	Initiation set of option k.	8
$\mathcal{I}_{\text{track}}$	Tracking-feature or reward-term index set.	7
$\mathcal{I}_k^{\text{io}}$	Benchmark observation-action interface.	11
$\mathcal{J}$	Joint-training loss-term or supervision-target index set.	5
$\mathcal{K}$	Class of comparison functions used in stability or barrier conditions.	2
$\mathcal{K}_{\text{contact}}, \mathcal{K}_{\text{hand}}$	Active point-contact index set in the floating-base body model, and active finger-object contact index set in the hand-object model, respectively.	7
$\mathcal{K}_e^{\text{eval}}$	Evaluation-task set for embodiment e.	6
$\mathcal{L}$	Training objective, with subscripts specifying its data, dynamics, or representation terms.	2, 5, 6, 7, 9
$\mathcal{L}_k^{\log}$	Traceable evidence record in a benchmark protocol.	11
$\mathcal{L}_{\mathbf{m}}^{\text{links}}$	Selected-link or keypoint set induced by a command mask.	7
$\mathcal{L}_{\text{sym}}$	Mirror-consistency training regularizer.	7
$\mathcal{L}_{\text{change}}$	Clean-latent change-supervision loss.	5
$\mathcal{L}_{\text{align}}$	Directional token-to-target feature-alignment loss.	5
$\mathcal{L}_{\text{geom}}$	Teacher geometry/motion-descriptor distillation loss.	5
$\mathcal{L}_{\text{WAM}}$	Combined video, action, change, alignment, and geometry objective.	5
$\mathcal{L}_{\text{phys}}, \mathcal{L}_{\text{physics}}$	Physical residual or constraint loss, with its local residual and sampling domain.	2, 5
$\mathcal{L}_{\text{Gauss}}$	Heteroscedastic Gaussian prediction loss.	5
$\mathcal{L}_{\text{multi}}$	Multi-horizon prediction loss.	5
$\mathcal{L}_{\text{pre}}$	Pretraining objective over available action, state, progress, outcome, and language supervision.	6
$\mathcal{L}_{\text{new}}, \mathcal{L}_{\text{retain}}, \mathcal{L}_{\text{reg}}$	New-task, capability-retention, and regularization terms in continual learning.	6
$\mathcal{L}_{\text{adapt}}, \Theta_{\text{upd}}$	Embodiment-adaptation objective and the selected parameter subset allowed to update.	6
$\mathcal{M}, \mathcal{M}_e$	MDP or POMDP environment tuple, with an embodiment subscript where applicable.	2, 6
$\mathcal{M}_{\text{mask}}$	Mask specifying target positions or withheld regions in representation training. Expectations average over its sampling procedure.	5
$\mathcal{M}_{\text{motion}}$	Reference motion clip or ordered reference-state sequence sampled from motion data.	7
$\mathcal{M}_k^{\text{meas}}$	Benchmark measurement and metric definitions.	11
$\mathcal{N}_{\text{PDE}}[u^{\text{field}}]$	Partial-differential-equation residual operator.	2
$\mathcal{N}$	Normal or Gaussian distribution.	5, 6
$\mathcal{N}_{\text{demo}} \subseteq \mathbb{N}_0$	Tested set of nonnegative integer demonstration budgets.	6
$\mathcal{O}_e$	Embodiment-specific observation space. Distinct from an observation likelihood O_e.	6
$\mathcal{P}_{\text{cloud}}$	Point cloud.	2

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\mathcal{P}_{\text{pref}}$	Preferences among acceptable task solutions.	8
$\mathcal{Q}$	Prediction-query set.	5
$\mathcal{S}, \mathcal{S}_e$	Physical-state space of a sequential-decision model.	2, 6
$\mathcal{T}_{\text{task}}$	Task specification comprising goals, constraints, preferences, and verification.	8
$\mathcal{T}_{\text{WBC}}$	Whole-body-control task index set.	7
$\mathcal{U}$	Admissible continuous-control set.	2
$\text{Unif}[0,1]$	Uniform distribution on the unit interval.	5
$\mathcal{U}_k^{\text{budget}}$	Benchmark resource and operating budgets.	11
$\mathcal{V}_{t:t+H}^{\text{viol}}$	Event that at least one trajectory-level constraint is violated. Its indicator is not a violation count.	5
$\mathcal{V}_{\text{verify}}$	Task-outcome and constraint-verification specification.	8
$\mathcal{W}_i^{\text{wrench}}(\sigma_{\text{contact}})$	Admissible foot-wrench set under contact mode σ_{contact}. The swing-foot set contains only the zero wrench.	7
$\mathcal{X}_{\text{adm}}, \mathcal{X}_{\text{term}}$	Admissible state set and terminal feasible set in MPC.	2
$\mathcal{X}$	Physical-agent state space.	8
$'$	Transformed quantity or successor-state integration variable. Not necessarily a derivative.	2, 5
$^+$	Future-variable label.	5, 6
*	Optimal-solution or target-object marker, as specified by the optimization or task definition.	4, 6, 7
$^{(b)}, ^{(j)}$	Rollout or diffusion-step indices, not powers.	5
$[u]_+$	Positive part, $\max(u, 0)$.	5
$\lceil u \rceil$	Ceiling, the smallest integer not less than u.	6
$\bmod 1$	Wrapping of normalized phase into a unit interval.	7
${}^{\text{S}_0}\mathbf{T}_{\text{S}_m}$	Transformation from sensor frame S_m to reference frame S_0. m is a variable index.	3
$\mathbf{R}_{\text{CW}}, \mathbf{t}_{\text{CW}}$	Rotation and translation mapping world coordinates to camera coordinates.	3
$\mathbf{p}_{\text{C}}, \mathbf{p}_{\text{W}}$	Position coordinates expressed in camera and world frames.	3, 8
$^\top$	Transpose.	2, 3, 5, 6, 7
$\mathbf{0}$	Zero vector or zero block of the required dimensions.	2, 7
$\mathbf{1}\{\cdot\}$	Event indicator, equal to one when its condition holds and zero otherwise.	5
$\arg\min, \arg\max$	Minimizing or maximizing argument, respectively.	4
$\min, \max$	Minimum or maximum value, including branches of clipping functions.	2, 5, 9
$\cup, \cap, \in, \subseteq, \setminus$	Set union, intersection, membership, inclusion, and difference.	2, 4
$\mathbb{E}$	Expectation under the stated distribution.	2, 4, 5, 6, 7, 9
Var	Variance.	5
G-MPKPE	Mean position error in the world frame, measured in meters.	7
L-MPKPE	Mean position error after expressing positions relative to horizontal root translation and heading.	7
G-MPKRE	Mean world-frame orientation error measured by geodesic angle on $\text{SO}(3)$.	7
L-MPKRE	Mean orientation error in the local root-heading frame.	7
$\hat{\mathbf{x}}_{\text{now}}, \hat{x}_t, \widehat{y}_{t+k\mid t}$	Current MPC state estimate in Chapter 2, estimated joint state in Chapter 11, or predicted readout in Chapter 5, respectively. $t+k \mid t$ denotes prediction for $t+k$ made at t.	2, 5, 11
$\widehat{\mathbf{p}}_i^{\text{human}}$	Human reference point used in motion retargeting.	6
$\wedge, \forall$	Logical conjunction and universal quantification.	8

Continued on the next page

Symbol	Meaning (continued)	Chapters
$\mathbb{N}_0$	Nonnegative integers.	6
∇	Gradient with respect to the indicated state or parameter variables.	2
$\mathrm{d}/\mathrm{d}t, \dot{x}, \ddot{x}$	Time derivative, first time derivative, and second time derivative.	2, 3, 7
$\mathrm{d}\mathbf{W}_t^{\mathrm{Wiener}}$	Increment of the d_{noise}-dimensional Wiener process in the stochastic differential equation.	2
$\odot$	Elementwise product, including application of a command mask.	7
sg	Stop-gradient operation. It does not imply that teacher parameters never update.	5, 7
clip	Clipping to the explicitly specified interval.	2, 7
sat	Actuator-command saturation.	7
Collect	Physical interaction-data collection.	4
Curate	Data-quality assessment, filtering, and organization.	4
TrainAndCalibrate	Policy training and model calibration in the data-improvement loop.	4
Accept	Held-out closed-loop acceptance evaluation.	4
Update, Supervise	Belief update and execution supervision, respectively.	8
$\cos_{\mathrm{sim}}(\mathbf{u}, \mathbf{v})$	Cosine similarity of two feature vectors.	5
$\exp, \log, \log_{10}$	Exponential, locally specified logarithm, and base-ten logarithm.	2, 4, 5, 6, 7, 12
arccos, tr	Inverse cosine and matrix trace, used in rotation-angle evaluation.	7
sin, cos	Trigonometric sine and cosine of a scalar angle.	7
Inside, Supported, Released, Upright, Closed	Task predicates for placement and cabinet manipulation.	8
$\mathrm{Succ}(\delta)$	Fraction of trials whose maximum selected-link position error over the entire evaluation horizon is within δ.	7
Pr	Probability of an event.	11
$p(\cdot \mid \cdot)$	Conditional density, likelihood, or posterior under the stated measure and context.	2, 4, 5, 6, 7, 8
$\mathbb{R}, \mathbb{R}^n, \mathbb{R}^{m \times n}$	Real numbers, real coordinate vectors, and real matrices of the stated dimensions.	2, 3, 6, 7
$\rightarrow, \longrightarrow$	Direction of time, control, or process flow. Not an identity relation.	7, 8
$\leftrightarrow$	Mutual dependency in a system diagram, unless a logical equivalence is explicitly defined.	8
$\mathrm{RMSE}(k)$	Root mean squared prediction error at horizon k.	5
BS	Brier score for binary event predictions.	5
$\mathrm{CRPS}(F_{\mathrm{CDF}}, y)$	Continuous ranked probability score of predictive cumulative distribution F_{CDF} and scalar outcome y.	5
$\sim$	Sampling from or following the specified probability distribution.	2, 4, 5
$\propto$	Proportionality. In homogeneous-coordinate projection, it denotes equality up to a nonzero scale. It also expresses a posterior proportional to likelihood times prior.	3, 4
$\approx, \simeq$	Approximate equality under the local modeling assumptions.	5, 9
$\mathrm{SR}_{N_{\mathrm{demo}}}(e, k), \mathrm{SR}_{\mathrm{before}}, \mathrm{SR}_{\mathrm{after}}$	Adaptation or transfer success rates. N_{demo} is the adaptation-demonstration budget. Evaluation trials are counted by N_{eval}.	6
$\succeq, \succ$	Positive-semidefinite and positive-definite order for symmetric matrices.	2
$\leq, \geq$	Scalar or componentwise inequalities, as defined by the operands.	2, 5, 7
$\sum, \prod, \int, \sup, \lim$	Summation, product, integration, supremum, and limit.	2
$\tilde{\pi}$	Candidate policy in simulator-assisted data or policy improvement.	4

Continued on the next page

Symbol	Meaning (continued)	Chapters
Hz, ms	Hertz and millisecond.	6, 7, 9, 11
m, mm, s, kg	Meter, millimeter, second, and kilogram.	7
$\|\cdot\|_2$, $\|\cdot\|^2$	Euclidean norm and squared Euclidean error for finite-dimensional vectors. The robust-control paragraph uses the subscript 2 for time-signal norms, as listed below. Matrix norms require a separate local definition.	2
$\|\cdot\|_{\mathrm{F}}$	Frobenius norm of an explicitly matrix-valued action residual. Its square is the sum of squared scalar entries.	6
$\|\mathbf{T}_{zw}\|_\infty$	For a stable proper LTI transfer matrix, the supremum over frequency of its largest singular value. Equal to its induced L_2 gain under zero initial conditions.	2
$\|\mathbf{w}\|_2$, $\|\mathbf{z}_{\mathrm{perf}}\|_2$	Finite-energy time-signal norms in the robust-control gain bound. For example, $\|\mathbf{w}\|_2 = (\int_0^\infty \mathbf{w}(t)^\top \mathbf{w}(t)\,\mathrm{d}t)^{1/2}$.	2

References

J. Achiam, D. Held, A. Tamar, and P. Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 22–31, 2017.

E. Acome, S. K. Mitchell, T. G. Morrissey, M. B. Emmett, C. Benjamin, M. King, M. Radakovitz, and C. Keplinger. Hydraulically amplified self-healing electrostatic actuators with muscle-like performance. *Science*, 359(6371):61–65, 2018.

A. D. Adesiji, S. E. Ibitoye, R. M. Mahamood, O. A. Olayemi, P. O. Omoniyi, T.-C. Jen, and E. T. Akinlabi. Safety considerations in deployment of robotic systems – a systematic review. *Journal of Field Robotics*, 43(1):5–33, 2026.

K. E. Adolph and J. E. Hoch. Motor development: Embodied, embedded, enculturated, and enabling. *Annual Review of Psychology*, 70:141–164, 2019.

A. Afram and F. Janabi-Sharifi. Theory and applications of HVAC control systems—a review of model predictive control (MPC). *Building and Environment*, 72:343–355, 2014.

C. Agia, R. Sinha, J. Yang, R. Antonova, M. Pavone, H. Nishimura, M. Itkina, and J. Bohg. CUPID: Curating data your robot loves with influence functions. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 2907–2932. PMLR, 2025a.

C. Agia, R. Sinha, J. Yang, Z. Cao, R. Antonova, M. Pavone, and J. Bohg. Unpacking failure modes of generative policies: Runtime monitoring of consistency and progress. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 689–723. PMLR, 2025b.

AgiBot-World-Contributors, Q. Bu, J. Cai, L. Chen, X. Cui, Y. Ding, S. Feng, S. Gao, X. He, X. Hu, X. Huang, S. Jiang, Y. Jiang, C. Jing, H. Li, J. Li, C. Liu, Y. Liu, Y. Lu, J. Luo, P. Luo, Y. Mu, Y. Niu, Y. Pan, J. Pang, Y. Qiao, G. Ren, C. Ruan, J. Shan, Y. Shen, C. Shi, M. Shi, M. Shi, C. Sima, J. Song, H. Wang, W. Wang, D. Wei, C. Xie, G. Xu, J. Yan, C. Yang, L. Yang, S. Yang, M. Yao, J. Zeng, C. Zhang, Q. Zhang, B. Zhao, C. Zhao, J. Zhao, and J. Zhu. AgiBot World Colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems, 2025. URL `https://arxiv.org/abs/2503.06669`.

P. Agrawal, A. V. Nair, P. Abbeel, J. Malik, and S. Levine. Learning to poke by poking: Experiential learning of intuitive physics. In *Advances in Neural Information Processing Systems*, volume 29, pages 5074–5082, 2016.

M. Ahn, D. Dwibedi, C. Finn, M. G. Arenas, K. Gopalakrishnan, K. Hausman, B. Ichter, A. Irpan, N. Joshi, R. Julian, S. Kirmani, I. Leal, E. Lee, S. Levine, Y. Lu, S. Maddineni, K. Rao, D. Sadigh, P. Sanketi, P. Sermanet, Q. Vuong, S. Welker, F. Xia, T. Xiao, P. Xu, S. Xu, and Z. Xu. AutoRT: Embodied foundation models for large scale orchestration of robotic agents, 2024. URL `https://arxiv.org/abs/2401.12963`.

B. Ai, L. Dai, N. Bohlinger, D. Li, T. Mu, Z. Wu, K. Fay, H. I. Christensen, J. Peters, and H. Su. Towards embodiment scaling laws in robot locomotion. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3483–3515. PMLR, 2025.

I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci. Wireless sensor networks: A survey. *Computer Networks*, 38(4):393–422, 2002.

A. Al-Tawaha, S. Gu, P. Niu, R. Jia, and M. Jin. Remembering more, risking more: Longitudinal safety risks in memory-equipped LLM agents, 2026. URL `https://arxiv.org/abs/2605.17830`.

A. Albu-Schäffer, S. Haddadin, C. Ott, A. Stemmer, T. Wimböck, and G. Hirzinger. The DLR lightweight robot: design and control concepts for robots in human environments. *Industrial Robot: An International Journal*, 34(5):376–385, 2007.

E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, and F. Fleuret. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems*, pages 58757–58791, 2024.

C. Amato. An introduction to centralized training for decentralized execution in cooperative multi-agent reinforcement learning, 2024. URL `https://arxiv.org/abs/2409.03052`.

A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.

A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431. IEEE, 2019.

A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. B. Connors, J. Darpinian, K. Dhabalia, J. Di Carlo, D. Driess, M. R. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Hancock, K. Hausman, G. Hussein, B. Ichter, S. Jakubczak, R. Jen, T. Jones, B. Katz, L. Ke, C. Kuchi, M. Lamb, D. Leblanc, S. Levine, A. Li-Bell, Y. Lu, V. Mano, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, C. Sharma, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, A. Swerdlow, J. Tanner, M. Torne, Q. Vuong, A. Walling, H. Wang, B. Williams, S. Yoo, L. Yu, U. Zhilinsky, and Z. Zhou. $\pi^*_{0.6}$: a VLA that learns from experience. In *Robotics: Science and Systems (RSS)*, 2026.

D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané. Concrete problems in AI safety, 2016. URL `https://arxiv.org/abs/1606.06565`.

H. Araujo, M. R. Mousavi, and M. Varshosaz. Testing, validation, and verification of robotic and autonomous systems: A systematic review. *ACM Transactions on Software Engineering and Methodology*, 32(2):1–61, 2023.

M. Asada, K. F. MacDorman, H. Ishiguro, and Y. Kuniyoshi. Cognitive developmental robotics as a new paradigm for the design of humanoid robots. *Robotics and Autonomous Systems*, 37(2–3): 185–193, 2001.

M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629, 2023.

M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. Robert Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning, 2025. URL `https://arxiv.org/abs/2506.09985`.

K. J. Åström and T. Hägglund. *Advanced PID Control*. ISA, Research Triangle Park, NC, 2005. ISBN 9781556179426.

K. J. Åström and R. M. Murray. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press, Princeton, NJ, 2008.

K. J. Åström and B. Wittenmark. *Adaptive Control*. Addison-Wesley, Reading, MA, 2 edition, 1995. ISBN 9780201558661.

ATI Industrial Automation. *Force/Torque (F/T) Manual: Axia80 Sensor and EtherCAT Axia Interface*, ethercat interface revision 03 edition, n.d. URL https://www.ati-ia.com/app_content/Documents/9610-05-EtherCAT%20Axia80.pdf.

P. Atreya, K. Pertsch, T. Lee, M. J. Kim, A. Jain, A. Kuramshin, C. Eppner, C. Neary, E. Hu, F. Ramos, J. Tremblay, K. Arora, K. Ellis, L. Macesanu, M. Leonard, M. Cho, O. Aslan, S. Dass, J. Wang, X. Yuan, X. Yang, A. Gupta, D. Jayaraman, G. Berseth, K. Daniilidis, R. Martin-Martin, Y. Lee, P. Liang, C. Finn, and S. Levine. RoboArena: Distributed real-world evaluation of generalist robot policies. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 336–364. PMLR, 2025.

A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr. Basic concepts and taxonomy of dependable and secure computing. *IEEE Transactions on Dependable and Secure Computing*, 1(1):11–33, 2004.

C. W. Bac, E. J. van Henten, J. Hemming, and Y. Edan. Harvesting robots for high-value crops: State-of-the-art review and challenges ahead. *Journal of Field Robotics*, 31(6):888–911, 2014.

A. Badithela, D. Snyder, L. Zha, J. Mikhail, M. O'Kelly, A. Dixit, and A. Majumdar. Reliable and scalable robot policy evaluation with imperfect simulators, 2025. URL https://arxiv.org/abs/2510.04354.

M. Baglietto, G. Battistelli, and P. Tesi. Packet loss detection in networked control systems. *International Journal of Robust and Nonlinear Control*, 30:6073–6090, 2020.

L. Bainbridge. Ironies of automation. *Automatica*, 19(6):775–779, 1983.

R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.

R. Bajcsy, Y. Aloimonos, and J. K. Tsotsos. Revisiting active perception. *Autonomous Robots*, 42(2): 177–196, 2018.

T. Baltrušaitis, C. Ahuja, and L.-P. Morency. Multimodal machine learning: A survey and taxonomy. *IEEE transactions on pattern analysis and machine intelligence*, 41(2):423–443, 2019.

M. Baniodeh, K. Goel, S. Ettinger, C. Fuertes, A. Seff, T. Shen, C. Gulino, C. Yang, G. Jerfel, D. Choe, R. Wang, B. Charrow, V. Kallem, S. Casas, R. Al-Rfou, B. Sapp, and D. Anguelov. Scaling laws of motion forecasting and planning – technical report, 2025. URL https://arxiv.org/abs/2506.08228.

Y. Bar-Shalom and E. Tse. Dual effect, certainty equivalence, and separation in stochastic control. *IEEE Transactions on Automatic Control*, 19(5):494–500, 1974.

A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. Revisiting feature prediction for learning visual representations from video. *Transactions on Machine Learning Research*, 2024.

J. Barreiros, A. Beaulieu, A. Bhat, R. Cory, E. Cousineau, H. Dai, C.-H. Fang, K. Hashimoto, M. Z. Irshad, M. Itkina, N. Kuppuswamy, K.-H. Lee, K. Liu, D. McConachie, I. McMahon, H. Nishimura, C. Phillips-Grafflin, C. Richter, P. Shah, K. Srinivasan, B. Wulfe, C. Xu, M. Zhang, A. Alspach, M. Angeles, K. Arora, V. C. Guizilini, A. Castro, D. Chen, T.-S. Chu, S. Creasey, S. Curtis, R. Denitto, E. Dixon, E. Dusel, M. Ferreira, A. Goncalves, G. Gould, D. Guoy, S. Gupta, X. Han, K. Hatch, B. Hathaway, A. Henry, H. Hochsztein, P. Horgan, S. Iwase, D. Jackson, S. Karamcheti, S. Keh, J. Masterjohn, M. Masuda, J. Mercat, P. Miller, P. Mitiguy, T. Nguyen, J. Nimmer, Y. Noguchi, R. Ong, A. Onol, O. Pfannenstiehl, R. Poyner, L. P. M. Rocha, G. Richardson, C. Rodriguez, D. Seale, M. Sherman, M. Smith-Jones, D. Tago, P. Tokmakov, M. Tran, B. Van Hoorick, I. Vasiljevic, S. Zakharov, M. Zolotas, R. Ambruș, K. Fetzer-Borelli, B. Burchfiel, H. Kress-Gazit, S. Feng, S. Ford, and R. Tedrake. A careful examination of large behavior models for multitask dexterous manipulation. *Science Robotics*, 11(113):eaea6201, 2026.

L. W. Barsalou. Perceptual symbol systems. *Behavioral and Brain Sciences*, 22(4):577–660, 1999.

L. W. Barsalou. Grounded cognition. *Annual Review of Psychology*, 59:617–645, 2008.

Basler AG. *Exposure Auto*. Basler Product Documentation, n.d. URL `https://docs.baslerweb.com/exposure-auto`.

P. W. Battaglia, J. B. Hamrick, and J. B. Tenenbaum. Simulation as an engine of physical scene understanding. *Proceedings of the National Academy of Sciences*, 110(45):18327–18332, 2013.

E. Bauer, E. Nava, and R. K. Katzschmann. Latent action diffusion for cross-embodiment manipulation. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 511–518, 2026.

D. M. Bear, E. Wang, D. Mrowca, F. J. Binder, H.-Y. F. Tung, R. T. Pramod, C. Holdaway, S. Tao, K. Smith, F.-Y. Sun, L. Fei-Fei, N. Kanwisher, J. B. Tenenbaum, D. L. K. Yamins, and J. E. Fan. Physion: Evaluating physical prediction from vision in humans and machines. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021.

C. Bédard, I. Lütkebohle, and M. Dagenais. ros2_tracing: Multipurpose low-overhead framework for real-time tracing of ROS 2. *IEEE Robotics and Automation Letters*, 7(3):6511–6518, 2022.

R. D. Beer. Dynamical approaches to cognitive science. *Trends in Cognitive Sciences*, 4(3):91–99, 2000.

BEHAVIOR Team. 2026 BEHAVIOR Challenge: Evaluation and rules. Official challenge documentation, 2026. URL `https://behavior.stanford.edu/challenge/evaluation.html`.

C. D. Bellicoso, C. Gehring, J. Hwangbo, P. Fankhauser, and M. Hutter. Perception-less terrain adaptation through whole body control and hierarchical optimization. In *2016 IEEE-RAS 16th International Conference on Humanoid Robots (Humanoids)*, pages 558–564, 2016.

Q. Ben, F. Jia, J. Zeng, J. Dong, D. Lin, and J. Pang. HOMIE: Humanoid loco-manipulation with isomorphic exoskeleton cockpit. In *Proceedings of Robotics: Science and Systems*, 2025.

Q. Ben, B. Xu, K. Li, F. Jia, W. Zhang, J. Wang, J. Wang, D. Lin, and J. Pang. Gallant: Voxel grid-based humanoid locomotion and local-navigation across 3-d constrained terrains. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 28086–28095, 2026.

S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.

D. Berckmans. General introduction to precision livestock farming. *Animal Frontiers*, 7(1):6–11, 2017.

M. Bettini, A. Shankar, and A. Prorok. Heterogeneous multi-robot reinforcement learning. In *Proceedings of the 22nd International Conference on Autonomous Agents and Multiagent Systems*, pages 1485–1494. International Foundation for Autonomous Agents and Multiagent Systems, 2023.

R. Bhirangi, T. Hellebrekers, C. Majidi, and A. Gupta. ReSkin: versatile, replaceable, lasting tactile skins. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 587–597. PMLR, 2022.

M. K. Bhuyan. *Computer vision and image processing: Fundamentals and applications*. CRC Press, 2019.

Z. M. Bi, Z. Miao, B. Zhang, and W. J. Zhang. The state of the art of testing standards for integrated robotic systems. *Robotics and Computer-Integrated Manufacturing*, 63:101893, 2020.

Z. M. Bi, C. Luo, Z. Miao, B. Zhang, W. J. Zhang, and L. Wang. Safety assurance mechanisms of collaborative robotic systems in manufacturing. *Robotics and Computer-Integrated Manufacturing*, 67:102022, 2021.

A. Bicchi. On the problem of decomposing grasp and manipulation forces in multiple whole-limb manipulation. *Robotics and Autonomous Systems*, 13(2):127–147, 1994.

G. Biggs, J. Perron, and S. Loretz. Actions. ROS 2 Design, 2020. URL `https://design.ros2.org/articles/actions.html`. Last modified May 2020.

Y. Bisk, A. Holtzman, J. Thomason, J. Andreas, Y. Bengio, J. Chai, M. Lapata, A. Lazaridou, J. May, A. Nisnevich, N. Pinto, and J. Turian. Experience grounds language. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 8718–8735, 2020.

K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems*, 2025a.

K. Black, M. Y. Galliker, and S. Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, volume 38, pages 37596–37620, 2025b.

M. Bloesch, M. Hutter, M. Hoepflinger, S. Leutenegger, C. Gehring, C. D. Remy, and R. Siegwart. State estimation for legged robots: Consistent fusion of leg kinematics and IMU. In *Robotics: Science and Systems*, 2012.

J. Blumenkamp, S. Morad, J. Gielis, and A. Prorok. CoViS-Net: A cooperative visual spatial foundation model for multi-robot applications. In P. Agrawal, O. Kroemer, and W. Burgard, editors, *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3780–3808. PMLR, 2025.

D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

J. Bongard, V. Zykov, and H. Lipson. Resilient machines through continuous self-modeling. *Science*, 314(5802):1118–1121, 2006.

D. M. Bossens, S. Ramchurn, and D. Tarapore. Resilient robot teams: a review integrating decentralised control, change-detection, and learning. *Current Robotics Reports*, 3(3):85–95, 2022.

K. Bousmalis, G. Vezzani, D. Rao, C. Devin, A. X. Lee, M. Bauza, T. Davchev, Y. Zhou, A. Gupta, A. Raju, A. Laurens, C. Fantacci, V. Dalibard, M. Zambelli, M. Martins, R. Pevceviciute, M. Blokzijl, M. Denil, N. Batchelor, T. Lampe, E. Parisotto, K. Żołna, S. Reed, S. Gómez Colmenarejo, J. Scholz, A. Abdolmaleki, O. Groth, J.-B. Regli, O. Sushkov, T. Rothörl, J. E. Chen, Y. Aytar, D. Barker, J. Ortiz, M. Riedmiller, J. T. Springenberg, R. Hadsell, F. Nori, and N. Heess. RoboCat: A self-improving generalist agent for robotic manipulation. *Transactions on Machine Learning Research*, 2024.

C. Breazeal. Toward sociable robots. *Robotics and Autonomous Systems*, 42(3–4):167–175, 2003.

R. P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM (JACM)*, 21(2):201–206, 1974.

E. Broadbent, R. Stafford, and B. MacDonald. Acceptance of healthcare robots for the older population: Review and future directions. *International Journal of Social Robotics*, 1(4):319–330, 2009.

A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, T. Jackson, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, K.-H. Lee, S. Levine, Y. Lu, U. Malla, D. Manjunath, I. Mordatch, O. Nachum, C. Parada, J. Peralta, E. Perez, K. Pertsch, J. Quiambao, K. Rao, M. Ryoo, G. Salazar, P. Sanketi, K. Sayed, J. Singh, S. Sontakke, A. Stone, C. Tan, H. Tran, V. Vanhoucke, S. Vega, Q. Vuong, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems XIX*, 2023.

R. A. Brooks. A robust layered control system for a mobile robot. *IEEE Journal on Robotics and Automation*, 2(1):14–23, 1986.

R. A. Brooks. Intelligence without representation. *Artificial Intelligence*, 47(1–3):139–159, 1991.

T. Brooks, B. Mildenhall, T. Xue, J. Chen, D. Sharlet, and J. T. Barron. Unprocessing images for learned raw denoising. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11028–11037. IEEE, 2019.

E. Brown, N. Rodenberg, J. Amend, A. Mozeika, E. Steltz, M. R. Zakin, H. Lipson, and H. M. Jaeger. Universal robotic gripper based on the jamming of granular material. *Proceedings of the National Academy of Sciences*, 107(44):18809–18814, 2010.

J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, Y. Aytar, S. M. E. Bechtle, F. Behbahani, S. C. Chan, N. Heess, L. Gonzalez, S. Osindero, S. Ozair, S. Reed, J. Zhang, K. Zolna, J. Clune, N. de Freitas, S. Singh, and T. Rocktäschel. Genie: Generative interactive environments. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 4603–4623, 2024.

D. Brugali and P. Scandurra. Component-based robotic engineering (part i) [tutorial]. *IEEE Robotics & Automation Magazine*, 16(4):84–96, 2009.

L. Brunke, M. Greeff, A. W. Hall, Z. Yuan, S. Zhou, J. Panerati, and A. P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5:411–444, 2022.

Q. Bu, Y. Yang, J. Cai, S. Gao, G. Ren, M. Yao, P. Luo, and H. Li. Learning to act anywhere with task-centric latent actions. In *Proceedings of Robotics: Science and Systems*, 2025.

B. Burger, P. M. Maffettone, V. V. Gusev, C. M. Aitchison, Y. Bai, X. Wang, X. Li, B. M. Alston, B. Li, R. Clowes, N. Rankin, B. Harris, R. S. Sprick, and A. I. Cooper. A mobile robotic chemist. *Nature*, 583(7815):237–241, 2020.

J. Cai, Z. Cai, J. Cao, Y. Chen, Z. He, L. Jiang, H. Li, H. Li, Y. Li, Y. Liu, Y. Lu, Q. Lv, H. Ma, J. Pang, Y. Qiao, Z. Qiu, Y. Shen, X. Shi, Y. Tian, B. Wang, H. Wang, J. Wang, T. Wang, X. Wei, C. Wu, Y. Xie, B. Xing, Y. Yang, Y. Yang, Q. Yu, F. Yuan, J. Zeng, J. Zhang, S. Zhang, S. Zhang, Z. Zhaxi, B. Zhou, Y. Zhou, Y. Zhou, H. Zhu, Y. Zhu, and Y. Zhu. InternVLA-A1: Unifying understanding, generation and action for robotic manipulation, 2026a. URL `https://arxiv.org/abs/2601.02456`.

J. Cai, Y. Mu, G. Yang, Z. Cao, Z. Tu, X. Gao, K. Li, X. Zhan, L. Yang, Y. Zhu, H. Ma, M. Zhou, Q. Yu, Y. Xue, L. He, Y. Yao, Y. Zhu, L. Ling, B. Jiang, H. Guo, X. Zhu, B. Zhou, B. Zhao, T. Xue, C. Shen, and W. Zhang. InternW0: A foundational physical world model for efficient real-world interactions, 2026b. URL `https://arxiv.org/abs/2609.27656`.

B. Calli, A. Walsman, A. Singh, S. Srinivasa, P. Abbeel, and A. M. Dollar. Benchmarking in manipulation research: Using the Yale-CMU-Berkeley object and model set. *IEEE Robotics & Automation Magazine*, 22(3):36–52, 2015.

M. Camurri, M. Fallon, S. Bazeille, A. Radulescu, V. Barasuol, D. G. Caldwell, and C. Semini. Probabilistic contact estimation and impact detection for state estimation of quadruped robots. *IEEE Robotics and Automation Letters*, pages 1023–1030, 2017.

A. Cangelosi, G. Metta, G. Sagerer, S. Nolfi, C. Nehaniv, K. Fischer, J. Tani, T. Belpaeme, G. Sandini, F. Nori, L. Fadiga, B. Wrede, K. Rohlfing, E. Tuci, K. Dautenhahn, J. Saunders, and A. Zeschel. Integration of action and language knowledge: A roadmap for developmental robotics. *IEEE Transactions on Autonomous Mental Development*, 2(3):167–195, 2010.

M. Y. Cao, S. Laws, and F. Rodriguez y Baena. Six-axis force/torque sensors for robotics applications: A review. *IEEE Sensors Journal*, 21(24):27238–27251, 2021.

N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision*, pages 213–229. Springer, 2020.

S. Caron. Floating base estimation. `https://scaron.info/robotics/floating-base-estimation.html`, n.d.

A. Cervin, D. Henriksson, B. Lincoln, J. Eker, and K.-E. Årzén. How does control timing affect performance? analysis and simulation of timing using Jitterbug and TrueTime. *IEEE Control Systems Magazine*, 23(3):16–30, 2003.

T. K. Chan, C. S. Chin, and X. Zhong. Review of current technologies and proposed intelligent methodologies for water distributed network leakage detection. *IEEE Access*, 6:78846–78867, 2018.

M. Chang, G. Chhablani, A. Clegg, M. D. Cote, R. Desai, M. Hlavac, V. Karashchuk, J. Krantz, R. Mottaghi, P. Parashar, S. Patki, I. Prasad, X. Puig, A. Rai, R. Ramrakhya, D. Tran, J. Truong, J. M. Turner, E. Undersander, and T.-Y. Yang. Partnr: A benchmark for planning and reasoning in embodied multi-agent tasks. In *International Conference on Learning Representations*, volume 2025, pages 65205–65268, 2025.

Y. Chang, N. Hughes, A. Ray, and L. Carlone. Hydra-Multi: Collaborative online construction of 3D scene graphs with multi-robot teams. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 10995–11002. IEEE, 2023.

D. Charatan, S. L. Li, A. Tagliasacchi, and V. Sitzmann. pixelSplat: 3D Gaussian splats from image pairs for scalable generalizable 3D reconstruction. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19457–19467. IEEE, 2024.

F. Chaumette and S. Hutchinson. Visual servo control. i. basic approaches. *IEEE robotics & automation magazine*, 13(4):82–90, 2006.

C. Cheang, S. Chen, Z. Cui, Y. Hu, L. Huang, T. Kong, H. Li, Y. Li, Y. Liu, X. Ma, H. Niu, W. Ou, W. Peng, Z. Ren, H. Shi, J. Tian, H. Wu, X. Xiao, Y. Xiao, J. Xu, and Y. Yang. GR-3 technical report, 2025. URL `https://arxiv.org/abs/2507.15493`.

Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox. Closing the sim-to-real loop: Adapting simulation randomization with real world experience. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8973–8979, 2019.

A. Chemero. An outline of a theory of affordances. *Ecological Psychology*, 15(2):181–195, 2003.

B. Chen, Z. Xu, S. Kirmani, B. Ichter, D. Sadigh, L. Guibas, and F. Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14455–14465, 2024.

J. Chen, C. Yu, X. Zhou, T. Xu, Y. Mu, M. Hu, W. Shao, Y. Wang, G. Li, and L. Shao. EMOS: Embodiment-aware heterogeneous multi-robot operating system with LLM agents. In *International Conference on Learning Representations*, volume 2025, pages 56670–56690, 2025a.

R. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

T. Chen, Z. He, and M. Ciocarlie. Hardware as policy: Mechanical and computational co-optimization using deep reinforcement learning. In *Proceedings of the 2020 Conference on Robot Learning*, volume 155 of *Proceedings of Machine Learning Research*, pages 1158–1173. PMLR, 2021.

T. Chen, Y. Chen, Z. Li, J. Tang, K. Su, H. Lu, W. Wan, B. Chen, S. Liu, H. Yan, H. Su, Z. Dou, K. Wang, D. Zhang, Y. Liu, Y. Qin, Q. Liang, Q. Wu, Z. Lin, W. Lin, Y. Wang, M. He, T. Wu, R. Wu, J. Zhou, K.-C. Lei, H. Yu, Y. Ji, W. Jin, G. Lin, X. Li, Q. Xiong, R. Xu, Z. Li, W. Chai, E. Xie, Z. Wang, Y. Mu, H. Dong, W. Matusik, M. Ding, W. Ding, P. Luo, and M. Tomizuka. RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies, 2026a. URL https://arxiv.org/abs/2607.04434.

X. Chen, Y. Chen, Y. Fu, N. Gao, J. Jia, W. Jin, H. Li, Y. Mu, J. Pang, Y. Qiao, Y. Tian, B. Wang, B. Wang, F. Wang, H. Wang, T. Wang, Z. Wang, X. Wei, C. Wu, S. Yang, J. Ye, J. Yu, J. Zeng, J. Zhang, J. Zhang, S. Zhang, F. Zheng, B. Zhou, and Y. Zhu. InternVLA-M1: A spatially guided vision-language-action framework for generalist robot policy, 2025b. URL https://arxiv.org/abs/2510.13778.

X. Chen, H. Guo, S. Guo, B. Jiang, C. Shen, X. Shen, T. Xue, Y. Xue, M. Yu, W. Zhang, B. Zhao, B. Zhou, and M. Zhou. A definition and roadmap for world models, 2026b. URL https://arxiv.org/abs/2607.06401.

X. Cheng, J. Li, S. Yang, G. Yang, and X. Wang. Open-TeleVision: Teleoperation with immersive active visual feedback. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2729–2749. PMLR, 2025.

C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023.

C. Chi, Z. Xu, C. Pan, E. Cousineau, B. Burchfiel, S. Feng, R. Tedrake, and S. Song. Universal Manipulation Interface: In-the-wild robot teaching without in-the-wild robots. In *Robotics: Science and Systems*, 2024.

F. Chollet. On the measure of intelligence, 2019. URL https://arxiv.org/abs/1911.01547.

L. P. Chovet, G. M. Garcia, A. Bera, A. Richard, K. Yoshida, and M. A. Olivares-Mendez. Performance comparison of ROS 2 middlewares for multi-robot mesh networks in planetary exploration, 2024. URL https://arxiv.org/abs/2407.03091.

K. Chua, R. Calandra, R. McAllister, and S. Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

A. Clark. *Being There: Putting Brain, Body, and World Together Again*. MIT Press, 1997.

A. Clark and D. J. Chalmers. The extended mind. *Analysis*, 58(1):7–19, 1998.

M. Colledanchise and P. Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, Boca Raton, FL, 2018. ISBN 9781138593732.

S. H. Collins, A. Ruina, R. Tedrake, and M. Wisse. Efficient bipedal robots based on passive-dynamic walkers. *Science*, 307(5712):1082–1085, 2005.

Council of Europe. Council of Europe Framework Convention on Artificial Intelligence and Human Rights, Democracy and the Rule of Law, 2024. URL https://www.coe.int/web/conventions/full-list?module=treaty-detail&treatynum=225. Council of Europe Treaty Series No. 225, opened for signature 5 September 2024.

Countries Attending the AI Safety Summit. The Bletchley Declaration by Countries Attending the AI Safety Summit, 1–2 November 2023, 2023. URL https://www.gov.uk/government/publications/ai-safety-summit-2023-the-bletchley-declaration.

Y. Cui, Y. Zhang, L. Tao, Y. Li, X. Yi, and Z. Li. End-to-end dexterous arm-hand VLA policies via shared autonomy: VR teleoperation augmented by autonomous hand VLA policy for efficient data collection, 2025. URL https://arxiv.org/abs/2511.00139v2.

A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret. Robots that can adapt like animals. *Nature*, 521: 503–507, 2015.

R. S. Dahiya, G. Metta, M. Valle, and G. Sandini. Tactile sensing—from humans to humanoids. *IEEE transactions on robotics*, 26(1):1–20, 2010.

T. Dai, S. Vijayakrishnan, F. T. Szczypiński, J.-F. Ayme, E. Simaei, T. Fellowes, R. Clowes, L. Kotopanov, C. E. Shields, Z. Zhou, J. W. Ward, and A. I. Cooper. Autonomous mobile robots for exploratory synthetic chemistry. *Nature*, 635(8040):890–897, 2024.

T. Dai, J. Wong, Y. Jiang, C. Wang, C. Gokmen, R. Zhang, J. Wu, and L. Fei-Fei. Automated creation of digital cousins for robust policy learning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4912–4943. PMLR, 2025.

T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, 2022.

P. de Haan, D. Jayaraman, and S. Levine. Causal confusion in imitation learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

M. P. de Looze, T. Bosch, F. Krause, K. S. Stadler, and L. W. O'Sullivan. Exoskeletons for industrial application and their potential effects on physical work load. *Ergonomics*, 59(5):671–681, 2016.

A. De Luca and R. Mattone. Actuator failure detection and isolation using generalized momenta. In *IEEE International Conference on Robotics and Automation (ICRA)*, volume 1, pages 634–639, 2003.

P. E. Debevec and J. Malik. Recovering high dynamic range radiance maps from photographs. In *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, pages 643–652. Association for Computing Machinery, 2023.

J. Degrave, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de las Casas, C. Donner, L. Fritz, C. Galperti, A. Huber, J. Keeling, M. Tsimpoukelli, J. Kay, A. Merle, J.-M. Moret, S. Noury, F. Pesamosca, D. Pfau, O. Sauter, C. Sommariva, S. Coda, B. Duval, A. Fasoli, P. Kohli, K. Kavukcuoglu, D. Hassabis, and M. Riedmiller. Magnetic control of tokamak plasmas through deep reinforcement learning. *Nature*, 602(7897):414–419, 2022.

J. Delmerico, S. Mintchev, A. Giusti, B. Gromov, K. Melo, T. Horvat, C. Cadena, M. Hutter, A. Ijspeert, D. Floreano, L. M. Gambardella, R. Siegwart, and D. Scaramuzza. The current state and future outlook of rescue robotics. *Journal of Field Robotics*, 36(7):1171–1191, 2019.

C. Deng, D. Zhu, K. Li, C. Gou, F. Li, Z. Wang, S. Zhong, W. Yu, X. Nie, Z. Song, G. Shi, and H. Fan. Emerging properties in unified multimodal pretraining, 2025a. URL `https://arxiv.org/abs/2505.14683`.

S. Deng, M. Yan, S. Wei, H. Ma, Y. Yang, J. Chen, Z. Zhang, T. Yang, X. Zhang, H. Cui, Z. Zhang, and H. Wang. GraspVLA: a grasping foundation model pre-trained on billion-scale synthetic action data. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 1004–1029. PMLR, 2025b.

A. Desai, S. Ghosh, S. A. Seshia, N. Shankar, and A. Tiwari. SOTER: A runtime assurance framework for programming safe robotics systems. In *IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 138–150. IEEE, 2019.

J. Di Carlo, P. M. Wensing, B. Katz, G. Bledt, and S. Kim. Dynamic locomotion in the MIT Cheetah 3 through convex model-predictive control. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1–9. IEEE, 2018.

C. Di Franco and N. Bezzo. Interpretable run-time monitoring and replanning for safe autonomous systems operations. *IEEE Robotics and Automation Letters*, 5(2):2427–2434, 2020.

Y. Diao, Y. Zhang, D. De Martini, P. G. Zhao, and E. L. Li. Task-oriented co-design of communication, computing, and control for edge-enabled industrial cyber-physical systems. *IEEE Journal on Selected Areas in Communications*, 43(9):3041–3055, 2025.

S. Dohare, J. F. Hernandez-Garcia, Q. Lan, P. Rahman, A. R. Mahmood, and R. S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024.

A. M. Dollar and R. D. Howe. The highly adaptive SDM hand: Design and performance evaluation. *The International Journal of Robotics Research*, 29(5):585–597, 2010.

S. Doncieux, D. Filliat, N. Díaz-Rodríguez, T. Hospedales, R. Duro, A. Coninx, D. M. Roijers, B. Girard, N. Perrin, and O. Sigaud. Open-ended learning: A conceptual framework based on representational redescription. *Frontiers in Neurorobotics*, 12:59, 2018.

R. C. Dorf and R. H. Bishop. *Modern Control Systems*. Pearson, Boston, 13 edition, 2017.

M. Dorigo, G. Theraulaz, and V. Trianni. Swarm robotics: Past, present, and future. *Proceedings of the IEEE*, 109(7):1152–1165, 2021.

A. D. Dragan, K. C. T. Lee, and S. S. Srinivasa. Legibility and predictability of robot motion. In *2013 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 301–308, 2013.

A. D. Dragan, S. Bauman, J. Forlizzi, and S. S. Srinivasa. Effects of robot motion on human-robot collaboration. In *Proceedings of the Tenth Annual ACM/IEEE International Conference on Human-Robot Interaction*, pages 51–58, 2015.

D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence. PaLM-E: An embodied multimodal language model. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 8469–8488, 2023.

D. Driess, J. T. Springenberg, B. Ichter, L. Yu, A. Li-Bell, K. Pertsch, A. Z. Ren, H. Walke, Q. Vuong, L. X. Shi, and S. Levine. Knowledge insulating vision-language-action models: Train fast, run fast, generalize better, 2025. URL `https://arxiv.org/abs/2505.23705`.

Eclipse Zenoh. What is Zenoh? Official documentation, n.d. URL `https://zenoh.io/docs/overview/what-is-zenoh/`.

A. Esquenazi, M. Talaty, A. Packel, and M. Saulino. The ReWalk powered exoskeleton to restore ambulatory function to individuals with thoracic-level motor-complete spinal cord injury. *American Journal of Physical Medicine & Rehabilitation*, 91(11):911–921, 2012.

European Parliament and Council of the European Union. Directive 2006/42/EC on machinery. Official Journal of the European Union, 2006. URL `https://eur-lex.europa.eu/eli/dir/2006/42/oj/eng`.

European Parliament and Council of the European Union. Regulation (EU) 2016/679 (General Data Protection Regulation). Official Journal of the European Union, 2016. URL `https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng`.

European Parliament and Council of the European Union. Regulation (EU) 2017/745 on medical devices. Official Journal of the European Union, 2017. URL `https://eur-lex.europa.eu/eli/reg/2017/745/oj/eng`.

European Parliament and Council of the European Union. Regulation (EU) 2023/1230 on machinery. Official Journal of the European Union, 2023. URL `https://eur-lex.europa.eu/eli/reg/2023/1230/oj/eng`.

European Parliament and Council of the European Union. Regulation (EU) 2024/1689: Artificial intelligence act. Official Journal of the European Union, 2024a. URL https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng.

European Parliament and Council of the European Union. Directive (EU) 2024/2853 of 23 October 2024 on liability for defective products and repealing Council Directive 85/374/EEC. Official Journal of the European Union, L 2024/2853, 18 November 2024, 2024b. URL https://eur-lex.europa.eu/eli/dir/2024/2853/oj.

European Parliament and Council of the European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act): Consolidated version of 27 july 2026. EUR-Lex, 2026. URL https://eur-lex.europa.eu/eli/reg/2024/1689/2026-07-27/eng.

K. Eykholt, I. Evtimov, E. Fernandes, B. Li, A. Rahmati, C. Xiao, A. Prakash, T. Kohno, and D. Song. Robust physical-world attacks on deep learning visual classification. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1625–1634, 2018.

B. Eysenbach, S. Gu, J. Ibarz, and S. Levine. Leave no trace: Learning to reset for safe and autonomous reinforcement learning. In *International Conference on Learning Representations*, 2018.

J. A. Falco, D. Hemphill, K. E. Kimble, E. R. Messina, A. Norton, R. F. Ropelato, and H. A. Yanco. Benchmarking protocols for evaluating grasp strength, grasp cycle time, finger strength, and finger repeatability of robot end-effectors. *IEEE Robotics and Automation Letters*, 5(2):644–651, 2020.

H. Fang, H.-S. Fang, Y. Wang, J. Ren, J. Chen, R. Zhang, W. Wang, and C. Lu. AirExo: Low-cost exoskeletons for learning whole-arm manipulation in the wild. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 15031–15038, 2024a.

H. Fang, C. Wang, Y. Wang, J. Chen, S. Xia, J. Lv, Z. He, X. Yi, Y. Guo, X. Zhan, L. Yang, W. Wang, C. Lu, and H.-S. Fang. AirExo-2: Scaling up generalizable robotic imitation learning with low-cost exoskeletons. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 198–220. PMLR, 2025.

H.-S. Fang, H. Fang, Z. Tang, J. Liu, C. Wang, J. Wang, H. Zhu, and C. Lu. RH20T: A comprehensive robotic dataset for learning diverse skills in one-shot. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 653–660. IEEE, 2024b.

P. Fankhauser, M. Bloesch, and M. Hutter. Probabilistic terrain mapping for mobile robots with uncertain localization. *IEEE Robotics and Automation Letters*, 3(4):3019–3026, 2018.

D. Farina, N. Jiang, H. Rehbaum, A. Holobar, B. Graimann, H. Dietl, and O. C. Aszmann. The extraction of neural information from the surface EMG for the control of upper-limb prostheses: Emerging avenues and challenges. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 22(4):797–809, 2014.

FAULHABER. *DC-Micromotors: Technical Information*. Dr. Fritz Faulhaber GmbH & Co. KG, n.d. URL https://www.faulhaber.com/fileadmin/Import/Media/EN_TI_DC-MOTORS.pdf.

N. Fazeli, M. Oller, J. Wu, Z. Wu, J. B. Tenenbaum, and A. Rodriguez. See, feel, act: Hierarchical learning for complex manipulation skills with multisensory fusion. *Science Robotics*, 4(26): eaav3123, 2019.

R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.

S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. LIBERO-Plus: A progressive robustness benchmark for visual-language-action models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 38574–38583, 2026.

C. Ferrari and J. Canny. Planning optimal grasps. In *IEEE International Conference on Robotics and Automation (ICRA)*, volume 3, pages 2290–2295, 1992.

R. E. Fikes and N. J. Nilsson. STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2(3–4):189–208, 1971.

C. Finn and S. Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation*, pages 2786–2793, 2017.

J. A. Fodor and Z. W. Pylyshyn. Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28(1–2):3–71, 1988.

T. Foote. Clock and time. ROS 2 Design, 2018. URL `https://design.ros2.org/articles/clock_and_time.html`.

M. Føre, K. Frank, T. Norton, E. Svendsen, J. A. Alfredsen, T. Dempster, H. Eguiraun, W. Watson, A. Stahl, L. M. Sunde, C. Schellewald, K. R. Skøien, M. O. Alver, and D. Berckmans. Precision fish farming: A new framework to improve production in aquaculture. *Biosystems Engineering*, 173:176–193, 2018.

S. Forestier, R. Portelas, Y. Mollard, and P.-Y. Oudeyer. Intrinsically motivated goal exploration processes with automatic curriculum learning. *Journal of Machine Learning Research*, 23(152):1–41, 2022.

J. Forlizzi and C. DiSalvo. Service robots in the domestic environment: A study of the Roomba vacuum in the home. In *Proceedings of the 1st ACM SIGCHI/SIGART Conference on Human-Robot Interaction*, pages 258–265, 2006.

C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza. On-manifold preintegration for real-time visual-inertial odometry. *IEEE Transactions on Robotics*, pages 1–21, 2017.

B. A. Francis and W. M. Wonham. The internal model principle of control theory. *Automatica*, 12(5): 457–465, 1976.

R. M. Francos and A. M. Bruckstein. On the role and opportunities in teamwork design for advanced multi-robot search systems. *Frontiers in Robotics and AI*, 10:1089062, 2023.

G. F. Franklin, J. D. Powell, and A. Emami-Naeini. *Feedback Control of Dynamic Systems*. Pearson, New York, 8 edition, 2019.

Z. Fu, X. Cheng, and D. Pathak. Deep whole-body control: Learning a unified policy for manipulation and locomotion. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 138–149. PMLR, 2023.

Z. Fu, T. Z. Zhao, and C. Finn. Mobile ALOHA: Learning bimanual mobile manipulation using low-cost whole-body teleoperation. In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *PMLR*, pages 4066–4083, 2025.

P. Furgale, J. Rehder, and R. Siegwart. Unified temporal and spatial calibration for multi-sensor systems. In *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 1280–1286. IEEE, 2013.

H. Furukawa, K. E. Cordova, M. O'Keeffe, and O. M. Yaghi. The chemistry and applications of metal-organic frameworks. *Science*, 341(6149):1230444, 2013.

J. Gao, B. Sarkar, F. Xia, T. Xiao, J. Wu, B. Ichter, A. Majumdar, and D. Sadigh. Physically grounded vision-language models for robotic manipulation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 12462–12469. IEEE, 2024.

J. Gao, J. Chang, H. Que, Y. Xiong, S. Zhang, X. Qi, Z. Liu, J.-J. Wang, Q. Ding, X. Li, Z. Pan, Q. Xie, Z. Yan, J. Yan, and L. Zhang. UniLabOS: An AI-native operating system for autonomous laboratories, 2025. URL https://arxiv.org/abs/2512.21766.

N. Gao, J. Zheng, X. Gao, H. Ma, H. Wang, Y. Wang, J. Chen, Z. Chen, S. Zhang, M. Jia, X. Jiang, Z. Zhu, X. Li, S. Wang, H. Li, W. Cai, Y. Yang, X. Xu, Z. Lyu, Y. Mu, T. Wang, J. Pang, J. Zeng, W. Zhang, and C. Shen. EBench: Elemental diagnosis of generalist mobile manipulation policies, 2026. URL https://arxiv.org/abs/2606.18239.

K. Garg, S. Zhang, O. So, C. Dawson, and C. Fan. Learning safe control for multi-robot systems: Methods, verification, and open challenges. *Annual Reviews in Control*, 57:100948, 2024.

C. Garrett, A. Mandlekar, B. Wen, and D. Fox. SkillMimicGen: Automated demonstration generation for efficient skill learning and deployment. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2750–2790. PMLR, 2025.

C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 30, pages 440–448, 2020.

C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4: 265–293, 2021.

Q. Garrido, N. Ballas, M. Assran, A. Bardes, L. Najman, M. Rabbat, E. Dupoux, and Y. LeCun. Intuitive physics understanding emerges from self-supervised pretraining on natural videos, 2025. URL https://arxiv.org/abs/2502.11831.

E. Gat. On three-layer architectures. In D. Kortenkamp, R. P. Bonasso, and R. Murphy, editors, *Artificial Intelligence and Mobile Robots: Case Studies of Successful Robot Systems*, pages 195–210. AAAI Press, Menlo Park, CA, 1998. ISBN 9780262611374.

D. V. Gealy, S. McKinley, B. Yi, P. Wu, P. R. Downey, G. Balke, A. Zhao, M. Guo, R. Thomasson, A. Sinclair, P. Cuellar, Z. McCarthy, and P. Abbeel. Quasi-direct drive for low-cost compliant robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 437–443, 2019.

T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé, III, and K. Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

Gemini Robotics Team, A. Abdolmaleki, S. Abeyruwan, J. Ainslie, J.-B. Alayrac, M. G. Arenas, A. Balakrishna, N. Batchelor, A. Bewley, J. Bingham, M. Bloesch, K. Bousmalis, P. Brakel, A. Brohan, T. Buschmann, A. Byravan, S. Cabi, K. Caluwaerts, F. Casarini, C. Chan, O. Chang, L. Chappellet-Volpini, J. E. Chen, X. Chen, H.-T. L. Chiang, K. Choromanski, A. Collister, D. B. D'Ambrosio, S. Dasari, T. Davchev, M. K. Dave, C. Devin, N. Di Palo, T. Ding, C. Doersch, A. Dostmohamed, Y. Du, D. Dwibedi, S. T. Egambaram, M. Elabd, T. Erez, X. Fang, C. Fantacci, C. Fong, E. Frey, C. Fu, R. Gao, M. Giustina, K. Gopalakrishnan, L. Graesser, O. Groth, A. Gupta, R. Hafner, S. Hansen, L. Hasenclever, S. Haves, N. Heess, B. Hernaez, A. Hofer, J. Hsu, L. Huang, S. H. Huang, A. Iscen, M. G. Jacob, D. Jain, S. Jesmonth, A. Jindal, R. Julian, D. Kalashnikov, M. E. Karagozler, S. Karp, M. Kecman, J. C. Kew, D. Kim, F. Kim, J. Kim, T. Kipf, S. Kirmani, K. Konyushkova, L. Y. Ku, Y. Kuang, T. Lampe, A. Laurens, T. A. Le, I. Leal, A. X. Lee, T.-W. E. Lee, G. Lever, J. Liang, L.-H. Lin, F. Liu, S. Long, C. Lu, S. Maddineni, A. Majumdar, K.-K. Maninis, A. Marmon, S. Martinez, A. H. Michaely, N. Milonopoulos, J. Moore, R. Moreno, M. Neunert, F. Nori, J. Ortiz, K. Oslund, C. Parada, E. Parisotto, A. Paryag, A. Pooley, T. Power, A. Quaglino, H. Qureshi, R. V. Raju, H. Ran, D. Rao, K. Rao, I. Reid, D. Rendleman, K. Reymann, M. Rivas, F. Romano, Y. Rubanova, P. P. Sampedro, P. R. Sanketi, D. Shah, M. Sharma, K. Shea, M. Shridhar, C. Shu, V. Sindhwani, S. Singh, R. Soricut, R. Sterneck, I. Storz, R. Surdulescu, J. Tan, J. Tompson, S. Tunyasuvunakool, J. Varley, G. Vesom, G. Vezzani, M. B. Villalonga, O. Vinyals, R. Wagner, A. Wahid, S. Welker, P. Wohlhart, C. Wu, M. Wulfmeier, F. Xia, T. Xiao, A. Xie, J. Xie, P. Xu, S. Xu,

Y. Xu, Z. Xu, J. Yan, S. Yang, S. Yang, Y. Yang, H. H. Yu, W. Yu, W. Yuan, Y. Yuan, J. Zhang, T. Zhang, Z. Zhang, A. Zhou, G. Zhou, and Y. Zhou. Gemini Robotics 1.5: Pushing the frontier of generalist robots with advanced embodied reasoning, thinking, and motion transfer, 2025. URL https://arxiv.org/abs/2510.03342.

Generalist AI. GEN-0: Embodied foundation models that scale with physical interaction. Technical blog, 2025. URL https://generalistai.com/blog/nov-04-2025-GEN-0.

B. P. Gerkey and M. J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International Journal of Robotics Research*, 23(9):939–954, 2004.

T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki. Act3D: 3D feature field transformers for multi-task robotic manipulation. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 3949–3965. PMLR, 2023.

D. Ghosh, H. R. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, Q. Vuong, T. Xiao, P. R. Sanketi, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024.

J. J. Gibson. *The Ecological Approach to Visual Perception*. Houghton Mifflin, Boston, 1979. ISBN 978-0-395-27049-3.

J. Gielis, A. Shankar, and A. Prorok. A critical review of communications in multi-robot systems. *Current robotics reports*, 3(4):213–225, 2022.

T. Gneiting and A. E. Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007.

K. Goldberg. Good old-fashioned engineering can close the 100,000-year "data gap" in robotics. *Science Robotics*, 10(105):eaea7390, 2025.

A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox. RVT: Robotic view transformer for 3D object manipulation. In *Conference on Robot Learning*, pages 694–710. PMLR, 2023.

K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, M. Martin, T. Nagarajan, I. Radosavovic, S. K. Ramakrishnan, F. Ryan, J. Sharma, M. Wray, M. Xu, E. Z. Xu, C. Zhao, S. Bansal, D. Batra, V. Cartillier, S. Crane, T. Do, M. Doulaty, A. Erapalli, C. Feichtenhofer, A. Fragomeni, Q. Fu, A. Gebreselasie, C. Gonzalez, J. Hillis, X. Huang, Y. Huang, W. Jia, W. Khoo, J. Kolar, S. Kottur, A. Kumar, F. Landini, C. Li, Y. Li, Z. Li, K. Mangalam, R. Modhugu, J. Munro, T. Murrell, T. Nishiyasu, W. Price, P. R. Puentes, M. Ramazanova, L. Sari, K. Somasundaram, A. Southerland, Y. Sugano, R. Tao, M. Vo, Y. Wang, X. Wu, T. Yagi, Z. Zhao, Y. Zhu, P. Arbelaez, D. Crandall, D. Damen, G. M. Farinella, C. Fuegen, B. Ghanem, V. K. Ithapu, C. V. Jawahar, H. Joo, K. Kitani, H. Li, R. Newcombe, A. Oliva, H. S. Park, J. M. Rehg, Y. Sato, J. Shi, M. Z. Shou, A. Torralba, L. Torresani, M. Yan, and J. Malik. Ego4D: Around the world in 3,000 hours of egocentric video. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18973–18990, 2022.

S. Grushko, A. Vysocky, J. Suder, L. Glogar, and Z. Bobovsky. Improving human awareness during collaboration with robot: Review. *MM Science Journal*, pages 5475–5480, 2021.

S. Gu, J. Grudzien Kuba, Y. Chen, Y. Du, L. Yang, A. Knoll, and Y. Yang. Safe multi-agent reinforcement learning for multi-robot control. *Artificial Intelligence*, 319:103905, 2023.

T. Gu, B. Dolan-Gavitt, and S. Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain, 2017. URL https://arxiv.org/abs/1708.06733.

Z. Gu, B. Zhu, G. Zhu, Y. Chen, M. Tang, and J. Wang. AnomalyGPT: Detecting industrial anomalies using large vision-language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 1932–1940, 2024.

J. Guiochet, M. Machin, and H. Waeselynck. Safety-critical advanced robots: A survey. *Robotics and Autonomous Systems*, 94:43–52, 2017.

C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.

Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn. VLAW: Iterative co-improvement of Vision-Language-Action policy and world model, 2026a. URL `https://arxiv.org/abs/2602.12063`.

Y. Guo, L. X. Shi, J. Chen, and C. Finn. Ctrl-World: A controllable generative world model for robot manipulation. In *International Conference on Learning Representations*, 2026b.

A. Gupta, S. Savarese, S. Ganguli, and F.-F. Li. Embodied intelligence via learning and evolution. *Nature Communications*, 12(1):5721, 2021.

A. Gupta, L. Fan, S. Ganguli, and L. Fei-Fei. Metamorph: Learning universal controllers with transformers. In *International Conference on Learning Representations*, 2022.

D. Ha and J. Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018.

S. Ha, S. Coros, A. Alspach, J. Kim, and K. Yamane. Computational co-optimization of design parameters and motion trajectories for robotic systems. *The International Journal of Robotics Research*, 37(13–14):1521–1536, 2018.

S. Haddadin, A. De Luca, and A. Albu-Schäffer. Robot collisions: A survey on detection, isolation, and identification. *IEEE Transactions on Robotics*, 33(6):1292–1312, 2017.

D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2555–2565. PMLR, 2019.

D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 640:647–653, 2025.

D. L. Hall and J. Llinas. An introduction to multisensor data fusion. *Proceedings of the IEEE*, 85(1): 6–23, 1997.

J. A. Hanley and A. Lippman-Hand. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA*, 249(13):1743–1745, 1983.

N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024.

S. Harnad. The symbol grounding problem. *Physica D: Nonlinear Phenomena*, 42(1–3):335–346, 1990.

R. Hartley and A. Zisserman. *Multiple view geometry in computer vision*. Cambridge University Press, 2 edition, 2003.

R. Hartley, M. Ghaffari, R. M. Eustice, and J. W. Grizzle. Contact-aided invariant extended Kalman filtering for robot state estimation. *The International Journal of Robotics Research*, 39(4):402–430, 2020.

H. Hauser, A. J. Ijspeert, R. M. Füchslin, R. Pfeifer, and W. Maass. Towards a theoretical foundation for morphological computation with compliant bodies. *Biological Cybernetics*, 105(5–6):355–370, 2011.

C. Hazirbas, L. Ma, C. Domokos, and D. Cremers. FuseNet: Incorporating depth into semantic segmentation via fusion-based CNN architecture. In *Computer Vision – ACCV 2016*, volume 10111 of *Lecture Notes in Computer Science*, pages 213–228. Springer, 2017.

K. He, G. Gkioxari, P. Dollár, and R. Girshick. Mask R-CNN. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 2961–2969, 2017.

T. He, J. Gao, W. Xiao, Y. Zhang, Z. Wang, J. Wang, Z. Luo, G. He, N. Sobanbabu, C. Pan, Z. Yi, G. Qu, K. Kitani, J. K. Hodgins, L. Fan, Y. Zhu, C. Liu, and G. Shi. ASAP: Aligning simulation and real-world physics for learning agile humanoid whole-body skills. In *Proceedings of Robotics: Science and Systems*, 2025a.

T. He, W. Xiao, T. Lin, Z. Luo, Z. Xu, Z. Jiang, J. Kautz, C. Liu, G. Shi, X. Wang, L. J. Fan, and Y. Zhu. Hover: Versatile neural whole-body controller for humanoid robots. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9989–9996, 2025b.

G. E. Healey and R. Kondepudy. Radiometric CCD camera calibration and noise estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(3):267–276, 1994.

HEIDENHAIN. Why dual encoders are the right choice for robotics design, 2023. URL `https://www.heidenhain.us/resources-and-news/4-reasons-dual-encoders-are-the-right-choice-for-robotics-design/`.

J. Hejna, C. Bhateja, Y. Jiang, K. Pertsch, and D. Sadigh. Re-Mix: Optimizing data mixtures for large scale imitation learning. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 145–164. PMLR, 2025a.

J. Hejna, S. Mirchandani, A. Balakrishna, A. Xie, A. Wahid, J. Tompson, P. Sanketi, D. Shah, C. Devin, and D. Sadigh. Robot data curation with mutual information estimators. In *Proceedings of Robotics: Science and Systems*, 2025b.

R. Held and A. Hein. Movement-produced stimulation in the development of visually guided behavior. *Journal of Comparative and Physiological Psychology*, 56(5):872–876, 1963.

T. A. Henzinger. The theory of hybrid automata. In *Proceedings of the 11th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 278–292. IEEE Computer Society, 1996.

C. Higuera, A. Sharma, C. K. Bodduluri, T. Fan, P. Lancaster, M. Kalakrishnan, M. Kaess, B. Boots, M. Lambeta, T. Wu, and M. Mukadam. Sparsh: Self-supervised touch representations for vision-based tactile sensing. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 885–915. PMLR, 2025.

G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015. URL `https://arxiv.org/abs/1503.02531`.

J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pages 6840–6851, 2020.

G. Hoffman. Evaluating fluency in human–robot collaboration. *IEEE Transactions on Human-Machine Systems*, 49(3):209–218, 2019.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. Rae, and L. Sifre. An empirical analysis of compute-optimal large language model training. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030. Curran Associates, Inc., 2022.

M. Hoffmann, H. G. Marques, A. H. Arieta, H. Sumioka, M. Lungarella, and R. Pfeifer. Body schema in robotics: A review. *IEEE Transactions on Autonomous Mental Development*, 2(4):304–324, 2010.

N. Hogan. Impedance control: An approach to manipulation: Part I—theory. *Journal of Dynamic Systems, Measurement, and Control*, 107(1):1–7, 1985.

P. F. Hokayem and M. W. Spong. Bilateral teleoperation: An historical survey. *Automatica*, 42(12): 2035–2057, 2006.

G. C. Holst and T. S. Lomheim. *CMOS/CCD Sensors and Camera Systems*. JCD Publishing, 2007.

R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang. EgoDex: Learning dexterous manipulation from large-scale egocentric video. In *International Conference on Learning Representations (ICLR)*, 2026.

R. Horaud, M. Hansard, G. Evangelidis, and C. Ménier. An overview of depth cameras and range scanners based on time-of-flight technologies. *Machine vision and applications*, 27(7):1005–1020, 2016.

A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard. OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous robots*, 34(3):189–206, 2013.

Y. Hou, Z. Liu, C. Chi, E. Cousineau, N. Kuppuswamy, S. Feng, B. Burchfiel, and S. Song. Adaptive compliance policy: Learning approximate compliance for diffusion guided control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 4829–4836, 2025.

E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022a.

Y. Hu, S. Fang, Z. Lei, Y. Zhong, and S. Chen. Where2comm: Communication-efficient collaborative perception via spatial confidence maps. In *Advances in Neural Information Processing Systems*, volume 35, pages 4874–4886, 2022b.

J. Huang, S. Wang, F. Lin, Y. Hu, C. Wen, and Y. Gao. Tactile-VLA: unlocking vision-language-action model's physical knowledge for tactile generalization, 2025a. URL https://arxiv.org/abs/2507.09160.

T. Huang, J. Ren, H. Wang, Z. Wang, Q. Ben, M. Wen, X. Chen, J. Li, and J. Pang. Learning humanoid standing-up control across diverse postures. In *Proceedings of Robotics: Science and Systems*, 2025b.

T. Huang, R. Liu, X. Tang, X. Zhang, J. Ren, H. Wang, F. Jia, Y. Qi, K. Yin, W. Zeng, L. Chen, X. Li, T. Wu, K. Li, R. Dai, J. Wang, L. Han, and J. Pang. Towards professional tennis styles for humanoid robots with adaptive motion planning and tracking. In *Conference on Robot Learning (CoRL)*, 2026a.

W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. VoxPoser: Composable 3D value maps for robotic manipulation with language models. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 540–562. PMLR, 2023a.

W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Jackson, N. Brown, L. Luu, S. Levine, K. Hausman, and B. Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 1769–1782. PMLR, 2023b.

Z. Huang, Y. Li, J. Chiu, X. Lyu, M. Zhou, Y. Yao, J. Lasenby, and S. Wu. LiteReality-Agent: An agentic system for interactable 3D indoor scene reconstruction, 2026b. URL https://arxiv.org/abs/2610.01863.

HugWBC Authors. HugWBC: Official implementation, 2025. URL https://github.com/InternRobotics/HugWBC/tree/eb831af76ea70ebe5d435786b3bbebee13c75bbd. Commit eb831af, 2 August 2025.

J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter. Learning agile and dynamic motor skills for legged robots. *Science robotics*, 4(26):eaau5872, 2019.

B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A. T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, O. Cortes, N. Sievers, C. Tan, S. Xu, D. Reyes, J. Rettinghouse, J. Quiambao, P. Pastor, L. Luu, K.-H. Lee, Y. Kuang, S. Jesmonth, N. J. Joshi, K. Jeffrey, R. J. Ruano, J. Hsu, K. Gopalakrishnan, B. David, A. Zeng, and C. K. Fu. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 287–318. PMLR, 2023.

IEC. Asset Administration Shell for Industrial Applications—Part 1: Asset Administration Shell Structure. International Standard IEC 63278-1:2023, International Electrotechnical Commission, Dec. 2023. URL `https://webstore.iec.ch/en/publication/65628`. Edition 1.0.

International Electrotechnical Commission. Adjustable speed electrical power drive systems – Part 5-2: Safety requirements – Functional. IEC 61800-5-2:2016, 2016. URL `https://webstore.iec.ch/en/publication/24556`. Edition 2.0.

International Organization for Standardization. *ISO 12100:2010: Safety of Machinery—General Principles for Design—Risk Assessment and Risk Reduction*. ISO, 2010. URL `https://www.iso.org/standard/51528.html`.

International Organization for Standardization. *ISO 13482:2014: Robots and Robotic Devices—Safety Requirements for Personal Care Robots*. ISO, 2014. URL `https://www.iso.org/standard/53820.html`.

International Organization for Standardization. *ISO 13850:2015: Safety of Machinery—Emergency Stop Function—Principles for Design*. ISO, 2015. URL `https://www.iso.org/standard/59970.html`.

International Organization for Standardization. ISO/TR 23482-1:2020: Robotics—application of ISO 13482—part 1: Safety-related test methods. Technical report, ISO, 2020. URL `https://www.iso.org/standard/71564.html`.

International Organization for Standardization. Safety of machinery—safety-related parts of control systems—part 1: General principles for design. Technical Report ISO 13849-1:2023, International Organization for Standardization, 2023a. URL `https://www.iso.org/standard/73481.html`.

International Organization for Standardization. *ISO 13849-1:2023: Safety of Machinery—Safety-Related Parts of Control Systems—Part 1: General Principles for Design*. ISO, 2023b. URL `https://www.iso.org/standard/73481.html`.

International Organization for Standardization. *ISO 10218-1:2025: Robotics—Safety Requirements—Part 1: Industrial Robots*. ISO, 2025. URL `https://www.iso.org/standard/73933.html`.

L. Iocchi, D. Holz, J. Ruiz-del Solar, K. Sugiura, and T. van der Zant. RoboCup@Home: Analysis and results of evolving competitions for domestic and service robots. *Artificial Intelligence*, 229: 258–281, 2015.

M. Iovino, E. Scukins, J. Styrud, P. Ögren, and C. Smith. A survey of behavior trees in robotics and AI. *Robotics and Autonomous Systems*, 154:104096, 2022.

M. Iskandar, A. Albu-Schäffer, and A. Dietrich. Intrinsic sense of touch for intuitive physical human-robot interaction. *Science Robotics*, 9(93):eadn4008, 2024.

S. James, Z. Ma, D. Rovick Arrojo, and A. J. Davison. RLBench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.

J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, Y. Fang, F. Hu, S. Huang, K. Kundalia, Y.-C. Lin, L. Magne, A. Mandlekar, A. Narayan, Y. L. Tan, G. Wang, J. Wang, Q. Wang, Y. Xu, X. Zeng, K. Zheng, R. Zheng, M.-Y. Liu, L. Zettlemoyer, D. Fox, J. Kautz, S. Reed, Y. Zhu, and L. Fan. DreamGen: Unlocking generalization in robot learning through video world models. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 5170–5194. PMLR, 2025.

M. Janner, J. Fu, M. Zhang, and S. Levine. When to trust your model: Model-based policy optimization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

A. K. S. Jardine, D. Lin, and D. Banjevic. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510, 2006.

K. Javed and R. S. Sutton. The big world hypothesis and its ramifications for artificial intelligence. OpenReview preprint, 2024. URL `https://openreview.net/forum?id=Sv7DazuCn8`.

F. Jenelten, J. He, F. Farshidian, and M. Hutter. Dtc: Deep tracking control. *Science Robotics*, 9(86): eadh5401, 2024.

Jet Propulsion Laboratory. NASA's self-driving Perseverance Mars rover "takes the wheel". NASA, 2021. URL `https://www.nasa.gov/solar-system/nasas-self-driving-perseverance-mars-rover-takes-the-wheel/`. Published 1 July 2021.

J. H. Jiang, F. Feng, P. E. Rosen, K. A. Fahy, P. Das, P. Obacz, A. Zhang, and Z.-H. Zhu. Avoiding the Great Filter: Predicting the timeline for humanity to reach Kardashev Type I civilization. *Galaxies*, 10(3), 2022.

Y. Jiang, Y. Chebotar, R. Zheng, F. Hu, Y. Ge, J. Wu, T. Dai, S. Reed, L. Fei-Fei, Y. Zhu, and L. J. Fan. RoboTTT: Context scaling for robot policies, 2026. URL `https://arxiv.org/abs/2607.15275`.

Z. Jiang, Y. Xie, K. Lin, Z. Xu, W. Wan, A. Mandlekar, L. Fan, and Y. Zhu. DexMimicGen: Automated data generation for bimanual dexterous manipulation via imitation learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 16923–16930, 2025.

M. Johnson, J. M. Bradshaw, P. J. Feltovich, C. M. Jonker, M. B. van Riemsdijk, and M. Sierhuis. Coactive Design: Designing support for interdependence in joint activity. *Journal of Human-Robot Interaction*, 3(1):43–69, 2014.

D. Jones, C. Snider, A. Nassehi, J. Yon, and B. Hicks. Characterising the digital twin: A systematic literature review. *CIRP Journal of Manufacturing Science and Technology*, 29:36–52, 2020.

K. Justmann, L. Leurs, and M. Ruskowski. Mapping of OPC UA FX to the asset administration shell and capability, skill and service model. In *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–4. IEEE, 2024.

L. P. Kaelbling and T. Lozano-Pérez. Hierarchical task and motion planning in the now. In *2011 IEEE International Conference on Robotics and Automation*, pages 1470–1477. IEEE, 2011.

L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

U. Kafader. Continuous operation range of BLDC (EC) motors. maxon Support, 2024. URL `https://support.maxongroup.com/hc/en-us/articles/360000350014-Continuous-operation-range-of-BLDC-EC-Motors`. Updated 29 August 2024.

P. Kaifosh, T. R. Reardon, and CTRL-labs at Reality Labs. A generic non-invasive neuromotor interface for human-computer interaction. *Nature*, 645(8081):702–711, 2025.

S. Kajita, F. Kanehiro, K. Kaneko, K. Fujiwara, K. Harada, K. Yokoi, and H. Hirukawa. Biped walking pattern generation by using preview control of zero-moment point. In *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*, volume 2, pages 1620–1626. IEEE, 2003.

D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke, and S. Levine. QT-Opt: Scalable deep reinforcement learning for vision-based robotic manipulation. In *Conference on Robot Learning (CoRL)*, 2018.

R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960a.

R. E. Kalman. Contributions to the theory of optimal control. *Boletín de la Sociedad Matemática Mexicana*, 5(2):102–119, 1960b.

S. S. Kannan, V. L. Venkatesh, and B.-C. Min. SMART-LLM: Smart multi-agent robot task planning using large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12140–12147. IEEE, 2024.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling Laws for Neural Language Models, 2020. URL `https://arxiv.org/abs/2001.08361`.

N. S. Kardashev. Transmission of information by extraterrestrial civilizations. *Soviet Astronomy*, 8 (2):217–221, 1964.

S. Kareer, D. Patel, R. Punamiya, P. Mathur, S. Cheng, C. Wang, J. Hoffman, and D. Xu. EgoMimic: Scaling imitation learning via egocentric video. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 13226–13233, 2025.

G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.

B. Katz, J. Di Carlo, and S. Kim. Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 6295–6301, 2019.

J. Kay. Introduction to real-time systems. ROS 2 Design, 2016. URL `https://design.ros2.org/articles/realtime_background.html`.

K. Kedia, T. G. W. Lum, J. Bohg, and C. K. Liu. SimToolReal: An object-centric policy for zero-shot dexterous tool manipulation. In *Proceedings of Robotics: Science and Systems*, 2026.

A. Kendall and Y. Gal. What uncertainties do we need in Bayesian deep learning for computer vision? In *Advances in Neural Information Processing Systems*, volume 30, 2017.

B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4):1–14, 2023.

L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, and A. Bhowmik. Intel (r) realsense (tm) stereoscopic depth cameras. In *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 1267–1276. IEEE, 2017.

H. K. Khalil. *Nonlinear Systems*. Prentice Hall, Upper Saddle River, NJ, 3 edition, 2002. ISBN 9780130673893.

W. Khalil and E. Dombre. *Modeling, identification and control of robots*. Butterworth-Heinemann, 2004.

O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.

O. Khatib, X. Yeh, G. Brantner, B. Soe, B. Kim, S. Ganguly, H. Stuart, S. Wang, M. Cutkosky, A. Edsinger, P. Mullins, M. Barham, C. R. Voolstra, K. N. Salama, M. L'Hour, and V. Creuze. Ocean one: A robotic avatar for oceanic discovery. *IEEE Robotics & Automation Magazine*, 23(4): 20–29, 2016.

A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Lee, Y. Lee, A. Jain, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. Hatch, S. Lin, J. Lu, A. Rehman, P. R. Sanketi, A. Sharma, C. Simpson, Q. Vuong, H. Walke, B. Wulfe, T. Xiao, J. Yang, A. Yavary, T. Z. Zhao, C. Agia, R. Baijal, M. G. Castro, D. Chen, Q. Chen, T. Chung, J. Drake, E. P. Foster, J. Gao, D. A. Herrera, M. Heo, K. Hsu, J. Hu, D. Jackson, C. Le, Y. Li, K. Lin, R. Lin, Z. Ma, A. Maddukuri, S. Mirchandani, D. Morton, T. Nguyen, A. O'Neill, R. Scalise, D. Seale, V. Son, S. Tian, A. Wang, Y. Wu, A. Xie, J. Yang, P. Yin, Y. Zhang, O. Bastani, G. Berseth, J. Bohg, K. Goldberg, A. Gupta, A. Gupta, D. Jayaraman, J. J. Lim, J. Malik, R. Martín-Martín, S. Ramamoorthy, D. Sadigh, S. Song, J. Wu, Y. Zhu, T. Kollar, S. Levine, and C. Finn. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024.

M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. In *Proceedings of Robotics: Science and Systems*, 2025a.

M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR, 2025b.

M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, and J. Gu. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. In *The Fourteenth International Conference on Learning Representations*, 2026a.

S. Kim, S. Jang, B. Yoon, D. Kim, J. Won, and J. Shin. RoboCurate: Harnessing diversity with action-verified neural trajectory for robot learning, 2026b. URL https://arxiv.org/abs/2602.18742.

K. Kimble, K. Van Wyk, J. Falco, E. Messina, Y. Sun, M. Shibata, W. Uemura, and Y. Yokokohji. Benchmarking protocols for evaluating small parts robotic assembly systems. *IEEE Robotics and Automation Letters*, 5(2):883–889, 2020.

R. D. King, J. Rowland, S. G. Oliver, M. Young, W. Aubrey, E. Byrne, M. Liakata, M. Markham, P. Pir, L. N. Soldatova, A. Sparkes, K. E. Whelan, and A. Clare. The automation of science. *Science*, 324 (5923):85–89, 2009.

D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014.

A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Segment anything. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 3992–4003. IEEE, 2023.

J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

H. Kitano. Nobel turing challenge: creating the engine for scientific discovery. *npj Systems Biology and Applications*, 7:29, 2021.

D. P. Klüner, L. Hegerath, A. D. Hatib, S. Kowalewski, B. Alrifaee, and A. Kampmann. Automotive middleware performance: Comparison of FastDDS, Zenoh and vSomeIP. In *2025 IEEE International Conference on Vehicular Electronics and Safety (ICVES)*, pages 303–310, 2025.

T. Kobayashi. L2C2: Locally Lipschitz continuous constraint towards stable and smooth reinforcement learning. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4032–4039, 2022.

J. Kober, J. A. Bagnell, and J. Peters. Reinforcement learning in robotics: A survey. *The International Journal of Robotics Research*, 32(11):1238–1274, 2013.

P. W. Koh, S. Sagawa, H. Marklund, S. M. Xie, M. Zhang, A. Balsubramani, W. Hu, M. Yasunaga, R. L. Phillips, I. Gao, T. Lee, E. David, I. Stavness, W. Guo, B. Earnshaw, I. Haque, S. M. Beery, J. Leskovec, A. Kundaje, E. Pierson, S. Levine, C. Finn, and P. Liang. WILDS: A benchmark of in-the-wild distribution shifts. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 5637–5664. PMLR, 2021.

Kollmorgen. How does a servo motor work?, Sept. 2020. URL `https://www.kollmorgen.com/en-us/blogs/how-servo-motors-work`. Published 29 September 2020.

L. Kong, S. Xie, H. Hu, L. X. Ng, B. R. Cottereau, and W. T. Ooi. RoboDepth: Robust out-of-distribution depth estimation under corruptions. In *Advances in Neural Information Processing Systems*, volume 36, pages 21298–21342, 2023.

M. V. Kothare, P. J. Campo, M. Morari, and C. N. Nett. A unified framework for the study of anti-windup designs. *Automatica*, 30(12):1869–1883, 1994.

H. Kress-Gazit, G. E. Fainekos, and G. J. Pappas. Temporal-logic-based reactive mission and motion planning. *IEEE Transactions on Robotics*, 25(6):1370–1381, 2009.

A. Kumar, Z. Fu, D. Pathak, and J. Malik. RMA: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems*, 2021.

M. Kyrarini, F. Lygerakis, A. Rajavenkatanarayanan, C. Sevastopoulos, H. R. Nambiappan, K. K. Chaitanya, A. R. Babu, J. Mathew, and F. Makedon. A survey of robots in healthcare. *Technologies*, 9(1):8, 2021.

B. M. Lake and M. Baroni. Human-like systematic generalization through a meta-learning neural network. *Nature*, 623:115–121, 2023.

B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

M. Lambeta, P.-W. Chou, S. Tian, B. Yang, B. Maloon, V. R. Most, D. Stroud, R. Santos, A. Byagowi, G. Kammerer, D. Jayaraman, and R. Calandra. DIGIT: A novel design for a low-cost compact high-resolution tactile sensor with application to in-hand manipulation. *IEEE Robotics and Automation Letters*, 5(3):3838–3845, 2020.

C. Laschi, B. Mazzolai, and M. Cianchetti. Soft robotics: Technologies and systems pushing the boundaries of robot abilities. *Science Robotics*, 1(1):eaah3690, 2016.

L. Le, J. Xie, W. Liang, H.-J. Wang, Y. Yang, Y. J. Ma, K. Vedder, A. Krishna, D. Jayaraman, and E. Eaton. Articulate-Anything: Automatic modeling of articulated objects via a vision-language foundation model. In *International Conference on Learning Representations (ICLR)*, 2025.

Y. LeCun. A path towards autonomous machine intelligence. Open Review, 2022. URL `https://openreview.net/forum?id=BZ5a1r-kVsf`.

Y. LeCun, Y. Bengio, and G. Hinton. Deep learning. *Nature*, 521:436–444, 2015.

E. A. Lee. Cyber physical systems: Design challenges. Technical Report UCB/EECS-2008-8, University of California, Berkeley, 2008. URL `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-8.html`.

J. Lee, B. Bagheri, and H.-A. Kao. A cyber-physical systems architecture for Industry 4.0-based manufacturing systems. *Manufacturing Letters*, 3:18–23, 2015.

J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47):eabc5986, 2020.

S. Lee, T. Kim, A. Corsaro, and K.-J. Park. The three dimensions of ROS 2 middleware, 2026. URL `https://arxiv.org/abs/2607.01304`.

W. Lee, J. Sim, J. Kim, S. Jo, W. Luo, and C. Nam. Merry-go-round: Safe control of decentralized multi-robot systems with deadlock prevention. In *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 4589–4595. IEEE, 2025.

Y. Lei, M. Liu, A. Maddukuri, Z. Jiang, and Y. Zhu. A mechanistic analysis of Sim-and-Real co-training in generative robot policies, 2026. URL `https://arxiv.org/abs/2604.13645`.

T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez. Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, 58:52–68, 2020.

N. G. Leveson. A new accident model for engineering safer systems. *Safety Science*, 42(4):237–270, 2004.

S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, 37(4–5):421–436, 2018.

F. L. Lewis, D. L. Vrabie, and V. L. Syrmos. *Optimal Control*. John Wiley & Sons, Hoboken, NJ, 3 edition, 2012. ISBN 9780470633496.

C. Li, J. Wen, Y. Peng, Y. Peng, and Y. Zhu. PointVLA: Injecting the 3D world into vision-language-action models. *IEEE Robotics and Automation Letters*, 11(3):2506–2513, 2026a.

F. Li, W. Song, H. Zhao, J. Wang, P. Ding, D. Wang, L. Zeng, and H. Li. Spatial forcing: Implicit spatial representation alignment for vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 132324–132345, 2026b.

K. Li, P. Li, T. Liu, Y. Li, and S. Huang. Maniptrans: Efficient dexterous bimanual manipulation transfer via residual learning. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6991–7003. IEEE, 2025a.

M. Li, Z. Ferguson, T. Schneider, T. Langlois, D. Zorin, D. Panozzo, C. Jiang, and D. M. Kaufman. Incremental potential contact: Intersection- and inversion-free large deformation dynamics. *ACM Transactions on Graphics*, 39(4):1–20, 2020.

M. Li, S. Zhao, Q. Wang, K. Wang, Y. Zhou, S. Srivastava, C. Gokmen, T. Lee, L. Li, R. Zhang, W. Liu, P. Liang, L. Fei-Fei, J. Mao, and J. Wu. Embodied Agent Interface: Benchmarking LLMs for embodied decision making. In *Advances in Neural Information Processing Systems*, volume 37, pages 100428–100534, 2024.

P. Li, Z. Chen, Y. Wu, P. Wei, Y. Li, T. Wang, J. Shi, M. Yu, B. Jia, S.-C. Zhu, T. Liu, and S. Huang. Towards human-level dexterous teleoperation, 2026c. URL `https://arxiv.org/abs/2607.11481v1`.

S. Li, R. Batra, D. Brown, H.-D. Chang, N. Ranganathan, C. Hoberman, D. Rus, and H. Lipson. Particle robotics based on statistical mechanics of loosely coupled components. *Nature*, 567:361–365, 2019a.

X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, S. Levine, J. Wu, C. Finn, H. Su, Q. Vuong, and T. Xiao. Evaluating real-world robot manipulation policies in simulation. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 3705–3728. PMLR, 2025b.

Y. Li, G. Carboni, F. Gonzalez, D. Campolo, and E. Burdet. Differential game theory for versatile physical human–robot interaction. *Nature Machine Intelligence*, 1(1):36–43, 2019b.

Y. Li, Y. Feng, S. Sun, J. Chen, H. Ren, B. Yang, W. Xu, Q. Wu, H. Cheng, and R. Xu. Agentic RSR: Real-to-sim-to-real through scene reconstruction and execution-grounded robot policies, 2026d. URL https://arxiv.org/abs/2610.10479.

Y. Li, M. Zhou, D. Zhou, Z. Ma, H. Wang, C. Zeng, D. Wang, Y. Zhou, Q. Yu, B. Jiang, Z. Cao, Z. Zhu, Z. Guo, X. Li, D. Feng, X. Yu, Y. Tang, J. Li, J. Cao, Z. Wang, Q. Zhang, L. Yin, X. Xu, Z. Lv, K. Li, M. Tao, D. An, T. Lang, S. Zheng, L. Tian, X. Chen, Y. Xu, Y. Mu, X. Yang, F. Shi, S. Wang, C. Lu, R. Wang, Z. Wang, M. Yao, G. Wang, X. Li, S. Zhang, H. Zhou, Y. Shen, J. Zhang, X. Fu, J. Meng, T. Xue, B. Zhao, C. Shen, L. Bai, W. Zhang, and B. Zhou. InternLab: A governed network of physically intelligent autonomous laboratories for open-world scientific discovery. Zenodo preprint, July 2026e.

Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE, 2023a.

W.-Y. Liang, Y. Yuan, and H.-J. Lin. A performance study on the throughput and latency of Zenoh, MQTT, Kafka, and DDS, 2023b. URL https://arxiv.org/abs/2303.09419.

Q. Liao, T. E. Truong, X. Huang, Y. Gao, G. Tevet, K. Sreenath, and C. K. Liu. BeyondMimic: From motion tracking to versatile humanoid control via guided diffusion. *Science Robotics*, 11(117): eadx8924, 2026.

D. Liberzon. *Switching in Systems and Control*. Birkhäuser, Boston, MA, 2003. ISBN 9781461200178.

F. Lin, Y. Hu, P. Sheng, C. Wen, J. You, and Y. Gao. Data scaling laws in imitation learning for robotic manipulation. In *International Conference on Learning Representations (ICLR)*, 2025.

X. Lin, G. Zhong, T. Lu, Z. Ye, Y. Zhu, Z. Wu, and Y.-G. Jiang. ActiveMimic: Egocentric video pretraining with active perception, 2026. URL https://arxiv.org/abs/2606.06194.

Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

H. L. Litaker, Jr., K. H. Beaton, O. S. Bekdash, E. Z. Crues, E. J. Paddock, B. A. J. Rohde, M. L. Reagan, C. M. Van Velson, and S. F. Everett. South pole lunar lighting studies for driving exploration on the lunar surface. In *2025 IEEE Aerospace Conference*, pages 1–16. IEEE, 2025.

M. L. Littman, R. S. Sutton, and S. P. Singh. Predictive representations of state. In *Advances in Neural Information Processing Systems*, volume 14. MIT Press, 2002.

B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 44776–44791, 2023a.

H. Liu, S. Nasiriany, L. Zhang, Z. Bao, and Y. Zhu. Robot learning on the job: Human-in-the-loop autonomy and learning during deployment. In *Robotics: Science and Systems (RSS)*, 2023b.

H. Liu, Y. Zhang, V. Betala, E. Zhang, J. Liu, C. Ding, and Y. Zhu. Multi-Task Interactive Robot Fleet Learning with Visual World Models. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4286–4313. PMLR, 2025a.

K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao. Coherent: Collaboration of heterogeneous multi-robot system with large language models. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 10208–10214. IEEE, 2025b.

S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, pages 38–55. Springer, 2024.

S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu. RDT-1B: a diffusion foundation model for bimanual manipulation. In *International Conference on Learning Representations*, volume 2025, pages 29982–30009, 2025c.

Y. Liu, W. Chen, Y. Bai, X. Liang, G. Li, W. Gao, and L. Lin. Aligning cyber space with physical world: A comprehensive survey on embodied AI. *IEEE/ASME Transactions on Mechatronics*, 30 (6):7253–7274, 2025d.

Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. L. Rus, and S. Han. Bevfusion: Multi-task multi-sensor fusion with unified bird's-eye view representation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2774–2781. ieee, 2023c.

A. C. Lo, P. D. Guarino, L. G. Richards, J. K. Haselkorn, G. F. Wittenberg, D. G. Federman, R. J. Ringer, T. H. Wagner, H. I. Krebs, B. T. Volpe, C. T. Bever, D. M. Bravata, P. W. Duncan, B. H. Corn, A. D. Maffucci, S. E. Nadeau, S. S. Conroy, J. M. Powell, G. D. Huang, and P. Peduzzi. Robot-assisted therapy for long-term upper-limb impairment after stroke. *New England Journal of Medicine*, 362(19):1772–1783, 2010.

F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems*, volume 33, pages 11525–11538, 2020.

J. Long, Z. Wang, Q. Li, J. Gao, L. Cao, and J. Pang. Hybrid internal model: Learning agile legged locomotion with simulated robot response. In *International Conference on Learning Representations (ICLR)*, 2024.

J. Long, J. Ren, M. Shi, Z. Wang, T. Huang, P. Luo, and J. Pang. Learning humanoid locomotion with perceptive internal model. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9997–10003, 2025a.

J. Long, W. Yu, Q. Li, Z. Wang, D. Lin, and J. Pang. Learning H-infinity locomotion control. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 1094–1108. PMLR, 2025b.

D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza. Learning high-speed flight in the wild. *Science Robotics*, 6(59):eabg5810, 2021.

G. Lu, R. Zhao, H. Lin, H. Zhang, and Y. Tang. Human-in-the-loop online rejection sampling for robotic manipulation, 2025. URL `https://arxiv.org/abs/2510.26406`.

T. G. W. Lum, K. Kedia, C. K. Liu, and J. Bohg. Play2Perfect: What matters in dexterous play pretraining for precise assembly?, 2026. URL `https://arxiv.org/abs/2606.26428v1`.

M. Lungarella and L. Berthouze. On the interplay between morphological, neural, and environmental dynamics: A robotic case study. *Adaptive Behavior*, 10(3–4):223–241, 2002.

M. Lungarella, G. Metta, R. Pfeifer, and G. Sandini. Developmental robotics: A survey. *Connection Science*, 15(4):151–190, 2003.

J. Luo, C. Xu, J. Wu, and S. Levine. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. *Science Robotics*, 10(105):eads5033, 2025.

Z. Luo, Y. Yuan, T. Wang, C. Li, F. Castañeda, S. Chen, Z.-A. Cao, J. Li, D. Minor, Q. Ben, J. Park, D. Sami, Z. Wang, X. Da, R. Ding, C. Hogg, L. Song, E. Lim, E. Jeong, T. He, H. Xue, W. Xiao, S. Yuen, J. Kautz, Y. Chang, U. Iqbal, L. J. Fan, and Y. Zhu. SONIC: Supersizing motion tracking for natural humanoid whole-body control. *Science Robotics*, 11(117):eaed4592, 2026.

K. Ly, N. Kellaris, D. McMorris, B. K. Johnson, E. Acome, V. Sundaram, M. Naris, J. S. Humbert, M. E. Rentschler, C. Keplinger, and N. Correll. Miniaturized circuitry for capacitive self-sensing and closed-loop control of soft electrostatic transducers. *Soft Robotics*, 8(6):673–686, 2021.

K. M. Lynch and F. C. Park. *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017. ISBN 978-1-107-15630-2.

F. Ma and S. Karaman. Sparse-to-dense: Depth prediction from sparse depth samples and a single image. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4796–4803. IEEE, 2018.

H. Ma, J. Cai, X. Xu, H. Li, Y. Yang, Y. Tian, J. Cao, H. Zhu, Z. Qiu, Zhaxizhuoma, Y. Yang, J. Peng, X. Wei, Y. Zhu, J. Jiang, X. Gao, H. Wang, F. Yuan, K. Li, X. Zhu, T. Wang, Y. Ding, J. Pang, J. Zeng, J. Zhang, B. Zhou, Y. Mu, C. Shen, and W. Zhang. InternVLA-A1.5: Unifying understanding, latent foresight, and action for compositional generalization, 2026a. URL `https://arxiv.org/abs/2607.04988`.

Y. Ma, Z. Chen, E. Zhou, Z. Liu, H. Zhang, K. Liu, and H. Fan. Table30 V2: Evaluating generalized models by real robots at scale. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 4461–4467, 2026b.

S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot Operating System 2: Design, architecture, and uses in the wild. *Science robotics*, 7(66):eabm6074, 2022.

B. P. MacLeod, F. G. L. Parlane, T. D. Morrissey, F. Häse, L. M. Roch, K. E. Dettelbach, R. Moreira, L. P. E. Yunker, M. B. Rooney, J. R. Deeth, V. Lai, G. J. Ng, H. Situ, R. H. Zhang, M. S. Elliott, T. H. Haley, D. J. Dvorak, A. Aspuru-Guzik, J. E. Hein, and C. P. Berlinguette. Self-driving laboratory for accelerated discovery of thin-film materials. *Science Advances*, 6(20):eaaz8867, 2020.

A. Maddukuri, Z. Jiang, L. Y. Chen, S. Nasiriany, Y. Xie, Y. Fang, W. Huang, Z. Wang, Z. Xu, N. Chernyadev, S. Reed, K. Goldberg, A. Mandlekar, L. Fan, and Y. Zhu. Sim-and-Real co-training: A simple recipe for vision-based robotic manipulation. In *Proceedings of Robotics: Science and Systems*, 2025.

V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State. Isaac Gym: High performance GPU based physics simulation for robot learning. In J. Vanschoren and S. Yeung, editors, *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021.

Z. Mandi, S. Jain, and S. Song. Roco: Dialectic multi-robot collaboration with large language models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 286–299. IEEE, 2024.

A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín. What matters in learning from offline human demonstrations for robot manipulation. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 1678–1690. PMLR, 2022.

A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox. MimicGen: A data generation system for scalable robot learning using human demonstrations. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1820–1864. PMLR, 2023.

MANUS. From finger movements to digital hands: EMF hand tracking, skeleton data and retargeted motion. MANUS technical documentation, n.d.a.

MANUS. Tracking systems, n.d.b. URL https://docs.manus-meta.com/2.5.0/Software/TrackingSystems/. MANUS Core 2.5.0 Knowledge Center.

G. B. Margolis and P. Agrawal. Walk these ways: Tuning robot control for generalization with multiplicity of behavior. In *Proceedings of The 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 22–31. PMLR, 2023.

J. Marshall, T. Barfoot, and J. Larsson. Autonomous underground tramming for center-articulated vehicles. *Journal of Field Robotics*, 25(6–7):400–421, 2008.

R. Martín-Martín, M. A. Lee, R. Gardner, S. Savarese, J. Bohg, and A. Garg. Variable impedance control in end-effector space: An action space for reinforcement learning in contact-rich tasks. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1010–1017, 2019.

M. T. Mason and J. K. Salisbury Jr. *Robot hands and the mechanics of manipulation*. MIT press, 1985.

E. Matheson, R. Minto, E. G. G. Zampieri, M. Faccio, and G. Rosati. Human–robot collaboration in manufacturing applications: A review. *Robotics*, 8(4):100, 2019.

A. Matthias. The responsibility gap: Ascribing responsibility for the actions of learning automata. *Ethics and Information Technology*, 6(3):175–183, 2004.

D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

T. McGeer. Passive dynamic walking. *The International Journal of Robotics Research*, 9(2):62–82, 1990.

O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

J. Merel, L. Hasenclever, A. Galashov, A. Ahuja, V. Pham, G. Wayne, Y. W. Teh, and N. Heess. Neural probabilistic motor primitives for humanoid control. In *International Conference on Learning Representations*, 2019.

M. Meribout, N. A. Takele, O. Derege, N. Rifiki, M. El Khalil, V. Tiwari, and J. Zhong. Tactile sensors: A review. *Measurement*, 238:115332, 2024.

G. Metta, G. Sandini, and J. Konczak. A developmental approach to visually-guided reaching in artificial systems. *Neural Networks*, 12(10):1413–1427, 1999.

X. Miao, Z. Li, B. Fang, K. Song, T. Wang, H. Zhang, Y. Wang, X. Li, Y. He, X. Wei, C. Gao, X. Yang, Y. Xu, K. Ren, W. Guo, J. Zhou, X. Wang, W. Zhao, N. Yang, Z. Cai, Y. Xue, H. Li, Z. He, Y. Zhou, R. Fu, J. Zhang, S. Cui, F. Huang, Y. Zhou, X. Gao, Y. Yao, Q. Yu, K. Li, M. Zhou, M. Huang, X. Li, W. Cui, B. Jiang, X. Zhu, J. Dong, H. Guo, T. Lu, M. Yu, B. Zhou, B. Zhao, T. Xue, W. Zhang, and C. Shen. InternW0-Δ: A world action model bridging predictive dynamics and actions with 20k+ hours of open data, 2026. URL https://arxiv.org/abs/2609.31394.

T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, and M. Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science robotics*, 7(62):eabk2822, 2022.

B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *Computer Vision – ECCV 2020*, pages 405–421. Springer International Publishing, 2020.

A. Milecki and P. Nowak. Review of fault-tolerant control systems used in robotic manipulators. *Applied Sciences*, 13(4):2675, 2023.

S. Mirchandani, S. Belkhale, J. Hejna, E. Choi, M. S. Islam, and D. Sadigh. So you think you can scale up autonomous robot data collection? In *Proceedings of the 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2791–2810. PMLR, 2025.

S. M. Mirvakili and I. W. Hunter. Artificial muscles: Mechanisms, applications, and challenges. *Advanced Materials*, 30(6):1704407, 2018.

K. Mo, L. J. Guibas, M. Mukadam, A. Gupta, and S. Tulsiani. Where2Act: From pixels to actions for articulated 3D objects. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6813–6823, 2021.

Model Context Protocol. Model Context Protocol Specification: Tools. Open protocol specification, version 2025-11-25, 2025. URL `https://modelcontextprotocol.io/specification/2025-11-25/server/tools`.

L. Monostori, B. Kádár, T. Bauernhansl, S. Kondoh, S. Kumara, G. Reinhart, O. Sauer, G. Schuh, W. Sihn, and K. Ueda. Cyber-physical systems in manufacturing. *CIRP Annals*, 65(2):621–641, 2016.

H. Moravec. *Mind Children: The Future of Robot and Human Intelligence*. Harvard University Press, Cambridge, MA, 1988.

M. R. Morris, J. Sohl-Dickstein, N. Fiedel, T. Warkentin, A. Dafoe, A. Faust, C. Farabet, and S. Legg. Position: Levels of AGI for operationalizing progress on the path to AGI. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 36308–36321. PMLR, 2024.

D. Mrowca, C. Zhuang, E. Wang, N. Haber, L. Fei-Fei, J. B. Tenenbaum, and D. L. K. Yamins. Flexible neural representation for physics prediction. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

N. Muennighoff, A. M. Rush, B. Barak, T. L. Scao, A. Piktus, N. Tazi, S. Pyysalo, T. Wolf, and C. Raffel. Scaling data-constrained language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 50358–50376, 2023.

M. S. Müller, T. Jung, N. Jazdi, and M. Weyrich. Safeguarding autonomous systems: emerging approaches, assumptions and metrics – a systematic literature review. *IFAC-PapersOnLine*, 55(6): 743–754, 2022.

R. R. Murphy. *Disaster Robotics*. MIT Press, 2014.

C. C. Murray and A. G. Chu. The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies*, 54:86–109, 2015.

S. K. Muthalagu, K. Mogensen, G. Gibler, and E. Hanna. Current sensing in humanoid robots. Technical Report SBOA627, Texas Instruments, June 2025. URL `https://www.ti.com/lit/pdf/sboa627`.

Y. Nagai, K. Hosoda, A. Morita, and M. Asada. A constructive model for the development of joint attention. *Connection Science*, 15(4):211–229, 2003.

S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A universal visual representation for robot manipulation. In *Proceedings of the 6th Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 892–909, 2023.

S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. RoboCasa: Large-scale simulation of household tasks for generalist robots. In *Proceedings of Robotics: Science and Systems*, 2024.

S. Nasiriany, S. Nasiriany, A. Maddukuri, and Y. Zhu. RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. In *International Conference on Learning Representations*, 2026.

National Aeronautics and Space Administration. Mars communications disruption and delay. Moon to mars architecture concept review white paper, NASA, 2023. URL https://www.nasa.gov/wp-content/uploads/2024/01/mars-communications-disruption-and-delay.pdf.

B. J. Nelson, I. K. Kaliakatsos, and J. J. Abbott. Microrobots for minimally invasive medicine. *Annual Review of Biomedical Engineering*, 12:55–85, 2010.

V. N. Nguyen, R. Jenssen, and D. Roverso. Automatic autonomous vision-based power line inspection: A review of current status and the potential role of deep learning. *International Journal of Electrical Power & Energy Systems*, 99:107–120, 2018.

S. Nikolaidis and J. Shah. Human-robot cross-training: Computational formulation, modeling and evaluation of a human team training strategy. In *2013 8th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 33–40, 2013.

S. Nikolaidis, A. Kuznetsov, D. Hsu, and S. Srinivasa. Formalizing human-robot mutual adaptation: A bounded memory model. In *2016 11th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 75–82, 2016.

D. Niu, Z. Liu, Z. Wang, B. Shao, Z.-H. Yin, A. Pai, Y. Sharma, S. Saravalle, R. Zheng, J. Wang, R. Punamiya, M. Xu, Y. Xie, Y. Jiang, L. Fu, K. Kallidromitis, M. Gioia, J. Zhang, J. Ge, H. Feng, F. Galasso, W. Zhan, D. M. Chan, Y. Bai, R. Herzig, J. Lei, L. Fei-Fei, K. Goldberg, J. Malik, P. Abbeel, Y. Zhu, D. Xu, L. Fan, and T. Darrell. T-rex: Tactile-reactive dexterous manipulation, 2026a. URL https://arxiv.org/abs/2606.17055.

P. Niu, W. Qu, S. Gu, T. Shi, Y. Li, A. Tawaha, H. Alzahrani, V. Siu, B. Li, C. Wang, J. Zhang, B. Alomair, M. Jin, M. Chen, C. Wang, C. Spanos, and D. Song. Understanding and evaluating Claw-like agent security through a computer-systems lens, 2026b. URL https://arxiv.org/abs/2606.30755.

NVIDIA. EgoScale: Scaling human video to unlock dexterous robot intelligence, 2026. URL https://research.nvidia.com/labs/gear/egoscale/.

NVIDIA, N. Agarwal, A. Ali, M. Bala, Y. Balaji, E. Barker, T. Cai, P. Chattopadhyay, Y. Chen, Y. Cui, Y. Ding, D. Dworakowski, J. Fan, M. Fenzi, F. Ferroni, S. Fidler, D. Fox, S. Ge, Y. Ge, J. Gu, S. Gururani, E. He, J. Huang, J. Huffman, P. Jannaty, J. Jin, S. W. Kim, G. Klár, G. Lam, S. Lan, L. Leal-Taixe, A. Li, Z. Li, C.-H. Lin, T.-Y. Lin, H. Ling, M.-Y. Liu, X. Liu, A. Luo, Q. Ma, H. Mao, K. Mo, A. Mousavian, S. Nah, S. Niverty, D. Page, D. Paschalidou, Z. Patel, L. Pavao, M. Ramezanali, F. Reda, X. Ren, V. R. N. Sabavat, E. Schmerling, S. Shi, B. Stefaniak, S. Tang, L. Tchapmi, P. Tredak, W.-C. Tseng, J. Varghese, H. Wang, H. Wang, H. Wang, T.-C. Wang, F. Wei, X. Wei, J. Z. Wu, J. Xu, W. Yang, L. Yen-Chen, X. Zeng, Y. Zeng, J. Zhang, Q. Zhang, Y. Zhang, Q. Zhao, and A. Zolkowski. Cosmos world foundation model platform for physical AI, 2025a. URL https://arxiv.org/abs/2501.03575.

NVIDIA, H. A. Alhaija, J. Alvarez, M. Bala, T. Cai, T. Cao, L. Cha, J. Chen, M. Chen, F. Ferroni, S. Fidler, D. Fox, Y. Ge, J. Gu, A. Hassani, M. Isaev, P. Jannaty, S. Lan, T. Lasser, H. Ling, M.-Y. Liu, X. Liu, Y. Lu, A. Luo, Q. Ma, H. Mao, F. Ramos, X. Ren, T. Shen, X. Sun, S. Tang, T.-C. Wang, J. Wu, J. Xu, S. Xu, K. Xie, Y. Ye, X. Yang, X. Zeng, and Y. Zeng. Cosmos-Transfer1: Conditional world generation with adaptive multimodal control, 2025b. URL https://arxiv.org/abs/2503.14492.

NVIDIA, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. GR00T N1: An open foundation model for generalist humanoid robots, 2025c. URL https://arxiv.org/abs/2503.14734.

NVIDIA Corporation. *Jetson Orin Nano Series, Jetson Orin NX Series and Jetson AGX Orin Series: Thermal Management*. NVIDIA Jetson Linux Developer Guide, n.d. URL https://docs.nvidia.com/jetson/archives/r36.5/DeveloperGuide/SD/PlatformPowerAndPerformance/JetsonOrinNanoSeriesJetsonOrinNxSeriesAndJetsonAgxOrinSeries.html. Release r36.5.

K. Obata, T. Aoki, T. Horii, T. Taniguchi, and T. Nagai. LiP-LLM: Integrating linear programming and dependency graph with large language models for multi-robot task planning. *IEEE Robotics and Automation Letters*, 10(2):1122–1129, 2025.

Object Management Group. Data Distribution Service (DDS), Version 1.4. Technical Report formal/2015-04-10, Object Management Group, 2015. URL https://www.omg.org/spec/DDS/1.4/PDF.

K. Ogata. *Modern Control Engineering*. Prentice Hall, Upper Saddle River, NJ, 5 edition, 2010. ISBN 978-0136156734.

H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto. Voxblox: Incremental 3D Euclidean signed distance fields for on-board MAV planning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1366–1373. IEEE, 2017.

R. Olfati-Saber, J. A. Fax, and R. M. Murray. Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*, 95(1):215–233, 2007.

OPC Foundation. OPC UA for Laboratory & Analytical Device Standard (LADS)—Part 1: Basics. OPC UA Companion Specification OPC 30500-1, OPC Foundation, Nov. 2023. URL https://reference.opcfoundation.org/specs/OPC-30500-1/v1.0.0. Release 1.0.0, 30 November 2023.

Open X-Embodiment Collaboration. Open X-Embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, IEEE, 2024.

OpenAI. GPT-6 Astra: A new generation of intelligence. OpenAI research and product announcement, September 2026, 2026. URL https://openai.com/index/gpt-6-astra/.

OpenAI, I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, J. Schneider, N. Tezak, J. Tworek, P. Welinder, L. Weng, Q. Yuan, W. Zaremba, and L. Zhang. Solving Rubik's cube with a robot hand, 2019. URL https://arxiv.org/abs/1910.07113.

OpenAI, M. Andrychowicz, B. Baker, M. Chociej, R. Józefowicz, B. McGrew, J. Pachocki, A. Petron, M. Plappert, G. Powell, A. Ray, J. Schneider, S. Sidor, J. Tobin, P. Welinder, L. Weng, and W. Zaremba. Learning dexterous in-hand manipulation. *The International Journal of Robotics Research*, 39(1):3–20, 2020.

J. K. O'Regan and A. Noë. A sensorimotor account of vision and visual consciousness. *Behavioral and Brain Sciences*, 24(5):939–973, 2001.

Organisation for Economic Co-operation and Development. Recommendation of the council on artificial intelligence. OECD Legal Instruments, 2019. URL https://legalinstruments.oecd.org/en/instruments/OECD-LEGAL-0449. OECD/LEGAL/0449.

Oriental Motor. Stepper motor basics, n.d. URL https://www.orientalmotor.com/stepper-motors/technology/stepper-motor-basics.html.

P. Ortega-Kral, E. Xing, A. Bucker, V. Luk, J. Kim, O. Kwon, A. Xie, N. Sobanbabu, Y. Yuan, M. Lee, D. Ameria, B. Ayapilla, J. Bussell, G. Shi, J. Francis, and J. Oh. RIO: Flexible real-time robot I/O for cross-embodiment robot learning. In *Proceedings of Robotics: Science and Systems*, 2026.

P.-Y. Oudeyer, F. Kaplan, and V. V. Hafner. Intrinsic motivation systems for autonomous mental development. *IEEE Transactions on Evolutionary Computation*, 11(2):265–286, 2007.

B. Paliwal, H. Etukuru, W. Liang, P. Abbeel, N. M. M. Shafiullah, and J. Malik. Do as I Do: Dexterous manipulation data from everyday human videos, 2026. URL https://arxiv.org/abs/2606.19333v1.

M. Pan, S. Feng, Q. Zhang, X. Li, J. Song, C. Qu, Y. Wang, C. Li, Z. Xiong, Z. Chen, Y. Liu, and J. Luo. SOP: A scalable online post-training system for Vision-Language-Action models, 2026. URL https://arxiv.org/abs/2601.03044.

R. Parasuraman, T. B. Sheridan, and C. D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics – Part A: Systems and Humans*, 30(3):286–297, 2000.

G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter. Continual lifelong learning with neural networks: A review. *Neural Networks*, 113:54–71, 2019.

F. C. Park and B. J. Martin. Robot sensor calibration: solving $AX = XB$ on the Euclidean group. *IEEE Transactions on Robotics and Automation*, 10(5):717–721, 1994.

L. E. Parker. Distributed intelligence: Overview of the field and its application in multi-robot systems. In *AAAI fall symposium: regarding the intelligence in distributed intelligent systems*, pages 1–6, 2007.

Participants Attending the Leaders' Session of the AI Seoul Summit. Seoul Declaration for Safe, Innovative and Inclusive AI, 2024. URL https://www.gov.uk/government/publications/seoul-declaration-for-safe-innovative-and-inclusive-ai-ai-seoul-summit-2024.

D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell. Curiosity-driven exploration by self-supervised prediction. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2778–2787, 2017.

C. Paul. Morphological computation: A basis for the analysis of morphology and control requirements. *Robotics and Autonomous Systems*, 54(8):619–630, 2006.

T. Pearce and J. Song. Reconciling Kaplan and Chinchilla scaling laws. *Transactions on Machine Learning Research*, 2024.

T. Pearce, T. Rashid, D. Bignell, R. Georgescu, S. Devlin, and K. Hofmann. Scaling laws for pretraining agents and world models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48542–48562, 2025.

S. Peng, K. Genova, C. Jiang, A. Tagliasacchi, M. Pollefeys, and T. Funkhouser. OpenScene: 3D scene understanding with open vocabularies. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 815–824. IEEE, 2023.

X. B. Peng, P. Abbeel, S. Levine, and M. Van de Panne. DeepMimic: Example-guided deep reinforcement learning of physics-based character skills. *ACM Transactions On Graphics (TOG)*, 37(4):1–14, 2018a.

X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel. Sim-to-real transfer of robotic control with dynamics randomization. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3803–3810, 2018b.

X. B. Peng, Z. Ma, P. Abbeel, S. Levine, and A. Kanazawa. AMP: Adversarial motion priors for stylized physics-based character control. *ACM Transactions on Graphics (ToG)*, 40(4):1–20, 2021.

X. B. Peng, Y. Guo, L. Halper, S. Levine, and S. Fidler. ASE: Large-scale reusable adversarial skill embeddings for physically simulated characters. *ACM Transactions On Graphics (TOG)*, 41(4):1–17, 2022.

K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. FAST: Efficient action tokenization for vision-language-action models. In *Proceedings of Robotics: Science and Systems*, 2025.

O. Pettersson. Execution monitoring in robotics: A survey. *Robotics and Autonomous Systems*, 53(2): 73–88, 2005.

N. Pfaff, T. Cohn, S. Zakharov, R. Cory, and R. Tedrake. SceneSmith: Agentic generation of simulation-ready indoor scenes. In *Proceedings of the 43rd International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 98499–98567, 2026.

T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.

R. Pfeifer and J. Bongard. *How the Body Shapes the Way We Think: A New View of Intelligence*. The MIT Press, 2006. ISBN 9780262162395.

R. Pfeifer and F. Iida. Embodied artificial intelligence: Trends and challenges. In F. Iida, R. Pfeifer, L. Steels, and Y. Kuniyoshi, editors, *Embodied Artificial Intelligence*, volume 3139 of *Lecture Notes in Computer Science*, pages 1–26. Springer, 2004.

R. Pfeifer, M. Lungarella, and F. Iida. Self-organization, embodiment, and biologically inspired robotics. *Science*, 318(5853):1088–1093, 2007.

Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky. $\pi_{0.5}$: A vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR, 2025.

Physical Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, V. Choudhary, F. Collins, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, M. Dhaka, J. DiCarlo, D. Driess, M. Equi, A. Esmail, Y. Fang, C. Finn, C. Glossop, T. Godden, I. Goryachev, L. Groom, H. Habeeb, H. Hancock, K. Hausman, G. Hussein, V. Hwang, B. Ichter, C. Jacobsen, S. Jakubczak, R. Jen, T. Jones, G. Kammerer, B. Katz, L. Ke, M. Khadikov, C. Kuchi, M. Lamb, D. LeBlanc, B. LeCount, S. Levine, X. Li, A. Li-Bell, V. Lialin, Z. Liang, W. Lim, Y. Lu, E. Luo, V. Mano, N. Marwaha, A. Mongush, L. Murphy, S. Nair, T. Patterson, K. Pertsch, A. Z. Ren, G. Schelske, C. Sharma, B. Shi, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, W. Stoeckle, J. Tang, J. Tanner, S. Tekeste, M. Torne, K. Vedder, Q. Vuong, A. Walling, H. Wang, J. Wang, X. Wang, C. Whalen, S. Whitmore, B. Williams, C. Xu, S. Yoo, L. Yu, W. Zhang, Z. Zhang, and U. Zhilinsky. $\pi_{0.7}$: A steerable generalist robotic foundation model with emergent capabilities, 2026. URL `https://arxiv.org/abs/2604.15483`.

T. Porian, M. Wortsman, J. Jitsev, L. Schmidt, and Y. Carmon. Resolving discrepancies in compute-optimal scaling of language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 100535–100570, 2024.

M. Posa, C. Cantu, and R. Tedrake. A direct method for trajectory optimization of rigid bodies through contact. *The International Journal of Robotics Research*, 33(1):69–81, 2014.

G. A. Pratt and M. M. Williamson. Series elastic actuators. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, volume 1, pages 399–406, 1995.

T. Prevot, J. Rios, P. Kopardekar, J. E. Robinson, M. Johnson, and J. Jung. UAS traffic management (UTM) concept of operations to safely enable low altitude flight operations. In *16th AIAA Aviation Technology, Integration, and Operations Conference*, 2016.

A. Prorok. Extending robot minds through collective learning. *Science Robotics*, 10(106):eadv4049, 2025.

N. Pudasaini, Y. Zhang, J. Lavering, M. Conway, A. Roncone, and N. Correll. FAME: Force-adaptive RL for expanding the manipulation envelope of a full-scale humanoid, 2026. URL https://arxiv.org/abs/2603.08961v3.

PX4 Development Team. Hardware in the loop simulation (HITL). PX4 User Guide, n.d. URL https://docs.px4.io/main/en/simulation/hitl.

D. Qu, H. Song, Q. Chen, D. Wang, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, and X. Li. SpatialVLA: Exploring spatial representations for visual-language-action model. In *Proceedings of Robotics: Science and Systems*, 2025.

J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang. WorldGym: World model as an environment for policy evaluation. In *International Conference on Learning Representations*, 2026.

M. N. Qureshi, S. Garg, F. Yandun, D. Held, G. Kantor, and A. Silwal. SplatSim: Zero-shot Sim2Real transfer of RGB manipulation policies using Gaussian splatting. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6502–6509, 2025.

A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, 2021.

I. Radosavovic, S. Kamat, T. Darrell, and J. Malik. Learning humanoid locomotion over challenging terrain, 2024. URL https://arxiv.org/abs/2410.03654.

M. H. Raibert and J. J. Craig. Hybrid position/force control of manipulators. *Journal of Dynamic Systems, Measurement, and Control*, 103(2):126–133, 1981.

M. H. Raibert, H. B. Brown Jr, and M. Chepponis. Experiments in balance with a 3D one-legged hopping machine. *The International Journal of Robotics Research*, 3(2):75–92, 1984.

M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

A. Rajvanshi, K. Sikka, X. Lin, B. Lee, H.-P. Chiu, and A. Velasquez. Saynav: Grounding large language models for dynamic planning to navigation in new environments. In *Proceedings of the International Conference on Automated Planning and Scheduling*, volume 34, pages 464–474, 2024.

Y. Rizk, M. Awad, and E. W. Tunstel. Cooperative heterogeneous multi-robot systems: A survey. *ACM Computing Surveys (CSUR)*, 52(2):1–31, 2019.

A. Robey, Z. Ravichandran, V. Kumar, H. Hassani, and G. J. Pappas. Jailbreaking LLM-controlled robots, 2024. URL https://arxiv.org/abs/2410.13691.

RobotEra. *X-Hand 1 Product Manual*, 1.0 edition, Oct. 2024. URL https://cdn.robotshop.com/media/E/ERA/RB-Era-01/pdf/x-hand-product-manual-1-0.pdf. Version 1.0, 15 October 2024.

RobotEra. XHAND 1: Product specifications, n.d. URL https://www.robotera.com/upload/goods/20241208/2ab64afaae0098db948e9d4063951c28.pdf.

E. Rodgers, E. Gertsen, J. Sotudeh, C. Mullins, A. Hernandez, H. Le, P. Smith, and N. Joseph. Space based solar power. White paper, National Aeronautics and Space Administration, Office of Technology, Policy, and Strategy, 2024. URL https://ntrs.nasa.gov/citations/20230018600. NASA NTRS Document ID: 20230018600.

M. Rolf, J. J. Steil, and M. Gienger. Goal babbling permits direct learning of inverse kinematics. *IEEE Transactions on Autonomous Mental Development*, 2(3):216–229, 2010.

R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10674–10685, 2022.

J. Romero, D. Tzionas, and M. J. Black. Embodied hands: Modeling and capturing hands and bodies together. *ACM Transactions on Graphics*, 36(6):1–17, 2017.

ROS 2 Project. rmw_zenoh: Design. Jazzy branch, n.d. URL `https://github.com/ros2/rmw_zenoh/blob/jazzy/docs/design.md`.

ros2_control Development Team. Getting started: Architecture. ros2_control documentation, Jazzy distribution, n.d. URL `https://control.ros.org/jazzy/doc/getting_started/getting_started.html`.

S. Ross and J. A. Bagnell. Efficient reductions for imitation learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9 of *Proceedings of Machine Learning Research*, pages 661–668, 2010.

S. Ross, G. J. Gordon, and J. A. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 15 of *Proceedings of Machine Learning Research*, pages 627–635, 2011.

D. K. Roy and A. P. Pentland. Learning words from sights and sounds: A computational model. *Cognitive Science*, 26(1):113–146, 2002.

S. Royo and M. Ballesta-Garcia. An overview of lidar imaging systems for autonomous vehicles. *Applied sciences*, 9(19):4093, 2019.

N. Rudin, D. Hoeller, P. Reist, and M. Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022.

D. Rus and M. T. Tolley. Design, fabrication and control of soft robots. *Nature*, 521(7553):467–475, 2015.

O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision*, 115(3):211–252, 2015.

S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020. ISBN 9780134610993.

C. Sagan. *The Cosmic Connection: An Extraterrestrial Perspective*. Anchor Press, Garden City, NY, 1973.

J. Sajid, S. S. Hassan, W. Liu, Y. K. Tun, Y. Fu, N. H. Tran, Z. Han, C. Stefanović, T. Ratnarajah, and M. M. Alam. Embodied AI in 6G networks: From intelligent connectivity to physical intelligence, 2026. URL `https://arxiv.org/abs/2606.20592`.

S. S. Sajjan, M. Moore, M. Pan, G. Nagaraja, J. Lee, A. Zeng, and S. Song. ClearGrasp: 3D shape estimation of transparent objects for manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 3634–3642, 2020.

A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia. Learning to simulate complex physics with graph networks. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 8459–8468, 2020.

S. Sartor and N. Thompson. Neural scaling laws in robotics, 2025. URL https://arxiv.org/abs/2405.14005.

V. G. Satorras, E. Hoogeboom, and M. Welling. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, pages 9323–9332, 2021.

M. Savva, A. Kadian, O. Maksymets, Y. Zhao, E. Wijmans, B. Jain, J. Straub, J. Liu, V. Koltun, J. Malik, D. Parikh, and D. Batra. Habitat: A platform for embodied AI research. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 9338–9346, 2019.

V. Saxena, M. Bronars, N. R. Arachchige, K. Wang, W. C. Shin, S. Nasiriany, A. Mandlekar, and D. Xu. What matters in learning from large-scale datasets for robot manipulation. In *International Conference on Learning Representations*, 2025.

S. Schaal. Is imitation learning the route to humanoid robots? *Trends in Cognitive Sciences*, 3(6): 233–242, 1999.

J. Schmidhuber. Curious model-building control systems. In *Proceedings of the International Joint Conference on Neural Networks*, volume 2, pages 1458–1463, 1991.

B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio. Toward causal representation learning. *Proceedings of the IEEE*, 109(5):612–634, 2021.

J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel, T. Lillicrap, and D. Silver. Mastering Atari, Go, chess and Shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems*, volume 28, pages 2503–2511, 2015.

C. Semini, N. G. Tsagarakis, E. Guglielmino, M. Focchi, F. Cannella, and D. G. Caldwell. Design of HyQ – a hydraulically and electrically actuated quadruped robot. *Proceedings of the Institution of Mechanical Engineers, Part I: Journal of Systems and Control Engineering*, 225(6):831–849, 2011.

S. Seok, A. Wang, M. Y. Chuah, D. J. Hyun, J. Lee, D. M. Otten, J. H. Lang, and S. Kim. Design principles for energy-efficient legged locomotion and implementation on the MIT Cheetah robot. *IEEE/ASME Transactions on Mechatronics*, 20(3):1117–1129, 2015.

P. Sermanet, T. Ding, J. Zhao, F. Xia, D. Dwibedi, K. Gopalakrishnan, C. Chan, G. Dulac-Arnold, S. Maddineni, N. J. Joshi, P. Florence, W. Han, R. Baruch, Y. Lu, S. Mirchandani, P. Xu, P. Sanketi, K. Hausman, I. Shafran, B. Ichter, and Y. Cao. RoboVQA: Multimodal long-horizon reasoning for robotics. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 645–652, 2024.

D. Shah, A. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine. ViNT: A foundation model for visual navigation. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 711–733. PMLR, 2023.

J. Shah, J. Wiken, B. Williams, and C. Breazeal. Improved human-robot team performance using Chaski, a human-inspired plan execution system. In *Proceedings of the 6th International Conference on Human-Robot Interaction*, pages 29–36, 2011.

Shanghai AI Laboratory. InternW0-Δ: Official project page. Project website, 2026. URL https://internrobotics.github.io/InternW0-Delta/.

A. K. Sharma, Y. Sun, N. Lu, Y. Zhang, J. Liu, and S. Yang. World-Gymnast: Training robots with reinforcement learning in a world model, 2026a. URL https://arxiv.org/abs/2602.02454.

Y. Sharma, D. Niu, A. Pai, Z. Wang, Z. Liu, B. Shi, S. Saravalle, B. Shao, R. Zheng, J. Wang, K. Kallidromitis, Y. Kato, F. Galasso, Y. Zhu, D. Xu, L. Fan, J. Malik, T. Darrell, and R. Herzig. Contrastive action-image pre-training for visuomotor control, 2026b. URL https://arxiv.org/abs/2606.17256v1.

Sharpa Robotics. Sharpa Wave W01 hand user manual, n.d.a. URL https://sharpa-robotics.github.io/sharpa-docs/.

Sharpa Robotics. Sharpa Wave: Released robot models, n.d.b. URL https://github.com/sharpa-robotics/sharpa-urdf-usd-xml. Wave 01 right-hand URDF; active finger joints, excluding floating base.

R. Shen, Ž. Kovačič, P. Kulits, X. Wang, Z. Li, J. B. Tenenbaum, A. Yuille, J. Chen, and J. Wu. 4DCodeBench: Benchmarking agents on inverse graphics of dynamic scenes, 2026. URL https://arxiv.org/abs/2610.03715.

B. Shih, D. Shah, J. Li, T. G. Thuruthel, Y.-L. Park, F. Iida, Z. Bao, R. Kramer-Bottiglio, and M. T. Tolley. Electronic skins and machine learning for intelligent soft robots. *Science Robotics*, 5(41): eaaz9239, 2020.

M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.

M. Shridhar, L. Manuelli, and D. Fox. CLIPort: What and where pathways for robotic manipulation. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 894–906, 2022.

M. Shridhar, L. Manuelli, and D. Fox. Perceiver-Actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799. PMLR, 2023.

M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene. SmolVLA: A vision-language-action model for affordable and efficient robotics, 2025. URL https://arxiv.org/abs/2506.01844.

P. Siano. Demand response and smart grids—a survey. *Renewable and Sustainable Energy Reviews*, 30:461–478, 2014.

B. Siciliano and O. Khatib, editors. *Springer Handbook of Robotics*. Springer, Cham, 2 edition, 2016. ISBN 9783319325507.

A. Sidelsky. Trapezoidal motors vs. sinusoidal motors. Texas Instruments E2E technical FAQ, 2020. URL https://e2e.ti.com/support/motor-drivers-group/motor-drivers/f/motor-drivers-forum/909911/faq-trapezoidal-motors-vs-sinusoidal-motors.

J. Siekmann, Y. Godse, A. Fern, and J. Hurst. Sim-to-real learning of all common bipedal gaits via periodic reward composition. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 7309–7315, 2021.

M. Sitti. Physical intelligence as a new paradigm. *Extreme Mechanics Letters*, 46:101340, 2021.

S. Skogestad and I. Postlethwaite. *Multivariable Feedback Control: Analysis and Design*. John Wiley & Sons, Chichester, UK, 2 edition, 2005. ISBN 9780470011683.

J.-P. Sleiman, F. Farshidian, M. V. Minniti, and M. Hutter. A unified MPC framework for whole-body dynamic locomotion and manipulation. *IEEE Robotics and Automation Letters*, 6(3):4688–4695, 2021.

J. Solà. Quaternion kinematics for the error-state Kalman filter, 2017. URL https://arxiv.org/abs/1711.02508.

SPIDER Contributors. SPIDER: Released XHAND robot assets, 2026. URL https://github.com/facebookresearch/spider. Right-hand URDF and STL meshes; CC BY-NC 4.0.

S. S. Srinivasa, D. Berenson, M. Cakmak, A. Collet, M. R. Dogar, A. D. Dragan, R. A. Knepper, T. Niemueller, K. Strabala, M. Vande Weghe, and J. Ziegler. HERB 2.0: Lessons learned from developing a mobile manipulator for the home. *Proceedings of the IEEE*, 100(8):2410–2428, 2012.

Standing Committee of the National People's Congress. Personal information protection law of the people's republic of china. Official Chinese text published by the Cyberspace Administration of China, 2021. URL https://www.cac.gov.cn/2021-08/20/c_1631050028355286.htm. Adopted August 20, 2021; effective November 1, 2021.

L. Steels and F. Kaplan. AIBO's first words: The social learning of language and meaning. *Evolution of Communication*, 4(1):3–32, 2001.

R. E. Stern, S. Cui, M. L. Delle Monache, R. Bhadani, M. Bunting, M. Churchill, N. Hamilton, R. Haulcy, H. Pohlmann, F. Wu, B. Piccoli, B. Seibold, J. Sprinkle, and D. B. Work. Dissipation of stop-and-go waves via control of autonomous vehicles: Field experiments. *Transportation Research Part C: Emerging Technologies*, 89:205–221, 2018.

D. E. Stewart and J. C. Trinkle. An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and Coulomb friction. *International Journal for Numerical Methods in Engineering*, 39(15):2673–2691, 1996.

P. Stone, G. A. Kaminka, S. Kraus, and J. S. Rosenschein. Ad hoc autonomous agent teams: Collaboration without pre-coordination. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 24, pages 1504–1509, 2010.

J. Su, Y. Zheng, M. Yan, L. Yi, Z. Zhang, and H. Wang. GPT-6 Astra as an embodied policy. Technical report, 2026. URL https://anonymous-report-421.github.io/public-website/.

S. Suresh, Z. Si, J. G. Mangelson, W. Yuan, and M. Kaess. Shapemap 3-d: Efficient shape mapping through dense touch and vision. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 7073–7080. IEEE, 2022.

R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

R. S. Sutton. The bitter lesson. Online essay, 2019. URL http://www.incompleteideas.net/IncIdeas/BitterLesson.html.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

R. S. Sutton, D. Precup, and S. Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

K. Suzuki, J. Liu, Y. Wang, C. Hori, M. Brand, D. Romeres, and T. Koike-Akino. Embedding morphology into transformers for cross-robot policy learning, 2026. URL https://arxiv.org/abs/2603.00182.

R. Szeliski. *Computer vision: algorithms and applications*. Springer London, 1 edition, 2011.

N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng, and G. Ceder. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624:86–91, 2023.

N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, T. He, D. Milsted, M. J. McDermott, M. Gallant, E. D. Cubuk, A. Merchant, H. Kim, A. Jain, C. J. Bartel, K. Persson, Y. Zeng, and G. Ceder. Author Correction: An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 650:E1, 2026.

E. Tabassi. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, National Institute of Standards and Technology, Gaithersburg, MD, 2023.

J. Tang, Z. Zhang, J. Yang, Y. Yan, K. Zhou, L. Mao, R. Han, J. Cao, W. Xu, L. He, M. Fu, F. Feng, and B. Huang. Video2World: Benchmarking coding agents for interactive world modeling from embodied videos, 2026. URL https://arxiv.org/abs/2610.04432.

S. Tao, F. Xiang, A. Shukla, Y. Qin, X. Hinrichsen, X. Yuan, C. Bao, X. Lin, Y. Liu, T.-k. Chan, Y. Gao, X. Li, T. Mu, N. Xiao, A. Gurha, V. N. Rajesh, Y. W. Choi, Y.-R. Chen, Z. Huang, R. Calandra, R. Chen, S. Luo, and H. Su. ManiSkill3: GPU parallelized robotics simulation and rendering for generalizable embodied AI, 2025a. URL https://arxiv.org/abs/2410.00425.

T. Tao, M. K. Srirama, J. J. Liu, K. Shaw, and D. Pathak. DexWild: Dexterous human interactions for in-the-wild robot policies. In *Proceedings of Robotics: Science and Systems*, 2025b.

S. Tarbouriech and M. C. Turner. Anti-windup design: an overview of some recent advances and open problems. *IET Control Theory & Applications*, 3(1):1–19, 2009.

M. Tegmark. *Life 3.0: Being Human in the Age of Artificial Intelligence*. Alfred A. Knopf, 2017. ISBN 9781101946596.

S. Tellex, T. Kollar, S. Dickerson, M. R. Walter, A. G. Banerjee, S. Teller, and N. Roy. Understanding natural language commands for robotic navigation and mobile manipulation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 25(1):1507–1514, 2011.

Texas Instruments. *DRV8874 H-Bridge Motor Driver with Integrated Current Sense and Regulation*. Texas Instruments, 2019. URL https://www.ti.com/product/DRV8874. Datasheet, Revision A.

N. Thomas, T. Smidt, S. Kearnes, L. Yang, L. Li, K. Kohlhoff, and P. Riley. Tensor field networks: Rotation- and translation-equivariant neural networks for 3D point clouds, 2018. URL https://arxiv.org/abs/1802.08219.

S. Thrun and T. M. Mitchell. Lifelong robot learning. *Robotics and Autonomous Systems*, 15(1–2): 25–46, 1995.

Y. Tian, Y. Yang, Y. Xie, Z. Cai, X. Shi, N. Gao, H. Liu, X. Jiang, Z. Qiu, F. Yuan, Y. Li, P. Wang, J. Cai, J. Zeng, H. Dong, and J. Pang. InternData-A1: Pioneering high-fidelity synthetic data for pre-training generalist policy, 2025. URL https://arxiv.org/abs/2511.16651.

J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012.

E. Tolstaya, F. Gama, J. Paulos, G. Pappas, V. Kumar, and A. Ribeiro. Learning decentralized controllers for robot swarms with graph neural networks. In *Proceedings of the Conference on Robot Learning*, volume 100 of *Proceedings of Machine Learning Research*, pages 671–682. PMLR, 2020.

G. Tom, S. P. Schmid, S. G. Baird, Y. Cao, K. Darvish, H. Hao, S. Lo, S. Pablo-García, E. M. Rajaonson, M. Skreta, N. Yoshikawa, S. Corapi, G. D. Akkoc, F. Strieth-Kalthoff, M. Seifrid, and A. Aspuru-Guzik. Self-driving laboratories for chemistry and materials science. *Chemical Reviews*, 124(16): 9633–9732, 2024.

M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz, K. Dhabalia, M. Equi, Q. Vuong, J. T. Springenberg, S. Levine, C. Finn, and D. Driess. Mem: Multi-scale embodied memory for vision language action models, 2026. URL https://arxiv.org/abs/2603.03596.

A. Torralba, P. Isola, and W. T. Freeman. *Foundations of computer vision*. MIT Press, 2024.

M. Tranzatto, T. Miki, M. Dharmadhikari, L. Bernreiter, M. Kulkarni, F. Mascarich, O. Andersson, S. Khattak, M. Hutter, R. Siegwart, and K. Alexis. Cerberus in the darpa subterranean challenge. *Science Robotics*, 7(66):eabp9742, 2022.

R. Y. Tsai and R. K. Lenz. A new technique for fully autonomous and efficient 3D robotics hand/eye calibration. *IEEE Transactions on Robotics and Automation*, 5(3):345–358, 1989.

UNESCO. Recommendation on the ethics of artificial intelligence. United Nations Educational, Scientific and Cultural Organization, 2021. URL https://www.unesco.org/en/legal-affairs/recommendation-ethics-artificial-intelligence.

UniDex Team. UniDex: Project documentation, 2026. URL https://unidex-ai.github.io/.

United Nations Economic Commission for Europe. *UN Regulation No. 156: Software Update and Software Update Management System*. United Nations Economic Commission for Europe, 2021. URL https://unece.org/transport/documents/2021/03/standards/un-regulation-no-156-software-update-and-software-update.

Unitree Robotics. G1 SDK Development Guide, 2026. URL https://support.unitree.com/home/en/G1_developer. Manufacturer documentation: About G1, Architecture Description, Basic Services Interface, Sport Services Interface, and Odometer Service Interface.

T. Unterthiner, S. van Steenkiste, K. Kurach, R. Marinier, M. Michalski, and S. Gelly. Towards accurate generative models of video: A new metric & challenges, 2018. URL https://arxiv.org/abs/1812.01717.

H. Van Brussel, J. Wyns, P. Valckenaers, L. Bongaerts, and P. Peeters. Reference architecture for holonic manufacturing systems: PROSA. *Computers in Industry*, 37(3):255–274, 1998.

A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

P. Varaiya. Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies*, 36:177–195, 2013.

F. J. Varela, E. Thompson, and E. Rosch. *The Embodied Mind: Cognitive Science and Human Experience*. MIT Press, 1991.

M. T. Villasevil, A. Simeonov, Z. Li, A. Chan, T. Chen, A. Gupta, and P. Agrawal. Reconciling reality through simulation: A real-to-sim-to-real approach for robust manipulation. In *Robotics: Science and Systems (RSS)*, 2024.

W3C. Web of Things (WoT) Thing Description 1.1. W3c recommendation, World Wide Web Consortium, Dec. 2023. URL https://www.w3.org/TR/2023/REC-wot-thing-description11-20231205/. 5 December 2023.

K. P. Wabersich and M. N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021.

H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine. BridgeData V2: A Dataset for Robot Learning at Scale. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736. PMLR, 2023.

C. Wang, H. Shi, W. Wang, R. Zhang, L. Fei-Fei, and C. K. Liu. DexCap: Scalable and portable mocap data collection system for dexterous manipulation. In *Robotics: Science and Systems (RSS)*, 2024a.

C. Wang, M. Wang, Y. Chen, Z. Wu, B. Huang, S. Chen, G. Chen, L. Wei, H. Zhang, Q. Zhang, Y. Yang, G. Yang, S. Xiong, P. Wang, C. Jiang, and P. Y. Chen. WorldAgent: Verification-guided agentic physical world construction, 2026a. URL https://arxiv.org/abs/2609.33208.

G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024b.

J. Wang and Z. Huang. The recent technological development of intelligent mining in China. *Engineering*, 3(4):439–444, 2017.

J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny. Vggt: Visual geometry grounded transformer. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5294–5306. IEEE, 2025a.

L. Wang, X. Chen, J. Zhao, and K. He. Scaling proprioceptive-visual learning with heterogeneous pre-trained transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 124420–124450, 2024c.

L. Wang, Y. Ling, Z. Yuan, M. Shridhar, C. Bao, Y. Qin, B. Wang, H. Xu, and X. Wang. GenSim: Generating robotic simulation tasks via large language models. In *International Conference on Learning Representations (ICLR)*, 2024d.

S. Wang, J. Shi, Z. Fu, X. He, F. Liu, C. Yang, Y. Zhou, Z. Fei, J. Gong, J. Fu, M. Z. Shou, X. Huang, X. Qiu, and Y.-G. Jiang. World action models: The next frontier in embodied AI, 2026b. URL `https://arxiv.org/abs/2605.12090`.

Y. Wang, Z. Xian, F. Chen, T.-H. Wang, Y. Wang, K. Fragkiadaki, Z. Erickson, D. Held, and C. Gan. RoboGen: Towards unleashing infinite data for automated robot learning via generative simulation. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 51936–51983. PMLR, 2024e.

Y. Wang, H. Zhu, M. Liu, J. Yang, H.-S. Fang, and T. He. VQ-VLA: Improving vision-language-action models via scaling vector-quantized action tokenizers. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11089–11099. IEEE, 2025b.

Y. Wang, S. Huang, M. Li, C. Zhang, J. Liang, W. Jin, Y. Chen, X. Chi, D. Zhou, Q. Yu, Y.-K. Wang, Y. Rui, S. Yao, Z. Yuan, Z. Shen, K. Zhu, Z. Zhu, N. Gao, X. Chi, G. He, S. Zhang, H. Dong, L. Shao, and H. Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining, 2026c. URL `https://arxiv.org/abs/2609.07398`.

Z. Wang, H. Zheng, Y. Nie, W. Xu, Q. Wang, H. Ye, Z. Li, K. Zhang, X. Cheng, W. Dong, C. Cai, L. Lin, F. Zheng, and X. Liang. All robots in one: A new standard and unified dataset for versatile, general-purpose embodied agents, 2024f. URL `https://arxiv.org/abs/2408.10899`.

A. Wei, A. Agarwal, B. Chen, R. Bosworth, N. Pfaff, and R. Tedrake. Empirical analysis of Sim-and-Real cotraining of diffusion policies for planar pushing from pixels. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5625–5632, 2025.

J. Weng, J. L. McClelland, A. Pentland, O. Sporns, I. J. Stockman, M. Sur, and E. Thelen. Autonomous mental development by robots and animals. *Science*, 291(5504):599–600, 2001.

P. M. Wensing, A. Wang, S. Seok, D. Otten, J. Lang, and S. Kim. Proprioceptive actuator design in the MIT Cheetah: Impact mitigation and high-bandwidth physical interaction for dynamic legged robots. *IEEE Transactions on Robotics*, 33(3):509–522, 2017.

P. M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, and A. Del Prete. Optimization-based control for dynamic legged robots. *IEEE Transactions on Robotics*, 40:43–63, 2024.

J. Werfel, K. Petersen, and R. Nagpal. Designing collective behavior in a termite-inspired robot construction team. *Science*, 343(6172):754–758, 2014.

D. E. Whitney. Quasi-static assembly of compliantly supported rigid parts. *Journal of Dynamic Systems, Measurement, and Control*, 104(1):65–77, 1982.

N. Wiener. *Cybernetics: Or Control and Communication in the Animal and the Machine*. John Wiley & Sons, New York, 1948.

A. D. Wilson and S. Golonka. Embodied cognition is not what you think it is. *Frontiers in Psychology*, 4:58, 2013.

M. Wilson. Six views of embodied cognition. *Psychonomic Bulletin & Review*, 9(4):625–636, 2002.

T. Wisspeintner, T. van der Zant, L. Iocchi, and S. Schiffer. RoboCup@Home: Scientific competition and benchmarking for domestic service robots. *Interaction Studies*, 10(3):392–426, 2009.

World AI Conference and High-Level Meeting on Global AI Governance. Shanghai Declaration on Global AI Governance, 2024. URL https://www.fmprc.gov.cn/eng./zy/gb/202407/t20240704_11448351.html.

World Artificial Intelligence Conference and High-Level Meeting on Global AI Governance. Global AI Governance Action Plan, 2025. URL https://www.mfa.gov.cn/eng/zy/gb/202507/t20250729_11679232.html.

World Labs Team. Marble: A multimodal world model. World Labs blog, 12 November 2025, 2025a. URL https://www.worldlabs.ai/blog/marble-world-model.

World Labs Team. RTFM: A real-time frame model, 2025b. URL https://www.worldlabs.ai/blog/rtfm.

World Labs Team. Atlas: A world model for spatial intelligence. World Labs Blog, Sept. 2026. URL https://www.worldlabs.ai/blog/atlas. Published September 1, 2026.

J. T. Wright. Dyson spheres. *Serbian Astronomical Journal*, (200):1–18, 2020.

D. Wu, X. Wei, G. Chen, H. Shen, X. Wang, W. Li, and B. Jin. Generative multi-agent collaboration in embodied ai: A systematic review, 2025a. URL https://arxiv.org/abs/2502.11518.

G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang. 4d gaussian splatting for real-time dynamic scene rendering. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20310–20320. IEEE, 2024a.

K. Wu, C. Hou, J. Liu, Z. Che, X. Ju, Z. Yang, M. Li, Y. Zhao, Z. Xu, G. Yang, S. Fan, X. Wang, F. Liao, Z. Zhao, G. Li, Z. Jin, L. Wang, J. Mao, N. Liu, P. Ren, Q. Zhang, Y. Lyu, M. Liu, H. Jingyang, Y. Luo, Z. Gao, C. Li, C. Gu, Y. Fu, D. Wu, X. Wang, S. Chen, Z. Wang, P. An, S. Qian, S. Zhang, and J. Tang. RoboMIND: Benchmark on multi-embodiment intelligence normative data for robot manipulation. In *Proceedings of Robotics: Science and Systems*, 2025b.

L. Wu, C. Yu, J. Ren, L. Chen, Y. Jiang, R. Huang, G. Gu, and H. Li. FreeTacMan: Robot-free visuotactile data collection system for contact-rich manipulation. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5367–5374, 2026a.

P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg. DayDreamer: World models for physical robot learning. In *Conference on Robot Learning*, volume 205 of *Proceedings of Machine Learning Research*, pages 2226–2240, 2023.

P. Wu, Y. Shentu, Z. Yi, X. Lin, and P. Abbeel. GELLO: A general, low-cost, and intuitive teleoperation framework for robot manipulators. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12156–12163, 2024b.

W. Wu, F. Lu, Y. Wang, S. Yang, S. Liu, F. Wang, Q. Zhu, H. Sun, Y. Wang, S. Ma, Y. Ren, K. Zhang, H. Yu, J. Zhao, S. Zhou, Z. Qiu, H. Xiong, Z. Wang, Z. Wang, R. Cheng, Y.-L. Li, Y. Huang, X. Zhu, Y. Shen, and K. Zheng. A pragmatic VLA foundation model, 2026b. URL https://arxiv.org/abs/2601.18692.

Wuji Technology. Wuji Description: Robot model assets, 2026. URL https://github.com/wuji-technology/wuji-description. First-generation right-hand body URDF and visual meshes.

Wuji Technology. Wuji Glove: Product overview, n.d.a. URL https://docs.wuji.tech/docs/en/wuji-glove/latest/introduction/.

Wuji Technology. Wuji Hand teleoperation: SteamVR tracking solution, n.d.b. URL https://github.com/wuji-technology/wuji-hand-teleop/blob/main/docs/STEAMVR.md.

Wuji Technology. Wuji Hand: Product introduction, n.d.c. URL https://docs.wuji.tech/docs/en/wuji-hand/v1/overview/. Archived first-generation documentation.

P. R. Wurman, R. D'Andrea, and M. Mountz. Coordinating hundreds of cooperative, autonomous vehicles in warehouses. *AI Magazine*, 29(1):9–19, 2008.

R. B. Wynn, V. A. Huvenne, T. P. Le Bas, B. J. Murton, D. P. Connelly, B. J. Bett, H. A. Ruhl, K. J. Morris, J. Peakall, D. R. Parsons, E. J. Sumner, S. E. Darby, R. M. Dorrell, and J. E. Hunt. Autonomous underwater vehicles (AUVs): Their past, present and future contributions to the advancement of marine geoscience. *Marine Geology*, 352:451–468, 2014.

H. Xia, X. Li, Z. Li, Q. Ma, J. Xu, M.-Y. Liu, Y. Cui, T.-Y. Lin, W.-C. Ma, S. Wang, S. Song, and F. Wei. SAGE: Scalable agentic 3D scene generation for embodied AI, 2026. URL https://arxiv.org/abs/2602.10116.

J. Xie, W. Mao, Z. Bai, D. J. Zhang, W. Wang, K. Q. Lin, Y. Gu, Z. Chen, Z. Yang, and M. Z. Shou. Show-o: One single transformer to unify multimodal understanding and generation. In *International Conference on Learning Representations*, 2025.

T. Xie, Z. Zong, Y. Qiu, X. Li, Y. Feng, Y. Yang, and C. Jiang. PhysGaussian: Physics-integrated 3D Gaussians for generative dynamics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4389–4398, 2024.

X. Xiong, T.-H. Wang, Y.-L. Qiao, T. Du, and M. Li. Text2Sim: Agentic physics-based simulation generation with distilled expertise, 2026. URL https://arxiv.org/abs/2609.36593.

XPolicyLab Community, T. Chen, Y. Chen, T. Nian, Z. Cai, G. Chen, W. Lin, Q. Liang, Z. Chen, P. Xiang, K. Su, Z. Li, J. Tang, Y. Qin, Q. Chen, S. Zhu, T. Jiang, Y. Wang, X. Li, J. Zhang, W. Wan, B. Chen, H. Su, K. Ye, S. Liu, K. Wang, H. Liang, Y. Liu, M. Li, Y. Wang, B. Chen, H. Bi, S. Huang, H. Tan, J. Cai, Y. Mu, J. Guo, X. Wang, Z. Zhu, W. Ke, H. Li, Y. Tang, X. Li, G. Yang, Z. Tu, S. Yang, W. Song, P. Ding, K. Zhang, Y. Sun, J. Guo, T. Zhang, Y. Chen, R. Cui, Z. Zhang, H. Ma, J. Cai, H. Zhang, S. Yang, J. Ye, P. Chen, S. Liu, X. Su, W. Fang, W. Li, Y. Cao, C. Wang, Q. Chen, P. Luo, and W. Ding. XPolicyLab: A unified standard and open ecosystem for robot policy evaluation and deployment, 2026. URL https://arxiv.org/abs/2608.09892.

C. Xu, T. K. Nguyen, E. Dixon, C. Rodriguez, P. Miller, R. Lee, P. Shah, R. Ambrus, H. Nishimura, and M. Itkina. Can we detect failures without failure data? Uncertainty-aware runtime failure detection for imitation learning policies. In *Robotics: Science and Systems (RSS)*, 2025a.

C. Xu, Y. Xu, M. Xue, J. Lin, J. Wang, X. Zhang, Y. Wang, and J. Yu. The curse of precision: A data scaling law for high-precision robotic manipulation. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 21380–21387, 2026.

H. Xu, P. Liu, X. Chen, and S. Shen. D^2-SLAM: Decentralized and distributed collaborative visual-inertial SLAM system for aerial swarm. *IEEE Transactions on Robotics*, 40:3445–3464, 2024.

J. Xu, E. Heiden, I. Akinola, D. Fox, M. Macklin, and Y. Narang. Neural robot dynamics. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 3915–3935. PMLR, 2025b.

M. Xu, H. Zhang, Y. Hou, Z. Xu, L. Fan, M. Veloso, and S. Song. DexUMI: Using human hand as the universal manipulation interface for dexterous manipulation. In *Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 437–459, 2025c.

Y. Xue, W. Dong, M. Liu, W. Zhang, and J. Pang. A unified and general humanoid whole-body controller for versatile locomotion. Project manuscript, 2025. URL https://hugwbc.github.io/resources/HugWBC.pdf. Manuscript, HUGWBC project.

A. Yakefu, B. Xie, C. Xu, E. Zhang, E. Zhou, F. Jia, H. Yang, H. Fan, H. Zhang, H. Peng, J. Tan, J. Huang, K. Liu, K. Liu, K. Gu, Q. Zhang, R. Zhang, S. Huang, S. Cheng, S. Liu, T. Wang, T. Wang, W. Sun, W. Tang, Y. Wei, Y. Chen, Y. Gui, Y. Zhao, Y. Ma, Y. Wei, Y. Yang, Y. Guo, Z. Chen, Z. Du, Z. Zhang, Z. Liu, and Z. Yan. RoboChallenge: Large-scale real-robot evaluation of embodied policies, 2025. URL https://arxiv.org/abs/2510.17950.

Z. Yan, N. Jouandeau, and A. A. Cherif. A survey and analysis of multi-robot coordination. *International Journal of Advanced Robotic Systems*, 10(12):399, 2013.

B. Yang, Z. Li, Y. Sun, C. Miao, Y. Yang, M. Luo, X. Yan, F. Jiang, J. Shi, Y. Fu, N. Chen, J. Zhao, P. Wang, G. Yao, S. Zhang, H. Chen, Z. Li, and K. Zhu. AoE: Always-on egocentric human video collection for embodied AI, 2026a. URL https://arxiv.org/abs/2602.23893.

C. Yang, C. Lu, Y. Wang, and B. Zhou. Towards AI-45° law: A roadmap to trustworthy AGI, 2024a. URL https://arxiv.org/abs/2412.14186.

J. X. Yang, R. Dagli, A. Zook, H. Hadfield, A. Goyal, S. Birchfield, F. Ramos, and J. Tremblay. RoboLab: A high-fidelity simulation benchmark for analysis of task generalist policies. In *Robotics: Science and Systems*, 2026b.

S. Yang, Y. Du, K. Ghasemipour, J. Tompson, L. Kaelbling, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. In *International Conference on Learning Representations (ICLR)*, pages 45210–45234, 2024b.

Y. Yang, F.-Y. Sun, L. Weihs, E. VanderBilt, A. Herrasti, W. Han, J. Wu, N. Haber, R. Krishna, L. Liu, C. Callison-Burch, M. Yatskar, A. Kembhavi, and C. Clark. Holodeck: Language guided generation of 3D embodied AI environments. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16277–16287, 2024c.

R. D. Yates, Y. Sun, D. R. Brown, III, S. K. Kaul, E. Modiano, and S. Ulukus. Age of information: An introduction and survey. *IEEE Journal on Selected Areas in Communications*, 39(5):1183–1210, 2021.

S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, A. Malik, K. Lee, W. Liang, N. Ranawaka, J. Gu, Y. Xu, G. Wang, F. Hu, A. Narayan, J. Bjorck, J. Wang, G. Kim, D. Niu, R. Zheng, Y. Xie, J. Wu, Q. Wang, R. Julian, D. Xu, Y. Du, Y. Chebotar, S. Reed, J. Kautz, Y. Zhu, L. J. Fan, and J. Jang. World action models are zero-shot policies, 2026a. URL https://arxiv.org/abs/2602.15922.

Y. Ye, Y. Fu, Y. Lv, B. Hou, J. Cen, L. Kong, D. Zheng, T. Chen, J. Liu, Z. Cao, Y. Lou, W. Chow, X. Sun, Y. Wang, K. Ge, X. Chi, X. Zhang, Z. Pang, Y. Zhong, S. Han, Z. Lu, W. Yuan, Q. Chen, M. Y. Wang, Y. Mu, Z. Liu, J. Yang, P. Luo, and S. Zhang. Data Pyramid for Embodied Manipulation: A Survey, 2026b. URL https://arxiv.org/abs/2607.24744.

M. Yim, W.-M. Shen, B. Salemi, D. Rus, M. Moll, H. Lipson, E. Klavins, and G. S. Chirikjian. Modular self-reconfigurable robot systems [Grand Challenges of Robotics]. *IEEE Robotics & Automation Magazine*, 14(1):43–52, 2007.

J. Yin, Z. Zhao, X. Tan, Y. Liu, C. Wang, and X. Gu. WM-Craftnet: World synesthesia model for generalizable and robust dexterous in-hand manipulation, 2026a. URL https://arxiv.org/abs/2609.07002v1.

T. Yin, Z. Mei, Z. Zheng, M. Yamane, D. Wang, J. Sceats, S. M. Bateman, L. Zha, A. Badithela, O. Shorinwa, and A. Majumdar. PlayWorld: Learning robot world models from autonomous play, 2026b. URL https://arxiv.org/abs/2603.09030.

Z.-H. Yin, B. Huang, Y. Qin, Q. Chen, and X. Wang. Rotating without seeing: Towards in-hand dexterity through touch. In *Proceedings of Robotics: Science and Systems*, Daegu, Republic of Korea, 2023.

Z.-H. Yin, C. Wang, L. Pineda, F. Hogan, K. Bodduluri, A. Sharma, P. Lancaster, I. Prasad, M. Kalakrishnan, J. Malik, M. Lambeta, T. Wu, P. Abbeel, and M. Mukadam. Dexteritygen: Foundation controller for unprecedented dexterity. In *Proceedings of Robotics: Science and Systems*, 2025.

T. Yoshikawa. Manipulability of robotic mechanisms. *The International Journal of Robotics Research*, 4(2):3–9, 1985.

J. Yu, L. Fu, H. Huang, K. El-Refai, R. A. Ambrus, R. Cheng, M. Z. Irshad, and K. Goldberg. Real2Render2Real: Scaling robot data without dynamics simulation or robot hardware. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 547–577. PMLR, 2025a.

J. Yu, H. Liu, Q. Yu, J. Ren, C. Hao, H. Ding, G. Huang, G. Huang, Y. Song, P. Cai, W. Zhang, and C. Lu. ForceVLA: Enhancing VLA models with a force-aware MoE for contact-rich manipulation. In *Advances in Neural Information Processing Systems*, volume 38, pages 93409–93439, 2025b.

W. Yu, G. Turk, and C. K. Liu. Learning symmetric and low-energy locomotion. *ACM Transactions on Graphics (TOG)*, 37(4):1–12, 2018.

W. Yuan, S. Dong, and E. H. Adelson. GelSight: High-resolution robot tactile sensors for estimating geometry and force. *Sensors*, 17(12):2762, 2017.

W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox. RoboPoint: A vision-language model for spatial affordance prediction in robotics. In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 4005–4020, 2025.

E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda. A survey of autonomous driving: Common practices and emerging technologies. *IEEE Access*, 8:58443–58469, 2020.

P. Zanuttigh, G. Marin, C. Dal Mutto, F. Dominio, L. Minto, and G. M. Cortelazzo. *Time-of-Flight and Structured Light Depth Cameras: Technology and Applications*. Springer, 2016.

Y. Ze, S. Zhao, W. Wang, A. Kanazawa, R. Duan, P. Abbeel, G. Shi, J. Wu, and C. K. Liu. TWIST2: Scalable, portable, and holistic humanoid data collection system, 2025. URL https://arxiv.org/abs/2511.02832.

W. Zeng, S. Lu, K. Yin, X. Niu, M. Dai, J. Wang, and J. Pang. Behavior foundation model for humanoid robots, 2025. URL https://arxiv.org/abs/2509.13780.

W. Zeng, K. Yin, X. Niu, S. Lu, W. Zhong, J. Chen, F. Jia, X. Chen, Z. Wang, F. Xu, M. Zhou, K. Li, W. Zhang, H. Wang, L. Yi, D. Lin, J. Pang, and J. Wang. Scaling behavior foundation model for humanoid robots, 2026. URL https://arxiv.org/abs/2607.15163.

S. Zhai, Q. Zhang, T. Zhang, F. Huang, H. Zhang, M. Zhou, S. Zhang, L. Liu, S. Lin, and J. Pang. A Vision-Language-Action-Critic Model for Robotic Real-World Reinforcement Learning, 2025. URL https://arxiv.org/abs/2509.15937.

S. Zhai, Q. Zhang, T. Zhang, H. Zhang, F. Huang, Z. Lin, Z. Xu, and W. Zhang. HELP: Human-efficient large-scale robot post-training with rollout segmentation, 2026. URL https://arxiv.org/abs/2607.09776.

B. Zhang and R. Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

G. Zhang, Q. Xu, H. Zhang, J. Ma, L. He, Y. Bao, Z. Ping, Z. Yuan, C. Lu, C. Yuan, T. Liang, X. Tian, M. Shao, F. Zhang, M. Ding, Y. Gao, H. Zhao, H. Zhao, and H. Xu. UniDex: A robot foundation suite for universal dexterous hand control from egocentric human videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1841–1852, 2026a.

H. Zhang, Z. Wang, Q. Lyu, Z. Zhang, S. Chen, T. Shu, B. Dariush, K. Lee, Y. Du, and C. Gan. Combo: Compositional world models for embodied multi-agent cooperation. In *International Conference on Learning Representations*, volume 2025, pages 49996–50019, 2025a.

H. Zhang, C. Zhu, X. Wang, Z. Zhou, C. Yin, M. Li, L. Xue, Y. Wang, S. Hu, A. Liu, P. Guo, and L. Zhang. BadRobot: Jailbreaking embodied LLM agents in the physical world. In *The Thirteenth International Conference on Learning Representations*, 2025b.

J. Zhang, X. Yu, S. Ha, J. Peña Queralta, and T. Westerlund. Comparison of middlewares in edge-to-edge and edge-to-cloud communication for distributed ROS 2 systems. *Journal of Intelligent & Robotic Systems*, 110:162, 2024.

R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 586–595, 2018.

S. Zhang, O. So, K. Garg, and C. Fan. GCBF+: A neural graph control barrier function framework for distributed safe multiagent control. *IEEE Transactions on Robotics*, 41:1533–1552, 2025c.

W. Zhang, K. Wang, Y. Ouyang, X. Huang, L. Li, K. Su, W. Jin, W. Chai, H. Liang, Z. Dou, Y. Chen, and T. Chen. An unexpected robot policy: Early evaluations of GPT-6 Astra on RoboDojo and beyond, 2026b. URL `https://arxiv.org/abs/2609.24170`.

Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision*, pages 1–21. Springer, 2022.

Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on pattern analysis and machine intelligence*, 22(11):1330–1334, 2000.

Z. Zhang, H. Xu, Z. Yang, C. Yue, Z. Lin, H.-a. Gao, Z. Wang, and H. Zhao. Elucidating the design space of torque-aware vision-language-action models. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 4019–4037, 2025d.

T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023.

Z. Zhao, H. Jing, X. Liu, J. Mao, A. Jha, H. Yang, R. Xue, S. Zakharov, V. Guizilini, and Y. Wang. Humanoid Everyday: A comprehensive robotic dataset for open-world humanoid manipulation. In *2026 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6415–6422, 2026.

J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan. X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model. In *International Conference on Learning Representations*, volume 2026, pages 60580–60606, 2026a.

R. Zheng, D. Niu, Y. Xie, J. Wang, M. Xu, Y. Jiang, F. Castañeda, F. Hu, Y. L. Tan, L. Fu, T. Darrell, F. Huang, Y. Zhu, D. Xu, and L. Fan. EgoScale: Scaling dexterous manipulation with diverse egocentric human data, 2026b. URL `https://arxiv.org/abs/2602.16710`.

B. Zhou, H. Xu, and S. Shen. RACER: Rapid collaborative exploration with a decentralized multi-UAV system. *IEEE Transactions on Robotics*, 39(3):1816–1835, 2023.

H. Zhou, F. Yang, Z. Chen, L. Zhang, C. Fu, J. Li, J. Zhu, X. Liu, Y. Yang, L. Meng, L. Wang, X. Han, J. Sun, W. Ye, G. Wang, W. Li, M. Zhou, B. Li, M. Jin, K. Shao, M. Chen, G. Chen, L. Ma, J. T. Zhou, C. Spanos, X. Hu, M. Schwager, W. Zhang, Q. Feng, and S. Gu. Who Improves the Robot? Toward Physical Recursive Self-Improvement, 2026. URL https://arxiv.org/abs/2610.18842.

K. Zhou, J. C. Doyle, and K. Glover. *Robust and Optimal Control*. Prentice Hall, Upper Saddle River, NJ, 1996. ISBN 9780134565675.

Z. Zhou, P. Atreya, Y. L. Tan, K. Pertsch, and S. Levine. AutoEval: Autonomous evaluation of generalist robot manipulation policies in the real world. In *Proceedings of the 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 1997–2017. PMLR, 2025.

A. Zhu, M. Zhu, B. J. Kim, J. V. S. H. Ramos, Y. Shi, Y. Wu, R. Dhar, F. Yang, R. Hou, H. Fang, Q. Wang, Y. Cui, and D. W. Hong. DexEXO: A wearability-first dexterous exoskeleton for operator-agnostic demonstration and learning, 2026. URL https://arxiv.org/abs/2603.17323v1.

C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Robotics: Science and Systems (RSS)*, 2025.

W. Zhu, S. Oğuz, M. K. Heinrich, M. Allwright, M. Wahby, A. L. Christensen, E. Garone, and M. Dorigo. Self-organizing nervous systems for robot swarms. *Science Robotics*, 9(96):eadl5161, 2024.

J. G. Ziegler and N. B. Nichols. Optimum settings for automatic controllers. *Transactions of the ASME*, 64:759–765, 1942.

B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023.